MAKI

Media Asset Keeper & Indexer

User Manual

Version 4.12.0

2026-10-12

Media Asset Keeper & Indexer

A digital asset manager for photographers and media professionals

Apache-2.0 License

Contents

MAKI User Manual **10**
User Guide . . . 10
Reference Guide . . . 10
Developer Guide . . . 12

Overview & Concepts **13**
Overview . . . 13
Editions . . . 14
Core Concepts . . . 14
Content-Addressable Storage . . . 15
Two-Tier Storage . . . 16
Multi-Volume Support . . . 16
Variant Grouping . . . 16
Bidirectional XMP Sync . . . 17
Architecture . . . 18
Preview Generation . . . 19
Collections, Stacks, and Saved Searches . . . 20
The Round-Trip Workflow . . . 20

Setup **21**
Installation . . . 21
Initializing a Catalog . . . 22
Registering Volumes . . . 23
Configuration (maki.toml) . . . 27
AI Models *(Pro)* . . . 28

Ingesting Assets **31**
Import Pipeline . . . 31
Basic Import . . . 33
Auto-Grouping . . . 33
Recipe Handling . . . 35
Metadata Extraction . . . 36
Preview Generation . . . 37
Import Options . . . 39
Duplicate Handling . . . 40
Watching for New Files . . . 41
Audio Files . . . 41
AI Auto-Tagging *(Pro)* . . . 42
VLM Image Descriptions *(Pro)* . . . 42
Working with Video . . . 46
Common Import Scenarios . . . 47

Organizing Assets **50**
Tags . . . 50
Editing Metadata . . . 52
Manual Grouping . . . 53
Auto-Grouping . . . 53
Splitting Assets . . . 55
Collections (Static Albums) . . . 55
Stacks . . . 56
Saved Searches (Smart Albums) . . . 58
People Management (Face Recognition) *(Pro)* . . . 59
Deleting Assets . . . 62
Putting It All Together . . . 63

Browsing & Searching **64**
Searching Assets . . . 64
Search Filters Quick Reference . . . 65
Path Prefix Filter . . . 67
Output Formats . . . 67
Sort Options . . . 68
Scripting with maki . . . 69
Showing Asset Details . . . 69
Finding Duplicates . . . 70
Catalog Statistics . . . 72
Catalog Health . . . 74
Exporting Files . . . 74
Visual Similarity *(Pro)* . . . 75
Contact Sheets . . . 76
Related Topics . . . 77

Web UI **78**
Starting the Web UI . . . 78
Navigation bar . . . 79
Dark Mode . . . 80
Browse Page . . . 80
Navigation Between Views . . . 87
Lightbox . . . 88
Asset Detail Page . . . 91
Batch Operations . . . 94
Keyboard Navigation . . . 98
Compare View . . . 99
Tags Page . . . 101
Collections Page . . . 102
Saved Searches Page . . . 103
Stats Page . . . 103
People Page *(Pro)* . . . 104
Stroll Page *(Pro)* . . . 104
Duplicates Page . . . 106
Analytics Page . . . 107
Backup Status Page . . . 107
Volumes Page . . . 108
Maintain Dialog . . . 109
Settings Dialog . . . 111

Maintenance **112**

Edit History and Undo . . . 112
Verification . . . 113
Consistency Check (doctor) . . . 115
Sync . . . 115
Refresh . . . 117
Write Back *(Pro)* . . . 118
Sync Metadata *(Pro)* . . . 120
Working with External Tools . . . 121
Cleanup . . . 123
Trash . . . 124
Relocating Assets . . . 124
Updating File Locations . . . 126
Rebuilding the Catalog . . . 126
Fix Dates . . . 127
Fix Roles . . . 128
Fix Recipes . . . 128
Preview Management . . . 129
Storage Hygiene . . . 130
Aligning Flat Tags to the AI Vocabulary *(Pro)* . . . 131
Recommended Maintenance Workflow . . . 133
Backing Up Your Catalog with Git . . . 135
Recovering from a Drive Failure . . . 136

Scripting **138**
Bash Scripting . . . 138
Python Scripting . . . 140
Real-World Example: Fix Orphaned XMP Files . . . 142
Bulk Operations on a List of IDs . . . 142
Tips . . . 143
Related Topics . . . 144

Interactive Shell **145**
Starting the Shell . . . 145
Basic Usage . . . 145
Variables . . . 146
Session Defaults . . . 147
Tab Completion . . . 148
Script Files . . . 149
Session Management . . . 150
Quote Handling . . . 150
Tilde Expansion . . . 151
Built-in Commands . . . 151
Practical Examples . . . 151
Blocked Commands . . . 153
Tips . . . 153
Related Topics . . . 154

Organizing and Culling **155**
Three Populations . . . 155
Rating vs. Curation . . . 155
Workflow Approaches . . . 155
The Default Filter . . . 156
Practical Workflow Examples . . . 158
Tips . . . 158

Tagging Guide 160
Why Tags Matter . . . 160
Principles . . . 160
Recommended Vocabulary Structure . . . 162
How Many Tags? . . . 169
Auto-Tagging and the Label Vocabulary . . . 170
Industry Standards and References . . . 173
Cleaning Up an Existing Catalog . . . 174
Do's and Don'ts . . . 176
Tag Input in the Web UI . . . 176
The Vocabulary File . . . 177
Putting it all together . . . 180
Quick-Start Checklist . . . 181

Visual Discovery 182
Why Visual Discovery Matters . . . 182
Face Recognition . . . 183
Similarity Search . . . 185
Stroll . . . 186
Common Problems . . . 187
Cheatsheet . . . 188

The Archive Lifecycle 189
The Big Picture . . . 189
Stage 1: Import to Working Storage . . . 190
Stage 2: Cull and Organize . . . 191
Stage 3: Archive Completed Work . . . 191
Stage 4: Create Backup Copies . . . 192
Stage 5: Verify and Maintain . . . 192
Stage 6: Export and Deliver . . . 193
Putting It All Together: A Photographer's Monthly Workflow . . . 194
When Things Go Wrong . . . 195

CLI Conventions 196
Global Flags . . . 196
Catalog Discovery . . . 197
Asset ID Matching . . . 197
Exit Codes . . . 198
Output Conventions . . . 198
Scripting Patterns . . . 198
Safe Defaults . . . 199
Command Categories . . . 200

Setup Commands 202
maki init . . . 203
maki volume add . . . 205
maki volume list . . . 207
maki volume set-purpose . . . 209
maki volume remove . . . 210
maki volume combine . . . 212
maki volume split . . . 214
maki volume rename . . . 215

Ingest Commands 216
maki import . . . 217

maki watch . . . 220
maki delete . . . 222
maki tag . . . 224
maki tag rename . . . 226
maki tag split . . . 228
maki tag delete . . . 230
maki tag clear . . . 232
maki tag fix-unicode . . . 233
maki tag scan . . . 235
maki tag expand-ancestors . . . 237
maki tag export-vocabulary . . . 238
maki edit . . . 240
maki group . . . 242
maki split . . . 243
maki auto-group . . . 244
maki auto-tag *(Pro)* . . . 246
maki embed *(Pro)* . . . 249
maki ai export-vocabulary *(Pro)* . . . 251
maki audio analyze . . . 252
maki describe *(Pro)* . . . 254

Organize Commands **257**
maki collection create . . . 258
maki collection list . . . 259
maki collection show . . . 260
maki collection add . . . 261
maki collection remove . . . 262
maki collection delete . . . 263
maki saved-search save . . . 264
maki saved-search list . . . 265
maki saved-search run . . . 266
maki saved-search delete . . . 267
maki stack create . . . 268
maki stack add . . . 269
maki stack remove . . . 270
maki stack pick . . . 271
maki stack dissolve . . . 272
maki stack list . . . 273
maki stack show . . . 274
maki stack from-tag . . . 275
maki auto-stack *(Pro)* . . . 276
maki faces detect *(Pro)* . . . 278
maki faces cluster *(Pro)* . . . 279
maki faces clean *(Pro)* . . . 281
maki faces similarity *(Pro)* . . . 282
maki faces dump-aligned *(Pro)* . . . 283
maki faces people *(Pro)* . . . 284
maki faces name *(Pro)* . . . 285
maki faces merge *(Pro)* . . . 286
maki faces delete-person *(Pro)* . . . 287
maki faces unassign *(Pro)* . . . 288
maki faces export *(Pro)* . . . 289
maki faces download *(Pro)* . . . 290
maki faces status *(Pro)* . . . 291

Retrieve Commands **292**
maki search . . . 293
maki show . . . 297
maki preview . . . 298
maki export . . . 299
maki contact-sheet . . . 301
maki duplicates . . . 303
maki stats . . . 305
maki status . . . 307
maki backup-status . . . 309
maki doc . . . 311
maki licenses . . . 312
maki serve . . . 313
maki shell . . . 315

Maintain Commands **318**
maki undo . . . 319
maki history . . . 320
maki doctor . . . 321
maki verify . . . 322
maki sync . . . 324
maki refresh . . . 326
maki sync-metadata *(Pro)* . . . 328
maki writeback *(Pro)* . . . 330
maki cleanup . . . 333
maki dedup . . . 335
maki trash . . . 337
maki relocate . . . 338
maki update-location . . . 340
maki generate-previews . . . 342
maki fix-roles . . . 344
maki fix-dates . . . 345
maki fix-recipes . . . 347
maki create-sidecars . . . 349
maki rebuild-catalog . . . 351
maki migrate . . . 352

Search Filter Reference **353**
Negation and OR Syntax . . . 353
Free Text . . . 354
type . . . 354
tag . . . 355
format . . . 356
rating . . . 357
label . . . 358
camera . . . 358
lens . . . 358
description . . . 359
iso . . . 359
focal . . . 360
f (aperture) . . . 360
width . . . 360
height . . . 360
meta . . . 361

path . . . 361
date . . . 362
dateFrom . . . 362
dateUntil . . . 363
collection . . . 363
volume . . . 363
id . . . 364
orphan . . . 364
missing . . . 365
stale . . . 365
stacked . . . 365
copies . . . 366
variants . . . 366
tagcount . . . 367
scattered . . . 367
duration . . . 368
codec . . . 368
key . . . 369
bpm . . . 369
geo . . . 369
faces . . . 370
person *(Pro)* . . . 370
similar *(Pro)* . . . 371
min_sim *(Pro)* . . . 372
–image (search by query image) *(Pro)* . . . 372
embed . . . 373
text *(Pro)* . . . 373
Combining Filters . . . 374
Quoting and Special Characters . . . 375
Filter Availability . . . 376
Related Topics . . . 377

Format Templates Reference **378**
Presets . . . 378
Custom Templates . . . 379
Placeholders (search) . . . 379
Placeholders (duplicates) . . . 380
Escape Sequences . . . 380
Examples . . . 380
Format Flag Behavior . . . 381
Related Topics . . . 382

Configuration Reference (maki.toml) **383**
File Location . . . 383
Top-Level Fields . . . 383
preview Section . . . 384
[serve] Section . . . 385
[import] Section . . . 387
[dedup] Section . . . 389
[verify] Section . . . 390
[watch] Section . . . 390
[group] Section . . . 390
[contact_sheet] Section . . . 391
[ai] Section *(Pro)* . . . 392

[vlm] Section *(Pro)* . . . 394
[browse] Section . . . 398
trash Section . . . 400
history Section . . . 400
[writeback] Section . . . 401
[cli] Section . . . 403
[audio] Section . . . 404
Full Example . . . 404
Minimal Examples . . . 408
Defaults Summary . . . 408
Related Topics . . . 411

Data Model **412**
Entities . . . 412
Variant Roles . . . 419
Display Priority . . . 420
Storage . . . 420

VLM Model Guide *(Pro)* **425**
Quick Start . . . 425
Should I Upgrade? . . . 425
Tested Models . . . 426
Backends . . . 430
Choosing a Model . . . 431
Model Comparison . . . 432
Thinking Models . . . 433
Tips . . . 433
See Also . . . 435

Licenses & Acknowledgements **436**
MAKI . . . 436
Third-party Rust crates . . . 436
AI models *(Pro)* . . . 437
External tools . . . 438
Reporting license concerns . . . 438

REST API Reference **439**
Page Routes . . . 439
Search API . . . 441
Asset Editing . . . 441
Batch Operations . . . 446
Face & People APIs . . . 456
Data APIs . . . 459
Volume Management . . . 464
Import API . . . 466
Static Assets . . . 467
Error Handling . . . 467
Content Types Summary . . . 468

Module Reference **469**
Module Dependency Graph . . . 469
Module Table . . . 469
Generated API Documentation . . . 474

Building & Testing **475**

Building 475
Testing 475
Documentation 476
Release Process 477
Dependencies 477

MAKI User Manual

MAKI is a command-line digital asset manager built in Rust, designed for photographers and media professionals who manage large collections across multiple storage devices.

This manual is organized into three sections:

User Guide

Workflow-oriented guides that walk you through common tasks.

1. Overview & Concepts — Data model, architecture, and the round-trip workflow
2. Setup — Installation, initialization, volumes, and configuration
3. Ingesting Assets — Importing files, auto-grouping, metadata extraction, and previews
4. Organizing Assets — Tags, editing, grouping, collections, and saved searches
5. Browsing & Searching — CLI search, filters, output formats, and statistics
6. Web UI — Browser interface, batch operations, and keyboard navigation
7. Maintenance — Verification, sync, refresh, cleanup, and relocation
8. Scripting — Shell and Python scripting patterns, workflow automation
9. Interactive Shell — Variables, tab completion, script files, and session management
10. Organizing & Culling — Rating vs. curation, default filters, and workflow patterns
11. Tagging Guide — Tagging principles, vocabulary structure, and practical techniques
12. Visual Discovery — Face recognition, similarity search, and strolling: finding photos by what they look like
13. The Archive Lifecycle — Storage strategy, backup workflows, and long-term library management

Reference Guide

Man-page style documentation for every command, filter, and configuration option.

- CLI Conventions — Global flags, scripting patterns, exit codes
- Setup Commands — `init`, `volume` (`add`, `list`, `set-purpose`, `combine`, `split`, `rename`, `remove`)
- Ingest Commands — `import`, `watch`, `delete`, `tag` (`rename`, `split`, `delete`, `clear`, `fix-unicode`, `scan`, `expand-ancestors`, `export-vocabulary`), `edit`, `group`, `split`, `auto-group`, `auto-tag`, `embed`, `ai export-vocabulary`, `audio analyze`, `describe`
- Organize Commands — `collection`, `saved-search`, `stack`, `auto-stack`, `faces`
- Retrieve Commands — `search`, `show`, `preview`, `export`, `contact-sheet`, `duplicates`, `stats`, `status`, `backup-status`, `doc`, `licenses`, `serve`, `shell`
- Maintain Commands — `undo`, `history`, `doctor`, `verify`, `sync`, `refresh`, `sync-metadata`, `write-back`, `cleanup`, `dedup`, `trash`, `relocate`, `update-location`, `generate-previews`, `fix-roles`, `fix-dates`, `fix-recipes`, `create-sidecars`, `rebuild-catalog`, `migrate`
- Search Filters — Complete filter syntax reference
- Format Templates — Output format presets, custom templates, placeholders
- Configuration — `maki.toml` reference

- Data Model — Asset, Variant, Recipe, Volume, and FileLocation entities
- VLM Model Guide — Vision-language models for `maki describe`: tested models, backends, hardware guide
- Licenses & Acknowledgements — MAKI license, third-party Rust crates, AI models, external tools

Developer Guide

Technical documentation for integrators and contributors.

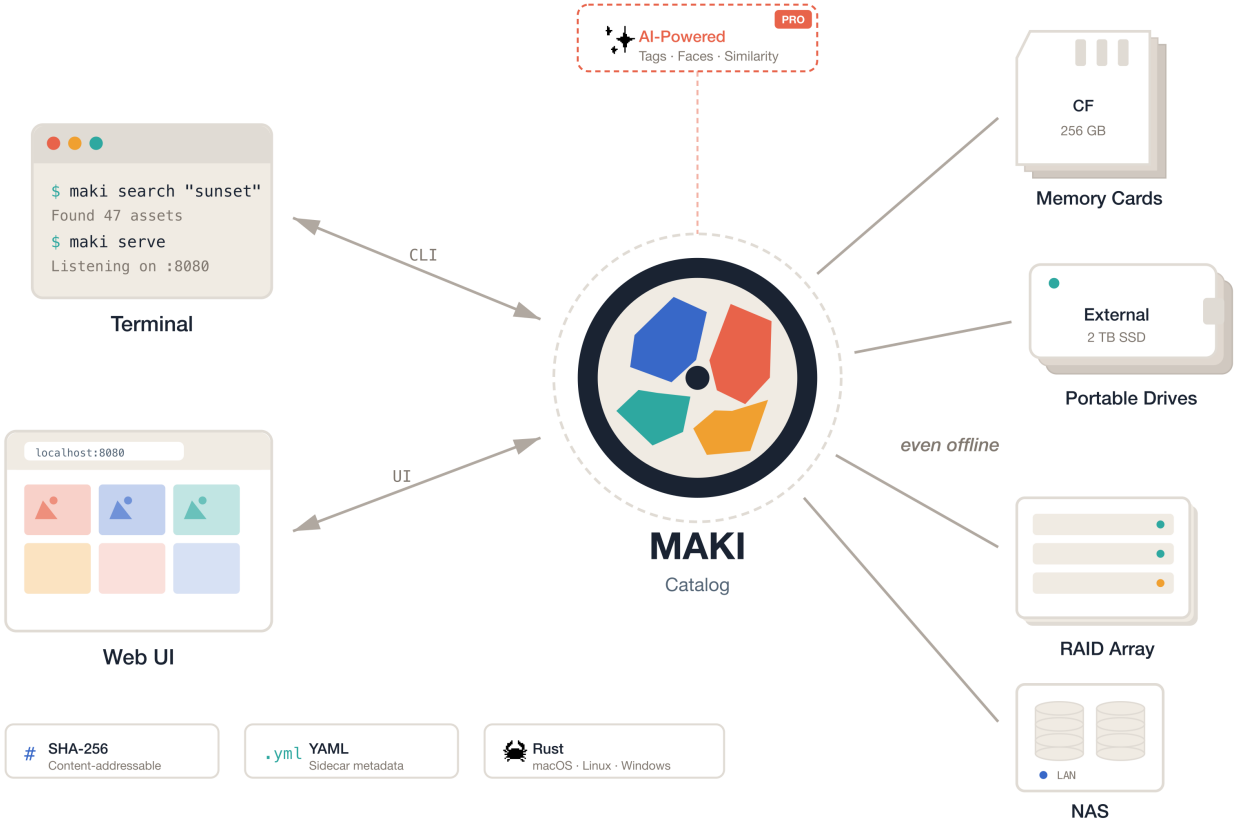
1. REST API — Complete web API documentation
2. Module Reference — Rust module overview and dependency graph
3. Building & Testing — Build commands, tests, and release process

---

Version: v4.12.0 | **Source**: GitHub | **License**: Apache-2.0

Overview & Concepts

Overview



MAKI is a digital asset manager built in Rust for photographers and media professionals who manage large collections — terabytes of images, videos, and processing files — across multiple storage devices.

At its core, MAKI maintains a **local catalog** on your machine that tracks every file across all your volumes: memory cards, portable SSDs, RAID arrays, and network storage. Files are identified by their SHA-256 content hash, enabling deduplication and integrity verification regardless of where copies are stored. Even when drives are offline, you can browse, search, and organize your assets using cached metadata and preview thumbnails.

You interact with MAKI through two interfaces: a **command-line interface** (CLI) for scripting, batch operations, and automation, and a **web UI** served locally for visual browsing, lightbox viewing, strolling through similar images, and interactive editing. Both interfaces work against the same catalog.

Metadata is stored as human-readable **YAML sidecar files** — the source of truth that survives database

rebuilds and is friendly to version control. A SQLite database serves as a fast query index that can be fully reconstructed from the sidecars at any time.

MAKI runs on **macOS**, **Linux**, and **Windows**.

Editions

MAKI is available in two editions:

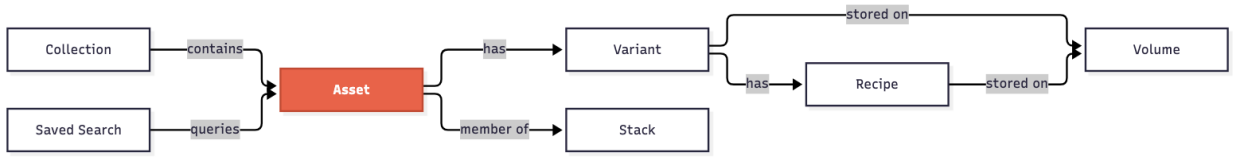
Edition	Features
MAKI	The full digital asset manager: import, organize, browse, search, export, contact sheets, stacks, collections, web UI, and all maintenance commands.
MAKI Pro	Everything in MAKI, plus: **auto-tagging**, **face detection and recognition**, **visual similarity search** (including Stroll), **text-to-image search**, **VLM-powered image descriptions**, and **bidirectional XMP writeback/sync** — all running locally with no cloud dependency.

Pre-built binaries for both editions are available on the GitHub releases page. The Pro edition includes ONNX Runtime for on-device AI inference. On macOS, GPU acceleration via CoreML (Neural Engine on Apple Silicon, Metal on Intel) is included automatically.

Throughout this manual, Pro features are marked with *(Pro)* in section headings, table entries, and option descriptions.

Core Concepts

MAKI organizes your files around these core concepts.



Asset

The central entity. An Asset represents a single logical image or media item – "photo of sunset at the beach" – regardless of how many files exist for it. An Asset has a deterministic UUID derived from the content hash of its first variant, plus optional metadata: name, description, tags (keywords), rating (1–5 stars), and color label (Red, Orange, Yellow, Green, Blue, Pink, Purple – a superset of Lightroom's palette that matches CaptureOne). Tags support hierarchy using `|` as a separator (e.g., `animals|birds|eagles`), aligned with Lightroom and CaptureOne conventions; searching for a parent tag matches all descendants.

Variant

A concrete file belonging to an Asset. A RAW file, its JPEG conversion, and a high-res TIFF export are three Variants of the same Asset. Each Variant is identified by its **SHA-256 content hash** – the same file always produces the same hash, regardless of where it is stored.

Variants carry a **role** that describes their purpose:

Role	Meaning
Original	Camera original (RAW, in-camera JPEG)
Alternate	Another original of the same shot (e.g. in-camera JPEG alongside RAW)
Processed	Intermediate edit (PSD, layered TIFF)
Export	Final deliverable (resized JPEG, web TIFF)
Sidecar	Accompanying non-media file

Each Variant tracks one or more **file locations** – a volume plus a relative path. A single Variant can exist on multiple drives (copies, backups on a NAS), and MAKI tracks them all. Each location stores a `verified_at` timestamp from the last integrity check.

Recipe

A processing sidecar file (`.xmp`, `.cos`, `.cot`, `.cop`, `.pp3`, `.dop`, `.on1`) attached to a Variant. Recipes store edits from tools like CaptureOne, Lightroom, RawTherapee, DxO, and ON1. Unlike Variants, Recipes are identified by **location** (volume + path) rather than content hash, because external tools routinely modify them in place. This means re-importing after an external edit updates the existing Recipe rather than creating a duplicate.

MAKI reads metadata from Recipes during import (rating, tags, description, color label) and can write changes back to XMP files via `maki writeback` or `maki sync-metadata`.

Volume

A registered storage device: local disk, external drive, or network share. Each Volume has a human-readable label and a mount point. MAKI detects online/offline status at runtime, so you can browse and search assets on unmounted drives using cached metadata and preview thumbnails.

Stack

A group of related but independent assets – burst shots, exposure brackets, or similar scenes that you want to collapse into a single entry in the browse grid. Each asset can belong to at most one stack. Members are position-ordered; position 0 is the **pick** (the representative image shown when the stack is collapsed). Unlike variants (which merge files into one asset), stacked assets keep their own ratings, tags, and descriptions.

Collection

A manual album – a named list of assets you curate by hand. Collections are static: adding or removing assets requires an explicit action. Use collections for portfolios, client deliveries, print selections, or any grouping that doesn't follow a search pattern.

Saved Search

A smart album – a named search query that dynamically matches assets. Saved searches appear as quick-access chips in the browse toolbar. When your catalog changes (new imports, rating changes, tag edits), the results update automatically because the query is re-evaluated each time.

Content-Addressable Storage

Every file imported into MAKI is hashed with SHA-256. This hash becomes the file's identity:

- **Deduplication**: Importing the same file twice (even from different paths or drives) recognizes it as the same content and adds the new location to the existing Variant rather than creating a duplicate.
- **Integrity verification**: The `verify` command re-hashes files on disk and compares against stored hashes to detect corruption or bit rot.
- **Transparent relocation**: Moving a file to a different drive does not change its identity. The `relocate` and `update-location` commands update the catalog to reflect the new path.

Originals (RAW files, camera JPEGs) are immutable – their content never changes, so their hash is stable forever. Recipe files are the exception: they are modified by external tools, so MAKI tracks them by location and updates their stored hash when changes are detected.

Two-Tier Storage

MAKI uses a dual-storage architecture. Neither tier alone is sufficient; together they provide both robustness and performance.

YAML sidecar files (source of truth): One `.yml` file per Asset, stored in the catalog's `metadata/` directory. Contains the complete Asset record: metadata, all Variants, all Recipes, all FileLocations. Human-readable, diffable, and version-control friendly. If the SQLite database is ever lost or corrupted, running `rebuild-catalog` reconstructs it entirely from these files.

SQLite catalog (derived cache): A single `catalog.db` file providing fast indexed queries. Contains denormalized columns for efficient browse-grid rendering (best variant hash, primary format, variant count). The catalog is always rebuildable from sidecars – it is a performance optimization, not a source of truth.

This design means you never lose data from a database corruption event. The YAML sidecars are the authoritative record.

Multi-Volume Support

Real-world photo libraries span multiple storage devices: a fast internal SSD for current projects, external drives for archives, a NAS for backups. MAKI handles this natively:

- **Register volumes** with `maki volume add` – give each drive a label and mount point.
- **Import from any volume** – MAKI auto-detects which registered volume a file path belongs to and stores a volume-relative path.
- **Offline browsing** – when a drive is unmounted, its assets remain searchable and browsable via cached metadata and preview thumbnails in the local catalog.
- **Cross-volume operations** – `relocate` copies or moves assets between volumes, `verify` checks file integrity across all online volumes, and `cleanup` detects missing files on online volumes.

See Setup for volume registration and Maintenance for cross-volume operations.

Variant Grouping

During import, MAKI automatically groups related files into a single Asset using **stem-based matching**: files that share the same filename stem in the same directory are grouped together.

For example, importing a directory containing:

```
DSC_4521.NEF       --> Original variant (RAW)
DSC_4521.jpg       --> Original variant (JPEG)
DSC_4521.xmp       --> Recipe (attached to NEF variant)
```

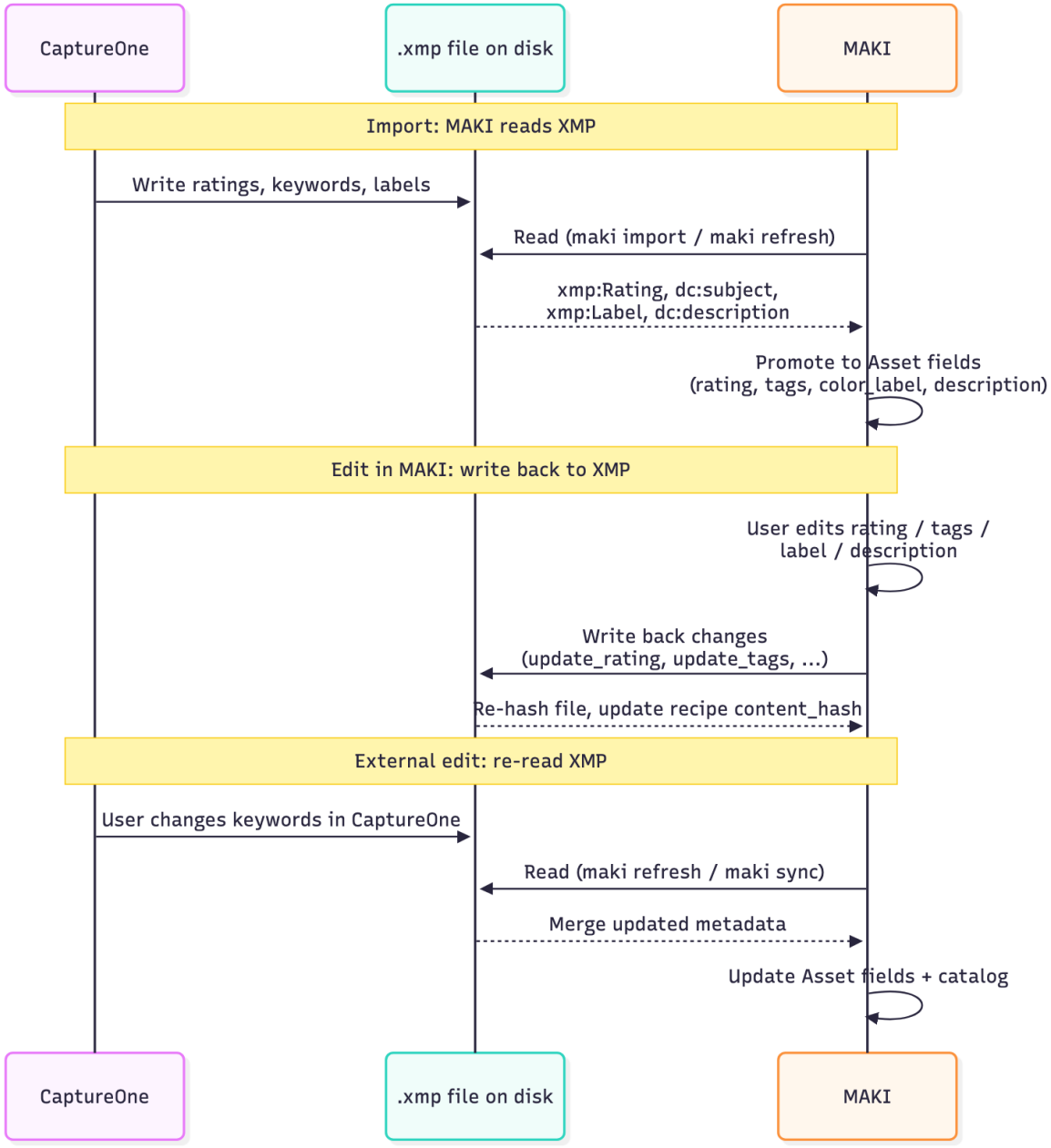
produces one Asset with two Variants and one Recipe. The RAW file takes priority as the primary variant (defining the asset's identity, EXIF metadata, and deterministic UUID).

For files that end up in different directories – common with CaptureOne exports that land in an `output/` subfolder – the `auto-group` command performs **fuzzy prefix matching** across the entire catalog. It matches stems like `DSC_4521` and `DSC_4521-1-HighRes` by checking that the shorter stem is a prefix of the longer one and the next character is a non-alphanumeric separator (`-`, `_`, space, `(`).

See Ingesting Assets for import details and Organizing Assets for manual and automatic grouping.

Bidirectional XMP Sync

MAKI maintains two-way synchronization with `.xmp` sidecar files, enabling a round-trip workflow with tools like CaptureOne and Lightroom.



Catalog to disk: When you change a rating, tag, description, or color label via the CLI (`maki edit`, `maki tag`) or the web UI, MAKI can write the change back to any associated `.xmp` files on disk, then re-hash them and update the stored recipe hash. Writeback must be explicitly enabled in `maki.toml` – by default, edits are stored only in the catalog and YAML sidecars:

```
[writeback]
enabled = true
```

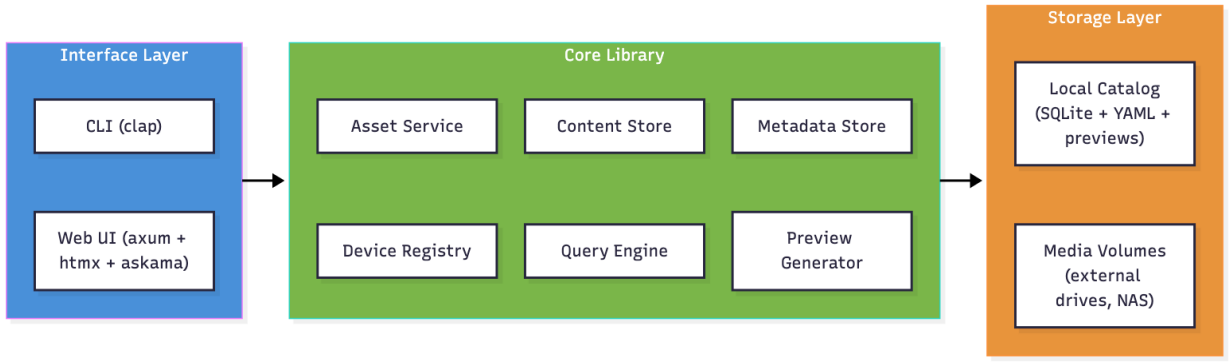
Disk to catalog: When CaptureOne or another tool modifies an `.xmp` file, running `maki refresh` or `maki sync` detects the changed hash, re-extracts the XMP metadata, and updates the Asset in both the catalog and sidecar YAML.

Tags added independently in CaptureOne are preserved during write-back – MAKI uses operation-level deltas (add/remove specific tags) rather than overwriting the entire tag list.

See Ingesting Assets for XMP extraction during import and Maintenance for the `refresh` and `sync` commands.

Architecture

MAKI is structured in three layers.



Interface Layer

CLI (clap): Subcommand-based interface (`maki import`, `maki search`, `maki serve`, etc.). Supports `--json` for machine-readable output, `--log` for per-file progress, `--format` for custom output templates.

Web UI (axum + htmx + askama): Browser-based interface started with `maki serve`. Provides a browse grid with filters, inline editing, batch operations, and keyboard navigation. Uses htmx for partial page updates without a JavaScript framework.

Core Library

Component	Responsibility
Asset Service	Orchestrates import, grouping, relocation, verification, preview generation
Content Store	SHA-256 hashing, deduplication, hash-to-location mapping
Metadata Store	YAML sidecar read/write, the authoritative data layer
Device Registry	Volume tracking, mount point detection, online/offline status
Query Engine	SQLite search, metadata editing (tags, rating, label), auto-grouping

Component	Responsibility
Preview Generator	Thumbnail creation via `image` crate, `dcraw`/LibRaw (RAW), `ffmpeg` (video)

Additional modules handle EXIF extraction, XMP reading/writing, configuration parsing (`maki.toml`), output formatting, and collection/saved-search management.

Storage Layer

Local Catalog (always available on local disk):

- `catalog.db` – SQLite database (derived index, rebuildable)
- `metadata/` – YAML sidecar files (source of truth)
- `previews/` – JPEG thumbnails for offline browsing (800px)
- `smart-previews/` – high-resolution previews for zoom/pan (2560px)
- `volumes.yaml` – registered volume definitions
- `searches.toml` – saved searches
- `collections.yaml` – static album definitions
- `stacks.yaml` – stack definitions (member order and pick)
- `maki.toml` – configuration
- `vocabulary.yaml` – planned tag hierarchy for autocomplete

Media Volumes (may be offline): The actual asset files on external drives, NAS, or local directories. MAKI never moves or modifies original media files unless explicitly asked (via `relocate --remove-source`).

Preview Generation

MAKI generates preview thumbnails (800px JPEG by default) for each variant during import, enabling offline browsing even when media volumes are unmounted. Optionally, high-resolution smart previews (2560px) can be generated alongside thumbnails for zoom and pan in the web UI lightbox — enable with `maki import --smart` or `[import] smart_previews = true` in `maki.toml`. Different file types use different strategies:

File type	Strategy
Standard images (JPEG, PNG, TIFF, WebP)	`image` crate (native Rust)
RAW files (NEF, ARW, CR3, …)	`dcraw` or `dcraw_emu` (LibRaw)
Video files (MP4, MOV, …)	`ffmpeg` (frame extraction)
Audio, documents, other	Info card (metadata summary rendered as JPEG)

Preview failure never blocks import. If an external tool is missing, an info card is generated as a fallback.

The display logic prefers Export > Processed > Original variant previews, with standard image formats preferred over RAW within the same role. This means your best-quality deliverable is shown in the browse grid, not the camera original.

Preview settings (max edge size, format, quality) are configurable in `maki.toml`. See Setup for configuration details and Ingesting Assets for the `generate-previews` command.

Collections, Stacks, and Saved Searches

MAKI provides three ways to organize assets into groups:

Collections (static albums): Manually curated lists of asset IDs. You explicitly add and remove assets. Backed by both SQLite (for fast queries) and a `collections.yaml` file (for persistence across catalog rebuilds). Use these for curated sets like "Portfolio" or "Client Deliverables".

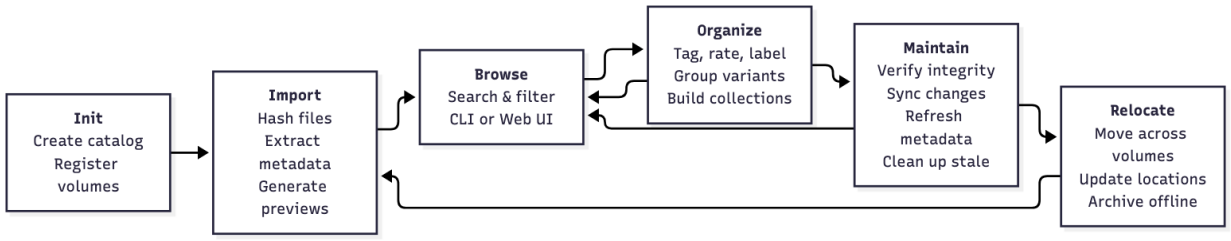
Stacks: Lightweight anonymous groups for burst shots, bracketing, or similar scenes. The browse grid collapses each stack to show only the pick image with a count badge, reducing visual clutter. Backed by a `stacks.yaml` file for persistence across catalog rebuilds. Use these to declutter the browse grid without permanently merging variants.

Saved searches (smart albums): Named queries stored in `searches.toml`. A saved search is a set of filter criteria (tags, rating, date range, volume, etc.) that is re-evaluated every time you run it. Results update automatically as your catalog changes. Use these for dynamic views like "5-star landscapes" or "untagged imports from last week".

All three are accessible from the CLI and the web UI. See Organizing Assets for detailed usage.

The Round-Trip Workflow

A typical MAKI workflow follows this cycle:



1. **Init** – Create a catalog directory and register your storage volumes. See Setup.
2. **Import** – Point MAKI at directories of files. It hashes everything, extracts EXIF/XMP metadata, groups related files into Assets, generates preview thumbnails, and builds the catalog. See Ingesting Assets.
3. **Browse** – Search and filter your catalog by tags, rating, color label, camera, date, format, volume, path, collection, or free text. Use the CLI for scripting or the web UI for visual browsing. See Browsing & Searching and Web UI.
4. **Organize** – Add tags, set ratings and color labels, write descriptions, group variants, build collections, and save searches. Changes write back to XMP files for interoperability. See Organizing Assets.
5. **Maintain** – Verify file integrity against stored hashes, sync the catalog with disk after external tools move or rename files, refresh metadata from modified XMP sidecars, and clean up stale location records. See Maintenance.
6. **Relocate** – Copy or move assets between volumes (external drives, NAS, archive storage). Update location records after manual file moves. The cycle continues as you import new work or re-process existing assets.

Each step is covered in detail in the following chapters.

Next: Setup – Installation, catalog initialization, volume registration, and configuration.

Setup

This chapter covers building MAKI from source, initializing a catalog, registering storage volumes, and configuring the system to match your workflow.

Installation

Pre-built binaries

Download a pre-built binary from the GitHub releases page. Each release provides two editions for every platform:

Archive name	Edition
`maki-VERSION-PLATFORM.tar.gz`	MAKI (standard)
`maki-VERSION-PLATFORM-pro.tar.gz`	MAKI Pro (with AI features)

Extract the archive and copy the `maki` binary to a directory on your `PATH`:

```
tar xzf maki-4.2.1-macos-arm64-pro.tar.gz
cp maki /usr/local/bin/
```

Verify the installation:

```
maki --version
# MAKI:     maki 4.2.1
# MAKI Pro: maki 4.2.1 Pro
```

Building from source

MAKI is written in Rust. You need a working Rust toolchain (rustc + cargo), version 1.89 or newer. Install one via rustup if you have not already.

```
git clone https://github.com/thoherr/maki.git
cd maki
cargo build --release                    # standard edition
cargo build --release --features pro     # Pro edition
cargo build --release --features pro,ai-gpu  # Pro with GPU acceleration (macOS only)
```

The binary is at `target/release/maki`. Copy or symlink it to a directory on your `PATH`.

Supported platforms

MAKI builds and runs on **macOS**, **Linux**, and **Windows**. Both x86_64 and ARM (Apple Silicon) are supported.

External tools (highly recommended)

MAKI handles standard image formats (JPEG, PNG, TIFF, WebP) natively. The following external tools extend its capabilities:

Tool	Purpose	Install
dcraw or **dcraw_emu** (LibRaw)	RAW file previews (NEF, ARW, CR2, CR3, etc.)	`brew install dcraw` or `brew install libraw` on macOS; your package manager on Linux; `winget install LibRaw.LibRaw` or `scoop install libraw` on Windows
ffmpeg / **ffprobe**	Video thumbnail extraction and video metadata (duration, codec, resolution, framerate). `ffprobe` is included with the ffmpeg package.	`brew install ffmpeg` on macOS; your package manager on Linux; `winget install Gyan.FFmpeg` or `scoop install ffmpeg` on Windows
curl	AI model download and VLM image descriptions	Pre-installed on macOS and most Linux distributions; `winget install cURL.cURL` or `scoop install curl` on Windows
keyfinder-cli / **beat_this** *(optional)*	Musical key and tempo detection for `maki audio analyze`	keyfinder-cli and beat_this; commands configurable via `[audio]` in `maki.toml`

When an external tool is missing, MAKI prints a warning on first use explaining what is needed and why. It still imports RAW and video files, but generates an info card (a placeholder JPEG showing file metadata) instead of a rendered preview. You can install the tools later and run `maki generate-previews --force` to regenerate real previews.

Initializing a Catalog

A catalog is a directory that holds MAKI's database, configuration, metadata sidecars, and preview images. Create one by navigating to the directory you want as the catalog root and running `maki init`:

```
mkdir ~/Photos
cd ~/Photos
maki init
```

Output:

```
Initialized new maki catalog in /Users/you/Photos
```

This creates the following structure:

```
~/Photos/
  maki.toml          # Configuration file
  vocabulary.yaml    # Planned tag hierarchy for autocomplete
  .gitignore         # Excludes derived files for optional git backup
  catalog.db         # SQLite database (cache/index)
  volumes.yaml       # Registered storage volumes
  metadata/          # YAML sidecar files (source of truth)
  previews/          # Preview thumbnails (800px)
  smart-previews/    # High-resolution previews for zoom/pan (2560px)
```

The `vocabulary.yaml` defines your planned tag hierarchy — MAKI uses it to offer autocomplete suggestions for tags you've planned but haven't used yet. Edit it to match your photography focus. See the Tagging Guide for vocabulary design recommendations.

The `.gitignore` is set up so you can optionally track the catalog with git for backup and history. See Backing Up Your Catalog with Git for details.

How catalog detection works

After initialization, you can run `maki` commands from the catalog root or any subdirectory. MAKI locates the catalog by walking up from your current working directory, looking for a `maki.toml` file. This means you can organize files in subdirectories and still run commands without specifying the catalog path.

```
cd ~/Photos
maki stats            # works -- maki.toml is here

cd ~/Photos/metadata
maki stats            # also works -- finds maki.toml in parent

cd /tmp
maki stats            # fails -- no maki.toml above /tmp
```

If no catalog is found, MAKI prints:

```
Error: No maki catalog found. Run `maki init` to create one.
```

Reinitializing

`maki init` refuses to overwrite an existing catalog. If you need to start fresh, delete the catalog files first (`maki.toml`, `catalog.db`, `metadata/`, `previews/`, `smart-previews/`, `volumes.yaml`) and run `maki init` again.

Registering Volumes

A **volume** represents a storage location – a local directory, an external drive, or a NAS mount point. Before importing files, you must register the volume they live on.

Adding a volume

```
maki volume add "Photos 2024" /Volumes/PhotosDrive
```

Output:

```
Registered volume 'Photos 2024' (a1b2c3d4-e5f6-7890-abcd-ef1234567890)
  Path: /Volumes/PhotosDrive
```

Each volume gets a UUID that stays constant even if the drive letter or mount point changes. The label is a human-readable name you choose.

If you omit the label, it is auto-derived from the last component of the path. This is handy for transient volumes like memory cards:

```
# Auto-label: derives "EOS_DIGITAL" from the path
maki volume add /Volumes/EOS_DIGITAL --purpose media
```

A few more examples:

```
# External SSD
maki volume add "Travel SSD" /Volumes/TravelPhotos
```

```
# Network-attached storage
maki volume add "NAS Archive" /Volumes/nas/photos

# Local directory
maki volume add "Local Work" /Users/you/Photography
```

Listing volumes

```
maki volume list
```

Output:

```
Photos 2024 (a1b2c3d4-e5f6-7890-abcd-ef1234567890) [online]
  Path: /Volumes/PhotosDrive
Travel SSD (b2c3d4e5-f6a7-8901-bcde-f12345678901) [offline]
  Path: /Volumes/TravelPhotos
```

Online vs. offline

MAKI checks whether each volume's mount point directory exists on disk:

- **Online**: The directory exists. maki can read files, generate previews, and verify integrity.
- **Offline**: The directory does not exist (drive disconnected, NAS unreachable). maki still knows about the volume and its assets, but file operations are skipped gracefully.

This design lets you manage a photo library that spans multiple external drives. You can search, browse, and view cached previews for assets on offline volumes. When you reconnect a drive, MAKI picks it up automatically.

Multiple volumes

There is no limit on the number of volumes. A typical setup might look like:

- A fast local SSD for current work
- One or more external drives for archive storage
- A NAS for backups

Assets can have files on multiple volumes simultaneously (see Maintenance for the `relocate` command).

Volume purposes

Each volume can optionally be assigned a **purpose** that describes its role in your storage hierarchy:

Purpose	Meaning
`media`	Transient source media — memory cards, card readers, camera USB
`working`	Active editing drive — fast SSD with current projects
`archive`	Long-term primary storage — the "master" copy
`backup`	Redundancy copy — exists purely for safety

Purpose	Meaning
`cloud`	Cloud-synced folder (Dropbox, iCloud, Google Drive)

```
maki volume add "Card"       /Volumes/CARD         --purpose media
maki volume add "Laptop SSD" /Volumes/MacintoshHD --purpose working
maki volume add "Archive"    /Volumes/MediaDrive   --purpose archive
maki volume add "Backup A"   /Volumes/BackupDisk   --purpose backup
maki volume add "Dropbox"    ~/Dropbox/Photos      --purpose cloud
```

Purpose metadata drives two features:

- **Duplicate analysis** (`maki duplicates`): Distinguishes unwanted duplicates (same file twice on the same working drive) from wanted redundancy (same file on an archive and a backup).
- **Backup coverage** (`maki backup-status`): Reports which assets lack copies on archive or backup volumes and flags at-risk assets. Media volumes are excluded from coverage calculations — a file only on a memory card is not considered backed up.

You can set or change a purpose at any time:

```
maki volume set-purpose "Laptop SSD" archive
maki volume set-purpose "Laptop SSD" none      # clear
```

Volumes without a purpose are treated as unclassified — they still work for import, search, and all other operations, but are excluded from purpose-based analysis.

Symlinks and path resolution

MAKI resolves symlinks when registering volumes and importing files. All paths stored in the catalog are **physical (canonical) paths**, not the symlink paths you may see in your filesystem.

This matters when your directory layout uses symlinks to span multiple disks. For example, if you have:

```
/Volumes/Pictures/masters/2025/   (real directory)
/Volumes/Pictures/masters/2026 → /Volumes/Dropbox/masters/2026/  (symlink)
```

and you register `/Volumes/Pictures` as a volume, then:

- Files under `.../masters/2025/` are tracked on the "Pictures" volume as expected.
- Files under `.../masters/2026/` resolve through the symlink to `/Volumes/Dropbox/masters/2026/`, which is **outside** the Pictures volume mount point. Import will fail with "No registered volume contains path" unless the Dropbox path is also registered as a volume.

Why MAKI resolves symlinks:

- **Reliable verification.** `maki verify` re-hashes files by their catalog path. If the catalog stored a symlink path and the symlink later changed target, verification would silently check the wrong file — or fail when the link breaks.
- **Correct offline detection.** A volume is "online" when its mount point exists. A broken symlink inside an online volume would cause confusing partial failures.
- **Predictable sync behavior.** `maki sync` detects moved and missing files by comparing disk state to catalog paths. Symlink changes would cause false "moved" or "missing" reports for every file under the changed link.
- **Unambiguous volume mapping.** Each file belongs to exactly one volume (determined by longest mount-point prefix match on the physical path). Symlinks could cause the same file to appear to belong to two different volumes depending on which path you use.

The recommended workaround is to register each physical storage location as its own volume:

```
maki volume add "Pictures"  /Volumes/Pictures  --purpose archive
maki volume add "Dropbox"   /Volumes/Dropbox   --purpose cloud
```

Both volumes work independently. Import auto-detects the correct volume based on the resolved physical path. `backup-status` shows correct copy counts across both. You can still navigate your symlinked directory structure normally — maki simply tracks where files physically reside.

Removing a volume

If a volume is no longer needed (drive decommissioned, cloud service cancelled), you can cleanly remove it:

```
maki volume remove "Old Drive"          # report what would be removed
maki volume remove "Old Drive" --apply  # actually remove
```

This removes all file location and recipe records on that volume from the catalog and sidecar files, deletes any assets that become orphaned (no remaining file locations), and cleans up orphaned preview files. Without `--apply`, it runs in report-only mode so you can review the impact first.

See the volume remove reference for details.

Combining volumes

If you initially registered year-based subdirectories as separate volumes (e.g., `media_2024`, `media_2025` under `/Volumes/Media`) and now want to consolidate them into a single disk-level volume, use `volume combine`:

```
# First register the parent directory as a volume
maki volume add "Media" /Volumes/Media

# Preview what combining would do
maki volume combine "media_2024" "Media"
# Would combine 'media_2024' into 'Media': 4832 locations, 312 recipes (3210 assets, prefix 'media_2024/')

# Execute the combination
maki volume combine "media_2024" "Media" --apply
```

The source volume's mount point must be a subdirectory of the target's. All file paths are automatically rewritten with the appropriate prefix (e.g., `photo.jpg` becomes `media_2024/photo.jpg`). After combining, the source volume is removed. You can repeat this for each year-volume to consolidate into one.

See the volume combine reference for details.

Splitting a volume

The inverse of combine: split a subdirectory off into its own volume. This is useful when you physically move a folder to a new drive:

```
# Preview what splitting would do
maki volume split "Photos" "Archive 2024" --path "Archive/2024"

# Execute the split
maki volume split "Photos" "Archive 2024" --path "Archive/2024" --apply
```

All file locations under the specified path are reassigned to the new volume, with path prefixes rewritten accordingly. You can optionally assign a purpose to the new volume:

```
maki volume split "Photos" "Archive 2024" --path "Archive/2024" --purpose archive --apply
```

See the volume split reference for details.

Renaming a volume

If a drive label changes or you want a clearer name:

```
maki volume rename "Old Label" "New Label"
```

This updates the volume label everywhere (catalog, sidecar YAML files, volume registry). No files are moved or modified on disk.

Configuration (maki.toml)

The `maki.toml` file at the catalog root controls MAKI's behavior. All sections are optional – an empty file (or one with only comments) uses sensible defaults.

Here is an example showing common options (see the Configuration Reference for the full list):

```
# Default volume for import when auto-detection is ambiguous
default_volume = "a1b2c3d4-e5f6-7890-abcd-ef1234567890"

[preview]
max_edge = 800        # Maximum width/height in pixels (default: 800)
format = "jpeg"       # "jpeg" or "webp" (default: "jpeg")
quality = 85          # JPEG quality 1-100 (default: 85; ignored for webp)

[serve]
port = 8080           # Web UI port (default: 8080)
bind = "127.0.0.1"    # Bind address (default: "127.0.0.1")

[import]
exclude = [           # Glob patterns to skip during import
    ".DS_Store",
    "Thumbs.db",
    "*.tmp",
]
auto_tags = [         # Tags automatically applied to every new asset
    "inbox",
    "unreviewed",
]
```

Section summary

`default_volume` – UUID of the volume to use when `maki import` cannot auto-detect the correct volume from the file path. Useful if you always import from the same drive.

`[preview]` – Controls preview generation. `max_edge` sets the longest side of generated thumbnails. `format` chooses between JPEG (smaller, lossy) and WebP (lossless via the `image` crate). `quality` only applies to JPEG output.

`[serve]` – Web UI server settings. Change `bind` to `"0.0.0.0"` to allow access from other machines on your network. CLI flags `--port` and `--bind` override these values.

`[import]` – `exclude` patterns are matched against filenames (not full paths) using glob syntax. Common choices: OS junk files, editor temp files, thumbnail caches. `auto_tags` are merged with any tags extracted from XMP metadata during import; useful for tagging everything in a session as "unreviewed" for later triage.

Recommended settings

If you use Lightroom, CaptureOne, or another tool that reads `.xmp` sidecar files, enable XMP writeback so your edits in MAKI are visible to those tools. Also make sure your photo editor is configured to **write XMP sidecar files** — most tools don't do this by default. See Working with External Tools for per-tool settings.

```
[writeback]
enabled = true
```

> **Warning:** Enabling writeback causes MAKI to modify `.xmp` files on your storage volumes whenever you change ratings, tags, labels, or descriptions. If you prefer MAKI to keep its edits strictly in its own catalog (YAML sidecars + SQLite), leave writeback disabled (the default). You can always enable it later and run `maki writeback --all` to push all edits at once.

For a complete reference of every option and its behavior, see the Configuration Reference.

AI Models *(Pro)*

MAKI Pro uses SigLIP vision-language models for `text:` search (find images by typed description), similarity browsing, and `auto-tag`. The model is downloaded once on demand and cached in `~/.maki/models/`.

Available models

Model ID	Size	Embedding dim	Languages	Use case
`siglip-vit-b16-256` *(default)*	~207 MB	768	English only	Fast, good quality, English queries
`siglip-vit-l16-256`	~670 MB	1024	English only	English, higher accuracy, slower
`siglip2-base-256-multi`	~410 MB	768	**Multilingual**	German, French, Spanish, Italian, Japanese, Chinese, etc.
`siglip2-large-256-multi`	~920 MB	1024	**Multilingual**	Same languages, higher accuracy, slower

Embeddings are stored per `(asset_id, model_id)` in both the SQLite catalog and the on-disk embedding store. This means you can switch back and forth between models without losing any data — each model has its own independent embedding set, and the others stay untouched.

Choosing a model

If you only ever type `text:` queries in English, the default `siglip-vit-b16-256` is the right choice. If you want to type queries in your native language — `text:"Sonnenuntergang am Strand"`, `text:"coucher de soleil sur la plage"`, `text:"atardecer en la playa"` — switch to `siglip2-base-256-multi`.

Switching models

Switching the active AI model is a three-step process:

Step 1: Update `maki.toml` to point at the new model:

```
[ai]
model = "siglip2-base-256-multi"
```

Step 2: Download the model files (one-time, ~410 MB for the multilingual base model; ~920 MB for large):

```
maki auto-tag --download siglip2-base-256-multi
```

Step 3: Generate embeddings for the new model:

```
maki embed ''
```

This processes only the assets that **don't yet have an embedding for the new model** and skips everything that does. Because embeddings are keyed by `(asset_id, model_id)`, the existing English-model embeddings count for nothing here — every asset starts as "missing" for the new model and gets embedded.

> **Important — don't use `--force` here.** The `--force` flag re-embeds *every* asset, including those that already have an embedding for the current model. That's the right thing for a "rebuild from scratch" scenario (e.g. after replacing the on-disk model file with a corrected version), but for a model switch it just wastes hours of CPU/GPU time. On a 260k catalog, embedding from scratch takes ~36 hours; without `--force` the same command resumes from where it left off and only does the missing work.

If you interrupted an earlier `maki embed` run (Ctrl-C, machine sleep, power loss), simply running `maki embed ''` again resumes — all assets that were already embedded for the active model are skipped. This makes the migration restart-safe and crash-safe.

Verifying the active model and embedding counts

To see how many embeddings exist per model in your catalog:

```
sqlite3 .maki/catalog.db "SELECT model, COUNT(*) FROM embeddings GROUP BY model;"
```

You'll see one row per model that has any embeddings. After switching, the count for the old model stays where it was, and the count for the new model grows from zero as you run `maki embed ''`.

Switching back

Because the old embeddings are still in the catalog, switching back is instant — just put the old model id back in `maki.toml`. No re-embedding needed; the existing embeddings are already there.

Listing and managing models

```
maki auto-tag --list-models                 # show all known models with downloaded status and sizes
maki auto-tag --download <model-id>         # download a specific model
maki auto-tag --remove-model <model-id>     # delete cached files for a model (frees disk space)
```

The active model is the one referenced in `[ai] model` in `maki.toml`, or the CLI `--model` flag if given.

Disk space cleanup

Old model embeddings remain in the catalog and embedding store after a switch. They are small (~3 KB per asset for base, ~4 KB for large) and don't slow anything down — `text:` and similarity search only query the active model. If you're certain you don't want to switch back, you can delete unused embeddings directly:

```
sqlite3 .maki/catalog.db "DELETE FROM embeddings WHERE model = 'siglip-vit-b16-256';"
maki cleanup    # removes orphaned on-disk embedding binaries
```

But unless disk space is a real concern, just leaving old embeddings in place is the simplest option.

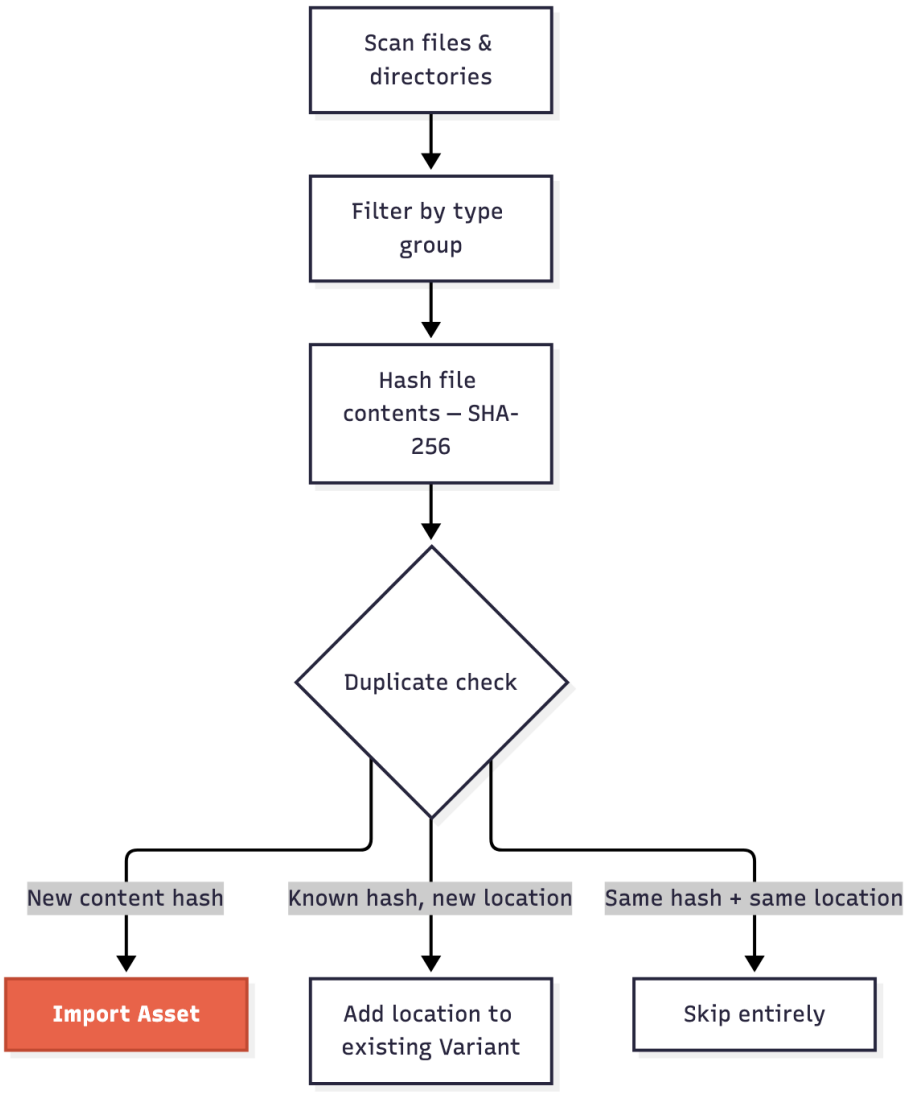
Next: Ingesting Assets – importing files into the catalog, auto-grouping, and metadata extraction.

Ingesting Assets

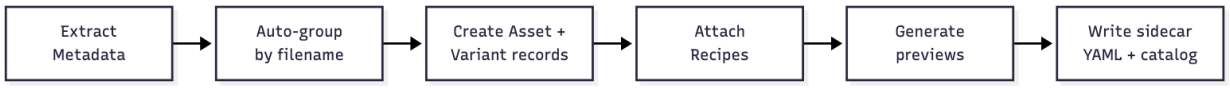
Importing is how files enter the MAKI catalog. Unlike traditional asset managers that copy files into a managed library folder, MAKI catalogs files **in place** on your existing volumes. The import process hashes each file (SHA-256), extracts metadata, generates previews, and records everything in the catalog – but your originals stay exactly where they are.

Import Pipeline

The following diagrams show the stages a file passes through during import:


Scan files & directories
Filter by type group
Hash file contents — SHA-256
Duplicate check
New content hash
Known hash, new location
Same hash + same location
Import Asset
Add location to existing Variant
Skip entirely

For each new file, the import asset pipeline runs:



Metadata precedence: On initial import, fields are set by the first source that provides them: EXIF data is extracted first, then embedded XMP fills any remaining empty fields, then sidecar XMP fills what is still unset. Tags (keywords) are merged from all sources as a union. On subsequent recipe updates (e.g. after editing in Capture One or Lightroom), sidecar XMP takes highest precedence and overwrites rating, description, and color label. The `created_at` date is always derived from EXIF and never overwritten by XMP.

Basic Import

Import a directory of photos:

```
maki import /Volumes/PhotosDrive/Photos/2026-02-20/
```

Import specific files:

```
maki import /Volumes/PhotosDrive/DSC_0042.nef /Volumes/PhotosDrive/DSC_0043.nef
```

The path you provide must reside on a registered volume. MAKI resolves the volume automatically from the file path. If auto-detection picks the wrong volume (for example, when mount points are nested), specify it explicitly:

```
maki import --volume "Photos 2024" /path/to/files
```

Preview Before Committing

Use `--dry-run` to see what would happen without writing anything to the catalog:

```
maki import --dry-run /Volumes/PhotosDrive/Photos/2026-02-20/
```

Files are still hashed during a dry run (so duplicate detection works), but no assets, variants, recipes, or previews are created. The output shows the same counters as a real import:

```
Dry run — would import: 47 imported, 3 skipped, 2 locations added, 5 recipes attached
```

Add `--json` for structured output (includes a `dry_run: true` field) or `--log` to see per-file details.

What Import Does NOT Do

Import does not copy or move your files. The catalog stores references to files on their volumes. This means:

- Your directory structure stays intact.
- Files are not renamed or reorganized.
- The same file can be cataloged from multiple volumes (e.g., original drive and a backup).
- If a volume is disconnected, the catalog remembers the files but marks the volume as offline.

Auto-Grouping

When MAKI imports a directory, it groups files by **filename stem** – the filename without its extension. Files sharing the same stem in the same directory become a single Asset with multiple Variants.

How It Works

Consider a directory containing:

```
DSC_0042.nef
DSC_0042.jpg
DSC_0042.xmp
```

MAKI creates **one Asset** with:

File	Role	Record Type
`DSC_0042.nef`	Original (primary)	Variant
`DSC_0042.jpg`	Alternate variant	Variant
`DSC_0042.xmp`	Processing sidecar	Recipe

RAW files always take priority as the primary variant because they represent the original capture. The primary variant defines the asset's identity and provides the authoritative EXIF data.

Practical Example

A typical CaptureOne session directory might contain:

```
Z91_8561.ARW          # RAW original
Z91_8561.xmp          # XMP sidecar from CaptureOne
Z91_8561.ARW.cos      # CaptureOne settings
Z91_8561.jpg          # JPEG export
```

After import:

```
maki show <asset-id>
```

```
Asset: Z91_8561
  Variant 1 (Original): Z91_8561.ARW [abc123...]
  Variant 2 (Alternate): Z91_8561.jpg [def456...]
  Recipe 1: Z91_8561.xmp
  Recipe 2: Z91_8561.ARW.cos
```

For more advanced grouping across different directories or after the fact, see the auto-group command and group command.

Cross-Directory Grouping with –auto-group

Standard auto-grouping only matches files within the same directory. But photo tools like CaptureOne often place RAW originals and exports in separate folders:

```
2026-02-22/
  Capture/
    DSC_0042.ARW
  Output/
    DSC_0042.JPG
    DSC_0042-1-HighRes.tif
```

Without `--auto-group`, importing both directories creates separate assets for the RAW and exports. The `--auto-group` flag runs a post-import grouping step scoped to the "neighborhood" of the imported files:

```
maki import --auto-group /Volumes/Photos/2026-02-22/Capture /Volumes/Photos/2026-02-22/Output
```

```
Import complete: 3 imported, 1 recipe(s) attached, 3 preview(s) generated
Auto-group: 1 stem group(s), 2 donor(s) merged, 2 variant(s) moved
```

How the neighborhood is determined: For each imported file's directory (e.g., `2026-02-22/Capture/`), MAKI finds the "session root": the deepest folder in the path whose name matches `[group] session_root_pattern` – by default a folder starting with a date, so `2026-02-22/`. It works however deep the exports are nested: importing `2026-02-22/Output/Final/Web/` still resolves to `2026-02-22/`. If no folder matches the pattern, the directory you handed to `maki import` is the session (import a whole client folder and everything below it may group), and if that directory holds the files itself, MAKI goes one level up so `Output/` sees its sibling `Capture/`. It then searches the catalog for all assets under those session roots on the same volume. Only those assets participate in grouping, and the grouping step partitions them by the same rule.

This scoping prevents false positives from restarting camera counters – `DSC_0001` from a January shoot won't be grouped with `DSC_0001` from a June shoot because they live under different session roots.

Incremental workflow: You can also use `--auto-group` when adding exports to a previously imported shoot:

```
# First import: RAW originals only
maki import /Volumes/Photos/2026-02-22/Capture

# Later: CaptureOne exports are ready
maki import --auto-group /Volumes/Photos/2026-02-22/Output
```

The second import detects the existing RAW assets in the sibling `Capture/` directory and groups the exports with them.

Combine with `--dry-run` to preview what would be grouped without making changes:

```
maki import --auto-group --dry-run /Volumes/Photos/2026-02-22/Capture /Volumes/Photos/2026-02-22/Output
```

Recipe Handling

Processing sidecars are files created by RAW processors and image editors to store non-destructive edit settings. maki recognizes the following recipe formats:

Extension	Tool
`.xmp`	Adobe Lightroom, CaptureOne, generic XMP
`.cos`, `.cot`, `.cop`	CaptureOne sessions/catalogs
`.pp3`	RawTherapee
`.dop`	DxO PhotoLab
`.on1`	ON1 Photo RAW

Attachment Rules

Recipes are attached to the **primary variant** of the matching asset (matched by filename stem in the same directory). They are identified by **location** (volume + path), not by content hash. This is an important distinction:

- If you edit settings in CaptureOne and re-import, MAKI updates the existing recipe record rather than creating a duplicate.
- The recipe's content hash is updated to reflect the new file contents.

Standalone Recipe Resolution

You can import recipe files after their media files:

```
# First import: RAW files
maki import /Volumes/PhotosDrive/Shoot/

# Later: CaptureOne creates XMP sidecars
maki import /Volumes/PhotosDrive/Shoot/*.xmp
```

MAKI finds the parent variant by matching the filename stem and directory, then attaches the recipe to it. Both sidecar naming conventions resolve to the same parent: `DSC_1234.xmp` (Capture One, Lightroom) and `DSC_1234.NEF.xmp` (darktable).

Importing never *creates* sidecars. If you tag and rate freshly imported RAW files in MAKI and want those values visible in Capture One or Lightroom, run `maki create-sidecars --apply` before the first session there, or set `[writeback] enabled = true` and `create_sidecars = true`, after which an import that assigns tags writes the sidecars in the same run (see Working with External Tools).

Enabling Non-Default Recipe Groups

By default, only `.xmp` recipe files are imported. CaptureOne, RawTherapee, DxO, and ON1 sidecars require opting in:

```
maki import --include captureone /Volumes/PhotosDrive/Shoot/
maki import --include captureone --include rawtherapee /Volumes/PhotosDrive/Shoot/
```

See Import Options below for the full list of type groups.

Metadata Extraction

MAKI extracts metadata from three sources during import, each contributing different information about the asset.

EXIF Data

Extracted directly from image files (RAW, JPEG, TIFF). Provides camera and capture information:

- Camera make and model
- Lens
- ISO, aperture, shutter speed, focal length
- Image dimensions
- Capture date and time
- GPS coordinates (if present)

EXIF data is stored in the variant's `source_metadata` and used to populate the asset's `captured_at` timestamp.

XMP Sidecar Files

When an `.xmp` file is attached as a recipe, its contents are parsed and merged into the asset:

XMP Field	maki Field	Notes
`dc:subject`	Tags	Merged with existing tags (union)
`dc:description`	Description	Set if not already present
`xmp:Rating`	Rating	Integer 1-5

XMP Field	maki Field	Notes
`xmp:Label`	Color label	Red, Orange, Yellow, Green, Blue, Pink, Purple
`dc:creator`	Source metadata	Stored as `creator`
`dc:rights`	Source metadata	Stored as `rights`

Video Metadata

When `ffprobe` is available (included with the ffmpeg package), MAKI automatically extracts video-specific metadata at import time: duration, codec, resolution, and framerate. This information is stored in the variant's `source_metadata` and used for duration badges in the web UI browse grid. If `ffprobe` is not installed, video files are still imported successfully but without video metadata.

Embedded XMP

JPEG and TIFF files can contain XMP metadata embedded in their binary structure (APP1 marker for JPEG, IFD tag 700 for TIFF). MAKI extracts the same fields as from sidecar XMP. This captures keywords, ratings, and labels from tools like CaptureOne and Lightroom that embed XMP directly in exported files.

Other file formats (RAW, video, audio, etc.) are skipped for embedded XMP extraction – zero I/O overhead.

Processing Order and Precedence

Metadata sources are processed in this order during import:

1. **EXIF** — extracted first. Sets `created_at`, camera/lens info, exposure data, GPS coordinates.
2. **Embedded XMP** (JPEG/TIFF only) — fills any fields still empty after EXIF (insert-if-absent).
3. **Sidecar XMP** — fills any fields still empty after embedded XMP (insert-if-absent).

For rating, description, and color label, the **first source to set a value wins** on initial import. Tags are always merged from all sources (union of all keywords, no removal).

When a sidecar recipe is **updated later** (e.g. after editing in Capture One or Lightroom), the behavior changes: sidecar XMP **overwrites** rating, description, and color label. This ensures that edits made in photo editing software take effect. The `created_at` date is never overwritten — it is always derived from EXIF.

> **Note:** The reverse direction — writing MAKI edits back to `.xmp` files — has two modes. `[write-back] enabled = true` in `maki.toml` flushes every edit immediately. With it off (the default safety net), edits land in the catalog and YAML sidecar and are flagged `pending_writeback` until you run `maki writeback` — that command always works regardless of the flag. See the Configuration Reference for the full split.

Preview Generation

MAKI generates preview thumbnails during import so you can browse assets without accessing the original files. Optionally, high-resolution **smart previews** (2560px) can be generated alongside thumbnails, enabling zoom and pan in the web UI lightbox.

By File Type

File type	Method	Result
Standard images (JPEG, PNG, TIFF, WebP, etc.)	`image` crate	800px JPEG thumbnail
RAW files (NEF, ARW, CR3, DNG, etc.)	`dcraw` or `dcraw_emu` (LibRaw)	800px JPEG thumbnail
Video (MP4, MOV, MKV, etc.)	`ffmpeg` frame extraction	800px JPEG thumbnail
Audio, documents, unknown	Info card renderer	800x600 JPEG with file metadata

The maximum edge size (default 800px) and output format (JPEG or WebP) are configurable in `maki.toml`. Smart previews have separate size and quality settings:

```
[preview]
max_edge = 800          # regular thumbnail max edge (default 800)
format = "jpeg"         # jpeg or webp
quality = 85            # JPEG/WebP quality (1-100)
smart_max_edge = 2560   # smart preview max edge (default 2560)
smart_quality = 92      # smart preview quality (default 92)
```

To generate smart previews during import, use `--smart` or enable permanently:

```
[import]
smart_previews = true
```

Info Cards

When a file has no visual preview (audio files, documents) or when external tools (`dcraw`, `ffmpeg`) are not installed, MAKI generates an **info card** – an 800x600 JPEG displaying the file's metadata:

- Filename and format
- File size
- For audio: duration, bitrate, sample rate (extracted via the `lofty` crate)

Storage and Failure

Previews are stored under the catalog's `previews/` and `smart-previews/` directories:

```
previews/
  ab/
    ab3f7c9e2d...1a4b.jpg
  f2/
    f29a4bc81e...7d3c.jpg
smart-previews/
  ab/
    ab3f7c9e2d...1a4b.jpg
  f2/
    f29a4bc81e...7d3c.jpg
```

The first two characters of the content hash serve as a subdirectory prefix to avoid having too many files in a single directory. Smart previews use the same naming scheme in a separate directory.

Preview generation never blocks import. If `dcraw` is missing, a RAW file still imports successfully – it just gets an info card instead of a rendered preview. You can regenerate previews later with `maki generate-previews`.

Import Options

File Type Groups

MAKI organizes recognized file extensions into groups. Some are enabled by default; others require opting in.

Group	Extensions	Default
`images`	jpg, jpeg, png, gif, bmp, tiff, tif, webp, heic, heif, svg, ico, psd, xcf, raw, cr2, cr3, crw, nef, nrw, arw, sr2, srf, orf, rw2, dng, raf, pef, srw, mrw, 3fr, fff, iiq, erf, kdc, dcr, mef, mos, rwl, bay, x3f	On
`video`	mp4, mov, avi, mkv, wmv, flv, webm, m4v, mpg, mpeg, 3gp, mts, m2ts	On
`audio`	mp3, wav, flac, aac, ogg, wma, m4a, aiff, alac	On
`xmp`	xmp	On
`documents`	pdf, doc, docx, xls, xlsx, ppt, pptx, txt, md, rtf, csv, json, xml, html, htm	Off
`captureone`	cos, cot, cop	Off
`rawtherapee`	pp3	Off
`dxo`	dop	Off
`on1`	on1	Off

Enable additional groups with `--include`, disable default groups with `--skip`:

```
# Import photos plus CaptureOne sidecars, but skip audio files
maki import --include captureone --skip audio /Volumes/PhotosDrive/Shoot/
```

Exclude Patterns

Configure glob patterns in `maki.toml` to permanently exclude files by name:

```toml
[import]
exclude = ["Thumbs.db", "*.tmp", ".DS_Store"]
```

Auto-Tags

Apply tags automatically to every newly imported asset:

```toml
[import]
auto_tags = ["inbox", "unreviewed"]
```

Auto-tags are merged with any tags extracted from XMP metadata (deduplicated).

All Flags

Flag	Effect
`--volume <label>`	Use a specific volume instead of auto-detecting from the file path
`--profile <name>`	Use a named import profile from `maki.toml` (see Configuration)
`--dry-run`	Show what would happen without writing to catalog, sidecar, or disk
`--auto-group`	After import, group new assets with nearby catalog assets by filename stem
`--include <group>`	Enable an additional file type group (repeatable)
`--skip <group>`	Disable a default file type group (repeatable)
`--smart`	Generate smart previews (2560px) for zoom/pan in the web UI
`--embed`	Generate image embeddings for visual similarity search *(Pro)*
`--json`	Structured JSON output to stdout
`--log`	Per-file progress to stderr (`filename -- status (duration)`)
`--time`	Show total elapsed time

Duplicate Handling

MAKI uses content hashing (SHA-256) to detect duplicates at the file level.

Three Outcomes

When importing a file, one of three things happens:

1. **New content hash** – The file is new to the catalog. A new Variant is created (and potentially a new Asset).
2. **Known hash, new location** – The file's content already exists in the catalog, but this is a new path (perhaps on a different volume, or the file was copied to a new directory). The new location is **added** to the existing Variant. The asset is not duplicated.
3. **Same hash and same location** – Both the content and the path are already recorded. The file is truly skipped.

Example: Backup Drive

```
# Import from primary drive
maki import /Volumes/Photos/Shoot-2026-02-20/

# Later, import the same files from a backup
maki import /Volumes/Backup/Shoot-2026-02-20/
```

The second import adds backup locations to the existing variants. Each variant now has two file locations (one per volume), but the catalog contains only one copy of each asset.

Finding Duplicates

After importing from multiple volumes, the same file will appear at multiple locations – this is expected and healthy. Use `maki duplicates --cross-volume` to confirm that your backup copies were recognized:

```
maki duplicates --cross-volume
```

```
abc3f7c9...  DSC_0042.nef
  Photos[working] → Shoot-2026-02-20/DSC_0042.nef
  Backup[backup]  → Shoot-2026-02-20/DSC_0042.nef
```

If you suspect accidental duplicates *within* a single volume (e.g., files imported twice from different paths), use `--same-volume` instead:

```
maki duplicates --same-volume
```

See Finding Duplicates for the full workflow and Maintenance for cleaning up unwanted copies.

Watching for New Files

`maki import` is a one-shot operation. For a tethered session, a card that fills up during the day, or a hot folder that other tools drop files into, `maki watch` keeps importing without you:

```
maki watch                                  # all online volumes, poll every 30 s
maki watch --interval 10 /Volumes/Photos/2026-06-11-shoot
maki watch --volume Archive --log
```

Watch is **poll-based**: the watched roots are re-scanned every N seconds (`--interval`, or `[watch] poll_seconds`, default 30), which behaves identically on macOS, Linux, and Windows. A new file is imported only once its size and modification time are unchanged on two consecutive scans — a file still being copied never gets half-imported. Everything a normal import does applies (`[import]` config: `auto_tags`, `smart_previews`, `embeddings`, `descriptions`, `exclude`), and `[watch] exclude` adds patterns on top.

Two things watch deliberately does *not* do: it never imports what was already on disk when it started (run `maki import` once for the backlog — the first scan only establishes a baseline), and it never removes anything (deletions are logged with `--verbose` only; catalog cleanup stays an explicit `maki cleanup`). Changed `.xmp` sidecars that already belong to the catalog are refreshed in place, so ratings and tags edited in Capture One or Lightroom flow in as well.

`maki watch --once` runs a single cycle and exits — the cron-friendly form. Unlike the continuous loop it diffs against the catalog rather than a start-time baseline, so untracked stable files are imported. See the watch reference for the full option list.

Audio Files

Audio imports like everything else — content-hashed, sidecar-backed, deduplicated — and since v4.9.3 it is indexed rather than merely stored. Import reads duration, bitrate, sample rate, channels, and embedded title / artist / album tags into the variant's source metadata; the preview is an info card with a waveform strip rendered by ffmpeg (a plain info card when ffmpeg is missing). `duration:` works on audio just as on video, and the web asset page gets an inline player plus technical badges.

Musical key and tempo are **not** detected at import, because those analyzers only make sense on full mixes. Run them explicitly on the material where the answer means something:

```
maki audio analyze "path:BackingTracks" --log
maki search "key:am bpm:100-140"
```

`maki audio analyze` shells out to `[audio] key_command` (default `keyfinder-cli`) and `[audio] bpm_command` (default `beat_this`); a missing tool warns and skips its half. MAKI never processes audio itself — it records

what the tools report. See the audio analyze reference.

AI Auto-Tagging *(Pro)*

After importing, you can use AI to automatically suggest tags for your images:

```
# First time: download the default SigLIP model (~207 MB)
maki auto-tag --download

# Preview suggested tags (report-only)
maki auto-tag "type:image"

# Apply suggested tags
maki auto-tag "type:image" --apply
```

The command uses SigLIP vision-language models for zero-shot classification against ~100 built-in photography categories (landscape, portrait, architecture, animals, etc.). Tags above the confidence threshold (default 0.1) are suggested. Use `--threshold` to adjust sensitivity.

Two models are available: the default ViT-B/16-256 (~207 MB) and the larger ViT-L/16-256 (~670 MB) for higher accuracy. Switch with `--model` or set `[ai] model` in `maki.toml`:

```
# Download and use the larger model
maki auto-tag --download --model siglip-vit-l16-256
maki auto-tag "type:image" --model siglip-vit-l16-256 --apply
```

You can provide a custom label vocabulary:

```
maki auto-tag --labels my-labels.txt "" --apply
```

Image embeddings are stored per model in the catalog, enabling visual similarity search:

```
maki auto-tag --similar <asset-id>
```

Embedding Generation During Import

To generate embeddings automatically during import (so "Find similar" works immediately), use the `--embed` flag:

```
maki import --embed /Volumes/PhotosDrive/Photos/2026-02-20/
```

Or enable it permanently in `maki.toml`:

```
[import]
embeddings = true
```

This runs embedding generation as a post-import phase using the preview image for each imported asset. If the AI model is not downloaded, the embedding phase is silently skipped. You can also batch-generate embeddings for existing assets with `maki embed ""`.

See the auto-tag reference for all options and the configuration reference for `[ai]` settings in `maki.toml`.

VLM Image Descriptions *(Pro)*

While SigLIP auto-tagging classifies images against a fixed vocabulary (~100 labels), vision-language models (VLMs) generate free-form text descriptions that capture scene context, spatial relationships, lighting, and mood. The `maki describe` command sends preview images to a VLM server and stores the generated text as the asset's description.

Setting Up a Local VLM Server

The recommended approach is Ollama, a one-command install that manages model downloads, quantization, and GPU memory automatically.

1. Install Ollama:

```
# macOS (also available as .dmg from ollama.com)
brew install ollama

# Linux
curl -fsSL https://ollama.com/install.sh | sh

# Windows (also available as installer from ollama.com)
winget install Ollama.Ollama
```

2. Start the server:

```
ollama serve
```

Ollama runs in the background on `http://localhost:11434`. On macOS and Windows, the Ollama desktop app starts the server automatically.

3. Pull a vision model:

```
# Recommended: good balance of quality, speed, and memory
ollama pull qwen2.5vl:3b

# Faster, lighter alternative for batch processing
ollama pull moondream

# Higher quality for important assets (needs ~6 GB RAM)
ollama pull qwen2.5vl:7b
```

4. Verify the setup:

```
maki describe --check
# Connected to http://localhost:11434. 2 model(s) available: qwen2.5vl:3b, moondream:latest
```

Model Recommendations

Model	Size	RAM	Speed (M3 Pro)	Best For
Moondream 2B `moondream`	1.7 GB	~2 GB	~3–5s	Large batch processing
Qwen2.5-VL 3B `qwen2.5vl:3b`	2.0 GB	~3 GB	~8–12s	Daily use (default)
Gemma 3 4B `gemma3:4b`	3.3 GB	~4 GB	~10–15s	Strong reasoning
Qwen2.5-VL 7B `qwen2.5vl:7b`	4.7 GB	~6 GB	~20–36s	Best quality, key assets

These are the well-tested defaults. The current model generations — **Qwen3.5** (`qwen3.5:4b` as the everyday upgrade from the default, `qwen3.5:9b` for a quality jump) and **Gemma 4** (`gemma4:e4b` as the upgrade from

Gemma 3 4B) — are covered with size and speed comparisons in the VLM Model Guide.

All models use GPU automatically on macOS (Metal), Linux (CUDA), and Windows (CUDA/DirectML) when run through Ollama.

Alternative Local Servers

Any server implementing the OpenAI-compatible `/v1/chat/completions` API works:

- **LM Studio** – GUI-based model manager with built-in server. Default endpoint: `http://localhost:1234`
- **llama.cpp server** – Lightweight C++ inference. Default endpoint: `http://localhost:8080`
- **vLLM** – High-throughput production server. Good for processing large catalogs on a dedicated GPU machine.

```
# Example: LM Studio
[vlm]
endpoint = "http://localhost:1234"
model = "qwen2.5vl-3b"
```

Using Cloud APIs

Cloud vision APIs that implement the OpenAI chat completions format can also be used. Note that these charge per request.

OpenAI:

```
[vlm]
endpoint = "https://api.openai.com"
model = "gpt-6-astra"
```

Requires `OPENAI_API_KEY` in your environment. Charged per image; see OpenAI's current pricing.

Groq (free tier available):

```
[vlm]
endpoint = "https://api.groq.com/openai"
model = "<current Groq vision model>"   # check console.groq.com/docs/models
```

Requires `GROQ_API_KEY`. Free tier allows ~30 requests/minute.

Together AI:

```
[vlm]
endpoint = "https://api.together.xyz"
model = "<current Together vision model>"   # check docs.together.ai
```

Requires `TOGETHER_API_KEY`.

Cloud model rosters rotate quickly (OpenAI's `gpt-4o`, its GPT-5.x line, and the Llama 3.2 vision previews once listed here have all been retired) – always take the exact identifier from the provider's current model list.

> **Note on authentication:** maki currently sends requests without authentication headers. For cloud APIs that require bearer tokens, you may need a local proxy that adds the header, or use a server like Ollama as a gateway. Native API key support may be added in a future version.

Generating Descriptions

```
# Preview descriptions without saving (report-only)
maki describe "description:none type:image" --log
```

```
# Apply descriptions to undescribed assets on a volume
maki describe --volume "Photos 2024" --apply --log

# Describe a single asset
maki describe --asset a1b2c3d4 --apply

# Overwrite existing descriptions with a better model
maki describe "rating:5" --model qwen2.5vl:7b --force --apply

# Use a custom prompt for specific content
maki describe --prompt "List the key subjects and artistic techniques." "tag:art" --apply

# Generate tags instead of descriptions
maki describe --mode tags "tag:untagged" --apply

# Generate both descriptions and tags (two VLM calls per asset)
maki describe --mode both --asset a1b2c3d4 --apply

# Dry run to see what would be processed
maki describe "date:2024-06" --dry-run
```

Three modes are available: `--mode describe` (default) generates descriptions, `--mode tags` generates tag suggestions, and `--mode both` runs both — making two separate VLM calls per asset so each uses its optimal prompt. Tags are deduplicated and merged with existing asset tags.

The report-only default (no `--apply`) lets you review generated descriptions and tags before committing them. This is especially useful when tuning prompts or trying different models.

Configuration

Persistent settings go in `maki.toml`:

```
[vlm]
endpoint = "http://localhost:11434"
model = "qwen2.5vl:3b"
max_tokens = 500
timeout = 300
temperature = 0.7
concurrency = 1  # parallel VLM requests (increase for faster batch processing)
```

To offer multiple models in the web UI (e.g., a fast model for routine work and a larger model for difficult images), add a `models` list:

```
[vlm]
model = "moondream"
models = ["moondream", "qwen3-vl:4b"]
```

A dropdown appears next to the Describe button when two or more models are configured. CLI flags (`--endpoint`, `--model`, `--prompt`, `--max-tokens`, `--timeout`, `--temperature`, `--mode`) override `maki.toml` for one-off runs.

SigLIP vs VLM: When to Use Which

	SigLIP Auto-Tag	VLM Describe
Output	Tags from fixed vocabulary	Free-form text, tags, or both
Speed	~50–150 ms/image	~3–36s/image
Best for	Categorical filtering, similarity	Documentation, open-ended tagging
Requires	*(Pro)*, ONNX models	*(Pro)*, running VLM server
GPU	Included on macOS (CoreML)	Automatic via Ollama

The two approaches are complementary. Use SigLIP for fast categorical tagging across large batches, and VLM for rich descriptions when quality matters more than speed.

See the describe reference for all options and the configuration reference for `[vlm]` settings.

Working with Video

MAKI handles video files alongside images — they flow through the same import pipeline, appear in the same browse grid, and support the same metadata operations (tags, ratings, labels, descriptions, collections, stacks).

What MAKI extracts from video

When `ffprobe` is available (installed with ffmpeg), MAKI extracts:

- **Duration** in seconds
- **Codec** (e.g., h264, hevc, prores)
- **Resolution** (width and height in pixels)
- **Frame rate**

This metadata is stored in the catalog and available as search filters.

Video previews

During import, MAKI uses `ffmpeg` to extract a representative frame as the preview thumbnail. If ffmpeg is not installed, the video gets an info card instead. You can install ffmpeg later and regenerate previews:

```
maki generate-previews --include video --force --log
```

In the web UI, video assets show a play icon overlay. Hovering plays a short proxy clip (generated automatically when ffmpeg is available).

Searching for video

Use the video-specific search filters to work with footage:

```
# All video assets
maki search "type:video"

# Short clips under 30 seconds
maki search "type:video duration:1-30"

# Long-form footage over 5 minutes
```

```
maki search "type:video duration:300+"

# H.264 footage
maki search "type:video codec:h264"

# HEVC/H.265 footage
maki search "type:video codec:hevc"
```

Mixed photo and video shoots

Modern cameras produce both photos and videos in the same shoot. MAKI handles this naturally — `--auto-group` groups by filename stem regardless of file type, and you can filter by type when needed:

```
# Import a mixed shoot from your working drive
maki import /Volumes/FastSSD/Capture/2026-03-28/ --auto-group --log

# Browse only the photos
maki search "path:Capture/2026-03-28 type:image"

# Browse only the videos
maki search "path:Capture/2026-03-28 type:video"

# Rate and tag both together in the web UI
maki serve
```

Common Import Scenarios

The way you import differs depending on the source. Here are patterns for common situations.

Card reader / memory card

Traditional approach — copy first, then import:

```
# Copy card to working SSD (using rsync, Finder, or your preferred tool)
rsync -av /Volumes/CARD/DCIM/ /Volumes/FastSSD/Capture/Johnson-Wedding/

# Import from the fast local copy
maki import /Volumes/FastSSD/Capture/Johnson-Wedding/ \
  --add-tag "shoot:johnson-wedding" \
  --auto-group --smart --log
```

Card-first approach — import from the card, cull on previews, copy only keepers:

```
# Register the card (label auto-derived from mount path)
maki volume add /Volumes/EOS_DIGITAL --purpose media

# Import from card — generates previews but doesn't copy files
maki import --profile card /Volumes/EOS_DIGITAL/DCIM \
  --add-tag "shoot:johnson-wedding" --auto-group --log

# Eject card, cull in web UI on smart previews, rate keepers
maki serve
```

```
# Copy only rated images to working drive (with XMP sidecars for CaptureOne)
maki relocate --query "volume:EOS_DIGITAL rating:1+" \
  --target "Work SSD" --create-sidecars --log

# Clean up the card volume
maki volume remove "EOS_DIGITAL" --apply
```

The card-first approach avoids copying rejects to your SSD entirely. On a 1000-shot day where you keep 300, that's 70% less data to copy. See The Archive Lifecycle for the full storage strategy.

The `--add-tag` flag stamps every imported asset with a session tag for easy retrieval later. `--auto-group` pairs RAW+JPEG files automatically.

Tethered shooting

When tethering to a capture folder, MAKI handles incremental imports naturally — files already in the catalog are skipped by content hash:

```
# First import
maki import /Volumes/Work/Capture/2026-03-28/ --auto-group --log

# Later, after more shots are captured
maki import /Volumes/Work/Capture/2026-03-28/ --auto-group --log
# Only new files are imported; existing ones are skipped
```

Migrating from another DAM

When importing a folder structure managed by CaptureOne, Lightroom, or another tool, include the tool's recipe files to preserve your processing work:

```
# Import with CaptureOne sidecars
maki import /Volumes/Media/CaptureOne --include captureone --auto-group --log

# Import with RawTherapee sidecars
maki import /Volumes/Media/RawTherapee --include rawtherapee --auto-group --log
```

After importing, run `maki refresh` to read metadata from the XMP sidecar files (ratings, keywords, labels, descriptions):

```
maki refresh --log
```

Cloud-synced folders

Register cloud-synced directories (Dropbox, iCloud, Google Drive) as volumes with the `cloud` purpose:

```
maki volume add "Dropbox" ~/Dropbox/Photos --purpose cloud
maki import ~/Dropbox/Photos/ --log
```

Be aware that partially-synced files (placeholders) may fail to import. Import again after sync completes to pick up the remaining files.

Importing selectively

Skip file types you don't need, or include types that are off by default:

```
# Photos only — skip video and audio
maki import /Volumes/FastSSD/Capture/2026-03/ --skip video --skip audio --log
```

```
# Include documents alongside media
maki import /Volumes/FastSSD/Projects/ --include documents --log
```

Next: Organizing Assets – tags, ratings, labels, grouping, collections, and saved searches.

Organizing Assets

Once your files are imported, MAKI provides several tools for organizing them: tags, metadata editing, variant grouping, static collections, and saved searches. All changes are persisted in both the SQLite catalog and YAML sidecar files. When `.xmp` recipe files are present, edits to ratings, tags, descriptions, and color labels are written back to disk for bidirectional sync with tools like CaptureOne and Lightroom.

Tags

Tags are free-form keywords attached to an asset. They are deduplicated automatically – tags from XMP sidecar files, embedded XMP in JPEGs, and manual additions are merged into a single set.

Hierarchical tags

Tags can contain `|` as a hierarchy separator (aligned with Lightroom and CaptureOne), allowing you to organize keywords into a tree structure:

```
maki tag a1b2c3d4 "animals|birds|eagles"
maki tag a1b2c3d4 "location|europe|germany"
```

Adding a hierarchical tag automatically stores all ancestor paths – tagging with `animals|birds|eagles` also adds `animals` and `animals|birds`. Searching for `tag:animals` matches because the asset has `animals` as a tag. This works in both CLI and web UI searches.

Hierarchical tags interoperate with Lightroom's `lr:hierarchicalSubject` XMP field. When importing XMP files that contain `lr:hierarchicalSubject` entries, MAKI reads the hierarchy and stores the full path as a tag. Write-back preserves the hierarchy in the XMP file.

The tags page in the web UI displays hierarchical tags as a collapsible tree (see Web UI for details).

Adding tags

Pass one or more tag names after the asset ID:

```
maki tag a1b2c3d4 landscape nature golden-hour
```

This adds three tags to the asset. If a tag already exists on the asset, it is silently skipped.

Removing tags

Use the `--remove` flag:

```
maki tag a1b2c3d4 --remove unwanted blurry
```

Tags that are not present on the asset are silently ignored.

XMP write-back

When an asset has `.xmp` recipe files on disk, tag changes are written back immediately. Added tags are inserted into the `dc:subject` / `rdf:Bag` block; removed tags are deleted from it. Tags that were added independently in CaptureOne or Lightroom (not managed by maki) are preserved – maki operates on deltas, not full replacement.

If the volume containing the `.xmp` file is offline, the write-back is skipped with a warning.

Renaming tags

To rename a tag across all assets that have it — for example, reorganizing a flat tag into a hierarchy, normalizing casing, or fixing a typo:

```
# Preview what would change
maki tag rename "Munich" "location|Germany|Bavaria|Munich"

# Apply the rename
maki tag rename "Munich" "location|Germany|Bavaria|Munich" --apply --log
```

Matching is case-insensitive: `maki tag rename "Concert" "concert"` finds "Concert", "CONCERT", and "concert" and normalizes them all.

When renaming to a hierarchical tag, all ancestor paths are automatically added. For example, renaming "Munich" to "location|Germany|Bavaria|Munich" also adds `location`, `location|Germany`, and `location|Germany|Bavaria`. Removing a hierarchical tag also cleans up orphaned ancestor tags (ancestors no longer needed by any other descendant).

Splitting tags

Replace one tag with two or more across the whole catalog. Useful when a single tag was carrying two orthogonal meanings — e.g. the legacy `subject|event|wedding-jane-2025` actually meant both "wedding scene" and "Jane's 2025 wedding occasion":

```
# Replace the legacy tag with two facet-aligned ones
maki tag split "subject|event|wedding-jane-2025" \
    "subject|event|wedding" \
    "event|wedding-jane-2025" \
    --apply
```

Splits act on **leaf occurrences only**. Pass `--keep` to add the new tags alongside the old (additive copy) instead of replacing.

Deleting tags

Remove a tag — and, by default, its entire descendant branch — from every asset that has it:

```
# Drop a typo'd tag everywhere
maki tag delete "lansdcape" --apply

# Remove an entire event branch
maki tag delete "event|wedding-jane-2025" --apply

# Leaf-only: delete just the parent value where it's not specialised
maki tag delete "=subject|nature" --apply
```

Same `=` (leaf-only), `^` (case-sensitive) markers as `tag rename`. Newly-orphaned ancestors on each asset are cleaned up automatically. The web UI's tags page has an equivalent trash button on every row with a

Preview/Apply confirmation modal.

Browsing tags in the web UI

The web UI provides a dedicated tags page at `/tags` with sortable columns, live text filter, and a hierarchical tree view. Each tag row carries inline Rename / Split / Delete buttons (Preview→Apply, Enter-twice to commit). The headline count is "assets carrying this tag in any form" (matches `tag:foo`); a parenthesised "(N as leaf)" hint surfaces parent rows with sloppily-tagged assets — it's clickable and lands on the browse page filtered to exactly those candidates for finer tagging. See Web UI → Tags Page for the full tour.

Editing Metadata

The `maki edit` command sets or clears first-class metadata fields on an asset. At least one flag is required.

Setting fields

```
maki edit a1b2c3d4 --name "Sunset at Beach"
maki edit a1b2c3d4 --description "A beautiful sunset over the Pacific coast"
maki edit a1b2c3d4 --rating 5
maki edit a1b2c3d4 --label Red
maki edit a1b2c3d4 --date "2024-12-25"
```

Multiple flags can be combined in a single invocation:

```
maki edit a1b2c3d4 --name "Sunset at Beach" --rating 5 --label Orange
```

Clearing fields

Each field has a corresponding `--clear-*` flag:

```
maki edit a1b2c3d4 --clear-name
maki edit a1b2c3d4 --clear-description
maki edit a1b2c3d4 --clear-rating
maki edit a1b2c3d4 --clear-label
maki edit a1b2c3d4 --clear-date
```

An empty description string is normalized to a clear operation – these two commands are equivalent:

```
maki edit a1b2c3d4 --description ""
maki edit a1b2c3d4 --clear-description
```

Color labels

The label field accepts one of seven colors: **Red**, **Orange**, **Yellow**, **Green**, **Blue**, **Pink**, **Purple**. Input is case-insensitive and stored in canonical title-case:

```
maki edit a1b2c3d4 --label red      # stored as "Red"
maki edit a1b2c3d4 --label YELLOW   # stored as "Yellow"
```

This is a superset of Lightroom's 5-color palette, matching CaptureOne's 7 colors.

XMP write-back for metadata

Changes to **rating**, **description**, and **color label** are automatically written back to any `.xmp` recipe files attached to the asset. This enables round-trip editing with external tools:

1. Rate an image in maki (`maki edit ... --rating 4`).

2. Open CaptureOne – the rating is already reflected in the XMP sidecar.
3. Change the rating in CaptureOne.
4. Run `maki refresh` to pick up the change.

Structured output

Use the global `--json` flag for machine-readable output:

```
maki --json edit a1b2c3d4 --rating 5
```

Manual Grouping

The `maki group` command merges multiple variants (identified by their content hashes) into a single asset. This is useful when related files were imported separately and ended up as distinct assets.

Basic usage

```
maki group abc123def456 789fed321cba
```

You can pass any number of content hashes. maki identifies the assets that own those variants and merges them.

Merge rules

- The **oldest asset** (by `created_at`) is chosen as the target.
- Donor variants that had the **Original** role are re-roled to **Alternate** (in both the YAML sidecar and the SQLite catalog) to avoid multiple originals on one asset.
- **Tags** from donor assets are merged into the target (union, deduplicated).
- **Recipes** from donor assets are moved to the target.
- Donor assets are **deleted** after merging.

Finding content hashes

Use `maki show` to display an asset's variants and their content hashes:

```
maki show a1b2c3d4
```

Or use `maki search --format '{id}\t{name}'` to locate the assets first, then inspect them with `maki show`.

Auto-Grouping

The `maki auto-group` command finds assets that belong together by matching their filename stems across the catalog. This handles the common case where a RAW original and its processed exports end up as separate assets – for example, when CaptureOne exports land in a different directory from the originals.

Report mode (dry run)

Without `--apply`, auto-group runs in report-only mode. Nothing is modified:

```
maki auto-group
```

Output shows which groups would be formed, how many donors would be merged, and how many variants would be moved.

Applying changes

```
maki auto-group --apply
```

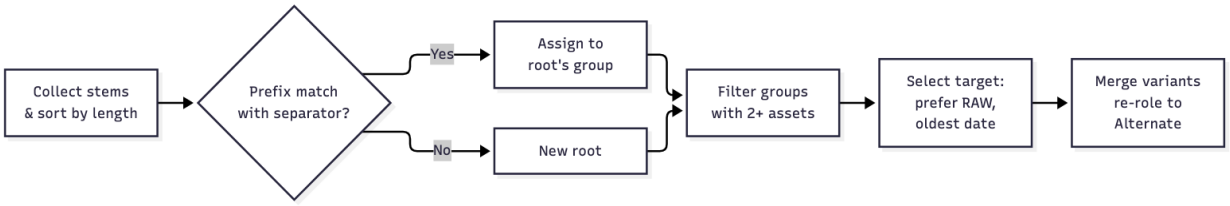
Scoping to a search query

Pass a search query to limit the scope:

```
maki auto-group "tag:landscape" --apply
maki auto-group "path:Capture/2024" --apply
```

The matching algorithm

Auto-group uses fuzzy prefix matching with separator-boundary detection:



Steps in detail:

1. Collect the filename stem (without extension) of each asset and sort by length (shortest first).
2. For each stem, check if any existing shorter stem is a prefix match with a separator boundary — the character after the prefix must be non-alphanumeric (`-`, `_`, space, `(`). If so, assign to that group. Otherwise, register as a new root.
3. Filter to groups with 2+ assets. Select a target per group (prefer assets with a RAW variant, then oldest by `created_at`). Merge donor variants into the target and re-role Original to Alternate.

Matching examples:

Shorter stem	Longer stem	Next char	Result
`Z91_8561`	`Z91_8561-1-HighRes`	`-` (hyphen)	Match
`Z91_8561`	`Z91_8561_edit`	`_` (underscore)	Match
`DSC_001`	`DSC_0010`	`0` (digit)	No match
`IMG_2000`	`IMG_2000 (1)`	(space)	Match

Chain resolution: When multiple stems form a chain, they all resolve to the shortest root. For example, `Z91_8561`, `Z91_8561-1`, and `Z91_8561-1-HighRes` all group under `Z91_8561`.

Target selection: Within each group, the target asset is the one with a RAW variant (if any), breaking ties by oldest `created_at` timestamp. Donor variants with the Original role are re-roled to Alternate, just like manual grouping.

Output options

```
maki auto-group --json            # structured JSON result
maki auto-group --log --apply     # per-group progress on stderr
```

Web UI

In the web UI, select multiple assets on the browse page, then click the **"Group by name"** button in the batch toolbar. A confirmation dialog shows how many groups would be formed before applying.

Splitting Assets

The inverse of grouping: extract one or more variants from an asset into new standalone assets.

```
# View variants of an asset
maki show abc12345

# Extract the JPEG variant into its own asset
maki split abc12345 sha256:def456...
```

Each extracted variant becomes a new asset with: - Role set to `original` - Tags, rating, color label, and description copied from the source - Associated recipes moved with the variant

In the web UI, the asset detail page shows checkboxes next to each variant. Select the variants to extract and click "Extract as new asset(s)".

Constraints: At least one variant must remain in the source asset.

Cleaning up after a mis-group

When assets are accidentally grouped, their tags get merged. After splitting them apart, the new assets inherit the merged tags. To clean up:

```
# Split the mis-grouped variant out
maki split abc12345 --variant def456...

# Clear the inherited merged tags from the new asset
maki edit <new-asset-id> --clear-tags

# Re-read the original tags from the XMP sidecar
maki refresh --asset <new-asset-id>
```

Collections (Static Albums)

Collections are manually curated lists of assets – the equivalent of albums in a traditional photo manager. They are backed by both SQLite (for fast queries) and a `collections.yaml` file at the catalog root (which survives `rebuild-catalog`).

The alias `maki col` can be used in place of `maki collection` for brevity.

Creating a collection

```
maki collection create "Best of 2024" --description "Top picks from the year"
```

Listing collections

```
maki collection list
```

Viewing collection contents

```
maki collection show "Best of 2024"
maki collection show "Best of 2024" --format full
```

Adding assets

```
maki collection add "Best of 2024" a1b2c3d4 e5f6a7b8
```

You can pipe asset IDs from a search:

```
maki search -q "rating:5 tag:landscape" | xargs maki col add "Best of 2024"
```

This is a powerful pattern – combine any search filter to build a collection in a single command.

Removing assets

```
maki collection remove "Best of 2024" a1b2c3d4
```

Deleting a collection

```
maki collection delete "Best of 2024"
```

This removes the collection definition and all membership records. The assets themselves are not affected.

Searching within a collection

Use the `collection:` filter in any search:

```
maki search "collection:\"Best of 2024\" rating:4+"
```

Note the quoted value for collection names with spaces.

Web UI

- The **`/collections`** page lists all collections with a **"+ New Collection"** button.
- On the **asset detail page**, collection membership is shown as chips with a remove button.
- In the **batch toolbar**, a collection dropdown lets you add or remove selected assets from a collection. The dropdown includes a **"New..."** option that creates a collection inline.

Stacks

Stacks are lightweight anonymous groups of assets – burst shots, exposure brackets, or similar scenes that you want to collapse into a single entry in the browse grid. Unlike grouping (which permanently merges variants into one asset), stacks keep each asset independent. You can dissolve a stack at any time to separate the assets again.

Each asset can belong to at most one stack. Members are position-ordered; position 0 is the **pick** – the representative image shown when the stack is collapsed in the browse grid. Stacks auto-dissolve when they have one or fewer members.

The alias `maki st` can be used in place of `maki stack` for brevity.

Creating a stack

Pass two or more asset IDs to create a new stack:

```
maki stack create a1b2c3d4 e5f6a7b8 c9d0e1f2
```

The first asset becomes the pick (position 0). The remaining assets are ordered by their position in the argument list.

Adding assets to a stack

```
maki stack add <stack-id> f3a4b5c6
```

The new asset is appended to the end of the stack.

Setting the pick

Promote any member to the pick position:

```
maki stack pick <stack-id> e5f6a7b8
```

The previous pick moves to position 1; all other positions shift accordingly.

Removing assets from a stack

```
maki stack remove <stack-id> c9d0e1f2
```

If the stack has one or fewer members after removal, it auto-dissolves.

Dissolving a stack

Dissolve a stack entirely, returning all members to standalone assets:

```
maki stack dissolve <stack-id>
```

Listing stacks

```
maki stack list
```

Shows all stacks with their member counts.

Showing stack details

```
maki stack show <stack-id>
```

Displays the stack's members in position order, with the pick marked.

Searching for stacked assets

Use the `stacked:` filter in any search:

```
maki search "stacked:true"      # assets that are in a stack
maki search "stacked:false"     # standalone assets
```

Converting tags to stacks

Some tools (notably CaptureOne) represent auto-stacking as tags rather than a native stacking structure. The `from-tag` subcommand converts these tags into proper MAKI stacks:

```
# Preview: find assets with CaptureOne auto-stack tags
maki stack from-tag "Aperture Stack {}"

# Apply and remove the tags afterwards
maki stack from-tag "Aperture Stack {}" --remove-tags --apply
```

The `{}` wildcard matches a group identifier — assets sharing the same matched value are grouped into a stack. For example, if you have assets tagged `Aperture Stack 1` and `Aperture Stack 2`, this creates two stacks.

This works with any tag pattern, not just CaptureOne:

```
# Convert Lightroom-style stack tags
maki stack from-tag "Stack {}"

# Convert bracketing tags
maki stack from-tag "Bracket {}"
```

Structured output

All stack commands support `--json`:

```
maki --json stack list
maki --json stack show <stack-id>
```

Web UI

See Web UI for the stack experience in the browse grid, batch operations, and asset detail page.

Saved Searches (Smart Albums)

Saved searches are named queries stored in `searches.toml` at the catalog root. Unlike collections, they are dynamic – running a saved search always reflects the current state of the catalog.

The alias `maki ss` can be used in place of `maki saved-search`.

Saving a search

```
maki saved-search save "Landscapes" "tag:landscape rating:3+" --sort date_desc
```

This stores the query string and sort order under the name “Landscapes”.

Listing saved searches

```
maki saved-search list
```

Running a saved search

```
maki saved-search run "Landscapes"
maki saved-search run "Landscapes" --format full
maki saved-search run "Landscapes" --format json
```

Deleting a saved search

```
maki saved-search delete "Landscapes"
```

Favorites

Mark a saved search as a **favorite** to pin it as a quick-access chip on the web UI browse toolbar:

```
maki ss save "Five Stars" "rating:5" --favorite
maki ss save "Unrated"    "rating:0 type:image" --favorite
maki ss save "At Risk"    "copies:1" --favorite
```

Favorite searches appear as clickable chips above the browse grid — one click to filter, another to clear. This is the fastest way to access your most-used views.

Practical examples

Save commonly used filters for quick access:

```
maki ss save "Unrated"     "rating:0"
maki ss save "Five Stars"  "rating:5"
maki ss save "Red Label"   "label:Red"
maki ss save "Recent"      "path:Capture/2024" --sort date_desc
maki ss save "Orphans"     "orphan:true"
maki ss save "Theater"     "tag:\"Fools Theater\""
```

Combine saved searches with other commands:

```
maki ss run "Five Stars" --format ids | xargs maki col add "Portfolio"
```

Web UI

- **Clickable chips** on the browse page load saved searches into the filter UI with a single click.
- The **Save button** captures the current search state (query, filters, sort order) as a new saved search.
- **Hover** over a chip to reveal rename and delete buttons.

People Management (Face Recognition) *(Pro)*

MAKI detects faces in your images, groups them into people, and lets you search by person. Uses YuNet for face detection (outputs 5-point landmarks) and ArcFace ResNet-100 FP32 for face recognition (512-dim embeddings). Faces are aligned to a canonical 112×112 template before embedding — the same preprocessing ArcFace was trained with.

Setup

Download the required models first:

```
maki faces download
```

This downloads the YuNet face detection model (~230 KB) and the ArcFace ResNet-100 FP32 recognition model (~261 MB) to `~/.maki/models/faces/`.

Detecting faces

Run face detection on your catalog:

```
maki faces detect --query "type:image" --apply
```

This finds faces in each image's preview, stores bounding boxes + embeddings + the face model id, and generates a small JPEG thumbnail per face. Use `--log` for per-asset progress.

You can scope detection to a specific shoot, volume, or single asset:

```
maki faces detect --query "path:Capture/2026-03" --apply
maki faces detect --volume "Photos" --apply
maki faces detect --asset a1b2c3d4 --apply
```

Use `--force` to re-detect even on assets that already have faces — useful after upgrading the recognition model or tweaking detection parameters:

```
maki faces detect --force --query 'tag:"person|refugee"' --apply
```

Clustering faces into people

After detection, cluster similar faces into groups:

```
maki faces cluster --apply
```

Uses agglomerative hierarchical clustering (average linkage, UPGMA) — order-independent and produces tighter clusters than simpler greedy approaches. Each resulting group becomes an **unnamed** person ("Unknown (abc12345)" on the people page) until you name it.

Only faces embedded with the **current** recognition model participate; faces from an older model variant are skipped with a warning (re-run `faces detect --force` to update them).

The two main knobs:

- `--threshold <F>` — minimum average cosine similarity for two clusters to merge. Default **0.35**.
- `--min-confidence <F>` — drop low-quality face detections before clustering. Default **0.7**.

```
maki faces cluster --threshold 0.45 --apply     # stricter (tighter clusters)
maki faces cluster --threshold 0.3 --apply      # looser (bigger clusters)
```

Tuning the threshold

The right threshold depends on your data — portrait-heavy collections, group photos, and family albums all have different similarity distributions. MAKI provides two diagnostic commands to help.

`maki faces similarity` — prints percentile stats and a histogram of pairwise cosine similarities for a scope:

```
maki faces similarity --query 'tag:"person|refugee"' --min-confidence 0.7
```

A healthy distribution is **bimodal**: a big hump near zero (different-person pairs) and a smaller hump in the 0.5–0.9 range (same-person pairs), with a valley between. Pick a threshold in that valley.

If the distribution is unimodal or flat (all pairs in a narrow band), the embeddings aren't discriminating well — check detection confidence, face size, and the recognition model. Add `--top 5` to list each face's nearest neighbors (with asset IDs) for spot-checking.

`maki faces dump-aligned` — saves the 112×112 aligned crops to disk so you can visually verify alignment is working:

```
maki faces dump-aligned --query 'tag:"person|refugee"' --limit 30
```

Open `./aligned-faces/` and check: faces should be centered, eyes level, face filling most of the frame. If crops look mirrored, rotated, or squashed, alignment is broken.

Naming and managing people

List discovered people and name them:

```
maki faces people
maki faces name 550e8400-... "Alice"
maki faces name 661f9511-... "Bob"
```

Merge two person groups (e.g., when the same person ended up in two clusters):

```
maki faces merge <target_id> <source_id>
```

Remove a face from a person (misidentification), or delete a person entirely:

```
maki faces unassign <face-uuid>
maki faces delete-person <person-id>      # unassigns the person's faces
```

Then sweep up the unassigned orphans:

```
maki faces clean --apply
```

Searching by face and person

Use the `faces:` and `person:` search filters:

```
maki search "faces:any"                 # assets with detected faces
maki search "faces:none type:image"     # images without faces
maki search "faces:2+"                  # group photos (2+ faces)
maki search "person:Alice"              # assets containing Alice
maki search "person:Alice rating:4+"    # Alice's best photos
```

Web UI

The web UI provides the full face management experience:

- **People page** (`/people`) — gallery of person cards with thumbnails, names, and face counts. Clicking any card (named or "Unknown") navigates to the browse view filtered to that person's assets. Inline rename (pencil icon) and delete (×).
  - **Filter input** above the grid does client-side substring matching against names — useful when you have hundreds of people.
  - **Merge suggestions** panel appears above the grid when MAKI detects pairs of clusters whose centroid embeddings are similar enough to likely be the same person (common after clustering, where splinter clusters often form alongside a main one). Each suggestion shows both sides with a match score, a swap button to flip merge direction, and **Merge** / **Not the same person** actions.
  - **Batch merge** — check the box on any two or more person cards and a toolbar appears. Click the bullseye badge on any selected card to pick the merge target; others become sources. Confirmation modal shows the target + sources with thumbnails before committing.
- **Asset detail page** — faces section with crop thumbnails, confidence, and assignment controls. Already-assigned faces show the person name (or "Unknown (…)" for unnamed clusters) as a clickable link. Unassigned faces get a **searchable combobox** (type to filter, ↑/↓/Enter to pick, or "+ Create new person" always offered at the bottom) — scales cleanly to hundreds of people.
- **Browse filters** — people are chips on the top filter row, next to the tag chips. Layout: `[tags] [people] [path]`. Type to filter; ↑/↓/Enter adds a chip; backspace in the empty input removes the last chip. Multiple chips **AND** (asset must contain all selected people). `faces:` and `person:` filters in the query input work as documented. Chips and filters persist across pagination and sort changes.
- **Batch toolbar** — "Detect faces" processes all selected assets in one go.

Configuration

In `maki.toml`:

```toml
[ai]
face_cluster_threshold = 0.35   # clustering similarity threshold (HAC average-linkage)
face_min_confidence = 0.7       # drop low-quality detections from clustering
```

Upgrading from older face recognition

v4.4.0 replaced the face recognition pipeline wholesale (new model, proper alignment, corrected preprocessing). Existing face embeddings live in a different vector space than new ones and are skipped by clustering. To migrate:

```
maki faces status               # see how many faces are stale
maki faces clean --apply        # delete stale unassigned faces (optional)
maki faces detect --force --query <scope> --apply   # re-embed with the new pipeline
```

For named people you want to keep, `faces detect --force` will preserve the person assignments — it just re-embeds. You'll probably want to re-cluster any auto-clusters afterward.

Deleting Assets

Tagging images as `rest` or leaving them unrated hides them from browsing (see Organizing & Culling), but the files and catalog records remain. When you want to permanently remove assets — confirmed rejects, test shots, accidental imports — use `maki delete`.

When to delete vs. cull

For most images, culling (tagging as `rest` or leaving unrated) is the better choice. Deletion is permanent and cannot be undone. Consider deleting only when:

- The files genuinely have no value (accidental shutter fires, blank frames, duplicates you've already deduplicated)
- You need to reclaim disk space and are certain you have backups
- You imported files by mistake (wrong folder, wrong file types)

Report-only default

Without `--apply`, the command shows what would happen without changing anything:

```
maki delete a1b2c3d4
```

```
Would delete: DSC_0042 (2 variants, 3 locations, 1 recipe)
  Dry run — no changes made. Run with --apply to delete.
```

Catalog-only deletion

With `--apply` but without `--remove-files`, MAKI removes the asset from the catalog and sidecar files but leaves the physical files untouched on disk:

```
maki delete a1b2c3d4 --apply
```

This is useful when you want to stop tracking a file in MAKI but don't want to touch the originals. You can re-import the files later if needed.

Physical file deletion

With `--remove-files`, the physical files are also deleted from all online volumes:

```
maki delete a1b2c3d4 --apply --remove-files
```

Files on offline volumes are not deleted — they remain on disk until you reconnect the volume and run `maki cleanup`.

Batch deletion

Combine search with delete for bulk cleanup:

```
# Preview: which assets would be deleted?
maki search "tag:reject" --format short

# Delete all rejected assets (catalog only)
maki search -q "tag:reject" | maki delete --apply --log

# Delete rejected assets AND their physical files
maki search -q "tag:reject" | maki delete --apply --remove-files --log
```

For the full command reference, see delete.

Putting It All Together

A typical organizing workflow after import might look like this:

1. **Auto-group** to merge related variants:

   ```
   maki auto-group --apply --log
   ```

2. **Tag** a batch of assets from the CLI:

   ```
   maki search -q "path:Capture/2024-08-15" | \
     xargs -I{} maki tag {} vacation beach
   ```

3. **Rate and label** standouts in the web UI using keyboard shortcuts (1-5 for ratings, r/o/y/g/b/p/u for labels).

4. **Build a collection** from your top picks:

   ```
   maki search -q "rating:5 tag:vacation" | xargs maki col add "Summer 2024"
   ```

5. **Save a smart search** for ongoing curation:

   ```
   maki ss save "Vacation Selects" "tag:vacation rating:4+" --sort date_desc
   ```

Next: Browsing & Searching | Previous: Ingesting Assets | Back to Manual

Browsing & Searching

This chapter covers how to find, inspect, and analyze assets in your catalog using the CLI. For the web-based browser interface, see Web UI.

Searching Assets

The `maki search` command is the primary way to find assets. It accepts free-text keywords and structured filters in a single query string.

Free-text search matches against asset names, filenames, descriptions, and source metadata:

```
maki search "sunset"
maki search "beach vacation"
```

Structured filters use a `key:value` syntax and can be combined freely with free-text:

```
maki search "tag:landscape type:image rating:3+"
maki search "sunset tag:landscape camera:fuji iso:100-800"
maki search "format:nef focal:35-70 f:1.4-2.8"
```

Tokens that don't match a known filter prefix are treated as free-text, so you can mix them in any order:

```
maki search "type:image sunset rating:4+ golden hour"
```

Values with spaces are supported using double quotes inside the query:

```
maki search 'tag:"Fools Theater"'
maki search 'camera:"Canon EOS R5" lens:"RF 50mm f/1.2"'
maki search 'collection:"My Favorites"'
maki search 'path:"Photos/Family Trip"'
```

Note the outer single quotes to prevent your shell from stripping the inner double quotes.

Negation excludes matching assets by prefixing any filter or free-text term with `-`:

```
maki search "-tag:rejected"              # exclude rejected assets
maki search "landscape -tag:processed"   # landscapes not yet processed
maki search "-format:xmp -type:other"    # exclude XMP files and "other" types
maki search "-sunset"                    # exclude free-text match on "sunset"
```

Negation is available for free text and for `tag:`, `type:`, `format:`, `label:`, `volume:`, `camera:`, `lens:`, `description:`, `path:`, `collection:` and `person:`; on other filters the leading `-` is ignored.

OR within a filter matches any of several values using commas:

```
maki search "tag:alice,bob"              # tagged alice OR bob (or both)
maki search "format:nef,cr3"             # NEF or CR3 format
maki search "type:image,video"           # images or videos
```

```
maki search "label:Red,Orange"           # Red or Orange labeled
```

Repeated `tag:` filters = AND. To require multiple tags, repeat the filter:

```
maki search "tag:landscape tag:sunset"    # BOTH landscape AND sunset tags
```

This AND-on-repeat rule holds for `tag:` and `meta:` only. Repeating any other filter ORs the values, exactly like a comma list (`type:image type:video` = `type:image,video`).

Search Filters Quick Reference

All filters can be combined in a single query. Remaining tokens become free-text search terms.

Filter	Syntax	Example
Asset type	`type:<type>`	`type:image`, `type:video`
Tag	`tag:<name>`	`tag:landscape`, `tag:"Fools Theater"`
Format	`format:<ext>`	`format:nef`, `format:jpg`
Rating (exact)	`rating:<N>`	`rating:5`
Rating (minimum)	`rating:<N>+`	`rating:3+`
Rating (range)	`rating:<N>-<M>`	`rating:3-5`
Rating (OR)	`rating:<N>,<M>`	`rating:2,4`, `rating:2,4+`
Color label	`label:<color>`, `label:none`	`label:Red`, `label:none`
Camera	`camera:<text>`	`camera:fuji`, `camera:"Canon EOS R5"`
Lens	`lens:<text>`	`lens:56mm`, `lens:"RF 50mm f/1.2"`
Description	`description:<text>`, `desc:<text>`	`description:sunset`
ISO (exact or range)	`iso:<N>` or `iso:<min>-<max>`	`iso:100`, `iso:100-800`
Focal length	`focal:<N>` or `focal:<min>-<max>`	`focal:50`, `focal:35-70`
Aperture	`f:<N>` or `f:<min>-<max>`	`f:2.8`, `f:1.4-2.8`
Width	`width:<N>`, `width:<N>+`, `width:<min>-<max>`	`width:4000+`
Height	`height:<N>`, `height:<N>+`	`height:2160`
Source metadata	`meta:<key>=<value>`	`meta:software=CaptureOne`
Path prefix	`path:<prefix>`	`path:Capture/2026-02-22`
Collection	`collection:<name>`	`collection:Favorites`
Date (prefix match)	`date:<prefix>`	`date:2026-02-25`, `date:2026-02`, `date:2026`
Date (from)	`dateFrom:<date>`	`dateFrom:2026-01-01`
Date (until)	`dateUntil:<date>`	`dateUntil:2026-12-31`
Volume	`volume:<label>`, `volume:none`	`volume:Archive`, `volume:none`
Asset ID	`id:<prefix>`	`id:72a0bb4b`
Copies	`copies:<N>`, `copies:<N>+`	`copies:1`, `copies:2+`

Filter	Syntax	Example
Variants	`variants:<N>`, `variants:<N>+`	`variants:2+`
Tag count (leaves)	`tagcount:<N>`, `tagcount:<N>+`	`tagcount:0` (untagged), `tagcount:5+`
Scattered	`scattered:<N>`, `scattered:<N>+`	`scattered:2+` (files in 2+ session roots)
Duration (seconds)	`duration:<N>`, `duration:<N>+`, `duration:<min>-<max>`	`duration:60`, `duration:30+`, `duration:10-120`
Codec	`codec:<text>`	`codec:h264`, `codec:hevc`, `codec:prores`
Musical key	`key:<key>`	`key:Am`, `key:F#m`
Tempo (BPM)	`bpm:<N>`, `bpm:<N>+`, `bpm:<min>-<max>`	`bpm:100-140`
GPS (bounding box)	`geo:<south>,<west>,<north>,<east>`	`geo:47.5,11.0,48.5,13.0`
GPS (radius)	`geo:<lat>,<lng>,<km>`	`geo:52.52,13.405,5`
GPS (any/none)	`geo:any`, `geo:none`	`geo:none`
Orphan assets	`orphan:true`	`orphan:true`
Missing files	`missing:true`	`missing:true`
Stale verification	`stale:<days>`	`stale:30`
Stacked	`stacked:true` or `stacked:false`	`stacked:true`
Face count	`faces:any`, `faces:none`, `faces:N`, `faces:N+`	`faces:2+`
Person *(Pro)*	`person:<name>`	`person:Alice`, `person:"John Smith"`, `person:Alice,Bob` (either), `person:Alice person:Bob` (both)
Visual similarity *(Pro)*	`similar:<id>` or `similar:<id>:<limit>`	`similar:72a0bb4b`, `similar:72a0bb4b:50` (default 40)
Query image *(Pro)*	`maki search --image <file>`	`maki search --image preview.jpg`
Similarity threshold *(Pro)*	`min_sim:<percent>`	`min_sim:90` (with `similar:`, `text:`, `--image`)
Text search *(Pro)*	`text:<query>`, `text:"<query>":<limit>`	`text:"sunset beach"` (quote multi-word queries)
Embedding status	`embed:any`, `embed:none`	`embed:none type:image`

Hierarchical tag matching: The `tag:` filter matches hierarchically. Searching for `tag:animals` finds assets tagged `animals`, `animals|birds`, `animals|birds|eagles`, and any other descendant of `animals` — and a full path like `tag:animals|birds` still includes its descendants. To match one tag value and nothing else, use the whole-path marker: `tag:=animals|birds`. Three more leading markers exist: `tag:/x` (leaf only — the tag with no deeper child on the asset), `tag:^x` (case-sensitive), and `tag:|x` (prefix anchor: any component starting with `x`). They combine, e.g. `tag:=^Legoland`. See tag in the reference.

Pro filters on a standard build: `similar:`, `min_sim:`, `text:` and `--image` are silently ignored when MAKI was built without the `ai` feature; `person:` matches nothing there.

Multilingual `text:` search *(Pro)*: The default AI model is English-only. To search in German, French, Spanish, etc., switch to the multilingual model in `maki.toml` and re-embed: see AI Models in the setup guide.

For the complete filter reference with detailed syntax and behavior, see Search Filters.

Path Prefix Filter

The `path:` filter restricts results to assets whose file locations start with a given prefix. This is useful for finding everything imported from a particular directory:

```
maki search "path:Capture/2026-02-22"
maki search "path:Events/Wedding"
```

Automatic path normalization in the CLI handles several convenience cases:

- `~` expands to your home directory
- `./` and `../` resolve relative to your current working directory
- Absolute paths that match a registered volume's mount point are automatically stripped to volume-relative, and the corresponding volume filter is applied

For example, if you have a volume mounted at `/Volumes/Photos`:

```
maki search "path:/Volumes/Photos/Capture/2026"
# Equivalent to: maki search "path:Capture/2026" on the Photos volume
```

An explicit `volume:` filter in the same query takes precedence over the auto-detected volume.

Output Formats

Control how search results are displayed with `--format` or `-q`.

Preset Formats

Default (short) – one line per result with a result count:

```
$ maki search "sunset"
a1b2c3d4  IMG_1234.jpg [image] (JPEG) — 2026-01-15T10:30:00
e5f6a7b8  DSC_5678.nef [image] (NEF) — 2026-01-14T16:45:00

2 result(s)
```

IDs only (`--format ids` or `-q`) – one UUID per line, ideal for scripting:

```
$ maki search -q "tag:landscape"
a1b2c3d4-5678-9abc-def0-123456789abc
e5f6a7b8-1234-5678-9abc-def012345678
```

Full (`--format full`) – includes tags and description:

```
$ maki search "sunset" --format full
a1b2c3d4  IMG_1234.jpg [image] (JPEG) — 2026-01-15T10:30:00 tags:sunset,landscape A golden sunset over the mountains
```

JSON (`--format json` or `--json`) – structured JSON array:

```
$ maki search "sunset" --json
[
  {
    "asset_id": "a1b2c3d4-5678-9abc-def0-123456789abc",
    "original_filename": "IMG_1234.jpg",
```

```
    "asset_type": "image",
    ...
  }
]
```

Custom Templates

Build your own output format using `{placeholder}` syntax:

```
maki search "sunset" --format '{id}\t{name}\t{tags}'
maki search "type:image" --format '{short_id} {filename} [{format}] {label}'
maki search "rating:5" --format '{name}\n  {description}\n'
```

Available placeholders:

Placeholder	Content
`{id}`	Full asset UUID
`{short_id}`	First 8 characters of the UUID
`{name}`	Asset name (falls back to filename)
`{filename}`	Original filename
`{type}`	Asset type (image, video, audio, etc.)
`{format}`	File format (JPEG, NEF, MOV, etc.)
`{date}`	Creation date
`{tags}`	Comma-separated tags
`{description}`	Asset description
`{hash}`	Content hash of the best variant
`{label}`	Color label (Red, Blue, etc.)

Escape sequences: `\t` (tab), `\n` (newline), `\\` (literal backslash).

When `--format` is explicitly provided, the result count line at the end is suppressed, keeping the output clean for piping and parsing.

For the complete format reference, see Format Templates.

Sort Options

The web UI provides inline sort toggle buttons (Name, Date, Size, and Similarity on a similarity view) with ascending/descending direction indicators. In the CLI, sort order is available when saving searches:

```
maki saved-search save "landscapes" "tag:landscape" --sort name_asc
```

Available sort values:

Sort	Description
`date_desc`	Newest first (default)
`date_asc`	Oldest first

Sort	Description
`name_asc`	Alphabetical A-Z (by name, falling back to filename)
`name_desc`	Alphabetical Z-A
`size_asc`	Smallest first
`size_desc`	Largest first
`similarity_desc`	Most similar first *(Pro; the default for `similar:` and query-image views)*
`similarity_asc`	Least similar first *(Pro)*

`maki search` itself has no `--sort` flag: output is newest first, except that `similar:` and `--image` searches come back ordered by similarity score.

Scripting with maki

The `-q` flag and format options make `maki` composable with standard Unix tools.

Loop over search results:

```
for id in $(maki search -q "tag:landscape"); do
    maki show "$id"
done
```

Pipe results into collection commands:

```
maki search -q "rating:5" | xargs maki col add "Best"
```

Extract specific fields with jq:

```
maki search "sunset" --json | jq '.[].asset_id'
maki search "type:video" --json | jq '.[] | {name: .original_filename, date: .created_at}'
```

Count results:

```
maki search -q "format:nef" | wc -l
```

Tab-separated output for spreadsheets:

```
maki search "type:image" --format '{name}\t{format}\t{date}\t{tags}'
```

Combine with other maki commands:

```
# Tag all 5-star images as "portfolio"
maki search -q "rating:5 type:image" | xargs -I{} maki tag {} portfolio

# Generate previews for a specific path
maki search -q "path:Capture/2026-02" | xargs -I{} maki generate-previews --asset {}
```

Showing Asset Details

The `maki show` command displays comprehensive information about a single asset.

```
maki show a1b2c3d4
```

Asset ID prefix matching is supported – you only need to type enough characters to uniquely identify the asset:

```
maki show a1b2     # Works if "a1b2" is a unique prefix
```

Output

The human-readable output includes:

```
Asset: a1b2c3d4-5678-9abc-def0-123456789abc
Name:  Golden Sunset
Type:  image
Date:  2026-01-15T10:30:00
Tags:  sunset, landscape, mountains
Rating: ★★★★★ (5/5)
Label: Red
Description: A golden sunset over the mountain range
Preview: /path/to/catalog/previews/a1/a1b2c3d4...jpg

Variants:
  [original] DSC_1234.NEF (NEF, 48.2 MB)
    Hash: abc123def456...
    Location: Photos → Capture/2026-01-15/DSC_1234.NEF
    camera_make: NIKON CORPORATION
    camera_model: NIKON Z 9
    focal_length: 70.0
    f_number: 8.0
    iso: 200
  [export] DSC_1234-Edit.tif (TIFF, 120.5 MB)
    Hash: def789abc012...
    Location: Photos → Export/2026-01-15/DSC_1234-Edit.tif

Recipes:
  [xmp] CaptureOne (e4f5a6b7c8...)
    Path: Capture/2026-01-15/DSC_1234.NEF.xmp
```

For structured output, use the `--json` flag:

```
maki show a1b2c3d4 --json
```

This returns a full `AssetDetails` JSON object with all variants, locations, recipes, and source metadata.

Quick location check

Use `--locations` to see just where an asset's files live, without the full detail output:

```
$ maki show a1b2c3d4 --locations
Photos:Capture/2026-01-15/DSC_1234.NEF
Backup:Capture/2026-01-15/DSC_1234.NEF
```

This is handy before relocating or deleting an asset, to confirm which volumes hold copies.

Finding Duplicates

Files with identical content can exist in multiple locations for very different reasons. A file copied twice into different folders on the same drive is probably an accident. The same file on your working drive *and* your

backup drive is exactly what you want. The `maki duplicates` command helps you distinguish between these two situations.

Same-volume duplicates: finding unwanted copies

Files that appear more than once on the **same** volume are almost always accidental – imported twice from different paths, copied into a staging folder and forgotten, or left behind after a reorganization:

```
$ maki duplicates --same-volume
DSC_0042.NEF (NEF, 48.2 MB)
  Hash: abc123def456...
    Photos[working] → Capture/2026-01-15/DSC_0042.NEF  [SAME VOLUME]
    Photos[working] → Import/Unsorted/DSC_0042.NEF      [SAME VOLUME]

1 file(s) with same-volume duplicate locations
```

These are candidates for cleanup with `maki dedup` (see Maintenance).

Cross-volume copies: verifying backups

Files that exist on **different** volumes represent your backup strategy at work. Use `--cross-volume` to verify that important files actually have copies on your archive or backup drives:

```
$ maki duplicates --cross-volume
DSC_0042.NEF (NEF, 48.2 MB)
  Hash: abc123def456...
    Photos[working] → Capture/2026-01-15/DSC_0042.NEF
    Backup[backup]  → Capture/2026-01-15/DSC_0042.NEF

1 file(s) with cross-volume locations
```

Cross-volume copies are healthy – they mean your backups are working. If you see *fewer* cross-volume copies than expected, that is a signal to check backup coverage with `maki backup-status`.

All duplicates (default)

Without flags, the command shows all files with 2+ locations regardless of volume layout:

```
$ maki duplicates
```

This gives the full picture but mixes wanted backups with unwanted copies, so the filtered modes are usually more actionable.

Related tools

- **`copies:` search filter**: Find assets by their total copy count. `maki search "copies:1"` shows single-copy assets at risk of data loss.
- **`maki backup-status`**: Reports which assets lack copies on archive or backup volumes. Works best when volumes have a purpose assigned.
- **`maki dedup`**: Removes same-volume duplicates automatically. See Storage Hygiene.

Duplicate output formats

The `--format` flag supports the same presets as search, plus `{locations}` and `{volumes}` placeholders for templates:

```
maki duplicates --format ids
maki duplicates --format full
maki duplicates --format json
maki duplicates --format '{filename}\t{format}\t{volumes} volumes\t{locations}'
```

Additional filtering narrows results to a specific volume, format, or path:

```
maki duplicates --same-volume --volume "Photos" --filter-format nef
maki duplicates --cross-volume --path "Capture/2026"
```

Catalog Statistics

The `maki stats` command provides an overview of your catalog's contents and health.

Overview (Default)

```
$ maki stats
Catalog Overview
  Assets:    12,847
  Variants:  18,203
  Recipes:   9,614
  Volumes:   3 (2 online, 1 offline)
  Total size: 1.8 TB
```

Asset Types and Formats

```
$ maki stats --types
Catalog Overview
  ...

Asset Types
  image         11,204  (87.2%)
  video          1,412  (11.0%)
  audio            231  (1.8%)

Variant Formats
  NEF            8,100
  JPEG           6,543
  TIFF           2,104
  MOV            1,412

Recipe Formats
  xmp            7,200
  cos            2,414
```

Per-Volume Details

```
$ maki stats --volumes
Catalog Overview
  ...

Volumes
  Photos [online]
```

```
    Assets: 10,234  Variants: 14,567  Recipes: 7,890
    Size: 1.2 TB  Directories: 342
    Formats: NEF, JPEG, TIFF, MOV
    Verified: 14,200/14,567 (97.5%)
    Oldest verification: 2026-01-01T08:00:00
  Backup [offline]
    Assets: 8,412  Variants: 12,100  Recipes: 5,200
    Size: 980.4 GB  Directories: 298
    Formats: NEF, JPEG, TIFF
    Verified: 12,100/12,100 (100.0%)
    Oldest verification: 2025-12-15T14:30:00
```

Tag Statistics

```
$ maki stats --tags
Tags
  Unique tags:     145
  Tagged assets:   9,876
  Untagged assets: 2,971

  Top Tags
    landscape               1,234
    portrait                  987
    street                    654
    ...
```

Verification Health

```
$ maki stats --verified
Verification
  Total locations:    26,667
  Verified:           26,300
  Unverified:            367
  Coverage:           98.6%
  Oldest verified:    2025-12-15T14:30:00
  Newest verified:    2026-02-22T19:00:00

  Per Volume
    Photos [online]: 14,200/14,567 (97.5%)
    Backup [offline]: 12,100/12,100 (100.0%)
```

Combined and Tuned Output

Use `--all` to show every section at once:

```
maki stats --all
```

Control the number of entries in top-N lists (default is 20):

```
maki stats --tags --limit 50
```

For structured output:

```
maki stats --all --json
```

Catalog Health

Where `maki stats` answers *what's in the catalog*, `maki status` answers *what needs attention*. It's a read-only survey that aggregates signals from cleanup, backup-status, and the AI/embedding stores into one prioritized report — every actionable line ends with a `→ command` suggestion so you don't have to consult docs to know the next step.

```
$ maki status
Gathering catalog status (scanning derived files; may take a moment)...
MAKI catalog status — /Users/you/.maki

Catalog
  Schema:   v8 (current)
  Counts:   12,847 assets · 18,203 variants · 9,614 recipes · 21,118 file locations
  Storage:  1.8 TB across 3 volume(s) (2 online, 1 offline)

Cleanup
  ✗ 5 locationless variant(s)                          → maki cleanup --apply
  ✗ 47 orphaned embedding file(s) on disk              → maki cleanup --apply

Pending work
  ✗ 28 pending XMP writeback(s) on offline volume(s)   → mount the volumes, then `maki writeback`
  ✗ 142 asset(s) without an embedding                  → maki embed

Backup coverage
  ✗ 124 of 12847 asset(s) (1.0%) have fewer than 2 copies → maki backup-status --at-risk

Volumes
  ● Photos       /Volumes/Photos    10234 asset(s), 1.2 TB [media]
  ● Backup-A     /Volumes/Backup-A  10234 asset(s), 1.2 TB [backup]
  ○ Travel-2026  /Volumes/Travel    810 asset(s), 35 GB [working] (offline)
```

Run it as part of a routine maintenance pass. The orphan-on-disk scan is the slow part (the same pass `maki cleanup --dry-run` runs); on a 12k-asset catalog expect ~30 seconds. The command is informational only — exit code is always 0 even when action items are present.

`--min-copies N` raises the backup-coverage floor; `--json` produces a structured `StatusReport` for scripting.

Exporting Files

The `maki export` command copies files matching a search query to a target directory — useful for client deliveries, sharing, or copying to external media.

Basic Export

Export the best variant of each matching asset:

```
maki export "rating:5 tag:portfolio" /tmp/delivery/
```

Layout Modes

Flat (default) — all files in one directory. Best for client deliveries where folder structure doesn't matter:

```
maki export "collection:Selects" /tmp/flat/
```

Filename collisions from different assets are resolved by appending a hash suffix (e.g., `DSC_001_a1b2c3d4.jpg`).

Mirror — preserves source directory structure. Best for handoff to other tools, archival exports, or when folder organization carries meaning:

```
maki export "tag:landscape" /Volumes/USB/export --layout mirror
```

When assets span multiple volumes, each volume's files are placed under a `<volume-label>/` prefix.

ZIP export

For email or upload delivery, export directly to a ZIP archive instead of a directory:

```
maki export "collection:Wedding Selects" delivery.zip --zip
```

This creates a single file that's easy to share. Combines with all other options (`--all-variants`, `--include-sidecars`, etc.).

Common export scenarios

Client delivery — flat layout, best variants only:

```
maki export "collection:Selects" /tmp/delivery/
```

Handoff to CaptureOne/Lightroom — mirror layout with sidecar files, so the other tool picks up your folder structure and XMP metadata:

```
maki export "tag:project-x" /Volumes/USB/handoff --layout mirror --include-sidecars
```

Temporary working folder — symlinks instead of copies, useful when you want to open files in Photoshop or another tool without copying gigabytes:

```
maki export "tag:retouch" ~/Desktop/retouch-batch/ --symlink
```

The symlinks point to the original files on your volumes. Delete the symlink folder when you're done — the originals are untouched.

Full archive export — all variants (RAW + processed + exports) with sidecars:

```
maki export "date:2025" /Volumes/USB/archive-2025 --layout mirror --all-variants --include-sidecars
```

Other options

Preview without writing files:

```
maki export "rating:4+" /tmp/test/ --dry-run
```

Re-running an export skips files that already exist with matching content. Use `--overwrite` to force re-copy.

For the full command reference, see export.

Visual Similarity *(Pro)*

With image embeddings generated (via `maki embed` or `maki import --embed`), you can find visually similar assets:

```
maki search "similar:72a0bb4b"              # top 40 similar assets (source first)
maki search "similar:72a0bb4b:50"           # top 50 similar
maki search "similar:72a0bb4b rating:4+"    # similar AND 4+ stars
maki search "similar:72a0bb4b min_sim:85"   # only close matches
```

```
maki search --image ~/Downloads/preview.jpg # similar to a file that is NOT in the catalog
```

The `similar:` filter uses SigLIP image embeddings for fast dot-product similarity search. It composes with all other filters, results are ordered by score, and `min_sim:` floors the score. `--image` does the same for a local file (a byte-identical copy in the catalog is pinned first as `100% exact`); see Finding the Original Behind a Preview.

For an interactive visual exploration experience, the web UI offers a **Stroll page** (`/stroll`) where you can navigate through your collection by clicking visually similar neighbors arranged around a center image. A fan-out slider (0–10) controls L2 transitive neighbor discovery: when fan-out is greater than 0, hovering or arrow-keying to a satellite reveals its own nearest neighbors fanning out behind it – useful for exploring broader visual themes. See Web UI – Stroll Page for details.

Contact Sheets

The `maki contact-sheet` command generates a PDF with thumbnail grids of your assets — useful for client proofing, archival reference, portfolio review, or simply getting an overview of a shoot on paper.

Basic usage

Generate a contact sheet from any search query:

```
maki contact-sheet "tag:wedding date:2026-03" wedding-proofs.pdf
```

This produces an A4 PDF with the standard layout (4 columns, 6 rows per page) using smart previews when available.

Client proofing

When preparing proofs for a client, add a title, copyright notice, and metadata fields below each thumbnail:

```
maki contact-sheet "tag:wedding rating:3+" proofs.pdf \
  --title "Johnson Wedding — March 2026" \
  --copyright "Jane Doe Photography" \
  --fields "filename,rating"
```

The `--fields` option accepts a comma-separated list: `filename`, `date`, `rating`, `label`, `format`, `size`, `dimensions`.

Shoot overview

For a quick visual overview of a multi-day shoot, group by date and use the dense layout:

```
maki contact-sheet "path:Capture/2026-03" march-overview.pdf \
  --group-by date --landscape --layout dense
```

Grouping options: `date` (by shooting day), `volume`, `collection`, `label` (by color label). Each group gets a section header in the PDF.

Layout presets and paper sizes

Three built-in layout presets control thumbnail density:

Preset	Columns	Rows	Best for
`dense`	6	8	Index prints, archival reference

Preset	Columns	Rows	Best for
`standard`	4	6	Client proofing (default)
`large`	3	4	Portfolio review, detail inspection

Override columns and rows individually with `--columns` and `--rows`. Paper sizes: `a4` (default), `letter`, `a3`.

```
# Large thumbnails on A3 landscape for portfolio review
maki contact-sheet "rating:5" portfolio.pdf \
  --layout large --paper a3 --landscape

# Dense index of an entire archive volume
maki contact-sheet "volume:Archive" archive-index.pdf \
  --layout dense --fields "filename,date,format" --group-by date
```

Color labels and sorting

The `--label-style` option controls how color labels appear: `border` (colored border around thumbnail), `dot` (small colored circle), or `none`. Sort with `--sort`: `date`, `name`, `rating`, `filename`.

```
maki contact-sheet "label:Green,Blue" selects.pdf \
  --sort rating --label-style border
```

Dry run

Preview page count and asset count before generating:

```
maki contact-sheet "tag:landscape" test.pdf --dry-run
```

For the full command reference, see contact-sheet.

Related Topics

- Organizing Assets – tags, editing, collections, and saved searches
- Web UI – browser-based search with interactive filters, sort controls, and batch operations
- Search Filters Reference – complete filter syntax documentation
- Format Templates Reference – all placeholders and template syntax
- CLI Conventions – global flags (`--json`, `--log`, `--time`)

Web UI

MAKI includes a browser-based interface for browsing, searching, and editing your catalog. It runs as a local web server and provides a responsive grid view of your assets, inline metadata editing, batch operations, and keyboard-driven navigation. All changes made in the web UI are written to both the SQLite catalog and YAML sidecar files, and any editable fields (rating, tags, description, color label) are automatically synced back to XMP recipe files on disk.

Starting the Web UI

Launch the server with `maki serve`:

```
maki serve
```

Output (on stderr):

```
MAKI 4.10.0 web UI: http://127.0.0.1:8080
```

Open that URL in your browser to start browsing. The server runs in the foreground; press Ctrl+C to stop it. Two more lines may follow the address: `Read-only mode: metadata edits and jobs are disabled.` when started with `--read-only` (or `[serve] read_only = true`), and `Basic authentication: enabled.` when `[serve] username`/`password` are set. Binding to a non-loopback address without either protection prints a three-line `WARNING:` block instead — see Read-only mode and authentication below.

Custom port and bind address

Use `--port` and `--bind` to change the listening address:

```
maki serve --port 9090 --bind 0.0.0.0
```

Binding to `0.0.0.0` makes the UI accessible from other devices on your local network. Anyone who can reach the address can then edit metadata and delete files, so combine it with `--read-only` or basic authentication (next section).

Read-only mode and authentication

`maki serve --read-only` starts the server in safe-sharing mode: every request that is not `GET`/`HEAD` — metadata edits, batch operations, maintenance jobs, deletions — is rejected with `403`, and the UI hides its edit affordances (`/api/build-info` reports `read_only`). The one exception is the query-image upload behind "Find by image", which is a POST by necessity but writes nothing. The mode can be made permanent with `[serve] read_only = true` in `maki.toml`.

HTTP Basic authentication protects every route — pages, API, previews, and the job event streams — when `[serve] username` and `[serve] password` are set. The password may also come from the `MAKI_SERVE_PASSWORD` environment variable, which takes precedence over the config value, so the secret need not live in `maki.toml`. Both mechanisms compose: a read-only, password-protected server is the recommended way to expose a catalog beyond the local machine. See the Configuration Reference for the exact keys.

Request logging

Add the `--log` flag to print each HTTP request to stderr:

```
maki serve --log
```

Output on stderr:

```
GET / -> 200 (12ms)
GET /static/style.css -> 200 (1ms)
GET /preview/ab/ab3f...jpg -> 200 (2ms)
```

This is useful for debugging slow requests or understanding access patterns.

Configuration via maki.toml

You can set default values for port and bind address in the `[serve]` section of `maki.toml`, alongside the settings that have no CLI flag:

```
[serve]
port = 9090
bind = "127.0.0.1"
per_page = 60              # results per browse page
read_only = false          # same as --read-only
# username = "thomas"      # HTTP Basic auth (password may come from MAKI_SERVE_PASSWORD)
# password = "secret"
stroll_neighbors = 12      # stroll page defaults, see the Stroll section
stroll_neighbors_max = 25
stroll_fanout = 5
stroll_fanout_max = 10
stroll_discover_pool = 80
```

Command-line flags always override `maki.toml` settings. See the Configuration Reference for details.

Navigation bar

Every page has a horizontal nav bar across the top. From left to right:

- **Brand** (MAKI logo) — link to the browse page.
- **Browse** and **Stroll** *(Pro)* — primary content views.
- **Tags**, **People** *(Pro)*, **Collections**, **Searches** — catalog organisation pages (see Saved Searches Page for **Searches**).
- **Catalog** ▾ dropdown — collapses Stats, Analytics, Backup, Volumes, Duplicates into one menu so the bar stays compact. Click the trigger to open; click outside or press Escape to close. The caret rotates when open. These pages are read-mostly catalog inspection views, used a few times a week rather than daily.
- **Import**● and **Maintain**● — long-running command launchers (see Importing from a volume and Maintain Dialog below). The pulsing dot lights up when a job of that kind is running anywhere in the catalog; clicking the entry while a job is in flight re-attaches to its progress feed.
- **Settings** — opens the schema-driven `maki.toml` editor (see Settings Dialog below).
- **Help (?)** and **Theme toggle** — keyboard-shortcut overlay and dark-mode switch.

The dropdown is keyboard-accessible (`Tab` to focus, `Enter` to open, `Esc` to close) and screen-reader friendly (`role="menu"` with `aria-expanded` on the trigger).

Dark Mode

The web UI supports dark mode. A toggle button (sun/moon icon) in the navigation bar switches between light and dark themes. Your preference is saved in the browser and applied instantly on every page load.

If you haven't chosen a theme explicitly, the UI follows your operating system preference automatically (via `prefers-color-scheme`). Dark mode covers all pages: browse, asset detail, tags, collections, stats, and backup status.

Browse Page

The browse page is the main entry point for the web UI. It shows a searchable, filterable grid of asset thumbnails.

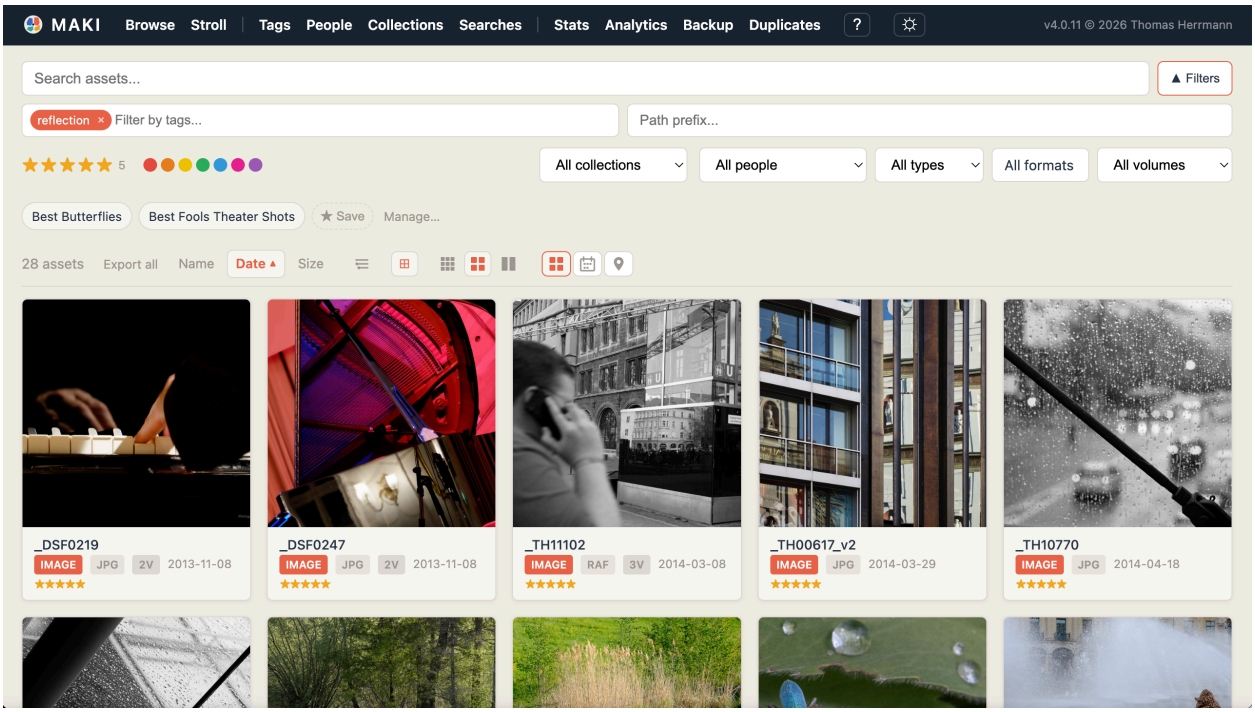


Figure 1: Browse page with search bar, filter controls, and results grid

Search bar

The search bar has two rows:

Row 1 – a full-width text input for free-text search. Type any keyword, filename, or structured filter (like `camera:"Canon EOS R5"`) and results update as you type with a 300ms debounce. Press Enter to search immediately without waiting for the debounce.

In the Pro edition the first row also carries a **Find by image** button: pick a local image file that is not in the catalog and the grid ranks the catalog by visual similarity to it (see Similarity browse below). You can also drop an image file anywhere on the browse page, or paste one from the clipboard.

The filter bar is collapsible. Press **Shift+F** to toggle its visibility on the browse page. Your preference is saved in the browser. This is useful for maximizing screen space when you want to focus on the grid without changing filters.

Row 2 – a row of filter controls, left to right:

- **Tag filter**: a chip-based input with autocomplete. Type to see tag suggestions, click or press Enter to add a tag chip. Multiple tags narrow the results (AND logic). Remove a tag by clicking the x on its chip, or press Backspace in an empty input to remove the last chip. Each chip has three inline toggles: `+/−` (include vs exclude — a `-` chip is the inline equivalent of `-tag:foo` in the search field), `▾/=/⁄⁄` (mode: include descendants vs whole-path vs leaf-only), and `cc/Cc` (case-insensitive vs sensitive). Adding, removing, or toggling a chip triggers an immediate search.
- **Star rating filter**: five clickable stars with a three-state cycle. First click sets a minimum match (e.g. "3+"), second click narrows to an exact match ("3"), third click clears the filter. Star 5 is two-state (exact and clear, since 5 and 5+ are identical). Triggers an immediate search.
- **Color label filter**: seven colored dots (Red, Orange, Yellow, Green, Blue, Pink, Purple). Click a dot to filter by that label. Click the active dot again to clear. Triggers an immediate search.
- **Type dropdown**: filter by asset type (Image, Video, Audio, Document). Triggers immediately on change.
- **Format filter**: a grouped multi-select dropdown. Click the trigger button to open a panel showing all formats in your catalog organized by category (RAW, Image, Video, Audio, Other). Each format shows its variant count. Check individual formats or use the group-level toggles ("All RAW", "All Image", etc.) to select entire categories. Multiple selections combine with OR logic (e.g., selecting NEF and CR3 shows assets in either format). The trigger button shows: "All formats" when nothing is selected, the format name for a single selection, the group name when a full group is selected (e.g., "All RAW"), or a compact summary like "nef +3…" for mixed selections. Hover the button to see the full list in a tooltip. Triggers an immediate search on every checkbox change.
- **Volume dropdown**: filter by storage volume. Only shown when you have registered volumes. Triggers immediately on change.
- **Collection dropdown**: filter by collection membership. Only shown when you have collections. Triggers immediately on change.
- **Path prefix input**: filter by file location path prefix. Type a directory path to see only assets stored under that path, or pick one from the autocomplete dropdown. Focus the field to browse the directory tree from scratch (empty prefix lists all top-level directories); type to narrow. Each accepted directory ends in `/` and the dropdown immediately fetches the next level — drill down one segment at a time without retyping, shell-style. Accepting a file leaf commits the filter. Use `↑/↓` to move through suggestions, `Tab` or `Enter` to accept the highlighted one, `Escape` to close the dropdown. Wildcard patterns (anything containing `*`) disable autocomplete — the filter already understands `*` itself. If you paste an absolute path that matches one of your registered volume mounts, the mount prefix is stripped automatically and the dropdown pins to that volume. If a volume is selected in the Volume dropdown, path suggestions are narrowed to that volume. Debounces at 120ms; Enter with no highlighted suggestion fires the search immediately.

All filters compose with each other. You can combine a text search with a tag, a minimum rating, a color label, and a volume restriction at the same time.

Results grid

Below the search bar, results appear as a grid of thumbnail cards. Each card shows:

- A preview thumbnail (lazy-loaded for performance)
- The asset name (or filename as fallback)
- Badges for type and primary format (e.g., "image" and "NEF")
- A duration badge on video assets showing the video length (e.g., "1:23")
- A variant count badge when the asset has multiple variants (e.g., "3v")
- A stack badge when the asset is in a stack (e.g., a small grid icon with the member count)
- Star rating (filled stars)
- Color label dot

Cards that belong to a stack have a colored left border for quick visual identification.

Stack collapsing: By default, stacks are collapsed – only the pick image (position 0) is shown in the grid,

with a stack count badge indicating how many assets are hidden. A collapse toggle in the results bar lets you expand all stacks so every member is visible. When expanded, all stack members appear in position order with the colored border indicating membership.

Result-count hints: when stacks are collapsed (or a `[browse] default_filter` is active), the count line surfaces *additional matches that exist but aren't currently rendered*. For example, `152 assets … · 73 more in stacks · 12 more without default filter`. Each `more …` segment is a clickable link that re-runs the search with the corresponding flag flipped (`&stacks=0` or `&nodefault=1`) and updates the URL so the back button works. Each segment is shown only when it'd actually surface more results, so the count line stays clean when there's nothing hidden behind a flag.

Per-stack expand/collapse: Click the stack badge (⊞ N) on any browse card to expand or collapse just that individual stack, regardless of the global collapse setting. When stacks are globally collapsed, clicking a badge fetches and displays that stack's members inline. When stacks are globally expanded, clicking a badge collapses only that stack while others remain expanded. Re-clicking restores the previous state.

Click a card to open the asset detail page.

View mode toggle

Next to the density controls, three view mode buttons switch between **Grid**, **Calendar**, and **Map** views:

- **Grid** (default): the standard thumbnail grid described above
- **Calendar**: a year-at-a-glance heatmap showing when assets were created
- **Map**: a map showing geotagged assets as clustered markers

Your view mode preference is saved in the browser and preserved across reloads.

Calendar heatmap view

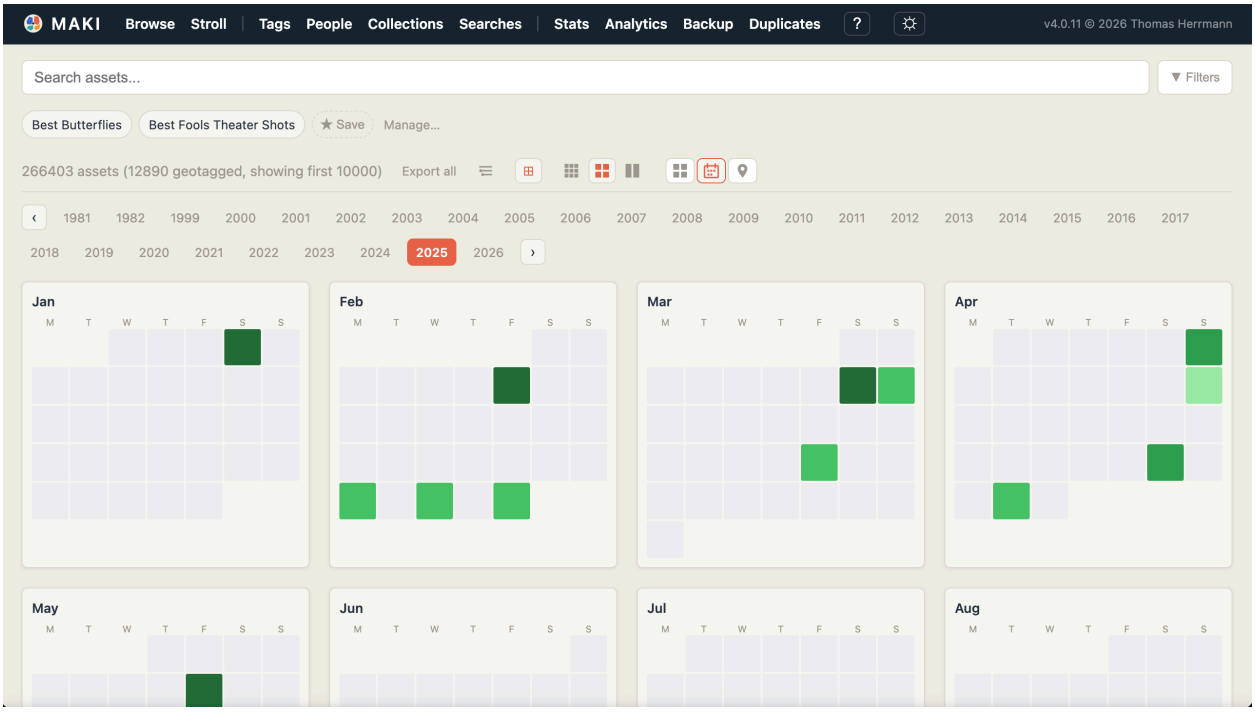


Figure 2: Calendar heatmap view with monthly grids and year navigation

The calendar view displays a GitHub-style heatmap with 12 months laid out in a responsive grid (4 columns on wide screens, adapting down to 2 on narrow ones). Each day cell is colored by the number of assets

created on that date, using a quartile-based 5-level color scale from empty (no assets) through increasingly intense shades.

Year navigation: Arrow buttons and clickable year chips above the calendar let you browse between years. Only years that contain assets appear as chips.

Filter integration: All browse filters (text search, tag, rating, label, type, format, volume, collection, path) apply to the calendar aggregation. For example, adding `tag:landscape` shows only landscape photos in the heatmap, letting you see which dates had landscape shots.

Day click: Click any day cell to filter the browse grid to that date. The view switches back to grid mode with `date:YYYY-MM-DD` added to the search query, showing only assets from that day. You can then clear the date filter from the search input to return to unfiltered browsing.

Legend: A color scale legend at the bottom of the calendar shows the meaning of each intensity level.

Map view

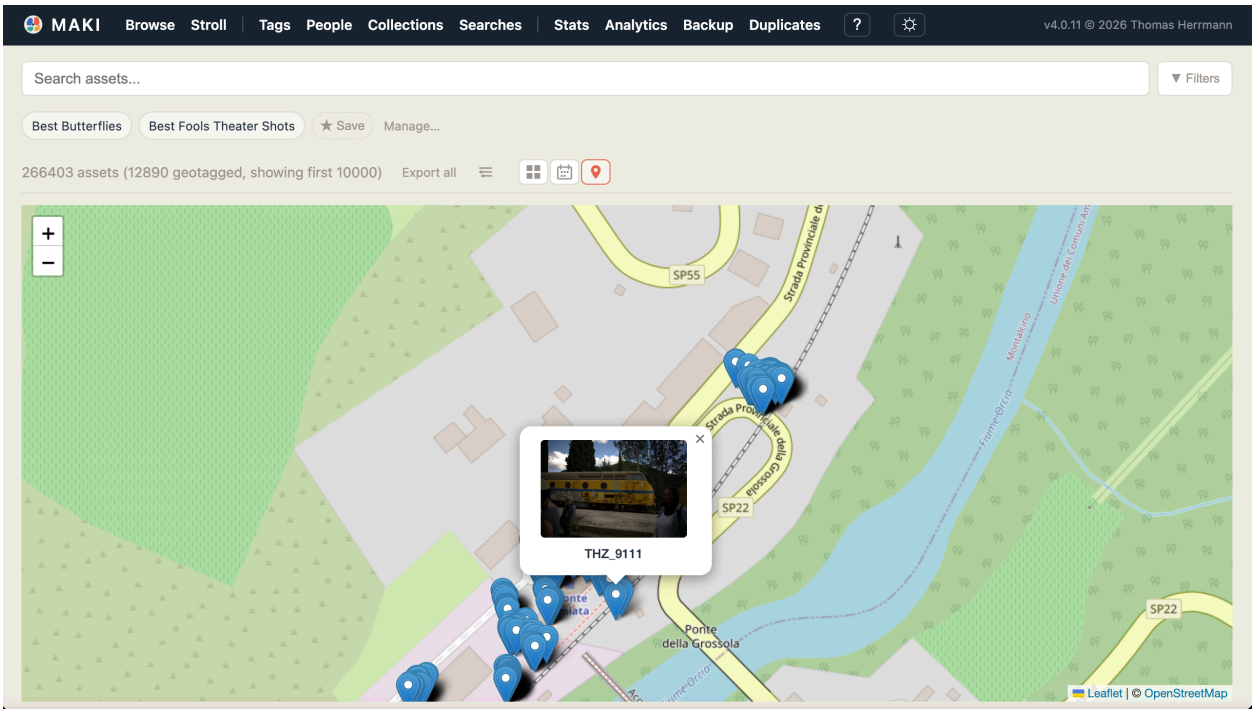


Figure 3: Map view with geotagged assets on OpenStreetMap

The map view displays geotagged assets on an interactive OpenStreetMap map powered by Leaflet.js. Assets with GPS coordinates appear as markers, automatically clustered at wider zoom levels for performance.

Markers and popups: Click a marker (or cluster) to see a popup with a thumbnail preview, asset name, rating stars, and color label dot. Click the thumbnail to open the asset in the lightbox with full prev/next navigation through visible markers. Click the name or metadata area to navigate to the asset detail page.

Filter integration: All browse filters (text search, tag, rating, label, type, format, volume, collection, path, date) apply to the map. Adding `tag:landscape` shows only landscape photos on the map. The `geo:` search filter also works: `geo:any` shows only geotagged assets, `geo:none` shows assets without GPS data, and `geo:52.5,13.4,10` restricts to a 10km radius around a point.

Dark mode: Map tiles are automatically inverted for the dark theme to maintain visual consistency. Popups and controls adapt to the current color scheme.

Keyboard shortcut: Press `m` to switch to map view from any other view mode.

Embedded libraries: Leaflet.js and MarkerCluster are embedded as static assets — no external CDN or internet connection is required. The map tiles themselves are loaded from OpenStreetMap servers, so an internet connection is needed for the tile layer (but not for the application code).

Grid density

Next to the sort controls, three grid density buttons let you adjust the thumbnail size:

- **Compact** (3×3 grid icon): smaller thumbnails, metadata and star ratings hidden, tighter spacing. Good for scanning many images quickly.
- **Normal** (2×2 grid icon): the default layout with full metadata.
- **Large** (1×2 grid icon): bigger thumbnails with room for two-line titles. Good for detailed visual comparison.

Your density preference is saved in the browser and preserved across page reloads and navigation.

Faceted sidebar

The faceted sidebar shows what's *in* the current result set — broken down by rating, color label, tag, year, format, volume, and geolocation — and lets you click any row to narrow the result further. It's the primary tool for **tag strolling**: pick a starting facet, see what other tags co-occur with it, click one to drill in, repeat.

Toggle: click the facet icon in the results bar (next to the stack-collapse button) or press `f`. Your preference is saved per browser. Hidden on viewports narrower than 768 px.

Section order: Ratings, Labels, Tags, Years, Formats, Volumes, Geotagged. Tags are above formats and volumes because the curated tag taxonomy is most users' main filtering axis; formats and volumes are usually set up once and rarely toggled. Each section is collapsible (open/closed state remembered per browser).

Tag hierarchy: tags render as an indented tree, not a flat list. MAKI auto-expands every hierarchical tag to its ancestor paths on storage, so the panel can reshape that flat data back into the original hierarchy. Counts on parent rows reflect the union of all descendants — `event` showing 2,419 means "any photo with anything under `event`". Clicking a node always applies the **full path**, so picking `2024` under `Holzkirchner Blues- und Jazztage` filters to that specific year of that specific festival, not every photo from 2024 across the catalog.

Click-to-narrow: every facet row is clickable, with the action matching the type of facet:

Facet	Click action
Tag	Adds the full tag path as a chip on the filter bar.
Rating	Sets the star widget to that exact rating.
Label	Sets the color-dot widget to that label.
Year	Appends `date:YYYY` to the search box.
Format	Toggles the format checkbox (additive — multi-select).
Volume	Sets the volume dropdown to that volume.
Geotagged	Appends `geotagged:1` to the search box.

Rows are keyboard-accessible: Tab to focus, Enter or Space to activate. Clicking a tag that's already in the active filter is a no-op (de-duplicated). After every click the panel re-fetches against the new filter, so the next layer of facets is computed live.

Worked example: exploring a festival

Suppose you want to see what's left under-tagged at the *Holzkirchner Blues- und Jazztage*, a recurring festival you've shot 13 years running.

1. Open the browse page with no filters. The Tags section shows top-level facets: `event`, `person`, `subject`, `location`, etc.
2. Click `event` → results narrow to all photos tagged as some kind of event. The panel re-fetches; tag tree now shows `event > anniversary`, `event > concert`, `event > festival`, …
3. Click `event > festival > Holzkirchner Blues- und Jazztage` → narrows to all 13 years. The Tags section now lists `…|2026`, `…|2025`, `…|2024`, … with per-year counts. The Years section also shows the matching calendar years.
4. Click a Years row, say `2018`. The query box gains `date:2018` — and you discover the asset count there *is greater than* the festival's `…|2018` row count: those extra photos are likely festival photos missing the festival tag. Use the result grid to spot-check and re-tag them.

This is the "what other tags appear in the same result set?" exploration mode — useful for both navigation (find a specific year/band) and audit (find inconsistently tagged photos).

Tip: combine with chips

Each click adds a filter; the chips at the top of the filter bar show your current path. Click a chip's × to back out one step without rebuilding the query. The browser back button works too — every filter change updates the URL.

Sorting

Above the results grid, sort toggle buttons let you order results by **Name**, **Date**, or **Size**. Each button toggles between ascending and descending. The active sort shows a direction arrow. Clicking a sort button updates results immediately.

Pagination

When results span multiple pages, pagination controls appear both above and below the grid:

- First page, previous page, numbered page links, next page, last page
- A "Page X of Y" indicator

Page numbers with ellipsis keep the controls compact for large result sets. The number of results per page defaults to 60 and is configured via `[serve] per_page` in `maki.toml` (there is no CLI flag for it).

Use **Shift+Left/Right arrow** keys to quickly turn pages from the keyboard. The results grid fades during loading to provide visual feedback while the new page loads. In the lightbox, regular arrow keys at page boundaries automatically navigate to the next/previous page with a loading spinner overlay. On the detail page the IDs of the neighbouring page are fetched ahead of time, a few assets before you reach the boundary; if you press the arrow while that fetch is still running, the keypress is remembered and carried out as soon as the IDs arrive.

Similarity browse

When searching with `similar:<asset-id>`, the browse grid shows visually similar assets with similarity percentage badges. A "Similarity" sort button appears for sorting by visual closeness. Use `min_sim:90` to filter by minimum similarity threshold.

Search by query image. The source does not have to be a catalog asset. Click **Find by image** in the search row, drop an image file anywhere on the page, or paste one from the clipboard, and the grid ranks the catalog against that file — the way to locate the original behind a preview or export someone sent back. The upload is hashed first: a byte-identical copy in the catalog is pinned at the top as 100%, and the

query pill above the grid says “exact copy in catalog”. Otherwise (or in addition) the file is encoded with the SigLIP model and ranked like `similar:`. The search field carries a `similar:@<token>` reference to the uploaded image, so every other filter, select-all, facets and export compose with it as usual. Starting the search switches the grid to similarity order whatever sort was active before; clearing the pill (its ×) returns to the previous default order. Query images live in server memory only, for an hour; a stale link shows an empty grid and a “drop it again” note. Nothing is imported or written to the catalog, so the feature also works on a `--read-only` server. The CLI equivalent is `maki search --image <file>`.

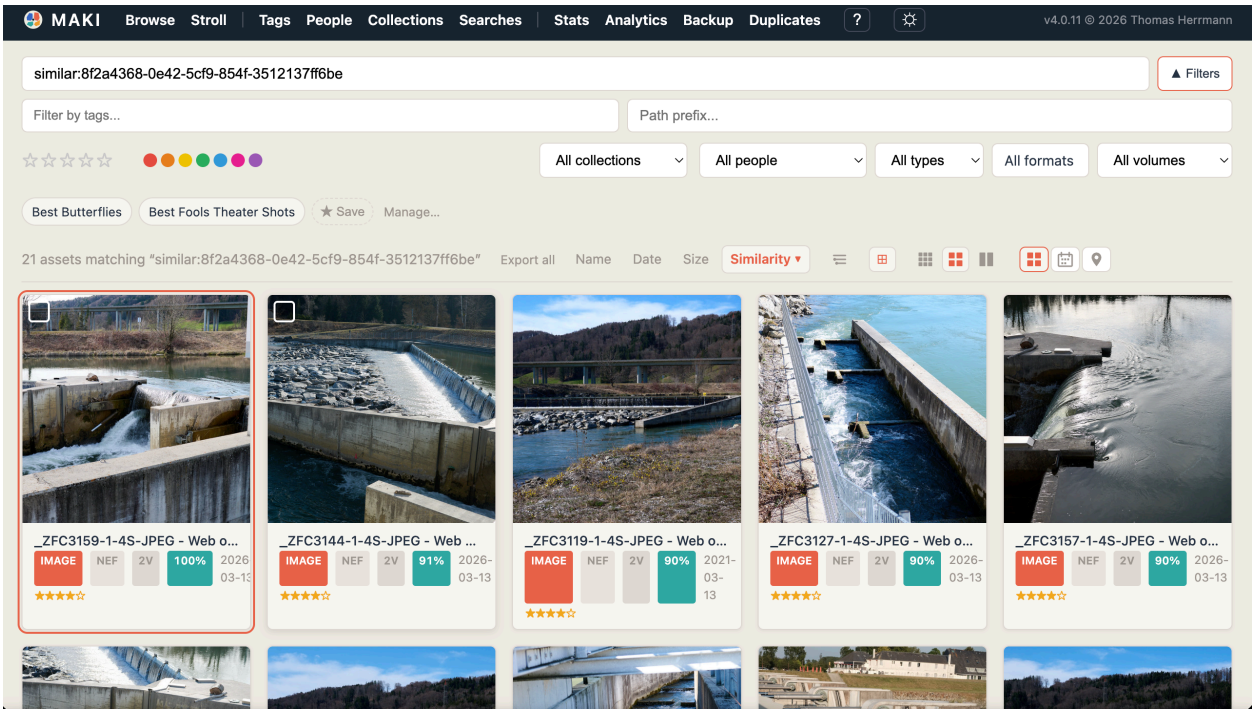


Figure 4: Similarity browse with percentage badges and sort control

Saved search chips

Below the search bar, a row of saved search chips provides quick access to your **favorite** named searches (smart albums). Each chip loads its stored query, filters and sort order when clicked.

- **Save the current search**: click the “Save” button at the end of the chip row. A prompt asks for a name, and the current search state (text query, filters, sort order) is saved and marked as a favorite, so the new chip appears immediately.
- Only favorites are shown as chips. Renaming, deleting, and toggling the favorite star are done on the Saved Searches Page (**Searches** in the navigation bar).

Saved searches are stored in `searches.toml` at the catalog root and work identically to CLI saved searches (see Organizing Assets for the `maki saved-search` command). Searches saved from the CLI start out as non-favorites (unless `maki saved-search save --favorite` was used) and do not show up as chips until starred.

How page updates work

The web UI uses htmx for partial page updates. When you search, filter, sort, or paginate, only the results area reloads – the search bar and its state remain intact. This makes interactions fast and fluid.

Because the URL updates with every search (via `hx-push-url`), the browser back button, forward button,

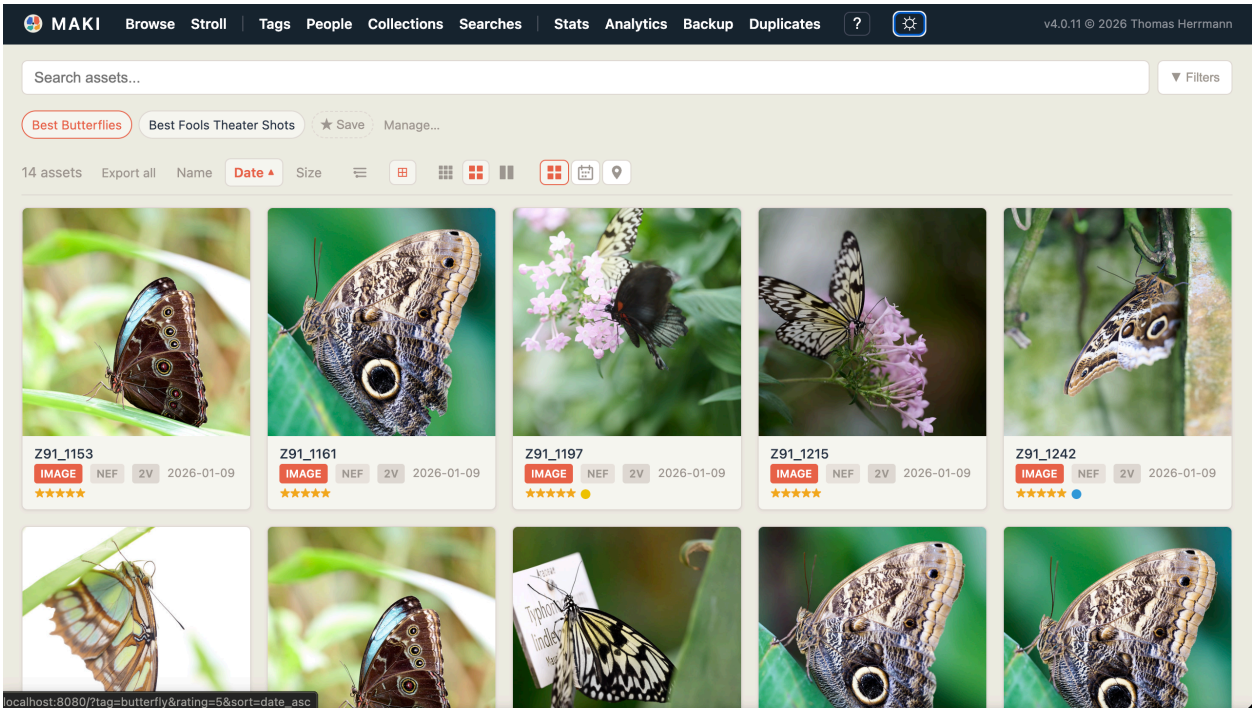
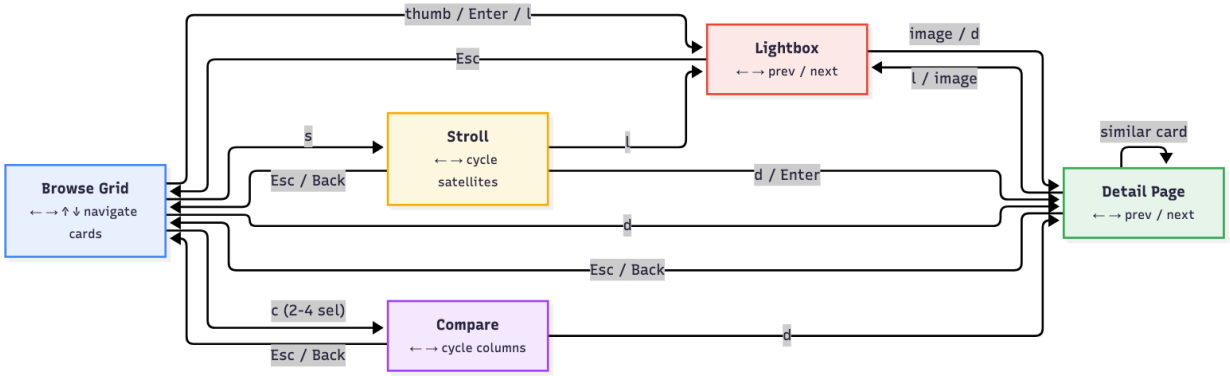


Figure 5: Saved search chips row on the browse page

reload, and bookmarks all work correctly. Navigating back from an asset detail page also refreshes results to reflect any edits you made.

Navigation Between Views

The browse grid, lightbox, detail page, stroll, and compare form a connected navigation graph. You can switch between views with keyboard shortcuts, clicks, and the browser back button. Focus, scroll position, and selection are preserved when you return to browse.



From	Action	To
Browse	Thumbnail click / Enter / `l`	Lightbox
Browse	`d`	Detail page
Browse	`s`	Stroll (from focused card)

From	Action	To
Browse	`c` (2–4 selected)	Compare
Lightbox	Click image / `d`	Detail page
Lightbox	`s`	Stroll (from current asset)
Lightbox	Escape	Close overlay (return to host page)
Lightbox	Left / Right arrows	Prev / next asset
Detail	Click image / `l`	Lightbox
Detail	Left / Right arrows	Prev / next detail page
Detail	Similar card click	Detail (chained navigation)
Detail	`s`	Stroll (from current asset)
Detail	Escape / Back	Previous page (`history.back()`)
Stroll	`l`	Lightbox (center or focused satellite)
Stroll	`d` / Enter	Detail page
Stroll	Escape / Back	Previous page (`history.back()`)
Compare	`d`	Detail page (focused column)
Compare	Escape / Back	Previous page (`history.back()`)
Any page	Shift+B	Jump directly to browse grid

Back navigation

There are two distinct "go back" actions:

- **Escape** (or browser Back button) — step back one level. On a page, this returns to wherever you came from via `history.back()`. In the lightbox overlay, it closes the overlay. On the browse grid, it clears the current selection or focus.
- **Shift+B** — jump directly to the browse grid, regardless of how deep you are. Useful when you've followed a chain of similar images or navigated through stroll and want to return to your search results.

The lightbox is available on browse, detail, and stroll pages. It uses the same shared component with page-specific callbacks for rating/label changes and navigation. Rating and color label editing (stars, color dots, 0–5 / Alt+1–7 keys) works everywhere the lightbox is open.

Focus and scroll position are preserved when navigating back to the browse page, even when the browser restores the page from its back/forward cache. Arrow key navigation in the lightbox and detail page updates which card will be focused when you return to browse.

Lightbox

Clicking a thumbnail opens a full-screen lightbox overlay. The lightbox is available on the browse grid, detail page, and stroll page.

Navigation

- **Arrow buttons**: click the left (‹) or right (›) side panels to move to the previous or next asset
- **Keyboard**: Left and Right arrow keys navigate between assets
- **Counter**: the top bar shows your position (e.g., "3 / 156")
- **Close**: click the × button or press Escape to close the overlay

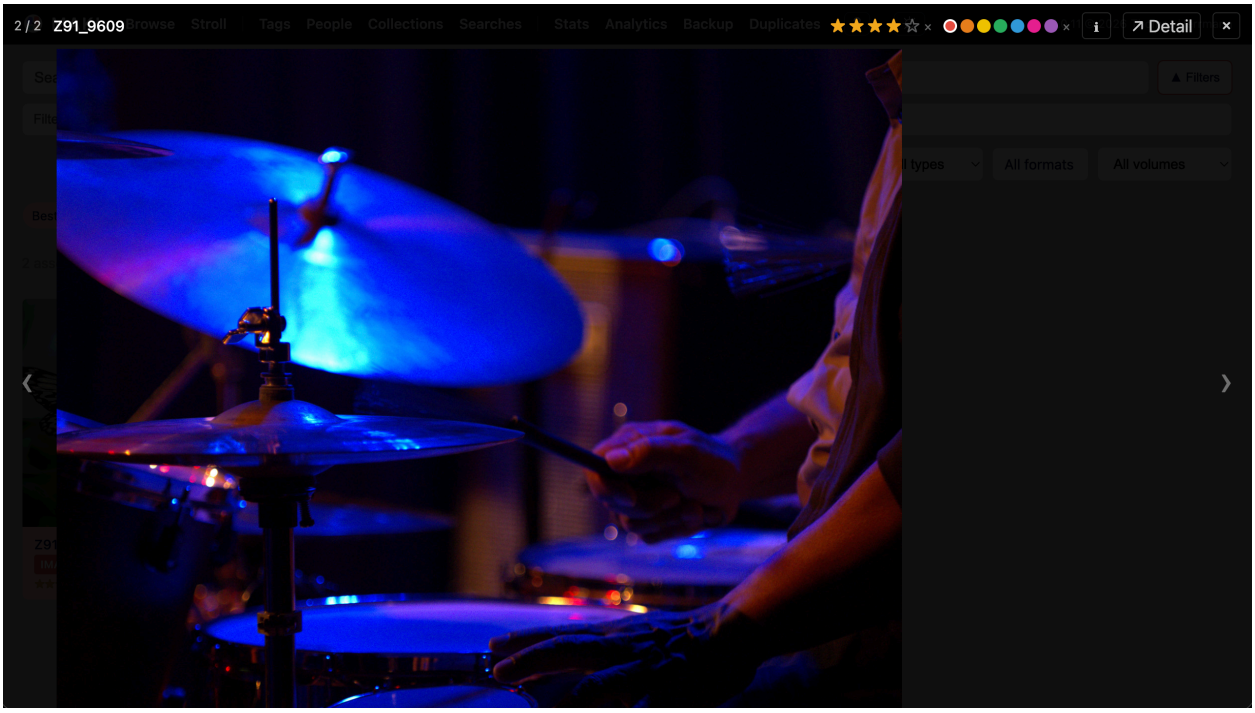


Figure 6: Lightbox with full-screen preview, rating, and navigation

- **Detail page**: click the image, press `d`, or click "↗ Detail" to open the full detail page for the current asset
- **Stroll**: press `s` to start strolling from the current asset

Info panel

Click the **i** button in the top bar (or press `i`) to toggle a side panel showing:

- **Metadata**: asset type, format, date, and variant count
- **Rating**: interactive stars — click to set a rating (1-5), click × to clear
- **Color label**: interactive color dots — click to set a label, click × to clear

Rating and label changes made in the lightbox are saved immediately via the API and reflected on the host page. The top bar also shows interactive rating stars and color label dots for quick editing without opening the info panel.

The info panel has its own × close button.

Rating and label

The lightbox top bar shows interactive rating stars and color label dots, always visible without opening the info panel. Click a star to set rating, click × to clear. Click a color dot to set a label, click the active dot again to clear.

These controls are also available in the info panel with more detail, but the top bar provides quick access during rapid browsing.

Smart preview and zoom

When a smart preview (high-resolution, 2560px) is available, the lightbox loads the regular preview instantly for display, then background-loads the smart preview. A pulsing **HD** badge in the top-right corner provides

visual feedback during loading. Once the smart preview loads, the badge briefly shows with solid opacity before fading out.

With a smart preview loaded, you can zoom and pan:

- **Mouse wheel**: zoom in/out centered on cursor position
- **Drag**: pan the image when zoomed in (cursor changes to grab/grabbing)
- **Keyboard**: `,` fit to screen, `.` 100% zoom, `+` zoom in, `-` zoom out

Smart previews can be generated via `maki import --smart`, `[import] smart_previews = true` in config, the "Regenerate previews" button on the detail page, or on-demand via `[preview] generate_on_demand = true`.

Keyboard shortcuts

All browse keyboard shortcuts for rating and label work inside the lightbox:

Key	Action
Left / Right arrow	Previous / next asset
Escape	Close lightbox
Click image	Open detail page
i	Toggle info panel
d	Open detail page
1-5	Set rating
0	Clear rating
Alt+1 through Alt+7	Set color label
Alt+0, x	Clear color label
r/o/y/g/b/p/u	Set label by color initial
, (comma)	Fit to screen (zoom)
. (period)	100% zoom
+	Zoom in *(or bump slideshow speed +1 s when slideshow is running)*
-	Zoom out *(or bump slideshow speed −1 s when slideshow is running)*
Space	Start / pause slideshow

Video playback: Video assets display inline in the lightbox. Click the video to play or pause it. Use the "Detail" button or press `d` to navigate to the detail page for full playback controls.

When the lightbox is open, browse keyboard navigation (arrow keys for card focus, Enter, Space) is suppressed — `Space` toggles the slideshow instead.

Slideshow

Two buttons in the top toolbar (next to Info / Detail / Close):

- ▶ / ▮▮ — start / pause. Same as pressing **Space**.
- ↻ / → — loop after last asset. ↻ (active) wraps to the first asset when the slideshow reaches the end; → stops at the last asset.

While the slideshow is running, + / – bump the cadence by one second (clamped to 1–60 s). The current value flashes briefly next to the play button. Initial cadence and loop state come from `[browse] slideshow_seconds` and `[browse] slideshow_loop` in `maki.toml` (defaults: 5 s, loop on).

The slideshow auto-pauses on any deliberate user action: manual prev/next click, rating change, label change, Escape. Closing the lightbox stops the slideshow entirely.

Asset Detail Page

Click the "↗ Detail" link in the lightbox top bar, or press `d` in the lightbox, to open the asset detail page.

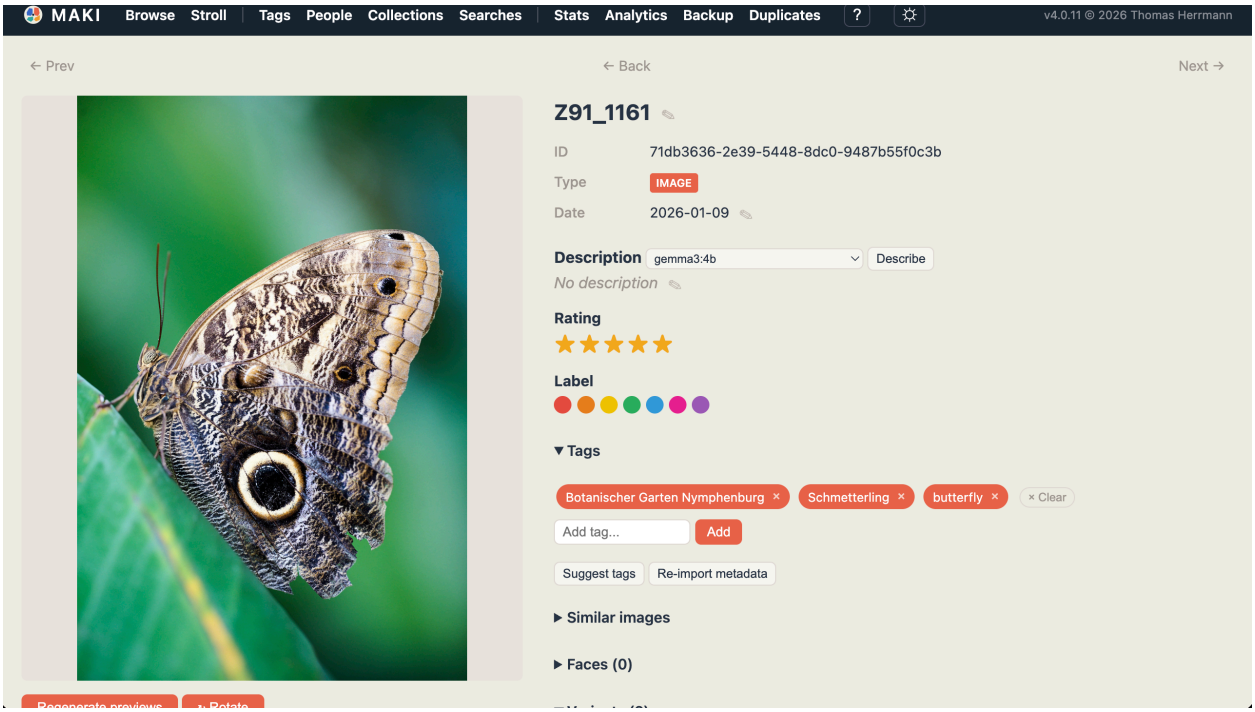


Figure 7: Asset detail page with preview, metadata editing, and variants

Navigation

A navigation bar at the top of the page provides Prev, Back, and Next buttons:

- **Prev / Next**: navigate to adjacent assets in the browse results. These buttons use the browse page's card list (stored in sessionStorage) for unlimited multi-hop navigation — you can press Next repeatedly to step through the entire result set. When accessed via a direct URL without browse context, these buttons are hidden.
- **Back**: returns to the browse page, preserving your search and filters.

Keyboard navigation:

Key	Action
Left arrow	Previous asset
Right arrow	Next asset
Click image	Open in lightbox

Key	Action
l	Open in lightbox
Escape	Return to browse page

Rating, label, and zoom keyboard shortcuts also work on the detail page:

Key	Action
1-5	Set rating
0	Clear rating
Alt+1 through Alt+7	Set color label
Alt+0, x	Clear color label
r/o/y/g/b/p/u	Set label by color initial
, (comma)	Fit to screen (zoom)
. (period)	100% zoom
+	Zoom in
-	Zoom out

Preview

The left side shows a large preview image (or, for video assets, an HTML5 video player with native browser controls for play, pause, seek, and volume). This is the best available preview for the asset, preferring export variants over processed variants over originals, and standard image formats over RAW.

When a smart preview is available, it loads in the background with a pulsing "HD" badge. Once loaded, the image supports zoom and pan via mouse wheel, drag, and keyboard shortcuts (`, . + -`). Clicking the preview image opens the asset in the lightbox for full-screen viewing. Buttons below the preview let you regenerate the regular preview, generate/regenerate the smart preview, or rotate the image 90° clockwise. Rotation cycles through 0° → 90° → 180° → 270° → 0° and is persisted per asset. Previews are automatically oriented using EXIF tags during generation; manual rotation is applied on top.

Editable metadata

The right side contains the asset's metadata, all editable inline:

Name – displayed as a heading. Click the pencil icon to switch to an inline text input with Save and Cancel buttons. Saving an empty name clears it, and the display falls back to the original filename in muted italic. The name is stored on the asset.

Description – click the pencil icon to switch to a textarea with Save and Cancel buttons. Saving an empty description clears it. Changes are written back to XMP recipe files on disk. In the Pro edition, when a VLM server is available, a "Describe" button appears next to the heading. Click it to run the configured vision-language model in the mode set by `[vlm] mode` in `maki.toml` (default `describe`) — the button shows "Generating…" while waiting for the VLM response, then updates the description in-place. With `mode = "tags"` the VLM's keyword suggestions are applied directly as tags with `vlm` provenance (they appear in the tag list after a reload); with `mode = "both"` you get the description and the tags from two VLM calls. When multiple VLM models are configured via `[vlm] models` in `maki.toml`, a model selector dropdown appears next to the Describe button, letting you choose which model to use per request (e.g., a fast model like Moondream for routine descriptions and a larger model like Qwen3-VL for difficult images).

Rating – five clickable stars. Click a star to set that rating. Click the same star again to clear the rating. Changes are written back to XMP recipe files on disk.

Color label – seven colored dots (Red, Orange, Yellow, Green, Blue, Pink, Purple) with a label name shown below the active dot. Click a dot to set that label; click the active dot again to clear it. Changes are written back to XMP recipe files on disk.

Tags – displayed as removable chips. Click the x on a chip to remove that tag. Use the text input below to add new tags – it offers autocomplete suggestions from your catalog's tag list as you type. Changes are written back to XMP recipe files on disk with operation-level deltas (tags added independently in CaptureOne or Lightroom are preserved).

Suggest tags *(Pro)* – a "Suggest tags" button appears below the tag input in the Pro edition. Click it to analyze the asset image with the SigLIP vision model. The button shows "Analyzing…" while the model processes (the first request may take a few seconds while the model loads). Results appear as suggestion chips, each showing the tag name and a confidence percentage. Tags already on the asset appear dimmed with an "already applied" label. Click ✓ to accept a new tag (it is added immediately), click × to dismiss it, or click "Accept new" to apply all unapplied suggestions at once.

Similar images *(Pro)* – a collapsible section with three buttons: **Embed** builds the asset's SigLIP embedding (no tags applied), **Find similar** lists the nearest assets inline with similarity percentages, **Browse similar** opens the browse grid as a `similar:<id>` search, and **Stack similar** proposes a stack from the nearest neighbours above a threshold.

Re-import metadata – a "Re-import metadata" button appears below the tag section. Click it to clear the asset's tags, description, rating, and color label, then re-extract metadata from variant source files (XMP recipe sidecars and embedded XMP in JPEG/TIFF files). A confirmation dialog asks before proceeding since the operation is destructive. This is useful for cleaning up metadata after splitting mis-grouped assets, where tags from multiple unrelated images may have been merged together. The page reloads after completion to reflect all changes. Offline volumes are silently skipped during re-extraction.

Asset information

Below the editable fields:

- **ID**: the asset's UUID
- **Type**: image, video, audio, or document
- **Date**: the asset's creation date (from EXIF or import time). Click the pencil icon to edit it inline with a date input and Save/Cancel buttons.

The badge row under the title shows the type, the identity format, and — for media — codec, resolution, and duration. Audio assets add their musical **key** and **BPM** (set by `maki audio analyze`) plus a technical badge (sample rate / channels / bitrate), and the preview area shows an HTML5 `<audio>` player above the waveform info card.

Collections

If the asset belongs to any collections, they appear as clickable chips. Click a chip to browse that collection. Click the x button on a chip to remove the asset from that collection.

Stack members

If the asset belongs to a stack, a section shows all members in position order with thumbnail previews. The pick is indicated. You can click any member to navigate to its detail page. Three buttons act on the stack from here: **Set as pick** makes this asset the stack's cover, **Remove from stack** takes it out (a stack left with one member auto-dissolves), and **Dissolve stack** removes the stack entirely, leaving every member as an independent asset.

Variants

An expandable section lists all variants of the asset in a table with columns for role, filename, format, size, and file locations (volume and path). This gives you a complete picture of where the asset's files live across your storage volumes.

For assets with multiple variants, two more controls appear. An **Extract as new asset(s)** button above the table splits the checked variants out into standalone assets (tags, rating, and label are copied) — the web equivalent of `maki split`. The last column holds a star per row: the filled star marks the variant whose preview represents the asset in grids and the lightbox; click the hollow star on another row to make it the **preview representative** instead (persisted per asset).

For assets with multiple variants, the **role column** is an inline dropdown. Click it to change a variant's role (original, alternate, processed, export, sidecar). Changes are saved immediately — a green flash confirms success. This is useful for correcting roles after import or `fix-roles` assigns an incorrect role (e.g. marking a re-exported JPEG as "alternate" when it should be "export").

For locations on online volumes, two action buttons appear next to each path: - **Reveal (file manager)** — opens the system file manager with the file selected (Finder on macOS, Explorer on Windows, file manager on Linux). - **>_ Terminal** — opens a terminal window in the file's parent directory (Terminal.app on macOS, cmd on Windows, system terminal emulator on Linux).

Recipes

An expandable section lists attached recipe files (XMP sidecars, CaptureOne settings, etc.) with columns for recipe type, software, and locations. Recipes are grouped by content: identical copies of the same XMP file on different volumes (e.g. from a backup) show as "1 recipe, 3 locations" rather than three separate entries — mirroring how variants display their locations. Different XMP content (e.g. pre/post edit) shows as separate recipe rows. The section header shows the unique recipe count and total location count when they differ.

Recipes with pending write-back changes (edits made while the volume was offline) show an orange sync indicator per location. When any recipe has pending changes, a "Write back to XMP" button appears at the top of the section — clicking it replays the queued edits to the recipe files on disk (equivalent to `maki writeback --asset`).

Source metadata

If the primary variant has EXIF or XMP metadata, an expandable section displays it as key-value pairs (camera model, lens, exposure settings, GPS coordinates, etc.).

Batch Operations

The web UI supports applying changes to multiple assets at once. Select assets in the browse grid and use the batch toolbar to tag, rate, label, group, or organize them into collections.

Selecting assets

Each browse card has a checkbox that appears on hover. Once any card is selected, all checkboxes become permanently visible (until the selection is cleared) so you can see what is selected at a glance. Selected cards receive a visible selection border.

- **Click a checkbox** to toggle selection of individual cards
- **"Select page"** button selects all cards on the current page
- **"Clear"** button deselects everything

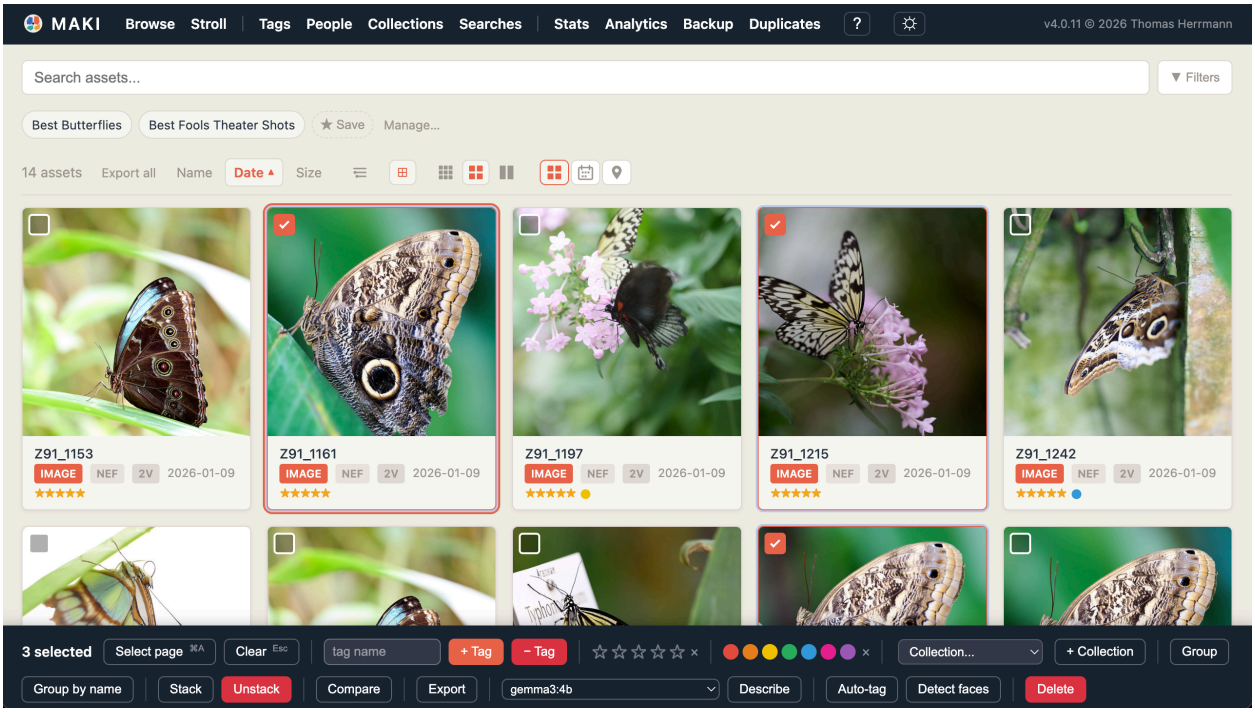


Figure 8: Batch toolbar with selection count, tag/rating/label controls

Batch toolbar

A fixed toolbar appears at the bottom of the screen whenever one or more assets are selected. It shows the selection count and provides these controls:

Tags: a text input with autocomplete, plus "+ Tag" and "- Tag" buttons. Type a partial tag name to see suggestions from your catalog's tag list; navigate with arrow keys, Enter or click to select a suggestion. Click "+ Tag" to add the tag to all selected assets, or "- Tag" to remove it. Press Enter in the input to add. Newly created tags are immediately available in the autocomplete.

Rating: five clickable stars and a clear button (x). Click a star to set that rating on all selected assets. Click x to clear rating.

Color label: seven colored dots and a clear button (x). Click a dot to set that label on all selected assets. Click x to clear label.

Collection: a dropdown listing your collections (plus a "New…" option to create one inline). The buttons next to it are context-sensitive: - When you are **not** browsing a collection, a "+ Collection" button adds the selected assets to the chosen collection. - When you **are** browsing a collection (the collection filter is active), a "- Collection" button removes the selected assets from that collection. The dropdown auto-selects the current collection.

Stack: creates a stack from the selected assets. The first selected asset becomes the pick. This is a lightweight, reversible grouping – assets remain independent. When the selection already touches an existing stack, the button gives way to three context-sensitive ones: **+ Stack** adds the unstacked selection to that stack, **− Stack** removes the selected members from it, and **Set pick** makes the single selected member the stack's pick.

Unstack: removes the selected assets from their stacks. If a stack has one or fewer members after removal, it auto-dissolves.

Group: merges the selected assets into one asset with multiple variants, exactly as selected — no filename

matching. A confirmation dialog explains the action. This cannot be undone.

Group by name: merges the selected assets by filename stem. A confirmation dialog explains the action. Assets whose filenames share a common prefix (e.g., `DSC_001.nef` and `DSC_001.jpg`) are merged into a single asset with multiple variants. This cannot be undone.

Compare: opens a side-by-side compare view with the selected assets (2–4). See Compare View below.

Describe *(Pro)*: sends each selected asset's preview image to the configured VLM server in the mode set by `[vlm] mode` (default `describe`; `tags` applies VLM tag suggestions, `both` does description and tags). Assets that already have a description are skipped in `describe` and `both` mode. A confirmation dialog shows the count and warns about per-asset timing. After processing, a summary reports how many descriptions were set. This button only appears in the Pro edition when a VLM server is reachable at startup (see VLM Setup).

Auto-tag *(Pro)*: analyzes each selected asset with the SigLIP vision model and applies suggested tags above the configured confidence threshold. A confirmation dialog shows the count of selected assets. After processing, a summary reports how many tags were applied. This is the one-shot "trust the model" path — every suggestion above threshold is written immediately.

Review tags... *(Pro)*: same model pass as Auto-tag, but instead of writing immediately, opens a two-pane review modal so you can prune the AI's picks before any tag is committed. Operates on the current selection if non-empty, otherwise on every asset matching the live filter. The modal opens immediately in a "Computing…" phase that streams progress while the model loads and scores assets (the first request can take 30–60 s while the model warms up — the progress bar reassures you nothing is hung). When the pass completes, the modal flips to a two-pane review:

- **Left pane**: every tag the model proposed, sorted by candidate count, with a mean-confidence bar per row. Click a tag to inspect its candidates in the right pane. Three states per tag — pending (default), applied (after you commit it, strikethrough on the row), or skipped (dismissed without applying).
- **Right pane**: thumbnail grid of every asset the selected tag was proposed for, pre-checked with a confidence overlay. A `from: <label> →` caption appears when the AI vocabulary mapping is non-identity (e.g. SigLIP scored `skyscraper` but the vocabulary maps that to `subject|urban|architecture|skyscraper`). A confidence slider auto-toggles cells above/below the threshold; a "Skip already-tagged assets" filter hides assets that already carry the tag.

The collection pass runs at half the configured `[ai].threshold` so the slider has range both above and below your normal floor. Commit applies the checked subset via the same batch tag endpoint as the toolbar's regular tag operations — no destructive cleanup, no removals, just additive writes. This is the interception point for cleanup-after-auto-tag workflows: where Auto-tag commits to disk immediately, Review tags lets you reason about each AI choice in context with thumbnails.

Embed *(Pro)*: generates the SigLIP visual-similarity embedding for each selected asset without applying any tags — coverage for `similar:`, "Find by image", auto-stack, and the stroll page. Runs as a background job with the progress toast.

Detect faces *(Pro)*: runs face detection on each selected asset's preview image using YuNet and ArcFace. Detected faces are stored with bounding boxes, embeddings, and crop thumbnails. A summary reports how many faces were found.

Model dropdown *(Pro)*: when `[vlm] models` lists more than one model, a small dropdown next to Describe picks the VLM for that request.

Delete: removes the selected assets (and their sidecars) from the catalog after a confirmation dialog that previews the selection. An "Also delete media files from disk" checkbox in the dialog moves the files into the catalog trash as well — the web equivalent of `maki delete --apply [--remove-files]`.

Export: downloads the selected assets as a ZIP archive. A modal dialog lets you choose layout (flat — all files in root with hash-suffix collision resolution, or mirror — preserves original directory structure), the content source (**Originals**, **Previews**, or **Smart previews** — the preview sources come from the catalog

directory, so they work even when the originals' volumes are offline; picking one disables the all-variants toggle since previews are keyed to the best variant), whether to include all variants or just the best, and whether to include sidecar/recipe files. The ZIP is streamed to the browser as a download. If any files could not be included — originals on offline volumes, previews not yet generated — a warning reports how many were skipped and why; the ZIP still contains everything that was available.

Export all results

The results bar includes an "Export all" link that exports all assets matching the current search and filter state — not just the current page. Clicking it opens the same export modal as the batch Export button. The full browse state (query, type, tag, format, volume, rating, label, collection, path, person, stack collapse, default-filter override) is sent to the server, which resolves the matching assets through the exact same pipeline as the browse grid — so the archive contains precisely the assets the grid shows. With stacks collapsed, only the stack covers are exported; switch the view to expanded stacks first if you want every member.

After every batch operation, the selection clears and the results grid refreshes to reflect the changes. All toolbar buttons are disabled during the operation to prevent double submissions.

Drag-and-drop

The web UI supports drag-and-drop for two common operations:

Adding to collections: Drag one or more browse cards onto the collection dropdown in the batch toolbar. If cards are selected, dragging any selected card moves the entire selection. A toast notification confirms how many assets were added. The collection dropdown highlights as a valid drop target during the drag.

Reordering stack members: On the asset detail page, stack members can be dragged to reorder. Dropping a member into the first position sets it as the stack pick.

Keyboard shortcuts for selection

Key	Action
Cmd+A (Mac) / Ctrl+A	Select all cards on the current page
Shift+Cmd+A (Mac) / Shift+Ctrl+A	Select **all assets matching the current filter**, across every page
Escape	Clear selection (if any), otherwise clear keyboard focus

These shortcuts are suppressed when focus is in a text input, textarea, or dropdown.

Shift+Cmd+A is the answer to "I forgot to embed / auto-tag / add a tag during import — how do I fix all 500 photos at once?" Filter the browse page down to the set you want (`path:concert-2024`, `tag:family`, etc.), press the shortcut, and confirm. The full set lands in the selection and any batch toolbar action runs across every page in one shot.

When the result spans more than one page, a confirmation dialog appears showing the asset count and how many pages it covers. If you already have an existing selection, the dialog offers two radios:

- **Replace current selection** (default) — the selected set becomes exactly the matching assets.
- **Add to current selection** — union with what was already selected.

Single-page filters skip the dialog and merge silently. After a multi-page select-all the bottom toolbar reads `487 selected (across 9 pages)` so the wider scope is always visible. Manually toggling a checkbox afterward drops the "(across N pages)" hint, since the count would no longer reliably correspond to a known scope.

The selection persists across page navigation, view-mode switches (grid → calendar etc.), and page reloads (until the tab is closed) — so you can browse to verify what you've grabbed before running the batch operation.

Keyboard Navigation

The browse page supports full keyboard navigation for efficient photo culling and rating workflows. No mouse required.

Keyboard help

Press `?` on any page (or click the "?" button in the navigation bar) to open a keyboard shortcuts overlay. The overlay shows all shortcuts available on the current page, organized by category. Press Escape, click ×, or click outside the panel to close it.

Movement

Key	Action
Arrow Left	Move focus to the previous card
Arrow Right	Move focus to the next card
Arrow Up	Move focus up one row (column-aware)
Arrow Down	Move focus down one row (column-aware)

The focused card has a blue outline, visually distinct from the selection highlight. If no card is focused, the first arrow key press focuses the first card.

Actions on the focused card

Key	Action
Shift+Left	Previous page
Shift+Right	Next page
Enter, l	Open the focused card in the lightbox
d	Open the focused card's detail page
Space	Toggle selection of the focused card
c	Open the compare view for the selected cards (2–4)
1-5	Set rating (applies to focused card, or to all selected if a batch selection is active)
0	Clear rating
Alt+1 through Alt+7	Set color label (1=Red, 2=Orange, 3=Yellow, 4=Green, 5=Blue, 6=Pink, 7=Purple). On macOS, use Option+number.
Alt+0	Clear color label
r	Set Red label
o	Set Orange label

Key	Action
y	Set Yellow label
g	Set Green label
b	Set Blue label
p	Set Pink label
u	Set Purple label
x	Clear label
Shift+F	Toggle filter bar visibility
f	Toggle the facet sidebar
m	Toggle the map view
s	Open the stroll page for the focused card (visual similarity exploration)

Single-letter label shortcuts and number keys for rating operate on the focused card when no batch selection is active. When assets are selected (selection count > 0), rating keys apply to the entire batch.

Focus persistence

Focus position is preserved across pagination and sort changes. If you are focused on card 5 and sort by name, card 5 (by position) remains focused after the grid reloads.

Input suppression

All keyboard shortcuts are suppressed when focus is in a text input, textarea, or select dropdown. This prevents accidental rating or label changes while typing a search query or tag name.

Compare View

Select 2–4 assets in the browse grid and click the "Compare" button in the batch toolbar to open a side-by-side comparison view.

Layout

Each asset appears in its own column with a preview image on top and metadata below. The metadata section shows the display name (clickable link to detail page), creation date, interactive rating stars, color label dots, and EXIF data (camera, lens, focal length, aperture, shutter speed, ISO).

Zoom and pan

All columns support zoom and pan:

- **Mouse wheel**: zoom in/out centered on cursor position (0.5× to 20×)
- **Drag**: pan the image when zoomed
- **Double-click**: reset zoom to 1×

Sync zoom: when enabled (on by default), zooming or panning one column applies the same transform to all columns. This lets you inspect the same area across all images simultaneously. Toggle sync with the checkbox in the top bar or press `s`.

Smart previews are loaded automatically when available, with a pulsing HD badge during loading.

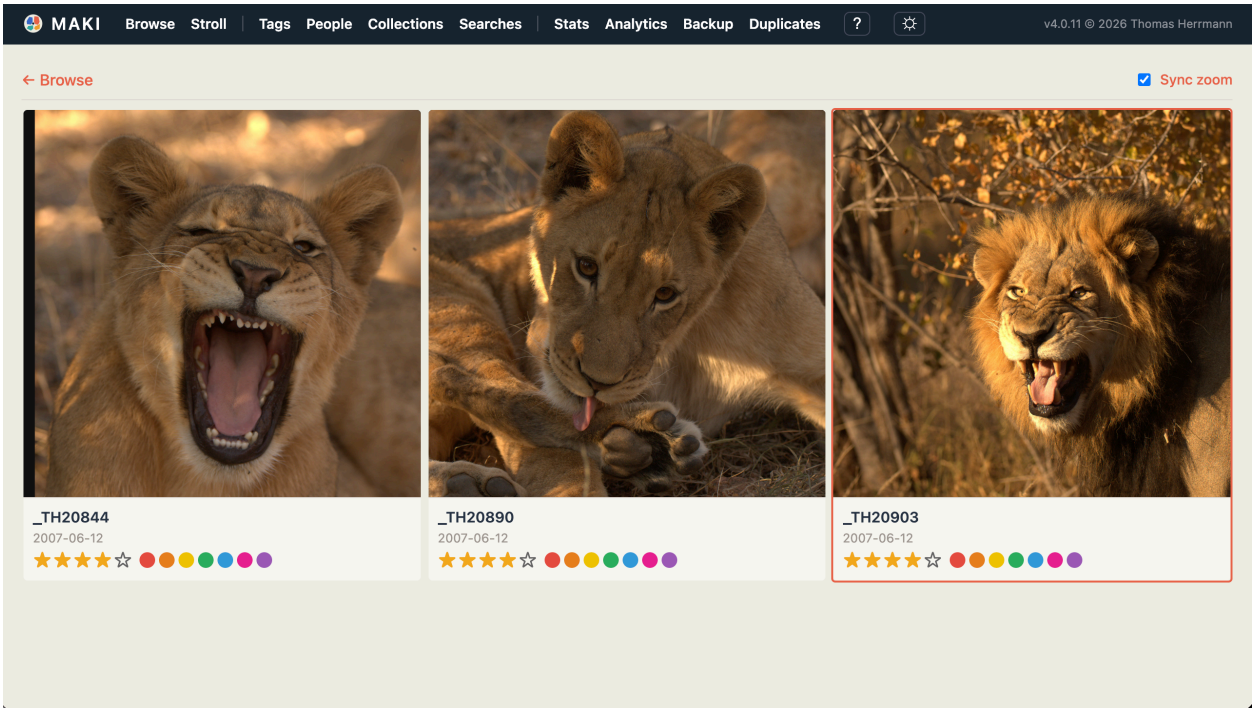


Figure 9: Compare view with side-by-side asset comparison

Keyboard shortcuts

Key	Action
Left / Right arrow	Move focus between columns
d	Open detail page for focused asset
s	Toggle sync zoom
Escape	Return to browse
1-5	Set rating on focused asset
0	Clear rating
Alt+1 through Alt+7	Set color label
Alt+0, x	Clear color label
r/o/y/g/b/p/u	Set label by color initial
, (comma)	Fit to view (zoom)
. (period)	Toggle 3× zoom
+	Zoom in
-	Zoom out

The focused column has a blue border. Rating and label changes are saved immediately.

Tags Page

The tags page shows every tag in your catalog with usage counts and inline edit actions.

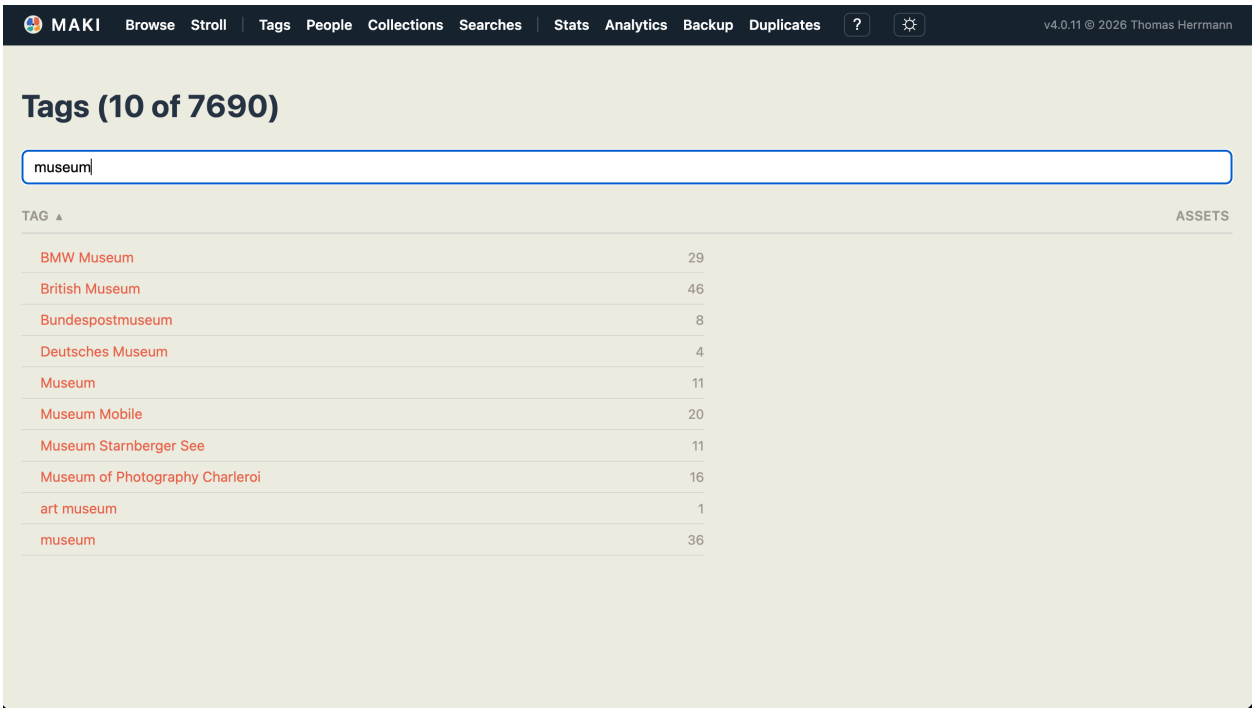


Figure 10: Tags page with sortable columns and text filter

Navigate to `/tags` or click "Tags" in the navigation bar.

Hierarchical tree

Tags containing `|` separators render as a collapsible tree. The tree is built in **pre-order**: a parent tag is always immediately followed by its descendants, so structurally-related rows stay grouped — even when a renamed pipe-child shares a prefix with flat siblings (e.g. `Bricking Bavaria` and `Bricking Bavaria 2012`). Click the disclosure triangle to expand or collapse a branch. Click any tag name to jump to the browse page filtered by that tag.

Counts: own vs leaf

Each tag row shows one or two numbers:

- **Headline (bold)** = assets that carry this tag in any form. Because MAKI auto-expands every hierarchical tag to its ancestor paths on storage, this includes assets that have any descendant on the same chain. Matches the default `tag:foo` browse semantic — same number you'd see in the result count after clicking the tag.
- **`(N as leaf)`** = assets where this tag is the *deepest* level on the asset (no descendant of it is also present). Matches `tag:/foo` (the leaf-only chip mode). Only shown when it's between 0 and the headline count exclusive — i.e. when some assets are tagged at exactly this level when they could plausibly be more specific. Properly-tagged parents (every photo specialised down to a deeper child) show no parens; true leaves (no children anywhere) show a single number.

The `(N as leaf)` text is **clickable** — it lands on the browse page with `tag:/<name>` applied, showing exactly those N candidate-for-finer-tagging assets. Useful for cleaning up legacy data: a `subject (3 as leaf)` row tells you three photos are tagged just `subject` with no specialisation; click to find and re-tag them.

Sort, filter, layout

- **Sortable columns**: click the *Tag* header to sort alphabetically, or the *Assets* header to sort by the headline count. Click again to reverse direction. The active sort shows a direction arrow.
- **Live text filter**: type in the filter input to narrow the tag list. Filtering begins at 2 characters. The count display updates to show "X of Y" tags. A leading `|` anchors to a hierarchy component start (e.g. `|wed` matches `wedding` and `events|wedding` but not `midweek`).
- **Multi-column layout**: tags flow into multiple columns automatically, adapting to the viewport width.

Export vocabulary

The **Export vocabulary...** button in the page header downloads the whole tag vocabulary through `GET /api/tags/export-vocabulary`. A small modal picks the format — YAML (the `[ai] labels` vocabulary format, hierarchy preserved), plain keyword text (one tag per line, importable into Lightroom / Capture One), or JSON — and two options: include asset counts, and prune entries that exist in the vocabulary file but are unused in the catalog. The CLI equivalent is `maki tag export-vocabulary`.

Inline edit actions

Hover over any tag row to reveal three icons that open Preview/Apply modals — same dry-run-by-default safety pattern across all three. Each modal supports the **Enter twice** rhythm: first Enter previews (gives a count), unlocks Apply; second Enter commits.

- **Rename (pencil icon)**: opens a Rename dialog with the tag pre-filled in *From* and an editable *To* input that has tag autocomplete from your existing vocabulary. Web equivalent of `maki tag rename`. The autocomplete and submit-on-Enter wiring is shared with the split modal via a single helper.
- **Split (branching arrow)**: shown only on leaf tags. Replaces the tag with two or more new tags across all matching assets, optionally keeping the source alongside (`Keep source` checkbox). Each target row has its own autocomplete-enabled input; Enter on a non-last row advances to the next, Enter on the last row submits. Web equivalent of `maki tag split`.
- **Delete (×)**: removes the tag — and, by default, the entire branch under it — from every asset that has it. Hover tint shifts to the destructive accent so the action reads clearly. The modal cascades by default (warns when the tag has children); clicking Preview shows how many assets would be affected, and a second click commits. Newly-orphaned ancestors on each asset are cleaned up automatically. Web equivalent of `maki tag delete`.

Collections Page

The collections page lists all your static collections (manually curated albums).

Navigate to `/collections` or click "Collections" in the navigation bar.

Features

- **Collection cards**: each collection is shown as a card with its name, asset count, and description (if set). Click a card to browse its assets on the main browse page.
- **"+ New Collection" button**: prompts for a name and optional description, then creates the collection immediately. The page reloads to show the new card.

Collections created here are the same as those created via the CLI `maki collection create` command. See Organizing Assets for full details on managing collections.

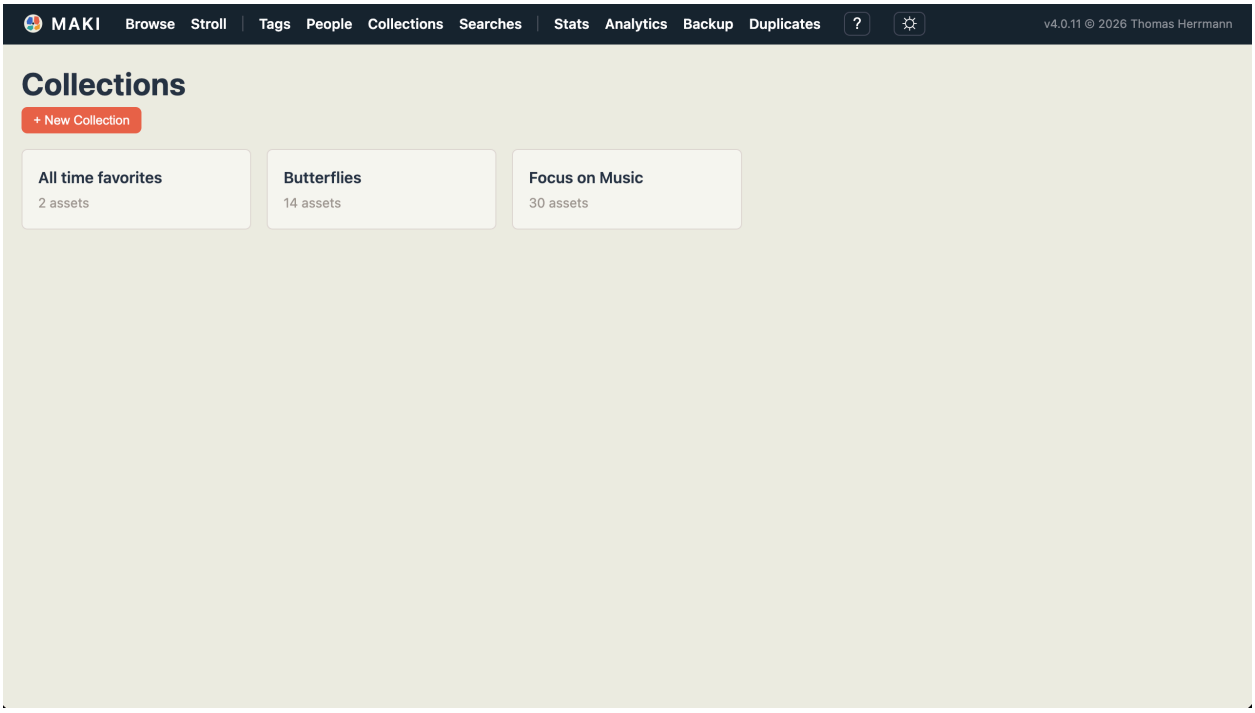


Figure 11: Collections page with collection cards and create button

Saved Searches Page

The saved searches page lists every named search (smart album) in the catalog, whether it was saved from the browse page or with `maki saved-search save`.

Navigate to `/saved-searches` or click "Searches" in the navigation bar.

Features

- **Table of searches**: one row per saved search with its name, the stored query string, and the sort order (`date_desc` when none was stored). If nothing has been saved yet, the page says so and points to the Save button on the browse page.
- **Run**: click the name to open the browse page with the stored query, filters and sort applied.
- **Favorite star**: the star in the first column toggles whether the search appears as a chip on the browse page. Filled means "shown on browse page"; hollow means the search is kept but hidden from the chip row.
- **Rename**: prompts for a new name. Renaming to a name that already exists is rejected.
- **Delete**: asks for confirmation, then removes the search.

Every action writes straight to `searches.toml` and reloads the page, so the CLI (`maki saved-search list`) sees the same state. All four actions are unavailable in `--read-only` mode.

Stats Page

Navigate to `/stats` or click "Stats" in the navigation bar to see a visual overview of your catalog.

The stats page displays:

- **Overview cards**: total assets, variants, recipes, online/total volumes, and total storage size
- **Asset types**: bar chart showing the distribution of images, videos, audio files, and documents

- **Variant and recipe formats**: bar charts showing format breakdowns (NEF, ARW, JPEG, XMP, etc.)
- **Volumes**: table with per-volume details including label, online/offline status, asset count, variant count, recipe count, total size, formats present, and verification coverage
- **Tags**: summary of unique tags, tagged assets, and untagged assets, plus a weighted tag cloud of the most-used tags
- **Verification health**: overall coverage bar, plus a per-volume breakdown of verification status

This is the web equivalent of `maki stats --all` on the command line. See Browsing & Searching for the CLI stats command.

People Page *(Pro)*

Navigate to `/people` or click "People" in the navigation bar to manage detected faces and named people.

The people page displays:

- **Person cards**: each person is shown as a card with a representative face crop thumbnail, name (or "Unknown" for unnamed clusters), and the number of detected faces. Click a card to browse assets containing that person. A name filter above the grid narrows the cards as you type.
- **Rename**: click the pencil button on a person card and enter the new name.
- **Merge**: tick the checkbox on two or more cards. A selection toolbar appears with a **Merge...** button; click the bullseye badge on one of the selected cards to make it the merge target (the others are merged into it), then confirm in the modal, which previews the people being merged. A **merge suggestions** panel lists pairs of clusters whose face embeddings are close enough to be the same person, each with a one-click merge.
- **Delete**: click the × button on a person card to remove the person (faces become unassigned).
- **Cluster button**: runs face auto-clustering from the UI, grouping unassigned faces into new person groups.

Faces are detected via `maki faces detect` on the CLI, the "Detect faces" button on the asset detail page, or the batch "Detect faces" button on the browse toolbar. After detection, use clustering (CLI or web UI) to group faces into people, then name them.

Asset detail faces section

On the asset detail page, a "Faces" section appears when faces have been detected for the asset:

- **Face chips**: each detected face is shown as a small chip with the crop thumbnail and confidence score.
- **Detect faces button**: triggers on-demand face detection for the asset.
- **Person assignment**: a dropdown on each face chip lets you assign or unassign the face to a person.

Browse face filters

- **Person dropdown**: in the browse filter row, a person dropdown (when people exist) lets you filter to assets containing a specific person.
- **Face count badge**: browse cards show a face count badge next to the variant count badge.
- **Query filters**: `faces:any`, `faces:none`, `faces:N`, `faces:N+`, `person:<name>` work in the query input.

Stroll Page *(Pro)*

The stroll page provides a visual similarity exploration experience. Instead of browsing a flat grid, you start with a center image and see its most visually similar neighbors arranged in a radial layout around it. Click any neighbor to make it the new center, and the neighbors update – letting you "stroll" through your collection by visual similarity.

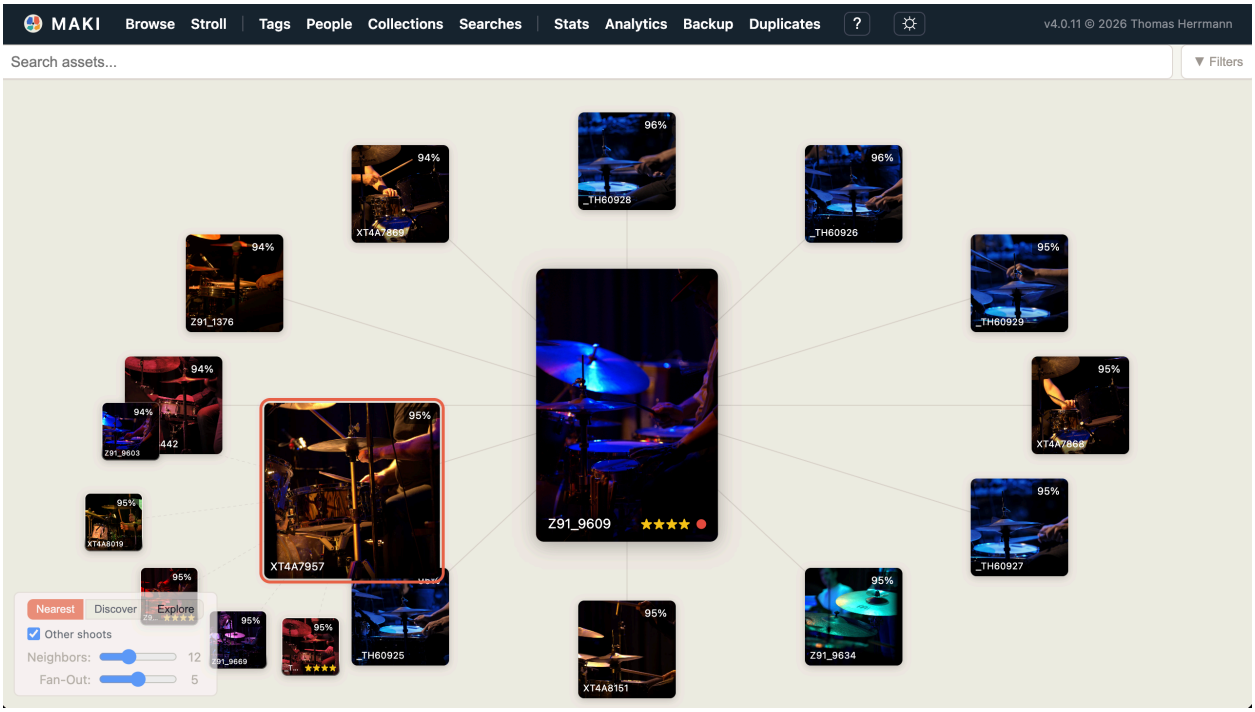


Figure 12: Stroll page with center image and similar neighbors

Navigate to `/stroll` or click "Stroll" in the navigation bar. You can also press `s` on the browse page (when a card is focused) or click the "Stroll" button on the asset detail page to start strolling from a specific asset.

Layout

The center image is displayed prominently in the middle of the page. Surrounding it, satellite thumbnails show the most similar assets based on SigLIP embedding similarity (dot-product distance), arranged in an elliptical layout that adapts to the viewport aspect ratio for optimal use of screen space. Each neighbor shows a preview thumbnail, and clicking it navigates to that asset as the new center.

Stroll modes

Three mode buttons in the control panel determine how neighbors are selected:

- **Nearest** (default): shows the top N most similar assets by embedding distance. Results are deterministic – the same center always produces the same neighbors.
- **Discover**: picks N random assets from a wider candidate pool (configurable via `[serve] stroll_discover_pool` in `maki.toml`, default 80). Each visit produces a different set of neighbors, encouraging serendipitous exploration. Useful for breaking out of tight visual clusters.
- **Explore**: skips past the K nearest neighbors to find more distant visual connections. A skip slider (0–200) appears in Explore mode, letting you control how far to reach. Higher skip values surface increasingly surprising matches.

"Other shoots" filter

A checkbox toggle labeled "Other shoots" excludes assets from the same directory or session as the center image. When enabled, only similar images from different shoots are shown as neighbors. This is useful for finding visual connections across your collection rather than seeing near-duplicates from the same session.

Neighbor count

A slider control (5–25, default 12, configurable via `[serve] stroll_neighbors` and `stroll_neighbors_max` in `maki.toml`) lets you adjust how many neighbor thumbnails are shown around the center image. Fewer neighbors give a cleaner view; more neighbors let you see a wider range of similar assets.

Filter bar

The stroll page includes the same collapsible filter bar as the browse page. When filters are active, only assets matching the filters are considered as neighbors. Press **Shift+F** to toggle the filter bar visibility.

Fan-Out slider (transitive neighbors)

A fan-out slider in the bottom-left corner (range 0–10, configurable via `[serve] stroll_fanout` and `stroll_fanout_max` in `maki.toml`) controls level-2 neighbor exploration. When fan-out is greater than 0 and you hover over or arrow-key to a satellite thumbnail, smaller thumbnails fan out from it showing that satellite's own nearest visual neighbors (excluding assets already visible on the page). The L2 thumbnails are arranged in a direction-dependent arc – the fan-out radius adapts based on each satellite's position, and satellites with L2 neighbors are pulled slightly toward the center to keep the layout balanced. L2 neighbor thumbnails show name, rating, and color label consistently with L1 satellites. Click any level-2 thumbnail to navigate to it as the new center. Results are cached per satellite, so moving focus back to a previously explored neighbor re-displays its fan instantly without another query.

Keyboard shortcuts

Key	Action
Arrow Left / Right	Navigate focus between neighbor thumbnails at the current level
Arrow Up	Focus the center image; from an L2 thumbnail, return to its parent L1 satellite
Arrow Down	Focus the first satellite; from an L1 satellite, drop into its L2 fan-out thumbnails (when fan-out > 0)
Enter	Make the focused neighbor the new center (or open the detail page when the center is focused)
d	Open the detail page for the focused asset
l	Open the focused asset in the lightbox
Shift+F	Toggle filter bar visibility
Shift+B	Back to the browse grid
Escape	Go back (browser history)
?	Open keyboard help

Duplicates Page

Navigate to `/duplicates` (under the Catalog ▾ menu) to review files that exist more than once. Summary cards show the number of duplicate groups, the wasted space, and how many groups are same-volume. Three tabs switch the mode: **All**, **Same Volume** (unwanted copies — the ones `maki dedup` targets), and **Cross Volume** (wanted backups). A filter row narrows the list by path prefix, format, and volume.

Each group lists its file locations with a per-location **Remove** button that deletes that copy (into the catalog trash, like the CLI). The **Auto-resolve** button removes every same-volume duplicate in the current filter

using the `maki dedup` heuristic (`[dedup] prefer` decides which copy survives). Click a group's thumbnail to open a lightbox: Left/Right arrows step through the groups, `d` opens the asset's detail page, Escape closes. This is the web equivalent of `maki duplicates` plus `maki dedup`; the endpoints are `POST /api/dedup/resolve` and `DELETE /api/dedup/location`. In read-only mode the page is view-only.

Analytics Page

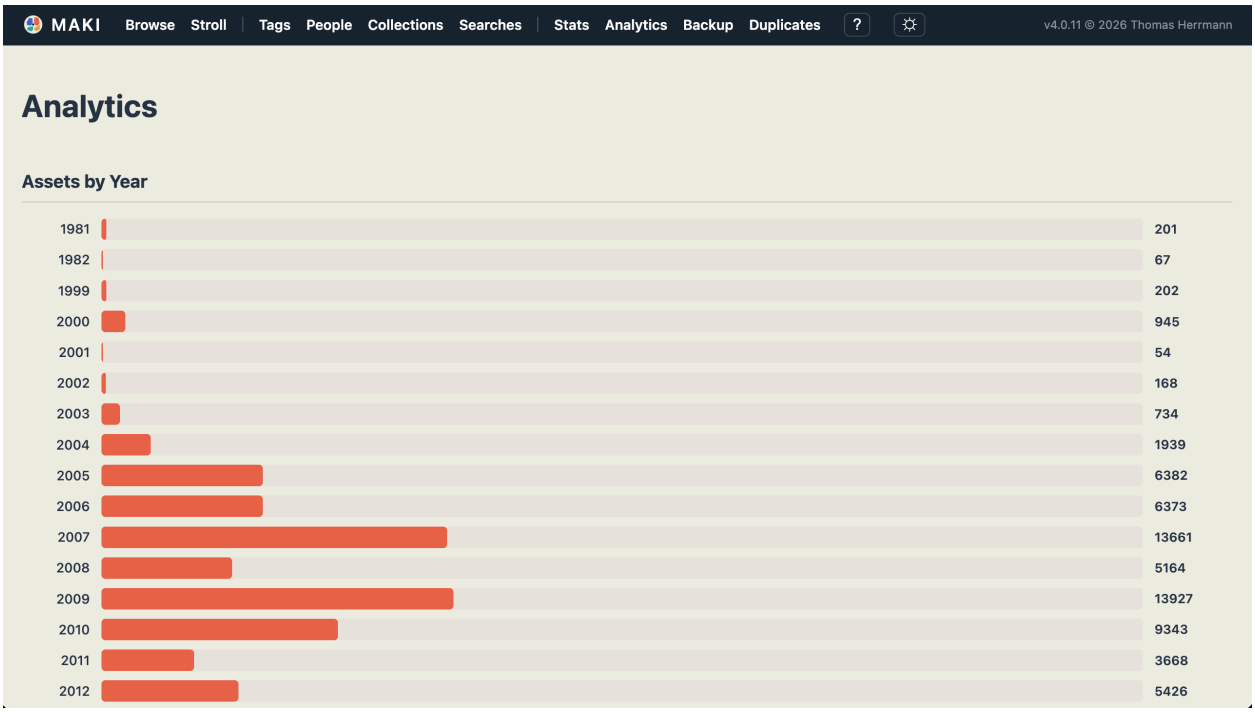


Figure 13: Analytics dashboard with charts and statistics

Navigate to `/analytics` or click "Analytics" in the navigation bar to see shooting and storage analytics.

The analytics page displays:

- **Yearly shooting counts**: bar chart of assets imported per year
- **Monthly sparkline**: monthly import volume for the most recent years
- **Camera usage**: top camera bodies by asset count
- **Lens usage**: top lenses by asset count
- **Rating distribution**: vertical bar chart showing how many assets have each star rating (1–5) plus unrated
- **Format distribution**: bar chart of file formats across the catalog
- **Storage by volume**: bar chart showing total size per volume

All charts auto-scale to their data. This complements the Stats Page with a more visual, photography-oriented perspective.

Backup Status Page

Navigate to `/backup` or click "Backup" in the navigation bar to see backup health at a glance.

The backup status page displays:

- **Summary cards**: total assets, at-risk count (highlighted in red when > 0), and minimum copies threshold (default 2)
- **At-risk link**: when at-risk assets exist, a prominent link navigates to the browse page filtered to `copies:1` for immediate review
- **Volume distribution**: horizontal bar chart showing how many assets exist on 0, 1, 2, or 3+ volumes, with red/amber/green coloring and "AT RISK" badges on under-backed-up buckets
- **Coverage by purpose**: table showing each volume purpose (working, archive, backup) with the number of volumes, asset count, and a coverage bar
- **Volume gaps**: table listing volumes with missing assets, showing the volume label, purpose, and missing count

This is the web equivalent of `maki backup-status` on the command line. See Maintenance for the CLI backup-status command.

Volumes Page

Navigate to `/volumes` or click "Volumes" in the navigation bar to manage registered volumes from the browser. This page is the web equivalent of `maki volume add/list/rename/set-purpose/remove` and is designed for users who don't want to drop into the CLI for routine volume management.

The page shows a table of all registered volumes with:

- **Label** and **mount point**
- **Purpose** badge (media, working, archive, backup, cloud, or none)
- **Status** badge (online/offline) — computed live from whether the mount point currently exists
- **Inline actions** per row: rename, set purpose, remove (with confirmation), and **Import** (only on online volumes)

A **Register Volume** button at the top opens an inline form: enter the path, the label is auto-derived from the directory name (editable), pick an optional purpose, and submit. This is the web equivalent of `maki volume add`.

Importing from a volume

The import dialog is reachable from two places:

- **Global "Import" entry** in the top navigation (every page). Opens the dialog with a volume-picker step first — pick which mounted volume to import from, then proceed to the form.
- **Per-volume "Import" button** on `/volumes`. Skips the picker and goes straight to the form for that volume.

Both routes share the same dialog. The form lets you:

- Specify an optional **subfolder** within the volume (e.g. `DCIM/100CANON` for a memory card). The field has **shell-style path autocomplete**: type to see directory entries under the volume's mount point, `↑`/`↓` to navigate, `Tab` or `Enter` on a directory drills in for the next level, on a file commits the value. `Esc` dismisses the dropdown. Server-side path resolution is clamped to the volume mount, so symlinks or `..` segments that would escape the mount are rejected.
- Choose an **import profile** from `[import.profiles.*]` in `maki.toml` (or default).
- Add **tags** to apply to every imported asset, via a chip picker matching the filter bar's tag UX. Type to autocomplete from existing tags, `Enter` or `,` adds a chip, `Backspace` on the empty input removes the last chip.
- Toggle **auto-group variants** and **smart previews**, plus — in the Pro edition — **Generate embeddings (visual similarity)** and **Generate descriptions (VLM)**, which run the same post-import phases as `maki import --embed` / `--describe`. Their initial state follows `[import] smart_previews` / `embeddings` / `descriptions` in `maki.toml`.
- Run a **dry run** first to see how many files would be imported.

- Click **Import** to start the actual import.

Dry-run and live import share the same flow: the dialog switches to a **live progress view** with a progress bar, running counts of imported/skipped/locations/recipes, and a scrolling log of the most recent files. Progress is streamed via Server-Sent Events, so you can keep the page open and watch in real time. Dry-run shows a "Dry run…" status line instead of "Importing…" but otherwise behaves identically — same SSE stream, same re-attach behaviour, same minimize-to-toast option.

A **pulsing red dot** next to the "Import" entry in the navigation indicates that a job is running anywhere in the catalog. Click the entry while a job is in flight and the dialog **re-attaches to the running job**: it replays the last ~100 events and continues with the live stream, so a page reload during a long import doesn't lose the activity feed. This works for both live imports and dry-runs. Only one import (live or dry) runs at a time, server-side.

When a live import finishes, the dialog shows a summary and a **Browse imported** link that scopes the browse view to the newly imported assets — exact `id:` filter for small batches (up to 80 assets), or the volume + subfolder filter sorted by date for larger ones.

When a dry run finishes, the dialog shows the same summary in "Would import N" form (the "Browse imported" link is hidden since nothing was actually written) plus a **Run for real** button that re-launches with the exact same form parameters as a live import — so you can review the dry-run counts, decide it's what you wanted, and commit in one click. This is the web equivalent of `maki import` for the most common workflow: plug in a card, register it (if needed), dry-run to sanity-check the count, then import.

Maintain Dialog

The **Maintain** entry in the top navigation (next to Import) opens a tabbed modal that exposes the long-running maintenance commands to the web UI. Seven tabs covering the most common workflows:

- **Writeback** — flushes pending metadata edits to `.xmp` recipe files on disk. Web equivalent of `maki writeback`. Always works regardless of `[writeback] enabled` (the config flag governs only *automatic* flush on every edit; this dialog is the explicit manual flush). The form exposes:
  - **Volume** dropdown — restrict to a single volume's recipes, or "All online volumes" to flush everywhere.
  - **Query** — optional search filter to scope to a subset (same syntax as the search bar; e.g. `tag:wedding`).
  - `Write all XMP recipes in scope (not just pending)` — the `--all` knob. Useful for rematerialising catalog metadata after large catalog-only restructuring (rename/split/delete tags) when the recipes weren't flagged pending.
  - `Mirror tags — remove XMP keywords the catalog no longer has` — the `--mirror-tags` knob. Reads each XMP's `dc:subject` and `lr:hierarchicalSubject`, diffs against the asset's catalog tags, and removes stale entries. Greyed out unless "Write all" is checked (mirror mode requires the broader sweep). **Destructive**: any keyword that's in the XMP but not in the catalog is dropped.
  - `Force — re-write every recipe in the explicit scope regardless of pending state` — the `--force` knob. More aggressive than mirror-tags: rebuilds both `dc:subject` and `lr:hierarchicalSubject` blocks from catalog state, discarding any pre-existing XMP entry. Use after a catalog import that may have stale pending flags, or to clean up runaway entity-escape damage (`Bobby &…`-style leftovers) or leaf-only stale entries (`Bavaria`, `Konzert`) that mirror-tags didn't catch. Combinable with `--mirror-tags` is redundant — `--force` is the stricter superset.
  - `Dry run — preview without writing` — reports counts without modifying files.
- **Sync metadata** — bidirectional XMP ↔ catalog sync. Web equivalent of `maki sync-metadata`. Inbound: external XMP edits (made in Lightroom / Capture One while the volume was outside MAKI) are pulled into the catalog. Outbound: pending catalog edits are flushed back to XMP for recipes that didn't change externally. Recipes that changed both on disk and in MAKI are reported as conflicts and skipped. Form options:
  - **Volume** dropdown.

  - `Re-extract embedded XMP from JPEG/TIFF media files` — the `--media` knob. Slow; use when you suspect external tools wrote XMP-in-JPEG that wasn't reflected in sidecars.
  - `Dry run`.
- **Verify** — content-hash check for media + recipes on disk. Web equivalent of `maki verify`. Re-hashes each file, compares against the stored content hash, reports mismatches (silent corruption, accidental edits, sync glitches) and missing files. Form options:
  - **Volume** dropdown.
  - **Skip files verified within last N days** — the `[verify] max_age_days` equivalent. Leave blank to re-verify all files.
- **Previews** — (re-)generates asset thumbnails. Web equivalent of `maki generate-previews` in catalog mode. Useful after a `fix-roles --apply` moved a different variant into the best-preview slot, or to add 2560 px smart previews to assets that only have the regular thumbnail. Form options:
  - **Volume** dropdown.
  - `Also generate the 2560 px smart preview` — the `--smart` knob. Generates the high-res preview alongside the regular thumbnail, enabling lightbox zoom.
  - `Upgrade only — regenerate when a different variant is now the best` — the `--upgrade` knob. Skips assets where the best-preview variant is already at index 0 (i.e. nothing changed since last generation). Cheap "fix what fix-roles broke" sweep.
  - `Force regenerate every preview, even existing ones` — the `--force` knob. Slow; use only when previews are visibly wrong.
- **Sync** — file-layer reconciliation: detect moved, modified, new, and missing files on disk vs. catalog locations. Web equivalent of `maki sync`. Different from "Sync metadata" above (which works at the *content* level inside `.xmp` files); this one works at the *file location* level. Form options:
  - **Volume** dropdown — required (the engine's API takes a `Volume`).
  - **Subpath** — optional volume-relative path to scope the scan (e.g. `2024/wedding`). Has filesystem autocomplete: ↑/↓/Tab/Enter to drill into a directory, Esc to dismiss. Drives the same `/api/volumes/{id}/browse` endpoint the import dialog uses.
  - `Apply changes` — without this, dry-run only.
  - `Remove stale catalog locations for missing files` — the `--remove-stale` knob, auto-disabled until "Apply" is checked.
- **Refresh** — re-read metadata from `.xmp` sidecar/recipe files that changed on disk. Web equivalent of `maki refresh`. Use after editing in Lightroom / Capture One while MAKI was offline. Form options:
  - **Volume** dropdown.
  - `Also re-extract embedded XMP from JPEG/TIFF media files` — the `--media` knob. Slow.
  - `Dry run`.
- **Cleanup** — remove orphaned records and files: stale catalog locations, locationless variants, orphaned assets, previews, smart previews, embeddings, and face crops. Web equivalent of `maki cleanup`. Form options:
  - **Volume** dropdown.
  - **Path prefix** — optional volume-relative path to restrict cleanup to a subtree. Same filesystem autocomplete as the Sync tab once a volume is chosen.
  - `Apply changes` — without this, report-only.

Each form's **Run** button posts to a server endpoint that immediately returns a job ID and runs the operation in the background. The dialog switches into a **progress phase** with a status line, the last 50 log entries, and a **Minimize to toast** button so you can dismiss the dialog and keep watching from the bottom-right progress toast (same as Embed / Auto-tag / Detect-faces). When the job finishes, the dialog flips into a **result phase** showing a counter list (`12 written`, `4 conflicts`, …) and a Close button.

The **pulsing dot on the Maintain entry** stays lit until the job finishes. Click the Maintain entry while a job is in flight to re-attach — the dialog opens directly into the progress phase showing the live feed.

Only one job per kind runs at a time; starting a second writeback / sync-metadata / verify / previews / sync / refresh / cleanup while one of the same kind is still running returns 409 from the server. Different kinds run in parallel (you can have a verify going while you also flush a writeback).

Settings Dialog

The **Settings** entry in the top navigation (next to Maintain) opens a modal that renders every option in `maki.toml` as a form widget. The form is generated from the `CatalogConfig` JSON Schema, so a new option added in the Rust source shows up here automatically — there's no separate UI registration step.

Widgets follow types: checkboxes for `bool`, number inputs for integers / floats, text inputs for strings, `<select>` for enums, comma-separated text for `Vec<String>`, collapsible `<details>` sections for nested structs (`[preview]`, `[serve]`, …). Hashmap-typed config (`[import.profiles.<name>]`, `[vlm.model_config."<model>"]`) renders each existing key as an editable sub-section.

Fields bound at server startup show a small **restart required** pill next to the label. After Save, a toast appears bottom-right: green means the file was saved and validated and every change is live; amber means a field flagged restart-required changed, so restart `maki serve` to apply it.

The pill and the amber toast cover every setting that `maki serve` reads once at startup: `[serve]` port/bind/per_page/stroll_*/read_only/username/password, all of `[preview]`, all of `[ai]` (model, thresholds, vocabulary path, prompt, text_limit – the labels file* itself is re-read on every request, so editing its contents needs no restart), all of `[browse]` (default filter and the slideshow fields), `[import]` smart_previews/embeddings/descriptions, `[dedup] prefer`, and the whole `[vlm]` section including per-model overrides. Settings the server re-reads from `maki.toml` per request – such as the import dialog's `[import]` exclude/profiles and the whole `[writeback]` section (`enabled`, `mirror_tags`, `sidecar_naming`, `create_sidecars`) – take effect immediately.

Before each save the previous file is copied to `maki.toml.bak`, so a bad edit is recoverable. Validation runs server-side by deserialising the submitted JSON back into the typed config struct — type errors and unknown keys are rejected before the file is written.

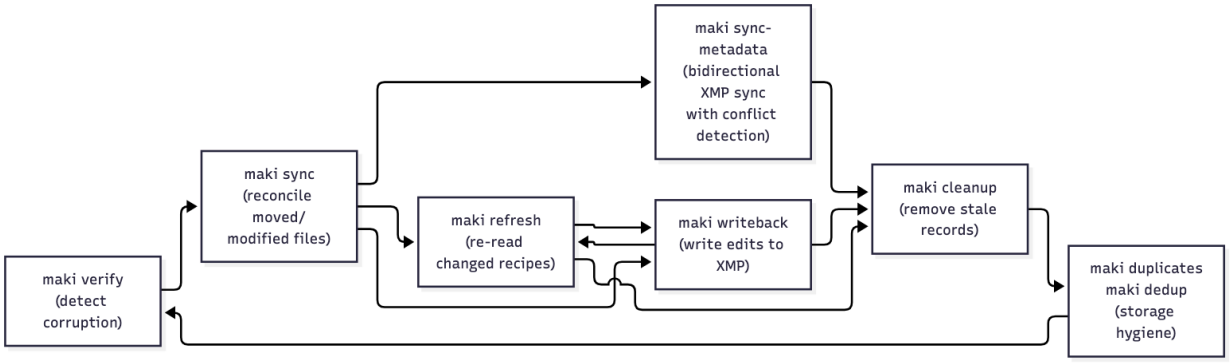
Adding or renaming hashmap keys stays manual: the schema can't express which keys are legal (e.g. VLM model names must match what's loaded on Ollama, profile names are referenced from `import --profile <name>` invocations), and a free-form name field would be too easy to get out of sync. The form shows a hint to that effect at the bottom of each hashmap section — open `maki.toml` in a text editor for those edits.

Next: Maintenance – verification, sync, refresh, cleanup, and file relocation.

Maintenance

Over time, files move, drives get swapped, external tools edit recipes, and storage devices accumulate stale references. This chapter covers the commands that keep your catalog accurate and your files healthy.

The core maintenance commands form a cycle:



After `sync`, there are two paths: `sync-metadata` handles both directions (reading recipe changes and writing back edits) in a single command with conflict detection. Alternatively, `refresh` and `writeback` can be run separately for finer control. After `cleanup`, the storage hygiene commands check for unwanted duplicates and verify backup coverage.

Each command is safe by default – destructive operations require an explicit `--apply` flag, and most commands support `--dry-run` or report-only mode.

Edit History and Undo

Every metadata edit — rating, label, description, name, date, tag add/remove/clear, single or batch, from the CLI, the shell, or the web UI — is journaled in `<catalog>/history/`. Two commands read that journal:

```
maki history                # last 20 operations, newest first
maki history a1b2c3d4       # what changed on one asset, field by field
maki undo --dry-run         # what the next undo would revert
maki undo                   # revert the most recent operation
```

Undo is LIFO at operation granularity: a batch that rated 500 assets is one operation and comes back as one. Before restoring an asset, undo checks that nothing else has touched it since; a conflicting asset is reported and skipped unless you pass `--force`. Import, delete, and structural operations (group, split, stack) are not covered — re-import is idempotent, delete is trash-backed (see Trash below), and structural undo is a planned follow-on.

The journal is neither a sidecar nor part of `catalog.db`: it survives `rebuild-catalog`, `doctor` ignores it, and deleting it loses only the ability to undo. Undo restores catalog and sidecar state; affected `.xmp` recipes

are flagged pending, so run `maki writeback` afterwards if you mirror edits to XMP. Retention is `[history] max_operations` in `maki.toml`.

Verification

`maki verify` re-hashes every file on disk and compares the result to the content hash stored in the catalog. This detects silent corruption, bit rot, and accidental modifications.

Verify everything

```
maki verify
```

This walks all file locations on all online volumes. Offline volumes are skipped automatically.

Sample output:

```
Verify complete: 1847 verified, 2 modified, 0 FAILED, 3 skipped
```

Verify specific files or directories

```
maki verify /Volumes/PhotosDrive/Capture/2026-02-01
```

Only files under the given path are checked.

Limit to a volume

```
maki verify --volume "Photos 2024"
```

Useful when you reconnect a drive and want to spot-check it before trusting its contents.

Verify a single asset

```
maki verify --asset a1b2c3d4
```

Asset IDs can be abbreviated to a unique prefix.

Filter by file type

```
# Only verify RAW files
maki verify --include raw

# Skip audio files
maki verify --skip audio
```

Two verification modes

Catalog mode (no paths): `maki verify` walks all file locations known to the catalog on online volumes. This checks whether your cataloged files are intact.

Path mode (with paths): `maki verify /some/path` scans files on disk and looks them up in the catalog. This also detects files that are not in the catalog at all.

What the results mean

- **verified** – file hash matches the catalog. The `verified_at` timestamp is updated.
- **modified** – a recipe file (`.xmp`, `.cos`, etc.) was changed externally. maki updates the stored hash and reports it as "modified" rather than "FAILED". This is expected when CaptureOne or Lightroom edits a sidecar.
- **FAILED** – a media file's hash does not match. This indicates corruption, accidental overwrite, or a file that was replaced. Investigate immediately.
- **MISSING** – a file referenced in the catalog no longer exists on disk (catalog mode only). The file's location record exists but the file is gone from an online volume.
- **UNTRACKED** – a file was found on disk but is not in the catalog (path mode only). The file's content hash does not match any known variant or recipe. Run `maki import` to bring it into the catalog, or ignore it if it is not a media file.
- **skipped** – the file is on an offline volume, or the path could not be read.

If any files fail verification (FAILED status), MAKI exits with code 1. Scripts can check `$?` to detect problems:

```
maki verify --volume "Archive" || echo "Integrity check failed!"
```

Incremental verification

On a large library, full verification can take hours. Use `--max-age` to skip files that were verified recently — this makes regular weekly or monthly runs practical:

```
# Skip files verified within the last 30 days
maki verify --max-age 30 --log

# Only verify files older than 7 days
maki verify --max-age 7 --volume "Work SSD" --log
```

You can set a default in `maki.toml` with `[verify] max_age_days`, so `maki verify` always skips recently-checked files unless you override with `--force`:

```
# Ignore max_age and re-verify everything
maki verify --force --log --time
```

Monitoring flags

```
# Per-file progress to stderr
maki verify --log

# Machine-readable output
maki verify --json

# Show elapsed time
maki verify --time

# Combine them
maki verify --volume "Photos 2024" --log --time
```

Checking verification age

Use the `stale:N` search filter to find assets that have not been verified recently:

```
# Assets not verified in the last 30 days (or never verified)
maki search "stale:30"
```

`stale:N` always includes never-verified files. To list *only* those, use `stale:never` – do not reach for `stale:0`, which matches every location verified before the current moment, i.e. effectively everything:

```
# Files no verify run has ever covered
maki search "stale:never"
```

See Browsing & Searching for more on search filters.

Consistency Check (doctor)

`maki verify` checks your files; `maki doctor` checks the catalog against itself. The YAML sidecars are the source of truth and SQLite is a derived cache — every write path is supposed to update both, and doctor confirms it did:

```
maki doctor                  # full comparison, summary by field
maki doctor --sample 500     # fast probabilistic check on a large catalog
maki doctor --log            # every finding, per asset
maki doctor --repair         # rebuild the SQLite side of every flagged asset from YAML
```

It flags sidecars without a catalog row and phantom rows without a sidecar, then compares metadata fields, denormalized columns, variant / location / recipe sets (including the pending-writeback flag), and face counts. `--repair` rewrites the catalog from the sidecar for flagged assets and deletes phantom rows; it never edits sidecars. Run it after an interrupted operation, before a big reorganisation, or whenever pending-writeback markers look wrong.

Sync

`maki sync` reconciles the catalog with what is actually on disk. Run it after moving, renaming, or deleting files with external tools (Finder, `mv`, CaptureOne's "move to folder", etc.).

Unlike `verify` (which only checks hashes), `sync` scans the filesystem for structural changes: files that moved, new files that appeared, and files that disappeared.

Report mode (safe default)

```
maki sync /Volumes/PhotosDrive/Photos/
```

Without `--apply`, sync scans the directory and reports what it finds, but changes nothing:

```
Sync complete: 1200 unchanged, 3 moved, 2 new, 1 modified, 4 missing
  Tip: run 'maki import' to import new files.
```

Apply changes

```
maki sync /Volumes/PhotosDrive/Photos/ --apply
```

With `--apply`, sync updates the catalog and sidecar files:

- **Moved files**: The catalog path is updated to the new location (same content hash found at a different path, old path gone).
- **Modified recipes**: Recipe hash is updated, and if it is an XMP file, metadata is re-extracted.
- **Modified media**: A media file (JPEG, TIFF, …) at the same path now has a different hash — typical case: CaptureOne / Lightroom re-exported the file with new edits. The stale file_location is dropped, a full variant for the new hash is inserted (inheriting the previous variant's role and original_filename, with EXIF / dimensions / embedded-XMP freshly extracted from the new bytes), and the asset's sidecar is updated to reference the new variant. **Previews are regenerated** in the same pass, keyed

by the new hash, so the asset is immediately viewable. The old variant becomes locationless and can be removed with `maki cleanup --apply` once you're sure no other volume still holds the previous bytes.
- **Missing files**: Reported but not removed (use `--remove-stale`).
- **New files**: Reported but not imported. Run `maki import` separately.

Removing stale records

```
maki sync /Volumes/PhotosDrive/Photos/ --apply --remove-stale
```

`--remove-stale` (which requires `--apply`) removes catalog location records for files that are confirmed missing. Use this when you intentionally deleted files and want the catalog to reflect that. If removing those locations leaves variants with no remaining locations (often the asset's *selected* preview variant), `sync` prints a follow-up tip pointing at `maki cleanup --apply` to remove the dangling variants and their orphaned previews/embeddings/face files.

Scoping to a volume

```
maki sync /Volumes/PhotosDrive/ --volume "Photos 2024"
```

Explicitly sets the volume context when auto-detection picks the wrong one.

Detection categories

Status	Meaning	Action with `--apply`
unchanged	Hash matches at expected path	None
moved	Known hash found at new path, old path gone	Path updated in catalog
new	Unknown hash, not in catalog	Reported (run `maki import`)
modified	Same path, different hash	Recipe: hash updated + XMP re-extracted. Media variant: stale file_location dropped, full variant for the new hash inserted (inheriting role + original_filename, with EXIF / dimensions / embedded-XMP freshly extracted), sidecar updated, previews regenerated keyed by the new hash. Old variant becomes locationless; clean with `maki cleanup --apply`.
missing	Catalog location exists but file is gone	Reported, or removed with `--remove-stale`

Monitoring flags

```
maki sync /Volumes/PhotosDrive/ --apply --log --time --json
```

`--log` shows per-file status, `--time` shows elapsed time, `--json` outputs structured results.

Refresh

`maki refresh` re-reads metadata from recipe and media files without scanning the full filesystem. It is lighter than `sync` – it only checks files the catalog already knows about, comparing their on-disk hash to the stored hash.

This is the right tool after editing in CaptureOne, Lightroom, or any other tool that modifies XMP sidecars.

Refresh all recipes

```
maki refresh
```

Checks every recipe file location on all online volumes. For each recipe whose hash has changed:

- **XMP recipes** (`.xmp`): re-extracts keywords, rating, description, and color label, then updates the catalog and sidecar YAML.
- **Non-XMP recipes** (`.cos`, `.pp3`, `.dop`, etc.): hash updated, but no metadata extraction (these formats are opaque to maki).

Refresh specific paths

```
maki refresh /Volumes/PhotosDrive/Capture/2026-02-01/
```

Only checks recipe files under the given path.

Limit to a volume or asset

```
# All recipes on a specific volume
maki refresh --volume "Photos 2024"

# Only recipes for a specific asset
maki refresh --asset a1b2c3d4
```

Re-extract embedded XMP from media files

```
maki refresh --media
```

The `--media` flag also scans JPEG and TIFF variant files, re-extracting embedded XMP metadata. This is useful in two scenarios:

1. **Retroactive extraction**: You imported files before the embedded XMP feature existed and want to pick up keywords/ratings/labels that were embedded all along.
2. **External edits**: A tool like CaptureOne or Lightroom modified the embedded XMP in a JPEG/TIFF file.

   Sidecar-wins rule (v4.5.14): MAKI skips embedded-XMP re-extraction for any variant whose asset already has an `.xmp` sidecar recipe. The sidecar is the authoritative metadata file (writeback keeps it current); a JPEG's embedded XMP is frozen at import time and would otherwise re-inject stale keywords on every refresh — which then can't be removed because the embedded-XMP merge is add-only. The `--log` output shows skipped variants as `skipped (sidecar present)`. If you need to force re-extraction from an embedded block (e.g. a sidecarless workflow where another tool edits the embedded XMP directly), delete the recipe first (`maki cleanup --recipes --asset <id>`) or re-import.

Dry run

```
maki refresh --dry-run
```

Shows what would change without applying anything. Combine with `--log` for detailed output:

```
maki refresh --dry-run --log
```

Sample output:

```
  DSC_001.xmp — changed (12ms)
  DSC_002.xmp — unchanged (3ms)
  DSC_003.xmp — changed (11ms)
Dry run — Refresh complete: 2 refreshed, 1 unchanged, 0 missing, 0 skipped (offline)
```

Monitoring flags

```
maki refresh --log --time --json
```

Write Back *(Pro)*

`maki writeback` flushes pending metadata writes to XMP recipe files. The pending queue accumulates in two situations:

1. You edit metadata (rating, label, tags, description) while the asset's volume is offline. The auto-writeback path skips the file write and marks the recipe `pending_writeback` until the volume comes back.
2. `[writeback] enabled = false` (the default safety setting). Every metadata edit lands in the catalog and YAML sidecar and is flagged `pending_writeback`, but no `.xmp` file is touched. Then `maki writeback` is the explicit, one-off flush — your safety net stays in place between flushes.

   Note: `maki writeback` always works regardless of `[writeback] enabled`. The config flag governs only *automatic* writeback on every edit; this command is the manual flush. See the Configuration Reference for the auto-vs-manual split.

Process pending write-backs

```
maki writeback
```

Without flags, only recipes with `pending_writeback=1` are processed. Each recipe's XMP file is updated with the current asset metadata (rating, label, tags, description), then re-hashed and the pending flag is cleared.

Write back all XMP recipes

```
maki writeback --all
```

The `--all` flag writes current metadata to every XMP recipe, not just pending ones. Useful for an initial sync or to force-push all MAKI metadata to XMP files.

Scope to a volume or asset

```
# Only write back to recipes on a specific volume
maki writeback --volume "Photos 2024"

# Only write back for a specific asset
maki writeback --asset a1b2c3d4
```

Dry run

```
maki writeback --dry-run
```

Shows what would be written without modifying any files. Combine with `--log` for detailed output.

How pending tracking works

When any metadata edit (CLI `maki edit`, `maki tag`, web UI stars/labels/tags/description) triggers XMP write-back, each recipe is checked:

- **Volume online, file exists**: XMP is written, `pending_writeback` stays 0.
- **Volume offline or file missing**: `pending_writeback` is set to 1. The flag persists until `maki writeback` clears it.

The flag records the *intent* to write back, not *what* changed. When writeback runs, it reads the current asset metadata and writes all four fields (rating, label, tags, description) to the XMP file.

Recommended workflow: volume comes back online

```
# 1. Push MAKI edits to XMP (MAKI wins for fields edited while offline)
maki writeback --volume "Archive 2025"

# 2. Pull any CaptureOne/Lightroom edits from XMP
maki refresh --volume "Archive 2025"
```

Order matters: writeback first ensures MAKI edits land in the XMP files. Then refresh picks up anything the external tool changed independently.

Monitoring flags

```
maki writeback --log --time --json
```

When recipes are skipped because their volume is offline, the summary names the offline volumes so you know which drives to reconnect before re-running:

```
Writeback: 0 written, 1 skipped (offline volumes: Archive 2025)
  → Reconnect "Archive 2025" and re-run `maki writeback`.
```

With `--json`, the same information is available as a `skipped_offline_volumes` array on the result object.

Using `rsync` alongside `maki writeback`

A common backup pattern: keep the master files on one drive (say "Archive") and rsync them to a backup drive ("Archive-Backup") periodically. If both volumes are registered in MAKI, each asset has recipes on both volumes, and edits made while one drive is offline queue up as `pending_writeback` until that drive comes back online.

The workflow that takes the least effort:

```
# 1. Curate normally. Master volume is online, backup is offline.
#    MAKI edits flush inline to the master's XMP files; backup recipes
#    accumulate pending_writeback.

# 2. Bulk-sync the backup drive outside MAKI when ready:
rsync -av --delete /Volumes/Archive/ /Volumes/Archive-Backup/
```

```
# 3. Run a single writeback to reconcile:
maki writeback
```

In step 3, MAKI walks every pending recipe on the backup volume, reads each XMP, and finds that the rsync already wrote the values MAKI was about to write. No file writes happen. The pending flag clears, and the recipe's stored `content_hash` is reconciled to match the rsync'd file. Output looks like:

```
Writeback: 0 written, 90000 already in sync
```

A subsequent `maki verify --volume Archive-Backup` runs cleanly against the now-accurate hashes — no false-positive mismatches. The `already_in_sync` counter on the JSON result and the web toast surfaces the same information.

This works because MAKI's writeback is idempotent at the field level (the underlying `xmp_reader::update_*` primitives skip when nothing changed) and now also self-reconciles drifted content hashes on the no-write path. You don't need a separate "post-rsync refresh" step.

Sync Metadata *(Pro)*

`maki sync-metadata` performs bidirectional XMP metadata sync in a single command — combining the inbound (refresh) and outbound (writeback) steps with conflict detection.

> **Note:** The outbound (writeback) phase honours `[writeback] enabled`. With auto-flush off (the default), `sync-metadata` performs inbound sync only — the catalog absorbs external XMP changes, but MAKI edits stay queued as `pending_writeback` until you run `maki writeback` (which always works regardless of the flag). With `enabled = true`, both directions run inline. See the Configuration Reference.

Basic usage

```
maki sync-metadata
```

This runs three phases:

1. **Inbound**: Detects externally modified XMP recipe files and re-reads their metadata (keywords, rating, description, color label).
2. **Outbound**: Finds recipes marked `pending_writeback` and writes current MAKI metadata back to the XMP file.
3. **Conflict detection**: When both the XMP file changed on disk AND the recipe has pending MAKI edits, the recipe is reported as a conflict and skipped.

Scope to a volume or asset

```
maki sync-metadata --volume "Photos 2024"
maki sync-metadata --asset a1b2c3d4
```

Include embedded XMP

```
maki sync-metadata --media
```

The `--media` flag adds a third phase that re-extracts embedded XMP from JPEG/TIFF variant files — useful after external tools modify embedded metadata.

Dry run

```
maki sync-metadata --dry-run --log
```

Shows what would change without modifying any files.

When to use sync-metadata vs. writeback + refresh

- **`sync-metadata`**: The recommended single command for most workflows. Handles both directions and detects conflicts.
- **`writeback` + `refresh`**: Use separately when you want explicit control over direction (e.g., force MAKI edits to win with `writeback --all`, then pull external changes with `refresh`).

Working with External Tools

MAKI is designed to coexist with tools like CaptureOne, Lightroom, RawTherapee, and DxO. Metadata flows between MAKI and these tools through XMP sidecar files. This section describes common round-trip scenarios.

Prerequisite: enable XMP sidecar writing in your photo editor

Most photo editors do **not** write XMP sidecar files by default. You need to enable this in each tool's settings, otherwise MAKI has no metadata to read.

Tool	Setting	Location
CaptureOne	Enable "Auto-write XMP sidecar"	Preferences → General → Metadata, or per-session in the tool tab
Lightroom Classic	Enable "Automatically write changes into XMP"	Catalog Settings → Metadata
RawTherapee	XMP sidecars are always written	No setting needed
DxO PhotoLab	Enable "Write to XMP sidecar"	Preferences → General
darktable	Enable "Write sidecar file for each image"	Settings → Storage

Without this setting, your ratings, keywords, and labels stay locked inside the tool's proprietary database and are invisible to MAKI.

Prerequisite on the MAKI side: sidecars must exist before the first session

Importing never creates `.xmp` files. Tags and ratings you assign right after a RAW import live in the catalog only, and writeback (automatic or `maki writeback`) can only update sidecars that already exist. If you open such a shoot in Capture One now, it creates its own empty sidecars and never sees your MAKI metadata; a later `maki refresh` then merges Capture One's values *over* yours for rating, label, and description.

So, before the first Capture One or Lightroom session on a freshly imported shoot:

```
maki create-sidecars "volume:Work SSD"          # report only
maki create-sidecars "volume:Work SSD" --apply  # write DSC_1234.xmp files
```

Each sidecar carries the asset's rating, label, description, and tags and is registered as a recipe, so from then on both directions work: `maki refresh` reads the tool's edits, `maki writeback` (or `[writeback] enabled = true`) pushes yours. To skip the manual step entirely, set `[writeback] enabled = true` together with

`create_sidecars = true`: an import that assigns tags (`--add-tag`, the web import dialog's tag field, `[import] auto_tags`) creates the sidecars in the same run, and the first edit of any other sidecar-less asset creates the file on the spot.

Sidecars are named `DSC_1234.xmp` (the Capture One / Lightroom convention). darktable users set `[writeback] sidecar_naming = "full"` for `DSC_1234.NEF.xmp`; MAKI imports either form.

"I rated and tagged in CaptureOne/Lightroom — how do I get those changes into MAKI?"

External tools write ratings, keywords, descriptions, and color labels to `.xmp` sidecar files. Use `refresh` to read those changes:

```
maki refresh --volume "Work SSD" --log
```

This detects XMP files that changed since the last import or refresh, and updates the catalog with the new metadata. Tags from XMP are merged (union) with existing MAKI tags.

If the external tool also modified embedded XMP in JPEG or TIFF files (e.g., Lightroom export with metadata), add `--media`:

```
maki refresh --media --volume "Work SSD" --log
```

"I rated and tagged in MAKI — how do I get those changes into CaptureOne/Lightroom?"

Use `writeback` to push MAKI metadata to XMP files on disk. The external tool picks up the changes when it re-reads the sidecar:

```
maki writeback --volume "Work SSD" --log
```

This writes rating, color label, tags, and description to every XMP recipe flagged `pending_writeback`. Works whether or not `[writeback] enabled = true` — that config controls only *automatic* writeback on every edit; the `writeback` command is the explicit manual flush.

For an initial sync (pushing all MAKI metadata to XMP, not just pending changes):

```
maki writeback --all --volume "Work SSD" --log
```

"I worked in both tools — how do I sync everything?"

Use `sync-metadata` for bidirectional sync with conflict detection:

```
maki sync-metadata --volume "Work SSD" --log
```

This reads external XMP changes, writes pending MAKI edits, and reports conflicts (files where both MAKI and the external tool made changes). Conflicts are skipped — you can resolve them by choosing which direction should win:

```
# Force MAKI edits to win, then pull any remaining external changes
maki writeback --all --volume "Work SSD"
maki refresh --volume "Work SSD"
```

"I imported old files that had XMP metadata I didn't capture"

If files were imported before MAKI supported embedded XMP extraction (or before sidecar XMP files existed), you can retroactively pick up that metadata:

```
# Re-extract embedded XMP from JPEG/TIFF files
maki refresh --media --log

# Re-read all XMP sidecar files
maki refresh --log
```

Summary: which command when?

Scenario	Command
External tool edited XMP sidecars	`maki refresh`
External tool edited embedded XMP in JPEG/TIFF	`maki refresh --media`
Push MAKI edits to XMP for external tools	`maki writeback`
Bidirectional sync with conflict detection	`maki sync-metadata`
Force MAKI metadata to all XMP files	`maki writeback --all`
Pick up metadata from old imports	`maki refresh --media`

Cleanup

`maki cleanup` scans all file locations and recipes across online volumes, checking whether the referenced files still exist on disk. It removes stale records, locationless variants, and orphaned derived files in eight passes.

Report mode (safe default)

```
maki cleanup
```

Without `--apply`, cleanup reports what it finds:

```
Cleanup complete: 1500 checked, 12 stale, 3 orphaned assets, 5 orphaned previews, 2 orphaned embeddings
  Run with --apply to remove stale records and orphaned files.
```

Apply cleanup

```
maki cleanup --apply
```

The eight passes:

1. **Stale location and recipe records**: Removes catalog entries for files that no longer exist on disk (variant file locations and recipe file locations). Updates sidecar YAML files accordingly.
2. **Locationless variants**: Removes variants with zero remaining file locations from assets that still have other located variants. Prevents ghost variants from accumulating after file moves or reimports.
3. **Orphaned assets**: Deletes assets where all variants have zero file locations remaining. Their recipes, variants, faces, embeddings, previews, smart previews, face crops, embedding binaries, catalog rows, and sidecar YAML files are removed.
4. **Orphaned previews**: Removes preview JPEG files whose content hash no longer matches any variant in the catalog.
5. **Orphaned smart previews**: Same for the `smart-previews/` directory.
6. **Orphaned embeddings**: Removes SigLIP embedding binaries whose asset ID no longer exists.
7. **Orphaned face crops**: Removes face crop thumbnails whose face ID no longer exists.
8. **Orphaned ArcFace embeddings**: Removes face embedding binaries whose face ID no longer exists.

Limit to a specific volume

```
maki cleanup --volume "Photos 2024" --apply
```

Only scans file locations on the specified volume. Useful after removing a drive's contents intentionally.

Limit to a specific path

```
maki cleanup --path "Capture/2026-02" --apply
maki cleanup --volume "Photos" --path "Archive/Old" --apply --log
```

Scopes stale-location scanning to files under a path prefix. Absolute paths are auto-detected to extract the volume and relative prefix.

List stale entries

```
maki cleanup --list
```

Prints stale entries to stderr (similar to `--log`, but only shows stale entries rather than every file checked).

Offline volumes

Offline volumes are skipped automatically with a note. MAKI never removes records for files on an offline volume – it cannot know whether the file is truly gone or just disconnected.

Monitoring flags

```
maki cleanup --apply --log --time --json
```

Trash

File-deleting operations — `maki delete --remove-files`, `maki dedup --apply`, and the web duplicates page — move files into `<catalog>/.trash/<date>/<volume>/<path>` instead of deleting them, so a mistaken batch is recoverable. Derived files (previews, embeddings, face crops) never enter the trash; they are regenerable.

```
maki trash list                                  # what is in the trash, with restore keys
maki trash restore 2026-09-01/Photos/2026/DSC_0042.nef
maki trash empty --dry-run                       # what would be purged (older than retention_days)
maki trash empty --all
```

`restore` moves the file back to its original location on its volume; the catalog is not updated, so re-register it with `maki import <path>` or `maki refresh`. `[trash] enabled` and `retention_days` (default 30) live in `maki.toml`; every deleting command accepts `--no-trash` for a permanent delete when freeing disk space is the whole point. The trash lives on the catalog volume, so trashing a file from another volume costs one copy.

Relocating Assets

`maki relocate` copies (or moves) all of an asset's files – variants and recipes – to another volume. Use this to migrate assets between drives, create backups on a second volume, or consolidate scattered files.

Copy to another volume

```
maki relocate a1b2c3d4 "Archive Drive"
```

This copies all files for the asset to the target volume, preserving their relative paths. The asset now has files on both volumes. After the copy, each file is verified via SHA-256 to ensure integrity.

Move (copy + delete source)

```
maki relocate a1b2c3d4 "Archive Drive" --remove-source
```

With `--remove-source`, the source files are deleted after successful copy and verification. The asset's locations are updated to point to the new volume only.

Dry run

```
maki relocate a1b2c3d4 "Archive Drive" --dry-run
```

Shows what would be copied or moved without making any changes:

```
Dry run — no changes made:
  Copy Capture/2026-02-01/DSC_001.nef → Archive Drive:Capture/2026-02-01/DSC_001.nef
  Copy Capture/2026-02-01/DSC_001.xmp → Archive Drive:Capture/2026-02-01/DSC_001.xmp
```

Batch relocation with `--query`

The single-asset examples above are useful for individual files, but the real power of `relocate` is moving entire shoots, years, or collections to another drive in one pass:

```
# Preview: migrate all 2025 images from the work SSD to archive
maki relocate --query "date:2025 volume:Work SSD" --target "Archive 2025" --dry-run

# Execute the copy
maki relocate --query "date:2025 volume:Work SSD" --target "Archive 2025" --log

# After verifying the archive, free the working drive
maki relocate --query "date:2025 volume:Work SSD" --target "Archive 2025" --remove-source --log
```

The `--query` flag accepts the same search syntax as `maki search`, so you can combine any filters:

```
# Move a specific shoot to archive
maki relocate --query "tag:johnson-wedding" --target "Archive 2026" --log

# Copy all single-copy images to a backup drive
maki backup-status --at-risk -q | maki relocate --target "Backup A" --log

# Move all videos to a dedicated video drive
maki relocate --query "type:video volume:Work SSD" --target "Video Archive" --remove-source --log
```

Tip: Always run without `--remove-source` first (copy), verify the result, then run again with `--remove-source` (move) in a second pass. This gives you a safety window before deleting source files.

Creating XMP sidecars on relocate

When copying files to a working drive where you'll open them in CaptureOne or Lightroom, use `--create-sidecars` to generate `.xmp` files with your MAKI metadata (ratings, tags, labels, descriptions):

```
maki relocate --query "rating:3+" --target "Work SSD" --create-sidecars --log
```

This creates an XMP sidecar alongside each variant that doesn't already have one on the target volume. The external tool picks up your ratings and tags immediately on import. The file is named `DSC_1234.xmp`

by default (`[writeback] sidecar_naming = "full"` switches to darktable's `DSC_1234.NEF.xmp`); a sidecar that already exists on disk but was never imported is left alone.

For files that stay where they are, `maki create-sidecars --apply` does the same thing in place (see below).

Updating File Locations

`maki update-location` fixes the catalog after you manually moved a single file on disk. Unlike `sync` (which scans a directory), this command targets one specific file.

Basic usage

```
maki update-location a1b2c3d4 \
    --from Capture/2026-02-01/DSC_001.nef \
    --to /Volumes/PhotosDrive/Archive/2026/DSC_001.nef
```

- `--to` must be an absolute path to the file's current location on disk.
- `--from` can be absolute or volume-relative (the path as it appears in the catalog).

How it works

1. maki resolves the asset by ID (or unique prefix).
2. The volume is auto-detected from `--to` by matching against registered volume mount points. You can override with `--volume`.
3. maki hashes the file at `--to` and compares it to the stored content hash. If they do not match, the command fails (safety check – you may have pointed to the wrong file).
4. The catalog and sidecar YAML are updated with the new path.

Specifying the volume explicitly

```
maki update-location a1b2c3d4 \
    --from Capture/old/DSC_001.nef \
    --to /Volumes/NewDrive/Capture/new/DSC_001.nef \
    --volume "New Drive"
```

This also handles recipe file locations, not just variant files.

Rebuilding the Catalog

`maki rebuild-catalog` wipes the SQLite database and reconstructs it from the YAML sidecar files, which are the source of truth.

```
maki rebuild-catalog
```

Sample output:

```
Rebuild complete: 1847 assets, 2914 variants, 312 recipes, 3 collections, 45 stacks
```

When to use it

- The SQLite database is corrupted or deleted.
- You manually edited sidecar YAML files and want the catalog to reflect those changes.
- Something seems out of sync and you want a clean slate.

What is preserved

- **Sidecar YAML files**: Untouched (they are the source, not the target).
- **Collections**: Restored from `collections.yaml` at the catalog root.
- **Stacks**: Restored from `stacks.yaml` at the catalog root (member order and pick assignments are preserved).
- **Faces and people** *(Pro)*: Restored from `faces.yaml` and `people.yaml`. ArcFace face embeddings restored from binary files in `embeddings/arcface/`.
- **Image embeddings** *(Pro)*: SigLIP embeddings restored from binary files in `embeddings/<model>/`.
- **Preview files**: Untouched (they are content-addressed by hash).
- **Volumes**: Re-registered from `volumes.yaml`.

What is regenerated

- All SQLite tables (assets, variants, recipes, file locations, tags, stacks).
- Denormalized columns (`best_variant_hash`, `primary_variant_format`, `variant_count`).
- Collection membership records (from `collections.yaml`).
- Stack membership records (from `stacks.yaml`).

This is a safe operation – only the derived cache (SQLite) is rebuilt. No files on disk are modified or deleted.

Fix Dates

`maki fix-dates` corrects asset dates that were set incorrectly during import. This commonly happens when files lack EXIF metadata (e.g., old JPEGs without DateTimeOriginal) — the import timestamp was used instead of the capture date.

How it works

For each asset, `fix-dates` collects candidate dates from all variants:

1. **EXIF DateTimeOriginal** from stored `source_metadata` (for assets imported since v1.3.1)
2. **Re-extracted EXIF** from the file on disk (for older assets without stored date)
3. **File modification time** on disk

The oldest date across all variants becomes the corrected `created_at`.

Report mode (safe default)

```
maki fix-dates
```

Shows what would be changed:

```
Warning: volume 'Archive' is offline — cannot read files for date extraction
Fix-dates: 5000 checked, 3200 fixed, 800 already correct, 1000 skipped (volume offline)
  Run with --apply to make changes.
  Mount offline volumes and re-run to fix remaining assets.
```

Apply fixes

```
maki fix-dates --apply --log
```

Updates both the SQLite catalog and sidecar YAML files. Also backfills EXIF dates into variant metadata so future runs work without needing the volume online.

Offline volumes

`fix-dates` needs file access for EXIF re-extraction and file modification times. Assets on offline volumes are skipped with a clear message. Mount the volume and re-run to fix them.

Scope to a volume or asset

```
# Fix dates for assets on a specific volume
maki fix-dates --volume "Photos 2024" --apply --log

# Fix a single asset
maki fix-dates --asset a1b2c3d4 --apply
```

Fix Roles

`maki fix-roles` corrects variant roles in assets that have both RAW and non-RAW variants. In these groups, the RAW variant should be the Original and non-RAW variants should be Exports. If roles are incorrect (e.g., both marked as Original), this command fixes them.

Report mode (safe default)

```
maki fix-roles
```

Shows what would be changed:

```
Dry run — Fix-roles: 150 checked, 3 fixed (5 variant(s)), 147 already correct
  Run with --apply to make changes.
```

Apply fixes

```
maki fix-roles --apply
```

Updates both the SQLite catalog and sidecar YAML files.

Scope to a volume or asset

```
# Only check assets on a specific volume
maki fix-roles --volume "Photos 2024" --apply

# Fix a single asset
maki fix-roles --asset a1b2c3d4 --apply
```

Fix Recipes

`maki fix-recipes` repairs a specific situation: recipe files (`.xmp`, `.cos`, `.pp3`, etc.) that were imported as standalone assets instead of being attached to their media files. This happens when recipe and media files are imported in separate passes, or when recipe files live in a different directory from their media.

The result is phantom assets in your catalog — assets whose only variant is a recipe file, when they should have been attached to the corresponding media asset.

```
# Preview what would be fixed
maki fix-recipes
```

```
# Apply fixes
maki fix-recipes --apply --log

# Scope to a specific volume or asset
maki fix-recipes --volume "Work SSD" --apply
maki fix-recipes --asset a1b2c3d4 --apply
```

The command matches recipe files to media files by filename stem (e.g., `DSC_001.xmp` attaches to the asset containing `DSC_001.nef`). When a match is found, the recipe is moved from its standalone asset to the correct media asset, and the now-empty standalone asset is removed.

For the full command reference, see fix-recipes.

Preview Management

During import, MAKI generates preview thumbnails (800px by default) for every media file. Over time, previews can become stale or insufficient — after processing in external tools, after changing preview settings, or when you need offline zoom capabilities. The `maki generate-previews` command lets you regenerate and upgrade previews.

Upgrading previews after processing

When you import RAW files, the auto-generated preview is rendered from the RAW data. If you later export a processed TIFF or JPEG from CaptureOne or Lightroom and import it as an additional variant, the preview still shows the original RAW rendering. Use `--upgrade` to regenerate previews from the better variant:

```
# Regenerate previews where a processed/export variant exists
maki generate-previews --upgrade --log
```

This checks each asset for export or processed variants and regenerates the preview from the highest-quality one. Assets with only an original variant are skipped.

Smart previews for offline browsing

Smart previews are high-resolution versions (2560px) that enable zoom and pan in the web UI — even when the original volume is offline. Generate them for assets you want to review without connecting the source drive:

```
# Generate smart previews for a volume before disconnecting it
maki generate-previews --volume "Work SSD" --smart --log

# Generate smart previews for your best images
maki search -q "rating:4+" | maki generate-previews --smart --log
```

Smart previews are also generated during import with `maki import --smart` or when `[import] smart_previews = true` is set in `maki.toml`.

Force regeneration

After changing preview settings in `maki.toml` (e.g., increasing `max_edge`, switching `format` from JPEG to WebP, or adjusting `quality`), existing previews still use the old settings. Use `--force` to regenerate all previews:

```
maki generate-previews --force --log --time
```

Scoping

Limit regeneration to a specific volume, asset, or path:

```
# Only regenerate previews for one volume
maki generate-previews --volume "Archive 2025" --log

# Only regenerate for a specific asset
maki generate-previews --asset a1b2c3d4

# Only process image files (skip video, audio)
maki generate-previews --skip video --skip audio --log
```

For the full command reference, see generate-previews.

Storage Hygiene

Over time, a multi-volume library accumulates duplicate files and uneven backup coverage. MAKI provides three commands that work together to answer the key storage questions: *Do I have unwanted copies wasting space?* and *Are my important files safely backed up?*

Finding and understanding duplicates

The first step is understanding what duplicates you have and *why* they exist. Same-volume and cross-volume duplicates answer fundamentally different questions:

Mode	Question it answers	Typical action
`--same-volume`	Do I have accidental copies on the same drive?	Clean up with `maki dedup`
`--cross-volume`	Are my files backed up across drives?	Verify coverage is sufficient

```
# Accidental duplicates — candidates for cleanup
maki duplicates --same-volume

# Backup verification — these copies are wanted
maki duplicates --cross-volume
```

See Finding Duplicates for detailed examples and output formats.

Removing unwanted duplicates

`maki dedup` removes same-volume duplicates automatically. It only targets files that appear more than once on the **same** drive — cross-volume copies (your backups) are never touched.

```
# Preview what would be removed
maki dedup

# Apply, preferring files under "Selects/" when choosing which copy to keep
maki dedup --prefer "Selects" --apply --log
```

The resolution heuristic decides which copy to keep: prefer paths matching `--prefer`, then most recently verified, then shortest path. Use `--min-copies` to ensure a minimum number of total copies survive across all volumes:

```
# Never reduce a file to fewer than 2 copies total
maki dedup --min-copies 2 --apply
```

See the Dedup Command Reference for all options.

Checking backup coverage

`maki backup-status` reports which assets lack copies on archive or backup volumes. This works best when your volumes have a purpose assigned:

```
maki backup-status
```

Finding at-risk assets

Use `--at-risk` to get a concrete list of under-backed-up assets:

```
# Assets with fewer than 2 copies (the default threshold)
maki backup-status --at-risk

# Stricter policy: require 3 copies
maki backup-status --min-copies 3 --at-risk

# Scope to rated images only
maki backup-status --at-risk -q "rating:1+"
```

Checking coverage for a specific volume

Use `--volume` to answer "which of my files aren't on this drive yet?":

```
# Which assets are missing from the backup drive?
maki backup-status --volume "Backup A" --at-risk

# Output as IDs for scripting
maki backup-status --volume "Backup A" --at-risk -q
```

Fixing coverage gaps

Pipe at-risk asset IDs directly into `relocate` to copy them to a backup drive:

```
maki backup-status --at-risk -q | maki relocate --target "Backup A" --log
```

For a quick check from search, the `copies:` filter finds under-backed-up assets directly:

```
# Single-copy assets — at risk if the drive fails
maki search "copies:1"

# Assets with 3+ copies — potential over-redundancy
maki search "copies:3+"
```

The web UI backup page provides a visual dashboard with volume distribution charts and gap analysis.

Aligning Flat Tags to the AI Vocabulary *(Pro)*

The AI vocabulary file (`[ai] labels = "my-labels.yaml"`) maps each flat label the SigLIP classifier sees to its canonical hierarchical home in your catalog. For new assets, MAKI applies the hierarchical form directly. But two situations still leave flat tags behind:

- **Vocabulary growth.** When you add a new mapping (`dog: subject|animal|domestic`), assets previously tagged with the bare `dog` keep the flat form. The mapping only steers *future* AI suggestions.
- **Inbound imports.** Sidecar XMP from Lightroom / CaptureOne typically arrives as flat keywords. MAKI imports them as-is, so `sunset`, `concert`, `landscape` land flat in the catalog.

The `scripts/apply-vocabulary.py` helper reads the vocabulary YAML and emits the `maki tag rename` / `maki tag split` commands needed to move every flat label into its hierarchical home. Run it whenever the vocabulary changes or after a batch of flat-tagged imports.

How it works

For each entry in the vocabulary:

- `label: tag` — emits `maki tag rename =label tag` (whole-path match so it touches only the flat form, not `something|label`).
- `label: [tag1, tag2]` — emits `maki tag split =label tag1 tag2` (one-to-many fan-out).
- `label: null` or identity (`label: label`) — skipped, no rename needed.

The script doesn't query the catalog — it emits every command for every mapping. Tags that don't exist in your catalog are no-ops at execution time (MAKI reports "no assets found" and moves on), so the output is safe to run unconditionally. Ancestor paths (`subject`, `subject|nature`, …) are auto-expanded by `maki tag rename` itself, so you don't need to spell them out.

Workflow

```
# 1. Review what would change — dry-run by default
python3 scripts/apply-vocabulary.py

# 2. Save for review, then execute
python3 scripts/apply-vocabulary.py --apply > apply-vocab.sh
less apply-vocab.sh
sh apply-vocab.sh
```

The script picks up your active `[ai] labels` automatically via `maki ai export-vocabulary`. Three other ways to scope it:

```
# Built-in default vocabulary
python3 scripts/apply-vocabulary.py --default

# Explicit file (e.g. test a candidate before adopting it)
python3 scripts/apply-vocabulary.py path/to/my-labels.yaml

# Pipe straight to sh — fine for the recurring case once you trust your vocab
python3 scripts/apply-vocabulary.py --apply | sh
```

After running the script, the dc:subject XMP keyword list still reflects every flat component (`subject`, `nature`, `landscape`, …) because `maki tag rename` writes back through the same hierarchical/flat pair MAKI uses everywhere — Lightroom and CaptureOne see the full keyword set.

When to run

Run after any of these:

- Editing `my-labels.yaml` (adding mappings, splitting flat labels, promoting branches).
- A batch import that brought in flat keywords from external XMP sidecars.
- Periodically — quarterly, or alongside `maki verify` — as a hygiene pass that catches drift.

Recommended Maintenance Workflow

A practical schedule for keeping your catalog healthy:

1. Periodic integrity checks

Run `maki verify` on a regular schedule – weekly for active volumes, monthly for archives:

```
# Verify the active working drive
maki verify --volume "Work SSD"

# Find assets that haven't been verified in 90 days
maki search "stale:90"
```

2. After editing in external tools

When you finish a session in CaptureOne, Lightroom, or another tool that edits XMP sidecars, run `maki refresh` to pick up the changes:

```
maki refresh --volume "Work SSD" --log
```

If the external tool also modified embedded metadata in JPEG/TIFF exports:

```
maki refresh --media --volume "Work SSD"
```

3. After moving or reorganizing files

If you moved files on disk (renamed directories, reorganized folder structure), run `maki sync` to reconcile:

```
# First, see what changed
maki sync /Volumes/PhotosDrive/Photos/

# Then apply
maki sync /Volumes/PhotosDrive/Photos/ --apply
```

If new files appeared (e.g., CaptureOne exported new TIFFs), import them:

```
maki import /Volumes/PhotosDrive/Photos/Exports/
```

4. Periodic cleanup and storage hygiene

Run `maki cleanup` periodically to remove stale records for files that no longer exist:

```
# See what would be cleaned up
maki cleanup --list

# Apply
maki cleanup --apply
```

After cleanup, check for accidental duplicates and verify backup coverage:

```
# Any unwanted same-volume duplicates?
maki duplicates --same-volume

# Are backups in good shape?
maki backup-status
```

See Storage Hygiene above for the full workflow.

5. After editing the AI vocabulary or importing flat-tagged sidecars *(Pro)*

When `my-labels.yaml` grows or external XMP sidecars bring in flat keywords, align the catalog to the vocabulary:

```
# Preview
python3 scripts/apply-vocabulary.py

# Execute
python3 scripts/apply-vocabulary.py --apply | sh
```

See Aligning Flat Tags to the AI Vocabulary above for the full workflow.

6. Nuclear option: rebuild

If the catalog seems fundamentally out of sync – searches return wrong results, show displays stale data – rebuild from the sidecar files:

```
maki rebuild-catalog
```

This is safe and fast. The sidecar YAML files are the source of truth; the SQLite database is just a derived cache.

Putting it all together

A typical maintenance session after reconnecting an archive drive:

```
# 1. Check file integrity
maki verify --volume "Archive 2025" --time

# 2. Push any MAKI edits made while the drive was offline
maki writeback --volume "Archive 2025"

# 3. Pick up any recipe changes made while the drive was connected elsewhere
maki refresh --volume "Archive 2025"

# 4. Reconcile any moved/renamed files
maki sync /Volumes/Archive2025/ --apply

# 5. Clean up any stale records
maki cleanup --volume "Archive 2025" --apply

# 6. Check for accidental duplicates on this volume
maki duplicates --same-volume --volume "Archive 2025"

# 7. Align flat tags from any imported sidecars to your hierarchy (Pro)
python3 scripts/apply-vocabulary.py --apply | sh

# 8. Verify backup coverage across all volumes
maki backup-status

# 9. Confirm everything looks good
maki stats --volumes
```

Backing Up Your Catalog with Git

MAKI's source of truth is the YAML sidecar files in the `metadata/` directory. The SQLite database, previews, and embeddings are all derived and can be regenerated. This makes the catalog an ideal candidate for git version control — you get full history, easy rollback, and protection against accidental bulk operations.

Setup

`maki init` creates a `.gitignore` that excludes derived files. To start tracking your catalog:

```
cd /path/to/your/catalog   # the directory containing maki.toml
git init
git add -A
git commit -m "Initial catalog snapshot"
```

The `.gitignore` excludes:

- `catalog.db` (rebuilt via `maki rebuild-catalog`)
- `previews/` and `smart-previews/` (regenerated via `maki generate-previews`)
- `embeddings/` and `faces/` (regenerated via `maki embed`, `maki faces detect`)

What gets tracked: `metadata/**/*.yaml` (all asset metadata), `volumes.yaml`, `collections.yaml`, `stacks.yaml`, `searches.toml`, and `maki.toml`.

Snapshotting before bulk operations

Before running bulk operations like `tag rename`, `fix-roles`, `dedup`, or `cleanup`, create a snapshot:

```
# Using the provided script
bash scripts/backup-catalog.sh "Before tag cleanup"

# Or manually
cd /path/to/your/catalog
git add -A && git commit -m "Before tag cleanup"
```

Recovering from mistakes

If a bulk operation goes wrong:

```
cd /path/to/your/catalog

# See what changed
git log --oneline
git diff HEAD~1 -- metadata/

# Restore all metadata to a previous state
git checkout HEAD~1 -- metadata/
maki rebuild-catalog

# Or restore just specific assets
git checkout HEAD~1 -- metadata/a1/a1b2c3d4-*.yaml
maki rebuild-catalog
```

The `maki rebuild-catalog` step regenerates the SQLite database from the restored YAML files.

What this protects against

- Accidental bulk tag renames or deletions

- Metadata lost during `cleanup` or `dedup`
- Any operation that modifies YAML sidecars

What it does NOT protect against: file deletions on disk (`maki delete --remove-files`), or XMP writeback changes (those modify files on your volumes, not in the catalog).

Recovering from a Drive Failure

When a storage device fails, MAKI's multi-volume architecture and content-addressable design help you assess the damage and recover. Here is a step-by-step playbook.

1. Assess the damage

Check which volume is affected and how many assets were on it:

```
# Which volumes are online/offline?
maki volume list

# How many assets had files on the failed drive?
maki search "volume:FailedDrive" --format ids | wc -l
```

2. Check what's backed up

Assets with copies on other volumes are safe. The key question is: which assets were *only* on the failed drive?

```
# Assets that exist ONLY on the failed drive (single-copy, at risk)
maki search "volume:FailedDrive copies:1"

# Assets that have copies elsewhere (safe)
maki search "volume:FailedDrive copies:2+"
```

For a broader view, use `backup-status`:

```
maki backup-status --at-risk
```

3. Recover what you can

If you have partial data recovery from the failed drive, import the recovered files. MAKI matches by content hash, so recovered files are automatically linked to existing assets:

```
maki import /path/to/recovered-files/ --log
```

4. Clean up stale records

Once you've recovered everything possible, remove the references to the dead drive:

```
# Preview what would be cleaned up
maki cleanup --volume "FailedDrive"

# Apply cleanup
maki cleanup --volume "FailedDrive" --apply --log
```

This removes file location records pointing to the failed volume. Assets that still have locations on other volumes are preserved. Assets with no remaining locations become orphaned and are removed in the cleanup pass.

5. Rebuild backup coverage

After cleanup, check what gaps remain and fill them:

```
# Which assets now have only one copy?
maki backup-status --at-risk --min-copies 2

# Copy at-risk assets to a new backup drive
maki backup-status --at-risk -q | maki relocate --target "New Backup" --log
```

6. If the catalog itself is lost

If the drive that held your MAKI catalog (the `.maki/` directory) fails, but your media volumes are intact, you can rebuild from scratch. The YAML sidecar files next to your media files are the source of truth — the SQLite database is just a derived cache:

```
# In a new directory
maki init

# Re-register your surviving volumes
maki volume add "Photos" /Volumes/PhotosDrive
maki volume add "Backup" /Volumes/BackupDrive

# Rebuild the catalog from sidecar files
maki rebuild-catalog
```

This restores all assets, variants, recipes, metadata, tags, ratings, and descriptions — everything that was stored in the YAML sidecars. Collections, stacks, and saved searches are also restored (they're stored as YAML in the catalog directory). Only preview thumbnails and AI embeddings need to be regenerated.

Next: Scripting – shell and Python scripting patterns for workflow automation.

For complete flag and option details on every maintenance command, see the Maintain Commands Reference.

Scripting

MAKI's CLI is designed for composability with shell scripts and external programs. Every command supports `--json` for structured output, `-q` for piping asset IDs, and stderr/stdout separation so human messages never pollute machine-readable output.

This chapter covers practical scripting patterns using bash and Python. For interactive multi-step workflows with variables and tab completion, see the Interactive Shell chapter — `maki shell` provides a REPL that eliminates much of the piping and quoting complexity shown here.

Bash Scripting

Batch Tagging by Path

Tag all assets from a specific session directory:

```
maki search -q "path:Capture/2026-02-15" | xargs -I{} maki tag {} session/february
```

Conditional Edits

Set a color label on all unrated images:

```
for id in $(maki search -q "type:image rating:0"); do
    maki edit "$id" --label Yellow
done
```

Rate all videos from a specific camera:

```
for id in $(maki search -q 'type:video camera:"NIKON Z 9"'); do
    maki edit "$id" --rating 3
done
```

Multi-Step Workflows

Import, auto-group, and generate smart previews for a new shoot:

```
#!/usr/bin/env bash
set -euo pipefail

SESSION="/Volumes/Photos/Capture/2026-02-28"

echo "Importing..."
maki import --smart --auto-group "$SESSION" --log --time

echo "Fixing roles..."
```

```
maki fix-roles --apply --log

echo "Done."
maki stats
```

Exporting for Delivery

Export rated picks to a client folder with sidecars:

```
maki export "rating:4+ collection:ClientProject" /tmp/delivery/ --include-sidecars --log
```

Mirror directory structure to a USB drive:

```
maki export "tag:portfolio" /Volumes/USB/export/ --layout mirror
```

Dry-run first to check what would be exported:

```
maki export "collection:Print" /tmp/test/ --dry-run --json | jq '{files: .files_exported, size: .total_bytes}'
```

Reporting with jq

Generate a tab-separated report of all 5-star assets with their tags:

```
maki search "rating:5" --json \
  | jq -r '.[] | [.original_filename, .created_at, (.tags // [] | join(", "))] | @tsv'
```

List volumes and their online status:

```
maki volume list --json | jq -r '.[] | "\(.label)\t\(if .is_online then "online" else "offline" end)\t\(.path // "
```

Count assets per format:

```
maki stats --types --json | jq -r '.types.variant_formats[] | "\(.format)\t\(.count)"'
```

Building Collections from Filters

Create a "Best of 2026" collection from highly-rated images:

```
maki col create "Best of 2026"
maki search -q "rating:4+ dateFrom:2026-01-01 dateUntil:2026-12-31" \
  | xargs maki col add "Best of 2026"
echo "Added $(maki col show 'Best of 2026' -q | wc -l | tr -d ' ') assets"
```

Verification and Health Checks

Weekly integrity check with notification:

```
#!/usr/bin/env bash
set -euo pipefail

LOG="/tmp/maki-verify-$(date +%Y%m%d).log"
if maki verify --log 2>"$LOG"; then
    echo "Verification passed" >> "$LOG"
else
    echo "VERIFICATION FAILURES DETECTED" >> "$LOG"
    # Add notification here (e.g., mail, Slack webhook)
fi
```

Find assets at risk (only one copy):

```
maki search -q "copies:1" | wc -l
```

Sync After External Edits

After editing files in CaptureOne or Lightroom:

```
# Detect what changed
maki sync /Volumes/Photos

# Apply catalog updates
maki sync /Volumes/Photos --apply --log

# Re-read modified XMP metadata
maki refresh /Volumes/Photos --log
```

Python Scripting

Python scripts can call `maki` as a subprocess and parse the `--json` output. This is useful for more complex logic that would be unwieldy in bash. The `scripts/` directory in the repository contains ready-to-use helper functions and example scripts.

Helper Functions

The `scripts/maki_helpers.py` module provides two functions for calling maki from Python:

```
from maki_helpers import maki_json, maki_ids

# Get parsed JSON output from any maki command
stats = maki_json("stats", "--tags")

# Get a list of asset IDs matching a search
ids = maki_ids("rating:5 tag:landscape")
```

The full source is at `scripts/maki_helpers.py`. Import it into your own scripts with `sys.path.insert(0, "scripts")` or copy the functions directly.

Example: Tag Analysis Report

Find tags that are only used once (potential typos or inconsistencies). Full script: `scripts/tag-analysis.py`.

```
python3 scripts/tag-analysis.py
```

```
stats = maki_json("stats", "--tags")
if stats and stats.get("tags"):
    for tag in stats["tags"].get("top_tags", []):
        if tag["count"] == 1:
            print(f"  Singleton tag: {tag['tag']}")
```

Example: Cross-Volume Backup Audit

Check which assets exist on only one volume. Full script: `scripts/backup-audit.py`.

```
python3 scripts/backup-audit.py --limit 20
```

```
at_risk = maki_ids("copies:1")
print(f"Assets with only 1 copy: {len(at_risk)}")

for asset_id in at_risk[:10]:  # show first 10
    details = maki_json("show", asset_id)
    if details:
        filename = details["variants"][0]["original_filename"]
        print(f"  {filename}")
```

Example: Batch Operations with Progress

Apply ratings based on an external CSV file. Full script: `scripts/batch-rate-from-csv.py`.

```
# Dry run
python3 scripts/batch-rate-from-csv.py ratings.csv

# Apply
python3 scripts/batch-rate-from-csv.py ratings.csv --apply
```

```
import csv

with open("ratings.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        asset_id = row["asset_id"]
        rating = row["rating"]
        result = subprocess.run(
            ["maki", "edit", asset_id, "--rating", rating],
            capture_output=True, text=True
        )
        if result.returncode == 0:
            print(f"  {asset_id[:8]}: rated {rating}")
        else:
            print(f"  {asset_id[:8]}: FAILED — {result.stderr.strip()}")
```

Example: Align Flat Tags to the AI Vocabulary *(Pro)*

Read the AI vocabulary YAML and emit `maki tag rename` / `maki tag split` commands that move flat catalog tags (e.g. `dog`, `sunset`, `concert`) into their canonical hierarchical home (`subject|animal|domestic`, …). Full script: `scripts/apply-vocabulary.py`. Covered in detail under Maintenance → Aligning Flat Tags.

```
# Preview commands against the active vocabulary
python3 scripts/apply-vocabulary.py

# Generate apply-mode commands and execute
python3 scripts/apply-vocabulary.py --apply | sh

# Test a candidate vocab file before adopting it in [ai].labels
python3 scripts/apply-vocabulary.py path/to/my-labels.yaml
```

Run after editing `my-labels.yaml` or after a batch import that brought in flat keywords from external XMP sidecars.

Real-World Example: Fix Orphaned XMP Files

The repository includes a complete Python script at `scripts/fix-orphaned-xmp.py` that demonstrates a real-world workflow automation.

Problem: XMP sidecar files that ended up in a different directory than their parent RAW file (e.g., XMP stayed in `Capture/` while the RAW was moved to `Selects/`). During import, these became standalone assets of type "other" instead of being attached as recipes.

Solution: The script uses maki's CLI to find orphaned XMPs, locates matching RAW files by filename stem, and moves the XMP files to the correct directory. It follows the standard two-phase pattern: make file changes, then let maki reconcile.

```
# Dry run — see what would be moved
python3 scripts/fix-orphaned-xmp.py --path 2026-02

# Apply changes
python3 scripts/fix-orphaned-xmp.py --path 2026-02 --apply

# Reconcile catalog with file moves
maki sync /Volumes/Photos --apply
maki fix-recipes --apply
```

Key patterns demonstrated in the script:

- **`maki search -q "type:other format:xmp"`** — find assets matching specific criteria
- **`maki show <id> --json`** — get full asset details including file locations
- **`maki volume list --json`** — enumerate volumes and their mount points
- **Dry-run by default** — `--apply` flag opts in to changes (following maki's convention)
- **Path scoping** — `--path` parameter limits the search to a subset of the catalog

---

Bulk Operations on a List of IDs

A common pattern: you have a file of asset IDs (one per line) and want to apply the same operation to all of them. There are several approaches, from simple to fast.

xargs (simple, one process per asset)

```
# Tag all assets in the list
cat ids.txt | xargs -I{} maki tag {} livestream

# Rate all assets
cat ids.txt | xargs -I{} maki edit {} --rating 3

# Delete (preview first, then apply)
cat ids.txt | xargs -I{} maki delete {}
cat ids.txt | maki delete --apply
```

This spawns a new `maki` process per line. Fine for hundreds of assets, slow for thousands.

Shell loop (simple, slightly faster)

```
while read id; do
    maki tag "$id" livestream
done < ids.txt
```

Same speed as xargs (one process per asset), but easier to add conditional logic.

maki shell script (fast, single process)

The interactive shell keeps the catalog open across commands. Generate a script file and run it in one shot:

```
# Generate a shell script from the ID list
awk '{print "tag " $0 " livestream"}' ids.txt > /tmp/retag.maki

# Execute all commands in a single maki process
maki shell /tmp/retag.maki
```

This is significantly faster for large batches (thousands of assets) because it avoids the overhead of opening the catalog for every command.

Commands that read IDs from stdin

Some commands accept asset IDs on stdin when no positional IDs are given:

```
# Delete assets from a list
cat ids.txt | maki delete --apply

# Relocate assets from a list
cat ids.txt | maki relocate --target "Archive" --log
```

Where do ID lists come from?

```
# Search results
maki search -q "tag:landscape rating:4+" > landscape-ids.txt

# Backup audit
maki backup-status --at-risk -q > at-risk-ids.txt

# External sources (CSV, spreadsheet, other tools)
cut -d, -f1 ratings.csv | tail -n +2 > ids.txt

# Saved from a previous session
maki shell -c 'search tag:concert' > concert-ids.txt
```

Tips

Avoid quoting pitfalls. When search queries contain inner quotes (e.g., `tag:"Fools Theater"`), use single quotes for the outer shell argument:

```
maki search -q 'tag:"Fools Theater"'
```

Use `--json` for reliable parsing. Human-readable output is designed for readability, not stability. Field positions, formatting, and labels may change between versions. JSON output is versioned and stable.

Stderr is for humans, stdout is for machines. Progress messages, warnings, and `--log` output go to stderr. When piping maki's output, only structured results appear on stdout.

Check exit codes. maki returns 0 on success and 1 on failure. Commands like `verify` return 1 if any mismatches are found, making them suitable for CI/cron health checks:

```
maki verify --volume "Photos" || echo "Integrity check failed!"
```

Dry-run first. Most destructive commands (sync, cleanup, auto-group, fix-roles) default to report-only mode. Always review the dry-run output before adding `--apply`.

Related Topics

- Interactive Shell – `maki shell` REPL with variables, tab completion, script files, and session management
- CLI Conventions – global flags, exit codes, and basic scripting patterns
- Browsing & Searching – search syntax and output format options
- Format Templates Reference – custom output templates for `--format`
- Search Filters Reference – all available filters
- REST API – programmatic access via HTTP

Interactive Shell

MAKI includes an interactive shell that keeps catalog state alive between commands, giving near-instant response times and enabling multi-step workflows with named variables, session defaults, tab completion, and script files. Instead of paying the startup cost of locating the catalog, loading configuration, and opening SQLite on every invocation, the shell does this once and reuses it for the entire session.

This chapter covers interactive use, variables, scripting, and practical workflows. For one-shot CLI scripting with bash or Python, see Scripting.

Starting the Shell

Interactive mode

Launch the shell from within a catalog directory:

```
maki shell
```

Output:

```
maki shell v3.2.2 — type 'help' or 'quit' to exit
photos>
```

The prompt shows the catalog name (the directory basename). Type any maki command without the `maki` prefix.

Run a script file

Execute a `.maki` script file and exit:

```
maki shell post-import.maki
```

Run a single command

Run one command and exit, useful in shell scripts or aliases:

```
maki shell -c 'search "rating:5 tag:landscape"'
```

The `-c` mode shares the same parsing and variable support as interactive mode, but exits after processing the command string.

Basic Usage

Inside the shell, type any maki subcommand directly. There is no need to type `maki` before each command:

```
photos> stats
photos> search "tag:portrait rating:4+"
  42 assets found
photos> show abc12345
photos> edit --rating 5 abc12345
```

Blank lines and comments

Blank lines are silently skipped. Lines starting with `#` are treated as comments:

```
photos> # This is a comment
photos>
photos> stats
```

History

The shell saves command history to `.maki/shell_history` inside your catalog directory. Use the up/down arrow keys to recall previous commands. History persists across sessions.

Keyboard shortcuts

Key	Action
Up/Down	Navigate command history
Ctrl-C	Cancel the current line (does not exit)
Ctrl-D	Exit the shell (same as `quit`)
Ctrl-A	Move cursor to start of line
Ctrl-E	Move cursor to end of line
Ctrl-K	Delete from cursor to end of line
Ctrl-W	Delete word before cursor
Tab	Trigger tab completion

Variables

The shell supports named variables that hold lists of asset IDs. Variables let you capture command results and reuse them in subsequent commands without re-running queries.

The implicit _ variable

Every command that produces asset IDs (such as `search`, `duplicates`, or `show`) automatically stores its results in the implicit _ variable. Use _ in the next command to refer to those results:

```
photos> search "tag:landscape rating:4+"
  12 assets found
photos> edit --rating 5 _
  12 assets edited
photos> tag --add "portfolio" _
  12 assets tagged
```

The _ variable is updated after every command that returns assets. It only expands when it appears as a standalone token – underscores inside words like `my_tag` or `_foo` are left alone.

Named variables

Store results in a named variable with the `$name = command` syntax:

```
photos> $picks = search "rating:5 date:2024"
  38 assets --> $picks
photos> $untagged = search "tags:none type:image"
  142 assets --> $untagged
```

Use variables in any command by referencing `$name`:

```
photos> tag --add "needs-review" $untagged
  142 assets tagged
photos> export $picks /tmp/best
  38 assets exported
```

You can also copy the last result into a named variable:

```
photos> search "rating:5"
  38 assets found
photos> $best = _
  38 assets --> $best
```

Variable rules

- Names start with `$`, followed by letters, digits, or underscores (`$picks`, `$session_2024`, `$batch1`)
- Variables hold asset ID lists – they are not re-evaluated queries
- Variables persist for the session and are cleared on `reload`
- Expanding an undefined variable leaves it as-is in the command
- **Position does not matter.** When a variable expands, its asset IDs are always placed at the end of the argument list (the trailing positional slot where commands expect asset IDs). This means `tag --add portfolio $picks` and `tag $picks --add portfolio` produce the same result. You never need to worry about where you place the variable in the command.

Listing and removing variables

Use `vars` to see all defined variables and their counts:

```
photos> vars
  Session defaults: --log
  _ = 38 assets
  $picks = 38 assets
  $untagged = 142 assets
```

Remove a variable with `unset`:

```
photos> unset $untagged
  Removed $untagged
```

Session Defaults

Use `set` to add flags that are automatically injected into every subsequent command. This is useful when you want `--log` or `--json` output for an entire session without typing it each time.

Setting defaults

```
photos> set --log
  Session default: --log
  Active defaults: --log
photos> set --json
  Session default: --json
  Active defaults: --json --log
```

The settable flags are: `--json`, `--log`, `--debug`, and `--time`.

After setting `--log`, every command will include progress output automatically:

```
photos> set --log
photos> generate-previews "rating:5"
  [1/38] abc12345 -- generated preview
  [2/38] def67890 -- generated preview
  ...
```

Removing defaults

```
photos> unset --log
  Removed session default: --log
```

Session defaults are also cleared by `reload`.

Tip: If a session default conflicts with what a particular command needs, you do not need to unset it first. The shell will not inject a flag that already appears in the command line.

Tab Completion

The shell provides context-aware tab completion powered by rustyline. Press Tab at any point to see suggestions.

What gets completed

Context	Completes	Example
First word	Subcommand names and built-in commands	`sea` -> `search`
`--` prefix	Common flags	`--j` -> `--json`
`$` prefix	Defined variable names	`$pi` -> `$picks`
`tag:` prefix	Tag names from the catalog	`tag:land` -> `tag:landscape`
`volume:` prefix	Volume labels from the catalog	`volume:Ph` -> `volume:Photos`
After `--volume`	Volume labels	`--volume Ph` -> `--volume Photos`

Volume labels containing spaces are automatically quoted in completions:

```
photos> search volume:<Tab>
volume:Photos  volume:"External SSD"  volume:Archive
```

Completion data is loaded from the catalog when the shell starts. Use `reload` to refresh it after adding new tags or volumes.

Script Files

A `.maki` script file is a sequence of commands, one per line. Scripts support the same syntax as interactive mode: variables, comments, blank lines, and `source`.

Basic script format

```
# post-import.maki -- run after each shoot day
import /Volumes/Cards/DCIM --log
auto-group
generate-previews --log
auto-tag --apply --log
describe "description:none" --apply --log
stats
```

Execute it:

```
maki shell post-import.maki
```

Variables in scripts

Variables work across lines, making multi-step workflows clean:

```
# curate-and-export.maki
$new = search "imported:today type:image"
auto-group $new
generate-previews $new --log
$picks = search "imported:today rating:4+"
tag --add "selects" $picks
export $picks /tmp/selects
stats
```

The `--strict` flag

By default, scripts continue after errors. Use `--strict` to stop on the first failure:

```
maki shell --strict critical-workflow.maki
```

In `--strict` mode, the shell exits with code 1 on any error. Without `--strict`, errors are printed with file and line number but execution continues:

```
critical-workflow.maki:3: Error: volume "Offline" is not mounted
```

The `source` command

Use `source` inside the shell (or another script) to run a script file inline, sharing the current session state:

```
photos> source post-import.maki
```

Variables defined in the sourced script are available after it finishes. Paths are resolved relative to the catalog root directory, or you can use absolute paths:

```
photos> source /path/to/shared/cleanup.maki
```

Session Management

`reload`

Re-reads `maki.toml` configuration, refreshes tab completion data (tags and volumes), and clears all variables and session defaults:

```
photos> reload
  Reloaded config, cleared variables and session defaults.
```

Use this after making external changes – editing `maki.toml`, importing from another terminal, or adding volumes.

`help`

Prints a quick reference of shell syntax, variable usage, session defaults, and built-in commands:

```
photos> help
```

`quit` / `exit`

End the session. Ctrl-D also exits. Command history is saved automatically on exit.

Quote Handling

The shell uses smart quote handling that lets search filter syntax pass through naturally. The rule is simple:

- **Quotes at the start of a token** are grouping quotes – they are stripped, and their content is treated as a single argument (standard shell behavior).
- **Quotes that appear mid-token** are preserved as part of the value (syntax quotes).

This means search filters with quoted values work without extra escaping:

```
photos> search text:"woman with glasses" tag:portrait
```

The shell splits this into two tokens: `text:"woman with glasses"` and `tag:portrait`. The quotes around `woman with glasses` are preserved because they start mid-token (after `text:`), so the search engine receives them intact.

Compare with grouping quotes, which wrap the entire token from the start:

```
photos> search "tag:landscape rating:4+"
```

Here the outer quotes are stripped (they start at the beginning of the token), producing two arguments: `tag:landscape` and `rating:4+`. This is equivalent to typing them without quotes.

You can mix both styles:

```
photos> search "type:image" camera:"NIKON Z 9" rating:4+
```

This produces three tokens: `type:image` (grouping quotes stripped), `camera:"NIKON Z 9"` (mid-token quotes preserved), and `rating:4+`.

Tip: If you are used to the standard CLI where you type `maki search 'text:"query"'`, inside the shell you can drop the outer quotes entirely and just type `search text:"query"`.

Tilde Expansion

The shell expands `~` to your home directory (`$HOME` on Unix, `%USERPROFILE%` on Windows) in any token:

```
photos> export $picks ~/Desktop/delivery
  38 assets exported to /Users/alice/Desktop/delivery
photos> export "rating:5" ~/exports/portfolio --zip
  Export complete: 12 files archived
```

Only `~` at the start of a token is expanded — `~` appearing elsewhere (e.g. inside a search query) is left as-is.

Built-in Commands

Most commands in the shell are maki subcommands run directly. A few commands are **built-ins** handled specially by the shell:

Command	Purpose
`export`	Export assets to a directory or ZIP archive (supports `$var` expansion for multi-ID export)
`preview`	Show asset previews in the terminal (supports `$var` and `_`)
`help`	Shell help and syntax reference
`reload`	Re-read config, clear variables and defaults
`set` / `unset`	Manage session defaults and variables
`source`	Run a script file inline
`vars`	List defined variables
`quit` / `exit`	End the session

Shell `export`

The `export` command is a built-in that supports variable expansion and all standard export flags:

```
photos> export <query|$var> <target> [--layout flat|mirror] [--all-variants]
        [--include-sidecars] [--dry-run] [--overwrite] [--symlink] [--zip]
```

When a variable expands to multiple asset IDs, all assets are exported in a single operation:

```
photos> $picks = search "rating:5 tag:landscape"
  24 assets --> $picks
photos> export $picks ~/Desktop/landscapes
  24 assets exported
photos> export $picks ~/Desktop/landscapes.zip --zip
  Export complete: 24 files archived
```

Practical Examples

Curate and export workflow

Browse a recent shoot, refine selections, and export:

```
photos> $candidates = search "date:2024-06 type:image rating:3+"
  287 assets --> $candidates
photos> $portraits = search "date:2024-06 tag:portrait rating:4+"
  42 assets --> $portraits
photos> tag --add "june-selects" $portraits
photos> export $portraits /tmp/june-portraits
  42 assets exported
```

Post-import script

Save this as `post-import.maki` and run after every import:

```
# post-import.maki
$new = search "imported:today"
auto-group $new
generate-previews $new --log
auto-tag "tags:none type:image" --apply --log
describe "description:none" --apply --log
stats
```

Run it:

```
maki shell post-import.maki
```

Or source it during an interactive session:

```
photos> import /Volumes/Cards/DCIM --log
photos> source post-import.maki
```

Audit workflow

Investigate and fix grouping issues:

```
photos> set --log
photos> $scattered = search "scattered:2 variants:3+"
  15 assets --> $scattered
photos> show $scattered
  ... review each asset ...
photos> split abc12345 --variants def678,ghi901
photos> auto-group $scattered
```

Batch tagging with variables

Tag assets from multiple searches without re-querying:

```
photos> $landscapes = search "tag:landscape rating:4+"
  85 assets --> $landscapes
photos> $portraits = search "tag:portrait rating:4+"
  42 assets --> $portraits
photos> tag --add "portfolio" $landscapes
photos> tag --add "portfolio" $portraits
photos> $portfolio = search "tag:portfolio"
  127 assets --> $portfolio
photos> export $portfolio /tmp/portfolio
```

Quick one-liners with -c

Use `-c` mode for cron jobs or shell aliases:

```
# Daily stats check
maki shell -c 'stats'

# Export today's picks
maki shell -c '$picks = search "imported:today rating:4+"; export $picks /tmp/daily'

# Verify with logging
maki shell -c 'set --log; verify'
```

Blocked Commands

Four commands are blocked inside the shell because they conflict with the shell session:

Command	Reason
`init`	Creates a new catalog; the shell is already inside one
`migrate`	Schema migration should be run standalone
`serve`	Starts a long-running web server that conflicts with the shell loop
`shell`	No nesting; use `source` to run script files instead

Attempting to run a blocked command prints a clear message:

```
photos> serve
'serve' cannot be used inside the shell.
```

Tips

The shell prompt tells you what is in memory. When named variables are defined, the prompt shows their counts in brackets:

```
photos [picks=38 untagged=142]> tag --add "review" $untagged
```

Variables are snapshots, not live queries. If you run `$picks = search "rating:5"` and then rate more assets, `$picks` still holds the original results. Run the assignment again to refresh.

Use `set --time` to profile commands. This is handy when exploring performance differences between queries during an interactive session.

Scripts are composable. A master script can `source` other scripts, building complex workflows from reusable pieces:

```
# nightly.maki
source post-import.maki
source cleanup.maki
source verify.maki
```

Ctrl-C is safe. It cancels the current input line but does not exit the shell or lose your variables. Use Ctrl-D or `quit` to exit.

Related Topics

- Scripting – bash and Python scripting with `maki --json` and `jq`
- Browsing & Searching – search syntax and output format options
- CLI Conventions – global flags, exit codes, and output conventions
- Search Filters Reference – all available search filters
- Configuration Reference – `maki.toml` settings including `[import]` and `[serve]` sections

Organizing and Culling

You shoot thousands of images but only show dozens. Every growing collection eventually faces the same challenge: separating the work you want to see from the work you don't – without losing anything permanently. This chapter covers practical strategies for culling and the default filter feature that ties them together.

Three Populations

In any large catalog, your assets fall into three groups:

1. **Picks** – curated keepers you actively browse, rate, and share.
2. **Backlog** – unreviewed imports you haven't looked at yet.
3. **Set aside** – images you've consciously decided to exclude from everyday browsing (rejects, duplicates, test shots, outtakes).

The goal of a culling workflow is to move assets between these populations efficiently, without deleting anything prematurely.

Rating vs. Curation

Star ratings and inclusion/exclusion serve different purposes. Conflating them leads to awkward workarounds.

Rating answers "how good is this image among my keepers?" It is a quality scale within your working set. A 3-star landscape and a 5-star portrait are both part of your active catalog – they just differ in quality or prominence.

Curation answers "should this image appear in my browsing views at all?" It is a visibility gate. A blurry test shot and a duplicate JPEG are not bad images that deserve 1 star – they are images you don't want to see while browsing.

Keep these concerns separate. Use ratings for quality within your keepers. Use tags, labels, or the absence of a rating for curation.

Workflow Approaches

There is no single correct approach. Choose one that matches how you work, and stay consistent within a project.

Tag-based culling

Add a `rest` tag to images you want to hide. Set a default filter of `-tag:rest` so they disappear from browsing. Everything else – rated or not – stays visible.

```
[browse]
default_filter = "-tag:rest"
```

Strengths: Simple to understand. Unreviewed imports remain visible by default, so you naturally encounter them. Opt-in exclusion means you only have to act on what you want to hide.

When to use it: Most users, especially if you import sporadically and want new work to show up immediately.

Rating-based culling

Rate your keepers 1–5. Leave rejects and backlog unrated. Set a default filter of `rating:1+` so only rated images appear.

```
[browse]
default_filter = "rating:1+"
```

Strengths: Clean browsing experience – only reviewed work appears. Ratings carry meaning across your entire catalog.

Trade-off: Unreviewed imports disappear until you rate them. You must do an explicit review pass before new work shows up in the browse grid.

Color-label workflow

Use color labels for workflow state rather than aesthetic preference. For example: Red = reject, Green = approved, Yellow = needs review.

```
[browse]
default_filter = "-label:Red"
```

Strengths: Visual and quick in the web UI (keyboard shortcuts r/o/y/g/b for labels). Leaves ratings free for quality scoring. Works well in teams where labels signal handoff state.

Combined approaches

You can combine filters. For example, show everything that is either rated or not yet tagged as `rest`:

```
[browse]
default_filter = "rating:1+ OR -tag:rest"
```

Or hide Red-labeled rejects and rest-tagged shots:

```
[browse]
default_filter = "-label:Red -tag:rest"
```

The Default Filter

The `[browse] default_filter` option in `maki.toml` applies a search filter automatically to all browse, search, and stroll views.

What it does

When set, every browse page load, search query, and stroll session starts with this filter pre-applied. Assets that don't match the filter are hidden from view – but they remain in the catalog and are fully accessible to CLI commands.

How to configure it

Add a `[browse]` section to your `maki.toml`:

```
[browse]
default_filter = "-tag:rest"
```

The value is any valid search filter string – the same syntax you use with `maki search`. See the Search Filters Reference for the full syntax.

Toggling in the web UI

The web UI shows a checkbox in the filter bar when a default filter is configured. Unchecking it temporarily disables the filter for the current session, letting you see everything. Re-checking it restores the filter. This makes it easy to peek at hidden assets without editing `maki.toml`.

Scope

The default filter applies to **browsing views only**: the browse grid, search results, and the stroll page. It does **not** affect operational commands like `maki export`, `maki describe`, `maki verify`, or `maki search` on the CLI. Those commands always operate on the full catalog unless you explicitly pass a query.

Examples for common workflows

```
# Hide images tagged "rest" (recommended starting point)
[browse]
default_filter = "-tag:rest"

# Show only rated images
[browse]
default_filter = "rating:1+"

# Hide Red-labeled rejects
[browse]
default_filter = "-label:Red"

# Show only images, hide videos and documents
[browse]
default_filter = "type:image"

# Combine: hide rest-tagged and Red-labeled
[browse]
default_filter = "-tag:rest -label:Red"
```

Practical Workflow Examples

Wedding photographer

1. Import the shoot: `maki import /Volumes/Cards/DCIM --log`
2. Quick scan in the web UI. Tag obvious rejects (out of focus, closed eyes, test flashes) as `rest` using keyboard shortcuts or batch select.
3. Rate keepers: 3 = good, 4 = strong, 5 = hero shots.
4. Deliver 4–5 star images to client: `maki export "rating:4+" /Volumes/Delivery/`
5. Build a portfolio collection from the best: `maki search -q "rating:5" | xargs maki col add "Wedding Portfolio"`

Default filter: `-tag:rest`

Travel and personal

1. Import after a trip: `maki import ~/Photos/Iceland/ --log`
2. Browse at leisure. Rate favorites as you go – no pressure to review everything immediately.
3. Revisit unrated images when you feel like it. Tag anything truly unwanted as `rest`.
4. Over time, the unrated backlog shrinks naturally.

Default filter: `-tag:rest`

Stock and archive

1. Import with descriptive tags: `maki import /Volumes/Stock/ --add-tag stock --log`
2. Use `maki describe` to generate AI descriptions for searchability.
3. Organize into collections by theme: "Architecture", "Food", "Travel".
4. Rate for commercial potential: 5 = lead image, 4 = strong, 3 = filler.
5. Default filter hides low-rated filler from the browse grid.

Default filter: `rating:3+`

Art and portfolio

1. Import selectively – only work you consider keeping.
2. Rate carefully for portfolio tiers:
   - 5 = hero image (exhibition, cover)
   - 4 = portfolio (website gallery)
   - 3 = extended portfolio (social media)
   - 2 = decent (personal archive)
   - 1 = keeper (not for sharing)
3. Build saved searches for each tier: `maki ss save "Portfolio" "rating:4+" --sort date_desc`
4. Use collections for specific series or exhibitions.

Default filter: `rating:2+` (hide the bottom tier from daily browsing)

Tips

Start simple. A single `rest` tag and `-tag:rest` default filter covers most needs. You can add ratings, labels, and more structure later without changing what you've already done.

The `rest` tag approach is recommended for most users because unreviewed images stay visible by default. You only act on what you want to hide, which is usually the smaller set.

Bulk-tag the remainder after a culling session. After rating your keepers from a shoot, tag everything unrated as `rest` in one command:

```
maki search -q "path:Capture/2026-03-15 -rating:1+" | xargs maki tag rest
```

This clears the backlog for that shoot without deleting anything.

Ratings are personal and subjective. Be consistent within a project, but don't worry about cross-project consistency. A 4-star wedding photo and a 4-star landscape don't need to be comparable.

Use saved searches for recurring views. Rather than remembering filter syntax, save your common queries:

```
maki ss save "Portfolio candidates" "rating:4+ -tag:rest" --sort date_desc
maki ss save "Unreviewed this month" "date:2026-03 -rating:1+ -tag:rest" --sort date_desc
maki ss save "Needs keywords" "-tag:rest rating:1+ tag:untagged" --sort date_desc
```

Don't delete – set aside. The `rest` tag is reversible. Deletion is not. You can always reconsider an image later by toggling off the default filter in the web UI.

Next: Tagging Guide – tagging principles, vocabulary structure, and practical techniques.

Previous: Interactive Shell | Back to Manual

Tagging Guide

Tags are the backbone of discoverability in a large photo catalog. Ratings tell you how good an image is; tags tell you what it *is*. A thoughtful tagging strategy pays compound interest – every tag you apply today makes thousands of future searches faster. This chapter covers principles, a recommended vocabulary structure, and practical techniques for building and maintaining tags across a growing collection.

For the mechanics of adding, removing, and searching tags, see Organizing Assets. This chapter focuses on *what* to tag and *how to think about it*.

Why Tags Matter

A 100-image portfolio doesn't need tags. A 10,000-image catalog can get by with folder names and memory. Beyond that, you need structured metadata to find what you're looking for – and tags are the most flexible tool for the job.

Tags serve three purposes:

1. **Search** – finding images by subject, location, technique, or concept. "Show me all concert photos from 2023 with dramatic lighting."
2. **Navigation** – browsing by category. The tags page gives you an overview of what your catalog contains.
3. **Interoperability** – IPTC keywords (`dc:subject`) are the industry standard for metadata exchange. Tags written by MAKI travel with your images to Lightroom, CaptureOne, stock agencies, and archives.

Principles

Tag what you see, not what you know

Primary tags should describe the visible content of the image: *sunset*, *concert*, *portrait*, *bridge*. Secondary tags can capture context that isn't visible: event names, project names, or moods. But the foundation is always observable content.

Be specific and general at the same time

Hierarchical tags handle this naturally. Tagging an image `subject|animal|bird|heron` automatically stores all ancestor paths (`subject`, `subject|animal`, `subject|animal|bird`) alongside the full tag – matching the CaptureOne/Lightroom convention. This means the image will match searches for `tag:subject`, `tag:subject|animal`, `tag:subject|animal|bird`, and `tag:subject|animal|bird|heron`.

Use one language consistently

Mixed-language tags (Konzert *and* concert, München *and* Munich) double your vocabulary without adding information. Pick one language for your tag vocabulary and stick with it. English is the practical default for descriptive terms – IPTC keywords, stock agencies, and most tools expect English.

The exception is **place names**: use **English for countries**, and **local names from regions downward**. Countries always have stable, unambiguous English names (`Germany`, `Italy`, `Japan`). Below that, local names avoid the constant judgment call of whether a place "has" an English name — famous cities do, small towns don't, and the cutoff is arbitrary. Local names match what you see on signs, maps, and receipts.

- Country: English — `Germany` not `Deutschland`
- Region: local — `Bayern` not `Bavaria`
- City: local — `München` not `Munich`, `Holzkirchen` stays `Holzkirchen`
- Venue: local — `Kulturbühne Hinterhalt`

Note on regions: whether to use English (`Bavaria`) or local (`Bayern`) for well-known regions is a matter of preference. Both are defensible. The important thing is to decide once and apply it consistently across your vocabulary. The recommendation above (local from regions down) is the simplest rule with the fewest judgment calls.

The same principle applies to venue names and event names — use the original language.

Use singular, lowercase forms – except proper nouns

Pick a convention and stay consistent:

- `concert` not `concerts` or `Concert`
- `mountain` not `mountains` or `Mountain`
- `black and white` not `Black And White` or `B&W`

The one exception is **proper nouns**: place names, person names, venue names, and project names keep their natural capitalization. This follows normal language rules that everyone already knows – no ambiguity about what gets capitalized:

- `location|Germany|Bayern|Gelting` – country in English, local names below
- `subject|nature|landscape|mountain` – generic terms lowercase
- `person|artist|Peter Schneider` – person name capitalized
- `project|Focus on Music` – project name capitalized

MAKI's tag search is case-insensitive for queries, but the stored tags should be consistent. A canonical form prevents duplicates from creeping in.

This convention applies to **all levels of the hierarchy**, including structural nodes. There is no visual distinction between "category" and "leaf" in casing – that's a UI concern handled by indentation and tree display, not by the data.

Separate content tags from workflow tags

Content tags describe the image: *landscape*, *wedding*, *jazz*. Workflow tags describe the image's state in your process: *unreviewed*, *to edit*, *portfolio candidate*. Keep them apart. Use a prefix like `@` for workflow tags, or better yet, use MAKI's ratings, color labels, and saved searches for workflow state instead of tags.

Don't tag what metadata already captures

EXIF records the camera, lens, date, and GPS coordinates. MAKI indexes file paths and folder names. Don't duplicate this information as tags – you'd just create a maintenance burden. Use search filters instead: `camera:Z9`, `lens:50mm`, `date:2024`, `path:Capture/2024-08-15`.

Recommended Vocabulary Structure

A well-structured tag vocabulary has two layers:

1. **Top-level facets** that partition the world into non-overlapping categories (the "what / where / who / how" of an image).
2. **Hierarchical terms** within each facet, going 2-4 levels deep.

The facets

```
subject          what is in the image?
event            which specific occasion is it from?
location         where was it taken?
person           who is in it? (named individuals)
technique        how was it made?
project          what project or assignment does it belong to?
color            what is the dominant color? (optional)
```

Not every image needs a tag from every facet – a landscape might only need subject and location; a studio portrait might need subject, person, and technique. `event` and `color` are opt-in facets for photographers who care about those axes.

Thinking in facets: when to promote a branch to top-level

Each facet answers a *distinct question* about the photo. That independence is the whole point. A concert photo taken at a specific wedding, of a specific musician, in a specific venue, with specific lighting, in a specific color palette carries a tag from each facet — and queries should be able to intersect any subset: "all photos of Alice at outdoor weddings in Bayern with golden-hour lighting."

When you're considering adding a new branch to your vocabulary, the test is simple: **what question does this tag answer about the photo?** If it answers a question that's genuinely distinct from what the existing facets answer, promote it to top-level. If it's a refinement of an existing axis, nest it. Facets are cheap to add; wrong nesting is expensive to fix later.

Two worked examples illustrate the choice.

Worked example: events

Many photographers start by filing events under `subject|event|*` — weddings, concerts, workshops. That feels natural at first, but there's friction: events are *date-driven instances*, while the rest of the subject tree is a *stable taxonomy*. New events get added every time one happens; the subject tree barely changes year over year. Jamming them together pollutes your subject vocabulary with names that are meaningful only in context ("Jane's Wedding 2025" is not a kind of thing photos can depict — it's a specific occasion).

The cleaner choice is a top-level `event` facet.

Pros:

- Keeps the subject tree stable and navigable. *Subject* answers "what's depicted"; *event* answers "what occasion is this from". Distinct questions, distinct facets.
- Event cardinality grows unboundedly (one new entry per wedding, per trip, per festival). Isolating it prevents the subject tree from getting skewed.
- Event names become their own namespace — easy to keep consistent (`event|wedding-jane-2025`, or grouped by year: `event|2025|wedding-jane`).
- Queries read clearly: `event:wedding-jane-2025 person:Alice` intersects two independent axes.
- Migration is cheap — `maki tag rename "subject|event|wedding-jane-2025" "event|wedding-jane-2025" --apply` handles the pure rename case. For the common situation where an old `subject|event|xxx` tag should become *both* the specific-occasion tag AND the generic scene-type tag, use `maki tag split`

`"subject|event|wedding-jane-2025" "event|wedding-jane-2025" "subject|event|wedding" --apply` to replace one tag with both in a single pass.

Cons:

- One more top-level to decide on when tagging.
- You need to distinguish two different questions a photo can answer about an event: *what kind of scene is this* (generic type) vs *which particular occasion is this from* (specific instance). They coexist cleanly — a photo of Jane's wedding carries **both** `subject|event|wedding` (it's a wedding scene) and `event|wedding-jane-2025` (specifically hers). The three layers:
  - `subject|performing arts|concert` — photo depicts a *performance*.
  - `subject|event|wedding` — photo depicts a *ceremony or non-performance gathering*.
  - `event|wedding-jane-2025` — photo is from *this specific occasion*.
- Edge cases where event blurs with subject — a parade photo is arguably both (the event *is* the depicted scene). Tag both; this is a feature, not a bug.

Recommendation: **add `event|` as a top-level facet** for specific instances, and keep `subject|event|*` for generic ceremony/gathering scene types that aren't performances. See the event hierarchy section below.

Worked example: color

Similar reasoning applies when you start tagging dominant color. You have three plausible homes:

- `subject|color|red` — treating color as a subject type
- `technique|style|red` — treating color as a photographic technique
- `color|red` — its own facet

Color isn't really a subject (unless the photo is *about* red — an abstract color study). It isn't really a technique (red isn't a choice you make, it's an attribute of the scene). What it *is* is an independent axis — orthogonal to what's depicted, where it was taken, how it was shot. So color gets its own facet: `color|red`, `color|blue`, `color|monochrome`.

One caveat: MAKI already has a first-class **color label** field (Red/Yellow/Green/Blue/Purple — the Lightroom-style editorial label). If you only ever tag the five standard colors, you're duplicating that label with more keystrokes. Content-color tagging earns its keep when you want finer distinctions (`teal`, `magenta`, `warm`, `cold`, `pastel`, `monochrome`) or dominant-color filtering across a catalog — which is a different use case from the workflow-oriented color label.

Worked example: striking objects that span subject and event

Some photo elements are *visually distinctive* (clearly subject material — they're what's in the frame) AND *almost always tied to a specific occasion* (clearly event material — they exist because of that gathering or festival). Fireworks, campfires, maypoles, parade floats, costumes, decorations, sparklers, bonfires — these all live on the boundary. The instinct to file them under one facet leaves the other axis unsearchable; the right move is to **always tag both**, with each tag answering its own facet's question.

Three concrete examples that come up often:

- **Fireworks.** Subject side: `subject|nature|sky|fireworks` (sibling to `aurora`, `lightning`, `sunset` — the photo *looks like* a sky shot, even though the source is human-made). Event side: `event|holiday|new-year`, `event|holiday|independence-day`, `event|festival|<name>|closing-fireworks`. A `setting|night` qualifier often comes along for the ride, useful when distinguishing dusk fireworks from full-dark.
- **Campfire.** Subject side: `subject|object|fire|campfire` (the `fire` parent earns its place because you'll eventually want bonfire, hearth, candle, gas lamp under it — useful for "show me all fire photos" cutting across kinds). Event side: `event|trip|<name>`, `event|gathering|<name>`, or just leave the occasion-facet empty if it's a generic camping shot.
- **Maypole / Maibaum.** Worth being deliberate here — the English "may pole" (ribbon-dance pole) and the Bavarian *Maibaum* (painted pole with guild shields, erected May 1, guarded against theft by the next village) are different cultural objects sharing one English name. If you photograph both,

keep them distinct: `subject|tradition|maypole` (general) with optional `|maypole|bavarian` and `|maypole|english` children, OR commit to the local name as the leaf: `subject|tradition|maibaum`. Event-side pair: `event|tradition|maibaum-aufstellen` for the erection-ceremony photos specifically. Location facet usually rides along too, since many Bavarian villages have a permanent Maibaum on the central square — `location|Germany|Bayern|<village>` co-occurs naturally.

The general pattern: if a photo element is **both** distinctive enough to search for visually **and** tightly coupled to an occasion, give it a home under `subject|...` (for queries like "show me all fireworks photos") and pair it with `event|...` at tagging time (for "all fireworks at New Year"). Neither tag alone gives you both axes; both together does.

A few neighbours that follow the same pattern, with suggested homes:

- Parade float → `subject|vehicle|float` + `event|parade|<name>`
- Costume / traditional dress → `subject|object|costume|<style>` + relevant `event|`
- Christmas tree → `subject|object|decoration|christmas-tree` + `event|holiday|christmas|<year>`
- Lantern festival lanterns → `subject|object|light|lantern` + `event|festival|<name>`
- Sparklers → `subject|object|light|sparkler` (or `subject|nature|sky|sparklers` if photographed sky-style) + appropriate `event|`

These leaves don't need to ship in the default vocabulary on day one — add them when your catalog demands them, and the hierarchy organically grows along the axis your photos actually exercise.

Worked example: when *not* to add a tag — "action"

Some words look like obvious tags but actually compress three different questions into one ambiguous string. "Action" is the textbook case. Walked through carefully, it usually splits into:

1. **"There's a recognisable activity in the frame."** A runner, a surfer, a dog jumping, a dancer mid-leap. → Already covered by `subject|person|activity|*` (or `subject|animal|...`, or `subject|performing arts|...`). The activity-ness is *implicit* in the activity tag. Adding a separate `action` here is redundant and dilutes the activity branch.

2. **"The photo's defining quality is motion / a decisive moment, regardless of subject."** A panning shot of a passing train, a frozen-mid-jump portrait, a wave crashing — the dynamism *is* the photo, not whatever happens to be moving. → A genuine cross-cutting qualifier, sibling to `style` / `condition` / `mood`:

   ```
   subject
   └── motion
       ├── action      (dynamic moment, decisive moment)
       ├── still       (statically posed, calm composition)
       ├── panning     (subject sharp, background streaked)
       ├── frozen      (high-shutter mid-action)
       └── flowing     (silky-water long exposure, blurred crowd)
   ```

   Combines with any subject branch — `subject|nature|water|wave` + `subject|motion|action` is a different photo than `subject|nature|water|wave` + `subject|motion|flowing`.

3. **"I used a specific motion-emphasis technique."** Long exposure, panning, fast-shutter freeze, slow-sync flash. → That's `technique|motion|*`, not subject. Describes *how the shot was made*, not what's in it.

The recommendation:

- Don't add a bare `action` tag — it's not searchable for anything specific.
- For sense (1), use the specific activity tag. "All my action photos" becomes a saved search like `tag:subject|person|activity OR tag:subject|animal OR tag:subject|performing arts`.
- For sense (2), `subject|motion|*` earns its keep only if you actively want to slice the catalog by motion-quality independent of subject. If you're not sure, you don't need it.

- For sense (3), `technique|motion|*` follows the same rule as other technique tags — only worth it if you tag technique systematically.

The wider lesson: **a tag that means "kinda this, kinda that" isn't a tag, it's a placeholder**. When you reach for one, the question to ask isn't "where should it go in the hierarchy" but "which of the three questions am I actually answering, and is there already a facet that handles it?". Often the answer is "yes, and I don't need a new tag at all".

A few more words that look like tags but usually decompose:

- `candid` → either `subject|mood|*` (relaxed, joyful) or implicit in `subject|person|portrait` vs `subject|person|activity`. Rarely needs its own tag.
- `mood` (as a tag, not a facet) → use one of the mood leaves (`subject|mood|melancholic` etc.). The word `mood` itself answers nothing.
- `nature` → use the specific nature subbranch (`subject|nature|landscape`, `subject|nature|flora`). Bare `nature` is just the parent facet name.
- `interesting`, `nice`, `cool` → not tags. These are ratings or color labels in disguise. Use MAKI's rating (1–5) or color label fields.

The general rule

Ask: *what distinct question does this tag answer about the photo?* If the answer matches an existing facet, nest it. If it's a new question — a new way someone might want to slice the catalog — promote it to top-level.

subject hierarchy

Subject is the largest facet. A starting structure for photography:

```
subject
├── nature
│   ├── landscape    (mountain, forest, beach, desert, valley, ...)
│   ├── flora        (flower, tree, leaf, moss, mushroom, ...)
│   ├── sky          (sunset, sunrise, cloud, fog, storm, aurora, ...)
│   └── water        (ocean, river, lake, waterfall, ...)
├── animal
│   ├── mammal       (deer, fox, bear, seal, whale, ...)
│   ├── bird         (eagle, owl, heron, kingfisher, swan, ...)
│   ├── reptile      (lizard, snake, turtle, frog, ...)
│   ├── invertebrate (butterfly, dragonfly, bee, spider, snail, ...)
│   ├── aquatic      (fish, jellyfish, coral, crab, ...)
│   └── domestic     (dog, cat, horse, cow, sheep, ...)
├── urban
│   ├── architecture (building, bridge, skyscraper, tower, facade, ...)
│   └── street       (road, alley, graffiti, neon sign, shop front, ...)
├── vehicle          (cross-cuts every setting — not constrained to urban)
│   ├── car
│   ├── truck
│   ├── motorcycle
│   ├── bicycle
│   ├── train
│   ├── airplane
│   └── boat
├── person
│   ├── portrait     (headshot, environmental portrait, candid, ...)
│   ├── group        (family, couple, crowd, ...)
```

```
│   └── activity     (dance, sports, hiking, cooking, ...)
├── performing arts
│   ├── concert      (live music, musician, singer, guitarist, ...)
│   ├── theatre      (actor, stage set, rehearsal, costume, ...)
│   └── dance        (ballet, contemporary, ...)
├── event            (ceremony / gathering scene types — not performances)
│   ├── wedding
│   ├── exhibition   (art exhibition, photo exhibition, ...)
│   ├── workshop     (photo workshop, craft workshop, ...)
│   ├── sports event (marathon, match, tournament, ...)
│   └── festival     (food, cultural, craft — music festivals go under performing arts)
├── object
│   ├── food         (coffee, wine, cake, cooking, restaurant, ...)
│   ├── instrument   (guitar, piano, drum, saxophone, ...)
│   └── other        (book, camera, flag, candle, sculpture, ...)
├── concept
│   ├── travel
│   ├── fashion
│   ├── documentary
│   └── abstract
├── style          (visual era/aesthetic of the SUBJECT — see note below)
│   ├── vintage
│   ├── modern
│   ├── retro
│   ├── rustic
│   ├── industrial
│   └── classic
├── condition      (physical state of the subject)
│   ├── abandoned
│   ├── ruined
│   ├── restored
│   ├── weathered
│   ├── pristine
│   └── under construction
└── mood           (emotional quality of the scene)
    ├── dramatic
    ├── serene
    ├── playful
    ├── mysterious
    ├── melancholic
    └── joyful
```

You don't need all of these on day one. Start with the top two levels and add leaf nodes as your collection demands them.

Subject qualifiers — style, condition, mood: These three branches are *cross-cutting qualifiers* that can be combined with any other subject category. They describe *how* the subject looks or feels, not *what* it is. Tag a vintage car photo as `subject|vehicle|car` + `subject|style|vintage`. Tag an abandoned factory as `subject|urban|architecture` + `subject|condition|abandoned` + `subject|mood|mysterious`.

Don't confuse `subject|style` (the subject's visual era — vintage, modern, retro) with `technique|style` (the photographic technique — black and white, high key, infrared). A vintage car shot in black and white gets both: `subject|style|vintage` + `technique|style|black and white`.

When categories overlap: `subject|person` describes the *composition* of the photo (a portrait, a group scene, someone doing an activity), not the profession or context. A musician on stage is `subject|performing`

`arts|concert|musician` — the photo shows a performance, not a person doing a generic activity. The same musician backstage in a headshot is `subject|person|portrait`. The rule: ask "what is the photo *about*?" — if the answer is "a concert" use `performing arts`, if it's "a portrait of someone" use `person`.

Similarly, `subject|person|activity` is for general activities (hiking, cooking, sports) where the activity defines the image. A wedding guest dancing → `subject|person|activity|dance`. A ballet performance → `subject|performing arts|dance|ballet`. Context matters more than the literal content.

location hierarchy

```
location
└── country
    └── region
        └── city
            └── venue   (optional, for recurring locations)
```

The structural levels are generic terms and lowercase. The actual values are proper nouns and capitalized:

Example: `location|Germany|Bayern|Gelting|Kulturbühne Hinterhalt`

Note how `location` (generic category) is lowercase, while `Germany` (country in English), `Bayern`, `Gelting`, and `Kulturbühne Hinterhalt` (local names) keep their natural capitalization.

Keep location tags for *significant* or *recurring* places. Don't tag every street corner – GPS data and folder paths handle that. Location tags are most useful for:

- Recurring venues (concert halls, studios, favorite spots)
- Travel destinations
- Places without GPS data (scanned film, older cameras)

person hierarchy

```
person
├── family
├── friend
├── artist         (musician, performer, model)
├── public figure
└── ensemble       (named groups of people)
    ├── band
    ├── choir
    ├── orchestra
    └── team
```

The `person` facet covers both individuals and named groups. Individual names go under their relationship category: `person|artist|musician|Peter Schneider`. Named groups go under `ensemble`: `person|ensemble|band|Alice Viola Trio`, `person|ensemble|choir|Vocalitas München`.

This distinction matters: a concert photo might be tagged both `person|ensemble|band|Alice Viola Trio` (the group) and `person|artist|musician|Alice Viola` (an individual member, possibly via face recognition).

Note: `subject|person` is a separate facet that describes *what the photo shows* (portrait, group, activity), not *who is in it*. An image can be `subject|person|portrait` AND `person|artist|musician|Peter Schneider`.

For collections with many named individuals, consider using MAKI's face recognition system instead of (or alongside) person tags. Face recognition scales better and doesn't require manual tagging. Use person tags for individuals who don't appear in photos (event organizers, clients) or as a complement to face recognition for search flexibility.

technique hierarchy

```
technique
├── style       (black and white, high key, low key, infrared, ...)
├── exposure    (long exposure, double exposure, HDR, ...)
├── lighting    (natural light, flash, studio, golden hour, blue hour, ...)
├── composition (minimalist, symmetry, leading lines, ...)
└── effect      (bokeh, motion blur, silhouette, reflection, lens flare, ...)
```

project hierarchy

```
project
├── 365 pictures 2018
├── Bricking Bavaria
├── Guido Karp Workshop LA 2019
└── (your projects)
```

Project names are proper nouns and keep their original capitalization. The category `project` itself is lowercase. Project tags are inherently personal and won't follow any standard. Use them to group assets that belong together by assignment or creative intent rather than by subject or location.

event hierarchy

```
event
├── wedding-jane-2025
├── workshop-la-2019
├── concert-alice-viola-2024-11-15
└── (your events)
```

Specific occasions — weddings, workshops, trips, named concerts — go here, one tag per event. You have two naming styles to pick from:

- **Flat with date in name**: `event|wedding-jane-2025` — simplest, easy to type, each event is one leaf tag.
- **Grouped by year**: `event|2025|wedding-jane`, `event|2024|workshop-la` — browsable by year in the tags tree, and year becomes a queryable ancestor (`tag:event|2025` finds everything from 2025).

Pick one style and stay consistent. Flat naming works well for most people; year grouping pays off when you have many events per year and want to browse chronologically.

What belongs under top-level `event|` vs `subject|event|*`:

- **Specific instances** (`event|wedding-jane-2025`, `event|fuji-trip-2023`) — always top-level `event|`. This axis answers *which particular occasion is this photo from?*
- **Generic ceremony / gathering scene types** (`subject|event|wedding`, `subject|event|exhibition`, `subject|event|sports event`) — stay under the subject tree. This axis answers *what kind of scene is this?*
- **Performances** (concerts, theatre, dance) — `subject|performing arts|*`, not `subject|event|*`. Performances are distinct from ceremonies and gatherings; they get their own subject branch.
- **Recurring annual events** — treat each year as its own specific instance (`event|christmas-2024`, `event|christmas-2023`), or group them (`event|recurring|christmas|2024`) if you want a single branch for all Christmases.

A photo from Jane's wedding typically carries both `subject|event|wedding` and `event|wedding-jane-2025`: one tag describes the scene, the other the occasion.

Event names are inherently personal — you won't find standard event taxonomies. Name them however you'll remember them, and keep the style consistent.

color (optional)

If you tag dominant color as a searchable attribute of the image, use it as its own facet:

```
color
├── red
├── orange
├── yellow
├── green
├── blue
├── purple
├── pink
├── brown
├── black
├── white
├── grey
├── monochrome
├── pastel
├── warm
└── cold
```

Low-cardinality vocabulary (15-30 terms covers almost everything). Keep it flat — a hierarchy like `color|warm|red` adds no real structure, since "red" is already obviously warm.

When to use it: content-color tagging earns its keep when you want finer distinctions than MAKI's built-in color label (Red/Yellow/Green/Blue/Purple — which is an editorial workflow field, not a content descriptor), or when you want to filter a catalog by dominant color. If you only ever tag the five standard colors and only use them as a workflow marker, use the color label field instead — that's what it's for.

How Many Tags?

Per image

Aim for **5-15 tags per image**:

- 2-4 subject tags (what's in the image)
- 1-2 location tags (where, if relevant and not covered by GPS)
- 0-2 person tags (who, if relevant)
- 1-2 technique tags (how, if noteworthy)
- 0-1 event tag (which specific occasion, if this photo belongs to one)
- 0-1 project tag
- 0-1 color tag (if you tag dominant color)

Example: a concert photo might carry `subject|performing arts|concert`, `subject|performing arts|concert|guitarist`, `location|Germany|Bayern|Gelting|Kulturbühne Hinterhalt`, `technique|lighting|stage lighting`, `event|concert-alice-viola-2024-11-15` — five intentional tags that you choose.

These counts refer to the **leaf-level tags you consciously assign**, not the total number of tags stored. MAKI automatically expands each hierarchical tag to include all ancestor paths (e.g., `subject|performing arts|concert|guitarist` also stores `subject`, `subject|performing arts`, and `subject|performing arts|concert`). The five tags in the example produce 15 stored entries — but you only think about the five leaves.

Fewer intentional tags means poor discoverability. More than 20 usually means you're tagging noise or duplicating information that belongs elsewhere.

Total vocabulary

For a serious amateur with 100k+ images:

Category	Expected range
`subject/` terms	150-300
`location/` entries	50-200 (grows with travel)
`person/` names	varies (consider face recognition)
`technique/` terms	30-50
`project/` entries	varies
`event/` entries	varies (grows with life)
`color/` terms	15-30 (if used)
Total (excl. names/events)	**250-500**

If your unique tag count is climbing past 1,000 (excluding person names and event-specific tags), it's time to review for duplicates, typos, and over-specificity.

Auto-Tagging and the Label Vocabulary

MAKI's auto-tagging uses a vision-language model (SigLIP) to suggest tags based on visual content. It works by matching image features against a list of text labels – the **AI vocabulary**.

The AI vocabulary is different from your **catalog vocabulary** (the tag tree this chapter is otherwise about), and the two interact:

	AI vocabulary	Catalog vocabulary
Purpose	What the AI model can *see*	What you want to *find*
Structure	Flat list of labels, each mapped to one or more hierarchical tags	Hierarchical tree
Size	100-400 labels	300-500+ tags
Content	Visually recognizable concepts	Includes abstract, contextual, personal terms
Examples	`sunset`, `concert`, `dog`, `bridge`	`subject|event|workshop`, `project|Bricking Bavaria`

The AI suggests tags from a flat vocabulary; the mapping converts each hit into the canonical hierarchical tag MAKI actually applies. Both vocabularies ship with sensible defaults, and the defaults are coherent – every tag the AI vocabulary maps to is also a leaf in the catalog vocabulary, so the two stay in sync.

Configuring labels

The AI vocabulary lives in a single YAML file. Keys are the labels the model sees; values are the hierarchical tag(s) to suggest when a label scores above threshold:

```yaml
# my-labels.yaml
sunset:
  - subject|nature|sky
  - technique|lighting|golden hour
concert: subject|performing arts|concert
portrait: subject|person|portrait
dog: subject|animal|domestic
abstract: subject|concept|abstract
# null = "leave the label flat, no canonical mapping"
weather:
```

Point `maki.toml` at it:

```toml
[ai]
labels = "my-labels.yaml"
threshold = 0.3
```

A higher threshold (0.3-0.5) produces fewer but more confident suggestions. A lower threshold (0.1-0.2) casts a wider net at the cost of more false positives.

To start from the built-in default (which already maps 96 photographic labels into the canonical hierarchy described in this chapter), dump it and edit:

```
maki ai export-vocabulary --default > my-labels.yaml
```

Legacy `.txt` flat-list files (one label per line, identity mapping) still load — the loader picks format by extension. To migrate an existing flat list to YAML with identity mappings as a starting point:

```
maki ai export-vocabulary > my-labels.yaml   # converts the active .txt file
```

Label design tips

- Use **short, concrete phrases**: `mountain` works better than `mountainous terrain`.
- Match the prompt template: with the default `"a photograph of {}"`, labels should read naturally after "a photograph of" – "a photograph of a sunset" (good), "a photograph of a subject|nature|sky|sunset" (bad).
- **Don't use hierarchy in label keys** – the model sees the key as text and scores the image against it. Use flat terms for keys; put hierarchy in the values.
- Include **genre labels** that vision models handle well: `concert`, `portrait`, `landscape`, `macro`, `street photography`, `architecture`.
- Include **compositional labels**: `silhouette`, `reflection`, `bokeh`, `long exposure`.
- Skip things the model can't see: person names, event names, abstract concepts, workflow state.
- Test and iterate: `maki auto-tag --asset <id> --log` on a few representative images shows what the model picks at your threshold.

When AI specificity meets your catalog hierarchy

The shipped default vocabulary maps to canonical leaves: `dog` and `cat` both land on `subject|animal|domestic`, every kind of book/jewelry/furniture on `subject|object|other`. The mapping is deliberately conservative – the default catalog vocabulary stops at the genus level for most branches, so the default AI vocabulary maps no deeper than that. It's a scaffold; no orphan tags appear in autocomplete out of the box. (One exception: the vehicle branch has species-level leaves out of the box – see strategy 3 below for the reasoning.)

But the AI saw something more specific. The model picked `dog` because it actually *looks* like a dog; collapsing that to `domestic` loses information that your tag tree may want to capture. Three coherent strategies, depending on how much your catalog has grown:

1. Conservative. Keep the default mapping. Tag genus-level (`subject|animal|domestic`, `subject|object|other`). Filter by what's in the photo via search rather than tag hierarchy depth. Good when you're starting out, or when species-level distinctions don't drive how you browse.

2. Extend both vocabularies together. If your catalog already has `subject|animal|domestic|dog`, `subject|animal|domestic|cat`, etc. as real tags, point the AI mapping at those leaves so AI suggestions land where you want them. Two files to edit:

```yaml
# my-labels.yaml — point AI labels at the leaves
dog:   subject|animal|domestic|dog
cat:   subject|animal|domestic|cat
horse: subject|animal|domestic|horse
```

```yaml
# vocabulary.yaml — declare those leaves in the catalog vocabulary
subject:
  animal:
    domestic:
      - dog
      - cat
      - horse
      - cow
      - sheep
```

The two files evolve together: every leaf you tag actively becomes a leaf in `vocabulary.yaml`, and the AI mapping points at it. The pair stays consistent.

3. Promote a branch. When the canonical nesting doesn't fit, lift the branch to its own facet. Worked example, taken from MAKI's own defaults: vehicles aren't always urban — a tractor in a field, a fishing boat at sea, an airplane in flight don't belong under an "urban" branch. The default vocabulary therefore puts vehicles in their own top-level subject facet:

```yaml
subject:
  vehicle:
    - car
    - truck
    - motorcycle
    - bicycle
    - train
    - airplane
    - boat
```

and the default AI mapping points at those leaves directly:

```yaml
car:     subject|vehicle|car
bicycle: subject|vehicle|bicycle
train:   subject|vehicle|train
airplane: subject|vehicle|airplane
boat:    subject|vehicle|boat
```

Apply the same reasoning to your own catalog when you notice a leaf that you keep wanting to use across multiple parent branches — that's the signal it's an independent axis, not a refinement. See Thinking in facets for the underlying principle.

Best practices

- **Start with the default.** Run `maki ai export-vocabulary --default > my-labels.yaml` and look through it before customizing. The default is a working baseline that's coherent with the default catalog vocabulary.

- **Add leaves you actually browse by.** Don't pre-emptively add every species name the AI knows – the catalog vocabulary should reflect axes you genuinely filter on, not theoretical completeness. Add a leaf the first time you'd want to filter by it.
- **Keep both files in sync.** When you add a leaf to `vocabulary.yaml`, update `my-labels.yaml` so the AI label maps to it. When you remove a leaf, scan `my-labels.yaml` for orphan references. (A future helper command may automate the consistency check.)
- **Use fan-out sparingly.** One label can map to multiple tags (sunset → sky subject AND golden-hour lighting). Useful for labels that genuinely span two facets, noisy otherwise – if every "wedding" suggestion adds three tags you didn't want, you've over-fanned.
- **Null where it doesn't fit.** Labels with no clean canonical home (weather, seasonal, generic terms) stay `null` and surface flat. Decide what to do with each *when you see it on a real photo*, not in the vocabulary file ahead of time.

Renaming legacy flat tags into hierarchy

If your catalog has flat tags from earlier AI runs (before the YAML mapping existed), batch-reorganize with `maki tag rename`:

```
maki tag rename "concert" "subject|performing arts|concert"
maki tag rename "dog" "subject|animal|domestic|dog"
```

Then update `my-labels.yaml` so future AI suggestions land at the new path directly.

Industry Standards and References

IPTC Photo Metadata

The IPTC Photo Metadata Standard defines the `dc:subject` field (keywords) and `lr:hierarchicalSubject` (hierarchical keywords). MAKI reads and writes both. Staying within this framework ensures your tags survive export to any IPTC-aware tool.

How MAKI stores hierarchical tags (the roundtrip)

Prerequisite: Your photo editor must be configured to write XMP sidecar files. Most tools don't do this by default — see Working with External Tools for per-tool settings.

CaptureOne and Lightroom write two representations of the same hierarchy to XMP:

- **`dc:subject`**: flat individual components — `location`, `Germany`, `Bayern`, `Wolfratshausen`
- **`lr:hierarchicalSubject`**: all ancestor paths — `location`, `location|Germany`, `location|Germany|Bayern`, `location|Germany|Bayern|Wolfratshausen`

On import, MAKI keeps only the `lr:hierarchicalSubject` entries (the pipe-separated paths) and discards the flat `dc:subject` components that are part of any hierarchy. This avoids storing redundant standalone tags like `Germany` alongside `location|Germany`.

On writeback, MAKI regenerates both formats: flat components for `dc:subject`, ancestor paths for `lr:hierarchicalSubject`. CaptureOne and Lightroom see exactly what they expect — no data loss in the roundtrip.

Internally, MAKI stores: `location`, `location|Germany`, `location|Germany|Bayern`, `location|Germany|Bayern|Wolfratshausen`. Searching `tag:Germany` matches `location|Germany` via prefix matching. Standalone `Germany` is not stored because it's redundant.

Controlled vocabularies

Several controlled vocabularies exist for photo tagging:

- **IPTC Media Topics** – ~1,200 terms in a 5-level hierarchy, 17 top-level categories. Freely available in multiple formats and 13 languages. Designed for news but applicable to photography. The most authoritative free taxonomy. Available at iptc.org/standards/media-topics.
- **David Riecks' Controlled Vocabulary Keyword Catalog (CVKC)** – ~11,000 terms, the most comprehensive photography-specific vocabulary. Commercial ($70/year). Includes Lightroom-compatible keyword lists. See controlledvocabulary.com.
- **Open keyword lists on GitHub** – several community-maintained lists in Lightroom's tab-indented text format, freely available:
  - LightroomKeywordHierarchy (MIT license, based on IPTC subject codes)
  - Open Keyword Catalog

You don't need to adopt any of these wholesale. Use them as references when building your own vocabulary – they show proven ways to partition and name categories.

Cleaning Up an Existing Catalog

If you've been tagging for years across different tools and approaches, you likely have duplicates, inconsistent casing, mixed languages, and orphaned tags. Here's a phased approach to cleanup.

Phase 1: Audit

Start by understanding what you have:

```
# Total unique tags
maki stats --tags

# Find singleton tags (used only once -- often typos)
python3 scripts/tag-analysis.py

# List all tags with usage counts
maki stats --tags
```

In the web UI, the `/tags` page shows your complete tag tree with counts. Sort by count to see your most-used tags; sort by name to spot near-duplicates.

Find under- and over-tagged assets with the `tagcount:` filter (counts intentional leaf tags, not auto-expanded ancestors):

```
maki search "tagcount:0" --format ids | wc -l     # completely untagged
maki search "tagcount:1 rating:4+"                # under-tagged keepers
maki search "tagcount:10+"                        # suspiciously heavy
maki search "tagcount:2-3 date:2024"              # recent light tagging
```

`tagcount:0` is the most useful for catching gaps — any asset that escaped tagging shows up.

Phase 2: Normalize

Fix the mechanical issues first – these can be done in bulk:

1. **Case normalization** – merge `Concert` into `concert`, `Blues` into `blues`, etc.
2. **Language normalization** – pick one language and merge translations.

3. **Remove dead workflow tags** – bulk-import markers, migration artifacts, tags from previous tools that no longer serve a purpose.

```
# Rename a tag across the entire catalog (case-insensitive matching)
maki tag rename "Concert" "concert"
maki tag rename "Konzert" "concert"
maki tag rename "Munich" "München"
```

All matching is case-insensitive — `maki tag rename "Concert" "concert"` catches "Concert", "CONCERT", and "concert".

Phase 3: Structure

Once the duplicates are resolved, introduce hierarchy:

```
# Move flat tags into hierarchy (ancestors are auto-expanded)
maki tag rename "concert" "subject|performing arts|concert"
maki tag rename "landscape" "subject|nature|landscape"
maki tag rename "München" "location|Germany|Bayern|München"
```

When renaming to a hierarchical tag, all ancestor paths are automatically added. For example, the last command replaces "München" with `location|Germany|Bayern|München` and also adds `location`, `location|Germany`, and `location|Germany|Bayern`.

If you have existing tags that were created before ancestor expansion was added, run `maki tag expand-ancestors --apply` to retroactively add the missing ancestor paths.

Do this for your most-used tags first (the top 50-100 tags cover most of your catalog). The long tail can be restructured gradually.

Phase 4: Prune

Review tags with very low usage (< 5 assets). For each one, decide:

- **Merge** into a broader tag (e.g., `nighthawk` → `bird`)
- **Fix** a typo (`landscpe` → `landscape`)
- **Keep** if it's genuinely specific and useful
- **Remove** if it's noise

Phase 5: Enrich

Run auto-tagging on untagged or under-tagged assets to fill in descriptive tags:

```
# Auto-tag all images (already-tagged assets get additional suggestions)
maki auto-tag "type:image" --apply --log

# Auto-tag a specific shoot
maki auto-tag "path:Capture/2024-08-15" --apply --log
```

Review suggestions in the web UI – accept good ones, dismiss bad ones, and adjust your label vocabulary or threshold if needed.

Reviewing suggestions before committing (web UI)

`maki auto-tag --apply` is unconditional: every suggestion above the configured threshold is written to every matching asset. For a curated collection – a vacation set, a wedding shoot, anything where false positives accumulate noticeably – the **Review tags...** button on the browse toolbar gives you a per-tag confirmation step without forcing you to select assets one by one.

Flow:

1. Filter the browse page down to the assets you want to review (or select a subset). Click **Review tags...**.
2. MAKI computes AI suggestions for every asset in scope – roughly 1-2 s per image on CPU, faster with the `ai-gpu` feature. Progress shows in the toast, same as Embed or Auto-tag.
3. When the job finishes, the review modal opens. The left pane lists every suggested tag sorted by candidate count (most-suggested first). The right pane shows the candidate thumbnails for the selected tag, all pre-checked, each with its confidence score and – when the AI's underlying label differs from the hierarchical tag MAKI mapped it to – a small "from: *skyscraper*" hint.
4. Uncheck false positives. Drag the **Threshold** slider to bulk-toggle by confidence. Tick **Skip already-tagged** to hide assets that already carry the tag.
5. Click **Apply *building*** to write the tag to the checked set. The row strikes through and the next un-reviewed tag opens. Use **Skip** to leave a tag for later.

The collection pass runs at half your configured `[ai].threshold` so the slider has room to drag both above and below your normal auto-tag floor. Suggestions are held in server memory on the job's record (page-reload survival within the session), so you can close the modal, finish another task, and reopen the review via the toast badge – a server restart drops them.

Do's and Don'ts

Do:

- Define your vocabulary before you start tagging. Spend an hour on structure to save hundreds of hours of cleanup.
- Use hierarchy. It scales; flat lists don't.
- Tag new imports immediately. The longer you wait, the less you remember about the context.
- Use auto-tagging for broad descriptive categories, then refine manually.
- Review your tag list periodically. A 15-minute audit every few months catches drift early.

Don't:

- Don't create tags you'll use fewer than ~10 times. One-off tags are noise.
- Don't tag camera or lens information – that's EXIF data.
- Don't tag dates or folder paths – those are already searchable.
- Don't go deeper than 4-5 hierarchy levels. Deeper trees are hard to navigate and maintain.
- Don't use tags for workflow state. Use ratings, color labels, or the `rest` tag pattern from Organizing & Culling.
- Don't aim for perfection. A 90% consistent catalog that you actually maintain beats a theoretically perfect taxonomy that you abandon after a week.

Tag Input in the Web UI

The browse filter bar's tag input and the asset-detail tag chip both share the same autocomplete dropdown. Two behaviours worth knowing:

Middle-of-hierarchy matches stay visible. Type `subject` and the dropdown shows `subject` itself even if your catalogue also has `subject|nature`, `subject|nature|landscape`, etc. as deeper matches. Earlier MAKI versions hid the parent row in favour of the more specific children, which made it hard to land at exactly the level you intended. Now any node whose own name (any segment) matches the query stays in the list alongside its descendants.

Drill into a hierarchy by clicking the prefix. Each suggestion that has a hierarchy renders the prefix in a dimmed style. Clicking that dimmed prefix narrows the autocomplete to that level — useful when a query produces dozens of suggestions across unrelated parents (`bird` matches `subject|nature|animal|bird`, `event|birding-trip`, etc.) and you want to focus on one branch. Clicking the leaf segment commits the full path as usual.

The same dropdown is reused on the tag rename / split modals on the tags page, so the behaviour is consistent everywhere a tag value is typed.

The Vocabulary File

MAKI creates a `vocabulary.yaml` file in your catalog root when you run `maki init`. This file defines your planned tag hierarchy — a skeleton of categories and terms you intend to use. MAKI reads it and offers these tags in autocomplete, even before you've tagged a single asset with them.

Why it matters

Without a vocabulary file, autocomplete can only suggest tags that already exist on at least one asset. When you add the first image of a new category, there's no guidance — you have to remember (or look up) your planned hierarchy. The vocabulary file bridges this gap: your planned structure is always available in autocomplete.

Editing the file

The file uses a nested YAML tree format:

```yaml
subject:
  nature:
    - landscape
    - flora
    - sky
  animal:
    - bird
    - mammal
technique:
  lighting:
    - natural light
    - stage lighting
```

Keys are hierarchy nodes, arrays are leaf lists. Edit the file in any text editor to add, remove, or reorganize categories as your collection grows. MAKI reads it on every command — changes take effect immediately.

Bootstrapping from existing tags

If you already have a catalog with organic tags, export the current tag tree as a starting point:

```
maki tag export-vocabulary
```

This generates `vocabulary.yaml` from your existing tags, grouped into a nested tree. Edit it to add planned categories and remove unwanted entries.

To see what the built-in default vocabulary looks like (without mixing in your catalog's tags), use `--default`:

```
maki tag export-vocabulary --default --output defaults.yaml
```

This is useful after a MAKI upgrade that adds new default categories (e.g. `subject|style`, `subject|condition`, `subject|mood`) — you can compare the defaults with your existing vocabulary and merge what you want. The `--default` flag ignores both your catalog tags and your existing `vocabulary.yaml`; it outputs only the built-in starter vocabulary.

Sharing your vocabulary with Lightroom and Capture One

The vocabulary you curate in MAKI is also useful inside your RAW processor — so that culling or initial tagging sessions in Lightroom or Capture One autocomplete against the same hierarchy. Export the tree as a tab-indented keyword text file:

```
maki tag export-vocabulary --format text --prune \
    --output ~/Desktop/maki-keywords.txt
```

Then import it:

- **Lightroom Classic**: *Metadata → Import Keywords…* and pick the file. Existing keywords are merged; the hierarchy is preserved.
- **Capture One**: *Image → Keywords → Import Keywords → Keyword Text File*, pick the file. The keywords become available in the Keywords tool and in autocomplete.

Re-run the export whenever your catalog grows enough new tags to be worth re-syncing. With `--prune`, you only push the subset that's actually in use; drop `--prune` to push the full planned hierarchy.

Annotating exports with asset counts

Pass `--counts` to attach the per-tag asset count to every entry. In YAML output the count appears as a `# N assets` trailing comment; the file is still valid YAML and MAKI's autocomplete loader ignores comments. Useful for spotting candidates to consolidate or retire:

```
maki tag export-vocabulary --counts
```

```yaml
subject:  # 5 assets
  nature:  # 3 assets
    - landscape  # 2 assets
    - bird  # 1 asset
- legoland  # 1 asset
```

In **text** format (Lightroom / Capture One) the flag is silently ignored — those tools reject comments. In **json** format every node already has a `count` field; the flag is implied.

Programmatic export (JSON)

For dashboards, integration scripts, or any tool that finds the keyword-text or YAML formats awkward to walk, export as nested JSON:

```
maki tag export-vocabulary --format json --prune > tags.json
```

Each node has a `count` field plus an optional `children` map keyed by child segment name. Leaf nodes omit `children`:

```
{
  "subject": {
    "count": 5,
    "children": {
      "nature": {
        "count": 3,
        "children": {
          "landscape": {"count": 2},
```

```
          "bird": {"count": 1}
        }
      }
    }
  },
  "legoland": {"count": 1}
}
```

The structure mirrors the tag tree exactly — root keys are top-level tags, nested keys are sub-tags, and counts are the per-asset totals from the catalog. Identical to `maki tag export-vocabulary --format yaml --counts` in information content; the JSON shape is just easier to consume from code.

Exporting from the web UI

The tags page (`/tags`) carries an **Export vocabulary...** button next to the page title — a one-click UI alternative to running `maki tag export-vocabulary` from a shell. Clicking it opens a small modal:

- Format radio: **YAML** (default), **Keyword text** (Lightroom / Capture One), or **JSON**.
- `Annotate with per-tag asset count` — equivalent to `--counts`. Auto-disabled when "Keyword text" is selected (LR / C1 reject comments).
- `Prune planned-but-unused entries` — equivalent to `--prune`.
- `Built-in default vocabulary only` — equivalent to `--default`. Greys out the counts and prune options since neither applies (the built-in tree has no asset counts).

Clicking **Download** triggers a browser save dialog with the appropriate filename (`vocabulary.yaml` / `.txt` / `.json`). The output is byte-identical to what the CLI command produces with the same flags — same rendering pipeline (`src/vocabulary.rs`), same sanitisation warnings (the Lightroom-import sanitisation step runs server-side; if any of your tags contain commas, semicolons, or XML entities, the export will silently rename them in the file but the catalog stays untouched).

Vocabulary vs. auto-tagging labels

The vocabulary file and the auto-tagging label file (`labels` in `[ai]` config) serve different purposes:

	`vocabulary.yaml`	AI label file
Purpose	Autocomplete guidance	Vision model classification
Format	Nested YAML tree	Flat text, one term per line
Content	Full hierarchy including abstract/personal terms	Only visually recognizable concepts
Used by	CLI and web UI autocomplete	`maki auto-tag`

Putting it all together

Every principle in this chapter plays out on a single photo. Consider a shot from Jane's wedding reception in Bayern:

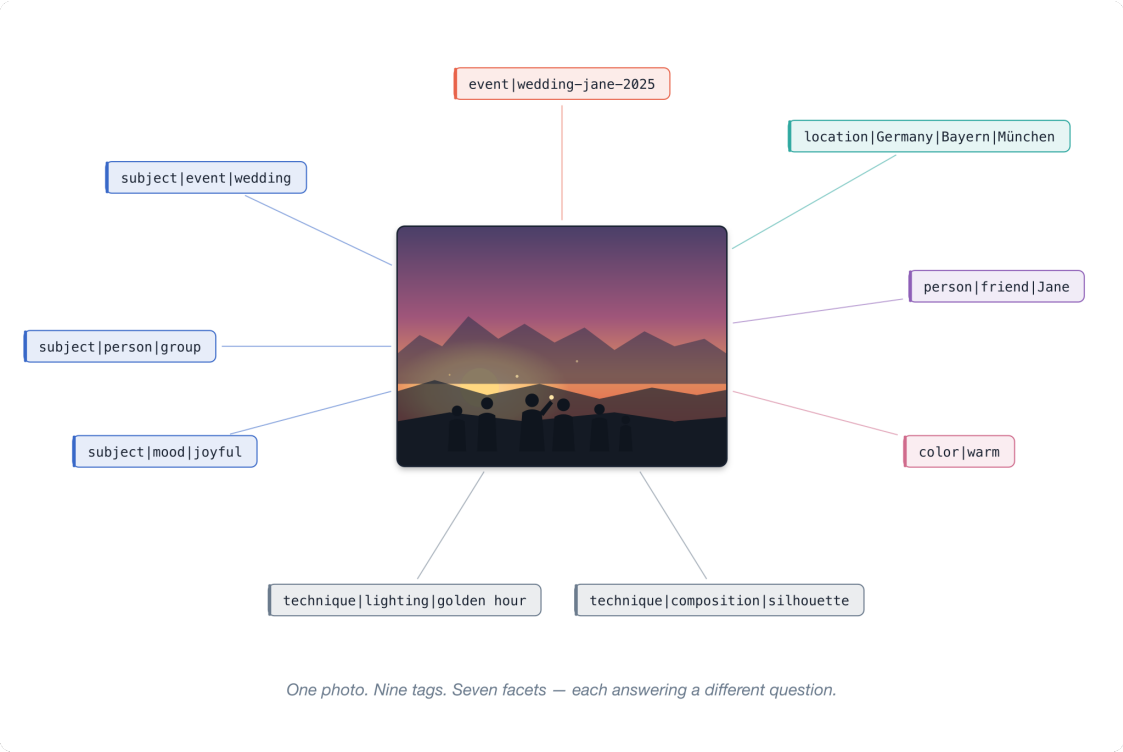


Figure 14: One photo, nine tags, seven facets — each answering a different question.

Nine leaf tags across seven facets. Ancestor expansion bumps the stored count to roughly fifteen — but you only think about the nine leaves you picked.

Each tag answers a different question about the same image:

- **Subject tags** describe *what the photo depicts* — a wedding scene, a group of people, a joyful mood. These are conceptually the deepest tree; most of your vocabulary lives here.
- **Event** ties the photo to a *specific occasion* — Jane's 2025 wedding, not any wedding. A date-driven instance that doesn't belong in the stable subject taxonomy.
- **Location** says *where in the world* it was taken — structured from country (English) down to local venue names.
- **Person** names *who* is in the frame. On a large catalogue, face recognition handles most of this automatically; person tags are useful for named individuals who won't be picked up by faces (or for precision when faces are ambiguous).
- **Technique** captures *how* it was made — golden hour lighting, silhouette composition. Two tags from the same facet but different sub-axes, because technique has multiple meaningful ones (lighting, style, composition, effect).
- **Color** records the *dominant palette* — useful when you later want to assemble a warm-toned or monochrome gallery across the catalog.
- **Project** would tie it to an assignment or body of work (a photo book, a 365 project) if applicable. Not every photo has one.

A search like `event:wedding-jane-2025 person:Jane color:warm` intersects three independent axes. You can recombine them however the question of the moment demands — `technique:golden` across every event, or

`color:monochrome subject:nature|landscape` across every location. The catalog becomes a dense multidimensional space you can slice any way you want, rather than a tree you have to walk in a fixed order.

That's the whole point of faceted tagging: invest up front in the vocabulary structure, and every photo you've ever tagged becomes available to arbitrary future queries you didn't think to ask when you tagged it.

Quick-Start Checklist

If you're starting fresh or resetting your tagging approach:

1. Choose a language (English recommended)
2. Choose a case convention (lowercase, proper nouns capitalized)
3. Define your top-level facets — start with the five essentials (subject, location, person, technique, project) and add `event` and/or `color` if you'll use them
4. Edit `vocabulary.yaml` in your catalog root to reflect your planned hierarchy
5. Write 50-100 tags for your most common subjects
6. Save them as your label vocabulary for auto-tagging (`labels = "my-labels.txt"` in `maki.toml`)
7. Set a confidence threshold (`threshold = 0.3` is a reasonable start)
8. Tag your next import using the new vocabulary
9. Run auto-tagging on a test batch and review the suggestions
10. Expand `vocabulary.yaml` as new subjects appear
11. Schedule a quarterly review of your tag list and vocabulary file

---

Next: Visual Discovery – face recognition, similarity search, and strolling: finding photos by what they look like.

Previous: Organizing & Culling | Back to Manual

Visual Discovery

Metadata — tags, ratings, dates, filenames — tells you what you know about an image. But sometimes you need to find photos by what they **look like**: *who is in this shot*, *which other frames show the same scene*, *what other photos does this one remind me of*. Visual discovery is MAKI's answer to that class of question.

Three features share a common foundation (neural-network embeddings that map an image to a vector of numbers, where visually similar images end up near each other):

- **Face recognition** — "who is in this photo?" Detects faces, maps each to an identity vector, and lets you search and group by person.
- **Similarity search** — "what other photos look like this one?" Powers the `similar:` filter, burst detection, and variant discovery.
- **Stroll** — "show me photos visually adjacent to this one, let me wander." A spatial navigation mode that uses similarity to build a graph of your catalog.

This chapter covers all three as workflows rather than controls. For the mechanics (commands, flags, config keys) see Organizing Assets, Browse & Search, and Web UI.

Why Visual Discovery Matters

Tags describe what you can put into words. Visual discovery finds what you recognize without having to name it.

Consider: you remember a sunset photo with a silhouetted figure, but you never tagged "silhouette" — and "sunset" alone returns 800 candidates. Open any silhouette you can find, hit the *similar* button, and the top 30 results are probably the right neighbourhood. You found what you needed without ever putting the feeling into words.

Three scenarios where this pays off:

1. **Finding forgotten photos.** Your catalog is larger than your memory. Visual similarity surfaces images that metadata alone would never connect.
2. **Organising bursts and variants.** RAW+JPEG already get grouped, but twenty near-identical frames from a concert need visual grouping — the filenames alone don't tell you which frames are duplicates.
3. **Connecting people across decades.** A face recognised in a 2003 shot and a 2024 shot are the same person despite everything else having changed.

None of these replace tags. They complement them: use tags for concepts you can name, visual discovery for what you can only recognise.

Face Recognition

MAKI uses two ONNX models in sequence: **YuNet** to find faces in an image (bounding boxes + five landmark points for each face), and **ArcFace ResNet-100** to produce a 512-dimensional identity embedding for each face after aligning it to a canonical pose. Two faces of the same person end up near each other in this 512-dim space; faces of different people end up far apart.

The Short Version

```
maki faces detect --query "*" --apply          # find faces on every asset (first run)
maki faces cluster --apply                     # group similar faces into people
# → Open /people in the web UI
# → Rename the big clusters: Alice, Bob, Carol
# → Use the "Merge suggestions" panel to fold splinter clusters
```

That gets you 80% of the value. The rest of this section is about when and why to deviate from that flow.

Detect vs. Cluster vs. Assign: Picking a Workflow

Face recognition has three distinct steps:

1. **Detect** — scan image pixels, find where the faces are, extract an identity embedding. Happens once per asset; `--force` re-runs it (e.g. after upgrading the recognition model).
2. **Cluster** — look at all unassigned face embeddings and group the similar ones into "people." Each cluster becomes an unnamed person ("Unknown") that you can rename later.
3. **Assign** — attach a specific face to a specific person (named or unnamed). Can be automatic (via clustering) or manual (via the asset detail page or a batch merge).

The interesting design choice is *when* to cluster versus assign per-asset. Three scenarios:

Bulk first-pass. You just imported 5 000 photos from a family archive. Run `faces detect --query "*" --apply`, then `faces cluster --apply` from the web UI. You'll end up with a handful of big clusters per regular subject (mother, grandmother, siblings) plus lots of singletons for one-off appearances. Name the big ones; ignore the rest until they matter.

Targeted subject. You're preparing a gift collection for a friend — you need every photo that includes them. Detect faces on that scope, cluster within the scope (`faces cluster --query "tag:family" --apply`), rename the result. Much faster than clustering the whole catalog.

One-off group photo. You just detected faces on a single old group photo. Clustering won't help here — it only creates groups of two or more similar faces, and each face in the photo is a singleton until more photos of that person exist. Just assign each face manually on the detail page: the face-assign combobox has a type-to-filter dropdown that always offers "+ Create new person" as its last option. (*This is the case we get asked about most often — clustering needs at least two similar faces to form a group, by design.*)

The Clustering Threshold

Clustering is governed by `face_cluster_threshold` — the minimum centroid similarity for two clusters to merge (0.0 to 1.0, higher = stricter). The default is 0.35, tuned for typical portrait-heavy catalogs using MAKI's aligned FP32 pipeline. Your mileage will vary:

- **0.3 and below** — aggressive; big clusters, risk of mixing similar-looking people (close family members, kids of the same age)
- **0.35 (default)** — a usable middle ground
- **0.45 and above** — tight; many singletons and small splinters, very unlikely to merge different people

If clustering is producing nonsense (one giant cluster with everyone in it, or hundreds of tiny clusters that should be merged), the threshold is probably wrong for your data. But before you guess, look at the actual similarity distribution.

Diagnosing Bad Clusters

`maki faces similarity` prints percentile statistics and a histogram of pairwise similarities for a scope. A healthy distribution is **bimodal**:

```
Histogram (10 buckets, -0.34 – 1.00):
  -0.343 – -0.209     138
  -0.209 – -0.075    4718  ████████
  -0.075 –  0.060   18237  ████████████████████████████████
   0.060 –  0.194    8710  ███████████████          ← different-people hump
   0.194 –  0.328     958  ██
   0.328 –  0.462    2036  ███                       ← valley between humps
   0.462 –  0.597    3350  ██████
   0.597 –  0.731    4032  ███████
   0.731 –  0.865    3287  ██████                    ← same-person hump
   0.865 –  1.000     590  █
```

The valley around 0.3–0.4 is where the threshold belongs. If instead you see a single narrow hump (everything at 0.85+, or everything between 0.9 and 1.0), the embeddings are not discriminating and no threshold will help — that points to a pipeline problem: bad crops, wrong model, a preprocessing mismatch.

`maki faces dump-aligned --query "..." --limit 30` saves the 112×112 aligned crops that actually get fed to the recognition model. Eyes should be roughly level, faces centered, nothing mirrored or squashed. If they look wrong, recognition cannot work no matter what you do downstream.

Merge Suggestions

Clustering almost always produces **splinter clusters**: a main cluster of 90 faces for Alice, plus a 5-face satellite of Alice taken under different lighting or at a much different age. The splinter's centroid is close enough to the main one that you'd call them the same person at a glance, but not close enough to have merged automatically.

The `/people` page surfaces these. The "Merge suggestions" panel above the grid lists candidate pairs with their match scores. Each suggestion shows both sides with an arrow indicating default merge direction (target → source), a swap button, and merge / dismiss actions. If neither side is named, the larger cluster becomes the target; if one is named, the named one wins so you keep your naming work.

Aim to keep this panel empty. Each morning's cleanup is usually five clicks.

Confidence Filtering

Not every detected face is worth clustering. Blurry faces, profile shots, and faces occluded by hair, sunglasses, or other people produce noisy embeddings that pollute clusters. The `--min-confidence` flag on `cluster` drops low-quality detections before they can cause damage:

```
maki faces cluster --min-confidence 0.85 --apply
```

The default is 0.7. Raising it to 0.8 or 0.85 usually produces cleaner clusters at the cost of leaving some real faces unassigned — they remain as unassigned faces on their assets, still visible and assignable manually. This is almost always the right trade.

Maintenance

Face recognition is not a one-time operation. New imports bring new faces; new faces sometimes fit existing clusters and sometimes form new ones. A reasonable rhythm:

Weekly (or after any big import):

```
maki faces detect --query "*" --apply    # rescans only assets with no faces yet
```

```
maki faces cluster --apply              # clusters any newly-unassigned faces
```

A note on what "already detected" means: without `--force`, detection skips any asset that has already been scanned, regardless of whether faces were found. Once an asset is scanned it's marked as done in both the catalog and the YAML sidecar — so a landscape or product shot is scanned exactly once and never again unless you explicitly re-scan with `--force`.

This also means that if you delete a bad detection (via the trash-can button on the detail page), the asset stays marked as scanned — the detection won't come back on the next run. The trade-off is that `--force` is the only way to re-run detection on assets you've already scanned (useful after a model upgrade or if you want to try a different `--min-confidence`).

Occasionally, when clustering quality feels off: - Run `faces similarity` on a scope to check the distribution - Re-cluster with a different `--threshold` or `--min-confidence` - Review the merge-suggestions panel and fold remaining splinters - Use `faces clean --apply` to sweep up orphan unassigned faces after pruning

After upgrading the recognition model (e.g. a MAKI release with pipeline changes): run `maki faces status` to see how many faces are from the old model, then `faces detect --force` on those scopes to re-embed with the new model. Old embeddings live in a different vector space and are automatically skipped by clustering until replaced.

Similarity Search

The same SigLIP (or SigLIP2) embeddings that power the text-to-image `text:` search also power image-to-image similarity. Every asset with a SigLIP embedding can become the starting point for a "what looks like this?" query.

The Use Cases

Finding burst shots and variants. Ten frames of the same concert moment, shot at 5 fps, with slightly different expressions. They share no tags, they have no filename convention to group them. But their embeddings cluster tightly. Open one, click the "Similar" button on the detail page, and the others are in the top results. Select them all and click "Stack" to collapse them behind a pick.

Rediscovering forgotten work. You remember a particular kind of light, a particular mood, a particular composition — but you can't name it. Open any photo that has the feeling you're after, browse by similarity, and the catalog gives you everything that reminds *it* of that photo.

Quality-controlling a stack. You've stacked 30 frames from a sequence, designated the pick. Is the pick actually the best one? Browse by similarity from the pick — if you immediately see a visibly better frame, reorder the stack.

The Mechanics

From the CLI:

```
maki search "similar:72a0bb4b"                # top 20 most similar
maki search "similar:72a0bb4b:50"             # top 50
maki search "similar:72a0bb4b min_sim:85"     # only >=85% similar
maki search "similar:72a0bb4b tag:portrait"   # similar AND portrait-tagged
```

In the web UI: click **Similar** on the asset detail page, or press `b` in the browse grid to go "browse similar" from the focused card.

Finding the Original Behind a Preview

The similarity source does not have to be in the catalog. Someone sends back a downsized JPEG from last year's export and asks for the full-resolution file; you have a screenshot of a picture you know you took; a contact sheet frame needs its RAW. Importing the throwaway just to search from it would pollute the catalog, so `search --image` takes the file directly:

```
maki search --image ~/Downloads/IMG_4711_preview.jpg
maki search --image ~/Downloads/IMG_4711_preview.jpg "min_sim:90"    # near-duplicates only
maki search --image frame.png "tag:events|wedding" --json            # narrowed, with scores
```

Two things happen. First the file is hashed and looked up among the catalog's variants — a byte-identical copy is answered without even loading the model and shows as `100% exact` at the top. Then the file is encoded with the SigLIP model and ranked against the embedding index like any `similar:` search. A resized copy of an original scores very close to 100% and stands well clear of unrelated images; a re-cropped or heavily edited derivative lands lower, which is why the scores are shown rather than a threshold applied. RAW and video query files are rendered through the preview generator first.

In the web UI the same search is the **Find by image** button in the search row (always visible, no selection needed), or simply drop an image file anywhere on the browse page, or paste one from the clipboard. A query pill above the grid shows the thumbnail and filename, notes an exact copy when there is one, and clears with its ×. All other filters keep working on top.

Why `min_sim:` Matters

Raw similarity search returns the top N no matter how dissimilar. With a small catalog that's fine. With 100 000 assets, the top 20 similar results might all be visually related; with 10 assets, you'd get back images that have nothing to do with the source.

`min_sim:85` floors the results at 85% similarity. Use it to cut off the long tail when you're doing exact-match work (finding burst duplicates, stacking variants). Skip it when you're exploring ("show me anything even loosely related").

Prerequisites

Similarity needs SigLIP embeddings. Either: - `maki import --embed` to embed at import time (cheap if you do it in batches) - `maki embed --query "..." --apply` to backfill existing assets

Embeddings are keyed per `(asset_id, model_id)`, so switching models only generates the missing ones — no `--force` needed.

Stroll

Stroll is MAKI's most underappreciated feature. It's what you use when you don't know exactly what you're looking for, or when you want your catalog to show you something you'd forgotten you had.

The Spatial Metaphor

A regular browse grid is a list: sorted by date, rating, filename. Stroll is a graph: you stand on one asset, and its visual neighbours arrange themselves around it. Click a neighbour to step onto it, and *its* neighbours appear. You wander through your catalog the way you'd wander through a museum — following what draws your eye, with no predetermined path.

Three Modes, Three Moods

- **Nearest** — the N most visually similar assets, same result every time. Use for finding burst duplicates, stacking candidates, or verifying that a similarity search is stable. This is the default.
- **Discover** — N randomly-sampled assets from a larger pool of similar ones (pool size configurable via `[serve] stroll_discover_pool`, default 80). Each visit produces a different neighbour set. Use for serendipity: breaking out of a tight visual cluster, reminding yourself of work you hadn't thought about in a year, or just browsing for pleasure.
- **Explore** — shows assets ranked N+1 through 2N by similarity, skipping the nearest neighbours entirely. Use when "nearest" keeps returning variants of the same shot and you want to see what else is *near-ish* in the catalog.

Switch modes with the dropdown at the top of the stroll page. The current mode persists per-session; next time you open stroll you're in the same mode you left it in.

When to Stroll vs. Search

Use **search** when you know the thing's name (a tag, a date, a person, a file pattern). Search gets you precise answers to precise questions.

Use **similarity** when you have one example and want its siblings. Precise question, imprecise filter.

Use **stroll** when you don't have a specific question. You're just looking — for inspiration, for rediscovery, for the moment when your catalog shows you something that stops you in your tracks.

Stroll is not a navigational tool. It's a looking tool.

Stroll + Filters

The stroll page inherits whatever filters were active on the browse page when you started strolling. Start from a browse filtered to `tag:portrait rating:4+` and your stroll only wanders among 4-star portraits. Start from `date:2015` and you're strolling through one year of your work. This turns stroll from "random walk of the whole catalog" into "focused walk of the subset I'm currently interested in."

Common Problems

"I ran cluster and nothing happened"

You detected faces on one photo, then clicked *Cluster unassigned faces* hoping MAKI would create new people for the unknowns. It won't: clustering needs at least two similar faces to form a group. If each face is a singleton (one new person appearing in one photo), the cluster command correctly skips all of them.

Use the face-assign combobox on the asset detail page instead. Type a name — even a placeholder like "man with beard" — and click *+ Create new person*. Later photos of the same person will attach to that person when you assign them, and clustering will start forming real groups once there are multiples.

"Clustering produced one giant cluster"

Threshold too low. Either raise `--threshold` (try 0.5 or 0.6), or raise `--min-confidence` to 0.85 so only high-quality detections participate. Run `faces similarity` on the scope to see the actual distribution before you guess.

"Clustering produced lots of tiny splinters"

Threshold too high, or the pipeline's embedding quality is poor. First check `faces dump-aligned` — do the crops look healthy? If yes, lower `--threshold` gradually (0.35 → 0.3 → 0.25) and re-cluster. If the crops look bad (mirrored, rotated, too small), check the detection settings and that you're on the current recognition model (`faces status`).

"Merge suggestions keep suggesting the same pair I dismissed"

Dismissals are per-session (stored in `sessionStorage`). Close the tab and they're gone. If a pair is genuinely not the same person but keeps getting suggested, the only permanent fix is to make one of them more distinct — assign more faces to the correct one so its centroid shifts away from the other.

"Similarity search returns nonsense"

Two possibilities. First, the asset may not have an embedding — `maki show <id>` shows whether an embedding is present. If not, run `maki embed --asset <id> --apply`. Second, the model may not be suited to the content: SigLIP works well on natural photography, less well on screenshots, diagrams, or heavily-abstract work.

"Stroll always shows me the same 12 neighbours"

You're in *Nearest* mode. Switch to *Discover* for randomized sampling, or *Explore* to skip the nearest set. Or lower the neighbour count (slider at the top) if you're seeing too much context.

Cheatsheet

Paste this above your desk for the first few weeks.

Face workflow:

```
maki faces detect --query "*" --apply       # detect (scans new assets only)
maki faces cluster --apply                  # group into people
# → /people: rename the big clusters
# → /people: clear the merge-suggestions panel
maki faces similarity --query "<scope>"     # diagnose cluster quality
maki faces dump-aligned --query "<scope>"   # visually verify alignment
maki faces status                           # check for stale embeddings
maki faces clean --apply                    # sweep up unassigned orphans
```

Similarity recipes:

```
maki search "similar:<id>"                  # top 20 similar
maki search "similar:<id>:50 min_sim:85"    # top 50, only >=85% match
maki search "similar:<id> tag:portrait"     # similar AND tagged
```

Stroll: - `/stroll` or press `s` on a browse card - Nearest / Discover / Explore — try all three - Stroll inherits your browse filters

---

For the reference pages behind this chapter, see:

- Face detection commands — full CLI flags for detect/cluster/similarity/etc.
- `similar:` and `person:` filters — search filter reference
- Web UI stroll page — the visual controls
- AI configuration — models, thresholds, execution providers

The Archive Lifecycle

Most photography tools focus on individual tasks — importing, tagging, exporting. But the harder challenge is the long game: managing a growing library across multiple drives over years and decades. This chapter ties together the commands you've learned into a coherent storage strategy.

The Big Picture

Files in a well-managed library flow through a predictable lifecycle. Each stage has a different purpose, and MAKI provides tools for every transition:

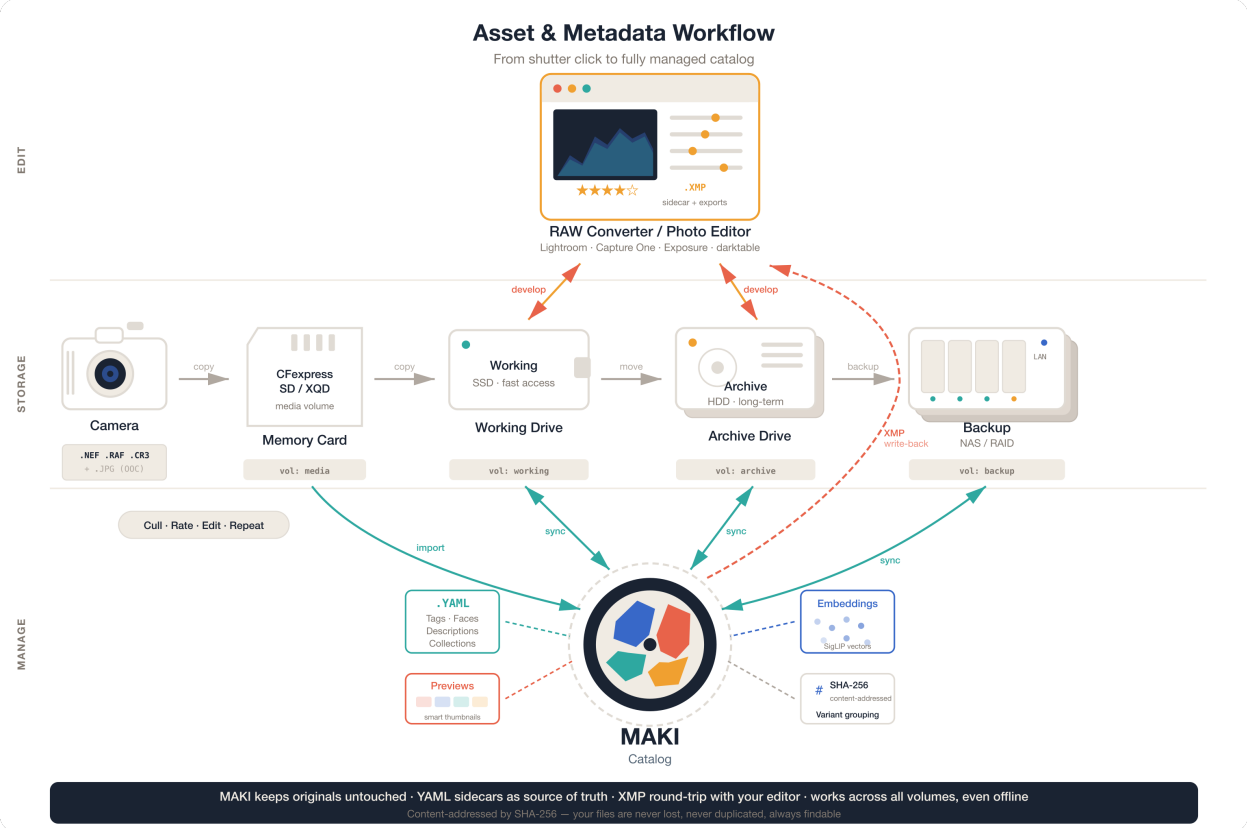


Figure 15: Asset & Metadata Workflow — from shutter click to fully managed catalog

The flow is:

1. **Import** new files onto a fast working drive
2. **Cull and organize** while files are easily accessible
3. **Archive** completed work to reliable long-term storage
4. **Back up** the archive to a second (or third) drive
5. **Verify and maintain** on a regular schedule
6. **Export and deliver** from any stage

Each MAKI volume can be assigned a **purpose** (`media`, `working`, `archive`, `backup`, `cloud`) that reflects its role in this lifecycle. These purposes drive `maki backup-status` and `maki duplicates` analysis. See Volume Purposes for setup details.

Stage 1: Import to Working Storage

A fast local SSD is the ideal landing zone for new files. Speed matters during import, culling, and preview generation. There are two approaches to getting files from a memory card into MAKI.

Traditional: copy first, then import

Copy everything to the working SSD using your preferred method, then import:

```
# Register the working drive
maki volume add "Work SSD" /Volumes/FastSSD --purpose working

# Copy files from card to working drive (outside MAKI)
rsync -av /Volumes/CARD/DCIM/ /Volumes/FastSSD/Capture/2026-03-15/

# Import into MAKI
maki import /Volumes/FastSSD/Capture/2026-03-15/ \
  --add-tag "shoot:johnson-wedding" --auto-group --smart --log
```

This is simple and reliable. The downside: you copy *everything* — including the 70% of shots you may reject during culling.

Card-first: cull before copying

Import directly from the memory card, cull on smart previews, and copy only keepers to the working drive:

```
# Register the card as a media volume (label auto-derived from mount path)
maki volume add /Volumes/EOS_DIGITAL --purpose media

# Import from card using a profile with smart previews enabled
maki import --profile card /Volumes/EOS_DIGITAL/DCIM \
  --add-tag "shoot:johnson-wedding" --auto-group --log

# Eject the card — previews are local, culling works offline
```

After import, cull in the web UI (Stage 2 below). Then copy only the keepers:

```
# Copy rated images to working drive, with XMP sidecars for CaptureOne/Lightroom
maki relocate --query "volume:EOS_DIGITAL rating:1+" \
  --target "Work SSD" --create-sidecars --log
```

```
# Clean up the card volume
maki volume remove "EOS_DIGITAL" --apply
```

The card-first approach skips copying rejects entirely. On a 1000-shot wedding day where you keep 300 images, that's 70% less data transferred to your SSD. It also lets you start reviewing immediately after inserting the card.

Tip: Set up an import profile in `maki.toml` to avoid repeating flags:

```
[import.profiles.card]
smart_previews = true
auto_tags = ["from-card"]
```

Then use `maki import --profile card /Volumes/EOS_DIGITAL/DCIM --add-tag "shoot:wedding"`.

Finding stale card volumes: After ejecting cards, old media volumes accumulate. Find and clean them up:

```
# List offline card volumes
maki volume list --purpose media --offline

# Remove a stale card volume
maki volume remove "EOS_DIGITAL" --apply
```

Stage 2: Cull and Organize

While files are on fast storage (or while working from smart previews in the card-first approach), do your creative work: rate, tag, describe, and build collections. This is the stage covered in Organizing Assets and Organizing & Culling.

A typical post-import session:

```
# Start the web UI for visual culling
maki serve

# Or from the CLI:
# 1. Auto-group RAW+JPEG pairs
maki auto-group --apply

# 2. Tag obvious rejects
maki search -q "path:Capture/2026-03-15 -rating:1+" | xargs -I{} maki tag {} rest

# 3. Build a collection from selects
maki search -q "tag:johnson-wedding rating:4+" | xargs maki col add "Johnson Wedding Selects"
```

Don't rush this stage. Files on the working drive are fast to browse and easy to revisit. Move to archive only when you're satisfied with your curation.

Stage 3: Archive Completed Work

Once a shoot or project is fully culled and organized, move it to long-term archive storage. This frees space on your working SSD while preserving everything in the catalog.

```
# Register the archive drive
maki volume add "Archive 2026" /Volumes/ArchiveDrive --purpose archive

# Preview the migration
maki relocate --query "tag:johnson-wedding" --target "Archive 2026" --dry-run

# Execute: copy files to archive
maki relocate --query "tag:johnson-wedding" --target "Archive 2026" --log

# After confirming the archive copy, free the working drive
maki relocate --query "tag:johnson-wedding volume:Work SSD" --target "Archive 2026" --remove-source --log
```

Important: Don't use `--remove-source` in the first pass. Copy first, verify, then remove the source in a second pass. This gives you a safety window.

After archiving, the files appear on both volumes during the transition. The working drive copy is removed only when you explicitly pass `--remove-source`.

Stage 4: Create Backup Copies

An archive is only as good as its backup. Copy your archived files to a physically separate drive:

```
# Register the backup drive
maki volume add "Backup A" /Volumes/BackupDisk --purpose backup

# Copy the entire archive to the backup
maki relocate --query "volume:Archive 2026" --target "Backup A" --log
```

Note: `relocate` without `--remove-source` copies files — the originals stay on the archive drive. This is intentional: you now have two copies.

The 3-2-1 rule

A solid backup strategy follows the 3-2-1 rule: **3 copies** of your data, on **2 different media types**, with **1 copy offsite**. With MAKI:

- **3 copies:** working + archive + backup (or archive + backup A + backup B)
- **2 media types:** SSD working drive + HDD archive + HDD backup
- **1 offsite:** A backup drive stored at a different physical location, or a `cloud` volume

Use `--min-copies` to enforce your policy:

```
# Check that every asset has at least 2 copies
maki backup-status --min-copies 2 --at-risk

# Stricter: require 3 copies for rated images
maki backup-status --min-copies 3 --at-risk -q "rating:1+"
```

Stage 5: Verify and Maintain

Storage devices degrade silently. A file that reads fine today may return corrupted data tomorrow. Regular verification catches problems while you still have good copies elsewhere.

Monthly routine

```
# Verify the archive drive (skip files checked within 30 days)
maki verify --volume "Archive 2026" --max-age 30 --log

# Check backup coverage
maki backup-status --at-risk

# Clean up any stale records
maki cleanup --apply
```

After reconnecting an offline drive

When you reconnect a backup or archive drive that's been offline, run the full maintenance sequence:

```
# 1. Verify file integrity
maki verify --volume "Backup A" --time

# 2. Push pending MAKI edits to XMP files
maki writeback --volume "Backup A"

# 3. Pick up external recipe changes
maki refresh --volume "Backup A"

# 4. Reconcile moved/renamed files
maki sync /Volumes/BackupDisk/ --apply

# 5. Clean up stale records
maki cleanup --volume "Backup A" --apply

# 6. Check for accidental duplicates
maki duplicates --same-volume --volume "Backup A"
```

Monitoring long-term health

Use saved searches and `stats` to keep an eye on your library:

```
# Save a search for single-copy assets (at risk)
maki ss save "At Risk" "copies:1" --favorite

# Save a search for assets not verified in 90 days
maki ss save "Stale" "stale:90" --favorite

# Overview of your storage
maki stats --volumes
```

Stage 6: Export and Deliver

At any point in the lifecycle, you can export files for delivery, sharing, or printing — regardless of which volume holds the originals (as long as it's online):

```
# Client delivery as ZIP
maki export "collection:Johnson Wedding Selects" delivery.zip --zip
```

```
# Contact sheet for review
maki contact-sheet "collection:Johnson Wedding Selects" proofs.pdf \
  --title "Johnson Wedding" --copyright "Jane Doe Photography"

# Full export with sidecars for handoff to another tool
maki export "tag:johnson-wedding" /Volumes/USB/handoff --layout mirror --include-sidecars
```

When the originals live on an offline archive drive, you can still deliver preview-sized copies — previews and smart previews are stored in the catalog directory, so no volume needs to be mounted:

```
# Social-media-sized copies straight from the catalog's smart previews
maki export "tag:johnson-wedding rating:4+" social.zip --zip --smart-previews
```

Files keep their original names with the extension swapped to the preview format (`IMG_0042.NEF` → `IMG_0042.jpg`). Assets without a generated (smart) preview are reported as skipped — run `maki generate-previews [--smart]` first for full coverage. The same option is available in the web UI's export dialog ("Content: Originals / Previews / Smart previews").

Putting It All Together: A Photographer's Monthly Workflow

Here is a concrete example of how these stages play out over a typical month:

Week 1–2: Shoot and import

```
# Copy files from card to working SSD, then import
rsync -av /Volumes/CARD/DCIM/ /Volumes/FastSSD/Capture/Corporate-2026-03/
maki import /Volumes/FastSSD/Capture/Corporate-2026-03/ \
  --add-tag "shoot:corporate-2026-03" --auto-group --smart --log

rsync -av /Volumes/CARD/DCIM/ /Volumes/FastSSD/Capture/Landscape-Spring/
maki import /Volumes/FastSSD/Capture/Landscape-Spring/ \
  --add-tag "shoot:landscape-spring" --auto-group --smart --log
```

Week 2–3: Cull and deliver

```
# Cull in the web UI, rate keepers, tag rejects as "rest"
maki serve

# Export selects for client
maki export "tag:corporate-2026-03 rating:4+" /tmp/delivery/ --zip
maki contact-sheet "tag:corporate-2026-03 rating:3+" proofs.pdf --title "Corporate Event"
```

Week 4: Archive and back up

```
# Move completed shoots to archive
maki relocate --query "tag:corporate-2026-03" --target "Archive 2026" --log
maki relocate --query "tag:landscape-spring" --target "Archive 2026" --log

# Remove from working drive after confirming archive
maki relocate --query "tag:corporate-2026-03 volume:Work SSD" --target "Archive 2026" --remove-source
maki relocate --query "tag:landscape-spring volume:Work SSD" --target "Archive 2026" --remove-source
```

```
# Back up new archive content
maki relocate --query "volume:Archive 2026 copies:1" --target "Backup A" --log

# Verify and check coverage
maki verify --volume "Archive 2026" --max-age 30
maki backup-status --at-risk
```

Result

After one month: - Working SSD is clean, ready for next month's shoots - Archive drive holds all culled and organized work - Backup drive has a copy of everything on the archive - `maki backup-status` confirms no at-risk assets - All metadata, tags, ratings, and collections travel with the files via YAML sidecars

---

When Things Go Wrong

Drives fail. It's not a question of if, but when. See Recovering from a Drive Failure for a step-by-step recovery playbook.

The best defense is prevention: regular verification, backup coverage checks, and the discipline of always having at least two copies of every file you care about.

Previous: Visual Discovery | Back to Manual

CLI Conventions

This page documents the global flags, output behavior, exit codes, and scripting patterns that apply across all maki commands.

Global Flags

Five flags are available on every command. They can appear before or after the subcommand name.

`--json`

Switches output to machine-readable JSON on stdout. Human-readable status messages are sent to stderr so they do not interfere with JSON parsing.

```
maki search "rating:5" --json
maki import /path/to/files --json
maki stats --json
```

All data types (`SearchRow`, `AssetDetails`, `ImportResult`, `VerifyResult`, `SyncResult`, `CleanupResult`, `RelocateResult`, `DuplicateEntry`, etc.) implement `serde::Serialize` and produce well-formed JSON.

`-l` / `--log`

Enables per-file progress logging to stderr. Applies to all multi-file commands: import, verify, sync, sync-metadata, refresh, cleanup, generate-previews, relocate, writeback, dedup, export, delete, fix-roles, fix-dates, fix-recipes, auto-group, describe, and `maki serve` (logs HTTP requests).

Each file prints a status line to stderr in the format `filename -- status (duration)`.

```
maki import /Volumes/Photos/2026 --log
maki verify --volume "Archive" --log
maki serve --log
```

`-v` / `--verbose`

Shows operational decisions and program flow to stderr. Useful for understanding what a command is doing without the low-level detail of `--debug`. Examples of verbose output:

- **import**: number of resolved files, detected volume, exclude patterns, auto-tags
- **describe**: VLM endpoint, model, mode, concurrency, candidate count
- **search**: parsed query, result count
- **preview generation**: source format, generation method (image/RAW/video/info card), max edge size

`--debug` implies `--verbose` — you never need to pass both.

```
maki import /Volumes/Photos/2026 --verbose
maki describe "rating:5" --verbose
maki search "tag:landscape" --verbose
```

`-d` / `--debug`

Shows stderr output from external tools (ffmpeg, dcraw, dcraw_emu) and implies `--verbose`. Useful for diagnosing preview generation issues. Prints both the command line and the tool's stderr via `eprintln`.

```
maki generate-previews --force --debug
maki import /path/to/raw-files --debug
```

`-t` / `--time`

Prints total elapsed wall-clock time after command execution.

```
maki import /Volumes/Photos/2026 --time
# ...
# Elapsed: 12.34s
```

Flags can be combined freely:

```
maki import /path --json --log --time
```

Catalog Discovery

MAKI locates the active catalog by searching for a `maki.toml` file:

1. Check the current working directory.
2. Walk up through parent directories until one containing `maki.toml` is found.
3. If the filesystem root is reached without finding one, exit with an error.

```
cd ~/Photos
maki stats            # works -- maki.toml is here

cd ~/Photos/metadata
maki stats            # works -- finds maki.toml in parent

cd /tmp
maki stats            # fails -- no maki.toml above /tmp
```

Error message when no catalog is found:

```
Error: No maki catalog found. Run `maki init` to create one.
```

Asset ID Matching

Most commands that accept an asset ID (e.g. `show`, `edit`, `relocate`, `generate-previews --asset`) support **unique prefix matching**. You do not need to type the full UUID – any unambiguous prefix is enough.

```
# Full UUID
maki show a1b2c3d4-e5f6-7890-abcd-ef1234567890

# Unique prefix (works if only one asset ID starts with "a1b2c")
maki show a1b2c
```

```
# Ambiguous prefix (multiple matches) -- maki reports an error
maki show a1
```

This applies to both command arguments and filter values where asset IDs are expected.

Exit Codes

Code	Meaning
0	Success. Command completed without errors.
1	Failure. Examples: `verify` found hash mismatches, a referenced asset/volume was not found, a required argument was missing, or any other command error.

Standard Rust/clap error handling applies for invalid arguments, missing subcommands, and unknown flags – these also exit with a non-zero code and print usage help to stderr.

Output Conventions

maki separates machine output from human messages:

Stream	Content
stdout	Command results: search results, asset details, JSON output, format-template output
stderr	Progress messages, per-file logs (`--log`), debug output (`--debug`), timing (`--time`), warnings, and errors

This separation means you can safely pipe or redirect stdout without capturing status noise:

```
maki search -q "tag:landscape" > asset-ids.txt
maki stats --json | jq '.total_size'
```

Search result count

`maki search` prints a result count header by default (e.g. `Found 42 assets`). This header is suppressed when an explicit `--format` is given (including `-q`, which is shorthand for `--format=ids`), keeping output clean for scripting.

Scripting Patterns

Get just IDs for piping

```
maki search -q "tag:landscape"
```

`-q` is shorthand for `--format=ids` and prints one UUID per line with no header.

JSON processing with jq

```
maki search "rating:5" --json | jq '.[].id'
maki show abc123 --json | jq '.variants[].filename'
maki stats --json | jq '{assets: .assets, size: .total_size}'
```

Pipe search results into a collection

```
maki search -q "rating:5 tag:travel" | xargs maki col add "Travel Best"
```

Batch operations via shell loop

```
for id in $(maki search -q "type:video"); do
  maki edit "$id" --label Blue
done
```

Dry-run before applying

Several commands default to report-only mode and require `--apply` to make changes:

```
# Preview what would be imported
maki import --dry-run /path/to/files

# See what auto-group would merge (no --apply = dry-run)
maki auto-group "tag:landscape"
maki auto-group "tag:landscape" --apply    # actually merge

# See what sync would update (no --apply = report-only)
maki sync /Volumes/Photos
maki sync /Volumes/Photos --apply          # actually update

# See what cleanup would remove (no --apply = report-only)
maki cleanup
maki cleanup --apply                       # actually remove stale records
```

Combining flags for verbose dry-runs

```
maki import --dry-run /path/to/files --log --time --json
maki sync /Volumes/Photos --log --json
```

Safe Defaults

Commands that modify or delete data use conservative defaults:

Command	Default behavior	Flag to commit changes
`delete`	Report-only (no changes)	`--apply`
`auto-group`, `auto-tag`, `auto-stack`	Report-only (no changes)	`--apply`

Command	Default behavior	Flag to commit changes
`describe`	Report-only (`--dry-run` previews without contacting the VLM)	`--apply`
`tag rename`, `tag split`, `tag delete`, `tag fix-unicode`, `tag expand-ancestors`	Report-only (no changes)	`--apply`
`sync`, `cleanup`, `dedup`	Report-only (no changes)	`--apply`
`fix-roles`, `fix-dates`, `fix-recipes`, `create-sidecars`	Report-only (no changes)	`--apply`
`volume remove`, `volume combine`, `volume split`	Report-only (no changes)	`--apply`
`stack from-tag`	Report-only (no changes)	`--apply`
`faces detect`, `faces cluster`, `faces clean`	Report-only (no changes)	`--apply`
`import`	Imports immediately	`--dry-run` to preview
`relocate`	Copies immediately	`--dry-run` to preview
`export`, `contact-sheet`	Writes immediately	`--dry-run` to preview
`refresh`, `sync-metadata`, `writeback`	Writes immediately	`--dry-run` to preview
`undo`	Reverts immediately	`--dry-run` to preview
`audio analyze`	Analyzes immediately	`--dry-run` to preview
`trash empty`	Deletes immediately	`--dry-run` to preview

The pattern is consistent: commands that scan and potentially alter many records default to showing you what they *would* do. You opt in to changes with `--apply`. Commands that operate on explicitly named files (import, relocate) run immediately but offer `--dry-run` for previewing.

Command Categories

Quick reference of all maki commands, organized by workflow stage:

Category	Commands	Purpose
Setup	`init`, `volume` (`add`, `list`, `combine`, `split`, `rename`, `remove`, `set-purpose`)	Create catalog, register and manage storage volumes

Category	Commands	Purpose
Ingest & Edit	`import`, `watch`, `delete`, `tag` (`rename`, `split`, `delete`, `clear`, `scan`, `expand-ancestors`, `fix-unicode`, `export-vocabulary`), `edit`, `group`, `split`, `auto-group`, `audio analyze`, `describe` *(Pro)*, `auto-tag` *(Pro)*, `auto-stack` *(Pro)*, `embed` *(Pro)*, `faces` *(Pro)* (`detect`, `download`, `status`, `cluster`, `clean`, `similarity`, `dump-aligned`, `people`, `name`, `merge`, `delete-person`, `unassign`, `export`), `ai export-vocabulary` *(Pro)*	Bring files in, apply metadata, merge variants, AI enrichment
Organize	`collection` (`col`: `create`, `list`, `show`, `add`, `remove`, `delete`), `saved-search` (`ss`: `save`, `list`, `run`, `delete`), `stack` (`st`: `create`, `add`, `remove`, `pick`, `dissolve`, `list`, `show`, `from-tag`)	Curate static and smart albums, scene stacks
Retrieve	`search`, `show`, `preview`, `export`, `contact-sheet`, `duplicates`, `stats`, `status`, `backup-status`, `doc`, `licenses`	Find assets, inspect details, export files, catalog health at a glance
Maintain	`undo`, `history`, `doctor`, `verify`, `sync`, `refresh`, `sync-metadata` *(Pro)*, `writeback` *(Pro)*, `cleanup`, `dedup`, `trash` (`list`, `restore`, `empty`), `relocate`, `update-location`, `generate-previews`, `fix-roles`, `fix-dates`, `fix-recipes`, `create-sidecars`, `rebuild-catalog`, `migrate`	Reversibility, integrity checks, disk reconciliation, housekeeping
Interactive	`serve`, `shell`	Web UI, interactive shell with variables and tab completion

Commands with aliases are shown with the alias in parentheses. For example, `maki col add` is equivalent to `maki collection add`, and `maki ss run` is equivalent to `maki saved-search run`.

Next: Setup Commands – `init`, `volume add`, `volume list`, `volume combine`, `volume split`, `volume rename`, `volume remove`, `volume set-purpose`.

Setup Commands

Commands for initializing a catalog and registering storage volumes. A catalog is a directory containing the SQLite database, YAML sidecars, previews, and configuration. Volumes are the storage locations (drives, directories, network mounts) where your media files live.

Command	Description
init	Create a new catalog in the current directory
volume add	Register a storage volume
volume list	List volumes and their online/offline status
volume set-purpose	Change a volume's logical role
volume remove	Unregister a volume and clean up its records
volume combine	Merge a child volume into its parent
volume split	Split a subdirectory into a new volume
volume rename	Change a volume's label

maki init

NAME

maki-init – initialize a new catalog in the current directory

SYNOPSIS

```
maki [GLOBAL FLAGS] init
```

DESCRIPTION

Creates a new MAKI catalog rooted in the current working directory. This sets up the directory structure, configuration file, SQLite database, and volume registry needed to begin managing assets.

The following files and directories are created:

- `maki.toml` – catalog configuration file (preview settings, serve settings, import exclusions)
- `metadata/` – directory for YAML sidecar files (source of truth for asset metadata)
- `previews/` – directory for generated preview thumbnails (800px)
- `smart-previews/` – directory for high-resolution smart previews (2560px, generated with `--smart`)
- `catalog.db` – SQLite database (derived cache for fast queries)
- `volumes.yaml` – storage volume registry
- `vocabulary.yaml` – default tag vocabulary (only written if the file does not exist yet)
- `.gitignore` – excludes the derived cache (`catalog.db*`), generated previews, and AI artifacts from an optional git-based backup of the catalog (only written if the file does not exist yet)

If `maki.toml` already exists in the current directory, the command fails with an error to prevent accidental re-initialization.

ARGUMENTS

None.

OPTIONS

This command only accepts global flags.

`--json` outputs `{"status": "initialized", "path": "<catalog_root>"}`.

EXAMPLES

Initialize a catalog in a new directory:

```
mkdir ~/Photos && cd ~/Photos
maki init
```

Initialize and verify with JSON output:

```
cd /Volumes/Archive/PhotoLibrary
maki init --json
# {"status": "initialized", "path": "/Volumes/Archive/PhotoLibrary"}
```

Attempt to re-initialize (fails safely):

```
cd ~/Photos
maki init
# Error: A maki catalog already exists in this directory.
```

SEE ALSO

volume add – register a storage volume after initialization. CLI Conventions – catalog discovery, global flags, exit codes.

---

maki volume add

NAME

maki-volume-add – register a new storage volume with the catalog

SYNOPSIS

```
maki [GLOBAL FLAGS] volume add <LABEL> <PATH> [--purpose <PURPOSE>]
maki [GLOBAL FLAGS] volume add <PATH> [--purpose <PURPOSE>]
```

DESCRIPTION

Registers a storage volume (a directory tree containing media files) with the catalog. Each volume is assigned a UUID and tracked by its label and mount point path. Volumes allow MAKI to manage files spread across multiple disks, external drives, and network mounts.

When two positional arguments are given, the first is the label and the second is the path. When only one argument is given, it is treated as the path and the label is auto-derived from the last component of the path (e.g., `/Volumes/EOS_DIGITAL` becomes `"EOS_DIGITAL"`). This is convenient for transient volumes like memory cards.

An optional `--purpose` flag assigns a logical role to the volume in the storage hierarchy. This metadata is used by duplicate analysis and backup coverage commands to distinguish between unwanted duplicates (same file on the same working volume) and wanted backups (same file on an archive or backup volume). The purpose can be changed later with `maki volume set-purpose`.

After registration, files under the volume's path can be imported and tracked. If the volume becomes unavailable (e.g., an external drive is disconnected), it is reported as "offline" in `maki volume list`, and commands that need to access its files will skip it gracefully.

ARGUMENTS

LABEL (optional) Human-readable name for the volume. Used in `--volume` flags across commands. Auto-derived from the path if omitted.

PATH (required) Absolute path to the volume's mount point or root directory.

OPTIONS

`--purpose <PURPOSE>` Logical role of the volume. Valid values: `media` (transient source — memory cards, card readers), `working` (active editing drive), `archive` (long-term primary storage), `backup` (redundancy copy), `cloud` (cloud sync folder). Optional — volumes without a purpose are treated as unclassified. Media volumes are excluded from `backup-status` coverage calculations.

`--json` outputs `{"id": "<uuid>", "label": "<label>", "path": "<path>", "purpose": "<purpose>"}`.

EXAMPLES

Register an external drive:

```
maki volume add "Photos" /Volumes/PhotoDrive
# Registered volume 'Photos' (a1b2c3d4-e5f6-7890-abcd-ef1234567890)
#   Path: /Volumes/PhotoDrive
```

Register a volume with a purpose:

```
maki volume add "Master Media" /Volumes/MediaDrive --purpose archive
# Registered volume 'Master Media' (e5f67890-...)
```

```
#   Path: /Volumes/MediaDrive
#   Purpose: archive
```

Register multiple volumes for a multi-disk workflow:

```
maki volume add "Laptop" /Volumes/MacintoshHD --purpose working
maki volume add "Master Media" /Volumes/MediaDrive --purpose archive
maki volume add "Backup A" /Volumes/BackupDisk --purpose backup
maki volume add "Dropbox" ~/Dropbox/Photos --purpose cloud
```

SEE ALSO

volume list – list registered volumes and their status. import – import files from a volume. relocate – copy or move asset files between volumes.

---

maki volume list

NAME

maki-volume-list – list all registered volumes and their online/offline status

SYNOPSIS

```
maki [GLOBAL FLAGS] volume list [--purpose <PURPOSE>] [--offline] [--online]
```

DESCRIPTION

Displays all storage volumes registered with the catalog, along with their UUIDs, labels, mount point paths, and current status. Optional filters narrow the output by purpose and online status.

A volume is reported as **online** if its mount point path exists on disk, and **offline** if the path is not accessible (e.g., the drive is disconnected or the network share is unmounted). Offline volumes are silently skipped by commands that access files on disk (import, verify, sync, cleanup, etc.).

ARGUMENTS

None.

OPTIONS

–**purpose <PURPOSE>** Filter by volume purpose. Valid values: `media`, `working`, `archive`, `backup`, `cloud`.
–**offline** Show only offline volumes. Mutually exclusive with `--online`.
–**online** Show only online volumes. Mutually exclusive with `--offline`.

`--json` outputs an array of `{"id", "label", "path", "volume_type", "purpose", "is_online"}` objects. Filters are applied before JSON output.

EXAMPLES

List all volumes:

```
maki volume list
# Laptop (a1b2c3d4-...) [online] [working]
#   Path: /Volumes/MacintoshHD
# Master Media (e5f67890-...) [online] [archive]
#   Path: /Volumes/MediaDrive
# Backup A (f1234567-...) [offline] [backup]
#   Path: /Volumes/BackupDisk
```

Find stale card volumes:

```
maki volume list --purpose media --offline
```

List volumes as JSON for scripting:

```
maki volume list --json | jq '.[] | select(.is_online) | .label'
```

Check if a specific volume is online:

```
maki volume list --json | jq '.[] | select(.label == "Archive") | .is_online'
```

SEE ALSO

volume add – register a new volume. volume set-purpose – change a volume's purpose. stats – `--volumes` flag shows per-volume asset counts and sizes. CLI Conventions – catalog discovery rules.

---

maki volume set-purpose

NAME

maki-volume-set-purpose – set or clear the logical purpose of a volume

SYNOPSIS

```
maki [GLOBAL FLAGS] volume set-purpose <VOLUME> <PURPOSE>
```

DESCRIPTION

Changes the purpose of an existing volume. The purpose describes the volume's role in the storage hierarchy and is used by duplicate analysis and backup coverage commands to distinguish between working copies, archives, and backups.

ARGUMENTS

VOLUME (required) Volume label or UUID.
PURPOSE (required) One of `media`, `working`, `archive`, `backup`, `cloud`, or `none` (to clear the purpose).

OPTIONS

This command only accepts global flags.

`--json` outputs `{"id": "<uuid>", "label": "<label>", "purpose": "<purpose>"}`.

EXAMPLES

Set a volume's purpose:

```
maki volume set-purpose "Photos" archive
# Volume 'Photos' purpose set to: archive
```

Clear a volume's purpose:

```
maki volume set-purpose "Photos" none
# Volume 'Photos' purpose cleared.
```

SEE ALSO

volume add – register a new volume (with optional `--purpose`). volume list – list volumes and their purposes.

maki volume remove

NAME

maki-volume-remove – remove a volume and all its associated catalog data

SYNOPSIS

```
maki [GLOBAL FLAGS] volume remove <VOLUME> [--apply]
```

DESCRIPTION

Removes a volume and all data associated with it: file location records, recipe records, assets that become orphaned (no remaining file locations on any volume), and preview files for those orphaned assets. Also removes the volume from `volumes.yaml` and the SQLite catalog.

By default, runs in **report-only mode** – shows what would be removed without making changes. Use `--apply` to execute the removal. This is consistent with `cleanup`, `sync`, and `dedup`.

The removal proceeds in phases:

1. **Locations**: Removes all file location records on the volume from the catalog and sidecar YAML files.
2. **Recipes**: Removes all recipe records on the volume from the catalog and sidecar YAML files.
3. **Orphaned assets**: Detects assets where all variants now have zero file locations. In apply mode, deletes these assets along with their variants, remaining recipes, catalog rows, and sidecar YAML files.
4. **Orphaned previews**: Detects preview files whose content hash no longer matches any variant in the catalog. In apply mode, deletes these files.
5. **Volume**: Removes the volume entry from `volumes.yaml` and the SQLite `volumes` table.

In report-only mode, orphaned assets and previews are predicted (what *would* become orphaned if the locations were removed).

ARGUMENTS

VOLUME (required) Volume label or UUID.

OPTIONS

`--apply` Execute the removal. Without this flag, only a report is printed.

This command also accepts global flags. `--json` outputs a `VolumeRemoveResult` object with fields: `volume_label`, `volume_id`, `locations`, `locations_removed`, `recipes`, `recipes_removed`, `orphaned_assets`, `removed_assets`, `orphaned_previews`, `removed_previews`, `apply`, `errors`. `--log` prints per-file progress to stderr.

EXAMPLES

Preview what removing a volume would do:

```
maki volume remove "Old Drive"
# Volume 'Old Drive' would remove: 1523 locations, 87 recipes, 412 orphaned assets, 412 orphaned previews
#   Run with --apply to remove.
```

Remove the volume:

```
maki volume remove "Old Drive" --apply
# Volume 'Old Drive' removed: 1523 locations removed, 87 recipes removed, 412 orphaned assets removed, 412 orphane
```

JSON output for scripting:

```
maki volume remove "Old Drive" --json
```

SEE ALSO

volume list – list volumes and their status. cleanup – remove stale records for missing files (works across volumes).

---

maki volume combine

NAME

maki-volume-combine – merge a source volume into a target volume by rewriting paths

SYNOPSIS

```
maki [GLOBAL FLAGS] volume combine <SOURCE> <TARGET> [--apply]
```

DESCRIPTION

Combines a source volume into a target volume. All file location and recipe records on the source volume are moved to the target volume, with their relative paths rewritten to include the directory prefix that differentiates the two mount points.

The source volume's mount point must be a subdirectory of the target volume's mount point. For example, if the target is `/Volumes/Media` and the source is `/Volumes/Media/media_2024`, the computed prefix is `media_2024/` and all paths are prepended accordingly.

This is useful when you have multiple volumes registered as subdirectories of the same physical disk (e.g., year-based volumes `media_2024`, `media_2025` under `/Volumes/Media`). Since they all go online and offline together, maintaining separate volumes adds complexity without benefit. Combining them into a single disk-level volume simplifies `volume list`, web UI dropdowns, and `backup-status` reporting.

By default, runs in **report-only mode** – shows what would be combined without making changes. Use `--apply` to execute the combination.

In apply mode, the operation proceeds in phases:

1. **Sidecars**: Loads each affected asset's YAML sidecar, rewrites matching variant locations and recipe locations (changes volume ID and prepends path prefix), and saves.
2. **Catalog**: Ensures the target volume exists in the SQLite catalog, then bulk-updates all `file_locations` and `recipes` rows in a single SQL statement each.
3. **Cleanup**: Removes the source volume from `volumes.yaml` and the SQLite `volumes` table.

Sidecars are updated first (source of truth). If the process crashes between sidecar and catalog updates, `maki rebuild-catalog` reconstructs the correct state from sidecars.

ARGUMENTS

SOURCE (required) Label or UUID of the volume to merge away. This volume is removed after combining.

TARGET (required) Label or UUID of the volume that receives all locations and recipes.

OPTIONS

`--apply` Execute the combination. Without this flag, only a report is printed.

This command also accepts global flags. `--json` outputs a `VolumeCombineResult` object with fields: `source_label`, `source_id`, `target_label`, `target_id`, `path_prefix`, `locations`, `locations_moved`, `recipes`, `recipes_moved`, `assets_affected`, `apply`, `errors`. `--log` prints per-asset progress to stderr.

EXAMPLES

Preview what combining would do:

```
maki volume combine "media_2024" "Media"
# Would combine 'media_2024' into 'Media': 4832 locations, 312 recipes (3210 assets, prefix 'media_2024/')
#   Run with --apply to combine.
```

Execute the combination:

```
maki volume combine "media_2024" "Media" --apply
# Volume 'media_2024' combined into 'Media': 4832 locations moved, 312 recipes moved (3210 assets, prefix 'media_2
```

Combine multiple year-volumes into a single disk volume:

```
for year in media_2010 media_2011 media_2012 media_2013 media_2014 media_2015; do
  maki volume combine "$year" "Media" --apply
done
```

SEE ALSO

volume list – list volumes and verify the result. volume remove – remove a volume and delete its data (rather than merging). rebuild-catalog – reconstruct SQLite from sidecars if needed after a crash.

---

maki volume split

NAME

maki-volume-split – split a subdirectory from an existing volume into a new volume

SYNOPSIS

```
maki [GLOBAL FLAGS] volume split <SOURCE> <NEW_LABEL> --path <SUBDIR> [--purpose <PURPOSE>] [--apply]
```

DESCRIPTION

Inverse of `volume combine`: creates a new volume from a subdirectory of an existing volume. All file locations and recipes under the specified path are moved to the new volume, with path prefixes stripped. The source volume is preserved with the remaining locations.

Without `--apply`, runs in report-only mode showing what would be moved.

OPTIONS

–**path <SUBDIR> (required)** Subdirectory path (relative to source volume mount point) to split off.
–**purpose <PURPOSE>** Assign a purpose to the new volume.
–**apply** Execute the split.

EXAMPLES

```
maki volume split "Photos" "Archive 2024" --path "Archive/2024" --purpose archive --apply
```

---

maki volume rename

NAME

maki-volume-rename – change a volume's label

SYNOPSIS

```
maki [GLOBAL FLAGS] volume rename <VOLUME> <NEW_LABEL>
```

DESCRIPTION

Changes the label of a registered volume. The volume's UUID, mount point, and all tracked file locations remain unchanged. Only the human-readable label is updated in the volume registry and YAML sidecars.

ARGUMENTS

VOLUME (required) Current volume label or UUID.
NEW_LABEL (required) New label.

EXAMPLES

```
maki volume rename "Old Drive" "Archive 2025"
```

Next: Ingest Commands – `import`, `tag`, `edit`, `group`, `auto-group`.

Ingest Commands

Commands for importing files, applying metadata, and merging asset variants. Import brings files into the catalog without copying them. Metadata commands (tag, edit, describe) modify asset properties. Grouping commands (group, split, auto-group) manage variant relationships.

Command	Description
import	Import files into the catalog
watch	Watch directories and auto-import new files (poll-based)
delete	Remove assets from the catalog
tag	Add or remove tags
tag rename	Rename a tag across all assets
tag split	Split one tag into multiple tags across all assets
tag delete	Delete a tag (and descendants) from every asset that has it
tag clear	Remove all tags from an asset
tag fix-unicode	Normalise tag values to Unicode NFC catalog-wide
tag scan	Find assets with malformed tags (whitespace, stray pipes, NFD, …)
tag expand-ancestors	Expand hierarchical tags to include ancestor paths
tag export-vocabulary	Export tag tree as a vocabulary.yaml file
edit	Edit metadata (name, rating, label, description, date)
group	Merge variants into one asset
split	Extract variants into standalone assets
auto-group	Auto-group by filename stem
auto-tag	AI-powered tag suggestions *(Pro)*
embed	Generate image embeddings *(Pro)*
audio analyze	Detect musical key and tempo of audio assets
describe	Generate VLM descriptions *(Pro)*

maki import

NAME

maki-import – import files into the catalog

SYNOPSIS

```
maki [GLOBAL FLAGS] import [OPTIONS] <PATHS...>
```

DESCRIPTION

Imports files and directories into the catalog. Each file is hashed (SHA-256), and its metadata is extracted from EXIF data and XMP sidecars. Preview thumbnails are generated during import.

Stem-based auto-grouping: Files sharing the same filename stem in the same directory are grouped into one asset. RAW files take priority as the primary variant (defining the asset's identity and EXIF data). Additional media files become extra variants on the same asset.

Recipe handling: Processing sidecar files (`.xmp`, `.cos`, `.cot`, `.cop`, `.pp3`, `.dop`, `.on1`) are attached as recipe records to the primary variant rather than imported as standalone assets. Re-importing after external edits updates recipes in place. Identical copies of an already-known recipe (same content hash, e.g. from a backup volume) are recorded for location tracking but do not re-merge metadata — this keeps your curated tags and ratings intact when importing backups.

XMP metadata extraction: When an `.xmp` sidecar is attached, its contents are parsed. Keywords become asset tags, `dc:description` sets the asset description, `xmp:Rating` becomes the asset rating, and `xmp:Label` becomes the color label. EXIF data takes precedence over XMP for overlapping fields.

Embedded XMP: JPEG and TIFF files have their embedded XMP metadata extracted automatically (keywords, ratings, descriptions, labels).

Duplicate handling: When a file's content hash already exists, the new file location is added to the existing variant. Only truly skips when the exact same location (volume + path) is already recorded.

Preview generation: Standard images get 800px JPEG thumbnails via the `image` crate. RAW files use `dcraw`/`dcraw_emu` (LibRaw). Videos use `ffmpeg`. Non-visual formats get an info card. Preview failure never blocks import.

The volume is auto-detected from the first path by matching against registered volume mount points. Use `--volume` to override.

ARGUMENTS

PATHS (required) One or more file paths or directory paths to import. Directories are scanned recursively.

OPTIONS

–**volume <LABEL>** Use a specific volume instead of auto-detecting from the path. Useful when auto-detection picks the wrong volume.

–**include <GROUP>** Include additional file type groups that are not imported by default. Can be specified multiple times. Example groups: `captureone`, `documents`.

–**skip <GROUP>** Skip file type groups that would normally be imported. Can be specified multiple times. Example groups: `audio`, `xmp`.

–**add-tag <TAG>** Add a tag to every imported asset. Can be specified multiple times. Merged with `[import] auto_tags` from `maki.toml` and XMP-extracted tags, deduplicated.

–**smart** Generate smart previews (high-resolution, 2560px) alongside regular thumbnails during import. Smart previews enable zoom and pan in the web UI. Can be enabled permanently via `[import] smart_previews = true` in `maki.toml`. Smart preview dimensions are controlled by `[preview] smart_max_edge`.

–**embed** ***(Pro)*** Generate SigLIP image embeddings for visual similarity search during import. Embeddings enable `maki auto-tag --similar` and the web UI "Find similar" button. Runs as a post-import phase using preview images (smart preview preferred, then regular preview). Silently skips if the AI model is not downloaded – run `maki auto-tag --download` first. Can be enabled permanently via `[import] embeddings = true` in `maki.toml`. Non-image assets are skipped. Uses the model configured in `[ai] model`.

–**describe** ***(Pro)*** Generate VLM descriptions for newly imported assets as a post-import phase. Requires a running Ollama instance (or compatible VLM endpoint configured in `[vlm]`). Runs after the embed phase if both are enabled. Uses the VLM model, prompt, and parameters from `[vlm]` config (including per-model overrides). Concurrency is controlled by `[vlm] concurrency`. Can be enabled permanently via `[import] descriptions = true` in `maki.toml`. Non-image assets are skipped.

–**dry-run** Show what would be imported without writing to catalog, sidecar, or disk. Files are still hashed to detect duplicates. Reports the same counters as a real import (imported, skipped, locations added, recipes attached/updated).

–**profile <NAME>** Use a named import profile from `[import.profiles.<name>]` in `maki.toml`. The profile overrides the base `[import]` config — unset profile fields inherit from the base. CLI flags (`--smart`, `--add-tag`, etc.) override both the profile and base config. The profile can also specify `include` and `skip` file type groups. See the Configuration Reference for profile syntax.

–**auto-group** After importing, automatically group newly imported assets with nearby catalog assets by filename stem. "Nearby" means assets on the same volume under the same *session root* as the imported files. The session root of each imported directory is the deepest path component matching `[group] session_root_pattern` (default: a directory starting with `YYYY-MM`), wherever it sits — above or below the directory you imported. Without a match, the directory given to `maki import` is the session, so a whole client folder imported at once groups across all its subfolders; when the imported directory itself holds the files (you imported just `Output/`), the scope is one level up so the sibling `Capture/` is seen. This handles the common CaptureOne/Lightroom pattern where RAW originals live in `Capture/` and exports in `Output/` (or deeper, `Output/Final/Web/`) under a shared session folder. Uses the same fuzzy prefix matching and the same session partitioning as `maki auto-group`. When combined with `--dry-run`, the auto-group phase also runs in dry-run mode.

Undated archives: importing a tree of several undated shoots with `--auto-group` makes the whole tree one session, so a camera counter that restarted between shoots can produce false groups. Set a `session_root_pattern` that matches your shoot folders, or import one shoot at a time; `--dry-run` previews the groups.

`--json` outputs an `ImportResult` object with `imported`, `skipped`, `locations_added`, `recipes_attached`, `recipes_updated`, `sidecars_created` (sidecars written by `[writeback] create_sidecars`) counters and a `dry_run` boolean. When `--auto-group` produces matches, an `auto_group` key is added with the full `AutoGroupResult`. When `--embed` generates embeddings, `embeddings_generated` and `embeddings_skipped` keys are added. When `--describe` generates descriptions, `descriptions_generated` and `descriptions_skipped` keys are added.

EXAMPLES

Import a directory of photos:

```
maki import /Volumes/Photos/Capture/2026-02-22
```

Import with explicit volume and progress logging:

```
maki import --volume "Archive" /Volumes/NAS/Photos/2025 --log --time
```

Preview what would be imported without making changes:

```
maki import --dry-run /Volumes/SD-Card/DCIM
```

Import only image files, skipping audio and XMP sidecars:

```
maki import --skip audio --skip xmp /Volumes/Photos/Mixed
```

Import with smart previews for high-resolution browsing:

```
maki import --smart /Volumes/Photos/Capture/2026-02-22
```

Import with embedding generation for visual similarity search *(Pro)*:

```
maki import --embed /Volumes/Photos/Capture/2026-02-22
```

Import with both smart previews and embeddings:

```
maki import --smart --embed /Volumes/Photos/Capture/2026-02-22
```

Tag assets during import:

```
maki import --add-tag landscape --add-tag "2026" /Volumes/Photos/Landscapes
```

Import a CaptureOne session and auto-group RAW+exports:

```
maki import --auto-group /Volumes/Photos/2026-02-22/Capture /Volumes/Photos/2026-02-22/Output
```

Import with JSON output for scripting:

```
maki import /Volumes/Photos/NewShoot --json | jq '.imported'
```

SEE ALSO

tag – add or remove tags after import. edit – set name, description, rating, or label. auto-group – group related assets by filename stem. generate-previews – regenerate or upgrade previews. CLI Conventions – `--log`, `--json`, `--time` behavior.

---

maki watch

NAME

maki-watch – watch directories and auto-import new files (poll-based)

SYNOPSIS

```
maki [GLOBAL FLAGS] watch [OPTIONS] [PATHS...]
```

DESCRIPTION

Watches directories for filesystem changes and feeds them into the catalog automatically. After a CaptureOne session (or any tethered/copy workflow), new files appear in the catalog without a manual `maki import`.

With no PATHS, the mount points of all online registered volumes are watched (or just the volume named by `--volume`). Explicit PATHS must lie inside registered volumes — the same requirement (and error message) as `maki import`.

Polling, not events: the watched roots are re-scanned every N seconds (default 30, configurable via `[watch] poll_seconds` or `--interval`). Poll-based watching behaves identically on macOS, Linux, and Windows and needs no OS-specific event APIs.

Stability debounce: a new or changed file is acted on only once its size and modification time are identical on two consecutive scans. Files still being copied — for example during a tethered capture session — stay "pending" until they stop growing and are never half-imported.

Actions per tick:

- *New stable files* go through the regular import pipeline (per owning volume). All `[import]` config applies: `auto_tags`, `smart_previews`, `embeddings`, `descriptions`, and `exclude` patterns. Import is content-addressed and idempotent, so a re-offered known file is cheap and harmless.
- *Changed stable `.xmp` recipe files* that already belong to the catalog go through the refresh pipeline, scoped to just those files — the owning asset's tags, rating, label, and description are re-read from the sidecar.
- *Deletions* are ignored (logged with `--verbose` only). Removing catalog records stays an explicit decision — run `maki cleanup`.
- *Changed media files* are ignored too (use `maki sync` to reconcile in-place edits).

Baseline: the initial scan establishes a baseline; files already present when the watch starts are NOT imported — watch means "from now on". Run `maki import` once for the backlog. A one-line startup summary reports the watched roots, poll interval, and baseline file count.

The catalog's own derived directories (`previews/`, `smart-previews/`, `metadata/`, `embeddings/`, `faces/`, `.trash/`) are never tracked, so a catalog living inside a watched volume does not import its own thumbnails.

Runs as a foreground process like `maki serve`. Quiet ticks print nothing; a tick that did something prints a one-line summary to stderr. Ctrl-C exits cleanly.

ARGUMENTS

PATHS (optional) Directories (or files) to watch. Each must lie inside a registered volume. Default: the mount points of all online registered volumes.

OPTIONS

–**volume <LABEL>** Watch a specific volume's mount point. When PATHS are also given, each must lie on that volume.

–**interval <SECONDS>** Poll interval between scans. Minimum 2 (lower values are clamped). Defaults to `[watch] poll_seconds` from `maki.toml` (30).

–**once** Run exactly one cycle and exit: scan, wait ~1 second, re-scan for stability, act, print the summary. Unlike the continuous loop (whose baseline is "what was on disk when the watch started"), `--once` diffs against the *catalog*: stable files not yet tracked are imported, and tracked `.xmp` recipes whose content changed are refreshed. Designed for scripting and cron-style invocation.

`--log` prints a per-file line for every import/refresh action. `--verbose` additionally logs quiet ticks, deletions, and ignored media changes. `--json` emits a machine-readable summary per active tick (a single pretty-printed object with `--once`).

EXAMPLES

Watch all online volumes with the default 30s interval:

```
maki watch
```

Watch a CaptureOne session folder, polling every 10 seconds:

```
maki watch --interval 10 /Volumes/Photos/2026-06-11-shoot
```

Watch one volume with per-file logging:

```
maki watch --volume "Archive" --log
```

One-shot cycle for a cron job or script:

```
maki watch --once --json | jq '.imported'
```

NOTES

Excludes are additive: `[import] exclude` and `[watch] exclude` patterns are both applied (glob match against file and directory names, same matching as import). Hidden files and directories are always skipped.

The stability window equals one poll interval in continuous mode (~1 second with `--once`). A file must therefore be unchanged for at least one full interval before it is imported.

SEE ALSO

import – one-shot import for backlogs and explicit ingestion. refresh – re-read metadata from changed sidecar files. sync – reconcile moved/modified/missing files. Configuration Reference – `[watch]` section options.

---

maki delete

NAME

maki-delete – remove assets from the catalog

SYNOPSIS

```
maki [GLOBAL FLAGS] delete [OPTIONS] [ASSET_IDS...]
```

DESCRIPTION

Removes assets from the catalog. By default runs in **report-only mode** – shows what would be deleted without making changes. Use `--apply` to execute the deletion.

When `--apply` is set, the following data is removed for each asset:

- The asset row from the SQLite catalog
- All variants belonging to the asset
- All file location records for those variants
- All recipe records attached to those variants
- All preview and smart preview files
- The YAML sidecar file
- Collection memberships (the asset is removed from all collections)
- Stack membership (the stack auto-dissolves if only one member remains)

With `--remove-files` (which requires `--apply`), physical media files and recipe files are also deleted from disk on online volumes. Files on offline volumes are skipped with a warning.

Asset IDs support unique prefix matching (see CLI Conventions).

When no asset IDs are given on the command line, IDs are read from stdin (one per line). This enables piping from `maki search -q`:

```
maki search -q "orphan:true" | maki delete --apply
```

ARGUMENTS

ASSET_IDS (optional) One or more asset IDs or unique prefixes. If omitted, reads from stdin.

OPTIONS

–**apply** Execute the deletion. Without this flag, the command only reports what it would do.

–**remove-files** Also delete physical files (variant media and recipe files) from disk. Requires `--apply`. Skips files on offline volumes with a warning. Files go to `<catalog>/.trash/` (recoverable via `maki trash restore`) unless trash is disabled.

–**no-trash** With `--remove-files`, delete permanently instead of moving files to the catalog trash.

`--json` outputs a `DeleteResult` with `deleted`, `not_found`, `files_removed`, `previews_removed`, `dry_run`, and `errors`.

`--log` prints per-asset status to stderr.

EXAMPLES

Preview what would be deleted (report-only):

```
maki delete a1b2c3d4
```

Delete an asset:

```
maki delete --apply a1b2c3d4
```

Delete multiple assets at once:

```
maki delete --apply a1b2c3d4 e5f6a7b8
```

Delete an asset and its files from disk:

```
maki delete --apply --remove-files a1b2c3d4
```

Delete all orphaned assets (no file locations):

```
maki search -q "orphan:true" | maki delete --apply
```

Delete with JSON output for scripting:

```
maki delete --apply a1b2c3d4 --json | jq '.deleted'
```

Use a short ID prefix:

```
maki delete --apply a1b2
```

SEE ALSO

cleanup – remove stale location records and orphaned assets automatically. search – find assets to delete (`orphan:true`, `missing:true`). show – inspect an asset before deleting it.

---

maki tag

NAME

maki-tag – add or remove tags on an asset

SYNOPSIS

```
maki [GLOBAL FLAGS] tag <ASSET_ID> [--remove] <TAGS...>
```

DESCRIPTION

Adds or removes tags on an asset. Tags are free-form text strings stored on the asset record. They are persisted in both the YAML sidecar file and the SQLite catalog.

When tags are changed, MAKI automatically writes the changes back to any `.xmp` recipe files associated with the asset. Tag write-back uses operation-level deltas: added tags are inserted into the XMP `dc:subject` / `rdf:Bag` block, and removed tags are deleted – tags added independently in external tools like CaptureOne are preserved.

Tags are deduplicated: adding a tag that already exists is a no-op.

Asset IDs support unique prefix matching (see CLI Conventions).

ARGUMENTS

ASSET_ID (required) The asset ID or a unique prefix of it.
TAGS (required) One or more tags to add or remove.

OPTIONS

–**remove** Remove the specified tags instead of adding them.

`--json` outputs a `TagResult` with `asset_id`, `tags_added` or `tags_removed`, and the full `tags` list after the operation.

EXAMPLES

Add tags to an asset:

```
maki tag a1b2c3d4 landscape mountains "golden hour"
```

Remove a tag:

```
maki tag a1b2c3d4 --remove landscape
```

Add a multi-word tag:

```
maki tag a1b2c "Fools Theater"
```

Tag assets in bulk via search:

```
for id in $(maki search -q "path:Capture/2026-02-22"); do
  maki tag "$id" "February 2026"
done
```

SEE ALSO

edit – set asset name, description, rating, or label. search – `tag:` filter for finding tagged assets. stats – `--tags` shows tag usage frequencies.

maki tag rename

NAME

maki-tag-rename – rename a tag across all assets that have it

SYNOPSIS

```
maki [GLOBAL FLAGS] tag rename <OLD_TAG> <NEW_TAG> [--apply]
```

DESCRIPTION

Finds all assets tagged with OLD_TAG and replaces that tag with NEW_TAG. This is an atomic bulk operation — every affected asset is updated in the catalog, YAML sidecar, and XMP recipe files (if writeback is enabled) in a single pass.

By default, matching is **case-insensitive** and **cascades to descendant tags**, consistent with the default `tag:` search filter behavior. `maki tag rename "Concert" "concert"` finds and renames "Concert", "CONCERT", and "concert" variants. The stored result uses the exact case specified in NEW_TAG.

Prefix markers for OLD_TAG mirror the `tag:` search syntax. The markers can be combined in any order:

OLD_TAG syntax	Behavior
`Foo`	Case-insensitive, includes descendants (default)
`=Foo`	Whole path only — does NOT touch `Foo|child` tags or other Foo occurrences
`/Foo`	Same as `=Foo` in rename context (both mean exact value match)
`^Foo`	Case-sensitive — `Foo` and `foo` are different tags
`=^Foo` or `^=Foo`	Both: whole path AND case-sensitive

In rename, `=` and `/` collapse to the same behavior because the underlying SQL uses equality on each individual tag value (which is whole-path by construction). The descendant-skip logic on top makes `=` behave correctly as "rename this exact tag, leave its children alone."

NEW_TAG is always taken literally (no prefix parsing) and is stored with the exact case given.

Ancestor expansion: When renaming a flat tag to a hierarchical one (e.g., "Munich" to "location|Germany|Bavaria|Munich"), all ancestor paths are automatically added (`location`, `location|Germany`, `location|Germany|Bavaria`). This matches the CaptureOne/Lightroom convention. The rename cascades to descendant tags by default — renaming a parent renames all children — unless you use the `=` whole-path marker.

If an asset already has NEW_TAG (in any case variant), the old tag is simply removed without creating a duplicate.

Without `--apply`, runs in report-only mode showing which assets would be affected.

ARGUMENTS

OLD_TAG (required) The tag to rename. Uses the same input conventions as `maki tag`: `|` (or `>`) for hierarchy. Default matching is case-insensitive and includes descendants. Optional `=` and/or `^` prefix markers tighten matching as described above.

NEW_TAG (required) The replacement tag. Always taken literally and stored with the exact case given.

OPTIONS

–**apply** Execute the rename. Without this flag, only reports what would change.

EXAMPLES

Rename a flat tag to a hierarchical one (also removes standalone “Germany” and “Bavaria”):

```
maki tag rename "Munich" "location|Germany|Bavaria|Munich" --apply
```

Normalize casing across all variants:

```
maki tag rename "Concert" "concert" --apply
```

Rename only the capitalized variant, leaving the lowercase one alone (e.g., when cleaning up case duplicates):

```
maki tag rename "^Landscape" "Landscape" --apply
```

Rename only the exact tag, not its descendants (e.g., promote `Animals` to `Wildlife` without touching `Animals|Cats`):

```
maki tag rename "=Animals" "Wildlife" --apply
```

Combined: rename only the exact-case, exact-level tag:

```
maki tag rename "=^Birds" "Aves" --apply
```

Move a branch deeper in the hierarchy (all descendants follow — `letter|a` → `abstract|letter|a`, etc.):

```
maki tag rename "subject|letter" "subject|abstract|letter" --apply
```

Move an entire subtree to a new root:

```
maki tag rename "animals" "nature|animals" --apply
# animals|birds|eagles → nature|animals|birds|eagles
```

Flatten a deep hierarchy into a single tag:

```
maki tag rename "location|Germany|Bayern|München" "München" --apply
```

Consolidate synonyms across languages (assets that already have the target tag just get the old one removed):

```
maki tag rename "Konzert" "performing arts|concert" --apply
```

Fix a typo:

```
maki tag rename "lanscape" "landscape" --apply
```

Preview what would change (dry run, no `--apply`):

```
maki tag rename "old-tag" "new-tag"
```

Preview with per-asset detail:

```
maki tag rename "old-tag" "new-tag" --log
```

SEE ALSO

tag – add or remove tags on a single asset. tag split – replace one tag with multiple tags (or add alongside with `--keep`). tag clear – remove all tags from an asset. search – `tag:` filter for finding tagged assets, with the same `=` and `^` prefix markers.

maki tag split

NAME

maki-tag-split – split one tag into multiple tags across all assets

SYNOPSIS

```
maki [GLOBAL FLAGS] tag split <OLD_TAG> <NEW_TAG>... [--keep] [--apply]
```

DESCRIPTION

Replaces OLD_TAG with every tag in NEW_TAG… across all matching assets. Useful for structural restructuring (`"concert-jane-2024"` → both `"subject|performing arts|concert"` and `"event|concert-jane-2024"`) and for separating combined tags (`"A & B"` → `"A"` and `"B"`).

Split is **exact-tag-only**: it acts on assets where OLD_TAG appears as a leaf (no descendants on that asset). Assets where OLD_TAG has descendants (e.g. the asset also carries `OLD_TAG|foo`) are skipped — splitting a non-leaf tag has ambiguous semantics. Use `tag rename` for structural renames that should cascade into descendants.

New tags are expanded to include all ancestor paths (same as regular `tag`), so `tag split foo subject|nature|landscape` automatically adds `subject` and `subject|nature` alongside the leaf.

With **`--keep`**, OLD_TAG is preserved and the new tags are added alongside — an additive / copy operation rather than a replace. This is the right tool when you want to supplement existing tags rather than replace them.

If writeback is enabled, all tag changes are also written back to XMP recipe files on disk.

ARGUMENTS

OLD_TAG (required) Source tag name. Accepts the same optional markers as `tag rename`: `=foo` / `/foo` (redundant no-ops — split is already exact-only), `^foo` (case-sensitive). The `|foo` prefix-anchor marker is rejected (split operates on one tag at a time).

NEW_TAG… (required, one or more) Target tag names. Always taken literally (no prefix parsing). `|` denotes hierarchy as usual.

OPTIONS

–**keep** Keep OLD_TAG in place and add the new tags alongside. Without this flag, OLD_TAG is removed (the replace / split semantics).

–**apply** Apply changes (default: report-only dry run).

EXAMPLES

Split a combined tag into its parts:

```
maki tag split "A & B" "A" "B" --apply
```

Restructure an event tag into the canonical pair (scene-type + specific-occasion):

```
maki tag split "concert-jane-2024" \
    "subject|performing arts|concert" \
    "event|concert-jane-2024" \
    --apply
```

Add a broader tag to everything currently carrying a specific tag (additive — keeps the source):

```
maki tag split "sunset" "color|warm" --keep --apply
```

Preview what a split would do (dry run, no `--apply`):

```
maki tag split "A & B" "A" "B"
```

SEE ALSO

tag – add or remove tags on a single asset. tag rename – rename a single tag (cascades to descendants). tag expand-ancestors – add ancestor paths for existing hierarchical tags.

---

maki tag delete

NAME

maki-tag-delete – remove a tag (and, by default, its descendants) from every asset that has it

SYNOPSIS

```
maki [GLOBAL FLAGS] tag delete <TAG> [--apply]
```

DESCRIPTION

Catalog-wide tag deletion: removes `TAG` from every asset that carries it. Mirrors the safety pattern of `tag rename` and `tag split` — runs as a dry-run by default and only modifies the catalog when `--apply` is passed. Same hierarchy semantics as `tag rename`:

- **Default cascade**: `maki tag delete event|wedding-jane-2025` removes `event|wedding-jane-2025` *and* every descendant (e.g. `event|wedding-jane-2025|ceremony`) from every asset that has them. The entire branch goes.
- **Leaf-only**: prefix the tag with `=` or `/` to opt out of the cascade. `maki tag delete =event|wedding-jane-2025` removes the exact tag value only, *and skips* assets where it has descendants — auto-expansion would re-add the parent on the next write anyway, so removing the parent without descendants would be incoherent.
- **Case-sensitive**: prefix with `^` for an exact-case match (`maki tag delete ^Landscape` won't touch `landscape` or `LANDSCAPE`).

After deletion, any **ancestor path that loses its last surviving descendant** on an asset is also cleaned up — same coherence rule that `maki tag <ASSET> --remove` uses (and the same `orphaned_ancestors` helper). This keeps the auto-expanded storage model coherent: a leaf delete won't leave an unused branch hanging on the asset's tag list.

The `|`-prefix anchor marker (used in search to mean "any tag whose path component starts with…") is rejected for delete — it would collapse distinct tags into one operation, which is rarely what you want. Use `maki search 'tag:|foo' --format ids` to find matching tags first, then run a targeted delete for each.

OPTIONS

<TAG> Tag to delete. Use `>` or `|` as the hierarchy separator (CaptureOne / Lightroom convention). Optional markers: `=` / `/` for leaf-only, `^` for case-sensitive. Markers are order-insensitive (`=^foo` and `^=foo` behave the same).

–apply Apply the deletion. Without this flag, the command counts matches and prints the summary without modifying the catalog or any sidecars.

EXAMPLES

Drop a typo'd tag everywhere:

```
maki tag delete "lansdcape" --apply
```

Remove an entire event branch (cascade default):

```
maki tag delete "event|wedding-jane-2025" --apply
```

Remove just the parent node `subject|nature` *only* on assets where it isn't already split into descendants:

```
maki tag delete "=subject|nature" --apply
```

Preview before committing:

```
maki tag delete "old|legacy|tag"          # dry run, prints count
maki tag delete "old|legacy|tag" --apply  # actually deletes
```

XMP WRITEBACK

When the catalog is configured with `[writeback] enabled = true`, removed tags are written back to each asset's XMP recipe(s) on next online-volume access. Until writeback runs, the catalog and sidecars reflect the deletion immediately but XMP files on disk still carry the old tags. See maki writeback.

SEE ALSO

tag rename – rename instead of delete. tag split – replace one tag with multiple alternatives. tag – add or remove tags on a single asset.

---

maki tag clear

NAME

maki-tag-clear – remove all tags from an asset

SYNOPSIS

```
maki [GLOBAL FLAGS] tag clear <ASSET_ID> [--source <SOURCE>]
```

DESCRIPTION

Removes all tags from the specified asset. This is equivalent to running `maki tag <id> --remove` with every tag the asset currently has, but without having to list them.

With `--source`, only tags with the given provenance are removed. Every tag records where it came from: `user` (manual edits, configured import auto_tags), `xmp-import` (XMP keywords absorbed at import/reimport), `auto-tag` (SigLIP auto-tagger), or `vlm` (VLM describe). Tags created before provenance tracking count as `user`. This makes it easy to wipe machine-applied tags — for example before re-running auto-tag with a better model — without touching human curation. `maki show` annotates machine tags (`(auto-tag)`, `(vlm)`, `(xmp)`) so you can preview what a source-filtered clear would remove.

If writeback is enabled, the tag removal is written back to XMP recipe files on disk.

ARGUMENTS

ASSET_ID (required) Asset ID or unique prefix.

OPTIONS

–**source SOURCE** Only clear tags with this provenance source: `user`, `xmp-import`, `auto-tag`, or `vlm`. Without the flag, all tags are cleared.

EXAMPLES

```
maki tag clear a1b2c3d4
maki tag clear a1b2c3d4 --source auto-tag   # drop machine tags, keep human curation
```

SEE ALSO

tag – add or remove individual tags. tag rename – rename a tag across all assets.

---

maki tag fix-unicode

NAME

maki-tag-fix-unicode – normalise tag values to Unicode NFC across the catalog

SYNOPSIS

```
maki [GLOBAL FLAGS] tag fix-unicode [--apply]
```

DESCRIPTION

One-shot migration that walks every asset in the catalog and rewrites its tag values to Unicode NFC (canonical composition). Background: external tools roundtrip through different normalisation forms — macOS path APIs lean **NFD** (decomposed: `O` + combining diaeresis), most XMP writers produce **NFC** (precomposed: single `Ö` code point). The two encodings render identically but are different byte strings, which means a single logical tag like `Ö-HA` can end up as two distinct catalog entries that sort to different positions on the tags page and don't collapse on the facet sidebar.

After this command runs:

- All tag values are NFC-normalised on disk (sidecar YAML and SQLite).
- Per-asset duplicates (where the same tag was present in both encodings) are collapsed to one.
- The tags page shows one row per logical tag instead of two.
- New tags going through MAKI's input pipeline are NFC-normalised at the chokepoint, so the catalog stays canonical without further intervention.

The command is idempotent — running it again on a normalised catalog reports `0 fixed`.

OPTIONS

--apply Apply the normalisation. Without this flag, the command counts what would change without modifying the catalog or any sidecars.

EXAMPLES

Preview impact:

```
maki tag fix-unicode
```

Sample output:

```
Tag fix-unicode: 12847 scanned, 23 would be fixed, 47 tag value(s) normalised, 8 with merged duplicates
  Run with --apply to normalise tags.
```

Apply:

```
maki tag fix-unicode --apply
```

Per-asset progress:

```
maki tag fix-unicode --apply --log
```

XMP WRITEBACK

When `[writeback] enabled = true`, removed encoding-variants are written back to each asset's XMP recipe(s) as tag deletions. The kept (NFC) form is identical to what was already there modulo encoding, so XMP semantics are unchanged for those values. Until writeback runs, sidecars and SQLite reflect the normalisation immediately but XMP files on disk still carry the old encodings. See maki writeback.

NOTES ON SORT ORDER

After NFC normalisation, accented characters such as `ö` are stored as a single precomposed code point (U+00D6). Under SQLite's default byte-comparison sort, that code point is greater than `z`, so umlauted tags appear at the **end** of the alphabetical tag list rather than next to their base letter. The decomposed (NFD) form happens to land near the base letter under the same collation, but only as an accident of byte ordering — Swedish/Finnish dictionaries place `ö` after `z` deliberately, German DIN 5007-1 places it next to `o`, and there is no single correct answer at the byte level.

NFC is still the right storage form: it matches the W3C/IETF recommendation, modern filesystems, XMP, and every external keyword tool MAKI rounds-trips through (Lightroom, Capture One, etc.). Locale-aware ordering on the tags page is a separate, presentation-layer concern and is not currently applied.

SEE ALSO

tag expand-ancestors – the other tag-data-cleanup migration; both are typically run once after major imports from external tools. tag rename, tag delete – targeted edits.

---

maki tag scan

NAME

maki-tag-scan – find assets whose tags don't pass the canonical normalisation

SYNOPSIS

```
maki [GLOBAL FLAGS] tag scan [--json]
```

DESCRIPTION

Read-only scan that walks every asset's tag list and reports any tag value that wouldn't survive a round-trip through `normalize_tag_for_storage` (the same filter every `maki tag` write site applies). Catches tags that bypassed normalisation at some prior ingestion point — typically an XMP keyword from Lightroom / CaptureOne / a manual `.xmp` edit that landed in `asset.tags` verbatim before the normaliser was applied to those code paths.

The check covers:

- Leading or trailing whitespace, including ASCII space, tab, and the various Unicode whitespace forms (non-breaking space U+00A0, zero-width space U+200B, ogham space U+1680, narrow no-break space U+202F, etc.).
- Leading or trailing `|` separators (`|foo`, `foo|` — semantically meaningless and break the `|`-prefix-anchor search syntax).
- Empty middle segments (`foo||bar`).
- Whitespace around `|` separators (`foo | bar`).
- Control characters.
- NFD-vs-NFC byte sequences for the same logical string (also fixable via `maki tag fix-unicode --apply`).
- Comma- or semicolon-joined "tags" that should have been split (`red, gold, white`).

For each offender, the output names the asset, dumps the raw tag with both its display form and its raw byte sequence in hex (so you can see which whitespace flavour actually snuck in), and shows the canonical form the normaliser would produce. Use it to find single problematic assets you can fix by hand, or to feed the IDs into `maki tag rename` / `maki tag delete --apply`.

OPTIONS

--json Emit JSON instead of human-readable lines. The result is `{scanned, malformed, assets: [{asset_id, name, tags: [{raw, raw_bytes_hex, normalised}]}]}`.

EXAMPLES

Default output:

```
maki tag scan
```

```
b6598546-2eb6-4126-b3e7-a6dbad5faee1  (unnamed)
    raw:  " |München"
    hex:  20 7c 4d c3 bc 6e 63 68 65 6e
    fix:  "München"
Tag scan: 12847 scanned, 1 with malformed tag(s).
```

JSON for tooling:

```
maki tag scan --json | jq '.assets[] | .asset_id'
```

SEE ALSO

tag fix-unicode – auto-fix the NFC/NFD subset of what `scan` reports. tag rename, tag delete – targeted fixes once you know which asset and tag to address.

---

maki tag expand-ancestors

NAME

maki-tag-expand-ancestors – expand hierarchical tags to include all ancestor paths

SYNOPSIS

```
maki [GLOBAL FLAGS] tag expand-ancestors [QUERY] [--asset <ID>] [--apply]
```

DESCRIPTION

Scans assets and adds missing ancestor paths for hierarchical tags. For example, if an asset has `subject|nature|landscape` but is missing `subject` and `subject|nature`, this command adds them.

This is a one-time cleanup for tags created before ancestor expansion was implemented. New tags added via `maki tag` already expand ancestors automatically.

ARGUMENTS

QUERY (optional) Optional search query (same syntax as `maki search`) selecting the assets to scan. Without a query and without `--asset`, all assets are scanned.

OPTIONS

–**asset <ID>** Limit the scan to a single asset (full or short ID).
–**apply** Execute changes. Without this flag, only reports what would change.

EXAMPLES

```
maki tag expand-ancestors              # preview
maki tag expand-ancestors --apply --log
```

---

maki tag export-vocabulary

NAME

maki-tag-export-vocabulary – export the current tag tree as a vocabulary file

SYNOPSIS

```
maki [GLOBAL FLAGS] tag export-vocabulary [--output <FILE>] [--format yaml|text|json] [--prune] [--default] [--counts]
```

DESCRIPTION

Generates a vocabulary file by merging the current catalog's tags with any existing vocabulary entries. Tags are grouped into a nested hierarchy.

Three output formats are supported:

- **yaml** (default): MAKI's native `vocabulary.yaml` — used by the catalog itself for autocomplete of planned tags.
- **text**: tab-indented keyword list compatible with Adobe Lightroom's *Import Keywords* and Capture One's *Import Keywords* (Keyword Text File). Use this to share your curated tag hierarchy with other tools so new ingest sessions there can autocomplete and tag against the same vocabulary.
- **json**: nested object suitable for programmatic consumption (UIs, dashboards, integration scripts). Every node has a `count` field; non-leaf nodes have a `children` map keyed by child segment name. Asset counts are always included regardless of `--counts`.

By default, planned-but-unused entries from the existing `vocabulary.yaml` are preserved in the export — the command adds new tags from usage without destroying your planned structure. Use `--prune` to remove unused entries and keep only tags that exist on at least one asset. Use `--default` to export only the built-in default vocabulary, ignoring both catalog tags and the existing vocabulary file.

OPTIONS

–**output <FILE>** Output file path. Defaults to `vocabulary.yaml`, `vocabulary.txt`, or `vocabulary.json` in the catalog root, depending on `--format`.

–**format <FORMAT>** Output format: `yaml` (default), `text`, or `json`. See *DESCRIPTION* for what each format is suited to.

–**prune** Remove vocabulary entries that have no assets. Only keep tags from actual usage.

–**default** Export only the built-in default vocabulary. Ignores catalog tags and existing `vocabulary.yaml`. Useful when the default vocabulary is updated (e.g. after a MAKI upgrade adds new categories) and you want to inspect or merge the changes without overwriting your curated file.

–**counts** Annotate each tag with its asset count. In **yaml**, counts appear as a trailing `# N assets` comment on every entry — still valid YAML; MAKI's autocomplete loader ignores comments. In **json**, every node has a `count` field already, so the flag is implied. In **text** (Lightroom / Capture One), comments aren't supported; the flag is silently ignored to keep the file importable.

EXAMPLES

```
maki tag export-vocabulary                                  # merge: keep planned + add used
maki tag export-vocabulary --prune                          # only used tags
maki tag export-vocabulary --output ~/my-vocab.yaml         # custom path
maki tag export-vocabulary --counts                         # YAML with `# N assets` trailing comments
maki tag export-vocabulary --default                        # built-in defaults only
maki tag export-vocabulary --default --output defaults.yaml # compare with your vocabulary
```

```
# Share the curated vocabulary with Lightroom / Capture One:
maki tag export-vocabulary --format text                    # vocabulary.txt in catalog root
maki tag export-vocabulary --format text --prune \
    --output ~/Desktop/maki-keywords.txt                    # used tags only, for import

# JSON for scripts / dashboards (always carries asset counts):
maki tag export-vocabulary --format json --prune > tags.json

# Lightroom:     Metadata → Import Keywords… → select maki-keywords.txt
# Capture One:   Image → Keywords → Import Keywords → Keyword Text File
```

YAML output with `--counts`

```
subject:  # 5 assets
  nature:  # 3 assets
    - landscape  # 2 assets
    - bird  # 1 asset
- legoland  # 1 asset
```

JSON output

```
{
  "subject": {
    "count": 5,
    "children": {
      "nature": {
        "count": 3,
        "children": {
          "landscape": {"count": 2},
          "bird": {"count": 1}
        }
      }
    }
  },
  "legoland": {"count": 1}
}
```

SEE ALSO

Tagging Guide – vocabulary file format and design recommendations.

maki edit

NAME

maki-edit – edit asset metadata (name, description, rating, color label, date, variant role)

SYNOPSIS

```
maki [GLOBAL FLAGS] edit <ASSET_ID> [OPTIONS]
```

DESCRIPTION

Sets or clears an asset's name, description, rating, color label, and creation date from the CLI. Can also change the role of a specific variant. At least one option must be provided.

Changes are written to both the YAML sidecar file (source of truth) and the SQLite catalog. Rating, description, and color label changes also trigger XMP write-back to any associated `.xmp` recipe files.

Rating is an integer from 1 to 5. Clearing it removes the rating entirely.

Color label accepts one of seven colors (case-insensitive): Red, Orange, Yellow, Green, Blue, Pink, Purple. The value is stored in canonical title-case. Clearing it removes the label.

Name is a custom display name for the asset. When set, it appears instead of the original filename in search results and the web UI.

Description is free-form text. Passing an empty string (`--description ""`) is equivalent to `--clear-description`.

Date accepts an ISO date string (e.g. `2024-12-25` or `2024-12-25T14:30:00`). This overrides the asset's `created_at` timestamp. Clearing it is not recommended since the field is always populated at import time.

Variant role (`--role` + `--variant`) changes the role of a specific variant within a multi-variant asset. Valid roles: `original`, `alternate`, `processed`, `export`, `sidecar`. This is useful when import or `fix-roles` assigns an incorrect role (e.g. marking a re-exported JPEG as "alternate" when it should be "export"). Role changes update both sidecar YAML and SQLite catalog, and recompute denormalized columns (best preview variant, primary format).

Asset IDs support unique prefix matching (see CLI Conventions).

ARGUMENTS

ASSET_ID (required) The asset ID or a unique prefix of it.

OPTIONS

–**name <TEXT>** Set the asset's display name.
–**clear-name** Remove the asset's display name (reverts to showing the original filename).
–**description <TEXT>** Set the asset's description. An empty string clears it.
–**clear-description** Remove the asset's description.
–**rating <1-5>** Set the asset's star rating (1 through 5).
–**clear-rating** Remove the asset's star rating.
–**label <COLOR>** Set the asset's color label. Accepts: Red, Orange, Yellow, Green, Blue, Pink, Purple (case-insensitive).
–**clear-label** Remove the asset's color label.
–**clear-tags** Remove all tags from the asset. Useful for cleaning up merged tags after splitting mis-grouped assets.
–**date <YYYY-MM-DD>** Set the asset's creation date (accepts ISO date or datetime strings).

–**clear-date** Reset the asset's creation date to the current time. The date is a required field, so it cannot be removed entirely; use `--date` to set a specific value.

–**role <ROLE>** Change a variant's role. Must be used with `--variant`. Accepts: original, alternate, processed, export, sidecar.

–**variant <HASH>** The content hash of the variant whose role should be changed. Must be used with `--role`. Use `maki show <id>` to find variant hashes.

`--json` outputs an `EditResult` with the fields that were changed and their new values.

EXAMPLES

Set a rating and description:

```
maki edit a1b2c3d4 --rating 5 --description "Best shot from the wedding ceremony"
```

Set a color label:

```
maki edit a1b2c --label Red
```

Give an asset a display name:

```
maki edit a1b2c3d4 --name "Sunset over Lake Constance"
```

Clear the rating and label:

```
maki edit a1b2c3d4 --clear-rating --clear-label
```

Correct an asset's date:

```
maki edit a1b2c3d4 --date "2024-12-25"
```

Clear the description (two equivalent forms):

```
maki edit a1b2c3d4 --clear-description
maki edit a1b2c3d4 --description ""
```

Remove all tags (e.g. after splitting a mis-grouped asset):

```
maki edit a1b2c3d4 --clear-tags
```

Change a variant's role from alternate to export:

```
maki edit a1b2c3d4 --role export --variant sha256:abcdef1234567890...
```

SEE ALSO

tag – add or remove tags. show – display full asset details including edited fields. search – `rating:`, `label:` filters for finding assets. fix-roles – batch re-role non-RAW variants in RAW+non-RAW groups.

---

maki group

NAME

maki-group – merge variants into a single asset

SYNOPSIS

```
maki [GLOBAL FLAGS] group <VARIANT_HASHES...>
```

DESCRIPTION

Merges multiple variants (identified by their content hashes) into a single asset. This is used to combine files that belong together but were imported as separate assets – for example, a RAW file and its exported TIFF, or multiple processing versions of the same shot.

The target asset is the oldest by creation date among the variants' current assets. All other ("donor") assets have their variants, tags, and recipes merged into the target.

Donor variants that have the role `original` are re-roled to `alternate` to avoid multiple originals on the same asset. This role change is applied in both the YAML sidecar and the SQLite catalog.

After merging, donor assets are deleted (their sidecar YAML files and catalog rows are removed). The target asset's denormalized columns (best variant hash, primary format, variant count) are updated.

ARGUMENTS

VARIANT_HASHES (required) Two or more content hashes (SHA-256 hex strings) of variants to group. Each hash must correspond to an existing variant in the catalog.

OPTIONS

This command only accepts global flags.

EXAMPLES

Group a RAW file and its export:

```
maki group abc123def456... 789012fed345...
```

Find variant hashes from asset details and group them:

```
maki show a1b2c --json | jq -r '.variants[].content_hash'
# Use the hashes from two different assets:
maki group <hash1> <hash2>
```

Group three variants (RAW, processed TIFF, final JPEG):

```
maki group abc123... def456... 789abc...
```

SEE ALSO

split – the inverse operation: extract variants into standalone assets. auto-group – automatically group assets by filename stem. show – display variant hashes for an asset. fix-roles – fix variant roles after grouping.

maki split

NAME

maki-split – extract variants from an asset into new standalone assets

SYNOPSIS

```
maki [GLOBAL FLAGS] split <ASSET_ID> <VARIANT_HASHES...>
```

DESCRIPTION

Splits one or more variants out of an existing asset, creating a new standalone asset for each extracted variant. This is the inverse of `maki group`.

Each extracted variant:

- Becomes the sole variant of a new asset with role `original`.
- Inherits tags, rating, color label, and description from the source asset.
- Takes associated recipes with it.
- Gets a deterministic UUID derived from its content hash.

The source asset retains all non-extracted variants. At least one variant must remain in the source asset. The **identity variant** (the one whose content hash generated the asset's UUID) cannot be split off — attempting to do so produces an error. Split the other variants instead.

ARGUMENTS

ASSET_ID (required) Asset ID (or unique prefix) to split.
VARIANT_HASHES (required) Content hashes of variants to extract.

OPTIONS

Standard global flags (`--json`, `--log`, `--time`).

EXAMPLES

Show variants of an asset:

```
maki show abc12345
```

Extract a specific variant into its own asset:

```
maki split abc12345 sha256:def456...
```

Extract multiple variants (each becomes a separate asset):

```
maki split abc12345 sha256:aaa... sha256:bbb...
```

JSON output for scripting:

```
maki --json split abc12345 sha256:def456...
```

SEE ALSO

group – the inverse operation: merge variants into a single asset. auto-group – automatically group assets by filename stem. show – display variant hashes for an asset.

maki auto-group

NAME

maki-auto-group – automatically group assets by filename stem

SYNOPSIS

```
maki [GLOBAL FLAGS] auto-group [QUERY] [--apply]
```

DESCRIPTION

Groups assets by filename stem using fuzzy prefix matching. This handles the common case where export tools (CaptureOne, Lightroom, Photoshop) append suffixes to the original filename: `Z91_8561.ARW` matches `Z91_8561-1-HighRes.tif` because `Z91_8561` is a prefix of `Z91_8561-1-HighRes` and the next character (`-`) is non-alphanumeric (a separator).

Directory-local grouping: By default, stem matching is restricted to assets within the same session root. A session root is the deepest directory component whose name matches the `[group] session_root_pattern` regex (default: `^\d{4}-\d{2}`, i.e. directories starting with a date like `2024-10-05-wedding`). Everything below a session root — `Capture/`, `Selects/`, `Output/` and their subdirectories — belongs to the same session. If no component matches, auto-group falls back to the parent directory (`maki import --auto-group` uses the imported directory instead, see `--auto-group`). This prevents `DSC_0001` from a 2019 shoot being grouped with `DSC_0001` from a 2024 shoot. Use `--global` to match across all directories (dangerous — only for carefully scoped queries). See the group configuration for customizing the session root pattern.

Fuzzy prefix matching rules: - Two stems match if the shorter is a prefix of the longer and the character immediately after the prefix in the longer string is non-alphanumeric (e.g., `-`, `_`, ` `, `(`). - This prevents false positives: `DSC_001` does not match `DSC_0010` because `0` is alphanumeric. - Stems are compared case-insensitively.

Chain resolution: When stems form a chain (e.g., `Z91_8561`, `Z91_8561-1`, `Z91_8561-1-HighRes`), all resolve to the shortest root (`Z91_8561`).

Target selection within each group: (1) prefer the asset that has a RAW variant, then (2) the oldest asset by creation date.

Without `--apply`, runs in report-only mode (dry run) and shows what would be grouped. With `--apply`, performs the merging: donor variants are moved to the target asset with their role set to `export` (for non-RAW in mixed RAW+non-RAW groups), tags and recipes are merged, and donor assets are deleted.

With `--log`, each stem group is printed as it's found during processing (real-time progress).

An optional search query scopes which assets are considered. Only assets matching the query participate in grouping.

ARGUMENTS

QUERY (optional) A search query (same syntax as `maki search`) to limit which assets participate in grouping.

OPTIONS

–**apply** Actually perform the grouping. Without this flag, the command only reports what it would do.

–**global** Match stems across all directories instead of restricting to the same directory neighborhood. **Dangerous**: may merge unrelated assets with the same camera-generated filename across different shoots. Only use with a carefully scoped query.

`--json` outputs an `AutoGroupResult` with `groups` (array of group details), `total_donors_merged`, `total_variants_moved`, and `dry_run` boolean.

`--log` prints per-group details to stderr.

EXAMPLES

Preview what auto-group would merge across the entire catalog:

```
maki auto-group
```

Auto-group only assets from a specific import path:

```
maki auto-group "path:Capture/2026-02-22"
```

Apply grouping after reviewing the dry-run report:

```
maki auto-group --apply
```

Auto-group assets with a specific tag:

```
maki auto-group "tag:wedding" --apply --log
```

Auto-group with JSON output for scripting:

```
maki auto-group --apply --json | jq '{merged: .total_donors_merged, moved: .total_variants_moved}'
```

SEE ALSO

group – manually group specific variants by content hash. fix-roles – fix variant roles after grouping. search – query syntax for scoping auto-group.

---

maki auto-tag *(Pro)*

NAME

maki-auto-tag – suggest or apply tags to images using AI (SigLIP zero-shot classification, multi-model)

SYNOPSIS

```
maki [GLOBAL FLAGS] auto-tag [QUERY] [OPTIONS]
maki [GLOBAL FLAGS] auto-tag [OPTIONS] --asset <ID>
maki [GLOBAL FLAGS] auto-tag [OPTIONS] --volume <LABEL>
maki [GLOBAL FLAGS] auto-tag --download [<MODEL_ID> | --model <ID>]
maki [GLOBAL FLAGS] auto-tag --remove-model [<MODEL_ID> | --model <ID>]
maki [GLOBAL FLAGS] auto-tag --list-models
maki [GLOBAL FLAGS] auto-tag --list-labels
maki [GLOBAL FLAGS] auto-tag --similar <ASSET_ID>
```

DESCRIPTION

Uses SigLIP vision-language models to classify images against a configurable tag vocabulary via zero-shot classification. Each image is encoded into an embedding, scored against all label embeddings using sigmoid scoring, and labels above the confidence threshold are suggested as tags.

Four models are available:

Model ID	Embedding	Size	Languages	Notes
`siglip-vit-b16-256`	768-dim	~207 MB	English only	Default, good balance
`siglip-vit-l16-256`	1024-dim	~670 MB	English only	Higher accuracy, slower
`siglip2-base-256-multi`	768-dim	~410 MB	**Multilingual**	German, French, Spanish, Italian, Japanese, Chinese, etc.
`siglip2-large-256-multi`	1024-dim	~920 MB	**Multilingual**	Same languages, higher accuracy, slower

Select with `--model <id>` or set `[ai] model` in `maki.toml`. The default is `siglip-vit-b16-256`. For non-English `text:` queries, use one of the multilingual SigLIP 2 variants — see Switching models in the setup guide.

By default runs in **report-only mode** – shows suggested tags without applying them. Use `--apply` to write suggested tags to assets.

Scope required: at least one scope (positional query, `--asset`, or `--volume`) must be specified to prevent accidental full-catalog processing. Use `""` (empty query) to process all assets.

Model files are downloaded from HuggingFace on first use. Use `--download` to pre-download, `--remove-model` to delete cached files, and `--list-models` to show all known models with download status, size, and active indicator.

Image selection: For each asset, the command looks for the best available image in this order: smart preview (2560px) → regular preview (800px) → original file on an online volume. Non-image assets (video, audio, documents) are skipped.

Embedding storage: Image embeddings are stored in the SQLite catalog (`embeddings` table) per model. Switching models does not corrupt existing embeddings; each model's embeddings are stored separately. The `--similar` flag uses stored embeddings from the active model to find visually similar assets via an in-memory index (dot product on L2-normalized vectors). Embeddings are also stored opportunistically by the web UI "Suggest tags", batch "Auto-tag", and "Review tags…" endpoints. Use `maki embed` to batch-generate embeddings without tagging.

Web UI review flow: For curated batches where false positives are costly (vacation sets, wedding shoots), the browse-page **Review tags...** button replaces unconditional `--apply` with a per-tag candidate-review UI. MAKI computes suggestions for the assets in scope, then opens a two-pane modal where each suggested tag is reviewed against its candidate set — uncheck false positives, drag the confidence slider, then Apply commits via the existing batch tag endpoint. See Tagging Guide → Reviewing suggestions for the full workflow.

Default labels: ~100 photography categories are built in (landscape, portrait, architecture, animals, food, etc.). Override with a custom labels file via `--labels`.

Prompt template: Each label is wrapped with a prompt template (default: `"a photograph of {}"`) before text encoding. Configurable via `[ai] prompt` in `maki.toml`.

OPTIONS

<QUERY> (positional, optional) Filter which assets to process using the same search syntax as `maki search`.

–**asset <ID>** Process a single asset by ID (supports prefix matching).

–**volume <LABEL>** Process only assets on a specific volume.

–**model <ID>** Select which SigLIP model to use. Available: `siglip-vit-b16-256` (default), `siglip-vit-l16-256`, `siglip2-base-256-multi`, `siglip2-large-256-multi`. Overrides `[ai] model` in `maki.toml`. Also applies to `--download` and `--remove-model`. For convenience, `--download` and `--remove-model` also accept the model id as a positional argument: `maki auto-tag --download siglip2-base-256-multi`.

–**threshold <FLOAT>** Minimum confidence score to suggest a tag (default: 0.1). Range: 0.0 to 1.0. Higher values produce fewer but more confident suggestions.

–**labels <FILE>** Path to a custom labels file (one label per line). Overrides the built-in default labels.

–**apply** Write suggested tags to assets. Without this flag, tags are only reported.

–**download** Download model files from HuggingFace if not already present. Returns early after download.

–**remove-model** Delete cached model files from disk. Returns early.

–**list-models** Show model download status and file sizes. Returns early.

–**list-labels** Print the active label list (one per line) and exit. Shows the built-in defaults, or the labels from `--labels` / `[ai] labels` config if set. Works without a catalog when using defaults. Pipe to a file to create a custom labels blueprint: `maki auto-tag --list-labels > my-labels.txt`. Supports `--json` (outputs a JSON array).

–**similar <ASSET_ID>** Find the 20 most visually similar assets to the given asset, using stored embeddings. If the target asset has no stored embedding, it is encoded on-the-fly. Other assets must have been previously processed by `auto-tag` or `maki embed`.

`--json` outputs an `AutoTagResult` with `assets_processed`, `assets_skipped`, `tags_suggested`, `tags_applied`, `errors`, `dry_run`, and per-asset `suggestions`.

`--log` prints per-asset status to stderr.

EXAMPLES

Download the default model (first-time setup):

```
maki auto-tag --download
```

Download the larger model for higher accuracy:

```
maki auto-tag --download --model siglip-vit-l16-256
```

Preview suggested tags for all images (report-only):

```
maki auto-tag ""
```

Auto-tag a specific asset and apply the tags:

```
maki auto-tag --asset a1b2c3d4 --apply
```

Auto-tag images matching a search query with a higher threshold:

```
maki auto-tag "type:image rating:4+" --threshold 0.4 --apply
```

Use the larger model for a specific query:

```
maki auto-tag "tag:unreviewed" --model siglip-vit-l16-256 --apply
```

Use a custom labels file:

```
maki auto-tag --labels my-labels.txt "" --apply
```

Find visually similar images:

```
maki auto-tag --similar a1b2c3d4
```

Export default labels as a blueprint for customization:

```
maki auto-tag --list-labels > my-labels.txt
# Edit the file, then use it:
maki auto-tag --labels my-labels.txt --apply
```

Show model status:

```
maki auto-tag --list-models
```

Auto-tag with JSON output for scripting:

```
maki auto-tag "tag:unreviewed" --json | jq '.suggestions[] | {asset: .asset_id, tags: [.suggested_tags[].tag]}'
```

SEE ALSO

embed – batch-generate embeddings for similarity search. tag – manually add or remove tags. search – `tag:` filter for finding tagged assets. Configuration – `[ai]` section for default threshold, labels, and prompt template.

---

maki embed *(Pro)*

NAME

maki-embed – batch-generate image embeddings for visual similarity search

SYNOPSIS

```
maki [GLOBAL FLAGS] embed [QUERY] [OPTIONS]
maki [GLOBAL FLAGS] embed --asset <ID> [OPTIONS]
maki [GLOBAL FLAGS] embed --volume <LABEL> [OPTIONS]
maki [GLOBAL FLAGS] embed --export
```

DESCRIPTION

Pre-computes image embeddings for visual similarity search (`maki auto-tag --similar` and the web UI "Find similar" button) without applying any tags. This is useful for building up the similarity search index across your catalog.

For each matching asset, the command finds the best available image (smart preview → regular preview → original file on an online volume), encodes it with SigLIP, and stores the embedding in the SQLite catalog's `embeddings` table and as a binary file under `embeddings/<model>/<prefix>/<asset_id>.bin`.

By default, assets that already have a stored embedding **for the active model** are skipped. Embeddings are keyed by `(asset_id, model_id)`, so switching to a different model and running `maki embed ''` will generate embeddings for the new model only — existing embeddings for other models are untouched and the run is restart-safe (resumes after Ctrl-C). See Switching models in the setup guide.

Use `--force` only when you want to **regenerate** embeddings that already exist for the active model — for example, after replacing the on-disk model file with a corrected version. For a model switch, do NOT use `--force`; it would re-embed everything and waste hours of compute.

Scope required: at least one scope (positional query, `--asset`, or `--volume`) must be specified to prevent accidental full-catalog processing. Use `""` (empty query) to process all assets.

Non-destructive: embedding generation does not modify any asset metadata, tags, or files. It only writes to the `embeddings` table.

OPTIONS

<QUERY> (positional, optional) Filter which assets to process using the same search syntax as `maki search`.

–**asset <ID>** Process a single asset by ID (supports prefix matching).

–**volume <LABEL>** Process only assets on a specific volume.

–**model <ID>** Select which SigLIP model to use. Available: `siglip-vit-b16-256` (default), `siglip-vit-l16-256`, `siglip2-base-256-multi`, `siglip2-large-256-multi`. Overrides `[ai] model` in `maki.toml`.

–**force** Re-generate embeddings even if they already exist for the active model. **Not needed when switching models** — embeddings are stored per `(asset_id, model_id)`, so a new model starts with an empty store and `maki embed ''` (without `--force`) generates the missing embeddings only.

–**export** Export all existing embeddings from SQLite to binary files. No scope filter required. Useful as a one-time migration to populate the file-based persistence layer from existing data.

`--json` outputs `{embedded, skipped, errors, model, force}`.

`--log` prints per-asset status to stderr.

EXAMPLES

Generate embeddings for all assets:

```
maki embed ""
```

Generate embeddings for a single asset:

```
maki embed --asset a1b2c3d4
```

Generate embeddings for a specific volume:

```
maki embed --volume "Photos 2024"
```

Switch to a different model and generate the missing embeddings (no `--force` — only assets without an embedding for the new model are processed):

```
maki embed "" --model siglip-vit-l16-256
```

Force re-generation of all embeddings for the active model (only when you really want to rebuild from scratch — takes hours on large catalogs):

```
maki embed "" --force
```

Generate embeddings for untagged images only:

```
maki embed "tag:none type:image"
```

Check progress with JSON output:

```
maki embed "" --json | jq '{embedded, skipped}'
```

Export all embeddings to binary files (one-time migration):

```
maki embed --export
```

SEE ALSO

auto-tag – AI tag suggestion and visual similarity search. Configuration – `[ai]` section for model and model directory settings.

---

maki ai export-vocabulary *(Pro)*

NAME

maki-ai-export-vocabulary – export the AI label vocabulary (labels + hierarchical mapping) as YAML

SYNOPSIS

```
maki [GLOBAL FLAGS] ai export-vocabulary [--output <FILE>] [--default]
```

DESCRIPTION

Writes the vocabulary that `auto-tag` and "Suggest tags" score against — the flat SigLIP labels and their mapping to hierarchical MAKI tags — as a YAML file you can edit and point `[ai] labels` at.

Without flags, emits the *active* vocabulary: whatever `[ai] labels` in `maki.toml` points to, or the built-in default when no labels file is configured. With `--default`, always emits the built-in default regardless of config. When the active vocabulary is a legacy `.txt` flat-list file, the export converts it to YAML with identity mappings (`label: null`) so you can start adding hierarchy without reordering anything.

Typical use: `maki ai export-vocabulary --default > my-labels.yaml`, edit the right-hand sides into your own taxonomy, set `[ai] labels = "my-labels.yaml"`, and every suggestion then lands on your hierarchy. See the Tagging Guide for the workflow. (Not to be confused with `tag export-vocabulary`, which exports the tags *in the catalog*.)

OPTIONS

-o, –output <FILE> Output file. Default: stdout (use shell redirection to save).

–default Emit the built-in default vocabulary instead of the active one.

–json (global) Emit a JSON object on stdout instead of the raw YAML. With `--output FILE`: `{"source": "...", "bytes": N, "output": "FILE"}`. Without `--output`: `{"source": "...", "bytes": N, "output": null, "yaml": "<the YAML document>"}`. `source` is `builtin-default`, `config-yaml` (the `[ai] labels` file was YAML and is passed through verbatim), or `config-txt` (a legacy flat list converted to YAML).

EXAMPLES

```
maki ai export-vocabulary --default > my-labels.yaml     # built-in default, raw YAML
maki ai export-vocabulary --output vocab.yaml            # active vocabulary to a file
maki ai export-vocabulary --json | jq -r .yaml           # same YAML, via the JSON wrapper
maki ai export-vocabulary --output vocab.yaml --json     # {"source":"config-yaml","bytes":2311,"output":"vocab.ya
```

SEE ALSO

auto-tag – apply the vocabulary to assets. tag export-vocabulary – export the catalog's tag vocabulary.

---

maki audio analyze

NAME

maki-audio-analyze – detect musical key and tempo of audio assets via external tools

SYNOPSIS

```
maki [GLOBAL FLAGS] audio analyze [QUERY] [OPTIONS]
```

DESCRIPTION

Runs external analyzers on audio assets and stores the results as searchable metadata: the musical key (via the configured `[audio] key_command`, default keyfinder-cli) and the tempo in BPM (via `[audio] bpm_command`, default beat_this; MAKI derives BPM from the median inter-beat interval of the tracker's beat timestamps, which is robust against occasional missed or extra beats).

Results are written to the analyzed variant's `source_metadata` (`audio_key` / `audio_bpm`) — sidecar first, catalog second — and denormalized into typed columns, so the `key:` and `bpm:` search filters pick them up immediately. The values also appear as badges on the web asset detail page.

Analysis is **explicitly opt-in** rather than part of import: key detection is only trustworthy on full mixes — running it on voice memos or single-instrument practice recordings produces confident-looking nonsense. Scope the query to the material where the answer means something (`path:BackingTracks`, `tag:genre|…`).

MAKI never processes audio itself — it records what the tools report. Missing tools are warned about and that half of the analysis is skipped; whichever analyzer exists still runs. Assets that already carry both fields are skipped (use `--force` to re-analyze); assets whose volumes are offline are reported and skipped.

ARGUMENTS

QUERY (optional) Search query to scope the analysis (same syntax as `maki search`). `type:audio` is always implied. Empty analyzes all audio assets.

OPTIONS

–**force** Re-analyze assets that already have key/BPM values.
–**dry-run** Report what would be analyzed without running the tools.

`--json` outputs an `AudioAnalyzeResult` object with fields: `assets_matched`, `analyzed`, `skipped_existing`, `skipped_offline`, `failed`, `keys_set`, `bpms_set`, `key_tool_available`, `bpm_tool_available`, `errors`.

EXAMPLES

Analyze all backing tracks:

```
maki audio analyze "path:BackingTracks" --log
```

Re-analyze a single asset after replacing the file:

```
maki audio analyze "id:6ba7b810" --force
```

Then search by the results:

```
maki search "key:am bpm:100-140"
```

SEE ALSO

Search filters – `key:` and `bpm:` filter reference. Configuration – `[audio]` section for analyzer commands.

maki describe *(Pro)*

NAME

maki-describe – generate image descriptions using a vision-language model (VLM)

SYNOPSIS

```
maki [GLOBAL FLAGS] describe [QUERY] [OPTIONS]
```

DESCRIPTION

Sends preview images to a VLM server and generates natural language descriptions and/or tags. The command uses the OpenAI-compatible chat completions API, which is implemented by Ollama, LM Studio, vLLM, and most local inference servers.

Three modes are available via `--mode` (default from `[vlm] mode` in `maki.toml`, or `describe`):

- **describe** — generates a natural language description for each asset.
- **tags** — asks the VLM to suggest tags, returned as a JSON array. Tags are deduplicated (case-insensitive) and existing asset tags are preserved.
- **both** — runs describe and tags as two separate VLM calls per asset, combining the results. This is equivalent to running `--mode describe` and `--mode tags` independently, so each call uses its optimal prompt.

By default, the command runs in **report-only mode**: results are generated and displayed but not saved. Use `--apply` to write descriptions/tags to assets. Use `--dry-run` to see what would be processed without calling the VLM at all.

The command requires at least one scope (positional query, `--asset`, or `--volume`) to prevent accidental processing of the entire catalog.

For each asset, the command: 1. Checks if a description already exists (skips unless `--force` is set; tags mode always runs) 2. Finds the best available image (smart preview > regular preview > original on an online volume) 3. Base64-encodes the image and sends it to the VLM endpoint 4. Parses the response and optionally saves descriptions/tags to the asset

OPTIONS

<QUERY> (positional, optional) Search query to scope which assets are described. Same syntax as `maki search`.

–**asset <ID>** Process a single asset (ID or unique prefix).

–**volume <LABEL>** Limit to assets on a specific volume.

–**model <NAME>** VLM model name. Default from `[vlm] model` in `maki.toml`, or `qwen2.5vl:3b`.

–**endpoint <URL>** VLM server base URL. Default from `[vlm] endpoint` in `maki.toml`, or `http://localhost:11434`.

–**prompt <TEXT>** Custom prompt sent to the VLM. Default from `[vlm] prompt` in `maki.toml`, or a built-in photography-focused prompt. In `--mode both`, custom prompts are ignored because each call uses its specialized built-in prompt.

–**max-tokens <N>** Maximum tokens in the VLM response. Default from `[vlm] max_tokens` in `maki.toml`, or `500`.

–**timeout <SECONDS>** Timeout for each VLM request. Default from `[vlm] timeout` in `maki.toml`, or `300`. Increase for larger models or first-time model loading.

–**mode <MODE>** Output mode: `describe`, `tags`, or `both`. Default from `[vlm] mode` in `maki.toml`, or `describe`. In `both` mode, two VLM calls are made per asset — one for description, one for tags.

–**temperature <FLOAT>** Sampling temperature controlling randomness. `0.0` = deterministic (always picks the most likely token), `0.7` = balanced (default), `1.0+` = more creative. Lower values produce more consistent but potentially blander output. Default from `[vlm] temperature` in `maki.toml`, or `0.7`.

–**num-ctx <N>** Context window size passed to the VLM server (Ollama `num_ctx`). When non-zero, overrides the model's default context length. Useful for models that benefit from a larger context (e.g., `--num-ctx 4096`). Default: `0` (server default).
–**top-p <FLOAT>** Nucleus sampling threshold. Only tokens whose cumulative probability exceeds this value are considered. Lower values produce more focused output. Default: `0.0` (server default).
–**top-k <N>** Top-k sampling: limit token selection to the k most likely candidates. Lower values produce more deterministic output. Default: `0` (server default).
–**repeat-penalty <FLOAT>** Repetition penalty factor. Values above `1.0` discourage the model from repeating tokens. Default: `0.0` (server default).
–**apply** Write descriptions and/or tags to assets. Without this flag, results are generated and displayed but not saved.
–**force** Overwrite existing descriptions. By default, assets that already have a description are skipped.
–**dry-run** Show what would be processed without calling the VLM. No network requests are made.
–**check** Test connectivity to the VLM endpoint. Prints the server status and available models, then exits. Does not process any assets.

EXAMPLES

Check that Ollama is running and see available models:

```
maki describe --check
```

Preview descriptions for undescribed assets (report-only):

```
maki describe "description:none type:image"
```

Generate and apply descriptions to a specific volume:

```
maki describe --volume "Photos 2024" --apply
```

Describe a single asset:

```
maki describe --asset a1b2c3d4 --apply
```

Use a faster model for batch processing:

```
maki describe "date:2024-06" --model moondream --apply
```

Use a remote server or larger model:

```
maki describe --endpoint http://gpu-server:11434 --model qwen2.5vl:7b --apply
```

Generate tags only:

```
maki describe --mode tags "tag:untagged" --apply
```

Generate both descriptions and tags in one pass (two VLM calls per asset):

```
maki describe --mode both --asset a1b2c3d4 --apply
```

Custom prompt for architectural photography:

```
maki describe --prompt "Describe the architectural style, materials, and features." "tag:architecture" --apply
```

Deterministic output (temperature 0) for reproducible batch tagging:

```
maki describe --mode tags --temperature 0 "date:2024-06" --apply
```

Increase timeout for a large model's first load:

```
maki describe --model qwen2.5vl:7b --timeout 300 --asset a1b2c3d4
```

Use a larger context window and nucleus sampling with a specific model:

```
maki describe --model qwen3-vl:4b --num-ctx 4096 --top-p 0.9 --apply
```

Overwrite existing descriptions with a better model:

```
maki describe "rating:4+" --model qwen2.5vl:7b --force --apply
```

Dry run with JSON output:

```
maki describe "rating:4+" --dry-run --json
```

Use a cloud API (OpenAI-compatible endpoint):

```
maki describe ""
```

SEE ALSO

edit – manually set or clear an asset's description. VLM Model Guide – tested models, backends, hardware guide, and quality comparison. Configuration – `[vlm]` section for endpoint, model, and prompt settings. VLM Setup (User Guide) – how to set up a local VLM server.

Previous: Setup Commands – `init`, `volume add`, `volume list`. Next: Organize Commands – `collection`, `saved-search`.

Organize Commands

Commands for curating static collections, managing saved searches (smart albums), grouping assets into stacks, and managing face recognition. Collections are manual albums; saved searches are dynamic queries that always reflect the current catalog state. Stacks collapse related assets in the browse grid.

Command group	Subcommands
collection	create, list, show, add, remove, delete
saved-search	save, list, run, delete
stack	create, add, remove, pick, dissolve, list, show, from-tag
auto-stack *(Pro)*	—
faces *(Pro)*	detect, download, status, cluster, clean, similarity, dump-aligned, people, name, merge, delete-person, unassign, export

maki collection create

NAME

maki-collection-create – create a new collection

SYNOPSIS

```
maki [GLOBAL FLAGS] collection create <NAME> [--description <TEXT>]
```

Alias: `maki col create`

DESCRIPTION

Creates a new empty collection. Collections are manually curated lists of asset IDs, similar to static albums in photo management tools. They are backed by SQLite tables for fast queries and a `collections.yaml` file at the catalog root for persistence across `rebuild-catalog`.

Collection names must be unique. Attempting to create a collection with an existing name produces an error.

ARGUMENTS

NAME (required) The name for the new collection.

OPTIONS

–**description <TEXT>** An optional description for the collection.

`--json` outputs the created collection's details.

EXAMPLES

Create a simple collection:

```
maki collection create "Best of 2026"
```

Create a collection with a description:

```
maki col create "Wedding Portfolio" --description "Final selects for client delivery"
```

Create with JSON output:

```
maki col create "Travel" --json
```

SEE ALSO

collection add – add assets to a collection. collection show – view collection contents. search – `collection:` filter for searching within a collection.

maki collection list

NAME

maki-collection-list – list all collections

SYNOPSIS

`maki [GLOBAL FLAGS] collection list`

Alias: `maki col list`

DESCRIPTION

Lists all collections in the catalog, showing each collection's name, description (if any), and the number of assets it contains.

ARGUMENTS

None.

OPTIONS

This command only accepts global flags.

`--json` outputs an array of collection objects.

EXAMPLES

List all collections:

```
maki collection list
```

List collections as JSON and extract names:

```
maki col list --json | jq '.[].name'
```

Count total collections:

```
maki col list --json | jq 'length'
```

SEE ALSO

collection create – create a new collection. collection show – view a specific collection's contents.

maki collection show

NAME

maki-collection-show – show the contents of a collection

SYNOPSIS

```
maki [GLOBAL FLAGS] collection show <NAME> [--format <FMT>]
```

Alias: `maki col show`

DESCRIPTION

Displays the assets belonging to a collection. Output format can be customized using the same format presets and template syntax as `maki search`.

ARGUMENTS

NAME (required) The name of the collection to display.

OPTIONS

–**format <FMT>** Output format. Presets: `ids`, `short` (default), `full`, `json`. Custom templates use `{placeholder}` syntax (e.g., `'{id}\t{name}'`).

EXAMPLES

Show a collection's contents:

```
maki collection show "Best of 2026"
```

Get just the asset IDs for piping:

```
maki col show "Wedding Portfolio" --format ids
```

Show full details including tags:

```
maki col show "Travel" --format full
```

Export collection as JSON:

```
maki col show "Favorites" --format json | jq '.[].id'
```

SEE ALSO

collection add – add assets to the collection. collection remove – remove assets from the collection. search – `collection:` filter for searching within collections.

maki collection add

NAME

maki-collection-add – add assets to a collection

SYNOPSIS

```
maki [GLOBAL FLAGS] collection add <NAME> <ASSET_IDS...>
```

Alias: `maki col add`

DESCRIPTION

Adds one or more assets to an existing collection. Asset IDs that are already in the collection are silently ignored (no duplicates are created).

Supports stdin piping for integration with `maki search -q` and shell scripting.

ARGUMENTS

NAME (required) The name of the collection to add assets to.
ASSET_IDS (required) One or more asset IDs to add. Also accepts IDs from stdin.

OPTIONS

This command only accepts global flags.

EXAMPLES

Add specific assets to a collection:

```
maki collection add "Favorites" a1b2c3d4-... e5f67890-...
```

Pipe search results into a collection:

```
maki search -q "rating:5 tag:travel" | xargs maki col add "Travel Best"
```

Add all 5-star landscape photos to a collection:

```
maki search -q "rating:5 tag:landscape" | xargs maki col add "Portfolio"
```

Add assets from a saved search:

```
maki ss run "Recent Imports" --format ids | xargs maki col add "Review Queue"
```

SEE ALSO

collection remove – remove assets from a collection. collection show – view collection contents. search – find assets to add.

maki collection remove

NAME

maki-collection-remove – remove assets from a collection

SYNOPSIS

```
maki [GLOBAL FLAGS] collection remove <NAME> <ASSET_IDS...>
```

Alias: `maki col remove`

DESCRIPTION

Removes one or more assets from a collection. The assets themselves are not deleted – only their membership in the collection is removed. Asset IDs not present in the collection are silently ignored.

ARGUMENTS

NAME (required) The name of the collection to remove assets from.
ASSET_IDS (required) One or more asset IDs to remove.

OPTIONS

This command only accepts global flags.

EXAMPLES

Remove a single asset from a collection:

```
maki collection remove "Favorites" a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

Remove multiple assets:

```
maki col remove "Review Queue" a1b2c3d4-... e5f67890-...
```

Remove all assets with a certain label from a collection:

```
maki search -q "collection:Portfolio label:Red" --format ids | xargs maki col remove "Portfolio"
```

SEE ALSO

collection add – add assets to a collection. collection delete – delete the entire collection.

maki collection delete

NAME

maki-collection-delete – delete a collection

SYNOPSIS

```
maki [GLOBAL FLAGS] collection delete <NAME>
```

Alias: `maki col delete`

DESCRIPTION

Deletes a collection entirely. This removes the collection record and all its membership entries. The assets themselves are not affected – only the collection is removed.

ARGUMENTS

NAME (required) The name of the collection to delete.

OPTIONS

This command only accepts global flags.

EXAMPLES

Delete a collection:

```
maki collection delete "Old Review Queue"
```

Delete using the alias:

```
maki col delete "Temporary"
```

Delete with JSON confirmation:

```
maki col delete "Drafts" --json
```

SEE ALSO

collection create – create a new collection. collection list – list all collections.

---

maki saved-search save

NAME

maki-saved-search-save – save a search query with a name

SYNOPSIS

```
maki [GLOBAL FLAGS] saved-search save <NAME> <QUERY> [--sort <SORT>] [--favorite]
```

Alias: `maki ss save`

DESCRIPTION

Saves a search query under a name for later re-use. Saved searches are stored in `searches.toml` at the catalog root and function as smart albums – the results update dynamically as the catalog changes.

If a saved search with the same name already exists, it is replaced.

Saved searches appear as clickable chips in the web UI browse page and can be executed from the CLI with `maki saved-search run`.

ARGUMENTS

NAME (required) A name for the saved search.
QUERY (required) The search query string, using the same syntax as `maki search`.

OPTIONS

–**sort <SORT>** Sort order for results. Values: `date_desc` (default), `date_asc`, `name_asc`, `name_desc`, `size_asc`, `size_desc`.

–**favorite** Mark the saved search as a favorite. Favorite searches are shown prominently as chips on the web UI browse page.

`--json` outputs the saved search entry.

EXAMPLES

Save a search for highly-rated landscapes:

```
maki saved-search save "Best Landscapes" "tag:landscape rating:4+"
```

Save with a custom sort order:

```
maki ss save "Recent Videos" "type:video" --sort date_desc
```

Save a search using quoted filter values:

```
maki ss save "Canon Portraits" 'camera:"Canon EOS R5" tag:portrait'
```

Save a path-scoped search:

```
maki ss save "February Shoot" "path:Capture/2026-02"
```

SEE ALSO

saved-search run – execute a saved search. saved-search list – list all saved searches. saved-search delete – delete a saved search. search – query syntax reference.

maki saved-search list

NAME

maki-saved-search-list – list all saved searches

SYNOPSIS

```
maki [GLOBAL FLAGS] saved-search list
```

Alias: `maki ss list`

DESCRIPTION

Lists all saved searches stored in the catalog, showing each search's name, query, and sort order.

ARGUMENTS

None.

OPTIONS

This command only accepts global flags.

`--json` outputs an array of saved search objects.

EXAMPLES

List all saved searches:

```
maki saved-search list
```

List as JSON:

```
maki ss list --json
```

Count saved searches:

```
maki ss list --json | jq 'length'
```

SEE ALSO

saved-search save – create or update a saved search. saved-search run – execute a saved search.

maki saved-search run

NAME

maki-saved-search-run – execute a saved search and display results

SYNOPSIS

```
maki [GLOBAL FLAGS] saved-search run <NAME> [--format <FMT>]
```

Alias: `maki ss run`

DESCRIPTION

Executes a previously saved search by name and displays the results. The stored query is run against the current state of the catalog, so results reflect any changes since the search was saved.

The sort order saved with the search is applied. Output format can be overridden with `--format`.

ARGUMENTS

NAME (required) The name of the saved search to execute.

OPTIONS

–**format <FMT>** Output format. Presets: `ids`, `short` (default), `full`, `json`. Custom templates use `{placeholder}` syntax.

EXAMPLES

Run a saved search:

```
maki saved-search run "Best Landscapes"
```

Run and get just IDs for piping:

```
maki ss run "Recent Videos" --format ids
```

Run a saved search and add results to a collection:

```
maki ss run "Best Landscapes" --format ids | xargs maki col add "Portfolio"
```

Run with JSON output:

```
maki ss run "Canon Portraits" --format json | jq '.[].id'
```

SEE ALSO

saved-search save – create or update a saved search. collection add – add search results to a collection.

maki saved-search delete

NAME

maki-saved-search-delete – delete a saved search

SYNOPSIS

```
maki [GLOBAL FLAGS] saved-search delete <NAME>
```

Alias: `maki ss delete`

DESCRIPTION

Deletes a saved search by name. The search is removed from `searches.toml`. This does not affect any assets or collections.

ARGUMENTS

NAME (required) The name of the saved search to delete.

OPTIONS

This command only accepts global flags.

EXAMPLES

Delete a saved search:

```
maki saved-search delete "Old Query"
```

Delete using the alias:

```
maki ss delete "Temporary Search"
```

Delete with JSON confirmation:

```
maki ss delete "Drafts" --json
```

SEE ALSO

saved-search save – create a new saved search. saved-search list – list all saved searches.

---

maki stack create

NAME

maki-stack-create – create a new stack from the given assets

SYNOPSIS

```
maki [GLOBAL FLAGS] stack create <ASSET_IDS...>
```

Alias: `maki st create`

DESCRIPTION

Creates a new stack from two or more assets. Stacks are lightweight anonymous groups for visually related images – burst shots, bracketing sequences, similar scenes. The first asset ID becomes the stack's "pick" (displayed in the browse grid).

Each asset can belong to at most one stack. Attempting to stack an asset that is already in a stack produces an error.

Stacks are persisted in `stacks.yaml` at the catalog root and restored during `rebuild-catalog`.

ARGUMENTS

ASSET_IDS (required) Two or more asset IDs. The first ID becomes the pick (position 0).

OPTIONS

This command only accepts global flags.

`--json` outputs the created stack's details (id, member_count, asset_ids).

EXAMPLES

Create a stack from three burst shots (first becomes pick):

```
maki stack create a1b2c3d4-... e5f67890-... f1a2b3c4-...
```

Create using the alias with JSON output:

```
maki st create a1b2c3d4-... e5f67890-... --json
```

SEE ALSO

stack add – add assets to an existing stack. stack pick – change which asset is the pick. stack dissolve – dissolve a stack entirely.

maki stack add

NAME

maki-stack-add – add assets to an existing stack

SYNOPSIS

```
maki [GLOBAL FLAGS] stack add <REFERENCE> <ASSET_IDS...>
```

Alias: `maki st add`

DESCRIPTION

Adds one or more assets to an existing stack. The reference asset identifies which stack to add to – it must already be a member of a stack. New assets are appended to the end of the stack's member list.

Assets that are already in a stack (any stack) cannot be added and will produce an error.

ARGUMENTS

REFERENCE (required) Any asset ID that is already in the target stack.
ASSET_IDS (required) One or more asset IDs to add to the stack.

OPTIONS

This command only accepts global flags.

`--json` outputs `{"added": N}` with the count of assets added.

EXAMPLES

Add an asset to the stack containing a1b2c3d4:

```
maki stack add a1b2c3d4-... new-asset-id-...
```

Add multiple assets:

```
maki st add a1b2c3d4-... b2c3d4e5-... c3d4e5f6-...
```

SEE ALSO

stack create – create a new stack. stack remove – remove assets from a stack.

---

maki stack remove

NAME

maki-stack-remove – remove assets from their stack

SYNOPSIS

```
maki [GLOBAL FLAGS] stack remove <ASSET_IDS...>
```

Alias: `maki st remove`

DESCRIPTION

Removes one or more assets from their stacks. Each asset is removed from whatever stack it belongs to. If removing an asset causes a stack to have fewer than 2 members, the stack is automatically dissolved.

Assets not currently in a stack are silently skipped.

ARGUMENTS

ASSET_IDS (required) One or more asset IDs to remove from their stacks.

OPTIONS

This command only accepts global flags.

`--json` outputs `{"removed": N}` with the count of assets removed.

EXAMPLES

Remove a single asset from its stack:

```
maki stack remove a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

Remove multiple assets:

```
maki st remove a1b2c3d4-... e5f67890-...
```

SEE ALSO

stack add – add assets to a stack. stack dissolve – dissolve an entire stack at once.

---

maki stack pick

NAME

maki-stack-pick – set the pick (top) of a stack

SYNOPSIS

```
maki [GLOBAL FLAGS] stack pick <ASSET_ID>
```

Alias: `maki st pick`

DESCRIPTION

Promotes an asset to position 0 (the "pick") of its stack. The pick is the asset shown in the browse grid when stacks are collapsed. The asset must already be a member of a stack.

ARGUMENTS

ASSET_ID (required) The asset ID to make the pick. Must be in a stack.

OPTIONS

This command only accepts global flags.

`--json` outputs `{"pick": "<asset_id>"}`.

EXAMPLES

Set a specific asset as the stack pick:

```
maki stack pick a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

Set pick using the alias:

```
maki st pick a1b2c3d4-...
```

SEE ALSO

stack show – view current stack members and pick. stack create – the first asset in create becomes the initial pick.

maki stack dissolve

NAME

maki-stack-dissolve – dissolve an entire stack

SYNOPSIS

```
maki [GLOBAL FLAGS] stack dissolve <ASSET_ID>
```

Alias: `maki st dissolve`

DESCRIPTION

Dissolves the stack that contains the given asset. All member assets become unstacked. The assets themselves are not affected – only the stack grouping is removed.

ARGUMENTS

ASSET_ID (required) Any asset ID in the stack to dissolve.

OPTIONS

This command only accepts global flags.

`--json` outputs `{"status": "dissolved"}`.

EXAMPLES

Dissolve a stack:

```
maki stack dissolve a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

Dissolve using the alias:

```
maki st dissolve a1b2c3d4-...
```

SEE ALSO

stack remove – remove individual assets instead of dissolving the whole stack. stack list – list all stacks.

maki stack list

NAME

maki-stack-list – list all stacks

SYNOPSIS

```
maki [GLOBAL FLAGS] stack list
```

Alias: `maki st list`

DESCRIPTION

Lists all stacks in the catalog, showing each stack's ID, member count, creation date, and pick asset ID.

ARGUMENTS

None.

OPTIONS

This command only accepts global flags.

`--json` outputs an array of `StackSummary` objects.

EXAMPLES

List all stacks:

```
maki stack list
```

List as JSON and count stacks:

```
maki st list --json | jq 'length'
```

SEE ALSO

stack show – view members of a specific stack. stack create – create a new stack.

---

maki stack show

NAME

maki-stack-show – show members of a stack

SYNOPSIS

```
maki [GLOBAL FLAGS] stack show <ASSET_ID> [--format <FMT>]
```

Alias: `maki st show`

DESCRIPTION

Displays the ordered member list of the stack containing the given asset. The pick (position 0) is shown first. Output format can be customized using the same format presets and template syntax as `maki search`.

ARGUMENTS

ASSET_ID (required) Any asset ID that belongs to a stack.

OPTIONS

–**format <FMT>** Output format. Presets: `ids`, `short` (default), `full`, `json`. Custom templates use `{placeholder}` syntax.

EXAMPLES

Show members of a stack:

```
maki stack show a1b2c3d4-...
```

Get just the member IDs:

```
maki st show a1b2c3d4-... --format ids
```

Show as JSON:

```
maki st show a1b2c3d4-... --json
```

SEE ALSO

stack pick – change the pick. stack list – list all stacks. search – `stacked:true` filter finds all stacked assets.

maki stack from-tag

NAME

maki-stack-from-tag – convert matching tags into stacks

SYNOPSIS

```
maki [GLOBAL FLAGS] stack from-tag <PATTERN> [--remove-tags] [--apply]
```

Alias: `maki st from-tag`

DESCRIPTION

Finds assets whose tags match a pattern containing a `{}` wildcard, groups them by the wildcard value, and creates a stack from each group. This automates the common workflow of converting tag-based groupings (e.g., from CaptureOne or Lightroom) into maki stacks.

Without `--apply`, runs in **report-only mode** showing what stacks would be created. With `--apply`, creates the stacks. With `--remove-tags`, the matched tag is removed from **every asset that carries it** — including orphan tags (single-asset tags that can't form a stack) and already-stacked assets. This makes `--remove-tags` a true cleanup flag for post-migration tag removal.

ARGUMENTS

PATTERN (required) A tag pattern with `{}` as a wildcard placeholder. For example, `"Aperture Stack {}"` matches tags like "Aperture Stack 1", "Aperture Stack 2", etc.

OPTIONS

–**remove-tags** Remove the matched tag from every asset that carries it, including orphans and already-stacked assets. Only effective with `--apply`. Useful for cleaning up leftover tags after a partial migration — re-run with `--remove-tags` to sweep up tags that earlier runs left behind.

–**apply** Actually create stacks. Without this flag, the command only reports what it would do.

EXAMPLES

Preview what stacks would be created from aperture stack tags:

```
maki stack from-tag "Aperture Stack {}"
```

Create stacks and remove the grouping tags:

```
maki st from-tag "Aperture Stack {}" --remove-tags --apply
```

Create stacks from bracket sequence tags:

```
maki stack from-tag "Bracket {}" --apply
```

SEE ALSO

stack create – manually create a stack from specific assets. stack dissolve – dissolve stacks if the result is unwanted. auto-stack – cluster visually similar assets into stacks. tag – manage asset tags.

maki auto-stack *(Pro)*

NAME

maki-auto-stack – cluster visually similar assets into stacks

SYNOPSIS

```
maki [GLOBAL FLAGS] auto-stack [QUERY] [--threshold <50-99>] [--min-size <N>] [--apply]
```

DESCRIPTION

Discovers natural visual clusters across the catalog using stored SigLIP embeddings and proposes a stack for each cluster. This is the catalog-wide companion to the `similar:` search filter: instead of asking "what looks like this asset?", it asks "which assets naturally belong together?" — burst sequences, bracket series, multiple shots of the same scene.

Candidates are assets that have a stored embedding for the active `[ai] model` (generated via `maki embed` or `maki import --embed`) and are **not already in a stack** — existing stacks are never disturbed. An optional positional QUERY narrows the candidate set (same syntax as `maki search`).

Clustering is greedy connected-components over the similarity graph: each candidate is connected to every other candidate whose embedding similarity meets the threshold, and each connected group of at least `--min-size` assets becomes one proposed stack. Because connectivity is transitive, a long burst chains into a single cluster even when its first and last frames are individually below the threshold.

The pick of each proposed stack is selected deterministically: highest rating first, then earliest capture date, then lowest asset ID.

Without `--apply`, runs in **report-only mode** listing every proposed cluster with its size, pick, members, and min/avg similarity. With `--apply`, creates the stacks (including the `stacks.yaml` export, same as `maki stack create`).

No model inference happens — only stored embeddings are read — so the command works without the AI model files downloaded and is fast even on large catalogs.

Safety: thresholds below 50% are refused (they produce one degenerate giant cluster), and a warning is printed when more than half of all candidates end up in a single cluster — a sign the threshold is too low for your material.

ARGUMENTS

QUERY (optional) A search query that narrows the candidate set (same syntax as `maki search`). Without it, all unstacked embedded assets are considered.

OPTIONS

–**threshold** ***PERCENT*** Similarity threshold in percent, 50–99 (default: **85**). Same scale as the `similar:` filter's minimum similarity. Higher values produce tighter, smaller clusters; lower values merge more aggressively.

–**min-size** ***N*** Minimum cluster size (default: **2**). Clusters with fewer members are not proposed.

–**apply** Actually create the stacks. Without this flag, the command only reports what it would do.

EXAMPLES

Preview clusters across the whole catalog:

```
maki auto-stack
```

Tighter clustering, only proposing groups of 3 or more:

```
maki auto-stack --threshold 92 --min-size 3
```

Cluster one shoot and create the stacks:

```
maki auto-stack 'path:2026-05-17' --apply
```

`--json` outputs the full structure: candidate count, per-cluster members with names and ratings, min/avg similarity, created stack IDs (with `--apply`), and any warnings. `--log` prints one line per cluster to stderr.

SEE ALSO

stack from-tag – convert existing tag-based groupings into stacks. stack dissolve – dissolve stacks if the result is unwanted. embed – generate the embeddings auto-stack clusters on. search filters – the `similar:` filter for per-asset similarity browse.

maki faces detect *(Pro)*

NAME

maki-faces-detect – detect faces in asset images

SYNOPSIS

```
maki [GLOBAL FLAGS] faces detect [--query <Q>] [--asset <id>] [--volume <label>]
                                 [--min-confidence <F>] [--force] [--apply]
```

DESCRIPTION

Detects faces in asset preview images using the YuNet ONNX model. For each detected face, computes a 512-dimensional ArcFace embedding and generates a 150×150 JPEG crop thumbnail.

Without `--apply`, runs in report-only mode showing how many faces would be detected. With `--apply`, stores face records in the catalog and generates crop thumbnails.

Requires at least one scope filter (`--query`, `--asset`, or `--volume`) to prevent accidental full-catalog processing. Models must be downloaded first with `maki faces download`.

OPTIONS

–**query <Q>** Search query to scope which assets are processed.
–**asset <id>** Process a single asset by ID.
–**volume <label>** Process assets on a specific volume.
–**min-confidence <FLOAT>** Minimum detection confidence threshold (0.0–1.0, default 0.5). Faces below this confidence are discarded.
–**apply** Actually store detected faces (default is dry run).
–**force** Re-detect faces even if faces already exist for an asset. Without this flag, assets with existing face records are skipped.

`--json`, `--log`, `--time` for output control.

EXAMPLES

Detect faces in all images (dry run):

```
maki faces detect --query "type:image"
```

Detect and store faces for a single asset:

```
maki faces detect --asset a1b2c3d4 --apply
```

Detect faces on a specific volume with logging:

```
maki faces detect --volume "Photos" --apply --log
```

SEE ALSO

faces cluster – group detected faces into people. faces download – download required models.

maki faces cluster *(Pro)*

NAME

maki-faces-cluster – group similar faces into people

SYNOPSIS

```
maki [GLOBAL FLAGS] faces cluster [--query <Q>] [--asset <id>] [--volume <label>]
                                  [--threshold <F>] [--min-confidence <F>] [--apply]
```

DESCRIPTION

Groups similar face embeddings into unnamed person groups using **agglomerative hierarchical clustering with average linkage** (UPGMA). This is order-independent and produces tighter, better-separated clusters than simpler greedy approaches. Faces already assigned to a person are skipped.

The threshold is the minimum average cosine similarity for two clusters to merge (0.0–1.0, higher = stricter). Default is **0.35**, configurable via `[ai] face_cluster_threshold` in `maki.toml`. The default is tuned for the aligned FP32 ArcFace pipeline where intra-person similarity typically falls in 0.5–0.9 and inter-person similarity is near zero or negative.

The `--min-confidence` flag filters out low-quality face detections (blurry, profile, partial) whose noisy embeddings hurt cluster purity. Default is **0.7**, configurable via `[ai] face_min_confidence`.

Only faces embedded with the **current** recognition model are clustered. If older embeddings from a previous model version exist, the command prints a warning listing how many were skipped — those assets need `maki faces detect --force` to be re-embedded.

Without `--apply`, shows a dry-run report of cluster sizes. With `--apply`, creates person records and assigns faces.

OPTIONS

–**query <Q>** Scope clustering to faces on assets matching this query.
–**asset <id>** Scope clustering to faces on a single asset.
–**volume <label>** Scope clustering to faces on assets on a specific volume.
–**threshold <F>** Similarity threshold for clustering (default 0.35).
–**min-confidence <F>** Skip face detections whose confidence is below this value (default 0.7).
–**apply** Actually create person groups (default is dry run).

`--json`, `--log`, `--time` for output control.

TUNING THE THRESHOLD

The right threshold depends on your dataset. Use `maki faces similarity` to see your embedding distribution, then pick a threshold in the valley between the inter-person and intra-person humps.

Threshold	Behavior
`0.3`	Aggressive merging — bigger clusters, may mix similar-looking people
`0.35` (default)	Balanced — usually a good starting point
`0.45`	Conservative — tighter clusters, more singletons/small groups

Threshold	Behavior
`0.6+`	Very strict — mostly splits same-person photos into separate clusters

Start with a dry run at the default, eyeball the cluster sizes, and adjust.

EXAMPLES

Preview clustering results:

```
maki faces cluster
```

Apply clustering with a stricter threshold:

```
maki faces cluster --threshold 0.45 --apply
```

Cluster only faces from a specific shoot:

```
maki faces cluster --query "path:Capture/2026-03" --apply
```

Use a higher confidence filter to skip profile/blurry faces:

```
maki faces cluster --min-confidence 0.85 --apply
```

SEE ALSO

faces detect – detect faces first. faces similarity – analyze embedding distribution. faces name – name the resulting person groups.

maki faces clean *(Pro)*

NAME

maki-faces-clean – delete unassigned face records

SYNOPSIS

```
maki [GLOBAL FLAGS] faces clean [--apply]
```

DESCRIPTION

Deletes face records that have no person assigned (`person_id IS NULL`). Useful for:

- Cleaning up after experimenting with clustering thresholds.
- Removing stale embeddings after a recognition-model upgrade — old embeddings sit idle, and this command sweeps them up before you re-detect with the new model.
- Dropping orphan faces left behind after `faces delete-person`.

Assigned faces (already part of a named or unnamed person cluster) are untouched.

Without `--apply`, reports how many faces would be deleted. With `--apply`, performs the deletion.

OPTIONS

–**apply** Actually delete the faces (default is dry run).

EXAMPLES

```
maki faces clean                # how many orphans are there?
maki faces clean --apply        # delete them all
```

SEE ALSO

faces cluster – group faces into people. faces delete-person – remove a person (unassigns their faces, which this command can then clean up).

---

maki faces similarity *(Pro)*

NAME

maki-faces-similarity – analyze face embedding distribution

SYNOPSIS

```
maki [GLOBAL FLAGS] faces similarity [--query <Q>] [--asset <id>] [--volume <label>]
                                     [--min-confidence <F>] [--top <N>] [--all]
```

DESCRIPTION

Diagnostic command. Computes all pairwise cosine similarities between faces in the given scope and prints:

- **Percentile statistics**: min, p10, p25, median, mean, p75, p90, p95, p99, max.
- **Histogram**: 10 buckets spanning the min–max range, each showing how many pairs fall into that similarity range.

Interpreting the histogram:

- **Bimodal (two clear humps)** — healthy embeddings. The lower hump is inter-person pairs (different people), the upper hump is intra-person pairs (same person). A good clustering threshold sits in the valley between them.
- **Single hump or narrow band** — embeddings are not discriminating well. Check that detection confidence is high enough, face crops are large enough, and the recognition model matches expectations.

With `--top N`, also prints each face's top-N nearest neighbors (face_id/asset_id/similarity) so you can spot-check whether a face's closest matches are actually the same person. This is invaluable for understanding clustering results.

OPTIONS

–query <Q>, –asset <id>, –volume <label> Scope the analysis.
–min-confidence <F> Ignore face detections with confidence below this value (default 0.0 — include all).
–top <N> For each face, list its top-N nearest neighbors with similarity scores (default 0 — summary only).
–all Include already-assigned faces (default: only unassigned).

EXAMPLES

Summary of unassigned refugee portraits:

```
maki faces similarity --query 'tag:"person|refugee"' --min-confidence 0.7
```

Per-face nearest neighbors to spot-check clustering:

```
maki faces similarity --query 'tag:"person|refugee"' --top 5
```

Include already-named people to see intra-cluster consistency:

```
maki faces similarity --query 'tag:"person|refugee"' --all
```

SEE ALSO

faces cluster – group faces using the threshold you picked. faces dump-aligned – save aligned crops for visual inspection.

---

maki faces dump-aligned *(Pro)*

NAME

maki-faces-dump-aligned – save aligned 112×112 face crops for visual debugging

SYNOPSIS

```
maki [GLOBAL FLAGS] faces dump-aligned [--query <Q>] [--asset <id>]
                                       [--output <DIR>] [--limit <N>]
```

DESCRIPTION

Detects faces on the fly and saves the resulting 112×112 ArcFace-aligned crops to disk. Used to verify that the alignment pipeline (5-point landmark similarity transform) is producing the crops the recognition model expects.

Healthy output: all crops at roughly the same face scale, eyes approximately level, face filling most of the 112×112 frame. If crops look mirrored, rotated, squashed, or offset, alignment math is wrong (or landmarks from the detector have an unexpected order).

Uses the same image-resolution path as `faces detect` — smart preview → regular preview → original file on an online volume.

OPTIONS

–query <Q>, –asset <id> Scope which assets to process.
–output <DIR> Output directory (default: `./aligned-faces`).
–limit <N> Maximum number of crops to save (default 20, 0 = unlimited).

EXAMPLES

```
maki faces dump-aligned --query 'tag:"person|refugee"' --limit 30
```

SEE ALSO

faces similarity – diagnose embedding quality numerically.

maki faces people *(Pro)*

NAME

maki-faces-people – list all people

SYNOPSIS

```
maki [GLOBAL FLAGS] faces people
```

DESCRIPTION

Lists all people in the catalog with their names (if assigned) and face counts.

EXAMPLES

```
maki faces people
maki faces people --json
```

SEE ALSO

faces name – name a person.

maki faces name *(Pro)*

NAME

maki-faces-name – name a person

SYNOPSIS

```
maki [GLOBAL FLAGS] faces name <PERSON_ID> <NAME>
```

DESCRIPTION

Assigns a human-readable name to a person. Person IDs are shown by `maki faces people`.

ARGUMENTS

PERSON_ID (required) The person's UUID.
NAME (required) The name to assign.

EXAMPLES

```
maki faces name 550e8400-... "Alice"
```

maki faces merge *(Pro)*

NAME

maki-faces-merge – merge two people

SYNOPSIS

```
maki [GLOBAL FLAGS] faces merge <TARGET_ID> <SOURCE_ID>
```

DESCRIPTION

Moves all faces from the source person to the target person, then deletes the source person. Useful for combining duplicate person groups after clustering.

ARGUMENTS

TARGET_ID (required) The person to keep.
SOURCE_ID (required) The person to merge into the target (deleted after merge).

EXAMPLES

```
maki faces merge 550e8400-... 661f9511-...
```

maki faces delete-person *(Pro)*

NAME

maki-faces-delete-person – delete a person

SYNOPSIS

```
maki [GLOBAL FLAGS] faces delete-person <PERSON_ID>
```

DESCRIPTION

Deletes a person record. All faces assigned to this person become unassigned (they are not deleted).

ARGUMENTS

PERSON_ID (required) The person's UUID.

EXAMPLES

```
maki faces delete-person 550e8400-...
```

---

maki faces unassign *(Pro)*

NAME

maki-faces-unassign – remove a face from its person

SYNOPSIS

```
maki [GLOBAL FLAGS] faces unassign <FACE_ID>
```

DESCRIPTION

Removes the person assignment from a single face. The face record is preserved; only the person link is cleared.

ARGUMENTS

FACE_ID (required) The face's UUID.

EXAMPLES

```
maki faces unassign a1b2c3d4-...
```

---

maki faces export *(Pro)*

NAME

maki-faces-export – export faces and people to YAML and binary files

SYNOPSIS

```
maki [GLOBAL FLAGS] faces export
```

DESCRIPTION

Exports all face and people data from the SQLite catalog to file-based storage:

- `faces.yaml` — face records (bounding boxes, confidence, person assignments)
- `people.yaml` — people records (names, representative faces)
- `embeddings/arcface/<prefix>/<face_id>.bin` — ArcFace face recognition embeddings as raw binary files

This is a one-time migration command for catalogs that have existing face data in SQLite but no corresponding YAML/binary files (i.e., data created before v2.2.1). After running this command, `rebuild-catalog` will be able to restore face and people data.

Going forward, all face/people write operations automatically persist to both SQLite and files.

EXAMPLES

Export all face data:

```
maki faces export
```

Export with JSON output:

```
maki faces export --json
```

SEE ALSO

faces detect – detect faces in images. rebuild-catalog – rebuilds catalog from files.

---

maki faces download *(Pro)*

NAME

maki-faces-download – download face detection models

SYNOPSIS

```
maki [GLOBAL FLAGS] faces download
```

DESCRIPTION

Downloads the YuNet face detection model (~230 KB) and ArcFace ResNet-100 FP32 face recognition model (~261 MB) from HuggingFace. Models are cached in `~/.maki/models/faces/` by default, configurable via `[ai] model_dir` in `maki.toml`.

The recognition model has normalization nodes baked into its ONNX graph; the pipeline passes raw pixel values and lets the model apply its own mean/std. Faces are aligned to a canonical 112×112 template using a 5-point similarity transform before embedding, matching how ArcFace was trained.

EXAMPLES

```
maki faces download
```

---

maki faces status *(Pro)*

NAME

maki-faces-status – show face detection model status

SYNOPSIS

```
maki [GLOBAL FLAGS] faces status
```

DESCRIPTION

Shows the download status of face models and the breakdown of stored face embeddings by recognition model. If the current recognition model differs from the one used for some stored embeddings (e.g., after upgrading), the command prints how many faces are "stale" and prompts you to re-run `maki faces detect --force` to re-embed them. Stale embeddings are silently skipped by clustering.

EXAMPLES

```
maki faces status
```

Typical output after an upgrade:

```
Face model directory: /Users/jane/.maki/models/faces
Models downloaded: yes
Current recognition model: arcface-resnet100-fp32-aligned-v2
Total faces detected: 387
Total people: 4
Faces by recognition model:
  arcface-resnet100-fp32-aligned-v2: 349 (current)
  arcface-resnet100-int8: 38
  38 face(s) use a different recognition model and will be
  ignored by clustering. Re-run `maki faces detect --force` to update.
```

SEE ALSO

faces download – download face models. faces detect – detect faces in images.

Previous: Ingest Commands – `import`, `tag`, `edit`, `group`, `auto-group`. Next: Retrieve Commands – `search`, `show`, `duplicates`, `stats`, `serve`.

Retrieve Commands

Commands for finding assets, inspecting details, exporting files, and browsing the catalog. Search is the primary discovery mechanism; all other retrieval commands build on it.

Command	Description
search	Find assets by text, filters, and metadata
show	Display full asset details
preview	Open asset preview in OS default image viewer
export	Copy matching files to a directory or ZIP
contact-sheet	Generate PDF contact sheets
duplicates	Find files with multiple locations
stats	Catalog statistics breakdown (types, formats, tags, verification)
status	Catalog health overview with actionable next steps
backup-status	Check backup coverage and find at-risk assets
doc	Open documentation in the browser
licenses	Show MAKI license and third-party crate licenses
serve	Start the web UI
shell	Interactive command shell

maki search

NAME

maki-search – search for assets using filters and free-text keywords

SYNOPSIS

```
maki [GLOBAL FLAGS] search <QUERY> [--format <FMT>] [-q]
maki [GLOBAL FLAGS] search --image <FILE> [QUERY] [--limit <N>] [--format <FMT>] [-q]
```

DESCRIPTION

Searches the catalog for assets matching the given query. The query string supports both free-text keywords (matched against filenames and metadata) and structured filter prefixes.

Filter syntax: `prefix:value`. Values containing spaces must be double-quoted: `tag:"Fools Theater"`.

Available filters:

Filter	Description	Examples
`type:`	Asset type	`type:image`, `type:video`, `type:audio`
`tag:`	Tag name (hierarchical: matches descendants)	`tag:landscape`, `tag:"golden hour"`, `tag:animals|birds`
`format:`	File format	`format:jpg`, `format:nef`, `format:mp4`
`rating:`	Star rating (exact, range, OR)	`rating:5`, `rating:3+`, `rating:3-5`, `rating:2,4`
`label:`	Color label (`none` = unlabeled, `-label:none` = any label)	`label:Red`, `label:Green`, `label:none`
`camera:`	Camera model (partial match)	`camera:fuji`, `camera:"Canon EOS R5"`
`lens:`	Lens model (partial match)	`lens:56mm`, `lens:"RF 50mm f/1.2"`
`description:` / `desc:`	Description text (partial match)	`description:sunset`, `desc:"colorful flowers"`
`iso:`	ISO value or range	`iso:100`, `iso:100-800`
`focal:`	Focal length or range (mm)	`focal:50`, `focal:35-70`
`f:`	Aperture or range	`f:1.4`, `f:1.4-2.8`
`width:`	Width in pixels (exact, minimum, range)	`width:3840`, `width:4000+`, `width:3000-5000`
`height:`	Height in pixels (exact, minimum, range)	`height:2160`, `height:2000+`, `height:2000-3000`
`meta:`	Source metadata key=value	`meta:Copyright=2026`
`path:`	File location path prefix	`path:Capture/2026-02`, `path:/Volumes/Photos/2026`
`collection:`	Collection membership	`collection:Favorites`, `collection:"My Picks"`

Filter	Description	Examples
`date:`	Creation date (prefix match)	`date:2026-02-25`, `date:2026-02`, `date:2026`
`dateFrom:`	Creation date lower bound (inclusive)	`dateFrom:2026-01-01`
`dateUntil:`	Creation date upper bound (inclusive)	`dateUntil:2026-12-31`
`copies:`	Number of distinct volumes holding the asset's files	`copies:1`, `copies:2+`, `copies:1-2`
`id:`	Asset ID (prefix match)	`id:72a0bb4b`
`variants:`	Variant count (exact, minimum, range)	`variants:2`, `variants:2+`, `variants:2-4`
`tagcount:`	Number of leaf tags (`0` = untagged)	`tagcount:0`, `tagcount:5+`, `tagcount:2-4`
`scattered:`	Files spread over N distinct session roots	`scattered:2+`, `scattered:1`
`duration:`	Audio/video duration in seconds	`duration:60`, `duration:30+`, `duration:10-120`
`codec:`	Video codec (partial match)	`codec:h264`, `codec:hevc`, `codec:prores`
`key:`	Musical key of analyzed audio (exact, case-insensitive)	`key:Am`, `key:F#m`
`bpm:`	Tempo of analyzed audio (exact, minimum, range)	`bpm:120`, `bpm:160+`, `bpm:100-140`
`orphan:true`	Assets with zero file locations	`orphan:true`
`orphan:false`	Assets with at least one file location	`orphan:false`
`missing:true`	Assets with files missing on disk	`missing:true`
`stale:`	Not verified in N days (or never); `never` = never verified	`stale:30`, `stale:90`, `stale:never`
`volume:`	Assets on a specific volume (`none` = no online copy, `-volume:none` = on an online volume)	`volume:Archive`, `volume:none`
`stacked:true`	Assets in a stack	`stacked:true`
`stacked:false`	Assets not in any stack	`stacked:false`
`geo:`	GPS geolocation	`geo:any`, `geo:none`, `geo:52.5,13.4,10`
`faces:`	Face count (counts populated by `faces detect`, Pro)	`faces:any`, `faces:none`, `faces:3`, `faces:2+`
`person:`	Named person *(Pro)*	`person:Alice`, `person:"John Smith"`
`similar:`	Visual similarity *(Pro)*	`similar:72a0bb4b`, `similar:72a0bb4b:50`
`min_sim:`	Minimum similarity threshold *(Pro)*	`min_sim:90`
`text:`	Semantic text-to-image search *(Pro)*	`text:sunset`, `text:"woman reading"`

Filter	Description	Examples
`embed:`	Embedding status (embeddings created by `embed`, Pro)	`embed:any`, `embed:none`

Filters can be freely combined. Free-text tokens that do not match a filter prefix are joined as a text search against filenames and metadata.

Path normalization: The `path:` filter automatically normalizes paths. `~` expands to `$HOME`, `./` and `../` resolve relative to the current working directory, and absolute paths matching a registered volume's mount point are stripped to volume-relative form with the volume filter implicitly applied (e.g., `path:/Volumes/Photos/Capture/2026` becomes `path:Capture/2026` scoped to the Photos volume).

Output formats: The `--format` flag controls output. Presets: `ids` (one UUID per line), `short` (default: abbreviated ID, name, type, format, date), `full` (adds tags, description, rating, label), `json` (JSON array). Custom templates use `{placeholder}` substitution with escape sequences (`\t`, `\n`).

Available placeholders: `{id}`, `{short_id}` (first 8 characters), `{name}`, `{filename}`, `{type}`, `{format}`, `{date}`, `{tags}`, `{description}`, `{label}`, `{hash}` (best-variant content hash), `{similarity}` (rounded percentage, empty unless the query is a `similar:`/`text:`/`--image` search), and `{exact}` (the word `exact` for a byte-identical `--image` match, otherwise empty). Unknown placeholders are left in place verbatim.

The result count header (e.g., "Found 42 assets") is suppressed when an explicit `--format` is given.

ARGUMENTS

QUERY (required unless `--image` is given) Search query string with optional filter prefixes and free-text keywords.

OPTIONS

–**format <FMT>** Output format preset or custom template.
-q / –**quiet** Shorthand for `--format=ids`. Prints one asset ID per line with no header, ideal for piping.
–**image <FILE>** Find assets visually similar to a local image file that is not in the catalog (e.g. locate the original behind a preview). A byte-identical copy is matched by content hash first and listed as `100% exact`; otherwise the file is encoded with the SigLIP model and ranked like `similar:`. The flag exists in every build; exact-hash matching works everywhere, while the similarity ranking needs a Pro (AI) build. The optional QUERY narrows the result (AND). See `--image` in the filter reference.
–**limit <N>** Result-set size for `--image` (default 40, counting the exact match).

`--json` (global flag) is equivalent to `--format json`.

EXAMPLES

Search by tag and minimum rating:

```
maki search "tag:landscape rating:4+"
```

Find all videos:

```
maki search "type:video"
```

Search with camera and aperture filters:

```
maki search 'camera:"Canon EOS R5" f:1.4-2.8'
```

Get IDs for scripting:

```
maki search -q "tag:travel label:Green"
```

Custom format template:

```
maki search "rating:5" --format '{id}\t{name}\t{label}'
```

Locate the original behind a preview you were sent (Pro):

```
maki search --image ~/Downloads/IMG_4711_preview.jpg
maki search --image scan.tif "min_sim:90" --json
```

Search within a path and pipe to a collection:

```
maki search -q "path:Capture/2026-02-22 rating:4+" | xargs maki col add "Feb Selects"
```

Find assets with files missing on disk:

```
maki search "missing:true"
```

Find assets with no backup (only one copy):

```
maki search "copies:1"
```

Find highly-rated assets with at least two copies:

```
maki search "copies:2+ rating:4+"
```

Find orphaned assets (no file locations):

```
maki search "orphan:true"
```

Find visually similar assets *(Pro + embeddings)*:

```
maki search "similar:72a0bb4b"
maki search "similar:72a0bb4b:50"
maki search "similar:72a0bb4b rating:4+ tag:landscape"
```

SEE ALSO

show – display full details for a specific asset. saved-search – save and re-run searches. CLI Conventions – output conventions, scripting patterns.

---

maki show

NAME

maki-show – display full details for an asset

SYNOPSIS

```
maki [GLOBAL FLAGS] show <ASSET_ID> [--locations]
```

DESCRIPTION

Displays comprehensive details for a single asset, including:

- Asset metadata: ID, name, description, type, tags, rating (as stars), color label, creation date.
- Variants: content hash, role (original/export/processed/sidecar), format, file size, original filename, file locations (volume + path), source metadata (camera, lens, ISO, etc.), and preview status.
- Recipes: content hash, format (XMP, COS, etc.), original filename, and file locations.

Asset IDs support unique prefix matching (see CLI Conventions).

The display logic for previews prefers Export > Processed > Original variant previews (skipping Sidecar). Within the same role, standard image formats are preferred over RAW, with file size as a tiebreaker.

ARGUMENTS

ASSET_ID (required) The asset ID or a unique prefix of it.

OPTIONS

–**locations** Print only the asset's file locations, one per line as `volume:path` — handy for scripting (`maki show abc123 --locations | xargs ...`).

`--json` outputs an `AssetDetails` object with full asset, variant, and recipe information.

EXAMPLES

Show an asset by full ID:

```
maki show a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

Show an asset by prefix:

```
maki show a1b2c
```

Show as JSON and extract variant filenames:

```
maki show a1b2c --json | jq '.variants[].original_filename'
```

Show as JSON and list file locations:

```
maki show a1b2c --json | jq '.variants[].locations[]'
```

SEE ALSO

search – find assets to inspect. edit – modify the fields shown here. tag – add or remove tags.

---

maki preview

NAME

`maki preview` – open an asset's preview in the OS default image viewer

SYNOPSIS

```
maki [GLOBAL FLAGS] preview <ASSET_ID>
```

DESCRIPTION

Resolves the asset's best preview file (smart preview if available, otherwise the regular preview) and opens it with the operating system's default image viewer (`open` on macOS, `xdg-open` on Linux, `start` on Windows). Useful for quickly inspecting an asset from the CLI without launching a full image browser.

Also available as a shell built-in: `preview <id>`, `preview $var`.

OPTIONS

- `<ASSET_ID>` – Asset ID or prefix (required)

EXAMPLES

Open a preview:

```
maki preview a1b2c3d4
```

In the shell with variable expansion:

```
photos> $picks = search "rating:5 date:2024"
photos [picks=12]> preview $picks
```

SEE ALSO

show – display full asset metadata. search – find assets to preview.

---

maki export

NAME

maki-export – copy files matching a search query to a directory or ZIP archive

SYNOPSIS

```
maki [GLOBAL FLAGS] export <QUERY> <TARGET> [OPTIONS]
```

DESCRIPTION

Exports files from the catalog to a target directory or ZIP archive. Searches for assets matching the query, resolves their file locations on online volumes, and copies (or symlinks) files to the target. With `--zip`, writes a single ZIP archive instead of individual files.

By default, only the **best variant** per asset is exported (Export > Processed > Original, image formats preferred, file size tiebreaker). Use `--all-variants` to export every variant.

Layout modes:

- **`flat`** (default): All files placed in the target root. Filename collisions (different content, same filename) are resolved by appending `_<8-char-hash>` before the extension. Files with identical content hashes reuse the same target path.
- **`mirror`**: Preserves the source volume-relative directory structure. When files span multiple volumes, each volume's files are placed under a `<volume-label>/` prefix to avoid path collisions.

Files are copied with SHA-256 integrity verification. Existing files at the target path are skipped if their content hash matches (use `--overwrite` to force re-copy).

ARGUMENTS

QUERY (required) Search query string (same syntax as `maki search`).

TARGET (required) Target directory path (or ZIP file path with `--zip`). Created automatically if it does not exist (except in `--dry-run` mode).

OPTIONS

–**layout <MODE>** File layout mode: `flat` (default) or `mirror`.

–**symlink** Create symbolic links instead of copying files. On Unix, uses `std::os::unix::fs::symlink`; on Windows, uses `std::os::windows::fs::symlink_file`.

–**all-variants** Export every variant of each matching asset. Default is best variant only.

–**include-sidecars** Also copy recipe/sidecar files (`.xmp`, `.cos`, `.cot`, `.cop`, `.pp3`, `.dop`, `.on1`) alongside the exported variants.

–**previews** Export the assets' preview images instead of the originals. Previews live in the catalog directory, so this works even when the originals' volumes are offline — handy for handing out web/social-media-sized copies without mounting archive drives. Exported files keep the original filename with the extension swapped to the preview format (`IMG_0042.NEF` → `IMG_0042.jpg`). Assets without a generated preview are reported as skipped; run `maki generate-previews` first for full coverage. Since a preview's bytes differ from the original's, no hash verification is performed for these copies, and `--all-variants` has no useful effect (previews are keyed to the best variant).

–**smart-previews** Like `--previews`, but exports the high-resolution smart previews from `<catalog>/smart-previews/`. Assets without a smart preview are skipped (no silent fallback to the standard preview); run `maki generate-previews --smart` first for full coverage. Mutually exclusive with `--previews`.

–**dry-run** Report the export plan without writing any files or creating directories.

–**overwrite** Re-copy files even if the target already contains a file with a matching content hash. Default behavior skips matching files.

–zip Write a ZIP archive to the target path instead of copying files to a directory. The archive uses stored (uncompressed) entries since media files are already compressed. The `.zip` extension is appended automatically if not present. Cannot be combined with `--symlink`. Layout and sidecar options work the same as directory export.

`--json` outputs an `ExportResult` object with fields: `dry_run`, `assets_matched`, `files_exported`, `files_skipped`, `sidecars_exported`, `total_bytes`, `errors`.

EXAMPLES

Export best-of picks to a delivery folder:

```
maki export "rating:5 tag:portfolio" /tmp/delivery/
```

Export with directory structure preserved:

```
maki export "collection:Print" /Volumes/USB/export --layout mirror
```

Include sidecars for another workstation:

```
maki export "tag:client" /tmp/handoff/ --include-sidecars
```

Hand out social-media-sized copies without mounting the archive drives:

```
maki export "tag:event|concert rating:4+" ~/for-the-band.zip --zip --smart-previews
```

Create symlinks instead of copies:

```
maki export "type:image rating:4+" ~/links/ --symlink
```

Export all variants (RAW + processed):

```
maki export "tag:portfolio" /tmp/all/ --all-variants
```

Export as a ZIP archive:

```
maki export "tag:client" ~/Desktop/delivery --zip
```

Dry run to see what would be exported:

```
maki export "collection:Best" /tmp/test/ --dry-run
```

JSON output for scripting:

```
maki --json export "rating:5" /tmp/out/ | jq '.files_exported'
```

SEE ALSO

search – find assets to export. contact-sheet – generate a PDF contact sheet from search results. relocate – move/copy asset files between volumes with catalog updates. CLI Conventions – global flags, scripting patterns.

maki contact-sheet

NAME

maki-contact-sheet – generate a PDF contact sheet from search results

SYNOPSIS

```
maki [GLOBAL FLAGS] contact-sheet <QUERY> <OUTPUT> [OPTIONS]
```

DESCRIPTION

Generates a printable PDF contact sheet with thumbnail grids from assets matching a search query. Pages are composed as images at 300 DPI and wrapped in PDF. Smart previews (2560px) are preferred with automatic fallback to regular previews (800px).

Layout presets control the grid density:

- **dense** (6x8): Maximum thumbnails per page.
- **standard** (4x5): Balanced view (default).
- **large** (3x3): Detailed thumbnails.

Color labels can be rendered as colored borders around cells, small dots next to filenames, or hidden entirely.

ARGUMENTS

QUERY (required) Search query (same syntax as `maki search`).
OUTPUT (required) Output PDF file path.

OPTIONS

–**layout <PRESET>** Layout preset: `dense`, `standard`, `large` (default: `standard`).
–**columns <N>** Number of columns (overrides layout preset).
–**rows <N>** Number of rows per page (overrides layout preset).
–**paper <SIZE>** Paper size: `a4`, `letter`, `a3` (default: `a4`).
–**landscape** Use landscape orientation.
–**title <TEXT>** Title printed in page headers.
–**fields <FIELDS>** Comma-separated metadata fields: `filename`, `date`, `rating`, `label`, `format`, `size`, `dimensions`.
–**sort <ORDER>** Sort order: `date`, `name`, `rating`, `filename`.
–**no-smart** Use regular previews instead of smart previews.
–**group-by <FIELD>** Group by field with section headers: `date`, `volume`, `collection`, `label`.
–**margin <MM>** Page margin in millimeters.
–**label-style <STYLE>** Color label display: `border`, `dot`, `none` (default: `border`).
–**quality <N>** JPEG quality for page images, 1-100.
–**copyright <TEXT>** Copyright text in center of page footer.
–**dry-run** Report page/asset count without generating.

EXAMPLES

Basic contact sheet of all 5-star images:

```
maki contact-sheet "rating:5" stars.pdf
```

Dense layout on A3 landscape with title:

```
maki contact-sheet "tag:landscape" landscapes.pdf --layout dense --paper a3 --landscape --title "Landscapes 2026"
```

Group by date with copyright:

```
maki contact-sheet "" all.pdf --group-by date --copyright "© 2026 Thomas Herrmann"
```

Dry run to check page count:

```
maki contact-sheet "format:nef" raw.pdf --dry-run
```

SEE ALSO

search – find assets matching a query. export – copy files to a directory. generate-previews – generate or upgrade previews.

---

maki duplicates

NAME

maki-duplicates – find files with identical content at multiple locations

SYNOPSIS

```
maki [GLOBAL FLAGS] duplicates [--format <FMT>] [--same-volume] [--cross-volume] [--volume <LABEL>] [--filter-format <FMT>] [--path <PREFIX>]
```

DESCRIPTION

Finds variants whose content hash appears at more than one file location. This detects files that exist on multiple volumes or at multiple paths, helping identify redundant copies or verify backup coverage.

Each result shows the content hash, filename, volume count, and all locations where the identical file exists. The `short` and `full` formats annotate locations with the volume's purpose (if set) and flag same-volume duplicate groups. The `full` format additionally shows the last verification timestamp for each location.

Duplicate modes:

- **Default** (no flag): All variants with 2+ file locations, regardless of volume layout.
- **`--same-volume`**: Only variants with 2+ locations on the **same** volume. These are likely unwanted copies (e.g., accidentally imported twice from different paths on the same drive).
- **`--cross-volume`**: Only variants with locations on 2+ **different** volumes. These represent intentional backups or copies across drives.

The `--volume`, `--filter-format`, and `--path` flags narrow results via SQL filtering. All filters compose with any mode.

ARGUMENTS

None.

OPTIONS

–**format <FMT>** Output format. Same presets as `maki search`: `ids`, `short` (default), `full`, `json`. Custom templates support all search placeholders plus `{locations}` and `{volumes}` (distinct volume count). Location strings include the volume purpose in brackets (e.g., `Photos[working]:Capture/photo.jpg`).
–**same-volume** Show only same-volume duplicates. Mutually exclusive with `--cross-volume`.
–**cross-volume** Show only cross-volume copies. Mutually exclusive with `--same-volume`.
–**volume <LABEL>** Filter results to entries involving this volume. Combines with any mode.
–**filter-format <FORMAT>** Filter to entries matching this file format (e.g. `nef`, `jpg`).
–**path <PREFIX>** Filter to entries with a location under this path prefix.

`--json` outputs an array of `DuplicateEntry` objects (includes `volume_count` and `same_volume_groups` fields).

EXAMPLES

Find all duplicates:

```
maki duplicates
```

Find likely unwanted same-volume duplicates:

```
maki duplicates --same-volume
```

Show cross-volume copies (backup verification):

```
maki duplicates --cross-volume
```

Filter to duplicates involving a specific volume:

```
maki duplicates --volume "Backup Drive"
```

Show full details with verification timestamps:

```
maki duplicates --format full
```

List duplicates as JSON:

```
maki duplicates --json | jq '.[].locations'
```

Cross-volume copies as JSON for a specific volume:

```
maki duplicates --cross-volume --volume Photos --json
```

Custom format showing hash and locations:

```
maki duplicates --format '{hash}\t{filename}\t{volumes} volumes\t{locations}'
```

SEE ALSO

verify – verify file integrity on disk. cleanup – remove stale location records. search – use `copies:` filter for location-count-based queries.

---

maki stats

NAME

maki-stats – show catalog statistics

SYNOPSIS

```
maki [GLOBAL FLAGS] stats [OPTIONS]
```

DESCRIPTION

Displays summary statistics about the catalog. Without any section flags, shows a compact overview: total assets, variants, recipes, volumes, and total file size.

Additional sections can be enabled with flags to show breakdowns by type, format, volume, tag usage, and verification health. `--all` enables all sections.

ARGUMENTS

None.

OPTIONS

–**types** Show asset type breakdown (image, video, audio, etc.) and format distribution.
–**volumes** Show per-volume details: asset count, total size, directory count, and verification status.
–**tags** Show tag usage frequencies (most-used tags first).
–**verified** Show verification health: how many files have been verified, when, and how many are overdue.
–**all** Enable all sections (equivalent to `--types --volumes --tags --verified`).
–**limit <N>** Maximum number of entries for top-N lists (default: 20). Applies to format breakdown, tag list, etc.

`--json` outputs a structured JSON object with all requested sections.

EXAMPLES

Quick overview:

```
maki stats
```

Full statistics:

```
maki stats --all
```

Show only tag frequencies:

```
maki stats --tags --limit 50
```

Show volume details as JSON:

```
maki stats --volumes --json | jq '.volumes[] | {label, assets, size}'
```

Show verification health:

```
maki stats --verified
```

SEE ALSO

search – find specific assets matching criteria. verify – run verification checks. volume list – list volumes with online/offline status.

maki status

NAME

maki-status – catalog health overview with actionable next steps

SYNOPSIS

```
maki [GLOBAL FLAGS] status [--min-copies N]
```

DESCRIPTION

Read-only survey that aggregates the catalog signals you'd otherwise have to check by running `cleanup --dry-run`, `backup-status`, and `volume list` separately. Designed as a one-shot "what needs attention?" report — every actionable item ends with a `→ command` suggestion so you don't have to consult docs to know the next step.

Sections:

- **Catalog** — schema version (with a migrate hint if stale), asset/variant/recipe/file-location counts, total size, and online/offline volume split.
- **Cleanup** — locationless variants, orphaned assets, and orphaned derived files on disk (previews, smart previews, embeddings, face crops). Each non-zero count points at `maki cleanup --apply`. If everything's clean, prints `✓ no cleanup needed`.
- **Pending work** — every actionable backlog the catalog can detect, each line ending in a `→ <command>` hint:
  - **Pending XMP writebacks**, split by online/offline target volume (hint: `→ maki writeback` regardless of `[writeback] enabled`, since manual flush always works). When the catalog has more than one registered volume, a per-volume breakdown follows the summary line so you can see exactly which drive is holding the queue:
    ```
    ✗ 103176 pending XMP writeback(s) on offline volume(s) → mount the volumes, then `maki writeback`
        └ 92596 on MediaBackup-2 (offline)
        └ 10045 on Media (offline)
        └   535 on MediaPortable (offline)
    ```
    The breakdown is sorted most-pending first. Single-volume catalogs skip it (the top-level line already names the only volume).
  - **Missing previews** — assets whose `best_variant_hash` has no preview file under `<catalog_root>/previews/` (hint: `→ maki generate-previews`).
  - **Missing smart previews** — same shape for `<catalog_root>/smart-previews/` (hint: `→ maki generate-previews --smart`). Only reported when the smart-previews directory exists; users who haven't opted into smart previews don't see this line.
  - **Assets without an embedding** (AI builds) and **assets unscanned for faces** (AI builds).
- **Backup coverage** — how many assets have fewer than `--min-copies` distinct volume copies. Points at `maki backup-status --at-risk` to drill down.
- **Volumes** — registered volumes sorted online-first with per-volume asset count, size, and purpose tag. ● = online, ○ = offline.

The orphan-on-disk counts come from the same dry-run scan `maki cleanup` runs without `--apply`; on a large catalog this is the most expensive part of `status` (typically a few seconds for tens of thousands of files). Status is read-only — it never modifies the catalog, sidecars, or filesystem.

OPTIONS

--min-copies N Minimum distinct volume copies for the **Backup coverage** section to consider an asset adequately backed up. Default `2` (matching `maki backup-status`).

EXAMPLES

Quick health check:

```
maki status
```

Treat 3 copies as the floor for backup adequacy:

```
maki status --min-copies 3
```

Pipe the structured report into a script:

```
maki status --json | jq '.cleanup'
```

EXIT STATUS

Always returns 0 on success. The presence of action items (✗ markers) does not change the exit code — `status` is informational, not a check.

SEE ALSO

cleanup – act on the cleanup hints surfaced here. backup-status – detailed backup coverage drill-down. stats – in-depth statistical breakdown (asset types, formats, tags, verification). writeback – act on pending writeback hints.

---

maki backup-status

NAME

maki-backup-status – check backup coverage and find under-backed-up assets

SYNOPSIS

```
maki [GLOBAL FLAGS] backup-status [QUERY] [OPTIONS]
```

DESCRIPTION

Answers the question: "Are my important assets safely backed up?"

In **overview mode** (default), displays aggregate statistics about backup coverage:

- **Totals**: asset count, variant count, file location count.
- **Coverage by volume purpose**: how many assets exist on volumes of each purpose (Working, Archive, Backup, Cloud), with percentages.
- **Volume distribution**: histogram of assets by number of distinct volumes they exist on (0, 1, 2, 3+), with "AT RISK" markers for assets below the threshold. Multiple variants or locations on the same volume count as one — what matters for backup safety is how many distinct volumes hold the asset.
- **At-risk summary**: count of assets with fewer than `--min-copies` locations, with hints for listing them.
- **Volume gaps**: per-volume count of missing assets (assets in scope but not on that volume).

In **at-risk listing mode** (`--at-risk`, `-q`, or `--format`), outputs a list of under-backed-up assets using the same output formats as `maki search`. When `--volume` is specified, lists assets missing from that specific volume instead of those with fewer than `--min-copies` locations overall.

An optional positional `QUERY` argument scopes the analysis to matching assets (same syntax as `maki search`).

ARGUMENTS

QUERY (optional) Search query to scope the asset universe. Same syntax as `maki search`. When omitted, all catalog assets are analyzed.

OPTIONS

–**at-risk** Switch to listing mode. Output under-backed-up assets instead of the overview.

–**min-copies <N>** Threshold for "adequately backed up" (default: 2). Assets on fewer than N distinct volumes are considered at-risk.

–**volume <LABEL>** In overview mode, adds a detailed volume coverage section for this volume. In at-risk listing mode, lists assets missing from this specific volume.

–**format <FMT>** Output format for at-risk listings. Same presets as `maki search`: `ids`, `short`, `full`, `json`, or a custom template.

-q / –**quiet** Shorthand for `--format=ids`. Prints one asset ID per line, ideal for piping to other commands.

`--json` (global flag) outputs a structured `BackupStatusResult` object in overview mode, or a JSON array of `SearchRow` objects in at-risk listing mode.

EXAMPLES

Quick overview of backup coverage:

```
maki backup-status
```

Scope to highly-rated images:

```
maki backup-status "rating:3+ type:image"
```

Require 3 copies and check coverage:

```
maki backup-status --min-copies 3
```

List at-risk asset IDs for scripting:

```
maki backup-status --at-risk -q
```

Find assets missing from a specific volume:

```
maki backup-status --volume "Master Media" --at-risk -q
```

Pipe at-risk assets to relocate:

```
maki backup-status --volume "Master Media" --at-risk -q "rating:3+" \
  | xargs -I{} maki relocate {} "Master Media"
```

Add at-risk assets to a collection for review:

```
maki backup-status --at-risk -q | xargs maki collection add "Needs Backup"
```

JSON output for scripting:

```
maki --json backup-status | jq '.at_risk_count'
```

SEE ALSO

search – use `copies:` filter for location-count-based queries. duplicates – find duplicate files across volumes. stats – general catalog statistics. verify – verify file integrity.

---

maki doc

NAME

maki-doc – open documentation in the browser

SYNOPSIS

```
maki [GLOBAL FLAGS] doc [DOCUMENT]
```

DESCRIPTION

Opens the specified MAKI documentation PDF in the default browser. The PDFs are served from the latest GitHub release — no local files needed, always up to date.

ARGUMENTS

DOCUMENT (optional, default: `manual`) Which document to open. Accepted values:

Value	Aliases	Document
`manual`	`man`, `guide`	User Manual
`cheatsheet`	`cheat`, `cs`	Cheat Sheet
`filters`	`search`, `filter`, `sf`	Search Filter Reference
`tagging`	`tags`, `tag`	Tagging Quick Guide

EXAMPLES

```
maki doc                # open user manual
maki doc cheatsheet     # open cheat sheet
maki doc filters        # open search filter reference
```

maki licenses

NAME

maki-licenses – show MAKI license and third-party crate licenses

SYNOPSIS

```
maki [GLOBAL FLAGS] licenses [--summary]
```

DESCRIPTION

Prints a summary of MAKI's own license (Apache-2.0), the third-party Rust crates compiled into the binary (all permissive open-source licenses), the AI models downloaded on demand from Hugging Face (Apache-2.0), and the external tools called by MAKI (dcraw, ffmpeg, curl — installed separately).

The full license text for every Rust dependency is in the file `THIRD_PARTY_LICENSES.md`, shipped alongside the `maki` binary in every release archive. It is also available at:

`https://github.com/thoherr/maki/releases/latest/download/THIRD_PARTY_LICENSES.md`

OPTIONS

–**summary** Show only the section headers and pointers to the full license text, without the explanatory paragraphs.

`--json` outputs a structured object with the license metadata for scripting.

EXAMPLES

```
maki licenses               # full overview
maki licenses --summary     # short version
maki licenses --json        # machine-readable
```

SEE ALSO

Licenses & Acknowledgements – the manual appendix with the complete licensing breakdown.

---

maki serve

NAME

maki-serve – start the web UI server

SYNOPSIS

```
maki [GLOBAL FLAGS] serve [--port <PORT>] [--bind <ADDR>] [--read-only]
```

DESCRIPTION

Starts a local web server that provides a browser-based interface for browsing, searching, and editing assets. The web UI offers a rich interactive experience including:

- **Browse page**: Grid of asset thumbnails with a two-row search bar. Row 1: full-width text input. Row 2: tag filter, star rating filter, color label dots, type/format/volume/collection dropdowns, and path prefix input. All filters auto-search with 300ms debounce on text inputs and immediate trigger on dropdowns, stars, labels, and tags.
- **Asset detail page**: Full preview, editable metadata (inline name, description, star rating, color label, tags), variant list, recipe list, and collection membership chips.
- **Tags page** (`/tags`): Sortable tag list with counts, live text filter, and multi-column layout.
- **Collections page** (`/collections`): List of all collections with creation button.
- **Stacks**: When stack collapsing is enabled (default), stacked assets are collapsed in the browse grid to show only the pick image with a stack count badge. A toggle button in the results bar switches between collapsed and expanded views. The `stacked:true` and `stacked:false` search filters are available in the query input.
- **Batch operations**: Checkbox selection on browse cards with a fixed bottom toolbar for batch tagging, rating, labeling, auto-grouping, stacking, and unstacking. "Stack" creates a new stack from selected assets; "Unstack" removes selected assets from their stacks.
- **Keyboard navigation**: Arrow keys navigate between cards, Enter opens details, Space toggles selection, 1-5 sets rating, 0 clears rating, Alt+1-7 sets label, single letters (r/o/y/g/b/p/u) set label by color initial, x clears label.
- **Saved search chips**: Clickable chips on the browse page load saved searches into the filter UI.
- **Tags page**: Shows a collapsible tree view for hierarchical tags (tags containing `/` as a hierarchy separator). Non-hierarchical tags continue to display in the flat multi-column layout.
- **Analytics page** (`/analytics`): Shooting frequency, camera/lens usage, rating distribution, format breakdown, monthly import volume, and storage per volume charts.
- **Drag-and-drop**: Drag browse cards onto the collection dropdown to add to a collection. Drag stack members on the detail page to reorder.
- **Per-stack expand/collapse**: Click the stack badge (⊞ N) on a browse card to expand/collapse just that stack, independent of the global toggle.

The server defaults to `127.0.0.1:8080`. These can be overridden by CLI flags or the `[serve]` section in `maki.toml`. CLI flags take precedence over configuration.

Requests check SQLite connections out of a small pre-opened pool. Previews are served from the catalog directory. Static assets (htmx, Leaflet, style.css, icons) are embedded at compile time.

ARGUMENTS

None.

OPTIONS

–**port <PORT>** Port to listen on. Default: 8080, or the value from `maki.toml [serve]` section.

–**bind <ADDR>** Address to bind to. Default: `127.0.0.1`, or the value from `maki.toml [serve]` section.
–**read-only** Safe-sharing mode: every request that is not `GET`/`HEAD` — metadata edits, jobs, deletions — is rejected with 403 and the UI hides its edit controls. Combine with `--bind` to let other people or devices browse without write access. Can be made permanent with `[serve] read_only = true`; HTTP Basic auth is available via `[serve] username`/`password` (the password may also come from the `MAKI_SERVE_PASSWORD` environment variable). Binding to a non-loopback address without either protection prints a startup warning.

`--log` (global flag) enables per-request logging to stderr in the format `METHOD /path -> STATUS (duration)`.

EXAMPLES

Start the web UI with defaults:

```
maki serve
# Listening on http://127.0.0.1:8080
```

Start on a custom port:

```
maki serve --port 9090
```

Bind to all interfaces (for LAN access):

```
maki serve --bind 0.0.0.0 --port 8080
```

Start with request logging:

```
maki serve --log
```

Start with all diagnostics:

```
maki serve --log --time
```

SEE ALSO

search – CLI equivalent of the web UI browse page. show – CLI equivalent of the web UI asset detail page. CLI Conventions – `maki.toml` configuration reference.

maki shell

NAME

maki-shell – interactive asset management shell

SYNOPSIS

```
maki [GLOBAL FLAGS] shell [SCRIPT]
maki [GLOBAL FLAGS] shell -c <COMMAND>
maki [GLOBAL FLAGS] shell [--strict] [SCRIPT | -c <COMMAND>]
```

DESCRIPTION

Starts an interactive shell that keeps catalog state cached across commands. Commands are entered without the `maki` prefix. The shell provides:

- **Readline editing** with persistent history (stored in `.maki/shell_history`) and tab completion for subcommand names, `--flags`, `$variables`, `tag:` names, and `volume:` labels.
- **Named variables** (`$name`) that store asset ID sets from command results, enabling multi-step workflows.
- **Implicit last result** (`_`) that expands to the asset IDs produced by the most recent command.
- **Session defaults** (`set --flag`) that inject flags into every command for the remainder of the session.
- **Script files** for repeatable workflows — plain text files with one command per line, comments with `#`.
- **Source command** that executes a script file within the current session, sharing variables and defaults.
- **Reload** that re-reads configuration, refreshes tab-completion data, and clears all variables and defaults.

In interactive mode, the prompt shows the catalog directory name and any active variable counts:

```
photos [picks=42 best=5]>
```

Certain commands are blocked inside the shell: `init`, `migrate`, `serve`, and `shell`.

ARGUMENTS

SCRIPT Path to a script file to execute. Each non-empty, non-comment line is run as a shell command. The shell exits after the script completes.

OPTIONS

-c, –command <COMMAND> Execute a single command string and exit. Supports multiple lines (separated by newlines) and variable assignments, just like a script.

–strict Exit with a non-zero status on the first error. Applies to script and `-c` modes. Without `--strict`, errors are printed but execution continues with the next line.

SHELL-ONLY COMMANDS

These commands are available only inside the shell (interactive, script, and `-c` modes):

Command	Description
`help`	Show a summary of shell features and syntax
`quit` / `exit`	End the session (also Ctrl-D)
`vars`	List all named variables with asset counts, plus active session defaults

Command	Description
`unset $name`	Remove a named variable
`unset --flag`	Remove a session default flag
`set --flag`	Add a session default flag. Settable flags: `--json`, `--log`, `--verbose`, `--debug`, `--time`
`reload`	Re-read config, refresh completions, clear all variables and defaults
`source <file>`	Execute a script file in the current session, sharing variables and defaults. Paths are resolved relative to the catalog root

VARIABLE SYNTAX

Assignment:

`$name = <command>`

Runs `<command>` and stores the resulting asset IDs in `$name`. The count is printed to stderr. Variable names may contain letters, digits, and underscores.

Expansion:

`$name` and `_` expand to asset IDs. The IDs are always placed at the **end** of the argument list, regardless of where the variable appears in the command. This means `tag $picks --add portfolio` and `tag --add portfolio $picks` are equivalent — the IDs always land in the trailing positional slot where commands expect asset IDs. Unknown variables are left as-is.

`_` (standalone, not inside a word like `_foo` or `foo_bar`) expands to the asset IDs from the most recent command that produced results.

QUOTE HANDLING

The shell uses a two-rule quoting model:

- **Token-start quotes** (quotes at the beginning of a token) are stripped. They act as grouping quotes, like in a POSIX shell: `"tag:landscape rating:4+"` becomes a single token `tag:landscape rating:4+`.
- **Mid-token quotes** (quotes appearing after other characters) are preserved. This keeps search filter syntax intact: `text:"woman with glasses"` passes through unchanged as a single token.

EXAMPLES

Interactive session with variables:

```
$ maki shell
photos> $picks = search "rating:5 date:2024"
  42 assets → $picks
photos [picks=42]> tag --add portfolio $picks
photos [picks=42]> export --target /tmp/best $picks
photos [picks=42]> quit
```

Run a script file:

```
maki shell workflow.maki
```

One-liner with `-c`:

```
maki shell -c 'search "tag:landscape rating:4+" --format ids'
```

Session defaults:

```
photos> set --log
  Session default: --log
  Active defaults: --log
photos> search tag:landscape
# (output includes per-file logging)
photos> unset --log
```

Source a file into the current session:

```
photos> source post-import.maki
```

Strict mode for CI scripts:

```
maki shell --strict batch-updates.maki
```

SEE ALSO

search – primary command for finding assets inside the shell. CLI Conventions – global flags (`--json`, `--log`, `--debug`, `--time`) usable as session defaults.

Previous: Organize Commands – `collection`, `saved-search`, `stack`. Next: Maintain Commands – `verify`, `sync`, `refresh`, `cleanup`, `relocate`, `update-location`, `generate-previews`, `fix-roles`, `fix-dates`, `rebuild-catalog`.

Maintain Commands

Commands for integrity checks, disk reconciliation, metadata sync, file relocation, preview generation, and catalog housekeeping. Most commands default to report-only mode; add `--apply` to execute changes.

Command	Description
undo	Revert the most recent metadata edit
history	Show recent metadata edits
doctor	Check YAML-sidecar ↔ SQLite-catalog consistency
verify	Check file integrity via SHA-256
sync	Reconcile catalog with moved/modified/missing files
refresh	Re-read metadata from changed recipe files
sync-metadata	Bidirectional XMP sync with conflict detection *(Pro)*
writeback	Write pending metadata to XMP files *(Pro)*
cleanup	Remove stale records and orphaned files
dedup	Remove same-volume duplicate file locations
trash	Inspect, restore, or empty the catalog trash
relocate	Copy or move files between volumes
update-location	Fix catalog path after manual file move
generate-previews	Generate or regenerate preview thumbnails
fix-roles	Correct variant roles (RAW=Original, non-RAW=Export)
fix-dates	Fix asset dates from EXIF metadata
fix-recipes	Re-attach recipe files imported as standalone assets
create-sidecars	Create XMP sidecars for assets without recipes
rebuild-catalog	Rebuild SQLite from YAML sidecars
migrate	Run database schema migrations

maki undo

NAME

maki-undo – revert the most recent metadata edit

SYNOPSIS

```
maki [GLOBAL FLAGS] undo [--dry-run] [--force]
```

DESCRIPTION

Reverts the newest not-yet-undone operation from the edit-history journal (`<catalog_root>/history/`), restoring each touched asset's prior state in both the SQLite catalog and the YAML sidecar.

Undo is **LIFO**: each invocation reverts the most recent surviving operation, and a reverted operation is moved aside (into `history/undone/`) so the next `maki undo` walks back to the one before it.

What undo covers — **field-level metadata edits only**: rating, color label, description, name, date, and tag add/remove/clear, made either singly or in a batch. A batch edit (e.g. rating 500 assets at once) is recorded as **one** operation and undone as one. Undo does **not** cover `import`, `delete`, or structural operations (group, split, stack, merge) — those have their own recovery paths (re-import is idempotent, delete is trash-backed; structural undo is a planned follow-on).

Conflict handling. Before restoring each asset, undo checks that the asset's *current* state still matches what the operation recorded as its result. If a later edit has touched the asset since, that asset is a **conflict**: it is reported and skipped, and the operation is left in place (not marked undone) so you can decide. Pass `--force` to restore conflicting assets anyway.

`--dry-run` reports what would be undone and changes nothing.

NOTES

Undo restores catalog + sidecar state only; it does **not** write to `.xmp` files live. Any affected `.xmp` recipes are flagged `pending_writeback`, so if you keep edits mirrored to XMP, run `maki writeback` afterwards to propagate the restored values to disk. (Most catalogs run with auto-writeback off — the documented default — so this is opt-in.)

The history journal is **non-authoritative**: it is neither a sidecar (source of truth) nor part of `catalog.db` (derived cache). It survives `rebuild-catalog`, is ignored by `maki doctor`, and may be deleted at any time — losing only the ability to undo past edits. Journaling and retention are configured under `[history]` in `maki.toml` (see the configuration reference).

SEE ALSO

history – list recorded edit operations. writeback – flush pending metadata changes to XMP files *(Pro)*.

---

maki history

NAME

maki-history – show recent metadata edits

SYNOPSIS

```
maki [GLOBAL FLAGS] history [<asset>] [--limit N]
```

DESCRIPTION

Lists operations from the edit-history journal, newest first. Each line shows a compact timestamp, the command label, a human summary, and the number of assets the operation touched.

With an `<asset>` argument (resolved by unambiguous id prefix, like other asset commands), `history` lists only operations that touched that asset and shows the **field-level change** each one made to it — e.g. `rating 2 → 4`, `tags +[nature, wildlife]`, `label red → —`.

`--limit N` caps the number of operations shown (default 20). `--json` emits the full operation records (asset-scoped when an asset is given).

ARGUMENTS

`<asset>` Optional asset id or unambiguous prefix to scope the history to.

OPTIONS

`--limit N` Maximum number of operations to display (default `20`).

EXAMPLES

Show the last 20 edits across the catalog:

```
maki history
```

Show the edit history for one asset:

```
maki history a1b2c3d4
```

SEE ALSO

undo – revert the most recent edit operation.

maki doctor

NAME

maki-doctor – check YAML-sidecar ↔ SQLite-catalog consistency

SYNOPSIS

```
maki [GLOBAL FLAGS] doctor [--sample N] [--repair]
```

DESCRIPTION

The YAML sidecars under `metadata/` are the source of truth; the SQLite catalog is a derived cache. Every write path is supposed to update both — `doctor` verifies that it actually happened, and can repair the catalog when it didn't.

Checked per asset:

- **Presence both ways** — sidecars without a catalog row, catalog rows without a sidecar (phantom rows), unreadable sidecar files.
- **Metadata fields** — name, date, type, tags, description, rating, color label, preview rotation/variant.
- **Denormalized columns** — `best_variant_hash`, `primary_variant_format`, `variant_count`, `leaf_tag_count`, recomputed from the sidecar with the same helpers the write path uses.
- **Variant, file-location, and recipe sets** — including each recipe's `content_hash` and `pending_writeback` flag (the exact divergence class behind the v4.5.15 stale-pending bug).
- **face_count** against the faces table.

`--repair` rebuilds the SQLite side from YAML for every flagged asset (faces, embeddings, and collection/stack memberships are preserved) and deletes phantom rows. Repair never edits sidecars.

`--sample N` limits the field-by-field comparison to ~N evenly spaced assets for a fast probabilistic check on large catalogs; the presence checks always cover everything.

With `--log`, each finding is printed per asset; the default summary aggregates mismatches by field. `--json` emits the full report.

NOTES

Doctor checks **store consistency**, not disk integrity — verifying file content hashes on your volumes is `maki verify`'s job. Run doctor after interrupted operations, before major reorganisations, or whenever pending-writeback markers look wrong.

SEE ALSO

verify – verify file content hashes on disk. rebuild-catalog – full rebuild of the SQLite catalog from sidecars.

---

maki verify

NAME

maki-verify – re-hash files on disk and compare against stored content hashes

SYNOPSIS

```
maki [GLOBAL FLAGS] verify [PATHS...] [--max-age DAYS] [--force] [OPTIONS]
```

DESCRIPTION

Verifies file integrity by re-hashing files on disk and comparing the computed SHA-256 hash against the stored content hash in the catalog. Detects corruption, bit rot, or unauthorized modification.

Catalog mode (no paths): Verifies all file locations on all online volumes. Checks whether cataloged files are intact, detecting OK, FAILED, MODIFIED, MISSING, and SKIPPED statuses. `--volume` limits to a specific volume; `--asset` limits to a specific asset.

Path mode (with paths): Scans files at the given paths on disk and looks them up in the catalog by content hash. In addition to the catalog mode statuses, can report UNTRACKED files — files on disk whose content hash does not match any known variant or recipe.

On successful verification, updates the `verified_at` timestamp on each file location record (persisted to both SQLite catalog and sidecar YAML for variant and recipe locations).

Incremental verify: Use `--max-age` to skip files that were verified recently. Only files whose `verified_at` timestamp is older than the given number of days (or never verified) are re-hashed. This enables fast periodic checks on large catalogs. `--force` overrides the skip and re-verifies everything. A default `max_age_days` can be set in `maki.toml` under `[verify]`.

Result statuses:

Status	Description	Mode
OK	File hash matches the catalog record.	Both
FAILED	Media file hash does not match (corruption or replacement).	Both
MODIFIED	Recipe file changed externally; hash updated in catalog.	Both
MISSING	Catalog location exists but file is gone from disk.	Catalog
UNTRACKED	File on disk not found in catalog.	Path
SKIPPED	Volume offline, path unreadable, or other error.	Both

Exit codes: Exits with code 1 if any hash mismatches (FAILED) are found. Exits with code 0 for all other statuses, including MODIFIED.

Recipe handling: Recipe files (XMP, COS, etc.) that have been modified externally are reported as "modified" rather than "FAILED" and do not trigger exit code 1. Their stored hash is updated to reflect the new content.

Offline volumes are silently skipped.

ARGUMENTS

PATHS (optional) One or more file paths or directories to verify. When omitted, verifies all file locations on online volumes.

OPTIONS

–**volume <LABEL>** Limit verification to a specific volume.
–**asset <ID>** Verify only the file locations of a specific asset. Supports prefix matching.
–**max-age <DAYS>** Skip files verified within the given number of days. Enables incremental verification.
–**force** Override `--max-age` and re-verify all files.
–**include <GROUP>** Include additional file type groups. Can be specified multiple times.
–**skip <GROUP>** Skip file type groups. Can be specified multiple times.

`--json` outputs a `VerifyResult` with `verified`, `failed`, `modified`, `skipped`, `skipped_recent`, `missing` counters and detail arrays.

`--log` prints per-file verification status and timing to stderr.

EXAMPLES

Verify the entire catalog:

```
maki verify
```

Verify a specific volume with progress logging:

```
maki verify --volume "Photos" --log --time
```

Verify a single asset:

```
maki verify --asset a1b2c3d4
```

Verify a specific directory:

```
maki verify /Volumes/Photos/Capture/2026-02-22
```

Incremental verify — skip files checked in the last 30 days:

```
maki verify --max-age 30
```

Force re-verify everything, ignoring recent timestamps:

```
maki verify --force --max-age 30
```

Verify and check for failures in a script:

```
if ! maki verify --volume "Archive"; then
  echo "Integrity check failed!"
fi
```

SEE ALSO

sync – reconcile catalog with disk after files are moved or modified. cleanup – remove stale location records for missing files. stats – `--verified` shows verification health overview.

---

maki sync

NAME

maki-sync – reconcile catalog with disk changes

SYNOPSIS

```
maki [GLOBAL FLAGS] sync <PATHS...> [OPTIONS]
```

DESCRIPTION

Scans paths on disk and reconciles the catalog with the current disk state. Detects files that have been moved, renamed, modified, or deleted by external tools since the last import.

Detected states:

State	Description
Unchanged	File at expected path with matching hash.
Moved	Known hash found at a new path; old path is gone.
New	Unknown hash at a new path (not yet imported).
Modified recipe	Recipe file at same path but with a different hash.
Missing	Catalog location points to a file that no longer exists.

Without `--apply`, runs in **report-only mode** (safe default) and shows what it found without making changes. With `--apply`, updates catalog and sidecar files for moved files and modified recipes. `--remove-stale` (requires `--apply`) removes catalog location records for confirmed-missing files.

New files are reported but not auto-imported – run `maki import` separately to bring them into the catalog.

ARGUMENTS

PATHS (required) One or more file paths or directories to scan.

OPTIONS

–**volume <LABEL>** Use a specific volume instead of auto-detecting from the path.
–**apply** Apply changes to catalog and sidecar files. Without this flag, only reports what it found.
–**remove-stale** Remove catalog location records for missing files. Requires `--apply`. Assets that become locationless (all locations removed) are automatically deleted along with their sidecar files.

`--json` outputs a `SyncResult` with counts and detail arrays for each state.

`--log` prints per-file status to stderr.

`--time` shows elapsed wall-clock time.

EXAMPLES

Preview what sync would find (report-only):

```
maki sync /Volumes/Photos/Capture
```

Apply changes for moved and modified files:

```
maki sync /Volumes/Photos --apply --log
```

Apply changes and remove stale location records:

```
maki sync /Volumes/Photos --apply --remove-stale
```

Sync a specific volume:

```
maki sync /Volumes/Archive --volume "Archive" --apply
```

Sync with full diagnostics:

```
maki sync /Volumes/Photos --apply --log --time --json
```

SEE ALSO

refresh – re-read metadata from changed recipe files. cleanup – remove stale records across all volumes. verify – check file integrity without reconciliation. import – import new files discovered by sync.

---

maki refresh

NAME

maki-refresh – re-read metadata from changed sidecar and recipe files

SYNOPSIS

```
maki [GLOBAL FLAGS] refresh [PATHS...] [OPTIONS]
```

DESCRIPTION

Checks recipe files for changes and re-extracts metadata when modifications are detected. For each recipe, compares the on-disk hash to the stored hash. If the file has changed, re-extracts XMP metadata (keywords, rating, description, color label) and updates catalog and sidecar.

This is useful for picking up changes made by external tools like CaptureOne or Lightroom that modify XMP sidecars outside of maki.

Without arguments, checks all recipe locations on all online volumes. With paths, scans recipe files under given paths. `--volume` limits to a specific volume; `--asset` limits to a specific asset's recipes.

–**media** additionally scans JPEG and TIFF variant files and re-extracts their embedded XMP metadata (keywords, rating, description, label, creator, rights). This is useful for retroactively extracting embedded XMP from files imported before the feature existed, or after external tools edit the embedded metadata.

Non-XMP recipes (COS, pp3, etc.) get their hash updated but no metadata extraction.

ARGUMENTS

PATHS (optional) One or more file paths or directories to scan for recipe files. When omitted, checks all recipe locations on online volumes.

OPTIONS

–**volume <LABEL>** Limit to a specific volume.
–**asset <ID>** Refresh only a specific asset's recipes. Supports prefix matching.
–**dry-run** Report what would change without applying updates.
–**media** Also re-extract embedded XMP from JPEG/TIFF media files (not just recipe files).
–**reimport** Clear all asset metadata and re-extract from source files (XMP sidecars, embedded XMP, EXIF). Also fully re-syncs SQLite with the sidecar YAML — deletes and re-inserts all variants, locations, and recipes. Deduplicates recipes and locations. Requires `--asset <ID>` or asset IDs as positional arguments. This is the CLI equivalent of the web UI "Re-import metadata" button.
–**exif-only** Re-extract only EXIF/source metadata (camera, lens, ISO, exposure, resolution, date) from media files, WITHOUT clearing tags, description, rating, or color label. Useful for backfilling missing EXIF data on assets whose tags and ratings have been manually curated. Requires `--asset <ID>` or asset IDs as positional arguments.

`--json` outputs a refresh result with changed/unchanged counts and detail arrays.

`--log` prints per-file status to stderr.

`--time` shows elapsed wall-clock time.

EXAMPLES

Check all recipes for changes (report-only by default):

```
maki refresh --dry-run
```

Refresh all recipes on a specific volume:

```
maki refresh --volume "Photos" --log
```

Refresh recipes for a single asset:

```
maki refresh --asset a1b2c3d4
```

Refresh recipes and also re-extract embedded XMP from media files:

```
maki refresh --media --log --time
```

Refresh a specific directory after editing in CaptureOne:

```
maki refresh /Volumes/Photos/Capture/2026-02-22 --log
```

SEE ALSO

sync – full reconciliation of catalog with disk. sync-metadata – bidirectional XMP metadata sync. verify – integrity checking without metadata re-extraction. import – initial import with XMP extraction.

---

maki sync-metadata *(Pro)*

NAME

maki-sync-metadata – bidirectional XMP metadata sync between MAKI and recipe files

SYNOPSIS

```
maki [GLOBAL FLAGS] sync-metadata [QUERY] [OPTIONS]
```

DESCRIPTION

Performs bidirectional metadata synchronization between the MAKI catalog and XMP recipe files on disk. Scope can be narrowed with a positional search query, `--asset`, or `--volume`. Runs in three phases:

1. **Inbound (read external changes)**: Detects XMP recipe files that have been modified externally (e.g., by CaptureOne or Lightroom). Re-reads keywords, rating, description, and color label from the changed XMP file and updates the catalog and sidecar YAML.
2. **Outbound (write pending edits)**: Finds recipes marked `pending_writeback` (edits made while the volume was offline) and writes the current MAKI metadata back to the XMP file on disk.
3. **Media (with `--media`)**: Re-extracts embedded XMP metadata from JPEG/TIFF variant files.

When both the disk file and the MAKI catalog have changed (phase 1 detects a change AND `pending_writeback` is set), the recipe is reported as a **conflict**. Conflicts are skipped — resolve manually or re-run with `refresh` or `writeback` as appropriate.

Without `--dry-run`, changes are applied immediately.

OPTIONS

<QUERY> (positional, optional) Search query to scope which assets are synced (same syntax as `maki search`).

–**volume <LABEL>** Limit to recipes on a specific volume.

–**asset <ID>** Limit to recipes belonging to a specific asset (prefix match).

–**dry-run** Report what would change without applying.

–**media** Also re-extract embedded XMP from JPEG/TIFF variant files (phase 3).

`--json` outputs a `SyncMetadataResult` with per-status counts and conflict details.

`--log` prints per-recipe status to stderr.

`--time` shows elapsed wall-clock time.

EXAMPLES

Sync only assets matching a query:

```
maki sync-metadata "volume:Photos date:2024"
```

Preview what sync-metadata would do:

```
maki sync-metadata --dry-run
```

Sync a specific volume after reconnecting:

```
maki sync-metadata --volume "Photos 2024" --log
```

Full sync including embedded XMP:

```
maki sync-metadata --media --log --time
```

SEE ALSO

refresh – one-way inbound metadata re-read. writeback – one-way outbound metadata write. sync – file-level reconciliation (moves, renames, missing files).

---

maki writeback *(Pro)*

NAME

maki-writeback – write pending metadata changes to XMP recipe files

SYNOPSIS

```
maki [GLOBAL FLAGS] writeback [QUERY] [OPTIONS]
```

DESCRIPTION

Flushes pending metadata writes to XMP recipe files on disk. The pending queue accumulates whenever an edit (rating, label, tags, description) cannot or should not be written to disk inline:

- The recipe lives on an offline volume.
- The XMP file is missing on disk.
- `[writeback] enabled = false` (the safety-net default in `maki.toml`) — every edit is tracked but no `.xmp` file is touched until this command runs.

This command **always works**, regardless of `[writeback] enabled`. The config flag governs only *automatic* writeback on every edit; `maki writeback` is the explicit manual flush, and is the supported way to keep auto-flush off as a safety net while still pushing staged metadata onto disk on demand. See the `[writeback]` configuration reference for the full split.

Without `--all`, only recipes with `pending_writeback=1` are processed. Each recipe's XMP file is updated with the current asset metadata (rating, label, tags, description), then re-hashed and the pending flag is cleared.

Scope can be narrowed with a positional search query, `--asset`, or `--volume`. Without scope options, all pending recipes across all online volumes are processed.

OPTIONS

<QUERY> (positional, optional) Search query to scope which assets are written back (same syntax as `maki search`).

–**volume <LABEL>** Limit to recipes on a specific volume.

–**asset <ID>** Limit to recipes belonging to a specific asset (prefix match).

–**all** Write current metadata to every XMP recipe in the matching set, not just those flagged pending. Useful for an initial sync, for rematerialising catalog metadata onto disk after large catalog-only restructuring (rename, split, rebuild), or for pushing accumulated edits when you've kept auto-flush off and now want a known asset set written through.

–**mirror-tags (requires --all)** Make XMP keyword lists exactly mirror the catalog. Without this flag, writeback is **additive**: catalog tags get pushed onto the XMP but XMP tags the catalog no longer has are left untouched. Renames, splits, deletions, and `tag fix-unicode` performed in MAKI therefore leave stale entries on disk — invisible drift that compounds across batches. With `--mirror-tags`, the writer reads the existing `dc:subject` and `lr:hierarchicalSubject`, diffs against the asset's catalog tags, and removes the stale entries before writing the current set. Pair with a query (`--mirror-tags --all "tag:NewName"` etc.) to scope a one-shot reconciliation; `--all` is required because the mirror operation is most useful as a broad sweep, not on the narrow pending-only default.

–**force** Rebuild the `dc:subject` and `lr:hierarchicalSubject` blocks from catalog state, discarding any pre-existing XMP entry. More aggressive than `--mirror-tags`: where `--mirror-tags` diffs against the catalog and only removes entries the catalog doesn't have, `--force` empties both blocks completely and re-adds the catalog's tags fresh. Use this when stale entries are stuck in a file that `--mirror-tags` doesn't catch — runaway entity-escape leftovers (`Bobby &… the BigTones`), leaf-only entries stranded in `lr:hierarchicalSubject` from years of multi-tool roundtrips, manually corrupted XMP — or as a guaranteed "this file now matches catalog" escape hatch after metadata drift. Unlike `--all`, `--force`

keeps the explicit scope (`--asset`, query, or `--volume`) but ignores the pending-flag filter. Combinable with `--all` if you want catalog-wide rebuild.

Recovery workflow for a catalog with accumulated runaway-escape damage across multiple volumes:

```
maki writeback --all --force --volume MediaPortable
maki writeback --all --force --volume Media
maki writeback --all --force --volume Archive
```

One pass per volume sweeps every recipe's tag blocks fresh from catalog state.

–**embed** Write embedded XMP into the asset's **JPEG variant files** instead of flushing `.xmp` recipe sidecars. `--embed` REPLACES the recipe flush for the run — the `.xmp` sidecars (and their pending flags) are left untouched. Some workflows need metadata inside the image file itself (stock submissions, tools that ignore sidecars); modern tools (Lightroom, Capture One, stock agencies) read embedded XMP, which is why v1 writes the XMP packet rather than IPTC IIM / EXIF IFD fields (both deferred, as is TIFF support — TIFF variants are skipped with a per-file status).
What happens per asset in scope: every variant with format `jpg/jpeg` that has an online file location gets its APP1 XMP segment replaced (or inserted, after the Exif segment) with a packet carrying the catalog's rating, label, description, and tags — `dc:subject` flat components plus `lr:hierarchicalSubject` ancestor paths, with the same `--force` / `--mirror-tags` remove-set semantics as the sidecar flush. The segment splice leaves every other byte of the JPEG untouched. Files whose embedded XMP already matches the catalog count as `already in sync` and are not rewritten.
Safety ladder per file: the original is first **copied into the catalog trash** (recover with `maki trash list` / `maki trash restore`; disable with `--no-trash`), the rewritten JPEG is written to a temp file, the temp file's embedded XMP is verified against the intended metadata (a mismatch fails the file and leaves the original untouched), then the temp file atomically replaces the original.
Because the file's bytes change, the variant's **content hash — its identity — changes too**. The hash is re-computed after the write and migrated across the catalog (variants, file locations, recipes, best-variant/preview references), the YAML sidecar, and the preview/smart-preview files on disk, so `maki verify` and `maki doctor` stay clean. Note this means other copies of the same variant on *offline* volumes now hold the pre-rewrite bytes under the new hash — they are reported as skipped (with the volume named); re-run `maki writeback --embed` per volume, or reconcile with `maki sync`, after reconnecting. There is no pending-flag queue for embedded targets.
Scope selection matches the sidecar flush: by default only assets with staged (pending) recipe changes are considered; `--all`/`--force` widen to every asset in scope. JPEG-only assets have no recipes (so nothing pending) — use `--force` (scoped) or `--all` to reach them. Packets too large for a single APP1 segment (>64 KB, would need ExtendedXMP) fail cleanly.

–**no-trash (requires --embed)** Skip preserving the original JPEGs in the catalog trash before rewriting them.

–**dry-run** Report what would be written without modifying any files.

`--json` outputs a result object with counts for written, skipped, failed, and cleared recipes.

`--log` prints per-recipe status to stderr.

`--time` shows elapsed wall-clock time.

EXAMPLES

Process all pending write-backs:

```
maki writeback
```

Write back only assets matching a query:

```
maki writeback "rating:4+ tag:landscape"
```

Write back to a specific volume after reconnecting:

```
maki writeback --volume "Photos 2024" --log
```

Force-write all metadata to XMP (initial sync):

```
maki writeback --all --log --time
```

Reconcile XMP keyword lists with the catalog after large tag restructuring (rename / split / delete / fix-unicode performed while auto-flush was off):

```
maki writeback --all --mirror-tags
maki writeback --all --mirror-tags "tag:NewName"   # scope to one tag's set
```

Rebuild XMP tag blocks from scratch — when `--mirror-tags` didn't catch some stale entries:

```
maki writeback --asset 016cc7dd --force
maki writeback --all --force --volume MediaPortable   # whole-volume cleanup
```

Write metadata into the JPEG files themselves (embedded XMP) — e.g. before a stock-agency export:

```
maki writeback --embed --force --asset 016cc7dd        # one asset's JPEGs
maki writeback --embed --all "rating:4+" --log         # all 4-star+ assets
maki writeback --embed --force --no-trash --asset 016cc7dd   # skip trash copies
```

Preview what would be written:

```
maki writeback --dry-run
```

Shell variable expansion:

```
# In maki shell
writeback $picks
writeback _
```

SEE ALSO

sync-metadata – bidirectional metadata sync (read + write). refresh – one-way inbound metadata re-read.

---

maki cleanup

NAME

maki-cleanup – remove stale location records, orphaned assets, and orphaned derived files

SYNOPSIS

```
maki [GLOBAL FLAGS] cleanup [OPTIONS]
```

DESCRIPTION

Scans all file locations and recipes across online volumes, checking for files that no longer exist on disk. Performs eight passes:

1. **Stale locations and recipes**: Removes catalog and sidecar records for files that are missing from disk.
2. **Locationless variants**: Removes variants with zero remaining file locations from assets that still have other located variants. Prevents ghost variants from accumulating after file moves or reimports. Cleans up derived files (previews, smart previews) and updates denormalized columns.
3. **Orphaned assets**: Deletes assets where all variants have zero remaining file locations, along with their recipes, variants, faces, embeddings, previews, smart previews, face crops, embedding binaries, catalog rows, and sidecar YAML files.
4. **Orphaned previews**: Removes preview files whose content hash no longer matches any variant in the catalog.
5. **Orphaned smart previews**: Same as above for the `smart_previews/` directory.
6. **Orphaned SigLIP embeddings**: Removes embedding binary files (`embeddings/<model>/`) whose asset ID no longer exists in the catalog.
7. **Orphaned face crops**: Removes face crop thumbnails (`faces/`) whose face ID no longer exists in the catalog.
8. **Orphaned ArcFace embeddings**: Removes face embedding binaries (`embeddings/arcface/`) whose face ID no longer exists in the catalog.

Without `--apply`, runs in **report-only mode** (safe default) and predicts what would be removed, including orphaned assets and derived files that would result from removing stale locations.

Offline volumes are skipped with a note.

ARGUMENTS

None.

OPTIONS

–**volume <LABEL>** Limit stale-location scanning to a specific volume. When omitted, checks all online volumes.

–**path <PREFIX>** Limit stale-location scanning to files under this path prefix. Accepts either a relative path on the volume or an absolute path (auto-detects the volume and converts to a relative prefix). Useful for scoping cleanup to a specific subdirectory without scanning the full volume.

–**list** Print stale entries to stderr (shows only stale items, unlike `--log` which prints all entries including OK ones).

–**apply** Apply changes: remove stale records, delete orphaned assets, and remove all orphaned derived files.

`--json` outputs a `CleanupResult` with counts for stale locations, locationless variants, orphaned assets, orphaned previews, orphaned smart previews, orphaned embeddings, and orphaned face files.

`--log` prints per-file status to stderr (both OK and stale entries).

`--time` shows elapsed wall-clock time.

EXAMPLES

Preview what cleanup would remove:

```
maki cleanup
```

List only the stale entries:

```
maki cleanup --list
```

Apply cleanup across all volumes:

```
maki cleanup --apply --log
```

Cleanup a specific volume:

```
maki cleanup --volume "Photos" --apply
```

Cleanup a specific directory:

```
maki cleanup --path "Capture/2026-02" --list
maki cleanup --volume "Photos" --path "Archive/Old" --apply --log
```

Cleanup with JSON output for scripting:

```
maki cleanup --apply --json | jq '{stale: .stale_locations, orphans: .orphaned_assets, previews: .orphaned_preview
```

SEE ALSO

sync – reconcile individual paths (more targeted than cleanup). verify – check integrity without removing records. search – `orphan:true` filter finds assets with no file locations.

---

maki dedup

NAME

maki-dedup – remove same-volume duplicate file locations

SYNOPSIS

```
maki [GLOBAL FLAGS] dedup [OPTIONS]
```

DESCRIPTION

Identifies variants with 2+ file locations on the **same** volume and removes the redundant copies. This targets accidental duplicates (e.g. files copied into multiple directories on the same drive) while leaving cross-volume copies untouched (those are intentional backups).

For each set of same-volume duplicate locations, a resolution heuristic selects which copy to **keep**:

1. If `--prefer` is given (or set in `[dedup] prefer` config), prefer locations whose relative path **contains** the specified string (substring match, not prefix-only).
2. Prefer more recently verified files (by `verified_at` timestamp; never-verified sorts oldest).
3. Prefer shorter relative paths (closer to the volume root).
4. Tiebreak: first alphabetically (deterministic).

Before removing a location, the command checks that the variant's total location count across **all** volumes won't drop below `--min-copies`.

When a file location is removed, co-located recipe files (XMP sidecars etc.) in the same directory are automatically cleaned up from disk, catalog, and sidecar YAML.

Without `--apply`, runs in **report-only mode** (safe default): shows what would be removed without making any changes. Recipe file counts are included in the dry-run report.

ARGUMENTS

None.

OPTIONS

–**volume <LABEL>** Limit deduplication to a specific volume. When omitted, processes same-volume duplicates on all volumes.

–**prefer <STRING>** Prefer keeping locations whose relative path contains this string (substring match). Useful for keeping files in a curated directory (e.g. `--prefer Selects`) while removing copies elsewhere. Falls back to the `[dedup] prefer` value in `maki.toml` when not given on the command line.

–**filter-format <FORMAT>** Filter to a specific file format (e.g. `nef`, `jpg`). Only processes duplicate groups matching this format.

–**path <PREFIX>** Filter to locations under this path prefix. Only processes duplicates with locations matching the prefix.

–**min-copies <N> (default: 1)** Minimum total locations to preserve per variant across all volumes. Prevents removing a location if it would leave fewer than N copies total. Set to 2 to ensure at least one backup copy survives.

–**apply** Apply changes: delete physical files and co-located recipe files from disk, remove location and recipe records from catalog and sidecar YAML. Deleted files go to `<catalog>/.trash/` (see trash).

–**no-trash** Delete files permanently instead of moving them to the catalog trash.

`--json` outputs a `DedupResult` with `duplicates_found`, `locations_to_remove`, `locations_removed`, `files_deleted`, `recipes_removed`, `bytes_freed`, `dry_run`, and `errors`.

`--log` prints per-location status to stderr (keep, remove, skipped).

`--time` shows elapsed wall-clock time.

EXAMPLES

Preview what dedup would remove:

```
maki dedup
```

Remove same-volume duplicates across all volumes:

```
maki dedup --apply --log
```

Dedup a specific volume, preferring files under `Selects/`:

```
maki dedup --volume "Photos" --prefer "Selects" --apply
```

Ensure at least 2 copies survive per variant:

```
maki dedup --min-copies 2 --apply
```

JSON output for scripting:

```
maki --json dedup --apply | jq '{groups: .duplicates_found, removed: .locations_removed, freed: .bytes_freed}'
```

SEE ALSO

duplicates – find duplicates without removing them. cleanup – remove stale locations for files that no longer exist on disk. verify – update `verified_at` timestamps used by the dedup heuristic. trash – removed files are recoverable from the trash.

---

maki trash

NAME

maki-trash – inspect, restore, or empty the catalog trash

SYNOPSIS

```
maki trash list [--json]
maki trash restore <DATE>/<VOLUME>/<RELATIVE-PATH>
maki trash empty [--older-than DAYS | --all] [--dry-run]
```

DESCRIPTION

File-deleting operations — `maki delete --remove-files`, `maki dedup`, and the web UI duplicates page — move files into `<catalog>/.trash/<date>/<volume>/<relative-path>` instead of deleting them permanently, so a mistaken batch delete is recoverable. Derived files (previews, embedding binaries, face crops) are exempt: they are regenerable and never enter the trash.

The mechanism is controlled by the `[trash]` section in `maki.toml` (`enabled`, default `true`; `retention_days`, default `30`). Every deleting command also accepts `--no-trash` to delete permanently on a per-run basis.

`list` shows every trashed file with its size and restore key.

`restore` moves a file back to its original location on its original volume (resolved by label; the volume must be online). The catalog is **not** updated — re-register the restored file with `maki import <path>` or `maki refresh`.

`empty` permanently deletes trashed files older than the cutoff: `--older-than DAYS` overrides the configured `retention_days`; `--all` empties everything; `--dry-run` only reports.

NOTES

The trash lives on the catalog volume. Removing a file from another (media) volume copies it into the trash before deleting the original — for very large files this costs one copy, which is the price of recoverability. When freeing disk space is the whole point, pass `--no-trash`.

SEE ALSO

delete – delete assets (optionally with files). dedup – remove same-volume duplicate copies.

---

maki relocate

NAME

maki-relocate – copy or move asset files to another volume

SYNOPSIS

```
maki [GLOBAL FLAGS] relocate <ASSET_ID> <VOLUME> [OPTIONS]
maki [GLOBAL FLAGS] relocate <ASSET_IDS...> --target <VOLUME> [OPTIONS]
maki [GLOBAL FLAGS] relocate --query <QUERY> --target <VOLUME> [OPTIONS]
```

DESCRIPTION

Copies all files belonging to one or more assets (variants and recipes) to a target volume. After copying, verifies file integrity via SHA-256 comparison. Preserves the relative path structure on the target volume.

Without `--remove-source`, files are copied and the asset gains additional file locations on the target volume. With `--remove-source`, source files are deleted after verified copy, effectively moving the asset.

Single-asset mode: `maki relocate <ID> <VOLUME>` — backward-compatible form with two positional arguments.

Batch mode: Use `--target` with one of: - Multiple positional IDs: `maki relocate <ID1> <ID2> ... --target <VOL>` - Search query: `maki relocate --query "tag:landscape" --target <VOL>` - Stdin: `maki search -q "rating:5" | maki relocate --target <VOL>`

Asset IDs support unique prefix matching. Batch mode reports per-asset progress with `--log` and a summary at the end.

ARGUMENTS

ASSET_IDS (positional) One or more asset IDs or unique prefixes. Reads from stdin when empty and no `--query` is given.

OPTIONS

–**target <VOLUME>** Target volume label or UUID. Required for batch mode. In single-asset mode, the volume can be given as the second positional argument instead.

–**query <QUERY>** Search query to select assets for batch relocate. Uses the same query syntax as `maki search`.

–**remove-source** Delete source files after successful copy and SHA-256 verification.

–**create-sidecars** After copying, create `.xmp` sidecar files at the destination for variants that have metadata (ratings, tags, labels, descriptions) but no existing XMP recipe on the target volume. The generated XMP files include `dc:subject` (tags), `lr:hierarchicalSubject` (hierarchical tags), `xmp:Rating`, `xmp:Label`, and `dc:description`. Each sidecar is registered as a recipe in the catalog and YAML sidecar. This enables CaptureOne, Lightroom, and other XMP-aware tools to pick up MAKI metadata immediately. File naming follows `[writeback] sidecar_naming` (default `DSC_1234.xmp`; see create-sidecars for the rules, including the existing-file guard).

–**dry-run** Show what would happen without making any changes.

`--json` outputs a JSON result with `assets`, `copied`, `skipped`, `removed`, `errors`, and `dry_run` fields.

EXAMPLES

Copy an asset to an archive volume:

```
maki relocate a1b2c3d4 "Archive"
```

Move an asset (copy + delete source):

```
maki relocate a1b2c3d4 "Archive" --remove-source
```

Batch relocate by search query:

```
maki relocate --query "tag:landscape rating:4+" --target "Archive" --dry-run
maki relocate --query "date:2024" --target "Archive 2024" --remove-source --log
```

Batch relocate via stdin:

```
maki search -q "volume:Working date:2023" | maki relocate --target "Cold Storage" --remove-source
```

Multiple IDs:

```
maki relocate a1b2c3d4 e5f6g7h8 --target "Backup" --dry-run
```

Relocate with full diagnostics:

```
maki relocate a1b2c3d4 "Archive" --remove-source --log --time
```

SEE ALSO

update-location – update path after a manual move. verify – verify file integrity after relocation. volume list – see available volumes.

---

maki update-location

NAME

maki-update-location – update a file's catalog path after it was manually moved on disk

SYNOPSIS

```
maki [GLOBAL FLAGS] update-location <ASSET_ID> --from <OLD_PATH> --to <NEW_PATH> [--volume <LABEL>]
```

DESCRIPTION

Updates the catalog to reflect a file that was manually moved on disk (outside of maki). The file at the new path is verified to have the same content hash as the catalog record (safety check against accidental mismatches).

`--from` specifies the old path as recorded in the catalog (absolute or volume-relative). `--to` must be an absolute path to the file's current location on disk.

The volume is auto-detected from `--to` by matching against registered volume mount points, or can be specified explicitly with `--volume`.

Handles both variant file locations and recipe file locations.

Asset IDs support unique prefix matching.

ARGUMENTS

ASSET_ID (required) The asset ID or a unique prefix of it.

OPTIONS

–**from <OLD_PATH> (required)** The old path where the file was before (absolute or volume-relative).
–**to <NEW_PATH> (required)** The new absolute path where the file is now.
–**volume <LABEL>** Volume label or UUID. Auto-detected from `--to` if omitted.

`--json` outputs the updated location details.

EXAMPLES

Update a file that was moved to a new directory:

```
maki update-location a1b2c3d4 \
  --from "Capture/2026-02-22/DSC_001.nef" \
  --to /Volumes/Photos/Processed/2026/DSC_001.nef
```

Update with explicit volume:

```
maki update-location a1b2c3d4 \
  --from /Volumes/OldDrive/Photos/IMG_001.jpg \
  --to /Volumes/NewDrive/Photos/IMG_001.jpg \
  --volume "NewDrive"
```

Update a recipe file location:

```
maki update-location a1b2c3d4 \
  --from "Capture/2026-02-22/DSC_001.xmp" \
  --to /Volumes/Photos/Processed/2026/DSC_001.xmp
```

SEE ALSO

relocate – copy or move files with maki managing the transfer. sync – automatic detection of moved files.

---

maki generate-previews

NAME

maki-generate-previews – generate or regenerate preview thumbnails

SYNOPSIS

```
maki [GLOBAL FLAGS] generate-previews [PATHS...] [OPTIONS]
```

DESCRIPTION

Generates preview thumbnails for assets. Standard image formats produce 800px JPEG thumbnails via the `image` crate. RAW files use `dcraw` or `dcraw_emu` (LibRaw). Videos use `ffmpeg`. Non-visual formats (audio, documents) get an info card showing file metadata.

Without PATHS: Iterates all catalog assets and generates previews for the best variant of each asset (Export > Processed > Original, standard image formats preferred over RAW, file size tiebreak). Can be filtered with `--asset` or `--volume`.

With PATHS: Resolves files on disk and looks up their variants in the catalog, generating previews only for those specific files.

Previews are stored in `previews/<hash-prefix>/<hash>.jpg`. Preview settings (max edge size, format, quality) are configured in the `[preview]` section of `maki.toml`.

ARGUMENTS

PATHS (optional) One or more files or directories to generate previews for.

OPTIONS

–**volume <LABEL>** Limit to variants on a specific volume.
–**asset <ID>** Generate preview only for a specific asset. Supports prefix matching.
–**include <GROUP>** Include additional file type groups. Can be specified multiple times.
–**skip <GROUP>** Skip file type groups. Can be specified multiple times.
–**force** Regenerate previews even if they already exist. For video assets, also backfills video metadata (duration, codec, resolution, framerate via `ffprobe`) when it is missing — useful for assets imported before video metadata extraction was available.
–**upgrade** Regenerate previews for assets where a better variant (export or processed) exists than what the current preview was generated from. Skips assets where the best variant is already the source of the preview.
–**smart** Also generate smart previews (high-resolution, 2560px) alongside regular thumbnails. Smart previews enable zoom and pan in the web UI lightbox. Dimensions controlled by `[preview] smart_max_edge` in `maki.toml`. Combines with `--force` to regenerate both preview types.

`--json` outputs a result with generated/skipped/failed counts.

`--log` prints per-file generation status to stderr.

`--debug` shows stderr output from external tools (dcraw, ffmpeg) for diagnosing failures.

EXAMPLES

Generate all missing previews:

```
maki generate-previews
```

Regenerate all previews (force):

```
maki generate-previews --force --log --time
```

Regenerate all previews including smart previews:

```
maki generate-previews --smart --force --log --time
```

Upgrade previews to use better variants (e.g., after grouping exports with originals):

```
maki generate-previews --upgrade --log
```

Generate preview for a single asset:

```
maki generate-previews --asset a1b2c3d4
```

Generate previews for files in a specific directory:

```
maki generate-previews /Volumes/Photos/Capture/2026-02-22
```

Generate previews with debug output for troubleshooting RAW files:

```
maki generate-previews --force --debug --asset a1b2c3d4
```

SEE ALSO

import – previews are generated automatically during import. serve – web UI displays preview thumbnails.

---

maki fix-roles

NAME

maki-fix-roles – fix variant roles in multi-variant assets

SYNOPSIS

```
maki [GLOBAL FLAGS] fix-roles [PATHS...] [OPTIONS]
```

DESCRIPTION

Scans assets that contain both RAW and non-RAW variants and re-roles non-RAW variants from `original` to `export`. This corrects role assignments that may have been missed during import or grouping.

In a properly organized asset, only the RAW file should have the `original` role. Non-RAW files (JPEG, TIFF, etc.) in the same asset should be `export` or `processed`.

Without `--apply`, runs in **report-only mode** and shows what roles would change. With `--apply`, updates both YAML sidecar files and the SQLite catalog.

ARGUMENTS

PATHS (optional) Files or directories to scope the fix. When omitted, checks all assets.

OPTIONS

–**volume <LABEL>** Limit to a specific volume.
–**asset <ID>** Fix only a specific asset. Supports prefix matching.
–**apply** Apply the role changes. Without this flag, only reports what would change.

EXAMPLES

Preview what roles would change:

```
maki fix-roles
```

Apply role fixes:

```
maki fix-roles --apply --log
```

Fix roles for a single asset:

```
maki fix-roles --asset a1b2c3d4 --apply
```

Fix roles on a specific volume:

```
maki fix-roles --volume "Photos" --apply --log --time
```

SEE ALSO

group – grouping merges variants and adjusts roles. auto-group – automatic grouping with role adjustment. show – inspect current variant roles.

maki fix-dates

NAME

maki-fix-dates – fix asset dates from variant EXIF metadata and file modification times

SYNOPSIS

```
maki [GLOBAL FLAGS] fix-dates [QUERY] [OPTIONS]
```

DESCRIPTION

Scans assets and corrects their `created_at` date by examining the EXIF DateTimeOriginal metadata and file modification times of their variants. This fixes assets that were imported with the wrong date (e.g., import timestamp instead of capture date). Scope can be narrowed with a positional search query, `--asset`, or `--volume`.

For each asset, the command collects candidate dates from all variants:

1. **Stored EXIF date**: The `date_taken` field in the variant's `source_metadata` (stored since v1.3.1).
2. **Re-extracted EXIF**: For variants imported before `date_taken` was stored, re-reads the file on disk and extracts EXIF DateTimeOriginal. Requires the volume to be online.
3. **File modification time**: The filesystem mtime of the variant file on disk. Requires the volume to be online.

The oldest date found across all variants is used as the corrected `created_at`. A 1-second tolerance is applied when comparing to the current date (to handle rounding).

Without `--apply`, runs in **report-only mode** and shows what dates would change. With `--apply`, updates both YAML sidecar files and the SQLite catalog. When applying, also backfills `date_taken` into variant `source_metadata` so future runs work from metadata alone without needing the volume online.

Offline volume handling: Assets whose only file locations are on offline volumes cannot be fixed (no file access for EXIF re-extraction or mtime). These are counted separately as "skipped (volume offline)" and a warning is printed for each offline volume. Mount the volume and re-run to fix these assets.

OPTIONS

<QUERY> (positional, optional) Search query to scope which assets are checked (same syntax as `maki search`).

–**volume <LABEL>** Limit to assets with locations on a specific volume.

–**asset <ID>** Fix only a specific asset. Supports prefix matching.

–**apply** Apply date corrections. Without this flag, only reports what would change.

`--json` outputs a `FixDatesResult` with `checked`, `fixed`, `already_correct`, `no_date`, `skipped_offline` counters and `offline_volumes` list.

`--log` prints per-asset status to stderr.

`--time` shows elapsed wall-clock time.

EXAMPLES

Preview what dates would be corrected:

```
maki fix-dates
```

Preview with per-asset detail:

```
maki fix-dates --log
```

Apply date corrections:

```
maki fix-dates --apply --log
```

Fix dates for a specific volume:

```
maki fix-dates --volume "Photos 2024" --apply --log
```

Fix a single asset:

```
maki fix-dates --asset a1b2c3d4 --apply
```

SEE ALSO

fix-roles – fix variant roles in multi-variant assets. fix-recipes – re-attach recipe files imported as standalone assets. refresh – re-read metadata from changed recipe files. import – import now uses EXIF date → file mtime → current time fallback chain.

---

maki fix-recipes

NAME

maki-fix-recipes – re-attach recipe files that were imported as standalone assets

SYNOPSIS

```
maki [GLOBAL FLAGS] fix-recipes [QUERY] [OPTIONS]
```

DESCRIPTION

Finds assets that consist of a single variant with a recipe-type extension (xmp, cos, cot, cop, pp3, dop, on1) and `asset_type = other`, then tries to re-attach them as recipe records on a matching parent variant. Scope can be narrowed with a positional search query, `--asset`, or `--volume`.

This fixes a situation where recipe files were imported before their corresponding media files, or when a media format wasn't recognized at import time (e.g., NRW before extension support was added). In both cases, the recipe file ends up as a standalone asset instead of being attached to the media asset.

Matching logic: For each standalone recipe asset, the command extracts the filename stem and directory from the recipe's file location, then searches for a media variant with the same stem in the same directory on the same volume. Compound extensions are handled: if `DSC_001.NRW.xmp` doesn't match directly (stem = `DSC_001.NRW`), the last extension is stripped and `DSC_001` is tried as the stem.

When a parent is found and `--apply` is specified:

1. A `Recipe` record is created on the parent variant.
2. For XMP files, metadata (keywords, rating, description, color label) is extracted and applied to the parent asset.
3. The parent asset's denormalized columns are updated.
4. The standalone recipe asset is fully deleted (recipes, file locations, variants, asset row, and sidecar YAML).

Without `--apply`, runs in **report-only mode** and shows what would change.

OPTIONS

<QUERY> (positional, optional) Search query to scope which assets are checked (same syntax as `maki search`).

–**volume <LABEL>** Limit to assets with locations on a specific volume.

–**asset <ID>** Fix only a specific asset. Supports prefix matching.

–**apply** Apply the changes. Without this flag, only reports what would change.

`--json` outputs a `FixRecipesResult` with `checked`, `reattached`, `no_parent`, `skipped` counters.

`--log` prints per-asset status to stderr.

`--time` shows elapsed wall-clock time.

EXAMPLES

Preview what would be reattached:

```
maki fix-recipes
```

Preview with per-asset detail:

```
maki fix-recipes --log
```

Apply the fixes:

```
maki fix-recipes --apply --log
```

Fix a specific asset:

```
maki fix-recipes --asset a1b2c3d4 --apply
```

Fix recipes on a specific volume:

```
maki fix-recipes --volume "Photos" --apply --log --time
```

SEE ALSO

fix-roles – fix variant roles in multi-variant assets. fix-dates – fix asset dates from EXIF metadata. refresh – re-read metadata from changed recipe files. import – import with recipe attachment logic.

---

maki create-sidecars

NAME

maki-create-sidecars – create XMP sidecar files for assets with metadata but no existing recipe

SYNOPSIS

```
maki [GLOBAL FLAGS] create-sidecars [QUERY] [--volume <LABEL>] [--asset <ID>] [--apply]
```

DESCRIPTION

Scans assets for variants that have metadata (ratings, tags, labels, descriptions) but no XMP sidecar recipe, and creates `.xmp` files alongside the variant files on disk. Each generated sidecar includes `dc:subject` (flat tags), `lr:hierarchicalSubject` (hierarchical tags), `xmp:Rating`, `xmp:Label`, and `dc:description`.

This enables CaptureOne, Lightroom, and other XMP-aware tools to pick up MAKI metadata for files that were imported without XMP sidecars. Unlike `maki writeback` (which updates *existing* XMP files), this command *creates new* XMP files where none exist. Importing never creates sidecars; run this once before the first Capture One or Lightroom session on a freshly imported shoot, or set `[writeback] create_sidecars = true` to have the first edit create them automatically.

File naming follows `[writeback] sidecar_naming`: the default `stem` writes `DSC_1234.xmp` next to `DSC_1234.NEF` (the Capture One / Lightroom convention); `full` writes `DSC_1234.NEF.xmp` (darktable). Under `stem`, a RAW+JPEG pair in one asset gets a single shared sidecar.

Skipped locations. A variant that already has an `.xmp` recipe on that volume is skipped, as is a sidecar path already registered for the asset. A file that exists at the sidecar path but is not in the catalog (typically written by Capture One after the import) is never overwritten; it is counted separately and reported with a hint to run `maki import`. Only online volumes are considered.

Without `--apply`, runs in report-only mode showing what would be created.

OPTIONS

QUERY (optional) Search query to select assets. Same syntax as `maki search`.
–**volume <LABEL>** Only create sidecars for variant locations on this volume.
–**asset <ID>** Only create sidecars for a specific asset.
–**apply** Create XMP files on disk and register them as recipes. Without this flag, only reports what would be created.

OUTPUT

```
Create-sidecars: 120 checked, 118 created, 340 skipped (no metadata), 2 existing on disk but not imported (run `maki imp
```

`--log` prints one line per sidecar (`created`, `would create`, or `exists on disk but is not imported`). `--json` emits `dry_run`, `checked`, `created`, `exists_unregistered`, `skipped_no_metadata`, and the effective `sidecar_naming`.

EXAMPLES

Preview what sidecars would be created:

```
maki create-sidecars
```

Create sidecars for all assets on the working drive:

```
maki create-sidecars --volume "Work SSD" --apply --log
```

Create sidecars for rated images only:

```
maki create-sidecars "rating:1+" --apply --log
```

Create sidecars for a specific asset:

```
maki create-sidecars --asset a1b2c3d4 --apply
```

SEE ALSO

relocate – `--create-sidecars` flag creates sidecars as part of a relocate operation. writeback – updates *existing* XMP files with MAKI metadata (does not create new ones).

---

maki rebuild-catalog

NAME

maki-rebuild-catalog – rebuild the SQLite catalog from YAML sidecar files

SYNOPSIS

```
maki [GLOBAL FLAGS] rebuild-catalog [--asset <ID>]
```

DESCRIPTION

Rebuilds the SQLite catalog database from the YAML sidecar files. The sidecar files are the source of truth; the SQLite database is a derived cache.

Full rebuild (no flags): wipes all SQLite data rows and rebuilds from all sidecar files. Also restores collections, stacks, faces, people, and embeddings. For large catalogs (200k+ assets), this can take over an hour.

Per-asset rebuild (`--asset <ID>`): deletes and re-inserts a single asset's SQLite rows (variants, locations, recipes, embeddings, faces) from its sidecar. Takes seconds. Use this to fix sync mismatches on individual assets without a full rebuild.

OPTIONS

–**asset <ID>** Rebuild only a specific asset by ID or prefix. Deletes all SQLite rows for the asset and re-inserts from the sidecar YAML, including embeddings and faces.

EXAMPLES

Full rebuild:

```
maki rebuild-catalog
```

Rebuild a single asset:

```
maki rebuild-catalog --asset a1b2c3d4
```

Rebuild with timing:

```
maki rebuild-catalog --time
```

Watch the full rebuild progress sidecar by sidecar:

```
maki rebuild-catalog --log --time
```

With the global `--log` flag, the full rebuild prints one line per sidecar to stderr as it is inserted (`<asset-id> <name>`), which is the only way to see progress during a multi-hour rebuild. Without it, only the summary is printed when finished (or JSON with `--json`). `--log` has no effect on `--asset` rebuilds.

SEE ALSO

init – initial catalog creation. stats – verify catalog statistics after rebuild. verify – verify file integrity after rebuild.

---

maki migrate

NAME

maki-migrate – run database schema migrations

SYNOPSIS

```
maki [GLOBAL FLAGS] migrate
```

DESCRIPTION

Runs all pending database schema migrations. Migrations are idempotent (safe to run repeatedly) and add new columns, indexes, and tables needed by newer versions of maki.

A `schema_version` table tracks the current schema version. On startup, MAKI performs a single-query fast-check against this version number and skips migration logic entirely when the schema is already up to date, avoiding the overhead of running dozens of idempotent `ALTER TABLE` statements on every launch.

Migrations also run automatically once at program startup for all commands, so this command is primarily useful for:

- Explicit migration after a version upgrade
- Scripting and automation
- Verifying the schema is up to date

Rating repair: The migration includes a `MicrosoftPhoto:Rating` normalization pass. Windows Photo Gallery and some other tools write a percentage-based rating scale (1/13/25/50/75/99) into `Microsoft-Photo:Rating` instead of the standard `xmp:Rating` (1–5). The `normalize_rating()` function maps these values to the 1–5 scale. The migration fixes affected ratings in both the SQLite catalog and YAML sidecar files.

EXAMPLES

Run migrations:

```
maki migrate
```

Verify with JSON:

```
maki migrate --json
```

SEE ALSO

rebuild-catalog – full catalog rebuild from sidecar files.

Previous: Retrieve Commands – `search`, `show`, `preview`, `export`, `contact-sheet`, `duplicates`, `stats`, `backup-status`, `serve`, `shell`.

Search Filter Reference

Complete reference for all search filters available in `maki search`, the web UI browse page, and saved searches. Filters use a `key:value` syntax and can be mixed freely with free-text terms in a single query string.

All filters combine with AND – every filter must match for an asset to appear in results.

Negation and OR Syntax

Negation (Excluding Results)

Prefix any filter or free-text term with `-` to exclude matches:

```
maki search "-tag:rejected"              # exclude assets with the "rejected" tag
maki search "-format:xmp"                # exclude XMP files
maki search "-type:other"                # exclude "other" type assets
maki search "-label:Red"                 # exclude Red labeled assets
maki search "-camera:phone"              # exclude phone camera shots
maki search "landscape -tag:processed"   # landscapes that aren't tagged processed
maki search "-sunset"                    # exclude free-text match on "sunset"
```

Negation is supported for free-text terms and for the filters that have an exclude form: `tag:`, `type:`, `format:`, `label:`, `volume:`, `camera:`, `lens:`, `description:`/`desc:`, `path:`, `collection:`, and `person:`. A leading `-` on any other filter (`-rating:3`, `-date:2026`, `-iso:800`, `-id:`, `-similar:`, …) is ignored and the filter applies as if it were positive.

The special `none` forms negate too: `-label:none` matches assets that *have* a color label (any color), and `-volume:none` matches assets with at least one file location on a currently online volume – the complement of `volume:none`.

OR Within a Filter (Comma Operator)

Use commas within a single filter value to match any of the given options (OR logic). This works for the list-valued filters (`tag:`, `type:`, `format:`, `label:`, `camera:`, `lens:`, `description:`/`desc:`, `path:`, `volume:`, `collection:`, `person:`) and for every numeric filter (`rating:`, `iso:`, `focal:`, `f:`, `width:`, `height:`, `copies:`, `variants:`, `tagcount:`, `scattered:`, `duration:`, `bpm:`, `faces:N`). It does **not** apply to `id:`, `codec:`, `key:`, `date:`/`dateFrom:`/`dateUntil:`, or `meta:` – there the comma is part of the value. (`id:` can be repeated instead: `id:abc id:def` matches either prefix.)

```
maki search "tag:alice,bob"              # assets tagged alice OR bob (or both)
maki search "format:nef,cr3"             # NEF or CR3 format
maki search "type:image,video"           # images or videos
maki search "label:Red,Orange"           # Red or Orange labeled assets
maki search "rating:4,5"                 # 4-star or 5-star assets
```

The comma operator works within a single filter value. To require multiple tags, repeat the filter:

```
maki search "tag:landscape tag:sunset"    # assets with BOTH landscape AND sunset tags
```

Repeating a filter means AND only for `tag:` and `meta:`. Every other repeatable filter flattens its repeated values into one OR list, exactly as if they had been written with commas: `type:image type:video` is the same as `type:image,video`, `camera:sony camera:nikon` matches either camera, `format:nef format:cr3` matches either format. The same holds for `label:`, `lens:`, `description:`, `path:`, `volume:`, and `collection:`. `person:` is the one filter whose repeat semantics differ between front ends — see person.

Combining Negation and OR

Negation and OR can be combined in complex queries:

```
maki search "type:image,video -format:xmp"              # images or videos, but not XMP files
maki search "tag:landscape,portrait -label:Red"       # landscape or portrait, excluding Red labeled
maki search "format:nef,cr3 -tag:rejected rating:3+"   # RAW files (NEF or CR3), not rejected, 3+ stars
```

Free Text

Syntax: any token that does not match a recognized `prefix:value` pattern; `-<word>` excludes a term

Description: Remaining tokens after all prefix filters are extracted are joined into a single free-text string. This text is matched against asset names, original filenames, descriptions, and source metadata values (partial, case-insensitive). A token prefixed with `-` is an exclusion instead: assets matching that term anywhere in the same columns are dropped from the result. Each excluded term is applied separately (`-draft -test` excludes both), while the positive terms are matched as one string.

Examples:

```
maki search "sunset"
maki search "golden hour mountains"
maki search "sunset tag:landscape"       # "sunset" is free text, tag:landscape is a filter
maki search "sunset -draft"              # "sunset" but not anything matching "draft"
```

SQL behavior: Terms of three or more characters go through the `assets_fts` FTS5 index (trigram tokenizer, schema v9): `WHERE a.id IN (SELECT asset_id FROM assets_fts WHERE assets_fts MATCH '"text"')` — an index-accelerated, case-insensitive substring match over name, filename, description, and source metadata. The term is always quoted as an FTS string literal, so it is never interpreted as FTS query syntax. Terms shorter than three characters fall back to `(a.name LIKE '%text%' OR bv.original_filename LIKE '%text%' OR a.description LIKE '%text%' OR bv.source_metadata LIKE '%text%')` against the best-variant columns. Negated free text (`-term`) uses the same two paths wrapped in `NOT IN` / `NOT (...)`.

---

type

Syntax: `type:<value>`

Values: `image`, `video`, `audio`, `document`, `other`

Description: Filters by asset type. Exact match, case-insensitive (`type:Image` and `type:image` are the same).

Examples:

```
maki search "type:image"
maki search "type:video rating:3+"
```

SQL behavior: `WHERE a.asset_type = ? COLLATE NOCASE` (comma lists become `COLLATE NOCASE IN (...)`, negation `(a.asset_type IS NULL OR a.asset_type != ? COLLATE NOCASE)`). Pure assets-table filter, no JOIN required.

---

tag

Syntax: `tag:<name>`, `tag:"<multi-word name>"`, `tag:=<name>` (whole path), `tag:/<name>` (leaf only), `tag:^<name>` (case-sensitive), or `tag:|<name>` (prefix anchor)

Description: Filters to assets that have a specific tag. Supports quoted values for multi-word tags.

Hierarchical matching: Tags can be organized hierarchically using `|` as a separator (e.g., `animal|bird|eagle`), aligned with Lightroom and CaptureOne conventions. `>` is also accepted as an alternative separator. Searching for a tag name matches it at any level in the hierarchy: `tag:eagle` finds assets tagged `animal|bird|eagle`, `tag:bird` finds both `animal|bird` and `animal|bird|eagle`. Parent-to-child matching also works: `tag:animal` finds `animal`, `animal|bird`, and `animal|bird|eagle`. Only exact component names match — `tag:eag` does NOT match `eagle`.

Whole-path match (`=`): Prefix with `=` to match the full tag value **exactly** — no level sliding, no descendants, no leaf variants. Use when a root-level tag shares a name with a leaf elsewhere in the hierarchy and you need to disambiguate.

Concrete example: if your catalog has three tags `Legoland`, `location|Denmark|Legoland`, and `location|Germany|Legoland`, `tag:Legoland` matches all three and `tag:/Legoland` also matches all three (each is a leaf in its branch). Only `tag:=Legoland` isolates the root-level one. Works at any depth: `tag:=location|Denmark|Legoland` matches exactly that path and nothing else. The `=` reads as "equals" — matches the user's mental model of exact equality.

Leaf-only match (`/`): Prefix with `/` to match assets tagged at any hierarchy level but only where the tag is a **leaf** (nothing deeper in the same branch). `tag:/location|Germany|Bayern` matches assets whose deepest tag in this branch is `Bayern` — NOT assets that also have `location|Germany|Bayern|München`. Niche use case for distinguishing "this is the deepest tagged level" from "this is a parent." For most disambiguation needs, `=` is what you want.

In the web UI, click the mode indicator on a tag chip to cycle through the three modes (▾ default, `=` whole path, `/` leaf only).

Case-sensitive match (`^`): Tag matching is case-insensitive by default (`tag:landscape` matches both `landscape` and `Landscape`). Prefix with `^` to make the match case-sensitive — useful when cleaning up tag duplicates like `landscape` vs `Landscape`, which the tags page shows as distinct entries. In the web UI, each tag chip has a small `cc`/`Cc` toggle: click it to flip that specific chip between case-insensitive (`cc`) and case-sensitive (`Cc`) matching.

Negation per chip (`-`): In the web UI, each tag chip has a `+` / `−` toggle on its left that flips the chip between include (default) and exclude. A negated chip routes into the search's NOT clause — assets carrying that tag are hidden, regardless of the other chips' modes or case settings. The chip is visually marked (red tint, strikethrough on the label) so the polarity is obvious at a glance. This is the chip-UI equivalent of typing `-tag:foo` in the query field, and works alongside the mode and case toggles on the same chip (e.g. a chip can be "exclude `cat` at any depth, case-sensitive" with all three toggles active).

Prefix anchor (`|`): Prefix the value with `|` to match any hierarchy component that **starts with** the rest of the value, at any level (root or descendant). `tag:|wed` matches assets tagged `wedding`, `wedding-2024`, `events|wedding`, `events|wedding|2024-05-12`, etc. — anything where the letter combination "wed" begins a tag component. This is the search-side counterpart of the `|xyz` autocomplete syntax in the web UI tag

filter and tags page. Useful for finding tag families with shared prefixes (e.g. all date-stamped tags `2024-*`, all event tags `wedding-*`). The `|` marker implicitly includes descendants (the `=` and `/` markers are silently ignored when `|` is present, since they conflict).

The `=`, `/`, `^`, and `|` prefixes can be combined in any order: `tag:=^Foo`, `tag:/^Foo`, `tag:^|Wed`, etc. `=` and `/` together **combine**: `tag:=/location|Germany` matches the exact path `location|Germany` *and* requires that the asset carries no descendant of it — this is the form the tags page uses for its "(N as leaf)" link. `|` paired with `=` or `/` resolves to `|` (the prefix anchor wins, the other markers are dropped).

`/` and `=` when positioned after another modifier or inside the name are treated as literal characters in tag names (e.g., `f/1.4` works naturally without escaping). Only a **leading** marker is interpreted as a prefix.

> **Note:** prior to v4.4.4 the `=` and `/` markers were swapped — `=` meant leaf-only-at-any-level and `/` meant whole-path. The swap was made because `=` reads naturally as "equals" / exact value match, which is what most users expect. If you have saved searches or scripts using the old semantics, swap `=` and `/` in the tag values to preserve the previous behavior.

Examples:

```
maki search "tag:landscape"
maki search 'tag:"Fools Theater"'
maki search 'tag:"Black and White" rating:4+'
maki search "tag:animal"                       # matches animal, animal|bird, animal|bird|eagle
maki search "tag:eagle"                       # matches animal|bird|eagle (any level in hierarchy)
maki search "tag:animal|bird"                  # matches animal|bird and animal|bird|eagle
maki search "tag:=Legoland"                    # whole path: only the standalone Legoland
maki search 'tag:="location|Denmark|Legoland"' # only that exact path
maki search "tag:/Holzkirchen"                 # leaf only: Holzkirchen with no deeper child
maki search "tag:^Landscape"               # case-sensitive: matches "Landscape" but NOT "landscape"
maki search "tag:=^Legoland"                   # whole path AND case-sensitive
maki search "tag:/^Animals"                    # leaf only AND case-sensitive
maki search "tag:|wed"                     # prefix anchor: wedding, wedding-2024, events|wedding, ...
maki search "tag:^|Wed"                       # case-sensitive prefix anchor: Wedding but not wedding
```

SQL behavior: Each `tag:` entry is served in one of two ways with identical semantics. If a bounded probe shows the entry matches fewer than 10,000 assets, its patterns run against `assets_tags_fts`, an FTS5 trigram copy of the `tags` column (schema v12) that evaluates ordinary `LIKE`/`GLOB` from its index: `WHERE a.id IN (SELECT asset_id FROM assets_tags_fts WHERE tags LIKE … OR …)` (`NOT IN` for negation). Broader entries keep the inline scan `WHERE (a.tags LIKE … OR …)`, which the planner can stop early under `ORDER BY … LIMIT` and which is not slower than materialising tens of thousands of ids. The decision is memoised per connection. Four LIKE patterns match the tag at any hierarchy level: `%"stored"%` (standalone), `%"stored|%` (parent from root), `%|stored"%` (leaf child), `%|stored|%` (mid-path). The JSON quoting provides natural word boundaries — `tag:eagle` cannot match `eagles`. With `=` prefix: a single `LIKE '%"stored"%'` — whole-path equality, bounded by the JSON quotes; with `=/` combined, that clause is ANDed with `NOT LIKE '%"stored|%' AND NOT LIKE '%|stored|%'` (no descendants). With `/` prefix: `WHERE (LIKE '%"stored"%' OR LIKE '%|stored"%') AND NOT LIKE '%"stored|%' AND NOT LIKE '%|stored|%'` — matches the tag at any level but only as a leaf (no descendants). With `^` prefix: SQLite `GLOB` is used instead of `LIKE` — GLOB is case-sensitive and uses `*` as the wildcard (literal `*` and `?` in tag names are an unsupported edge case for case-sensitive search).

---

format

Syntax: `format:<extension>`

Description: Filters by file format (extension) of any variant on the asset. Case-insensitive match against the variant's format field.

Examples:

```
maki search "format:nef"
maki search "format:jpg"
maki search "format:mp4 type:video"
maki search "format:tif tag:processed"
```

SQL behavior: `WHERE v.format = ?`. Triggers a JOIN to the variants table.

---

rating

Syntax: `rating:<N>` | `rating:<N>+` | `rating:<N>-<M>` | `rating:<N>,<M>` | `rating:<N>,<M>+`

Values: 0 through 5 (0 = unrated)

Description: Filters by the asset's star rating (1–5 scale, 0 = unrated). Supports the full numeric filter syntax:

Syntax	Meaning	Example
`N`	exactly N	`rating:5`
`N+`	N or more	`rating:3+`
`N-M`	from N to M	`rating:3-5`
`N,M`	exactly N OR M	`rating:2,4`
`N,M+`	exactly N OR M+	`rating:2,4+`

Ratings are extracted from XMP during import. Microsoft Photo ratings (percentage values 1–99) are automatically normalized to the 1–5 scale (1=1, 25=2, 50=3, 75=4, 99=5).

Unrated assets: Assets without a rating are stored with `rating = NULL` in the catalog. `rating:0` (and any filter that includes 0 in its range or value list) matches both NULL-rated assets and assets with an explicit rating of 0 — users mentally treat "unrated" and "0 stars" as the same thing. Conversely, `rating:1+` and any filter that excludes 0 also excludes unrated assets.

Web UI: Click the small ∅ marker to the left of the stars to filter for unrated assets (equivalent to `rating:0`). Click it again to clear the filter. Click any star to set the minimum rating as before.

Examples:

```
maki search "rating:5"          # exactly 5 stars
maki search "rating:3+"         # 3 stars or more (excludes unrated and 0)
maki search "rating:3-5"        # 3, 4, or 5 stars
maki search "rating:2,4"        # exactly 2 or 4 stars
maki search "rating:2,4+"       # exactly 2, or 4 stars and above
maki search "rating:0"          # unrated OR 0-star assets ("the rest")
```

SQL behavior: - Exact (non-zero): `WHERE a.rating = ?` - Exact 0: `WHERE (a.rating IS NULL OR a.rating = 0)` - Minimum (> 0): `WHERE a.rating >= ?` - Minimum 0 or less: `WHERE (a.rating IS NULL OR a.rating >= ?)` - Range including 0: `WHERE (a.rating IS NULL OR (a.rating >= ? AND a.rating <= ?))` - Values including 0: `WHERE (a.rating IS NULL OR a.rating IN (...))`

Rating filters use a dedicated `rating_clause` helper that detects whether the filter matches 0 and wraps the standard numeric clause in `(a.rating IS NULL OR ...)` when it does. Filters that don't match 0 (e.g., `rating:3+`, `rating:2-4`) use the standard numeric comparison and correctly exclude NULL.

Pure assets-table filter, no JOIN required.

---

label

Syntax: `label:<color>`, `label:none`, `-label:none`

Values: `Red`, `Orange`, `Yellow`, `Green`, `Blue`, `Pink`, `Purple` (case-insensitive input, stored as title-case), `none` (unlabeled); `-label:none` = any label

Description: Filters by color label. The 7-color set is a superset of Lightroom's 5 colors, matching Capture-One's label palette. Use `label:none` to find assets without any color label, and `-label:none` to find assets that carry any label at all (the complement). Individual colors can be excluded with `-label:Red`.

Examples:

```
maki search "label:Red"
maki search "label:blue"
maki search "label:none"
maki search "-label:none"                  # any color label set
maki search "label:Green rating:4+"
```

SQL behavior: `WHERE a.color_label = ? COLLATE NOCASE` for color values; `WHERE a.color_label IS NULL` for `label:none`, `WHERE a.color_label IS NOT NULL` for `-label:none`. Pure assets-table filter, no JOIN required. The `COLLATE NOCASE` makes the comparison case-insensitive so users can type `label:red`, `label:Red`, or `label:RED` interchangeably regardless of how the label is stored in the catalog.

---

camera

Syntax: `camera:<text>` or `camera:"<multi-word text>"`

Description: Partial match against the camera model of any variant. Case-insensitive substring match. Comma-separated values and repeated `camera:` entries are both OR.

Examples:

```
maki search "camera:fuji"
maki search 'camera:"Canon EOS R5"'
maki search 'camera:"NIKON Z 9" rating:4+'
```

SQL behavior: `WHERE v.camera_model LIKE '%value%'` on the typed `camera_model` column of the variants table (populated at import from EXIF). Negation: `(v.camera_model IS NULL OR v.camera_model NOT LIKE '%value%')`. Triggers a JOIN to the variants table.

---

lens

Syntax: `lens:<text>` or `lens:"<multi-word text>"`

Description: Partial match against the lens model of any variant. Case-insensitive substring match. Comma-separated values and repeated `lens:` entries are both OR.

Examples:

```
maki search "lens:56mm"
maki search 'lens:"RF 50mm f/1.2"'
maki search 'lens:"24-70" camera:sony'
```

SQL behavior: `WHERE v.lens_model LIKE '%value%'` on the typed `lens_model` column of the variants table. Triggers a JOIN to the variants table.

---

description

Syntax: `description:<text>`, `description:"<multi-word text>"`, or `desc:<text>` (short alias)

Description: Case-insensitive substring match against the asset's `description` field. Useful for finding assets by what's *in* them when you've added descriptions manually or via VLM auto-describe (Pro). Unlike free-text search, this filter targets ONLY the description column — it does not match against name, filename, or source metadata.

The short alias `desc:` is equivalent to `description:` and exists for convenience in interactive use. Both negation (`-description:`, `-desc:`) and OR (`description:foo,bar`) work the same as other text filters. Assets with NULL description are correctly excluded by negation.

Examples:

```
maki search "description:sunset"                    # any asset whose description mentions 'sunset'
maki search 'description:"colorful flowers"'        # multi-word match
maki search "desc:cat -desc:rejected"               # cats but not rejected
maki search "description:wedding rating:5+"         # 5-star photos described as wedding
maki search "description:sunset,dawn,dusk"          # any of three terms (OR)
```

Comparison with free-text: A bare term like `maki search "sunset"` matches the asset's name, filename, description, AND source metadata at once — convenient for broad lookups but noisy when you specifically want description hits. Use `description:sunset` to scope the match.

SQL behavior: `WHERE a.description LIKE '%value%'`. Pure assets-table filter, no JOIN required. Comma-separated values and repeated `description:` entries are both OR-combined (`description:cat description:dog` matches either); each negated entry adds `(a.description IS NULL OR a.description NOT LIKE '%value%')`.

---

iso

Syntax: `iso:<N>` (exact) | `iso:<N>+` (minimum) | `iso:<min>-<max>` (range) | `iso:<N>,<M>` (OR list) | `iso:<N>,<M>+`

Description: Filters by ISO sensitivity. Supports the full numeric filter syntax (see rating for the table).

Examples:

```
maki search "iso:3200"          # exactly ISO 3200
maki search "iso:3200+"         # ISO 3200 or higher
maki search "iso:100-800"       # ISO between 100 and 800 inclusive
```

SQL behavior: Numeric comparison on the typed `v.iso` column of the variants table (`= ?`, `>= ?`, `BETWEEN`, `IN (...)`), populated at import from EXIF. Triggers a JOIN to the variants table.

---

focal

Syntax: `focal:<N>` (exact) | `focal:<N>+` (minimum) | `focal:<min>-<max>` (range) | `focal:<N>,<M>`

Description: Filters by focal length in millimeters. Full numeric filter syntax.

Examples:

```
maki search "focal:50"          # exactly 50mm
maki search "focal:200+"        # 200mm or longer
maki search "focal:35-70"       # between 35mm and 70mm inclusive
```

SQL behavior: Same pattern as `iso` on the typed `v.focal_length_mm` column. Triggers a JOIN to the variants table.

---

f (aperture)

Syntax: `f:<N>` (exact) | `f:<N>+` (minimum) | `f:<min>-<max>` (range) | `f:<N>,<M>`

Description: Filters by f-number (aperture). Supports decimal values and the full numeric filter syntax.

Examples:

```
maki search "f:2.8"             # exactly f/2.8
maki search "f:1.4+"           # f/1.4 or wider (numerically >= 1.4)
maki search "f:1.4-2.8"        # between f/1.4 and f/2.8 inclusive
```

SQL behavior: Same pattern as `iso` on the typed `v.f_number` column (floating-point). Triggers a JOIN to the variants table.

---

width

Syntax: `width:<N>` (exact) | `width:<N>+` (minimum) | `width:<min>-<max>` (range) | `width:<N>,<M>`

Description: Filters by image width in pixels. A bare number is an **exact** match — use `+` for "at least".

Examples:

```
maki search "width:4000+"       # 4000 pixels wide or more
maki search "width:3840"        # exactly 3840 pixels (4K UHD width)
maki search "width:4000+ height:2000+"
```

SQL behavior: Numeric comparison on the typed `v.image_width` column (`= ?` for a bare number, `>= ?` with `+`, `BETWEEN` for a range). Triggers a JOIN to the variants table.

---

height

Syntax: `height:<N>` (exact) | `height:<N>+` (minimum) | `height:<min>-<max>` (range) | `height:<N>,<M>`

Description: Filters by image height in pixels. A bare number is an exact match.

Examples:

```
maki search "height:2000+"      # 2000 pixels tall or more
maki search "height:2160"       # exactly 2160 pixels (4K UHD height)
```

SQL behavior: Same pattern as `width` on the typed `v.image_height` column. Triggers a JOIN to the variants table.

---

meta

Syntax: `meta:<key>=<value>`

Description: Generic key-value match against variant source metadata. The key selects a field in the `source_metadata` JSON blob, the value is a case-insensitive substring match on that field. Useful for filtering on metadata fields that don't have a dedicated prefix filter. A `meta:` token without `=` is ignored.

Examples:

```
maki search "meta:software=CaptureOne"
maki search "meta:camera_make=SONY"
maki search "meta:color_space=sRGB"
```

SQL behavior: `WHERE json_extract(v.source_metadata, '$.<key>') LIKE '%value%'`. Triggers a JOIN to the variants table. Multiple `meta:` filters can be specified and all must match (AND). Negation is not supported.

---

path

Syntax: `path:<pattern>` or `path:"<pattern with spaces>"`

Description: Restricts results to assets that have at least one file location whose volume-relative path matches the given pattern. Without wildcards, this is a fast prefix match. The * character is a wildcard that matches any sequence of characters (including slashes).

Wildcards:

Pattern	Meaning	Speed
`path:Pictures/2026`	Prefix from root: matches `Pictures/2026...`	fast (index scan)
`path:Pictures/*/Capture`	Anything between `Pictures/` and `/Capture`	fast (left-anchored)
`path:*/2026/*/party`	Slash-anchored substring search	slow (full scan)
`path:*party`	Substring match anywhere	slow (full scan)

A trailing * is implicit — `path:Pictures/2026` and `path:Pictures/2026*` are equivalent.

Performance note: Patterns with no leading * use the SQLite index on `relative_path` and are fast even on large catalogs. Patterns starting with * force a full table scan and are noticeably slower on catalogs with hundreds of thousands of assets.

Automatic normalization (CLI only, not in web UI or saved searches):

Input	Behavior
`path:Capture/2026`	Used as-is (volume-relative prefix match)

Input	Behavior
`path:~/Photos/2026`	~ expanded to `$HOME`, then matched against volumes
`path:./subdir`	Resolved relative to current working directory
`path:../other`	Resolved relative to current working directory
`path:/Volumes/Photos/Capture/2026`	Stripped to `Capture/2026` with implicit `volume:` filter for the Photos volume

Patterns containing * skip the absolute-path / volume normalization (they pass through unchanged). An explicit `volume:` filter in the same query takes precedence over the auto-detected volume from path normalization.

Examples:

```
maki search "path:Capture/2026-02-22"
maki search 'path:"Photos/Family Trip"'
maki search "path:Capture/2026 rating:3+ tag:landscape"
maki search "path:/Volumes/Photos/Capture/2026"     # auto-normalized
maki search "path:*/2026/*/wedding"                  # find any wedding shoot in 2026
maki search "path:*party"                            # any path containing "party"
```

SQL behavior: `WHERE fl.relative_path LIKE 'pattern%' ESCAPE '\'`. The user's * is translated to SQL `%`; literal `%` and `_` in the user input are escaped. Triggers a JOIN to the file_locations table.

---

date

Syntax: `date:<prefix>`

Values: `YYYY-MM-DD` (day), `YYYY-MM` (month), or `YYYY` (year)

Description: Prefix match on the asset's creation date. Matches any asset whose `created_at` timestamp starts with the given prefix. This gives you day, month, or year granularity in a single filter.

Examples:

```
maki search "date:2026-02-25"       # assets created on Feb 25, 2026
maki search "date:2026-02"          # assets created in February 2026
maki search "date:2026"             # assets created in 2026
maki search "date:2026-02 tag:landscape"
```

SQL behavior: `WHERE a.created_at LIKE 'prefix%'`. Pure assets-table filter, no JOIN required. Uses the index on `assets.created_at`.

---

dateFrom

Syntax: `dateFrom:<date>`

Values: `YYYY-MM-DD` (or any date prefix)

Description: Inclusive lower bound on the asset's creation date. Commonly used with `dateUntil:` to define a date range.

Examples:

```
maki search "dateFrom:2026-01-01"                          # from Jan 1, 2026 onward
maki search "dateFrom:2026-01-01 dateUntil:2026-03-31"   # Q1 2026
maki search "dateFrom:2025-06-01 rating:4+"
```

SQL behavior: `WHERE a.created_at >= ?`. String comparison is correct for RFC 3339 timestamps. Pure assets-table filter, no JOIN required.

---

dateUntil

Syntax: `dateUntil:<date>`

Values: `YYYY-MM-DD`, `YYYY-MM`, or `YYYY`

Description: Inclusive upper bound on the asset's creation date. Internally converted to an exclusive next-day/month/year boundary for correct range semantics.

Examples:

```
maki search "dateUntil:2026-02-28"                         # up to and including Feb 28
maki search "dateFrom:2026-01-01 dateUntil:2026-12-31"   # full year 2026
maki search "dateUntil:2025-12-31 type:video"              # all videos before 2026
```

SQL behavior: Converts the inclusive bound to exclusive: `"2026-02-28"` → `a.created_at < "2026-03-01"`. Pure assets-table filter, no JOIN required.

---

collection

Syntax: `collection:<name>` or `collection:"<multi-word name>"`

Description: Restricts results to assets that belong to a specific static collection (album). The collection's member asset IDs are pre-loaded and passed as a filter set.

Examples:

```
maki search "collection:Favorites"
maki search 'collection:"My Favorites"'
maki search 'collection:"Travel 2026" rating:4+'
```

SQL behavior: Pre-computes the set of asset IDs in the named collection(s), then filters with `WHERE a.id IN (...)`; `-collection:` uses `NOT IN`. Comma-separated names and repeated `collection:` entries are both OR (the ID sets are unioned). An unknown collection name matches nothing.

---

volume

Syntax: `volume:<label>`, `volume:<label1>,<label2>`, `-volume:<label>`, `volume:none`, `-volume:none`

Description: Restricts results to assets that have at least one file location on the specified volume. Volume labels are matched case-insensitively. Supports negation (`-volume:Archive`) and comma-separated OR (`volume:Photos,Working`). The special value `volume:none` finds assets with no file locations on any currently online volume; its negation `-volume:none` finds the complement – assets with at least one file location on an online volume.

Examples:

```
maki search "volume:Photos"                  # assets on the Photos volume
maki search "volume:ScreenSaver type:image"  # images on the ScreenSaver volume
maki search "volume:Photos,Working"          # assets on either volume
maki search "-volume:Archive"                # exclude assets on the Archive volume
maki search "volume:none"                    # assets not on any online volume
maki search "volume:none orphan:false"       # has locations, but all on offline volumes
maki search "-volume:none"                  # reachable now: at least one file on an online volume
maki search "volume:\"External SSD\""        # volume label with spaces (quoted)
```

In the web UI, the volume filter is also available as a dropdown control that passes the volume UUID directly to the search backend.

SQL behavior: - `volume:<label>`: Label is resolved to a volume UUID via DeviceRegistry. `WHERE fl.volume_id IN (...)`. Triggers a JOIN to the file_locations table. - `-volume:<label>`: Resolved to UUID, then `WHERE a.id NOT IN (SELECT DISTINCT v2.asset_id FROM variants v2 JOIN file_locations fl2 ON fl2.content_hash = v2.content_hash WHERE fl2.volume_id IN (...))` — excludes assets with *any* file on those volumes. - `volume:none`: Pre-computes online volume IDs from DeviceRegistry. Adds `WHERE NOT EXISTS (SELECT 1 FROM file_locations fl JOIN variants v ... WHERE fl.volume_id IN (...online_ids...))`. - `-volume:none`: Same online-ID list, `WHERE EXISTS (...)` instead of `NOT EXISTS`. When no volume is online the clause is a constant false, so the result is empty.

---

id

Syntax: `id:<prefix>`

Description: Matches assets whose UUID starts with the given prefix. Useful for quickly finding a specific asset by its ID (or the beginning of it) in both CLI and web UI.

Examples:

```
maki search "id:c654e"
maki search "id:c654efa4-4e55"
```

SQL behavior: `WHERE a.id LIKE 'prefix%'`. Pure SQL prefix match.

---

orphan

Syntax: `orphan:true` | `orphan:false`

Description: Filters by whether an asset has any file location records.

- `orphan:true` — assets with zero file locations (files removed but metadata remains)
- `orphan:false` — assets with at least one file location (has files on disk)

Examples:

```
maki search "orphan:true"                  # assets with no files on disk
maki search "orphan:false type:image"      # images that have files
maki search "volume:none orphan:false"     # has files, but not on any online volume
```

SQL behavior: `orphan:true` uses `NOT EXISTS (...)`, `orphan:false` uses `EXISTS (...)` on the file_locations/variants subquery. Pure SQL, no disk I/O.

---

missing

Syntax: `missing:true`

Description: Finds assets where at least one file location points to a file that no longer exists on disk (on an online volume). This requires checking the filesystem and may be slow on large catalogs.

Examples:

```
maki search "missing:true"
maki search "missing:true type:video"
```

SQL behavior: Pre-computes affected asset IDs by iterating all file locations on online volumes and checking `Path::exists()`. The resulting ID set is passed as `WHERE a.id IN (...)`. Offline volumes are skipped.

---

stale

Syntax: `stale:<N>` or `stale:never`

Values: Number of days (0 or more), or the word `never`. The other numeric forms (`N+`, `A-B`, `A,B`) are accepted, but only the first (or lower) number is used as the day count.

Description: `stale:N` finds assets with at least one file location that has not been verified within the last N days, or that has never been verified. `stale:never` finds only the never-verified ones – locations whose `verified_at` is unset because no `maki verify` run has covered them yet. Both are useful for identifying files due for integrity checks.

`stale:0` is not a shortcut for `stale:never`: the cutoff is `now - 0 days`, so it matches every location verified before the current moment – in practice, everything. Use `stale:never` for never-verified files. The day comparison is exact to the second, so a location verified earlier today is not reported by `stale:1`.

Examples:

```
maki search "stale:30"          # not verified in 30+ days (or never)
maki search "stale:90 type:image"
maki search "stale:never"       # never verified at all
```

SQL behavior: `stale:N` adds `WHERE EXISTS (SELECT 1 FROM file_locations fl JOIN variants v ... WHERE (fl.verified_at IS NULL OR datetime(substr(fl.verified_at, 1, 19)) < datetime('now', '-N days')))` – `verified_at` is stored as an RFC 3339 UTC string, and the `substr`/`datetime` pair compares parsed timestamps rather than raw strings. `stale:never` adds the same `EXISTS` with `fl.verified_at IS NULL` alone. Pure SQL, no disk I/O.

---

stacked

Syntax: `stacked:true` or `stacked:false`

Description: Filters by whether an asset belongs to a stack. `stacked:true` finds assets that are members of a stack; `stacked:false` finds assets that are not in any stack.

Examples:

```
maki search "stacked:true"                  # all stacked assets
maki search "stacked:false rating:5"        # 5-star assets not in a stack
maki search "stacked:true type:image"       # stacked images
```

SQL behavior: - `stacked:true`: `WHERE a.stack_id IS NOT NULL` - `stacked:false`: `WHERE a.stack_id IS NULL`

Pure assets-table filter, no JOIN required.

---

copies

Syntax: `copies:<N>` (exact) | `copies:<N>+` (minimum) | `copies:<A>-<B>` (range) | `copies:<A>,<B>` (OR list)

Values: Non-negative integer

Description: Counts the number of **distinct volumes** where any file of this asset exists. An asset with a RAW file on volume Media and a backup on volume MediaBackup has `copies:2` — two volumes, regardless of how many individual files (variants, recipes) exist on each. This matches the backup-status page semantics and gives a meaningful answer to "how many drives would I have to lose before this asset is gone?"

Common patterns: - `copies:1` — assets on only one volume (at risk — matches the backup-status "AT RISK" count) - `copies:2+` — assets on at least two volumes (backed up) - `copies:0` — equivalent to `orphan:true` (no file locations at all)

Examples:

```
maki search "copies:1"              # at-risk assets (single volume)
maki search "copies:2"              # on exactly 2 volumes
maki search "copies:3+"             # on 3 or more volumes
maki search "copies:1 rating:4+"    # highly-rated assets with no backup
maki search "copies:2+ type:video"  # backed-up videos
```

SQL behavior: Scalar subquery: `(SELECT COUNT(DISTINCT fl2.volume_id) FROM file_locations fl2 JOIN variants v2 ON fl2.content_hash = v2.content_hash WHERE v2.asset_id = a.id)`. Self-contained, no outer JOIN flags needed.

---

variants

Syntax: `variants:<N>` (exact) | `variants:<N>+` (minimum) | `variants:<A>-<B>` (range) | `variants:<A>,<B>` (OR list)

Values: Non-negative integer

Description: Filters by the number of variants belonging to an asset. This counts distinct content-addressed files (originals, alternates, exports, processed) grouped under one asset. Useful for auditing mis-grouped assets (too many variants) or finding single-file assets.

Common patterns: - `variants:1` — assets with exactly one variant (single file) - `variants:3+` — assets with 3 or more variants (potentially mis-grouped) - `variants:5+` — suspiciously large groups worth manual review

Examples:

```
maki search "variants:1"              # single-variant assets
maki search "variants:3+"             # assets with 3+ variants
maki search "variants:5+ type:image"  # images with many variants
```

SQL behavior: Direct filter on the denormalized `a.variant_count` column. No JOIN required.

---

tagcount

Syntax: `tagcount:<N>` (exact) / `tagcount:<N>+` (minimum) / `tagcount:<A>-<B>` (range) / `tagcount:<A>,<B>` (OR list)

Values: Non-negative integer

Description: Filters by the number of **leaf tags** on an asset — the tags you intentionally applied, *not* including ancestor paths that MAKI auto-stores when you use hierarchical tags. An asset tagged with `subject|nature|landscape` has 3 stored tags (`subject`, `subject|nature`, `subject|nature|landscape`) but only 1 leaf. `tagcount:` uses the leaf count because that matches what you actually chose to apply.

A tag is a leaf when no other tag on the same asset starts with it plus `|`.

Common patterns:

- `tagcount:0` — **untagged assets** (no leaf = no intentional tags)
- `tagcount:1` — single-tag assets (often candidates for richer tagging)
- `tagcount:5+` — heavily tagged assets (audit for noise tags)
- `tagcount:10+` — suspiciously heavy, worth reviewing
- `tagcount:2-4` — lightly tagged (might want to expand)

Useful during tag restructuring: combine with other filters to narrow. `tagcount:0 type:image` finds untagged images specifically; `tagcount:1 date:2024` shows last year's under-tagged imports.

Examples:

```
maki search "tagcount:0"                    # completely untagged
maki search "tagcount:1"                    # single-tag assets
maki search "tagcount:5+"                   # heavily tagged
maki search "tagcount:0 rating:4+"          # untagged keepers (surprising)
maki search "tagcount:1 tag:sunset"         # assets tagged ONLY with sunset
```

SQL behavior: Direct filter on the denormalized `a.leaf_tag_count` column, populated at write time (schema v8). No JSON scan or correlated subquery — scales to very large catalogues.

---

scattered

Syntax: `scattered:<N>` (exact) | `scattered:<N>+` (minimum) | `scattered:<A>-<B>` (range) | `scattered:<A>,<B>` (OR list)

Values: Positive integer (number of distinct session roots)

Description: Finds assets whose variant files are spread across multiple **session roots** — the shoot/event boundaries detected by the `[group] session_root_pattern` regex (default: `^\d{4}-\d{2}`, matching directories starting with a date like `2024-10-05-wedding`). This is the same session root detection used by `auto-group`.

An asset with files in `2024-10/2024-10-05-event/Capture/` and `2024-10/2024-10-05-event/Output/Final/Web/` has `scattered:1` — both paths share the session root `2024-10/2024-10-05-event`, which is the same shoot. An asset with files in different session roots (e.g. `2024-10/2024-10-05-event/` and `2019-11/2019-11-30-other/`) has `scattered:2+`, indicating a potential cross-shoot mis-grouping.

This makes `scattered:2+` a precise diagnostic for finding assets that were accidentally merged across different shoots by `auto-group` — the original purpose of this filter. Subdirectories within the same shoot (Capture/, Selects/, Output/) do not inflate the count.

Examples:

```
maki search "scattered:2+"                  # files in 2+ distinct session roots (suspicious)
```

```
maki search "scattered:2+ variants:3+"       # scattered + many variants
maki search "scattered:1"                    # all files in the same session root (normal)
```

SQL behavior: Uses a custom `session_root(path, pattern)` function (registered at connection open) that implements the same deepest-date-component detection as `find_session_root()` in auto-group. Scalar subquery counting `DISTINCT session_root(relative_path, pattern)` across `file_locations` joined through `variants`. The pattern comes from `[group] session_root_pattern` in `maki.toml`.

---

duration

Syntax: `duration:<N>` | `duration:<N>+` | `duration:<min>-<max>` | `duration:<N>,<M>`

Values: Duration in seconds. Supports all standard numeric filter forms: exact (`60`), minimum (`30+`), range (`10-120`), OR (`30,60`), OR+min (`30,60+`).

Description: Filters by media duration — video **and** audio. Duration is extracted at import (ffprobe for video, lofty for audio) and stored in the shared denormalized `duration_seconds` column on the assets table. Images and documents have no duration and will not match this filter.

Examples:

```
maki search "duration:60"                    # exactly 60 seconds
maki search "duration:30+"                   # 30 seconds or longer
maki search "duration:10-120"                # between 10s and 2 minutes
maki search "duration:30,60"                 # exactly 30s or 60s
maki search "type:video duration:60+"        # videos at least 1 minute
maki search "type:audio duration:300+"       # tracks longer than 5 minutes
maki search "duration:10-30 rating:4+"       # short clips, highly rated
```

SQL behavior: Direct filter on the denormalized `a.duration_seconds` column using `NumericFilter` (exact/min/range/values). No JOIN required.

---

codec

Syntax: `codec:<text>`

Values: Partial, case-insensitive string match against the video codec.

Description: Filters by video codec. The codec is extracted from video files via ffprobe during import and stored as a denormalized `video_codec` column on the assets table. Common values include `h264`, `hevc` (H.265), `prores`, `av1`, `vp9`. Non-video assets have no codec and will not match this filter.

Examples:

```
maki search "codec:h264"                     # H.264 encoded videos
maki search "codec:hevc"                     # H.265/HEVC videos
maki search "codec:prores"                   # ProRes videos
maki search "type:video codec:h264 rating:4+"   # highly rated H.264 videos
maki search "codec:hevc duration:60+"        # HEVC videos at least 1 minute
```

SQL behavior: `WHERE a.video_codec LIKE '%value%'`. A plain SQLite `LIKE`, so the match is case-insensitive for ASCII letters only (`codec:HEVC` and `codec:hevc` are equivalent). Pure assets-table filter, no JOIN required.

---

key

Syntax: `key:<key>`

Values: Musical key in the notation your key tool emits — keyfinder-cli's standard notation by default (`A` = A major, `Am` = A minor, `Eb` = E-flat major, …). Case-insensitive exact match.

Description: Filters audio assets by detected musical key. Keys are not read from file tags — they are computed by `maki audio analyze` (external keyfinder-cli by default) and stored in the denormalized `audio_key` column. Assets that have not been analyzed will not match.

Examples:

```
maki search "key:am"                          # everything in A minor
maki search "key:F#m bpm:100-140"             # F-sharp minor at moderate tempo
maki search "type:audio key:c rating:4+"      # favorite tracks in C major
```

SQL behavior: `WHERE LOWER(a.audio_key) = LOWER(?)`. Pure assets-table filter, no JOIN required.

---

bpm

Syntax: `bpm:<N>` | `bpm:<N>+` | `bpm:<min>-<max>` | `bpm:<N>,<M>`

Values: Tempo in beats per minute. Supports all standard numeric filter forms.

Description: Filters audio assets by detected tempo. BPM is computed by `maki audio analyze` (beat_this beat tracker by default; MAKI derives BPM from the median inter-beat interval) and stored in the denormalized `audio_bpm` column. Assets that have not been analyzed will not match.

Examples:

```
maki search "bpm:120"                         # exactly 120 BPM
maki search "bpm:100-140"                     # moderate tempo range
maki search "type:audio bpm:160+ tag:genre|drum-and-bass"
```

SQL behavior: Direct filter on the denormalized `a.audio_bpm` column using `NumericFilter`. No JOIN required.

---

geo

Syntax: `geo:any` | `geo:none` | `geo:<lat>,<lng>,<radius_km>` | `geo:<south>,<west>,<north>,<east>`

Description: Filters by GPS geolocation. GPS coordinates are extracted from EXIF data during import and stored as denormalized `latitude`/`longitude` columns on the assets table.

Modes:

Form	Description
`geo:any`	Assets that have GPS coordinates
`geo:none`	Assets without GPS coordinates
`geo:52.5,13.4,10`	Assets within 10km of latitude 52.5, longitude 13.4 (bounding circle approximated as a bounding box)

`geo:48.0,11.0,53.0,14.0`	Assets within the bounding box: south 48.0, west 11.0, north 53.0, east 14.0

The 3-parameter form (lat, lng, radius) is converted to a bounding box internally using the approximation `dlat = radius / 111` and `dlng = radius / (111 * cos(lat))`.

Examples:

```
maki search "geo:any"                          # all geotagged assets
maki search "geo:none"                         # assets without GPS data
maki search "geo:52.52,13.405,5"               # within 5km of Berlin center
maki search "geo:48.0,11.0,53.0,14.0"          # bounding box covering Germany
maki search "geo:any rating:4+ tag:landscape"  # geotagged 4+ star landscapes
```

SQL behavior: - `geo:any`: `WHERE a.latitude IS NOT NULL AND a.longitude IS NOT NULL` - `geo:none`: `WHERE (a.latitude IS NULL OR a.longitude IS NULL)` - Bounding box/circle: `WHERE a.latitude >= ? AND a.latitude <= ? AND a.longitude >= ? AND a.longitude <= ?`

Pure assets-table filter, no JOIN required. Uses the composite index on `(latitude, longitude)`.

---

faces

Syntax: `faces:any` | `faces:none` | `faces:<N>` | `faces:<N>+` | `faces:<A>-<B>` | `faces:<A>,<B>`

Description: Filters by the number of detected faces on an asset. The filter itself works in every build; the counts are only populated by `maki faces detect` *(Pro)*, so on a standard build every asset has `faces:none`.

Modes:

Form	Description
`faces:any`	Assets with at least one detected face
`faces:none`	Assets with no detected faces
`faces:3`	Assets with exactly 3 detected faces
`faces:2+`	Assets with 2 or more detected faces

Examples:

```
maki search "faces:any"                    # all assets with faces
maki search "faces:none type:image"        # images without detected faces
maki search "faces:3"                      # group portraits (exactly 3 faces)
maki search "faces:2+ tag:portrait"        # portraits with multiple people
```

SQL behavior: Uses the denormalized `face_count` column on the assets table. Pure assets-table filter, no JOIN required.

---

person *(Pro)*

Syntax: `person:<name>` or `person:"<multi-word name>"`

Description: Filters to assets that contain at least one face assigned to the named person. Supports quoted values for multi-word names. Name resolution needs the face store, so in a standard (non-Pro) build a `person:` filter matches nothing.

Multiple people: the same rules as `tag:`. Commas within a single entry are OR (`person:Alice,Bob` — either person); separate `person:` entries are AND (`person:Alice person:Bob` — both in the photo). Excludes union: `-person:Alice -person:Bob` drops every asset showing either of them. CLI and web UI agree (until v4.10.1 the CLI OR-unioned separate entries).

```
maki search "person:Alice,Bob"              # either Alice or Bob
maki search "person:Alice person:Bob"       # both in the photo
maki search "-person:Alice -person:Bob"     # neither of them
```

In the web UI browse filter, each person chip is one `person:` entry, so multiple chips AND together. Type a comma-separated list (`Alice,Bob`) in the query input to get OR semantics.

Examples:

```
maki search "person:Alice"
maki search 'person:"John Smith"'
maki search "person:Alice rating:4+"
maki search "-person:Alice"                 # exclude assets with Alice
maki search "person:Alice person:Bob"       # both in the photo
```

SQL behavior: Resolves each name to the set of asset IDs carrying a face assigned to that person (via the face store), combines the sets per the rules above, then filters with `WHERE a.id IN (...)`; `-person:` uses `NOT IN`.

---

similar *(Pro)*

Syntax: `similar:<asset-id>` or `similar:<asset-id>:<limit>`

Description: Finds visually similar assets using stored SigLIP embeddings. Returns the top N most similar assets (default 40, counting the reference asset itself, which is always first at 100%). Requires embeddings to have been generated via `maki embed` or `maki import --embed`. The reference asset ID supports prefix matching (e.g. `similar:abc1` resolves like all other asset ID references).

Examples:

```
maki search "similar:72a0bb4b"                          # top 40 similar to this asset
maki search "similar:72a0bb4b:50"                       # top 50 similar
maki search "similar:72a0bb4b rating:3+ tag:landscape"  # similar AND 3+ stars AND landscape tag
maki search -q "similar:72a0bb4b"                       # just IDs, for scripting
```

Behavior: Looks up the stored embedding for the reference asset, loads all embeddings into an in-memory index, and performs a dot-product similarity search. If no embedding exists for the reference asset, exits with an error suggesting `maki embed --asset <id>`. The result set can be further filtered by all other search filters (AND logic). Results come back sorted by similarity, with the score shown as a badge in the web UI and appended to CLI short/full output (`92%`); JSON rows carry `similarity`.

`similar:`, `min_sim:`, `text:` and `--image` are Pro features: in a standard build these tokens are consumed without effect (the query runs as if they were not there).

---

min_sim *(Pro)*

Syntax: `min_sim:<percent>`

Description: Sets a minimum similarity threshold (0–100) for the three embedding searches: `similar:`, `text:`, and `--image`. Only assets with similarity at or above this percentage are included. Without `min_sim:`, all top-N results of the similarity search are returned. The pinned source asset of `similar:` (and the exact-copy hit of `--image`) is never filtered out.

Examples:

```
maki search "similar:72a0bb4b min_sim:90"          # only >= 90% similar
maki search "similar:72a0bb4b min_sim:80 type:image"  # >= 80% AND images only
maki search "text:sunset min_sim:20"                # drop weak text-to-image matches
maki search --image preview.jpg "min_sim:90"        # near-duplicates of a local file
```

Behavior: The percentage is converted to a 0.0–1.0 cosine similarity threshold internally. Values are clamped to the 0–100 range. Most useful for finding near-duplicates (min_sim:95) or visually very close images (min_sim:85).

–image (search by query image) *(Pro)*

Syntax: `maki search --image <file> [QUERY] [--limit <N>]`

Description: Finds assets visually similar to a local image file that is **not** in the catalog – a preview or downsized copy someone sent back, an export whose original needs locating, a screenshot. Nothing is imported: the query file is never copied, embedded, or stored. A flag rather than a `token:value` filter because Windows paths contain a colon.

Two stages:

1. **Exact copy first.** The file is hashed and looked up among the catalog's variants. A byte-identical copy is answered without loading the model; that asset comes first with `100% exact`.
2. **Visual similarity.** The file is encoded with the active SigLIP model and ranked against the embedding index, like `similar:`. RAW and video query files are rendered through the preview generator (dcraw / ffmpeg) first. Top N results (default 40, counting the exact match), sorted by similarity.

The `QUERY` is optional with `--image` and composes with AND: `min_sim:` floors the score, every other filter narrows the set.

Examples:

```
maki search --image ~/Downloads/IMG_4711_preview.jpg          # where is the original?
maki search --image scan.tif "min_sim:90" --json               # near-duplicates, scores in JSON
maki search --image frame.png "tag:events|wedding rating:3+"   # similar AND filtered
maki search --image shot.nef --limit 100 --format '{similarity}\t{exact}\t{id}\t{name}'
```

Output: Short and full formats append the score (`92%`, `100% exact`); JSON rows carry `similarity` (0.0–1.0) and `exact_match: true` on the hash hit; templates get `{similarity}` and `{exact}`. The exact-match asset ID is also printed to stderr.

Behavior: If the model is not downloaded and the file has an exact copy in the catalog, the exact match is still returned with a warning on stderr (the similarity ranking is skipped). Without an exact copy, a missing model is an error. `--image` cannot be combined with a `similar:` filter in the same query. In the web UI the same search is the **Find by image** button in the search row, drag-and-drop, or clipboard paste (see the web UI guide); the browse URL references the uploaded image as `similar:@<token>`.

---

embed

Syntax: `embed:any` | `embed:true` | `embed:none` | `embed:false`

Description: Filters by whether an asset has a stored AI embedding (SigLIP image embedding). The filter works in every build (the `embeddings` table always exists); embeddings are only generated by `maki embed` or `maki import --embed` *(Pro)*, so on a standard build `embed:none` matches everything.

Examples:

```
maki search "embed:any"                    # assets with AI embeddings
maki search "embed:true"                   # same as embed:any
maki search "embed:none"                   # assets without AI embeddings
maki search "embed:false"                  # same as embed:none
maki search "embed:none type:image"        # images that still need embeddings
```

SQL behavior: Uses an `EXISTS` / `NOT EXISTS` subquery on the `embeddings` table: `WHERE EXISTS (SELECT 1 FROM embeddings e WHERE e.asset_id = a.id)`. Pure subquery, no JOIN required.

---

text *(Pro)*

Syntax: `text:<query>`, `text:"<multi-word query>"`, or `text:"<query>":<limit>`

Multi-word queries **must** be quoted: `text:sunset beach` searches for "sunset" and treats `beach` as a separate free-text term.

Description: Natural language image search using SigLIP's text encoder. Encodes the query text into the same embedding space as image embeddings, then finds the most similar images via dot-product similarity. Requires embeddings to have been generated via `maki embed` or `maki import --embed`.

The result limit defaults to 50 and can be configured at three levels (highest priority wins):

1. **Inline syntax**: `text:"sunset":100` — per-query override
2. **`[ai] text_limit`** in `maki.toml` — catalog-wide default
3. **Hardcoded fallback**: 50

Examples:

```
maki search "text:sunset"                                  # images matching "sunset" (default limit)
maki search "text:\"colorful flowers in a garden\""          # multi-word query
maki search "text:\"person on a beach\" rating:3+"           # combined with other filters
maki search "text:\"mountain landscape\":100"                # return top 100 matches
```

Behavior: Loads the SigLIP model (text encoder), encodes the query string into an embedding vector, loads the in-memory embedding index, and returns the top N assets by dot-product similarity (default 50, configurable). `min_sim:` applies to the scores. The result set can be further filtered by all other search filters (AND logic). Since SigLIP is a vision-language model trained on image-text pairs, queries describe visual content ("red car", "sunset over water", "portrait of a woman") rather than metadata. Results quality depends on how well the SigLIP model generalizes.

Multilingual queries: The default model (`siglip-vit-b16-256`) is English-only. For German, French, Spanish, Italian, Japanese, Chinese, and many other languages, switch to the multilingual model:

```
[ai]
model = "siglip2-base-256-multi"
```

Then run `maki embed ''` (no `--force` needed — embeddings are stored per `(asset_id, model_id)`, so the new model only embeds assets that don't yet have one for it; existing English-model embeddings are untouched). Once switched, German queries like `text:"Sonnenuntergang am Strand"` or `text:"Hochzeit im Park"` work

natively without translation. See Switching models in the setup guide for the full workflow and verification commands.

Combining Filters

All filters are combined with AND logic. Every specified filter must match for an asset to appear in results. Free-text terms are also AND-combined with all prefix filters.

Example combinations:

```
# 5-star landscape images shot with a Nikon
maki search "rating:5 tag:landscape type:image camera:NIKON"

# High-ISO night shots in RAW format
maki search "iso:3200+ format:nef tag:night"

# Wide-angle portraits with shallow depth of field
maki search "focal:24-35 f:1.4-2.8 tag:portrait"

# Unverified images on the Photos volume, imported from a specific path
maki search "stale:30 type:image path:Capture/2026-01"

# 4K or larger images in a specific collection
maki search 'width:3840+ collection:"Best of 2026"'

# Orphaned video assets (no files on disk)
maki search "orphan:true type:video"

# Everything labeled Red with 4+ stars
maki search "label:Red rating:4+"

# Single-copy RAW files (backup risk)
maki search "copies:1 format:nef"

# Well-backed-up 5-star images
maki search "copies:2+ rating:5 type:image"

# Assets from a specific date range
maki search "dateFrom:2026-01-01 dateUntil:2026-03-31 tag:landscape"

# Everything shot in February 2026
maki search "date:2026-02"

# Unstacked 5-star images (candidates for stacking review)
maki search "stacked:false rating:5 type:image"

# Find stacked assets with a hierarchical tag
maki search "stacked:true tag:animals|birds"

# Visually similar assets, filtered to 4+ stars (Pro + embeddings)
maki search "similar:72a0bb4b rating:4+"

# Geotagged photos within 5km of a location
```

```
maki search "geo:52.52,13.405,5 rating:4+"

# All geotagged landscape images
maki search "geo:any tag:landscape type:image"
```

Quoting and Special Characters

Values with spaces

Use double quotes around filter values that contain spaces. When typing at the shell, wrap the entire query in single quotes to prevent the shell from stripping the inner double quotes.

```
# Shell-safe quoting: single quotes outside, double quotes for values
maki search 'tag:"Fools Theater"'
maki search 'camera:"Canon EOS R5" lens:"RF 50mm f/1.2"'
maki search 'collection:"My Favorites" rating:4+'
maki search 'path:"Photos/Family Trip" type:image'
```

Single-word values work without quotes:

```
maki search "tag:landscape"
maki search "camera:fuji"
```

Values with dashes

Without quotes, a leading `-` is interpreted as negation (`-tag:rejected` means "not tagged rejected"). When a dash appears *inside* a value (like a project name), quoting prevents misinterpretation:

```
# Wrong: "Geflüchtete" is parsed as negated free-text, not part of the tag
maki search "tag:project/Angekommen im Stadtbild - Geflüchtete im Portait"

# Correct: inner quotes keep the entire value together
maki search 'tag:"project/Angekommen im Stadtbild - Geflüchtete im Portait"'
```

Hierarchy separators

Tags use `|` as the hierarchy separator. In the shell, `|` is the pipe operator, so always quote tag values containing `|`:

```
maki search 'tag:"subject|performing arts|concert"'
maki search 'tag:subject'  # matches all descendants — no | needed
```

Quick reference

Character	In filter value	Solution
Space	Splits into multiple tokens	`tag:"golden hour"`
`-`	Interpreted as negation	`tag:"my - project"`
`|`	Shell pipe	`'tag:"subject|nature"'`

Character	In filter value	Solution
`"`	Ends quoted value	No escape sequence exists — a literal `"` cannot be typed in a query value. Match such tags via the web UI tag chips or the tags page.

Filter Availability

All filters work in the CLI (`maki search`), the web UI search box, and in saved searches. The web UI additionally provides dedicated controls for the most common filters:

Filter	Web UI control	Saved Searches
Free text	search box	yes
`type:`	dropdown (image/video/audio/document; `other` via search box) + search box	yes
`tag:`	tag chips + search box	yes
`format:`	multi-select panel + search box	yes
`rating:`	star clicks + search box	yes
`label:`	color dots + search box	yes
`camera:`	search box	yes
`lens:`	search box	yes
`description:` / `desc:`	search box	yes
`iso:`	search box	yes
`focal:`	search box	yes
`f:`	search box	yes
`width:`	search box	yes
`height:`	search box	yes
`meta:`	search box	yes
`path:`	text input + search box	yes
`collection:`	dropdown + search box	yes
`volume:`	dropdown + search box	yes
`copies:`	search box	yes
`variants:`	search box	yes
`tagcount:`	search box	yes
`scattered:`	search box	yes
`date:`	search box	yes

Filter	Web UI control	Saved Searches
`dateFrom:`	search box	yes
`dateUntil:`	search box	yes
`id:`	search box	yes
`orphan:`	search box	yes
`missing:true`	search box	yes
`stale:`	search box	yes
`stacked:`	search box	yes
`geo:`	search box (+ map view)	yes
`duration:`	search box	yes
`codec:`	search box	yes
`key:`	search box	yes
`bpm:`	search box	yes
`faces:`	search box	yes
`person:` *(Pro)*	people chips (multi-select with autocomplete) + search box	yes
`similar:` *(Pro)*	"Browse similar" button + search box	yes
`--image` *(Pro)*	"Find by image" button, drag-and-drop, paste	no (session-bound `similar:@<token>`)
`min_sim:` *(Pro)*	search box	yes
`text:` *(Pro)*	search box	yes
`embed:`	search box	yes

Related Topics

- Browse & Search (User Guide) – practical search workflows and output formatting
- Format Templates Reference – controlling output with `--format` and custom templates
- Configuration Reference – `maki.toml` settings

Format Templates Reference

The `--format` flag controls how `maki search` and `maki duplicates` display their results. It accepts preset names or custom template strings with placeholder substitution.

Presets

ids

One full UUID per line, with no header or decoration. Ideal for scripting and piping into other commands.

Shorthand: `-q` (equivalent to `--format=ids`)

```
$ maki search -q "tag:landscape"
a1b2c3d4-5678-9abc-def0-123456789abc
e5f6a7b8-1234-5678-9abc-def012345678
```

The result count line is suppressed.

short

Default compact view. One line per result showing short ID, filename, type, format, and date. A result count is printed at the end.

```
$ maki search "sunset"
a1b2c3d4  IMG_1234.jpg [image] (JPEG) -- 2026-01-15T10:30:00
e5f6a7b8  DSC_5678.nef [image] (NEF) -- 2026-01-14T16:45:00

2 result(s)
```

This is the default when no `--format` is specified.

full

Detailed view including tags and description on each line.

```
$ maki search "sunset" --format full
a1b2c3d4  IMG_1234.jpg [image] (JPEG) -- 2026-01-15T10:30:00 tags:sunset,landscape A golden sunset over the mountains
e5f6a7b8  DSC_5678.nef [image] (NEF) -- 2026-01-14T16:45:00 tags:sunset,nature
```

json

JSON array containing all result objects. Equivalent to using the global `--json` flag.

```
$ maki search "sunset" --format json
[
```

```
  {
    "asset_id": "a1b2c3d4-5678-9abc-def0-123456789abc",
    "original_filename": "IMG_1234.jpg",
    "asset_type": "image",
    ...
  }
]
```

Custom Templates

Build your own output format by passing a string containing `{placeholder}` tokens.

Syntax: `--format '{placeholder}\t{another}'`

Rules

- Placeholders are delimited by `{` and `}`.
- Known placeholders are replaced with the corresponding value for each result row.
- Unknown placeholders are left as-is in the output (e.g., `{bogus}` prints literally as `{bogus}`).
- An unclosed brace (no matching `}`) is emitted literally: `{world` prints as `{world`.
- The result count line is suppressed when a custom template (or any explicit `--format`) is used.

Placeholders (search)

Available in `maki search --format`:

Placeholder	Description
`{id}`	Full asset UUID
`{short_id}`	First 8 characters of the UUID
`{name}`	Asset name if set, otherwise falls back to the original filename
`{filename}`	Original filename (always the file's actual name, ignoring any custom asset name)
`{type}`	Asset type: `image`, `video`, `audio`, `document`, or `other`
`{format}`	Primary variant format/extension (e.g., `JPEG`, `NEF`, `MOV`)
`{date}`	Asset creation timestamp (ISO 8601 format)
`{tags}`	Comma-separated list of tags (empty string if no tags)
`{description}`	Description text (empty string if no description)
`{hash}`	Content hash of the best display variant
`{label}`	Color label name (e.g., `Red`, `Blue`; empty string if no label)
`{similarity}`	Similarity percentage (0–100) in a `similar:` / `--image` search; empty otherwise *(Pro)*

Placeholder	Description
`{exact}`	The word `exact` when the row is the byte-identical copy of a `--image` query file; empty otherwise *(Pro)*

Placeholders (duplicates)

Available in `maki duplicates --format`. Includes all search placeholders above, plus:

Placeholder	Description
`{locations}`	Comma-separated list of all file locations where the duplicate exists (format: `volume_label -> relative/path`)

Escape Sequences

Three escape sequences are recognized in template strings:

Sequence	Output
`\t`	Tab character
`\n`	Newline character
`\\`	Literal backslash

A backslash followed by any other character emits a literal backslash (the backslash is not consumed).

Examples

Tab-separated values for a spreadsheet

```
maki search "type:image" --format '{name}\t{format}\t{date}\t{tags}'
```

Output:

```
IMG_1234.jpg    JPEG    2026-01-15T10:30:00 sunset,landscape
DSC_5678.nef    NEF 2026-01-14T16:45:00 sunset,nature
```

One name per line with rating label

```
maki search "rating:4+" --format '{name} [{label}]'
```

Output:

```
Golden Sunset [Red]
Mountain Lake [Blue]
```

Multi-line detail view

```
maki search "tag:portfolio" --format '{name}\n  Format: {format}\n  Tags: {tags}\n  {description}\n'
```

Output:

```
Golden Sunset
  Format: NEF
  Tags: sunset,landscape,portfolio
  A golden sunset over the mountain range

Mountain Lake
  Format: JPEG
  Tags: nature,lake,portfolio
  Crystal clear reflections at dawn
```

Short IDs with filenames (compact scripting format)

```
maki search "format:nef" --format '{short_id} {filename}'
```

Output:

```
a1b2c3d4 DSC_1234.NEF
e5f6a7b8 DSC_5678.NEF
```

Hash-based file listing

```
maki search "type:image" --format '{hash}\t{filename}'
```

Output:

```
sha256:abc123def456...  IMG_1234.jpg
sha256:def789abc012...  DSC_5678.NEF
```

Duplicate locations as tab-separated

```
maki duplicates --format '{filename}\t{format}\t{locations}'
```

Output:

```
DSC_1234.NEF        NEF  Photos  ->  Capture/2026-01-15/DSC_1234.NEF,  Backup  ->  Capture/2026-01-
15/DSC_1234.NEF
```

Format Flag Behavior

Flag	Equivalent	Result count shown
(none)	`--format short`	yes
`--format short`	default	yes
`--format ids`	`-q`	no
`-q`	`--format ids`	no

Flag	Equivalent	Result count shown
`--format full`	–	no
`--format json`	`--json`	no
`--format '{...}'`	custom template	no

When `--format` is explicitly provided (any value including `short`), the result count line is suppressed, keeping output clean for piping and parsing. The count is only shown when no `--format` flag is given at all.

Related Topics

- Search Filters Reference – all available search filters
- Browse & Search (User Guide) – practical search and output examples
- CLI Conventions – global flags, scripting patterns, exit codes

Configuration Reference (maki.toml)

The `maki.toml` file stores catalog-level configuration. It lives at the root of your catalog directory (the same directory that contains the `metadata/`, `previews/`, and `catalog.db` files).

File Location

`maki.toml` is created automatically by `maki init`. maki locates it by searching the current directory and walking up through parent directories until it finds a directory containing `maki.toml`.

```
my-catalog/
  maki.toml              <-- configuration file
  catalog.db
  metadata/
  previews/
  searches.toml
  collections.yaml
```

All sections and fields are optional. A missing file or an empty file is equivalent to all-defaults. A comment-only file is also valid:

```
# maki catalog configuration
```

The web UI's **Settings** entry (top nav, next to Maintain) opens a schema-driven form that edits this file directly — see Settings Dialog. The form is generated from the underlying type definitions, so every option below appears as a widget. The previous `maki.toml` is copied to `maki.toml.bak` before each save. Hand-editing the file with a text editor remains fully supported and is the only way to add or rename hashmap-style sub-tables (e.g. `[import.profiles.<name>]`, `[vlm.model_config."<model>"]`).

Top-Level Fields

default_volume

- **Type:** UUID string (optional)
- **Default:** none

Fallback volume for `maki import` when auto-detection from the file path is ambiguous or fails. When set, import uses this volume if it cannot determine the correct volume from the first path argument.

```
default_volume = "550e8400-e29b-41d4-a716-446655440000"
```

Find your volume UUIDs with `maki volume list --json`.

---

preview Section

Controls how preview thumbnails are generated during import and by `maki generate-previews`.

max_edge

- **Type:** unsigned integer
- **Default:** `800`
- **Validation:** must be greater than 0

Maximum pixel size of the longest edge for generated thumbnails. The shorter edge is scaled proportionally. Applies to all preview types (standard images, RAW conversions, video frames, and info cards).

format

- **Type:** string (`"jpeg"` or `"webp"`)
- **Default:** `"jpeg"`

Output format for preview files. JPEG produces lossy compressed thumbnails controlled by the `quality` setting. WebP produces lossless thumbnails via the `image` crate (the `quality` setting is ignored for WebP).

quality

- **Type:** unsigned integer (1–100)
- **Default:** `85`
- **Validation:** must be between 1 and 100

JPEG compression quality. Higher values produce larger files with better visual quality. Only applies when `format = "jpeg"`. Ignored for WebP.

smart_max_edge

- **Type:** unsigned integer
- **Default:** `2560`
- **Validation:** must be greater than 0

Maximum pixel size of the longest edge for smart previews. Smart previews are a higher-resolution preview tier (separate from regular thumbnails) that enable zoom and pan in the web UI.

smart_quality

- **Type:** unsigned integer (1–100)
- **Default:** `85`

JPEG compression quality for smart previews. Only applies when the preview format is JPEG.

generate_on_demand

- **Type:** boolean
- **Default:** `false`

When `true`, the web server generates smart previews automatically on first request. The first load takes a few seconds while the preview is generated; subsequent requests are served from disk. A pulsing "HD" badge in the lightbox and detail page provides visual feedback during generation.

When `false`, smart previews must be generated explicitly via `maki import --smart`, `maki generate-previews --smart`, or the "Regenerate previews" button on the asset detail page.

Notes

Changing `max_edge` or `format` affects only newly generated previews. Existing previews are not automatically regenerated. Use `maki generate-previews --force` to regenerate all previews with the new settings.

Smart previews are stored in a separate directory (`smart_previews/`) and do not replace regular thumbnails.

```
[preview]
max_edge = 1200
format = "jpeg"
quality = 90
smart_max_edge = 2560
smart_quality = 85
generate_on_demand = true
```

[serve] Section

Controls the built-in web UI server started by `maki serve`.

port

- **Type:** unsigned 16-bit integer
- **Default:** `8080`

TCP port for the web server.

bind

- **Type:** string (IP address)
- **Default:** `"127.0.0.1"`

Bind address for the web server. Use `"127.0.0.1"` to restrict access to the local machine. Use `"0.0.0.0"` to allow connections from other devices on the network.

per_page

- **Type:** unsigned 32-bit integer
- **Default:** `60`

Number of results per page in the browse grid.

stroll_neighbors

- **Type:** unsigned 32-bit integer
- **Default:** `12`

Initial number of neighbor thumbnails shown around the center image on the stroll page. Controls the default position of the neighbor count slider.

stroll_neighbors_max

- **Type:** unsigned 32-bit integer
- **Default:** `25`

Maximum value for the neighbor count slider on the stroll page.

stroll_discover_pool

- **Type:** unsigned 32-bit integer
- **Default:** `80`

Candidate pool size for Discover mode on the stroll page. In Discover mode, the server fetches this many nearest neighbors and then randomly samples from the pool to produce the displayed satellites. Larger values increase variety; smaller values keep results closer to the center image.

stroll_fanout

- **Type:** unsigned 32-bit integer
- **Default:** `5`

Initial fan-out count for L2 transitive neighbors on the stroll page. Controls how many second-level neighbors fan out from a focused satellite. Set to `0` to disable fan-out by default.

stroll_fanout_max

- **Type:** unsigned 32-bit integer
- **Default:** `10`

Maximum value for the fan-out slider on the stroll page.

read_only

- **Type:** boolean
- **Default:** `false`

Safe-sharing mode: the server rejects every mutating request (metadata edits, batch operations, jobs, file-deleting endpoints) with `403 Forbidden`; browsing, search, previews, and the lightbox keep working. Enforcement is by HTTP method (anything other than GET/HEAD is rejected), so it covers every current and future write endpoint.

Use this when exposing the catalog to other people or devices, e.g. "show photos to clients on the iPad":

```
maki serve --bind 0.0.0.0 --read-only
```

The `--read-only` flag and the config value are OR-ed — either one enables the mode.

username / password

- **Type:** strings
- **Default:** unset (no authentication)

HTTP Basic-Auth credentials. Authentication is enforced on **every** route (pages, API, previews, static assets) once both values are non-empty. The password can also be supplied via the `MAKI_SERVE_PASSWORD` environment variable, which takes precedence over the config value — useful to keep the password out of `maki.toml`:

```
[serve]
username = "thomas"
# password via: MAKI_SERVE_PASSWORD=… maki serve
```

Combine with `read_only` for sharing, or use alone for full-access remote use. When binding to a non-loopback address with neither `read_only` nor authentication enabled, `maki serve` prints a warning at startup.

CLI Override

The `--port`, `--bind`, `--per-page`, and `--read-only` flags on `maki serve` override the values from `maki.toml`:

```
maki serve --port 9090 --bind 0.0.0.0 --per-page 100
```

```
[serve]
port = 8080
bind = "127.0.0.1"
per_page = 100
stroll_neighbors = 12
stroll_neighbors_max = 25
stroll_fanout = 5
stroll_fanout_max = 10
stroll_discover_pool = 80
read_only = false
# username = "thomas"
# password = "…"   # prefer MAKI_SERVE_PASSWORD env var
```

[import] Section

Controls import behavior for `maki import`.

exclude

- **Type:** array of glob pattern strings
- **Default:** `[]` (empty, nothing excluded)

Glob patterns matched against filenames (not full paths). Files matching any pattern are skipped during import. Useful for excluding OS-generated files, temporary files, and other non-media clutter.

Pattern matching uses the `glob-match` crate with standard glob syntax: `*` matches any sequence of characters, `?` matches a single character.

```
[import]
exclude = [
    "Thumbs.db",
    ".DS_Store",
    "*.tmp",
    "*.bak",
    "desktop.ini",
]
```

auto_tags

- **Type:** array of tag strings
- **Default:** `[]` (empty, no auto-tags)

Tags automatically applied to every newly imported asset. These are merged with any tags extracted from XMP metadata, CLI `--add-tag` values, and deduplicated (no duplicate tags are created).

Useful for marking import batches or applying a default workflow status:

```
[import]
auto_tags = ["inbox", "unreviewed"]
```

smart_previews

- **Type:** boolean
- **Default:** `false`

When `true`, import automatically generates smart previews (high-resolution, 2560px) alongside regular thumbnails. Equivalent to passing `--smart` on every `maki import` command. Smart preview dimensions are controlled by `[preview] smart_max_edge`. Also sets the initial checked-state of the "Smart previews" checkbox in the web UI import dialog.

```
[import]
smart_previews = true
```

embeddings

(Pro)

- **Type:** boolean
- **Default:** `false`

When `true`, import automatically generates SigLIP image embeddings for visual similarity search alongside previews. Equivalent to passing `--embed` on every `maki import` command. Embeddings enable `maki auto-tag --similar` and the web UI "Find similar" button.

Uses the model configured in `[ai] model`. Silently skips if the model is not downloaded. Non-image assets are skipped. Also sets the initial checked-state of the "Generate embeddings" checkbox in the web UI import dialog (`descriptions` does the same for the VLM "Generate descriptions" checkbox). The web defaults are read at server startup, so a `maki serve` restart is needed to pick up a config change.

```
[import]
embeddings = true
```

descriptions

- **Type:** boolean
- **Default:** `false`

When `true`, import automatically generates VLM descriptions for newly imported assets as a post-import phase. Equivalent to passing `--describe` on every `maki import` command.

Uses the VLM configured in `[vlm]` (endpoint, model, prompt, mode, temperature). Silently skips if the VLM endpoint is not available. Assets that already have descriptions are skipped. Works with all `[vlm] mode` settings (describe, tags, both).

```
[import]
descriptions = true
```

profiles

- **Type:** table of named profiles
- **Default:** (none)

Named preset configurations for different import scenarios. Each profile is a sub-table under `[import.profiles]` that overrides the base `[import]` config. Unset fields inherit from the base. CLI flags override both the profile and base config.

Profile fields (all optional):

Field	Type	Description
`exclude`	string array	Override exclude patterns
`auto_tags`	string array	Override auto-tags (replaces base, not merged)
`smart_previews`	boolean	Override smart preview generation
`embeddings`	boolean	Override embedding generation
`descriptions`	boolean	Override VLM description generation
`include`	string array	File type groups to include (e.g. `captureone`)
`skip`	string array	File type groups to skip (e.g. `audio`)

```toml
[import]
exclude = [".DS_Store", "Thumbs.db"]
auto_tags = ["inbox"]

[import.profiles.card]
auto_tags = ["from-card"]
smart_previews = true

[import.profiles.studio]
auto_tags = ["studio"]
smart_previews = true
embeddings = true
descriptions = true
include = ["captureone"]
skip = ["audio"]
```

Usage: `maki import --profile card /Volumes/CARD/DCIM`

[dedup] Section

Controls dedup behavior for `maki dedup` and the web UI's auto-resolve action.

prefer

- **Type:** string (optional)
- **Default:** none

Default path substring for the `--prefer` flag. When set, dedup prefers keeping file locations whose relative path contains this string. Useful for always keeping files in a curated directory (e.g. `Selects`) while removing copies elsewhere.

The CLI `--prefer` flag overrides this value. The web UI duplicates page pre-populates its "Prefer keeping" input from this setting.

```toml
[dedup]
prefer = "Selects"
```

[verify] Section

Controls incremental verify behavior for `maki verify`.

max_age_days

- **Type:** integer (optional)
- **Default:** none

Default value for the `--max-age` flag. When set, `maki verify` skips files verified within the given number of days. The CLI `--max-age` flag overrides this value. `--force` overrides both.

```
[verify]
max_age_days = 30
```

[watch] Section

Controls `maki watch`, the poll-based auto-import loop.

poll_seconds

- **Type:** integer
- **Default:** `30`

Seconds between filesystem scans of the watched roots. Minimum 2 — lower values are clamped at runtime. The `--interval` CLI flag overrides this value. Because a new or changed file must show an identical (size, mtime) signature on two consecutive scans before it is acted on, the poll interval is also the minimum time a file must be stable before import.

exclude

- **Type:** array of strings (glob patterns)
- **Default:** `[]`

Additional exclude patterns applied during watch scans, on top of `[import] exclude`. Same matching as import: glob patterns matched against file and directory names (`*.tmp`, `Thumbs.db`, `CaptureOne`). Hidden files and directories (leading `.`) are always skipped.

```
[watch]
poll_seconds = 10
exclude = ["*.tmp", "*.partial"]
```

[group] Section

Controls how `maki auto-group` identifies session boundaries (shoot/event roots) when partitioning assets into directory neighborhoods.

session_root_pattern

- **Type:** string (regex)
- **Default:** `^\d{4}-\d{2}`

A regular expression matched against each directory component in a variant's path. The deepest (rightmost) matching component becomes the session root — all files below that directory are considered part of the same session and eligible for stem matching.

The default pattern matches directory names starting with `YYYY-MM` (e.g., `2024-10`, `2024-10-05-wedding`, `2025-05-09-event`), which works for the common `year/year-month/year-month-day-event/` hierarchy.

Set to an empty string to disable session root detection entirely; auto-group then falls back to parent-directory grouping (each immediate parent directory is its own group scope). `maki import --auto-group` uses the directory handed to `import` as the session instead, falling back to the parent only when the files sit directly in that directory – see `--auto-group`.

```
[group]
# Default: date-prefixed directories
session_root_pattern = '^\d{4}-\d{2}'

# Match directories starting with "shoot-" or "project-"
# session_root_pattern = '^(shoot|project)-'

# Disable session root detection (parent-directory fallback)
# session_root_pattern = ''
```

[contact_sheet] Section

Default settings for `maki contact-sheet`. All fields are optional; CLI flags override these values.

Key	Type	Default	Description
`layout`	string	`"standard"`	Layout preset: `dense`, `standard`, `large`
`paper`	string	`"a4"`	Paper size: `a4`, `letter`, `a3`
`fields`	string	`"filename,date,rating"`	Comma-separated metadata fields
`margin`	float	`10.0`	Page margin in mm
`quality`	integer	`90`	JPEG quality for page images (1–100)
`label_style`	string	`"border"`	Color label display: `border`, `dot`, `none`
`copyright`	string	unset	Copyright text for page footer. Optional – omitted from the file when not set (no footer).

```
[contact_sheet]
layout = "dense"
paper = "a3"
label_style = "dot"
copyright = "© 2026 Thomas Herrmann"
```

[ai] Section *(Pro)*

Controls AI auto-tagging behavior for `maki auto-tag`.

model

- **Type:** string
- **Default:** `"siglip-vit-b16-256"`

Which SigLIP model to use. Available models:

Model ID	Size	Embedding dim	Notes
`siglip-vit-b16-256`	~207 MB	768	English-only, good balance (default)
`siglip-vit-l16-256`	~670 MB	1024	English-only, higher accuracy
`siglip2-base-256-multi`	~410 MB	768	**Multilingual** (German, French, Spanish, Italian, Japanese, Chinese, etc.)
`siglip2-large-256-multi`	~920 MB	1024	**Multilingual**, higher accuracy, slower

The CLI `--model` flag overrides this value. Embeddings are stored per `(asset_id, model_id)`, so switching models doesn't corrupt existing data — the old model's embeddings stay intact. After switching, run `maki embed ''` (without `--force`) to generate embeddings for the new model: it processes only assets that don't yet have an embedding for the active model, and is restart-safe. See Switching models in the setup guide for the full workflow.

Multilingual model: `siglip2-base-256-multi` is Google's SigLIP 2 base, trained on the WebLI dataset across many languages. Use this if you want to type `text:` queries in German or any non-English language. It uses the Gemma SentencePiece tokenizer (vocab 256k) and is a drop-in replacement for `siglip-vit-b16-256` in dimensions and image resolution.

> **Note:** earlier releases scored the SigLIP 2 models with SigLIP 1's calibration constants, which inflated auto-tag confidence values. The constants now match the released SigLIP 2 weights, so confidences are lower (and honest) — if you tuned `threshold` upward specifically for a SigLIP 2 model, re-tune it after upgrading. Similarity search and `text:` ranking are unaffected.

threshold

- **Type:** float (0.0–1.0)
- **Default:** `0.1`
- **Validation:** must be between 0.0 and 1.0

Minimum confidence score for a tag to be suggested. Higher values produce fewer but more confident suggestions. Lower values produce more suggestions with more noise.

labels

- **Type:** string (file path, optional)
- **Default:** none (uses built-in ~100 photography categories)

Path to a custom labels file (one label per line). When set, overrides the built-in default labels. The CLI `--labels` flag overrides this value.

model_dir

- **Type:** string (directory path)
- **Default:** `"~/.maki/models"`

Where to cache downloaded model files. The `~` prefix is expanded to the user's home directory.

prompt

- **Type:** string
- **Default:** `"a photograph of {}"`

Text encoder prompt template. The `{}` placeholder is replaced with each label name before encoding. Adjusting the prompt can improve classification accuracy for specific use cases (e.g., `"a photo of a {}"` or `"a professional photograph of {}"`).

execution_provider

- **Type:** string (`"auto"`, `"cpu"`, `"coreml"`)
- **Default:** `"auto"`

  GPU providers require the macOS Pro build (which includes CoreML support). On other platforms, all values fall back to CPU.

Selects the ONNX Runtime execution provider for AI inference (SigLIP, YuNet, ArcFace). `"auto"` uses CoreML when available on macOS (Neural Engine on Apple Silicon, Metal on Intel), falling back to CPU. `"cpu"` forces CPU-only inference. `"coreml"` explicitly requests CoreML (errors if unavailable).

face_cluster_threshold

- **Type:** float (0.0–1.0)
- **Default:** `0.35`

Minimum average cosine similarity for two face clusters to be merged during agglomerative clustering. Tuned for the aligned FP32 ArcFace pipeline where intra-person similarity typically falls in 0.5–0.9 and inter-person similarity is near zero or slightly negative.

Higher values produce tighter, more conservative clusters (more singletons, less risk of mixing people). Lower values produce bigger clusters at the risk of merging similar-looking different people. Use `maki faces similarity` to inspect your data's actual distribution before picking a value far from the default.

face_min_confidence

- **Type:** float (0.0–1.0)
- **Default:** `0.7`

Default value for the `--min-confidence` flag on `maki faces cluster` — face detections below this confidence are dropped before clustering (blurry, profile, or partial faces produce noisy embeddings that hurt cluster purity). Can also be passed per-invocation to override.

text_limit

- **Type:** unsigned integer
- **Default:** `50`

Maximum number of results returned by `text:` semantic search queries. This is the default limit used when no inline limit is specified in the query. Can be overridden per-query with the `text:"query":limit` syntax.

```
[ai]
threshold = 0.3
labels = "my-labels.txt"
model_dir = "~/.maki/models"
prompt = "a photograph of {}"
execution_provider = "auto"
face_cluster_threshold = 0.35
face_min_confidence = 0.7
text_limit = 50
```

[vlm] Section *(Pro)*

Controls the VLM (vision-language model) integration for `maki describe`.

endpoint

- **Type:** string (URL)
- **Default:** `"http://localhost:11434"`

Base URL of the VLM server. Any server implementing the OpenAI-compatible `/v1/chat/completions` endpoint works. Ollama also supports its native `/api/generate` endpoint as an automatic fallback.

Local servers: - Ollama – `http://localhost:11434` (default) - LM Studio – `http://localhost:1234` - llama.cpp server – `http://localhost:8080`

Cloud APIs (OpenAI-compatible, require API key in environment): - OpenAI – `https://api.openai.com` (model: `gpt-6-astra`) - Groq – `https://api.groq.com/openai` - Together AI – `https://api.together.xyz`

Cloud model rosters rotate quickly (OpenAI retired `gpt-4o` and the GPT-5.x line in 2026, and the Llama vision previews formerly listed here are gone) – check the provider's current vision-model list for the exact identifier.

> **Note:** Cloud APIs charge per request. maki does not set authentication headers – if your endpoint requires an API key, you may need to use a local proxy or set the key via the server's own configuration.

model

- **Type:** string
- **Default:** `"qwen2.5vl:3b"`

Model name passed to the VLM server. For Ollama, this is the model tag (e.g., `moondream`, `qwen2.5vl:3b`, `qwen3.5:4b`). For cloud APIs, this is the model identifier (e.g., `gpt-6-astra`).

Recommended models for photography (tested with Ollama on Apple Silicon):

Model	Size	RAM	Speed (M3 Pro)	Quality
Moondream 2B	1.7 GB	~2 GB	~3–5s	Good, fast for batch
Qwen2.5-VL 3B	2.0 GB	~3 GB	~8–12s	Very good (default)
Qwen3-VL 4B	2.8 GB	~4 GB	~10–15s	Very good
Gemma 3 4B	3.3 GB	~4 GB	~10–15s	Very good
Qwen3-VL 8B	5.2 GB	~6 GB	~15–20s	Excellent
Qwen2.5-VL 7B	4.7 GB	~6 GB	~20–36s	Excellent

The table above lists the long-tested baseline models. See the VLM Model Guide for the current generations — Qwen3.5 (`qwen3.5:4b`/`:9b`) and Gemma 4 (`gemma4:e4b`) — plus backend setup instructions and hardware recommendations.

max_tokens

- **Type:** unsigned 32-bit integer
- **Default:** `500`

Maximum number of tokens in the VLM response. 500 tokens is typically 3–5 sentences, leaving headroom for models that use internal reasoning tokens.

prompt

- **Type:** string (optional)
- **Default:** none (uses built-in photography-focused prompt)

Custom system prompt sent to the VLM. When not set, uses a built-in prompt appropriate for the mode (describe or tags). In `--mode both`, custom prompts are ignored because each of the two calls uses its specialized built-in prompt.

Override for specialized workflows:

```
[vlm]
prompt = "Describe the architectural style, materials, and notable design features."
```

mode

- **Type:** string
- **Default:** `"describe"`

Default output mode for every VLM pass: `maki describe` (when `--mode` is not given), the description phase of `maki import --describe` (and the web import dialog's "Generate descriptions"), and the web UI's Describe button and batch Describe. One of: `describe` (natural language description), `tags` (JSON tag suggestions, applied with `vlm` tag provenance), `both` (two separate VLM calls: one for description, one for tags).

Precedence for `maki describe` is CLI `--mode` > `[vlm] mode` > `describe`. The web UI reads this value at server startup (restart `maki serve` after changing it); a per-request `mode` in the JSON body of `/api/asset/{id}/vlm-describe` or `/api/batch/describe` still overrides it.

temperature

- **Type:** float
- **Default:** `0.7`

Sampling temperature controlling randomness in VLM output. `0.0` = deterministic (same input always produces the same output), `0.7` = balanced (default), `1.0+` = more creative/varied. Lower values give more consistent results; higher values give more diverse but potentially less accurate output.

timeout

- **Type:** unsigned 32-bit integer (seconds)
- **Default:** `300`

Maximum time to wait for a VLM response. Larger models on CPU may need higher timeouts. Assets that time out are reported as errors and skipped.

concurrency

- **Type:** unsigned 32-bit integer
- **Default:** `1`

Number of concurrent VLM requests. When greater than 1, `maki describe`, `maki import --describe`, and web UI batch describe process multiple assets in parallel. Each batch of `concurrency` assets sends VLM HTTP calls concurrently using scoped threads; preparation (skip checks, image lookup) and result application (catalog writes) remain sequential. Set to the number of simultaneous requests your VLM server can handle efficiently — for local Ollama this depends on available VRAM and model size.

models

- **Type:** array of strings (optional)
- **Default:** `[]` (empty — only the default model is shown)

List of VLM model names to offer in the web UI model selector dropdown on the asset detail page. When configured with two or more models (including the default `model`), a dropdown appears next to the "Describe" button letting you choose which model to use per request. The first entry is pre-selected. Models not in this list are not offered in the web UI (useful when your Ollama server has many models loaded but only some are suitable for image description).

```
[vlm]
model = "moondream"
models = ["moondream", "qwen3-vl:4b"]
```

The CLI `maki describe --model` flag is unaffected by this setting — it accepts any model name regardless of the `models` list.

max_image_edge

- **Type:** unsigned 32-bit integer
- **Default:** `0` (no resizing)

Maximum pixel size of the longest edge for images sent to the VLM. Larger images are downscaled before encoding, which cuts upload size and inference time on models that internally tile or resize anyway. `0` sends the source image (smart preview or original) unchanged. Overridable per model (see Per-Model Configuration).

num_ctx

- **Type:** unsigned 32-bit integer
- **Default:** `0` (not set — use server default)

Context window size passed to the VLM server. When non-zero, overrides the model's default context length. Useful for models that benefit from a larger context (e.g., `num_ctx = 4096` for Qwen models). A value of `0` means "not set" — the server uses its own default.

top_p

- **Type:** float
- **Default:** `0.0` (not set — use server default)

Nucleus sampling parameter. When non-zero, only tokens whose cumulative probability exceeds `top_p` are considered. Lower values produce more focused output. A value of `0.0` means "not set" — the server uses its own default.

top_k

- **Type:** unsigned 32-bit integer
- **Default:** `0` (not set — use server default)

Top-K sampling parameter. When non-zero, limits token selection to the K most probable candidates. A value of `0` means "not set" — the server uses its own default.

repeat_penalty

- **Type:** float
- **Default:** `0.0` (not set — use server default)

Repetition penalty applied to tokens that have already appeared. Values above `1.0` discourage repetition; `1.0` disables the penalty. A value of `0.0` means "not set" — the server uses its own default.

Per-Model Configuration

You can override any VLM setting for specific models using `[vlm.model_config."model-name"]` sections. When `maki describe` runs with a given model (via `--model` or `[vlm] model`), any matching per-model section is merged on top of the global `[vlm]` settings. CLI flags always win over both.

This is useful when different models need different timeouts, context sizes, or sampling parameters:

```
[vlm]
model = "qwen2.5vl:3b"
timeout = 300

[vlm.model_config."qwen3-vl:4b"]
max_image_edge = 384
timeout = 300
num_ctx = 4096

[vlm.model_config."moondream:latest"]
max_tokens = 200
temperature = 0.1
```

When running `maki describe --model qwen3-vl:4b`, the per-model overrides apply: `timeout` becomes 300, `num_ctx` becomes 4096, and `max_image_edge` becomes 384. All other settings fall through from the global `[vlm]` section.

Per-model sections support exactly these fields: `max_tokens`, `temperature`, `timeout`, `max_image_edge`, `num_ctx`, `top_p`, `top_k`, `repeat_penalty`, and `prompt`. The connection-level settings (`endpoint`, `mode`, `concurrency`) and `model`/`models`/`model_config` itself are global only.

CLI Override

The `--endpoint`, `--model`, `--prompt`, `--max-tokens`, `--timeout`, `--temperature`, `--mode`, `--num-ctx`, `--top-p`, `--top-k`, and `--repeat-penalty` flags on `maki describe` override the values from `maki.toml` (including per-model overrides).

```
[vlm]
endpoint = "http://localhost:11434"
model = "qwen2.5vl:3b"
max_tokens = 500
timeout = 300
temperature = 0.7
mode = "describe"
concurrency = 1
num_ctx = 0
top_p = 0.0
top_k = 0
repeat_penalty = 0.0
# models = ["qwen2.5vl:3b", "moondream"]
# prompt = "Custom prompt here."

# Per-model overrides (optional)
# [vlm.model_config."qwen3-vl:4b"]
# timeout = 300
# num_ctx = 4096
```

[browse] Section

Controls default browsing behavior for both the CLI `maki search` and the web UI.

Field	Type	Default	Description
`default_filter`	string	*(none)*	Search filter applied to all browse, search, and stroll views. Uses the same syntax as the search bar.
`slideshow_seconds`	integer	`5`	Initial advance interval (seconds) for the lightbox slideshow. The user can bump it at runtime with `+`/`-` while the slideshow is running (range clamped to 1–60).
`slideshow_loop`	bool	`true`	Initial loop-after-last state of the lightbox slideshow. When `true`, the slideshow wraps from the last visible asset back to the first; when `false`, it stops at the last asset. Toggle button in the lightbox toolbar overrides this per-session.

Field	Type	Default	Description
`remember_latest_filter`	bool	`true`	When `true`, the browse page persists the most-recently-viewed filter (filter + sort + page + visible asset IDs) to `localStorage` and restores it on a bare-`/` visit. A "Restored from last session" pill appears with a Clear button so the state is visible and dismissable. Page is recomputed by finding the first remembered asset still in the (possibly-changed) result set — survives intervening retags and deletions without landing on a stale page. Set `false` to disable persistence entirely.

The default filter is AND'd with whatever the user types — it acts as a persistent base filter. In the web UI, a toggle next to the search bar lets you temporarily disable it.

Scope: Applied to browse, search results, stroll, analytics, and map views. NOT applied to operational commands (`export`, `describe`, `contact-sheet`, `auto-tag`, etc.) where you pass your own explicit query.

Examples

```
# Only show rated assets (hide unreviewed and unrated)
[browse]
default_filter = "rating:1+"

# Hide assets tagged "rest" (recommended for most users)
[browse]
default_filter = "-tag:rest"

# Combine multiple conditions
[browse]
default_filter = "-tag:rest -tag:rejected"

# Show only images
[browse]
default_filter = "type:image"

# Slideshow — start at 8-second cadence, stop at end instead of looping
[browse]
slideshow_seconds = 8
slideshow_loop = false
```

Slideshow

The web UI's lightbox has a slideshow mode. Two buttons in the lightbox toolbar:

- ▶ / ❚❚ — start / pause. Keyboard shortcut: **Space**.
- ↻ / → — loop after last asset (toggle). When ↻ (loop on), the slideshow wraps to the first asset; when → (loop off), it stops at the last asset.

While the slideshow is running:

- **+** / **−** bump the cadence by one second (clamped to 1–60). The current value flashes briefly next to the play button.
- Clicking **Prev** / **Next**, changing the **rating** or **label**, or pressing **Esc** auto-pauses the slideshow so deliberate user actions take precedence over auto-advance.
- Closing the lightbox stops the slideshow.

The defaults come from `slideshow_seconds` and `slideshow_loop` in `[browse]`; runtime changes apply for the current session only.

---

trash Section

Controls the trash/quarantine for destructive file operations. When enabled (the default), `maki delete --remove-files`, `maki dedup`, and the web duplicates page move files into `<catalog>/.trash/` instead of deleting them permanently — recoverable via `maki trash list` / `maki trash restore`.

enabled

- **Type:** boolean
- **Default:** `true`

Set to `false` to delete files permanently everywhere (equivalent to passing `--no-trash` on every deleting command). Derived files (previews, embedding binaries, face crops) are never trashed — they are regenerable.

retention_days

- **Type:** unsigned 32-bit integer
- **Default:** `30`

Default age cutoff for `maki trash empty`: files trashed less than this many days ago are kept unless `--all` is passed.

```
[trash]
enabled = true
retention_days = 30
```

Note: the trash lives on the catalog volume. Removing a file from another (media) volume copies it into the trash before deleting the original, which costs one copy for large files — `--no-trash` is the escape hatch when freeing space is the whole point.

---

history Section

Controls the edit-history journal that backs `maki undo` and `maki history`. When enabled (the default), field-level metadata edits (rating, label, description, name, date, tag add/remove/clear — single and batch) are recorded as undoable operations in `<catalog_root>/history/`.

The journal is **non-authoritative**: it is neither a sidecar (source of truth) nor part of `catalog.db` (derived cache). It survives `rebuild-catalog`, is ignored by `maki doctor`, and may be deleted at any time — losing only the ability to undo past edits.

enabled

- **Type:** boolean
- **Default:** `true`

Set to `false` to disable journaling entirely. No operations are written, and `maki undo` then has nothing to revert.

max_operations

- **Type:** unsigned integer
- **Default:** `200`

How many operations to keep. After each recorded operation the journal is pruned to the newest `max_operations` files; older ones (and any undone counterparts) are deleted. Each operation stores the full before/after state of every asset it touched, so on very large batch edits the files are correspondingly larger — lower this if disk use matters more than deep undo history.

```
[history]
enabled = true
max_operations = 200
```

[writeback] Section

Controls whether metadata edits flow into the `.xmp` recipe files on your storage volumes **automatically** on every change.

enabled

- **Type:** boolean
- **Default:** `false`

This flag governs *automatic* (a.k.a. inline) writeback. It does **not** disable manual writeback — `maki writeback` always works regardless of this setting and is the supported way to keep auto-flush off as a safety net while still pushing staged metadata onto disk on demand.

`enabled`	What happens on rating/label/description/tag edits
`false` *(default)*	Edit lands in YAML sidecar + SQLite catalog. The recipe is flagged `pending_writeback = true`. The `.xmp` file on disk is **not** touched. Run `maki writeback` to flush staged changes whenever you want.
`true`	Edit lands in YAML/SQLite **and** is written to the recipe's `.xmp` immediately (or queued as `pending_writeback` if the recipe lives on an offline volume).

> **Note:** Edits are always tracked. Even with `enabled = false`, the catalog and the recipe's `pending_writeback` flag mark exactly which assets have staged edits — `maki status` surfaces the count, and `maki writeback` (optionally narrowed by query/volume) flushes them. The flag is purely "should every keystroke touch disk, or do I want a single explicit flush button?"

```
[writeback]
enabled = true   # auto-flush on every edit
```

To rematerialise the catalog metadata onto disk for a specific asset set without changing the config, use `maki writeback --all <query>`. That writes every XMP in the matching set whether or not it's flagged pending — useful right after a large catalog-only restructuring (rename, split, rebuild).

mirror_tags

- **Type:** boolean
- **Default:** `false`

When `true`, every `maki writeback` and every web Maintain → Writeback run defaults to **mirror mode**: the XMP's `dc:subject` and `lr:hierarchicalSubject` keyword lists are diffed against the asset's catalog tags and any stale entries are removed. Equivalent to passing `--mirror-tags` on the CLI for every invocation, but without the `--all` requirement (config-driven mirror mode applies to pending recipes only; CLI `--mirror-tags` still requires `--all` for explicit one-off cleanups).

This matters when catalog-only tag changes accumulate (renames, splits, deletions, fix-unicode) before the next flush. The default additive behaviour writes the new tags but leaves the old keywords stranded on disk — a subsequent re-import would absorb them back into the catalog and silently undo the rename. With `mirror_tags = true`, every flush keeps the XMP in lock-step with the catalog.

`mirror_tags`	What `maki writeback` does to XMP keywords
`false` *(default)*	Additive: catalog tags are written, XMP keywords from other sources are preserved. Safe for mixed-tool workflows where Lightroom / CaptureOne / Bridge may add their own keywords MAKI shouldn't strip.
`true`	Mirror: catalog is the source of truth — XMP keywords not in the catalog are removed alongside the new tag write. Recommended when MAKI is the only writer of keywords on the affected files.

```
[writeback]
mirror_tags = true   # every writeback strips stale XMP keywords
```

The CLI `--mirror-tags` flag and this config flag are OR'd. A user with `mirror_tags = true` doesn't need the CLI flag; a user with the config off can still trigger a one-off destructive cleanup via `maki writeback --all --mirror-tags`.

sidecar_naming

- **Type:** string (`"stem"` or `"full"`)
- **Default:** `"stem"`

How MAKI names the `.xmp` files it *creates* – `maki create-sidecars`, `maki relocate --create-sidecars`, and the `create_sidecars` auto-create path below. It does not rename anything that already exists, and import accepts both forms regardless of this setting.

`sidecar_naming`	File next to `DSC_1234.NEF`	Read by
`"stem"` *(default)*	`DSC_1234.xmp`	Capture One, Lightroom, Bridge, Photo Mechanic
`"full"`	`DSC_1234.NEF.xmp`	darktable (and MAKI before v4.11)

With `"stem"`, a RAW+JPEG pair in one asset shares a single `DSC_1234.xmp`, which is also how Capture One and Lightroom treat the pair. Choose `"full"` only when darktable is the tool that will read the sidecars.

```toml
[writeback]
sidecar_naming = "full"   # darktable-style DSC_1234.NEF.xmp
```

create_sidecars

- **Type:** boolean
- **Default:** `false`

When `true` **and** `enabled = true`, MAKI creates the `.xmp` sidecar itself whenever an asset ends up with metadata but no XMP recipe:

- **At import** (`maki import`, `maki watch`, the web import dialog): an asset that leaves the import with metadata — tags from `--add-tag`, the dialog's tag field, or `[import] auto_tags`; keywords or a rating from embedded XMP — gets its sidecar in the same run. The import summary reports `N sidecar(s) created`. An `.xmp` imported alongside the file in the same run counts as the recipe, so nothing is created for it.
- **On edit** (rating, label, description, tags, from CLI, shell or web): the first edit of a sidecar-less asset creates the file on the spot.

The new file carries the asset's complete current metadata and is registered as a recipe. From then on it behaves like any imported sidecar: later edits flow into it, `maki refresh` reads changes made by other tools.

This closes the gap where auto-writeback silently had nothing to write to: RAW files imported without sidecars gained tags and ratings in the catalog only. Only variant locations on online volumes are covered. A sidecar that already exists on disk but has not been imported (typically one Capture One wrote after the MAKI import) is left untouched – run `maki import` on it first.

Has no effect while `enabled = false`: there would be nothing to write into the new file. For a one-off pass over the existing catalog use `maki create-sidecars --apply`.

```toml
[writeback]
enabled = true
create_sidecars = true   # new sidecar on first edit of a sidecar-less asset
```

[cli] Section

Default global flags. Values are OR'd with command-line flags — setting `log = true` here is equivalent to always passing `--log`.

log

- **Type:** boolean
- **Default:** `false`

Enable per-file progress logging (`--log`) by default on all commands.

time

- **Type:** boolean
- **Default:** `false`

Show elapsed time (`--time`) by default after every command.

verbose

- **Type:** boolean
- **Default:** `false`

Enable verbose output (`--verbose`) by default.

```
[cli]
log = true
time = true
```

[audio] Section

External analyzers for `maki audio analyze` (musical key and tempo detection). The tools are shelled out per file — same policy as dcraw/ffmpeg: probed at run time, warned about when missing, never bundled. Commands are configurable so venv-installed tools or substitutes work.

key_command

- **Type:** string
- **Default:** `keyfinder-cli`

Command that prints an audio file's musical key to stdout, invoked as `<command> <file>`. The default, keyfinder-cli, emits standard notation (`Am`, `Eb`, …); empty output with exit 0 is treated as "no key detected" (silence), not an error. Any substitute with the same contract works.

bpm_command

- **Type:** string
- **Default:** `beat_this`

Command that writes beat timestamps for an audio file, invoked as `<command> <file> -o <output-dir>`. The default, beat_this, writes `.beats` files with one `<seconds> <beat-number>` line per beat. MAKI derives BPM from the median inter-beat interval, so occasional missed or extra beats don't skew the result. Any beat tracker with the same output contract works.

```
[audio]
# Point at a venv installation
key_command = "keyfinder-cli"
bpm_command = "/Users/me/.venvs/beats/bin/beat_this"
```

Full Example

A complete `maki.toml` with all options set and annotated:

```
# Default volume for import when auto-detection is ambiguous.
# Find volume UUIDs with: maki volume list --json
default_volume = "550e8400-e29b-41d4-a716-446655440000"

[preview]
# Maximum pixel size of the longest edge for thumbnails.
max_edge = 1200
# Output format: "jpeg" (lossy) or "webp" (lossless).
```

```
format = "jpeg"
# JPEG quality (1-100). Ignored for WebP.
quality = 90
# Smart preview: maximum pixel size of the longest edge.
smart_max_edge = 2560
# Smart preview: JPEG quality (1-100).
smart_quality = 85
# Generate smart previews on first web request.
generate_on_demand = true

[serve]
# Web UI port. Override with: maki serve --port 9090
port = 8080
# Bind address. Use "0.0.0.0" to allow network access.
bind = "127.0.0.1"
# Assets per browse-grid page.
per_page = 60
# Safe-sharing mode: reject every mutating request (also: maki serve --read-only).
read_only = false
# HTTP Basic Auth — enforced only when both are set. Prefer the
# MAKI_SERVE_PASSWORD environment variable over a password in this file.
# username = "thomas"
# password = "secret"
# Stroll page: initial neighbor count and slider maximum.
stroll_neighbors = 12
stroll_neighbors_max = 25
# Stroll page: initial fan-out count and slider maximum.
stroll_fanout = 5
stroll_fanout_max = 10
# Stroll page: candidate pool size for Discover mode.
stroll_discover_pool = 80

[import]
# Glob patterns to exclude during import (matched against filenames).
exclude = [
    "Thumbs.db",
    ".DS_Store",
    "*.tmp",
    "*.bak",
    "desktop.ini",
]
# Tags automatically applied to every new asset.
auto_tags = ["inbox", "unreviewed"]
# Generate smart previews during import.
smart_previews = true
# Generate embeddings for visual similarity search during import (Pro).
embeddings = true
# Generate VLM descriptions during import (requires running Ollama or compatible endpoint).
descriptions = true

# Named import profiles (maki import --profile <name>); unset fields inherit from [import].
[import.profiles.cards]
auto_tags = ["inbox"]
```

```
smart_previews = false
# File-type groups, not globs: images, video, audio, xmp, documents,
# captureone, rawtherapee, dxo, on1 (see [import.profiles.<name>] above).
include = ["captureone"]
skip = ["audio", "documents"]

[dedup]
# Default path substring for --prefer (keep files whose path contains this).
prefer = "Selects"

[verify]
# Skip files verified within this many days (incremental verify).
max_age_days = 30

[contact_sheet]
# Layout preset: "dense", "standard", "large".
layout = "standard"
# Paper size: "a4", "letter", "a3".
paper = "a4"
# Comma-separated metadata fields below each thumbnail.
fields = "filename,date,rating"
# Page margin in mm.
margin = 10.0
# JPEG quality for page images (1-100).
quality = 90
# Color label display: "border", "dot", "none".
label_style = "border"
# Copyright text for page footer (optional; omit for no footer).
copyright = "© 2026 Thomas Herrmann"

# AI auto-tagging settings (Pro).
[ai]
model = "siglip-vit-b16-256"
threshold = 0.3
# labels = "my-labels.txt"
model_dir = "~/.maki/models"
prompt = "a photograph of {}"
# GPU acceleration (included automatically on macOS Pro builds).
# execution_provider = "auto"
# Face recognition: clustering similarity threshold and minimum detection confidence.
face_cluster_threshold = 0.35
face_min_confidence = 0.7
# Default result count for the text: search filter.
text_limit = 50

# XMP writeback: disabled by default for safety.
# Enable to write rating/tags/label/description back to .xmp files on disk.
[writeback]
enabled = false
# Reconcile XMP keyword lists against the catalog on every writeback
# (removes stale keywords; default is additive only).
mirror_tags = false
# Name for sidecars MAKI creates: "stem" = DSC_1234.xmp (Capture One,
```

```
# Lightroom), "full" = DSC_1234.NEF.xmp (darktable).
sidecar_naming = "stem"
# With enabled = true: create the sidecar on first edit of an asset
# that has none yet (otherwise run `maki create-sidecars --apply`).
create_sidecars = false

# VLM image description settings (Pro).
[vlm]
endpoint = "http://localhost:11434"
model = "qwen2.5vl:3b"
max_tokens = 500
timeout = 300
temperature = 0.7
mode = "describe"
# Concurrent requests (for servers that handle parallelism).
concurrency = 1
# Downscale images to this longest edge before sending (0 = no resizing).
max_image_edge = 0
# Models offered in web UI dropdown (empty = only default model, no dropdown).
# models = ["qwen2.5vl:3b", "moondream"]
# prompt = "Describe this photograph concisely."
# Sampling parameters (0 = not set, use server default).
# num_ctx = 4096
# top_p = 0.9
# top_k = 40
# repeat_penalty = 1.1

# Per-model VLM overrides (optional).
# [vlm.model_config."moondream:latest"]
# max_tokens = 200
# temperature = 0.1

[browse]
# Default filter applied to browse/search/stroll views (standard search syntax).
# default_filter = "-tag:rejected"
# Lightbox slideshow: seconds per image, and wrap around at the end.
slideshow_seconds = 5
slideshow_loop = true
# Restore the last browse filter on a bare "/" visit.
remember_latest_filter = true

[trash]
# Move deleted files into <catalog>/.trash/ instead of unlinking them.
enabled = true
# Default age cutoff (days) for `maki trash empty`.
retention_days = 30

[history]
# Journal metadata edits under <catalog>/history/ for `maki undo` / `maki history`.
enabled = true
# Keep the newest N operations.
max_operations = 200
```

```toml
[cli]
# Default global flags (OR'd with the command line).
log = false
time = false
verbose = false

[group]
# Regex identifying session-root directories for auto-group and the scattered: filter.
session_root_pattern = '^\d{4}-\d{2}'

[watch]
# Poll interval in seconds for `maki watch` (minimum 2).
poll_seconds = 30
# Extra exclude globs on top of [import] exclude.
exclude = ["*.partial"]

[audio]
# External analyzers for `maki audio analyze`.
key_command = "keyfinder-cli"
bpm_command = "beat_this"
```

Minimal Examples

Preview-only: larger thumbnails in WebP

```toml
[preview]
max_edge = 1600
format = "webp"
```

Import-only: exclude system files and auto-tag

```toml
[import]
exclude = [".DS_Store", "Thumbs.db", "*.tmp"]
auto_tags = ["2026-import"]
```

Serve-only: custom port for LAN access

```toml
[serve]
port = 9090
bind = "0.0.0.0"
```

Defaults Summary

When a field is absent from `maki.toml`, these defaults apply:

Field	Default
default_volume	none
preview.max_edge	800
preview.format	"jpeg"
preview.quality	85
preview.smart_max_edge	2560
preview.smart_quality	85
preview.generate_on_demand	false
serve.port	8080
serve.bind	"127.0.0.1"
serve.per_page	60
serve.stroll_neighbors	12
serve.stroll_neighbors_max	25
serve.stroll_fanout	5
serve.stroll_fanout_max	10
serve.stroll_discover_pool	80
serve.read_only	false
serve.username / serve.password	none (no auth)
import.exclude	[]
import.auto_tags	[]
import.smart_previews	false
import.embeddings	false
import.descriptions	false
import.profiles	none
dedup.prefer	none
verify.max_age_days	none
contact_sheet.layout	"standard"
contact_sheet.paper	"a4"
contact_sheet.fields	"filename,date,rating"
contact_sheet.margin	10.0
contact_sheet.quality	90
contact_sheet.label_style	"border"
contact_sheet.copyright	none
ai.model	"siglip-vit-b16-256"
ai.threshold	0.1
ai.labels	none
ai.model_dir	"~/.maki/models"
ai.prompt	"a photograph of {}"

Field	Default
ai.execution_provider	"auto"
ai.face_cluster_threshold	0.35
ai.face_min_confidence	0.7
ai.text_limit	50
vlm.endpoint	"http://localhost:11434"
vlm.model	"qwen2.5vl:3b"
vlm.max_tokens	500
vlm.prompt	none (built-in)
vlm.mode	"describe"
vlm.temperature	0.7
vlm.timeout	300
vlm.concurrency	1
vlm.max_image_edge	0 (no resizing)
vlm.models	[] (empty)
vlm.num_ctx	0 (server default)
vlm.top_p	0.0 (server default)
vlm.top_k	0 (server default)
vlm.repeat_penalty	0.0 (server default)
vlm.model_config	none
writeback.enabled	false
writeback.mirror_tags	false
browse.default_filter	none
browse.slideshow_seconds	5
browse.slideshow_loop	true
browse.remember_latest_filter	true
trash.enabled	true
trash.retention_days	30
history.enabled	true
history.max_operations	200
cli.log / cli.time / cli.verbose	false
group.session_root_pattern	'^\d{4}-\d{2}'
watch.poll_seconds	30
watch.exclude	[]
audio.key_command	"keyfinder-cli"
audio.bpm_command	"beat_this"

Related Topics

- Setup (User Guide) – creating a catalog with `maki init`
- CLI Conventions – global flags and catalog discovery
- Search Filters Reference – search query syntax

Data Model

This page documents every entity in the MAKI data model, their fields, relationships, and storage mechanisms.

Entities

Asset

The top-level entity. An Asset represents a single logical media item – "photo of sunset at the beach" – regardless of how many physical files exist for it.

Field	Type	Description
`id`	UUID	Primary key. Deterministic UUID v5 derived from the content hash of the first variant. Same content always produces the same asset ID.
`name`	Option<String>	User-assigned display name. When absent, the UI shows `original_filename` as a fallback.
`original_filename`	String	Filename of the primary variant at import time (e.g. `DSC_4521.NEF`). Used as display fallback and for stem-based grouping.
`asset_type`	AssetType	One of: `image`, `video`, `audio`, `document`, `other`. Inferred from file extension at import.
`description`	Option<String>	Free-text description. Extracted from XMP `dc:description` during import, or set manually.
`tags`	Vec<String>	Keyword list. Merged from XMP `dc:subject`, embedded XMP, and manual tagging. Deduplicated.
`tag_sources`	BTreeMap<String, TagSource>	Provenance per tag value: `user`, `xmp-import`, `auto-tag`, or `vlm`. A tag absent from the map is `user`. Only tags with a machine source are stored, so pre-provenance sidecars stay byte-identical. Mutated only through `Asset::add_tags_with_source` / `remove_tags` / `rename_tag_value`; mirrored in the SQLite `tag_sources` column (schema v10) and checked by `maki doctor`.

Field	Type	Description
`rating`	Option<u8>	Star rating, 1–5. Extracted from XMP `xmp:Rating` during import, or set manually.
`color_label`	Option<String>	One of 7 canonical colors: Red, Orange, Yellow, Green, Blue, Pink, Purple. Extracted from XMP `xmp:Label`, or set manually. Stored as title-case English name.
`created_at`	DateTime<Utc>	Creation timestamp. Preferentially from EXIF `DateTimeOriginal`, falling back to filesystem modification time.
`face_scan_status`	Option<String>	`None` = never scanned for faces; `"done"` = scan completed (whether or not faces were found). Keeps zero-face assets from being re-scanned after a `rebuild-catalog`. *(Pro)*
`preview_rotation`	Option<u16>	Manual preview rotation override in degrees (0/90/180/270).
`preview_variant`	Option<String>	Content hash of a user-chosen preview representative, overriding the Display Priority algorithm.
`variants`	Vec<Variant>	The physical files belonging to this asset (in YAML sidecar).
`recipes`	Vec<Recipe>	Processing sidecars attached to this asset's variants (in YAML sidecar).

Denormalized columns (SQLite only, computed at write time to avoid expensive JOINs):

Column	Type	Description
`best_variant_hash`	String	Content hash of the best display variant (see Display Priority). Used for the browse grid JOIN.
`primary_variant_format`	String	Identity format of the asset. Prefers Original+RAW, then Original+any, then best variant's format. Shown on browse cards (e.g. "NEF").
`variant_count`	Integer	Number of variants. Shown as a badge on browse cards (e.g. "3v").
`face_count`	Integer	Number of detected faces. Shown as a badge on browse cards. *(Pro)*
`stack_id`	Option<UUID>	Foreign key to the Stack this asset belongs to. `None` if unstacked.
`stack_position`	Option<Integer>	Position within the stack (0 = pick). `None` if unstacked.
`latitude`, `longitude`	Option<Real>	GPS position lifted from variant metadata (map view, `geo:` filter).
`preview_rotation`	Option<Integer>	Mirror of the sidecar field (schema v3).

Column	Type	Description
`preview_variant`	Option<String>	Mirror of the sidecar field (schema v3).
`duration_seconds`	Option<Real>	Media duration for audio and video (`duration:` filter, duration badges). Added as `video_duration` in schema v4, renamed in v11 when audio started filling it.
`video_codec`	Option<String>	Video codec name (`codec:` filter, schema v5).
`face_scan_status`	Option<String>	Mirror of the sidecar field (schema v7).
`leaf_tag_count`	Integer	Number of leaf tags (`tagcount:` filter, schema v8).
`tag_sources`	Text (JSON)	Mirror of the sidecar map (schema v10).
`audio_sample_rate`, `audio_channels`, `audio_bitrate`	Option<Integer>	Typed audio properties from variant metadata (schema v11).
`audio_key`	Option<String>	Musical key (`key:` filter; from `maki audio analyze`, schema v11).
`audio_bpm`	Option<Real>	Tempo (`bpm:` filter; from `maki audio analyze`, schema v11).

The variant-related columns are updated by `insert_asset()`, `update_denormalized_variant_columns()`, and `fix_roles`. The stack columns are updated by `StackStore` operations, `face_count` by `FaceStore::update_face_count`. All are backfilled during schema migration and rebuilt by `rebuild-catalog`, with one exception: the v6→v7 migration does not rewrite every sidecar to stamp `face_scan_status` (too slow on large catalogs); `rebuild-catalog` stamps `done` on assets that have face records, and `faces detect` writes the flag for every asset it touches from then on.

Variant

A concrete file belonging to an Asset. A RAW file, its JPEG conversion, and a high-res TIFF export are three Variants of the same Asset.

Field	Type	Description
`content_hash`	String	Primary key. SHA-256 hash of the file contents. The same file always produces the same hash regardless of where it is stored.
`asset_id`	UUID	Foreign key to the parent Asset.
`role`	VariantRole	Purpose within the asset group: `original`, `alternate`, `processed`, `export`, or `sidecar`. See Variant Roles.
`format`	String	Lowercase file extension without dot (e.g. `nef`, `jpg`, `tif`, `mp4`).
`file_size`	u64	File size in bytes.
`original_filename`	String	Filename at import time (e.g. `DSC_4521.NEF`).

Field	Type	Description
`source_metadata`	HashMap<String, String>	Key-value pairs from EXIF and XMP extraction (camera model, lens, GPS coordinates, creator, rights, etc.).
`locations`	Vec<FileLocation>	Where this file physically exists on disk (in YAML sidecar; stored in a separate `file_locations` table in SQLite).

Indexed metadata columns (SQLite only, extracted from `source_metadata` for fast filtering):

Column	Type	Description
`camera_model`	String	Camera body (e.g. "NIKON Z 9")
`lens_model`	String	Lens (e.g. "NIKKOR Z 50mm f/1.2 S")
`focal_length_mm`	Real	Focal length in millimeters
`f_number`	Real	Aperture f-number
`iso`	Integer	ISO sensitivity
`image_width`	Integer	Image width in pixels
`image_height`	Integer	Image height in pixels

FileLocation

A pointer to where a Variant physically lives on disk. A single Variant can have multiple FileLocations – copies on different drives, backups, archives.

Field	Type	Description
`id`	Integer	Primary key (auto-increment, SQLite only).
`content_hash`	String	Foreign key to the parent Variant.
`volume_id`	UUID	Foreign key to the Volume where the file resides.
`relative_path`	String	Path relative to the volume's mount point (e.g. `Capture/2026-02-22/DSC_4521.NEF`).
`verified_at`	Option<DateTime<Utc»	Timestamp of the last successful integrity check via `maki verify`. `None` if never verified.

Recipe

A processing sidecar file attached to a Variant. Each Recipe record represents one physical file on one volume. When the same XMP file exists on multiple volumes (e.g. backup copies), there are multiple Recipe records with the same `content_hash` but different locations — conceptually one recipe with multiple locations, mirroring how Variants work. The detail page and stats page group recipes by `(variant_hash, content_hash)` to reflect this: "1 recipe, 3 locations" rather than "3 recipes".

When importing, if a recipe's content hash is already known for the asset (from any volume), the recipe file is tracked for location purposes but its metadata is **not re-merged** — this prevents stale XMP data from backup copies from overwriting curated tags or ratings.

Field	Type	Description
`id`	UUID	Primary key.
`variant_hash`	String	Foreign key to the parent Variant.
`software`	String	Processing tool identifier: `XMP`, `CaptureOne`, `RawTherapee`, `DxO`, `ON1`.
`recipe_type`	RecipeType	Either `sidecar` (external file) or `embedded_export`.
`content_hash`	String	SHA-256 hash of the recipe file. Updated when the file changes on disk.
`volume_id`	UUID	Foreign key to the Volume where the recipe file resides.
`relative_path`	String	Path relative to the volume's mount point.
`verified_at`	Option<DateTime<Utc»	Last verification timestamp.
`pending_writeback`	bool	`true` when metadata was edited in MAKI but the `.xmp` file could not be updated yet (volume offline, or `[writeback] enabled = false`). Cleared by a successful `maki writeback`. In SQLite since schema v2.

Supported recipe file extensions:

Extension	Software
`.xmp`	XMP (Lightroom, CaptureOne, Adobe)
`.cos`	CaptureOne settings
`.cot`	CaptureOne output
`.cop`	CaptureOne process
`.pp3`	RawTherapee
`.dop`	DxO PhotoLab
`.on1`	ON1 Photo RAW

Volume

A registered storage device. Volumes give MAKI a stable reference to storage that may come and go (external drives, network shares).

Field	Type	Description
`id`	UUID	Primary key (random UUID v4).
`label`	String	Human-readable name (e.g. "Photos SSD", "Archive NAS").
`mount_point`	PathBuf	Filesystem path where the volume is mounted (e.g. `/Volumes/Photos`).
`volume_type`	VolumeType	One of: `local`, `external`, `network`.

Field	Type	Description
`purpose`	Option<VolumePurpose>	Logical role: `media` (transient source — memory cards), `working` (active editing), `archive` (long-term primary), `backup` (redundancy), `cloud` (sync folder). Optional — unclassified if not set. Used by duplicate analysis and backup coverage commands. Media volumes excluded from backup coverage.
`is_online`	bool	Computed at runtime – `true` if `mount_point` exists on disk. Not persisted.

Collection

A manually curated list of assets (static album). Backed by both SQLite (for fast queries) and `collections.yaml` (for persistence across catalog rebuilds).

Field	Type	Description
`id`	UUID	Primary key.
`name`	String	Unique human-readable name (e.g. "Portfolio", "Client Deliverables").
`description`	Option<String>	Optional description text.
`created_at`	DateTime<Utc>	When the collection was created.
`asset_ids`	Vec<String>	Ordered list of asset UUIDs (in YAML). In SQLite, this is a separate `collection_assets` join table with `(collection_id, asset_id, added_at)`.

Stack

A lightweight anonymous group of assets for visually related images (burst shots, bracketing sequences, similar scenes). In the browse grid, stacked assets are collapsed to show only the "pick" image with a count badge.

Field	Type	Description
`id`	UUID	Primary key (random UUID v4).
`created_at`	DateTime<Utc>	When the stack was created.
`asset_ids`	Vec<String>	Ordered list of asset UUIDs. Index 0 is the pick (displayed in browse grid).

Constraints: - Each asset can belong to at most one stack. - A stack must have at least 2 members. Removing members that would leave fewer than 2 causes automatic dissolution. - Stack membership is denormalized onto the `assets` table as `stack_id` (FK to the stack) and `stack_position` (integer, 0 = pick). These columns enable efficient filtering (`stacked:true/false`) and stack collapsing in browse queries without joining a separate table.

Storage: Stacks are persisted in `stacks.yaml` at the catalog root (alongside `collections.yaml` and `searches.toml`). The SQLite `stacks` table and the `stack_id`/`stack_position` columns on `assets` are derived from this file and rebuilt by `rebuild-catalog`.

SavedSearch

A named query (smart album) stored in `searches.toml`. Re-evaluated every time it is run, so results update automatically as the catalog changes.

Field	Type	Description
`name`	String	Unique identifier.
`query`	String	Search filter string in the same syntax as `maki search` (e.g. `type:image tag:landscape rating:4+`).
`sort`	Option<String>	Sort order (e.g. `date_desc`, `name_asc`). Omitted means default (`date_desc`).
`favorite`	bool	Whether the search is shown as a chip on the browse page.

Embedding *(Pro)*

A stored SigLIP image embedding for an asset, produced by `maki embed`, `maki import --embed`, or as a side effect of `maki auto-tag`; it powers the `similar:` filter, `maki search --image`, the stroll page, and `maki auto-stack`.

Field	Type	Description
`asset_id`	String	Foreign key to the parent Asset. Primary key together with `model`.
`model`	String	Model identifier (default: `siglip-vit-b16-256`). Embeddings from different models are never compared; switching models only generates the missing rows.
`embedding`	Blob	float32 vector, stored as little-endian binary. The dimension is per model: 768 for `siglip-vit-b16-256` and `siglip2-base-256-multi` (3072 bytes), 1024 for `siglip-vit-l16-256` and `siglip2-large-256-multi` (4096 bytes).

Storage overhead: ~3–4 KB per asset and model. For 100,000 assets with the default model: ~300 MB in SQLite, plus a binary copy under `embeddings/<model>/` for rebuild resilience.

In-memory index: For fast similarity search, the web server loads all embeddings into an `EmbeddingIndex` — a contiguous `Vec<f32>` buffer — on first query. Search uses dot product (SigLIP embeddings are L2-normalized) with a min-heap for top-K selection. At 100k assets, search completes in <10ms. The index is updated in-place when new embeddings are stored.

Opportunistic storage: Embeddings are stored not only by `maki auto-tag` and `maki embed`, but also opportunistically by the web UI "Suggest tags" and batch "Auto-tag" endpoints. This means using AI features in the web UI gradually builds up the similarity search index.

Face *(Pro)*

A detected face within an asset image, with bounding box, confidence, recognition embedding, and optional person assignment.

Field	Type	Description
`id`	UUID	Primary key.
`asset_id`	UUID	Foreign key to the parent Asset.
`person_id`	Option<UUID>	Foreign key to the assigned Person. `None` if unassigned.
`bbox_x`	f32	Bounding box X position (normalized 0–1).
`bbox_y`	f32	Bounding box Y position (normalized 0–1).
`bbox_w`	f32	Bounding box width (normalized 0–1).
`bbox_h`	f32	Bounding box height (normalized 0–1).
`confidence`	f32	Detection confidence score (0–1).
`embedding`	Blob	512-dimensional float32 ArcFace vector (2048 bytes), stored as little-endian binary. The face is aligned to a canonical 112×112 template via a 5-point similarity transform before embedding.
`recognition_model`	Option<String>	Identifier of the ArcFace variant that produced this embedding (e.g. `arcface-resnet100-fp32-aligned-v2`). Added in schema v6. Clustering only mixes embeddings from the same model — older ones are skipped with a warning until re-embedded via `maki faces detect --force`.
`created_at`	DateTime<Utc>	When the face was detected.

The 150×150 JPEG crop thumbnail is not a field: its path is derived from the face ID as `faces/<2-char prefix>/<face_id>.jpg` under the catalog root. Storage overhead: ~2 KB per face (embedding + metadata). Face crops: ~5–15 KB each as JPEG.

Person *(Pro)*

A named or unnamed person group linking detected faces across assets.

Field	Type	Description
`id`	UUID	Primary key.
`name`	Option<String>	User-assigned name. `None` for unnamed clusters.
`representative_face_id`	Option<UUID>	Foreign key to the face used as the person's thumbnail.
`created_at`	DateTime<Utc>	When the person record was created.

Variant Roles

Each Variant carries a `role` that describes its purpose within the asset group.

Role	Meaning	Examples
Original	Camera source file. Each asset should have exactly one.	NEF, ARW, CR3, in-camera JPEG
Alternate	Secondary variant from grouping (e.g., JPEG paired with RAW original).	In-camera JPEG grouped with RAW
Processed	An edited or intermediate version, not straight from camera.	PSD, layered TIFF, edited DNG
Export	A derivative output produced by an editing tool.	Resized JPEG, web TIFF, final deliverable
Sidecar	A non-media sidecar file imported as a variant.	Embedded metadata files

When assets are merged via `maki group` or `maki auto-group`, donor variants with the `original` role are automatically re-roled to `alternate` to avoid having multiple originals in one asset.

Display Priority

The preview selection algorithm determines which variant is shown in the browse grid and asset detail page. The scoring follows this priority:

1. **Role** (highest weight): Export (300) > Processed (200) > Original (100) > Alternate (50) > Sidecar (0)
2. **Format bonus** (+50): Standard image formats (`jpg`, `jpeg`, `png`, `tiff`, `tif`, `webp`) are preferred over RAW
3. **File size tiebreak**: Larger files score slightly higher (up to +49 points)

This means a JPEG export is preferred over a RAW original for display, showing your best-quality deliverable in the browse grid rather than the camera file.

The `best_variant_hash` denormalized column caches this computation so the browse grid can join directly to the best variant without evaluating all variants per asset.

Storage

maki uses a dual-storage architecture. Neither tier alone is sufficient; together they provide both robustness and performance.

YAML Sidecar Files (source of truth)

One `.yaml` file per Asset, stored at `metadata/<id-prefix>/<id>.yaml` within the catalog directory. Writes take an advisory lock on `metadata/.write.lock` so concurrent MAKI processes cannot interleave sidecar writes. Contains the complete Asset record: metadata, all Variants (with their FileLocations and source_metadata), and all Recipes. Human-readable, diffable, and version-control friendly.

```
catalog/
  metadata/
    3a/
```

```
    3a7b1e02-4fd2-4a6b-9c1d-e75a0bf3284c.yaml
  f1/
    f1c8d9e0-...yaml
```

SQLite Catalog (derived cache)

A single `catalog.db` file providing fast indexed queries. Contains denormalized columns for efficient browse-grid rendering. The catalog is always rebuildable from the YAML sidecars via `maki rebuild-catalog` – it is a performance optimization, not a source of truth.

Tables: `assets`, `variants`, `file_locations`, `volumes`, `recipes`, `collections`, `collection_assets`, `stacks`, `embeddings` (created in every build; only filled by Pro), `schema_version` (single row, see Schema Migrations), `assets_fts` (FTS5 trigram index over name, filename, description and source metadata, kept current by six triggers on `assets` and `variants` – schema v9), `assets_tags_fts` (FTS5 trigram copy of the `tags` column that serves the `tag:` filter's LIKE/GLOB patterns from its index, three triggers on `assets` – schema v12), and `faces`, `people` (*(Pro)* builds only)

Performance indexes (created automatically via schema migrations):

- `variants(asset_id)`, `variants(format)`, `variants(camera_model)`, `variants(lens_model)`, `variants(iso)`, `variants(focal_length_mm)` — variant lookups and filter queries
- `file_locations(content_hash)`, `file_locations(volume_id)` — join and volume filter performance
- `assets(created_at)`, `assets(best_variant_hash)` — sort-by-date and best-variant join
- `assets(stack_id)`, partial `assets(stack_position, created_at DESC) WHERE stack_id IS NOT NULL` — stack collapsing in the browse grid
- partial `assets(latitude, longitude) WHERE latitude IS NOT NULL` — map view and `geo:` filter
- partial `assets(face_count) WHERE face_count > 0`, partial `assets(face_scan_status) WHERE face_scan_status IS NULL` — `faces:` filter and "assets still to scan" lookups
- `assets(leaf_tag_count)` — `tagcount:` filter
- `recipes(variant_hash)` — recipe lookups by variant
- `collection_assets(asset_id)` — collection membership queries
- `faces(asset_id)`, `faces(person_id)` — face lookups per asset and per person *(Pro)*

Schema Migrations

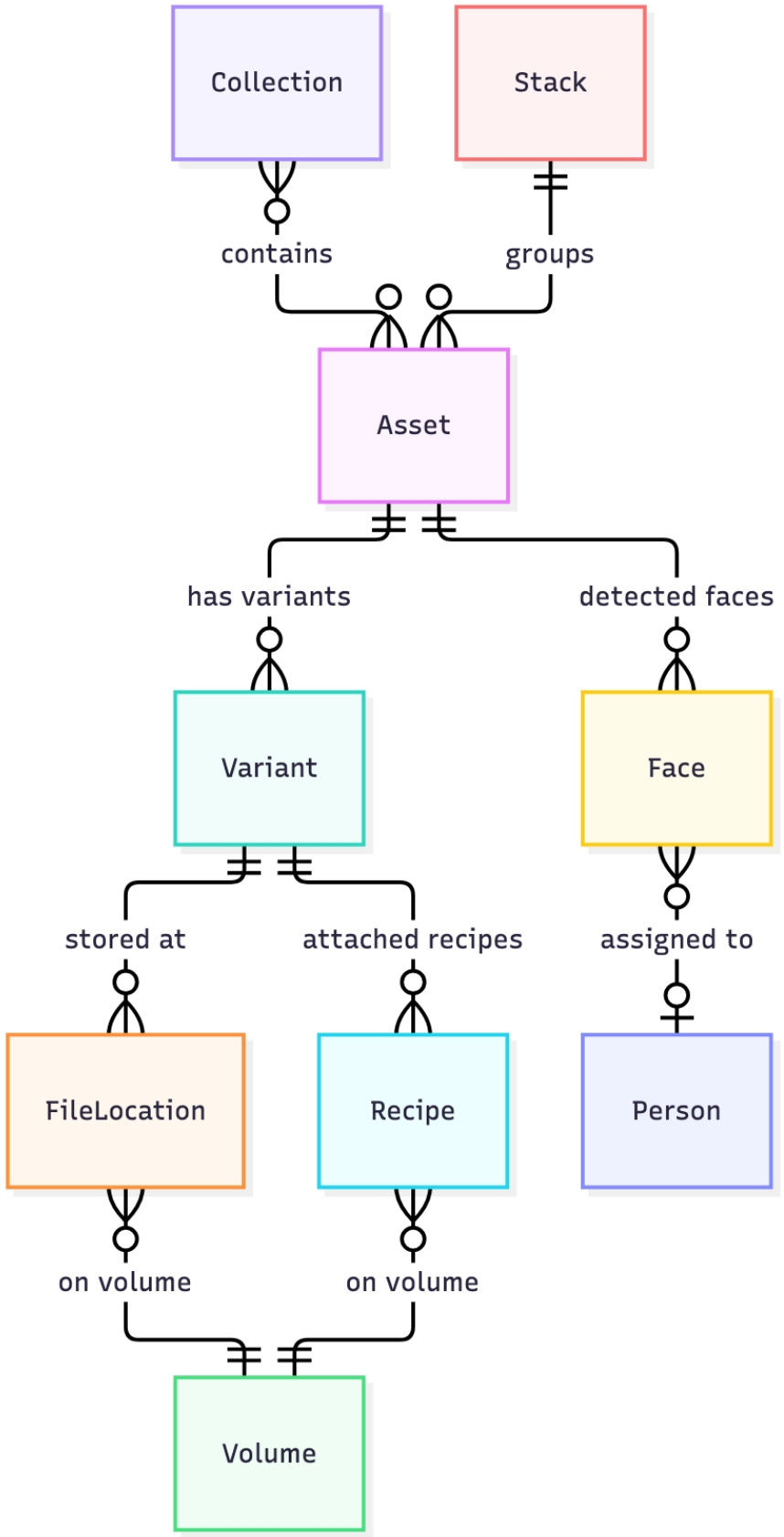
The `schema_version` table stores the catalog's schema version (`SCHEMA_VERSION` in `src/catalog.rs`, currently 11). Every command except `init` and `migrate` checks it at startup with one query and exits with an error if the catalog is older than the binary expects; migrations themselves run only in `maki init`, `maki migrate`, and `maki rebuild-catalog`. Migrations are idempotent `ALTER TABLE ... ADD COLUMN` / `CREATE INDEX IF NOT EXISTS` blocks with guarded backfills, executed only for versions above the stored one.

Other Files

File	Format	Contents
`volumes.yaml`	YAML	Registered volume definitions (id, label, mount_point, type)
`searches.toml`	TOML	Saved search definitions (name, query, sort)
`collections.yaml`	YAML	Collection definitions with ordered asset ID lists
`stacks.yaml`	YAML	Stack definitions with ordered asset ID lists
`maki.toml`	TOML	User configuration (see Configuration)

File	Format	Contents
`vocabulary.yaml`	YAML	Tag vocabulary for auto-tagging, seeded with the built-in default by `maki init`
`.gitignore`	text	Written by `maki init`; excludes the derived files below so a catalog can be version-controlled
`previews/<prefix>/<hash>.jpg`	JPEG	Preview thumbnails keyed by variant content hash (`.webp` when `[preview] format = "webp"`)
`smart-previews/<prefix>/<hash>.jpg`	JPEG	Larger smart previews (same keying)
`embeddings/<model>/<prefix>/<asset_id>.bin`	binary	Image embeddings, one file per asset and model — rebuild source for the `embeddings` table *(Pro)*
`embeddings/arcface/<prefix>/<face_id>.bin`	binary	Face recognition embeddings — rebuild source for `faces.embedding` *(Pro)*
`faces.yaml`, `people.yaml`	YAML	Face and person records — rebuild source for the `faces` / `people` tables *(Pro)*
`faces/<prefix>/<face_id>.jpg`	JPEG	Face crop thumbnails (150×150, *(Pro)*)
`history/<epoch-millis>-<opid>.json`, `history/undone/`	JSON	Edit-history journal for `maki undo` / `maki history`; undone operations move to `undone/`. Independent, non-authoritative, freely deletable
`.trash/`	files	Quarantine for deleted files (`delete --remove-files`, `dedup`, web duplicates page); recoverable via `maki trash`
`metadata/.write.lock`	lock file	Advisory lock serialising sidecar writes across processes

Entity Relationships



Content-Addressable Identity

Every file imported into MAKI is hashed with SHA-256. This hash is the file's identity:

- **Deduplication**: Importing the same file twice (even from different paths or drives) recognizes it as the same content and adds the new location to the existing Variant.
- **Integrity verification**: `maki verify` re-hashes files and compares against stored hashes to detect corruption or bit rot.
- **Transparent relocation**: Moving a file to a different drive does not change its identity. `maki relocate` and `maki update-location` update the catalog path.

Originals (RAW files, camera JPEGs) are immutable – their hash is stable forever. Recipe files are the exception: they are modified by external tools, so MAKI tracks them by location and updates their stored hash when changes are detected.

Previous: Configuration

VLM Model Guide *(Pro)*

This document covers vision-language models (VLMs) compatible with `maki describe`. It lists tested models, hardware requirements, quality notes, and setup instructions for each inference backend.

MAKI tries the **Ollama native API** (`/api/generate`) first, falling back to the **OpenAI-compatible** `/v1/chat/completions` endpoint. Both use base64-encoded images. Any server that implements either API works — Ollama, llama.cpp, vLLM, LM Studio, SGLang, or cloud providers.

> **Model swap is pure config.** Switching VLMs requires zero code changes. Pull the new model with your backend, change `[vlm] model` in `maki.toml` (or pass `--model` on the CLI), restart `maki serve` if it's running. MAKI knows nothing about which model is loaded — it sends a base64 image + prompt to an HTTP endpoint and accepts whatever JSON comes back.

Quick Start

1. Install Ollama (or another backend — see Backends below).
2. Pull a model: `ollama pull qwen2.5vl:3b`
3. Test connectivity: `maki describe --check`
4. Describe assets: `maki describe "description:none" --apply --log`

To switch models, either set `[vlm] model` in `maki.toml` or pass `--model` on the command line.

Should I Upgrade?

The answer is usually "stick with what works." The descriptions you're getting from your current model are good enough for *navigation* — that's the bar `maki describe` is trying to clear, not poet-grade prose. The clearest upgrade paths from popular defaults:

- `gemma3:4b` → `gemma4:e4b` — same effective compute, native multimodal (vs Gemma 3's bolt-on projector), 128K context, native system prompts. Similar latency on equivalent hardware.
- `qwen2.5vl:3b` → `qwen3.5:4b` — the current Qwen generation (natively multimodal, no separate `-VL`) at the same size class; or `qwen3.5:9b` for a clear quality jump. The `qwen3-vl` line (`qwen3-vl:4b`/`:8b`) is the well-trodden previous generation and still an excellent choice if you prefer it.
- `moondream` → `gemma4:e2b` — same speed bracket, much better visual reasoning.

Upgrading helps in specific cases:

- **Generic / repetitive descriptions on similar images.** A 3 B model often falls back to template language ("a beautiful landscape with mountains and a clear sky"). Stepping up to a 7–8 B model usually breaks the pattern.
- **Wrong subject identification.** If the model mistakes a horse for a dog, or a wedding for a concert, a larger model with stronger visual reasoning helps. Verify with a small A/B batch before committing.

- **OCR / text-in-image needs.** If you've been asking the model to read signage, license plates, or document scans, the Qwen-VL family (especially 7 B+) is significantly stronger here than Gemma 3.
- **Multilingual descriptions.** If you want descriptions in German, Japanese, etc., Qwen models cover 100+ languages out of the box; Gemma is English-centric.

Stay on your current model if:

- **Descriptions are useful for search and you don't read them word-by-word.** The marginal quality gain from 4 B → 8 B costs you ~2–3× the inference time on the same hardware.
- **You're batch-processing 1000+ images.** Doubling latency per image matters at scale.
- **Hardware is tight.** A 4 B model running comfortably in RAM beats an 8 B model swapping to disk by every metric.

Testing a Candidate Model on Your Own Photos

The honest way to decide is a small A/B on photos *you* care about. The web UI's per-asset model selector and the CLI's `--model` flag make this trivial:

```
# 1. Pull the candidate model.
ollama pull qwen3-vl:8b

# 2. Add it to your maki.toml so it appears in the web UI's per-asset
#    Describe dropdown without disturbing your default.
#
# [vlm]
# model = "gemma3:4b"                # your current default stays
# models = ["gemma3:4b", "qwen3-vl:8b"]  # both available in the dropdown

# 3. Pick 20 representative assets, generate descriptions with each
#    model, compare side-by-side. `--force` overwrites the existing
#    description on each iteration so you can re-run.
maki describe "rating:5 date:2026-05" --model gemma3:4b --apply --force --log
maki describe "rating:5 date:2026-05" --model qwen3-vl:8b --apply --force --log
```

Or use the web UI: open an asset's detail page, click **Describe** with the model selector set to one candidate, then re-run with the other. The descriptions overwrite each other so you're always seeing the latest.

Tested Models

All timings measured on Apple M3 Pro (18 GB) with Ollama, using preview images (~800px) at default Q4 quantization. Your results will vary with hardware, image size, and quantization. Verify model availability on your backend's library page (e.g. `ollama.com/library/<name>`) — model rosters move faster than these docs.

Recommended for Photography

Model	RAM Download	Speed	Notes
Qwen2.5-VL 3B `qwen2.5vl:3b`	**3 GB** (2.0 GB)	8–12s	**Default.** Best balance of speed, quality, and resource use. Very good quality.
Gemma 4 E4B `gemma4:e4b`	**10 GB** (9.6 GB)	10–15s	**Recommended upgrade from Gemma 3 4B.** Native multimodal, 128K context, native system prompts.

Model	RAM Download	Speed	Notes
Qwen3-VL 8B `qwen3-vl:8b`	**6 GB** (5.2 GB)	15–20s	Excellent quality, noticeably better than 2.5-VL. Previous-gen; see Qwen3.5/3.6 below for the current line.
Qwen3-VL 4B `qwen3-vl:4b`	**4 GB** (2.8 GB)	10–15s	Good step up from Qwen2.5-VL 3B without much extra RAM. Very good quality.
Qwen2.5-VL 7B `qwen2.5vl:7b`	**6 GB** (4.7 GB)	20–36s	Proven, widely tested. Excellent quality.
Gemma 3 4B `gemma3:4b`	**4 GB** (3.3 GB)	10–15s	Strong reasoning, good at scene understanding. Very good quality.

Budget / Batch Processing

Model	RAM Download	Speed	Notes
Moondream 2B `moondream`	**2 GB** (1.7 GB)	3–5s	Fastest option. Good for bulk first passes before refining with a larger model.
Gemma 4 E2B `gemma4:e2b`	**8 GB** (7.2 GB)	6–10s	Smaller Gemma 4 variant; faster than E4B while keeping the new generation's prompting + 128K context.

SmolVLM was previously listed here; it has been removed from the Ollama library (`ollama pull smolvlm` no longer resolves — only community-namespaced re-uploads remain), and Moondream / Gemma 4 E2B cover the same niche. Moondream 3 (a 9B-MoE preview with much stronger visual reasoning) exists on HuggingFace but is not in the Ollama library yet.

Large / High Quality

Model	RAM Download	Speed	Notes
Qwen3-VL 32B `qwen3-vl:32b`	**24 GB** (20 GB)	60–90s	Best Qwen quality via Ollama. Needs 32 GB+ system RAM. Outstanding quality.
Gemma 4 26B `gemma4:26b`	**22 GB** (18 GB)	50–80s	MoE architecture (25 B total / 3.8 B active); strong quality with moderate active-weight footprint. 256K context.
Gemma 4 31B `gemma4:31b`	**24 GB** (20 GB)	70–100s	Largest dense Gemma 4; near-cloud quality on a workstation. 256K context.
LLaVA 1.6 7B `llava:7b`	**6 GB** (4.7 GB)	15–25s	Well-established, wide compatibility. Good quality.

Qwen3.5, 3.6 & 3.8 (Current Native-Multimodal Generations)

The Qwen line moved on from the `-VL` split: **Qwen3.5** (shipped early 2026), **Qwen3.6**, and the brand-new **Qwen3.8** (open weights released mid-August 2026) are natively multimodal — no separate "-VL" variant — and generally out-reason the Qwen-VL models at similar parameter counts. Qwen3.5 is the mainstream recommendation; `qwen3-vl` is the still-solid previous generation.

Qwen3.5 — the everyday line, with the full size range on Ollama:

Model	RAM Download	Notes
Qwen3.5 2B `qwen3.5:2b`	**3 GB**	Budget / batch; fast.
Qwen3.5 4B `qwen3.5:4b`	**4 GB**	**Recommended everyday upgrade from Qwen2.5-VL 3B** — same class, current architecture.
Qwen3.5 9B `qwen3.5:9b`	**8 GB**	Clear quality jump; the sweet spot when RAM allows.
Qwen3.5 27B `qwen3.5:27b`	**20 GB**	High quality on a workstation.
Qwen3.5 35B `qwen3.5:35b`	**24 GB**	Largest widely-available variant.

Qwen3.6 — on Ollama only the larger sizes are published (`qwen3.6:27b` ~18 GB, `qwen3.6:35b` ~23 GB, plus `-mlx` variants for Apple Silicon), so it's a "best local quality" option rather than an everyday one.

Qwen3.8 — the newest generation (open weights August 2026); on Ollama as **27B only** (`qwen3.8:27b`, ~18 GB at Q4; `qwen3.8:27b-mlx` for Apple Silicon; a dozen quantization tags in total, all with image input), 256K context. Early benchmarks show the strongest image understanding of any local Qwen yet (document and real-world image tasks clearly ahead of Qwen3.6-27B), but there's no smaller size, so it needs ~24 GB RAM/VRAM. Worth trying if your hardware fits — with two caveats: run **Ollama v0.32.14 or newer** (release-day builds mis-parsed image input via the wrong parser; fixed within days), and A/B it against your current model on your own photos before switching the default. The related `qwen3.8-flash-next` (a 125 B MoE with 6 B active, vision-capable, 105–120 GB on disk) is a preview of the Qwen4 architecture and out of reach for ordinary workstations.

Not a VLM: Qwen-Image. Alibaba's *Qwen-Image* line (2.1 as of September 2026) is a diffusion model for generating and editing images, not for describing them — it cannot be used with `maki describe`, and it isn't on Ollama. The naming invites confusion; for descriptions you want Qwen3.5 / Qwen3-VL / Qwen3.8.

Vision on Ollama — 3.5 yes, 3.8 yes, 3.6 reportedly yes since late August 2026: current Ollama releases handle the **Qwen3.5** vision path correctly (only quite old versions silently dropped image input), and **Qwen3.8** runs on the native engine with vision wired up (on v0.32.14+, see above). **Qwen3.6** image input was *not* wired up in Ollama through mid-2026 (text generation worked, the vision projector wasn't loaded); the library rebuilt every `qwen3.6` tag in late August 2026 and now lists image input on all of them. We haven't re-verified it ourselves, so treat it as "works on current Ollama, confirm before a batch run" — or use llama.cpp / LM Studio, where Qwen3.6 vision has worked all along. Quick test for any new model: describe an obviously distinctive image (e.g. a sunset over water) — a generic answer that could fit any photo means vision isn't wired up.

Gemma 4 (Native Multimodal)

Google's Gemma 4 is natively multimodal — vision (and audio on the smaller variants) baked into the same model rather than glued on via a separate projector. Compared to Gemma 3: 128K context (vs 8K), native system-prompt support, configurable "thinking" modes, and a noticeable step up on coding / structured-output benchmarks. For photographic description specifically the practical wins are the longer context (room for richer prompts + multi-image comparisons in the future) and the native system-prompt slot (cleaner instruction-following than Gemma 3's in-content prompts).

Variant	Tag	Effective Params	Modalities	Notes
E2B	`gemma4:e2b`	2.3 B (5.1 B w/ embeddings)	text + image + audio	Compact, fast. Smaller-than-Gemma-3-4B effective compute.
E4B	`gemma4:e4b`	4.5 B (8 B w/ embeddings)	text + image + audio	**Direct upgrade path for `gemma3:4b` users.** Similar effective size, newer architecture, longer context.
12B	`gemma4:12b`	12 B	text + image	Mid-size dense; a step above E4B when RAM allows, below the 26B MoE.
26B MoE	`gemma4:26b`	25.2 B total / 3.8 B active	text + image	MoE — only ~3.8 B weights active per token, large total. Good quality / active-weight ratio.
31B Dense	`gemma4:31b`	30.7 B	text + image	Largest dense variant; best local Gemma 4 quality.

Accelerator-specific tags: check your Ollama version. Until Ollama v0.33.3 (late August 2026) the variants tagged `-mlx`, `-mlx-bf16`, `-mxfp8`, or `-nvfp4` were text-only optimisations that silently dropped image input. v0.33.3 wired up images (and audio) for Gemma 4 on the MLX engine, and the library now lists image input on those tags too. On older Ollama, stick with the default tags (`gemma4:e2b`, `gemma4:e4b`, `gemma4:12b`, `gemma4:26b`, `gemma4:31b`) — or the explicit `-it-q4_K_M` / `-it-q8_0` quants — for `maki describe`. Either way, run the smoke test from the Qwen section once: describe a distinctive image and check whether the answer is plausibly visual.

Audio support on E2B / E4B is interesting but `maki describe` only sends images — the audio modality is unused.

Backends

Ollama (Recommended)

The simplest setup. Ollama manages model downloads, quantization, and serves an OpenAI-compatible API.

Install: https://ollama.com

```
# Pull a model
ollama pull qwen2.5vl:3b

# The server starts automatically; default endpoint is http://localhost:11434
maki describe --check
```

```
# maki.toml
[vlm]
endpoint = "http://localhost:11434"
model = "qwen2.5vl:3b"
```

Supported models: All models with Ollama vision support — Qwen2.5-VL, Qwen3-VL, Qwen3.5, Qwen3.8, Gemma 3, Gemma 4, LLaVA, Moondream — and, on Ollama releases from late August 2026 onwards, Qwen3.6 (see the vision note above). Keep Ollama current (v0.34.x at the time of writing); most "vision silently missing" reports trace back to an old Ollama rather than the model.

Concurrency: Ollama handles one request at a time by default. For `concurrency > 1`, set `OLLAMA_NUM_PARALLEL` environment variable or increase `num_parallel` in Ollama's config.

Apple Silicon — MLX backend. Recent Ollama releases (early 2026 onwards) bundle Apple's MLX framework as the default inference path on Apple Silicon, with measurable wins on per-token latency and energy use vs the GGUF/llama.cpp backend. Most models pick this up automatically when the relevant `*-mlx-*` quants are available. Before Ollama v0.33.3 the MLX-only tags (notably the Gemma 4 `-mlx-bf16` series) were text-only and silently dropped image input; that is fixed on current releases, but if you're on an older Ollama use the standard tags for `maki describe`. No MAKI config changes needed; the speedup is transparent on the HTTP API.

llama.cpp

Direct inference with GGUF model files. Supports Qwen3.5 multimodal via the `--mmproj` flag.

Install: https://github.com/ggerganov/llama.cpp

```
# Download model files (e.g. from HuggingFace)
# You need both the main model and the vision projector (mmproj) file

# Start the server with vision support
llama-server \
  -m Qwen3.5-9B-Q4_K_M.gguf \
  --mmproj mmproj-BF16.gguf \
  --host 0.0.0.0 \
  --port 8080
```

```
# maki.toml
[vlm]
endpoint = "http://localhost:8080"
model = "Qwen3.5-9B"
```

Key points: - Vision requires the `--mmproj` flag with the separate vision projector file - Quantized GGUF files (Q4_K_M, Q5_K_M) reduce RAM needs significantly - No automatic model management — you download and manage files yourself - Serves the OpenAI-compatible API at `/v1/chat/completions`

vLLM

High-throughput inference server, best for GPU machines and batch processing. Supports Qwen3.5 natively.

Install: https://docs.vllm.ai

```
# Requires vLLM 0.17.0+ for Qwen3.5
pip install vllm

# Start the server
vllm serve Qwen/Qwen3.5-9B --host 0.0.0.0 --port 8000
```

```
# maki.toml
[vlm]
endpoint = "http://localhost:8000"
model = "Qwen/Qwen3.5-9B"
concurrency = 4   # vLLM handles parallel requests well
```

Key points: - Best throughput for batch processing (set `concurrency` higher) - GPU recommended (CUDA, ROCm); CPU inference is slow - Automatic model download from HuggingFace - Full OpenAI-compatible API

LM Studio

Desktop application with a GUI for model management and a built-in server.

Install: https://lmstudio.ai

```
# maki.toml
[vlm]
endpoint = "http://localhost:1234"
model = "qwen2.5-vl-3b"   # Use the model name shown in LM Studio
```

Cloud APIs

Any OpenAI-compatible cloud API works. Note that cloud APIs charge per request and maki does not set authentication headers — use a local proxy if your endpoint requires an API key.

```
# maki.toml — example with a self-hosted proxy that adds auth
[vlm]
endpoint = "https://api.openai.com"
model = "gpt-6-astra"        # or gpt-6-sol for a cheaper tier
```

Cloud model identifiers churn quickly (`gpt-4o` and the GPT-5.x line were both retired during 2026) — check your provider's current model list rather than copying an identifier from documentation.

Choosing a Model

By Use Case

Scenario	Recommended Model	Why
Daily use, modest hardware	Qwen2.5-VL 3B	Fast, 3 GB RAM, good quality
Direct upgrade from Gemma 3	Gemma 4 E4B	Native multimodal, 128K context, similar latency
Best quality via Ollama	Qwen3.5 9B, Qwen3-VL 8B, or Gemma 4 E4B	Excellent descriptions, reasonable speed
Bulk first pass	Moondream 2B or Gemma 4 E2B	3–10s per image, good-enough descriptions
Best local quality	Gemma 4 31B or Qwen3.5 27B	Strongest reasoning at workstation RAM
32 GB+ Mac or GPU server	Qwen3.8 27B / Qwen3-VL 32B / Gemma 4 31B	Near-cloud quality (Qwen3.8 needs Ollama 0.32.14+)
Tag suggestions	Qwen3-VL 8B or Gemma 4 E4B	Structured output reliability
Architectural / technical	Gemma 4 E4B or Qwen3-VL 8B	Good at detail and materials
Multilingual descriptions	Qwen3.5 (any size)	201 languages
Apple Silicon, low energy	any Gemma 4 / Qwen variant on recent Ollama	MLX backend kicks in automatically

By Hardware

System RAM	GPU VRAM	Suggested Models
8 GB	—	Moondream 2B, Gemma 4 E2B
16 GB	—	Qwen2.5-VL 3B (default), Gemma 3 4B, Qwen3-VL 4B, Gemma 4 E4B
24–32 GB	—	Qwen3-VL 8B, Qwen2.5-VL 7B, Qwen3.5 9B, Gemma 4 26B
32 GB+	—	Qwen3.8 27B, Qwen3-VL 32B, Qwen3.5 27B, Gemma 4 31B
—	8 GB	Qwen3-VL 8B (FP16), Qwen3.5 9B (Q4), Gemma 4 E4B
—	16 GB+	Qwen3.5 27B (Q4), Qwen3-VL 32B (Q4), Gemma 4 31B

Model Comparison

Qwen Model Generations

	Qwen2.5-VL	Qwen3-VL	Qwen3.5
Architecture	Late fusion (separate vision encoder)	Late fusion (improved encoder)	Early fusion (native multimodal)
Vision quality	Good	Very good	Best

	Qwen2.5-VL	Qwen3-VL	Qwen3.5
Document understanding	Good	Better	Best (90.8 OmniDocBench)
Context length	128K	128K	256K
Languages	~29	~119	201
Ollama vision	Yes	Yes	Yes (current releases)
llama.cpp vision	Yes	Yes	Yes (with mmproj)

For most users, **Qwen3.5 via Ollama** (`qwen3.5:4b` or `:9b`) is the practical sweet spot now that Ollama wires up its vision path; **Qwen3-VL** remains a well-tested fallback. Qwen3.6 vision on Ollama arrived with the late-August 2026 tag rebuild (see above); the newer **Qwen3.8** works on current Ollama but exists only as a 27B — both are high-end options, not everyday ones.

Description Quality (Subjective)

Based on describing the same set of 50 photographs (landscapes, portraits, architecture, street):

- **Moondream 2B**: Short, accurate, occasionally generic. Good at identifying subjects, weaker on mood/lighting.
- **Qwen2.5-VL 3B**: Solid all-rounder. Good detail, reasonable sentence structure. Occasionally repetitive across similar images.
- **Qwen3-VL 4B**: Noticeable step up in vocabulary and scene understanding over Qwen2.5-VL 3B.
- **Gemma 3 4B**: Strong on materials, textures, and spatial relationships. Slightly less natural prose.
- **Qwen3-VL 8B**: Rich, natural descriptions. Good at lighting, mood, composition. Rarely generic.
- **Qwen2.5-VL 7B**: Similar quality to Qwen3-VL 8B but slower.
- **Qwen3.5 9B**: Best visual reasoning. Catches subtle details (reflections, depth of field, lens characteristics). Runs on current Ollama; llama.cpp or vLLM also work.

Thinking Models

Some models (Qwen3-VL, Qwen3.5) support "extended thinking" — they wrap internal chain-of-thought reasoning in `<think>...</think>` tags before producing the actual answer. This can consume most of the `max_tokens` budget with reasoning text, leaving little room for the description itself.

MAKI handles this automatically:

- Sends `think: false` to disable extended thinking (works with Ollama's native API)
- Strips any `<think>...</think>` tags from the response as a fallback
- Tries the Ollama native API first (where `think: false` is honored), falling back to the OpenAI-compatible endpoint

No configuration is needed — thinking models work out of the box. If you see empty or truncated descriptions, try increasing `max_tokens` (default 500) to give the model more headroom.

Tips

Start small, upgrade later. Run `maki describe` with Moondream for a quick first pass, then re-describe the best assets with a larger model using `--force`:

```
maki describe "description:none" --model moondream --apply --log
maki describe "rating:4+" --model qwen3-vl:8b --apply --force --log
```

Use `--mode tags` for discovery. VLM-suggested tags can surface patterns you haven't tagged manually:

```
maki describe --mode tags "date:2024-06" --apply --log
```

Lower temperature for batch consistency. When describing hundreds of images, `--temperature 0` gives more uniform output:

```
maki describe "type:image" --temperature 0 --apply --log
```

Increase timeout for first load. Ollama loads the model into memory on first request, which can take 30–60 seconds for larger models:

```
maki describe --model qwen3-vl:8b --timeout 300 --asset a1b2c3d4
```

Use per-model config for multi-model workflows. If you switch between models frequently, set per-model overrides so each model gets its optimal settings automatically. Each `[vlm.model_config."name"]` section merges on top of the global `[vlm]` settings when that model runs, and CLI flags still win over both:

```
[vlm]
endpoint = "http://localhost:11434"
model = "qwen2.5vl:3b"                  # the default for plain `maki describe`
models = ["qwen2.5vl:3b", "qwen3-vl:8b", "moondream:latest"]  # web UI dropdown
max_image_edge = 512                    # global: resize before sending
timeout = 300                           # global: 5 min server-side

# Bigger model — needs more time to load + more context for richer prompts.
# Lower temperature because we want more deterministic output on careful runs.
[vlm.model_config."qwen3-vl:8b"]
timeout = 600                           # first-load can take 1–2 min on cold start
num_ctx = 8192                          # extended thinking benefits from headroom
temperature = 0.3                       # less creative, more grounded

# Fast bulk pass — short outputs, near-zero temperature for consistency,
# tiny image edge to maximise throughput.
[vlm.model_config."moondream:latest"]
max_tokens = 200
temperature = 0.1
max_image_edge = 384
```

Each override answers a specific question:

- `timeout`: how long the HTTP client waits for the server to respond. Larger models need more on cold start (Ollama loads weights into RAM the first time).
- `num_ctx`: the model's context window. Thinking models (Qwen3-VL, Qwen3.5) need 4 K+ for the `<think>…</think>` block plus the actual answer to fit.
- `temperature`: 0.0 = deterministic, 1.0 = creative. For batch-describing many photos consistently, low temperature reduces "look at this beautiful…" boilerplate.
- `max_image_edge`: longest-edge pixel cap before resize. Smaller = faster encode + faster model. 384–512 is plenty for caption-style descriptions; 768 if you're asking the model to OCR or count things.
- `max_tokens`: response length cap. Tiny for moondream-style one-liners; default 500 for richer captions.

See Configuration Reference — per-model config for the merge precedence rules and field-by-field reference.

Tune sampling parameters per model. Some models benefit from non-default sampling. Use `--num-ctx`, `--top-p`, `--top-k`, and `--repeat-penalty` on the CLI, or set them in `maki.toml` (globally or per-model). A value of zero means "not set — use server default":

```
maki describe "rating:5" --model qwen3-vl:8b --num-ctx 8192 --top-p 0.9 --apply --log
```

Use concurrency with capable servers. If your server handles parallel requests (vLLM, GPU Ollama with `OLLAMA_NUM_PARALLEL`):

```
[vlm]
concurrency = 4
```

See Also

- Configuration Reference — [vlm] section — all configuration options
- maki describe — command reference
- Ingesting Assets — auto-describe during import

Licenses & Acknowledgements

This appendix summarises the licensing of MAKI itself, the Rust crates compiled into the binary, the AI models downloaded on demand, and the external tools MAKI calls.

MAKI

MAKI is released under the **Apache License 2.0**. The full license text is in the `LICENSE` file at the root of the source repository and is available at https://www.apache.org/licenses/LICENSE-2.0.

```
Copyright 2024–2026 Thomas Herrmann
Licensed under the Apache License, Version 2.0
```

You may use MAKI for any purpose, modify it, and redistribute it (in source or compiled form), subject to the conditions of the Apache License 2.0 — primarily that you preserve the copyright notice and provide a copy of the license with any redistribution.

Third-party Rust crates

The MAKI binary statically links many open-source Rust crates. All bundled dependencies use **permissive open-source licenses**:

License	Examples	Notes
Apache-2.0	tokio, axum, serde, image, ort	Permissive; requires copyright preservation
MIT	clap, sha2, anyhow, rusqlite	Permissive; requires copyright preservation
BSD-2-Clause / BSD-3-Clause	various	Permissive; no advertising clause
ISC	various	Functionally equivalent to MIT
MPL-2.0	a few crates	Weak copyleft, file-level only
NCSA	rav1e (transitive)	University of Illinois — permissive
Unicode-3.0	text/Unicode crates	Unicode consortium license
Zlib, BSL-1.0, CC0, 0BSD	misc.	All permissive

The full text of every dependency's license — including copyright notices and the dependency tree showing which crates use which license — is in the file:

`THIRD_PARTY_LICENSES.md`

This file is shipped alongside the `maki` binary in every release archive (`maki-<version>-<platform>.tar.gz` / `.zip`). It is generated automatically with `cargo-about` during the release build.

You can also see a summary on the command line:

```
maki licenses             # full overview
maki licenses --summary   # short version
maki licenses --json      # machine-readable
```

License compatibility validation

The MAKI build pipeline runs `cargo-deny` on every release to validate that:

- All dependencies use only the licenses listed above (no copyleft licenses like GPL or LGPL slip in via transitive dependencies)
- No security advisories affect any compiled-in crate
- All dependencies come from `crates.io` (no untrusted git or alternate registries)

The configuration files are `about.toml` (for `cargo-about`) and `deny.toml` (for `cargo-deny`) at the repository root.

AI models *(Pro)*

MAKI Pro uses image-text encoder models (SigLIP, SigLIP 2) and face detection/recognition models. These are **downloaded on demand** from Hugging Face the first time you run `maki auto-tag --download` or trigger a feature that needs them; they are not bundled in the binary.

Model	Source	License	Credit
`siglip-vit-b16-256`	`Xenova/siglip-base-patch16-256`	Apache-2.0	Google Research
`siglip-vit-l16-256`	`Xenova/siglip-large-patch16-256`	Apache-2.0	Google Research
`siglip2-base-256-multi`	`onnx-community/siglip2-base-patch16-256-ONNX`	Apache-2.0	Google Research (SigLIP 2)
`siglip2-large-256-multi`	`onnx-community/siglip2-large-patch16-256-ONNX`	Apache-2.0	Google Research (SigLIP 2)
Face detection (YuNet)	`opencv/face_detection_yunet`	MIT	Shiqi Yu et al. / OpenCV Zoo
Face recognition (ArcFace ResNet-100)	`onnxmodelzoo/arcfaceresnet100-8`	Apache-2.0	ONNX Model Zoo; ArcFace method by the InsightFace project

By downloading these models, you agree to the terms of the license under which each is distributed (Apache-2.0, or MIT for YuNet). Because MAKI does not bundle the model weights, MAKI itself has no redistribution obligation for them; the obligation, if any, is on you when you copy a downloaded model elsewhere.

External tools

MAKI calls several external command-line tools as separate processes when they are present on the system. These tools are **not bundled** with MAKI — they are installed by the user, governed by their own licenses, and run as independent processes.

Tool	Used for	License
`dcraw` / `libraw`	RAW image preview generation	LGPL-2.1 (libraw), GPL-2 (dcraw)
`ffmpeg`	Video thumbnail and proxy generation	LGPL-2.1 / GPL-2 (build-dependent)
`ffprobe`	Video metadata extraction (duration, codec, resolution)	Same as ffmpeg
`curl`	AI model download and VLM HTTP calls	MIT/X-style
`keyfinder-cli`	Musical key detection (`maki audio analyze`)	GPL-3.0
`beat_this`	Tempo (BPM) detection (`maki audio analyze`)	MIT

Because MAKI invokes these tools via `std::process::Command` (a separate-process boundary), there is no linking and no derived-work relationship — MAKI's Apache-2.0 license is unaffected by their licenses, and vice versa.

If you redistribute MAKI bundled together with these tools (for example, in an installer that ships ffmpeg alongside the `maki` binary), you become the redistributor of those tools and must comply with their respective licenses. The default MAKI release archives only contain the `maki` binary, the manual PDF, and `THIRD_PARTY_LICENSES.md` — no external tools are bundled.

Reporting license concerns

If you believe a dependency listed in `THIRD_PARTY_LICENSES.md` is incorrectly attributed, or if you spot a license-compatibility issue we should address, please open an issue at https://github.com/thoherr/maki/issues.

This appendix is updated whenever the dependency tree changes. The authoritative, version-pinned license text for the binary you are running is always in the `THIRD_PARTY_LICENSES.md` file shipped in your release archive.

REST API Reference

The `maki serve` command starts an Axum-based web server (default `127.0.0.1:8080`) that exposes both HTML page routes and JSON/HTML-partial API endpoints. All endpoints use `spawn_blocking` for SQLite access and open a fresh catalog connection per request via `Catalog::open_fast()`.

Base URL: `http://127.0.0.1:8080` (configurable via `--bind` and `--port`)

Page Routes

These routes return full HTML pages (for direct browser requests) or HTML partials (for htmx requests).

`GET /` – Browse Page

Returns the asset browse grid with search, filtering, sorting, and pagination.

Parameter	Type	Default	Description
`q`	string	`""`	Free-text search query (supports filter syntax)
`type`	string	`""`	Asset type filter (e.g. `image`, `video`)
`tag`	string	`""`	Tag filter
`format`	string	`""`	File format filter, comma-separated for OR (e.g. `nef,cr3`)
`volume`	string	`""`	Volume label filter
`rating`	string	`""`	Rating filter (`5` exact, `3+` minimum)
`label`	string	`""`	Color label filter (e.g. `Red`, `Green`)
`collection`	string	`""`	Collection name filter
`path`	string	`""`	Path prefix filter
`sort`	string	`date_desc`	Sort order (see below)
`page`	u32	`1`	Page number (configurable results per page, default 60)

Sort values: `date_desc`, `date_asc`, `name_desc`, `name_asc`, `size_desc`, `size_asc`

Behavior: Detects `HX-Request` header. htmx requests receive a `ResultsPartial` (just the results grid). Direct browser requests receive the full `BrowsePage` with layout, CSS, filter controls, saved search chips, and all dropdown options.

```
# Full page (browser)
curl http://localhost:8080/
```

```
# With filters
curl "http://localhost:8080/?q=sunset&rating=4%2B&sort=date_desc&page=1"

# htmx partial
curl -H "HX-Request: true" "http://localhost:8080/?tag=landscape&page=2"
```

`GET /asset/{id}` – Asset Detail Page

Returns the detail page for a single asset, including preview image, metadata, editable tags, rating, label, name, description, variants, recipes, and collection memberships.

Parameter	Type	Description
`id`	string	Asset UUID (path param)

```
curl http://localhost:8080/asset/550e8400-e29b-41d4-a716-446655440000
```

Errors: Returns 404 with `<h1>Not Found</h1>` if the asset does not exist.

`GET /tags` – Tags Page

Returns an HTML page listing all tags with counts. Supports sortable columns and live text filtering.

```
curl http://localhost:8080/tags
```

`GET /stats` – Stats Page

Returns an HTML page showing catalog statistics: overview, type/format breakdown, per-volume details, tag frequencies, and verification health.

```
curl http://localhost:8080/stats
```

`GET /collections` – Collections Page

Returns an HTML page listing all collections with a "New Collection" button.

```
curl http://localhost:8080/collections
```

`GET /people` – People Page

(Pro)

Returns an HTML page showing all detected people with face crop thumbnails, names, face counts, and management controls (rename, merge, delete). Includes a "Cluster" button to run auto-clustering from the UI.

```
curl http://localhost:8080/people
```

`GET /backup` – Backup Status Page

Returns an HTML page showing backup health: summary cards (total assets, at-risk count, min copies), volume distribution bar chart, coverage by purpose table, and volume gaps table.

```
curl http://localhost:8080/backup
```

`GET /stroll` – Stroll Page

(Pro)

Returns an interactive visual similarity exploration page. Centers on a randomly selected asset (or a specific asset via query parameter) and displays its nearest neighbors by SigLIP embedding distance. Clicking a neighbor re-centers the view on that asset.

Parameter	Type	Default	Description
`id`	string	(random)	Asset UUID to center on (optional)

```
# Random starting point
curl http://localhost:8080/stroll

# Start from a specific asset
curl "http://localhost:8080/stroll?id=550e8400-e29b-41d4-a716-446655440000"
```

Search API

`GET /api/search`

Same query parameters as `GET /`. Intended for htmx consumption.

Behavior: Non-htmx requests (missing `HX-Request` header) are redirected to `/?{params}` so that direct browser loads, back-button navigations, and bookmarks render the full page. htmx requests receive a `ResultsPartial`.

```
# htmx request (returns HTML partial)
curl -H "HX-Request: true" "http://localhost:8080/api/search?q=sunset&page=1"

# Browser request (redirects to /?q=sunset&page=1)
curl -v "http://localhost:8080/api/search?q=sunset&page=1"
# < HTTP/1.1 303 See Other
# < location: /?q=sunset&page=1
```

Asset Editing

`POST /api/asset/{id}/tags` – Add Tags

Adds one or more tags to an asset. Triggers XMP write-back.

Field	Type	Description
`tags`	string	Comma-separated tag names (form-encoded)

Content-Type: `application/x-www-form-urlencoded`

Response: HTML partial – updated tag list fragment.

```
curl -X POST http://localhost:8080/api/asset/{id}/tags \
  -d "tags=landscape,nature,sunset"
```

`DELETE /api/asset/{id}/tags/{tag}` – Remove Tag

Removes a single tag from an asset. Triggers XMP write-back.

Parameter	Type	Description
`id`	string	Asset UUID (path param)
`tag`	string	Tag to remove (path param, URL-encoded)

Response: HTML partial – updated tag list fragment.

```
curl -X DELETE http://localhost:8080/api/asset/{id}/tags/landscape
```

`PUT /api/asset/{id}/rating` – Set Rating

Sets or clears the asset's star rating. A value of `0` or `null` clears the rating. Triggers XMP write-back.

Field	Type	Description
`rating`	u8 or null	Rating value 1-5, or 0/null to clear

Content-Type: `application/x-www-form-urlencoded`

Response: HTML partial – updated rating fragment.

```
# Set rating to 5 stars
curl -X PUT http://localhost:8080/api/asset/{id}/rating \
  -d "rating=5"

# Clear rating
curl -X PUT http://localhost:8080/api/asset/{id}/rating \
  -d "rating=0"
```

`PUT /api/asset/{id}/description` – Set Description

Sets or clears the asset's description. An empty string clears the description. Triggers XMP write-back.

Field	Type	Description
`description`	string or null	Description text, or empty to clear

Content-Type: `application/x-www-form-urlencoded`

Response: HTML partial – updated description fragment.

```
# Set description
curl -X PUT http://localhost:8080/api/asset/{id}/description \
  -d "description=A%20beautiful%20sunset%20over%20the%20mountains"
```

```
# Clear description
curl -X PUT http://localhost:8080/api/asset/{id}/description \
  -d "description="
```

`PUT /api/asset/{id}/name` – Set Name

Sets or clears the asset's display name. An empty string clears the name (reverts to filename fallback).

Field	Type	Description
`name`	string or null	Display name, or empty to clear

Content-Type: `application/x-www-form-urlencoded`

Response: HTML partial – updated name fragment (includes fallback filename display when name is cleared).

```
# Set name
curl -X PUT http://localhost:8080/api/asset/{id}/name \
  -d "name=Mountain%20Sunset"

# Clear name
curl -X PUT http://localhost:8080/api/asset/{id}/name \
  -d "name="
```

`PUT /api/asset/{id}/label` – Set Color Label

Sets or clears the asset's color label. Accepts case-insensitive color names from the 7-color set: Red, Orange, Yellow, Green, Blue, Pink, Purple. An empty string clears the label. Triggers XMP write-back.

Field	Type	Description
`label`	string	Color name (case-insensitive), or empty to clear

Content-Type: `application/x-www-form-urlencoded`

Response: HTML partial – updated label fragment.

```
# Set label
curl -X PUT http://localhost:8080/api/asset/{id}/label \
  -d "label=Red"

# Clear label
curl -X PUT http://localhost:8080/api/asset/{id}/label \
  -d "label="
```

`PUT /api/asset/{id}/date` – Set Date

Sets or clears the asset's creation date. Accepts an ISO date or datetime string. An empty string clears the date.

Field	Type	Description
`date`	string or null	ISO date string (e.g. `2024-12-25`), or empty to clear

Content-Type: `application/x-www-form-urlencoded`

Response: HTML partial – updated date fragment.

```
# Set date
curl -X PUT http://localhost:8080/api/asset/{id}/date \
  -d "date=2024-12-25"

# Clear date
curl -X PUT http://localhost:8080/api/asset/{id}/date \
  -d "date="
```

`POST /api/open-location` – Reveal in File Manager

Opens the system file manager with the specified file selected. macOS uses `open -R` (Finder); Windows uses `explorer /select,` (Explorer); Linux uses `xdg-open` on the parent directory.

Field	Type	Description
`volume_id`	string	Volume UUID
`relative_path`	string	File path relative to the volume mount point

Content-Type: `application/json`

Response: `{"ok": true}` on success, or error message.

```
curl -X POST http://localhost:8080/api/open-location \
  -H "Content-Type: application/json" \
  -d '{"volume_id":"abc-123","relative_path":"Photos/DSC_001.nef"}'
```

`POST /api/open-terminal` – Open Terminal

Opens a terminal window in the file's parent directory. macOS uses Terminal.app; Windows uses cmd; Linux tries common terminal emulators.

Field	Type	Description
`volume_id`	string	Volume UUID
`relative_path`	string	File path relative to the volume mount point

Content-Type: `application/json`

Response: `{"ok": true}` on success, or error message.

```
curl -X POST http://localhost:8080/api/open-terminal \
  -H "Content-Type: application/json" \
  -d '{"volume_id":"abc-123","relative_path":"Photos/DSC_001.nef"}'
```

`POST /api/asset/{id}/preview` – Regenerate Previews

Regenerates both the regular preview thumbnail and the smart preview for the asset's best variant. Requires the source file to be on an online volume. Returns cache-busted URLs so the browser displays the newly generated images without requiring a page reload.

Content-Type: None (no body required)

Response: HTML partial – updated preview fragment with cache-busted URLs.

```
curl -X POST http://localhost:8080/api/asset/{id}/preview
```

`POST /api/asset/{id}/rotate` – Rotate Preview

Cycles the preview rotation 90° clockwise (0° → 90° → 180° → 270° → 0°). Regenerates both regular and smart previews with EXIF auto-orientation applied. The rotation is persisted per asset.

Content-Type: None (no body required)

Response: HTML partial – updated preview fragment with cache-busted URLs.

```
curl -X POST http://localhost:8080/api/asset/{id}/rotate
```

`POST /api/asset/{id}/reimport-metadata` – Re-import Metadata

Clears the asset's tags, description, rating, and color label, then re-extracts metadata from all variant source files (XMP recipe sidecars and embedded XMP in JPEG/TIFF media files). Offline volumes are silently skipped.

Content-Type: None (no body required)

Response: HTML partial – updated tags fragment.

```
curl -X POST http://localhost:8080/api/asset/{id}/reimport-metadata
```

`PUT /api/asset/{id}/stack-pick` – Set Stack Pick

Sets this asset as the pick (position 0) of its stack. The previous pick swaps to this asset's former position. Persists to both SQLite and `stacks.yaml`.

Parameter	Type	Description
`id`	string	Asset UUID (path param)

Response:

```
{
  "status": "ok"
}
```

Errors: Returns 500 if the asset is not in a stack.

```
curl -X PUT http://localhost:8080/api/asset/{id}/stack-pick
```

`DELETE /api/asset/{id}/stack` – Dissolve Stack

Dissolves the entire stack that this asset belongs to. All members are unstacked. Persists to both SQLite and `stacks.yaml`.

Parameter	Type	Description
`id`	string	Asset UUID (path param)

Response:

```
{
  "status": "ok"
}
```

Errors: Returns 500 if the asset is not in a stack.

```
curl -X DELETE http://localhost:8080/api/asset/{id}/stack
```

Batch Operations

All batch endpoints accept JSON request bodies and return JSON responses. They loop over individual assets, collecting successes and failures independently.

`POST /api/batch/tags` – Batch Add/Remove Tags

Adds or removes tags on multiple assets. Each individual operation triggers XMP write-back.

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"],
  "tags": ["landscape", "nature"],
  "remove": false
}
```

Field	Type	Description
`asset_ids`	string[]	Array of asset UUIDs
`tags`	string[]	Tag names to add or remove
`remove`	bool	`false` to add tags, `true` to remove

Response:

```
{
  "succeeded": 3,
  "failed": 0,
  "errors": []
}
```

```
curl -X POST http://localhost:8080/api/batch/tags \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "tags": ["landscape"], "remove": false}'
```

`PUT /api/batch/rating` – Batch Set Rating

Sets or clears the rating on multiple assets. Each individual operation triggers XMP write-back.

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2"],
  "rating": 5
}
```

Field	Type	Description
`asset_ids`	string[]	Array of asset UUIDs
`rating`	u8 or null	Rating 1-5, or null/0 to clear

Response:

```
{
  "succeeded": 2,
  "failed": 0,
  "errors": []
}
```

```
# Set rating
curl -X PUT http://localhost:8080/api/batch/rating \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "rating": 4}'

# Clear rating
curl -X PUT http://localhost:8080/api/batch/rating \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "rating": null}'
```

`PUT /api/batch/label` – Batch Set Label

Sets or clears the color label on multiple assets. The label is validated against the 7-color set before processing any assets. Each individual operation triggers XMP write-back.

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2"],
  "label": "Red"
}
```

Field	Type	Description
`asset_ids`	string[]	Array of asset UUIDs
`label`	string or null	Color name (case-insensitive), or empty/null to clear

Response:

```
{
  "succeeded": 2,
  "failed": 0,
  "errors": []
}
```

```
curl -X PUT http://localhost:8080/api/batch/label \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "label": "Green"}'
```

`POST /api/batch/collection` – Add to Collection

Adds assets to a named collection. Persists to both SQLite and YAML.

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2"],
  "collection": "Best of 2025"
}
```

Field	Type	Description
`asset_ids`	string[]	Array of asset UUIDs
`collection`	string	Collection name

Response:

```
{
  "added": 2,
  "collection": "Best of 2025"
}
```

```
curl -X POST http://localhost:8080/api/batch/collection \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "collection": "Best of 2025"}'
```

`DELETE /api/batch/collection` – Remove from Collection

Removes assets from a named collection. Persists to both SQLite and YAML.

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2"],
  "collection": "Best of 2025"
}
```

Field	Type	Description
`asset_ids`	string[]	Array of asset UUIDs

Field	Type	Description
`collection`	string	Collection name

Response:

```
{
  "removed": 2,
  "collection": "Best of 2025"
}
```

```
curl -X DELETE http://localhost:8080/api/batch/collection \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "collection": "Best of 2025"}'
```

`POST /api/batch/stack` – Create Stack

Creates a stack from the selected assets. The first asset becomes the pick (position 0). Assets already in a stack are rejected. Persists to both SQLite and `stacks.yaml`.

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"]
}
```

Field	Type	Description
`asset_ids`	string[]	Array of asset UUIDs (minimum 2)

Response:

```
{
  "stack_id": "550e8400-e29b-41d4-a716-446655440000",
  "member_count": 3
}
```

Errors: Returns 500 if any asset is already in a stack, or fewer than 2 assets are provided.

```
curl -X POST http://localhost:8080/api/batch/stack \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2", "uuid-3"]}'
```

`DELETE /api/batch/stack` – Unstack Assets

Removes assets from their stacks. If a stack is left with 1 or fewer members, it is automatically dissolved. Persists to both SQLite and `stacks.yaml`.

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2"]
}
```

Field	Type	Description
`asset_ids`	string[]	Array of asset UUIDs

Response:

```
{
  "removed": 2
}
```

```
curl -X DELETE http://localhost:8080/api/batch/stack \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"]}'
```

`GET /api/stack/{id}/members` – Stack Members

Returns all members of a stack as a JSON array of card data, ordered by stack position. Used by the per-stack expand/collapse feature on the browse page.

Response: `application/json`

```
[
  {
    "asset_id": "uuid-1",
    "name": "DSC_001.NEF",
    "asset_type": "image",
    "format": "NEF",
    "date": "2026-01-15",
    "preview_url": "/previews/ab/abcdef1234.jpg",
    "rating": 4,
    "label": "Green",
    "variant_count": 2,
    "preview_rotation": 0,
    "stack_id": "stack-uuid"
  }
]
```

```
curl http://localhost:8080/api/stack/550e8400-e29b-41d4-a716-446655440000/members
```

`GET /analytics` – Analytics Dashboard

Returns the analytics page with shooting frequency, camera/lens usage, rating distribution, format breakdown, monthly import volume, and storage per volume charts.

`POST /api/asset/{id}/suggest-tags` – AI Tag Suggestions

(Pro)

Analyzes the asset's preview image with SigLIP and returns suggested tags with confidence scores. Tags already on the asset are included but marked with `existing: true`. The model is lazy-loaded on first request and cached in server memory.

Parameter	Type	Description
`id`	string	Asset UUID (path param)

Response: `application/json`

```
[
  {"tag": "landscape", "confidence": 0.85, "existing": false},
  {"tag": "mountain", "confidence": 0.42, "existing": false},
  {"tag": "nature", "confidence": 0.31, "existing": true}
]
```

Field	Type	Description
`tag`	string	Suggested tag name
`confidence`	float	Confidence score (0.0–1.0)
`existing`	bool	Whether the tag is already on the asset

```
curl -X POST http://localhost:8080/api/asset/550e8400-e29b-41d4-a716-446655440000/suggest-tags
```

`POST /api/batch/auto-tag` – Batch AI Auto-Tag

(Pro)

Auto-tags selected assets using SigLIP. For each asset, encodes the preview image, classifies against the configured label vocabulary, and applies tags above the threshold. Existing tags are not duplicated.

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"]
}
```

Field	Type	Description
`asset_ids`	string[]	Array of asset UUIDs

Response:

```
{
  "succeeded": 3,
  "failed": 0,
  "tags_applied": 12,
  "errors": []
}
```

Field	Type	Description
`succeeded`	u32	Assets successfully processed
`failed`	u32	Assets that failed
`tags_applied`	u32	Total new tags applied across all assets
`errors`	string[]	Error messages for failed assets

```
curl -X POST http://localhost:8080/api/batch/auto-tag \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2", "uuid-3"]}'
```

`POST /api/batch/detect-faces` – Batch Face Detection

(Pro)

Detects faces in the preview images of the selected assets using YuNet + ArcFace. Stores face records, embeddings, and crop thumbnails.

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"]
}
```

Field	Type	Description
`asset_ids`	string[]	Array of asset UUIDs

Response:

```
{
  "succeeded": 3,
  "failed": 0,
  "faces_detected": 7,
  "errors": []
}
```

```
curl -X POST http://localhost:8080/api/batch/detect-faces \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2", "uuid-3"]}'
```

`POST /api/asset/{id}/split` – Split Variants Out of Asset

Splits one or more variants out of a multi-variant asset into new standalone assets. Each specified variant becomes a new asset with its own sidecar YAML. Recipes attached to split variants move with them.

Content-Type: `application/json`

Request body:

```
{
  "variant_hashes": ["sha256:abcdef1234...", "sha256:567890abcd..."]
}
```

Field	Type	Description
`variant_hashes`	string[]	Content hashes of variants to split out

Response:

```
{
  "source_id": "550e8400-e29b-41d4-a716-446655440000",
  "new_assets": [
    {
      "asset_id": "660f9511-f30c-52e5-b827-557766551111",
      "variant_hash": "sha256:abcdef1234...",
      "original_filename": "DSC_001_edit.tif"
    }
  ]
}
```

Field	Type	Description
`source_id`	string	UUID of the original (source) asset
`new_assets[].asset_id`	string	UUID of the newly created asset
`new_assets[].variant_hash`	string	Content hash of the split variant
`new_assets[].original_filename`	string	Original filename of the variant

Errors: Returns 404 if the asset does not exist. Returns 400 if a variant hash is not found on the asset, or if splitting would leave the source asset with zero variants.

```
curl -X POST http://localhost:8080/api/asset/{id}/split \
  -H "Content-Type: application/json" \
  -d '{"variant_hashes": ["sha256:abcdef1234..."]}'
```

`POST /api/batch/group` – Merge Assets

Merges selected assets into a single asset. Donor variants are moved to the target asset, donor assets are removed. Tags and recipes are merged.

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"],
  "target_id": "uuid-1"
}
```

Field	Type	Description
`asset_ids`	string[]	Array of asset UUIDs to merge
`target_id`	string or null	Optional target asset UUID (default: auto-selected)

Response:

```
{
  "target_id": "uuid-1",
  "variants_moved": 2,
  "donors_removed": 2
}
```

Field	Type	Description
`target_id`	string	UUID of the target (surviving) asset
`variants_moved`	u32	Number of variants moved to target
`donors_removed`	u32	Number of donor assets removed

Errors: Returns 400 if fewer than 2 asset IDs are provided.

```
curl -X POST http://localhost:8080/api/batch/group \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2", "uuid-3"], "target_id": "uuid-1"}'
```

`POST /api/batch/auto-group` – Auto-Group by Stem

Groups selected assets by filename stem using fuzzy prefix matching. Merges donor variants into target assets (shortest stem, preferring RAW originals).

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2", "uuid-3"]
}
```

Field	Type	Description
`asset_ids`	string[]	Array of asset UUIDs

Response:

```
{
  "groups_merged": 1,
  "donors_removed": 2,
  "variants_moved": 2
}
```

```
curl -X POST http://localhost:8080/api/batch/auto-group \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2", "uuid-3"]}'
```

`POST /api/batch/export` – Export as ZIP

Downloads matching assets as a ZIP archive. Provide either explicit asset IDs (for selected assets) or browse filter parameters (for "export all" results).

Content-Type: `application/json`

Request body:

```
{
  "asset_ids": ["uuid-1", "uuid-2"],
  "filters": {
    "q": "sunset",
    "type": "image",
```

```
    "tag": "landscape",
    "format": "jpg",
    "volume": "Photos",
    "rating": "3+",
    "label": "Green",
    "collection": "Best",
    "path": "2026/March",
    "person": "Alice"
  },
  "layout": "flat",
  "all_variants": false,
  "include_sidecars": false
}
```

Field	Type	Default	Description
`asset_ids`	string[]	`[]`	Explicit asset IDs (used for selected-asset export)
`filters`	object	`null`	Browse filter params (used for "export all" results)
`filters.q`	string	`""`	Free-text search query
`filters.type`	string	`""`	Asset type filter (image, video, audio)
`filters.tag`	string	`""`	Tag filter
`filters.format`	string	`""`	Format filter
`filters.volume`	string	`""`	Volume label filter
`filters.rating`	string	`""`	Rating filter (e.g. "3+", "5")
`filters.label`	string	`""`	Color label filter
`filters.collection`	string	`""`	Collection name filter
`filters.path`	string	`""`	Path prefix filter
`filters.person`	string	`""`	Person name filter
`layout`	string	`"flat"`	`"flat"` (all files in root) or `"mirror"` (preserve dirs)
`all_variants`	bool	`false`	Include all variants (not just best)
`include_sidecars`	bool	`false`	Include recipe/sidecar files

When `asset_ids` is non-empty, `filters` is ignored. When `asset_ids` is empty, the server resolves matching assets from the filters using the same search pipeline as the browse page. If both are empty/null, all catalog assets are exported.

Response: `application/zip` with `Content-Disposition: attachment; filename="maki-export-N-assets.zip"`.

The ZIP uses `Stored` compression (no deflation) since media files are already compressed.

```
# Export specific assets
curl -X POST http://localhost:8080/api/batch/export \
  -H "Content-Type: application/json" \
  -d '{"asset_ids": ["uuid-1", "uuid-2"], "layout": "flat"}' \
  -o export.zip

# Export all results matching filters
```

```
curl -X POST http://localhost:8080/api/batch/export \
  -H "Content-Type: application/json" \
  -d '{"filters": {"tag": "landscape", "rating": "4+"}, "layout": "mirror"}' \
  -o export.zip
```

Face & People APIs

All face and people endpoints are Pro features.

`GET /api/asset/{id}/faces` – List Faces for Asset

Returns all detected faces for an asset as a JSON array.

Response: `application/json`

```
[
  {
    "id": "face-uuid-1",
    "asset_id": "asset-uuid",
    "person_id": "person-uuid",
    "bbox_x": 0.25,
    "bbox_y": 0.15,
    "bbox_w": 0.12,
    "bbox_h": 0.18,
    "confidence": 0.95,
    "created_at": "2026-03-05T10:30:00Z"
  }
]
```

```
curl http://localhost:8080/api/asset/{id}/faces
```

`POST /api/asset/{id}/detect-faces` – Detect Faces in Asset

Runs face detection on a single asset's preview image. Stores detected faces with embeddings and crop thumbnails.

Response: `application/json`

```
{
  "faces_detected": 3
}
```

```
curl -X POST http://localhost:8080/api/asset/{id}/detect-faces
```

`GET /api/people` – List All People

Returns all people with face counts as a JSON array.

Response: `application/json`

```
[
  {
    "id": "person-uuid",
    "name": "Alice",
    "face_count": 42,
```

```
    "representative_face_id": "face-uuid"
  }
]
```

```
curl http://localhost:8080/api/people
```

`PUT /api/people/{id}/name` – Name a Person

Sets or updates the name of a person.

Content-Type: `application/json`

Request body:

```
{
  "name": "Alice"
}
```

Response:

```
{
  "status": "ok"
}
```

```
curl -X PUT http://localhost:8080/api/people/{id}/name \
  -H "Content-Type: application/json" \
  -d '{"name": "Alice"}'
```

`POST /api/people/{id}/merge` – Merge People

Merges the source person into the target person. All faces from the source are reassigned to the target.

Content-Type: `application/json`

Request body:

```
{
  "source_id": "person-uuid-to-merge"
}
```

Response:

```
{
  "status": "ok",
  "faces_moved": 15
}
```

```
curl -X POST http://localhost:8080/api/people/{id}/merge \
  -H "Content-Type: application/json" \
  -d '{"source_id": "person-uuid-to-merge"}'
```

`DELETE /api/people/{id}` – Delete a Person

Deletes a person. All faces assigned to this person become unassigned.

Response:

```
{
  "status": "ok"
}
```

```
curl -X DELETE http://localhost:8080/api/people/{id}
```

`PUT /api/faces/{face_id}/assign` – Assign Face to Person

Assigns a face to a person.

Content-Type: `application/json`

Request body:

```
{
  "person_id": "person-uuid"
}
```

Response:

```
{
  "status": "ok"
}
```

```
curl -X PUT http://localhost:8080/api/faces/{face_id}/assign \
  -H "Content-Type: application/json" \
  -d '{"person_id": "person-uuid"}'
```

`DELETE /api/faces/{face_id}/unassign` – Unassign Face from Person

Removes the person assignment from a face.

Response:

```
{
  "status": "ok"
}
```

```
curl -X DELETE http://localhost:8080/api/faces/{face_id}/unassign
```

`POST /api/faces/cluster` – Auto-Cluster Faces

Runs auto-clustering on unassigned faces, grouping similar faces into new person records.

Content-Type: `application/json`

Request body (optional):

```
{
  "threshold": 0.5
}
```

Field	Type	Default	Description
`threshold`	float	0.5	Similarity threshold for clustering

Response:

```
{
  "people_created": 12,
  "faces_assigned": 87,
  "singletons_skipped": 5
}
```

```
curl -X POST http://localhost:8080/api/faces/cluster \
  -H "Content-Type: application/json" \
  -d '{"threshold": 0.5}'
```

Data APIs

`GET /api/calendar` – Calendar Heatmap Data

Returns per-day asset counts for a given year, respecting all search filters. Used by the browse page calendar heatmap view.

Parameter	Type	Default	Description
`year`	i32	current year	Year to aggregate
`q`	string	""	Free-text search query (supports filter syntax)
`type`	string	""	Asset type filter
`tag`	string	""	Tag filter
`format`	string	""	File format filter, comma-separated for OR
`volume`	string	""	Volume ID filter
`rating`	string	""	Rating filter
`label`	string	""	Color label filter
`collection`	string	""	Collection name filter
`path`	string	""	Path prefix filter

Response: `application/json`

```
{
  "year": 2026,
  "counts": {
    "2026-01-15": 23,
    "2026-01-16": 8,
    "2026-02-25": 14
  },
  "years": [2024, 2025, 2026]
}
```

Field	Type	Description
`year`	i32	The requested year
`counts`	object	Map of `"YYYY-MM-DD"` → asset count (days with 0 assets are omitted)
`years`	i32[]	All distinct years that have assets in the catalog

```
# Get 2026 calendar data
curl http://localhost:8080/api/calendar?year=2026
```

```
# With filters
curl "http://localhost:8080/api/calendar?year=2026&tag=landscape&rating=4%2B"
```

`GET /api/stroll/neighbors` – Embedding Neighbors

(Pro)

Returns the nearest neighbors of an asset by SigLIP embedding similarity. Used by the Stroll page for visual exploration and by the fan-out feature, which fetches transitive neighbors when a satellite is focused (L2 neighbors-of-neighbors).

Parameter	Type	Default	Description
`id`	string	–	Asset UUID to find neighbors for (required)
`n`	u32	`12`	Number of neighbors to return (range: 5–25)
`q`	string	`""`	Optional search query to scope neighbors

Response: `application/json`

```
{
  "center": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "name": "DSC_001",
    "preview_hash": "a1b2c3d4..."
  },
  "neighbors": [
    {
      "id": "660f9511-f30c-52e5-b827-557766551111",
      "name": "DSC_002",
      "preview_hash": "b2c3d4e5...",
      "similarity": 0.87
    }
  ]
}
```

Field	Type	Description
`center.id`	string	Asset UUID of the center asset
`center.name`	string	Display name or filename
`center.preview_hash`	string	Content hash for preview URL
`neighbors[].id`	string	Neighbor asset UUID
`neighbors[].name`	string	Display name or filename
`neighbors[].preview_hash`	string	Content hash for preview URL
`neighbors[].similarity`	float	Cosine similarity score (0.0–1.0)

```
# Get 12 nearest neighbors
curl "http://localhost:8080/api/stroll/neighbors?id=550e8400-e29b-41d4-a716-446655440000"
```

```
# Get 24 neighbors, scoped to landscapes
curl "http://localhost:8080/api/stroll/neighbors?id=550e8400-e29b-41d4-a716-446655440000&n=24&q=tag:landscape"
```

`GET /api/tags` – List All Tags

Returns all tags with their usage counts as a JSON array of `[name, count]` tuples.

Response: `application/json`

```
[
  ["landscape", 42],
  ["nature", 38],
  ["portrait", 15]
]
```

```
curl http://localhost:8080/api/tags
```

`GET /api/paths` – Path Autocomplete

Hierarchical path completion for the filter bar's Path input. Given a prefix `q`, returns the distinct set of next-segment completions found in `file_locations.relative_path`. Directory completions carry a trailing `/` (more to drill into), file completions don't (leaf).

Query parameters:

- `q` (optional, string): prefix the user has typed so far. Empty / missing returns top-level directory segments.
- `volume` (optional, UUID): scope completions to a single volume. If `q` is an absolute path matching a registered volume mount, the volume is inferred automatically and the mount prefix is stripped.
- `limit` (optional, integer): maximum completions to return. Default 20, clamped to 100.

Response: `application/json` — array of strings, sorted lexicographically.

```
["Capture/2024/", "Capture/2025/"]
```

```
# Top-level directories across all volumes:
curl 'http://localhost:8080/api/paths'

# Next level after a typed prefix:
curl 'http://localhost:8080/api/paths?q=Capture/2024/'

# Narrow to a specific volume:
curl 'http://localhost:8080/api/paths?q=Capture/&volume=68ddc72a-aa3e-4306-b2d2-c726dfedb1df'

# Absolute path — mount prefix stripped automatically, volume inferred:
curl 'http://localhost:8080/api/paths?q=/Volumes/NIKON%20Z9/Capture/'
```

Behaviour notes:

- Wildcard patterns (`*` anywhere in `q`) short-circuit to an empty array. The filter already understands `*`, and offering completions for a pattern would be misleading.
- LIKE metacharacters (`%`, `_`) in the prefix are escaped so paths containing those characters match literally.
- An absolute path that doesn't match any registered volume mount returns an empty array rather than leaking false positives from an unanchored search.
- Ordering is alphabetical; directories and files intermix at the same level.

`GET /api/stats` – Catalog Stats

Returns a JSON object with catalog overview statistics.

Response: `application/json`

```
{
  "assets": 1250,
  "variants": 2100,
  "recipes": 800,
  "total_size": 524288000000
}
```

```
curl http://localhost:8080/api/stats
```

`GET /api/collections` – List Collections

Returns all collections as a JSON array.

Response: `application/json`

```
curl http://localhost:8080/api/collections
```

`POST /api/collections` – Create Collection

Creates a new collection. Persists to both SQLite and YAML.

Content-Type: `application/json`

Request body:

```
{
  "name": "Best of 2025",
  "description": "Top picks from this year"
}
```

Field	Type	Description
`name`	string	Collection name (required, unique)
`description`	string or null	Optional description

Response (201 Created):

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "name": "Best of 2025",
  "description": "Top picks from this year"
}
```

Errors: Returns 409 Conflict if a collection with the same name already exists.

```
curl -X POST http://localhost:8080/api/collections \
  -H "Content-Type: application/json" \
  -d '{"name": "Best of 2025", "description": "Top picks from this year"}'
```

`GET /api/saved-searches` – List Saved Searches

Returns all saved searches as a JSON array.

Response: `application/json`

```
[
  {
    "name": "5-star landscapes",
    "query": "tag:landscape rating:5",
    "sort": "date_desc"
  }
]
```

```
curl http://localhost:8080/api/saved-searches
```

`POST /api/saved-searches` – Create/Update Saved Search

Creates a new saved search or updates an existing one with the same name. Persists to `searches.toml`.

Content-Type: `application/json`

Request body:

```
{
  "name": "5-star landscapes",
  "query": "tag:landscape rating:5",
  "sort": "date_desc"
}
```

Field	Type	Description
`name`	string	Search name (required)
`query`	string	Search query string (required)
`sort`	string or null	Sort order (optional)

Response:

```
{
  "status": "saved",
  "name": "5-star landscapes"
}
```

```
curl -X POST http://localhost:8080/api/saved-searches \
  -H "Content-Type: application/json" \
  -d '{"name": "5-star landscapes", "query": "tag:landscape rating:5", "sort": "date_desc"}'
```

`DELETE /api/saved-searches/{name}` – Delete Saved Search

Deletes a saved search by name. Persists the change to `searches.toml`.

Parameter	Type	Description
`name`	string	Saved search name (path param, URL-encoded)

Response:

```
{
  "status": "deleted",
  "name": "5-star landscapes"
}
```

Errors: Returns 404 if no saved search with that name exists.

```
curl -X DELETE http://localhost:8080/api/saved-searches/5-star%20landscapes
```

Volume Management

The `/volumes` page and the global import dialog talk to these endpoints to register, edit, and remove volumes, and to browse their filesystem for the import flow.

`GET /api/volumes` – List Volumes

Returns the registered volumes (online and offline). Used by the global import dialog's volume picker and the volumes page table.

Response:

```
[
  {
    "id": "f3a1b...",
    "label": "Pictures",
    "mount_point": "/Volumes/Pictures",
    "volume_type": "external",
    "purpose": "media",
    "is_online": true
  }
]
```

`POST /api/volumes` – Register a Volume

Registers a new volume. Equivalent to `maki volume add`.

Request body:

```
{
  "path": "/Volumes/PHOTO-2026",
  "label": "Photo 2026",
  "purpose": "archive"
}
```

`label` is required (UI auto-derives it from the path basename); `purpose` is optional (`media`, `working`, `archive`, `backup`, `cloud`). Returns 409 if a volume with the same label or path is already registered, 400 on invalid input.

`PUT /api/volumes/{id}/rename` – Rename a Volume

```
{ "label": "New label" }
```

`PUT /api/volumes/{id}/purpose` – Set Volume Purpose

```
{ "purpose": "archive" }
```

`purpose` may be `null` to clear.

`DELETE /api/volumes/{id}` – Remove a Volume

Removes the volume registration and any orphaned assets/locations/recipes/derived files left behind. Equivalent to `maki volume remove --apply`.

Response:

```
{
  "label": "Pictures",
  "locations_removed": 1234,
  "recipes_removed": 56,
  "assets_removed": 789
}
```

`GET /api/volumes/{id}/browse` – Browse Filesystem

Lists directory entries directly from disk under the volume's mount point. Distinct from `/api/paths` (which queries indexed catalog paths) — this returns paths not yet imported, used by the import dialog's subfolder autocomplete.

Parameter	Type	Description
`prefix`	string	Volume-relative path prefix (forward-slash separated). Empty = mount root.
`limit`	number	Max entries returned (default 50, capped at 500).
`hidden`	string	Set to `"1"` to include dotfiles. Default off.
`filter`	string	`"all"` (default), `"dirs"`, or `"files"`.

Response:

```
{
  "prefix": "2026/04",
  "entries": [
    { "name": "2026-04-22-paris", "is_dir": true,  "relative_path": "2026/04/2026-04-22-paris" },
    { "name": "2026-04-25.cr3",   "is_dir": false, "relative_path": "2026/04/2026-04-25.cr3" }
  ]
}
```

Sorted directories-first, alphabetical within each. Resolved paths are canonicalized and asserted to start with the canonical mount point — `..` traversal that escapes the mount, and symlinks pointing outside, return **403**. Returns **404** if the prefix doesn't exist or isn't a directory; **400** if the volume is offline.

Import API

A single import job runs at a time on the server. Clients start it via `POST /api/import`, observe via the SSE stream and the status endpoint, and re-attach to a running job by reconnecting the SSE stream.

`POST /api/import` – Start Import

Starts an import job (or runs a dry-run synchronously). Returns 409 if a job is already running.

Request body:

```
{
  "volume_id": "f3a1b...",
  "subfolder": "2026/04/2026-04-25-paris",
  "profile": "weekly",
  "tags": ["paris", "spring-2026"],
  "auto_group": true,
  "smart": false,
  "dry_run": false
}
```

All fields except `volume_id` are optional. `subfolder` is volume-relative (no leading `/`).

Response (live):

```
{ "job_id": "uuid-...", "status": "started" }
```

Response (dry_run = true):

```
{
  "dry_run": true,
  "imported": 142,
  "locations_added": 3,
  "skipped": 8,
  "recipes_attached": 142,
  "recipes_updated": 0,
  "previews_generated": 0,
  "sidecars_created": 0,
  "new_asset_ids": ["uuid", "..."]
}
```

`GET /api/import/progress` – SSE Progress Stream

Server-Sent Events stream of per-file events while a job runs. On connect: replays a ring buffer of up to 100 recent events first, then continues with the live broadcast — so a page reload mid-import doesn't lose the activity feed. Returns **404** if no job is running.

Each event payload is JSON:

```
{ "done": false, "file": "IMG_1234.jpg", "status": "imported",
  "imported": 17, "skipped": 0, "locations_added": 0, "recipes": 0 }
```

The terminal event has `"done": true` plus the final summary fields (or `"error"` on failure).

`GET /api/import/status` – Job Status

Cheap poll endpoint. Used by the nav badge to show whether a job is running.

Response (running):

```
{
  "running": true,
  "job_id": "uuid-...",
  "started_at": "2026-04-27T14:31:08Z",
  "imported": 17,
  "skipped": 0,
  "locations_added": 0,
  "recipes": 0
}
```

Response (idle):

```
{ "running": false }
```

`GET /api/import/profiles` – Named Import Profiles

Lists named import profiles from `[import.profiles.*]` in `maki.toml`. The global import dialog populates its profile dropdown from this so the partial template carries no template-variable dependencies.

Response:

```
{ "profiles": ["weekly", "video"] }
```

Static Assets

`GET /static/htmx.min.js`

Serves the embedded htmx library. Compiled into the binary at build time.

Response: `application/javascript`

`GET /static/style.css`

Serves the embedded stylesheet. Compiled into the binary at build time.

Response: `text/css`

`GET /preview/*`

Serves preview images from the `previews/` directory within the catalog root. Handled by `tower-http::ServeDir`.

Preview files are stored at `previews/<hash-prefix>/<hash>.jpg` (or `.webp` depending on configuration).

```
curl http://localhost:8080/preview/a1/a1b2c3d4e5f6...abc.jpg -o preview.jpg
```

Error Handling

All endpoints return standard HTTP status codes:

Code	Meaning
200	Success

Code	Meaning
201	Created (collection creation)
303	See Other (non-htmx redirect from `/api/search`)
404	Not Found (missing asset or saved search)
409	Conflict (duplicate collection name)
500	Internal Server Error (database or processing error)

Error responses are plain text with a descriptive message.

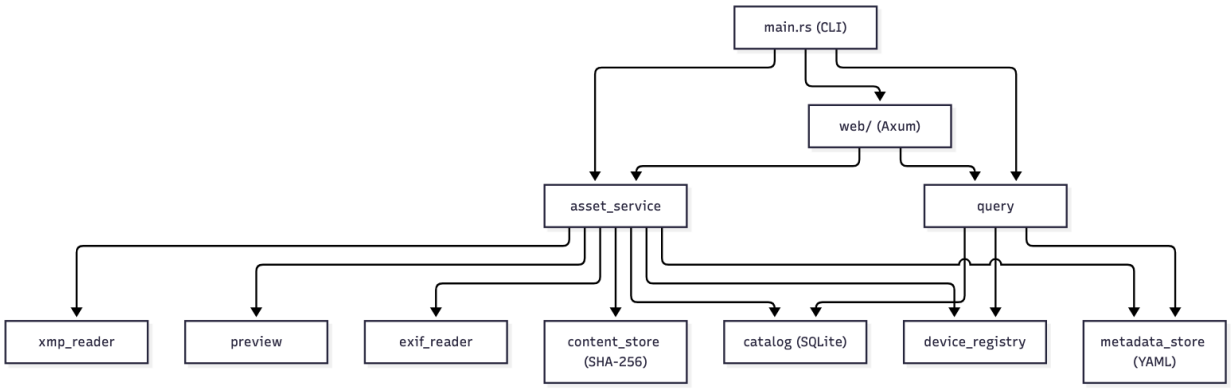
Content Types Summary

Endpoint Pattern	Request Content-Type	Response Content-Type
Page routes (`/`, `/asset/*`, etc.)	–	`text/html`
Single-asset editing (`PUT/POST/DELETE /api/asset/*`)	`application/x-www-form-urlencoded`	`text/html` (partial)
Batch operations (`/api/batch/*`)	`application/json`	`application/json`
Export (`POST /api/batch/export`)	`application/json`	`application/zip`
Data APIs (`GET /api/*`)	–	`application/json`
Mutation APIs (`POST/DELETE /api/*`)	`application/json`	`application/json`

Module Reference

This document describes the modules that make up the `maki` crate. All source code lives under `src/`.

Module Dependency Graph



Additional modules not shown: `config`, `models/`, `format`, `collection`, `saved_search`, `stack`, `embedded_xmp`, `tag_util`, `shell` — used across layers.

Module Table

Module	File(s)	Purpose
`ai`	`src/ai.rs`	*(Pro, `ai` feature.)* Multi-model SigLIP wrapper with `ModelSpec` registry. Supports SigLIP ViT-B/16-256 (768-dim), SigLIP ViT-L/16-256 (1024-dim), and the multilingual SigLIP 2 base (768-dim) and large (1024-dim) variants. ONNX session management for vision and text encoders, image preprocessing (256×256 squash resize, normalize to [-1,1] CHW tensor), SentencePiece tokenization (pad to 64 tokens), sigmoid scoring (`logit_scale * dot + logit_bias`), and ~100 default photography labels. Provides `ModelSpec`, `get_model_spec()`, `SigLipModel` with `encode_image()`, `encode_texts()`, and `classify()`.

Module	File(s)	Purpose
`asset_service`	`src/asset_service.rs`	Orchestrates import, export, grouping, auto-grouping, auto-tagging, face detection, VLM description, video metadata extraction, relocate, delete, and preview generation. Coordinates between content store, metadata store, catalog, EXIF/XMP readers, and preview generator. Shared workflow layer used by both CLI handlers and web routes to ensure identical behavior.
`catalog`	`src/catalog.rs`	SQLite database operations: schema creation, migrations, CRUD for assets/variants/recipes/locations, search queries with pagination and filtering, statistics, and tag management. Provides `Catalog::open()` (with migrations) and `Catalog::open_fast()` (skip migrations, for per-request use in the web server).
`collection`	`src/collection.rs`	Collection (static album) persistence. Dual storage: SQLite tables (`collections`, `collection_assets`) for fast queries, plus `collections.yaml` at catalog root for rebuild resilience. Provides `CollectionStore` with create, list, add/remove assets, and export/import operations.
`config`	`src/config.rs`	Parses and validates `maki.toml` configuration file. Defines `CatalogConfig` with sub-structs `PreviewConfig` (max_edge, format, quality), `ServeConfig` (port, bind), `ImportConfig` (exclude globs, auto_tags), and `AiConfig` (threshold, labels, model_dir, prompt). All sections are optional with sensible defaults.
`content_store`	`src/content_store.rs`	SHA-256 content hashing for content-addressable storage. Provides `hash_file()` for computing file digests used as variant identity and integrity verification.
`embedding_store`	`src/embedding_store.rs`	*(Pro, `ai` feature.)* Dual persistence for image embeddings: SQLite table for queries + binary files for rebuild resilience. Provides `EmbeddingStore` with `store()`, `get()`, `find_similar()`, `has_embedding()`, `count()`, `remove()`, `list_models()`, and `all_embeddings_for_model()` — all scoped by model ID. `EmbeddingIndex` loads all embeddings for a model into a contiguous `Vec<f32>` buffer for fast in-memory similarity search (dot product with `BinaryHeap` min-heap for top-K). Index is cached in `AppState` and updated in-place when new embeddings are stored. Embeddings stored as little-endian f32 BLOBs in SQLite and as raw binary files under `embeddings/<model>/<prefix>/<asset_id>.bin`. Free functions: `write_embedding_binary()`, `read_embedding_binary()`, `delete_embedding_binary()`, `scan_embedding_binaries()`. Table: `embeddings(asset_id TEXT, embedding BLOB, model TEXT, PRIMARY KEY (asset_id, model))`. Migrates old single-PK schema automatically.

Module	File(s)	Purpose
face	src/face.rs	*(Pro, ai feature.)* Face detection and recognition pipeline using ONNX models. FaceDetector wraps YuNet (face detection with bounding boxes, landmarks, confidence) and ArcFace (512-dim recognition embeddings). Multi-stride output decoder handles YuNet models with 12 separate tensors at strides 8/16/32. Provides detect_faces() (returns Vec<DetectedFace> with normalized bboxes), compute_embedding() (112×112 aligned face → 512-dim L2-normalized vector), and generate_crop() (150×150 JPEG thumbnail). Model specs: DETECTION_MODEL (YuNet), RECOGNITION_MODEL (ArcFace), FACE_MODEL_SPECS (for listing/downloading).
face_store	src/face_store.rs	*(Pro, ai feature.)* Dual persistence for face detections and named people: SQLite tables for queries + YAML/binary files for rebuild resilience. Tables: faces(id, asset_id, person_id, bbox_*, confidence, embedding, crop_path, created_at), people(id, name, representative_face_id, created_at). Files: faces.yaml, people.yaml, embeddings/arcface/<prefix>/<face_id>.bin. Provides FaceStore with insert_face(), faces_for_asset(), create_person(), name_person(), assign_face(), unassign_face(), merge_people(), delete_person(), auto_cluster(), list_people(), update_face_count(), save_all_yaml(), export_all_faces(), export_all_people(), import_faces_from_yaml(), import_people_from_yaml(), import_face_embedding(). Free functions: write_arcface_binary(), read_arcface_binary(), delete_arcface_binary(), scan_arcface_binaries(), load_faces_yaml(), save_faces_yaml(), load_people_yaml(), save_people_yaml(). Structs: StoredFace, Person, AutoClusterResult, FaceMatch, FaceRecord, PersonRecord, FacesFile, PeopleFile.
device_registry	src/device_registry.rs	Volume registration and online detection. Manages the volumes.yaml file at the catalog root. Detects whether volumes are online by checking mount point existence. Provides find_volume_for_path() to auto-detect which volume contains a given file path.
embedded_xmp	src/embedded_xmp.rs	Extracts XMP metadata embedded in JPEG and TIFF binary data (APP1 marker for JPEG, IFD tag 700 for TIFF). Returns parsed XMP data for merging during import. Supported formats: .jpg/.jpeg/.tif/.tiff; all other extensions return empty immediately with zero I/O.
exif_reader	src/exif_reader.rs	EXIF metadata extraction from image files using the kamadak-exif crate. Extracts camera make/model, lens, focal length, aperture, shutter speed, ISO, date/time, GPS coordinates, dimensions, and orientation. Provides apply_exif_orientation() for auto-rotating images based on EXIF orientation tags, and apply_rotation() for manual rotation overrides (used by preview generation).

Module	File(s)	Purpose
`format`	`src/format.rs`	Output format template engine for CLI commands. Supports preset formats (`ids`, `short`, `full`, `json`) and custom templates with `{placeholder}` substitution and escape sequences (`\t`, `\n`). Used by `search --format` and `duplicates --format`.
`model_manager`	`src/model_manager.rs`	*(Pro, `ai` feature.)* Download and cache management for SigLIP ONNX model files from HuggingFace. Model-aware: derives HuggingFace URL from `ModelSpec.hf_repo`. Downloads `vision_model_quantized.onnx`, `text_model_quantized.onnx`, and `tokenizer.json` via curl. Provides `ModelManager` with `ensure_model()`, `download_model()`, `remove_model()`, `list_files()`, `total_size()`, and `list_available_models()`. Default cache dir: `~/.maki/models/<model-id>/`.
`metadata_store`	`src/metadata_store.rs`	YAML sidecar file read/write operations. Sidecars are the source of truth for asset metadata. Stored at `metadata/<id-prefix>/<id>.yaml` within the catalog root. Handles serialization/deserialization of `Asset` structs with all nested variants, recipes, locations, and tags.
`models`	`src/models/`	Core data structures shared across all modules. Split into sub-modules:
	`src/models/mod.rs`	Re-exports and shared types (`Asset`, `FileLocation`)
	`src/models/asset.rs`	`Asset` struct: id, name, description, asset_type, tags, rating, color_label, created_at, variants. Includes `validate_color_label()` for the 7-color set.
	`src/models/variant.rs`	`Variant` struct: content_hash, original_filename, format, role, file_size, source_metadata, locations, recipes. Includes `best_preview_index()` / `best_preview_index_details()` for display variant selection, and `compute_best_variant_hash()` / `compute_primary_format()` for denormalized column computation.
	`src/models/recipe.rs`	`Recipe` struct: recipe_type, relative_path, volume_id, content_hash. Represents processing sidecars (XMP, COS, pp3, etc.) attached to variants.
	`src/models/volume.rs`	`Volume` struct: id, label, mount_point, is_online.
`preview`	`src/preview.rs`	Preview/thumbnail generation. Standard images use the `image` crate (resized to configurable max edge, default 800px). RAW files use `dcraw` or `dcraw_emu` (LibRaw). Videos use `ffmpeg`. Non-visual formats get info cards rendered with `imageproc`/`ab_glyph`. Previews stored at `previews/<hash-prefix>/<hash>.{jpg,webp}`. Smart previews (2560px) stored at `smart_previews/` with the same structure.

Module	File(s)	Purpose
`query`	`src/query.rs`	Search query parsing and `QueryEngine` orchestration. Parses filter syntax (`tag:X`, `rating:N+`, `label:Red`, `collection:X`, `path:prefix`, `orphan:true`, `missing:true`, `stale:N`, `volume:none`, `similar:<id>[:<limit>]`). `QueryEngine` wraps catalog + metadata store + device registry to provide high-level operations: `search`, `show`, `tag`, `edit`, `set_rating`, `set_description`, `set_name`, `set_color_label`, `auto_group`, `fix_roles`. Includes path normalization for volume-relative prefix matching.
`saved_search`	`src/saved_search.rs`	Saved search (smart album) persistence. Stores named queries in `searches.toml` at the catalog root. Provides `load()`, `save()`, and `SavedSearch` struct with `to_url_params()` for web UI chip rendering.
`stack`	`src/stack.rs`	Stack (scene grouping) persistence. Dual storage: SQLite `stacks` table for fast queries + `stacks.yaml` at catalog root for rebuild resilience. Provides `StackStore` with create, add, remove, set_pick, dissolve, list, and export/import operations. Assets in a stack have `stack_id` and `stack_position` columns on the `assets` table. Position 0 is the pick (displayed in browse grid when stacks are collapsed).
`web`	`src/web/`	Axum web server. Split into sub-modules:
	`src/web/mod.rs`	Server setup, router construction, `AppState`, request logging middleware, graceful shutdown.
	`src/web/routes.rs`	All HTTP route handlers: browse, asset detail, tags page, stats page, collections page, stroll page *(Pro)*, search API, asset editing endpoints, batch operations, saved search CRUD, collection CRUD, stroll neighbors API. Filter bar markup extracted into shared Askama includes (`filter_bar.html`, `filter_bar_js.html`). The L2 transitive neighbor feature (fan-out slider on the stroll page) is handled entirely in the frontend JavaScript within `templates/stroll.html`, reusing the existing `/api/stroll/neighbors` endpoint with chained requests — no additional backend route is needed.
	`src/web/templates.rs`	Askama HTML templates and template helper structs (`BrowsePage`, `ResultsPartial`, `AssetPage`, `AssetCard`, `TagsPage`, `StatsPage`, `CollectionsPage`, `StrollPage`, `StrollCenter`, `StrollNeighbor`, and HTML fragment types for htmx partial updates).
	`src/web/static_assets.rs`	Compile-time embedded static files (`htmx.min.js`, `style.css`) served as in-memory responses.

Module	File(s)	Purpose
`xmp_reader`	`src/xmp_reader.rs`	XMP sidecar parsing and write-back. Reads XMP files to extract `dc:subject` (tags), `dc:description`, `xmp:Rating`, `xmp:Label`, `dc:creator`, `dc:rights`. Write-back functions: `update_rating()`, `update_label()`, `update_tags()` (operation-level deltas), `update_description()`, `update_hierarchical_subjects()`. Handles XML escaping (`xml_escape` for writes, `xml_unescape` for reads — symmetric inverse pair so multi-pass writebacks don't accumulate `&…` escape layers; `&` decoded LAST in unescape so nested encodings like `<` decode to `<` not `<`), namespace-URI canonicalisation (any prefix bound to the Lightroom namespace — `lr:`, `lightroom:`, or exotic — collapses to one canonical `lr:hierarchicalSubject` block on write), namespace injection, and `rdf:Description` format conversion. The streaming reader uses `quick-xml`'s `Event::Text::unescape()` for entity decoding; the regex-based update paths apply `xml_unescape` at every `<rdf:li>` capture site to keep the in-memory tag set in literal form (matches catalog), preventing the comparison-mismatch bug that drove the runaway-escape behaviour fixed in v4.5.17.

Generated API Documentation

For detailed Rust API documentation with full type signatures, run:

```
cargo doc --no-deps --open
```

This generates HTML documentation from doc comments in the source code and opens it in your browser. The output is written to `target/doc/maki/`.

Building & Testing

Building

Debug Build

```
cargo build
```

Produces an unoptimized binary at `target/debug/maki` with debug symbols and runtime checks enabled. Fast compilation, suitable for development.

Release Build

```
cargo build --release
```

Produces an optimized binary at `target/release/maki`. Significantly faster runtime performance. Use this for production deployment.

Pro Build

```
cargo build --features pro
cargo build --release --features pro
```

Builds the **Pro** edition with SigLIP-based zero-shot image classification, face detection/recognition, and visual similarity search. The `pro` feature enables `ai` automatically. Adds ONNX Runtime (`ort`), `ndarray`, and `tokenizers` as dependencies. The ONNX Runtime shared library is downloaded automatically during build. Without `--features pro` (or `--features ai`), these dependencies are not compiled and AI commands (`auto-tag`, `embed`, `faces`) are not available.

Requirements

- **Rust edition**: 2021; minimum Rust **1.89** (std file locking; the bundled-SQLite stack needs 1.88). CI tracks stable.
- **Platforms**: macOS, Linux, Windows
- **SQLite**: Bundled via `rusqlite` with the `bundled` feature (no system SQLite required)

Testing

Run All Tests

```
cargo test
```

Runs approximately 693 tests total: ~465 unit tests and ~228 integration tests. With `--features pro` (or `--features ai`), adds ~41 unit tests and ~13 integration tests.

Unit Tests Only

```
cargo test --lib
```

Runs the ~465 unit tests embedded in library source files (`#[cfg(test)]` modules within `src/`).

Integration Tests Only

```
cargo test --test integration
```

Runs the ~228 integration tests defined in `tests/cli.rs`. These tests exercise the full system through the CLI binary and library API, using temporary catalogs and volumes.

Run a Specific Test

```
cargo test test_name_pattern
```

Runs only tests whose names match the given pattern.

Test Helpers

The integration test suite provides helper functions for setting up test catalogs:

- **`setup_search_catalog()`** – Creates a catalog with pre-populated assets for search testing. Requires `asset.variants` to be populated before calling `catalog.insert_asset()` (because denormalized columns `best_variant_hash`, `primary_variant_format`, and `variant_count` are computed at insert time).
- **`setup_metadata_catalog()`** – Creates a catalog with assets that have metadata (tags, ratings, descriptions, recipes) for metadata operation testing. Same requirement: variants must be populated before insert.

Documentation

Rust API Docs

```
cargo doc --no-deps --open
```

Generates HTML documentation from doc comments and opens it in your browser. The `--no-deps` flag skips building docs for third-party dependencies, which speeds up the build considerably. Output is at `target/doc/maki/`.

PDF Manual

```
bash doc/manual/build-pdf.sh
```

Generates `doc/manual/maki-manual.pdf` from the 21 Markdown source files. The script concatenates all sections in order, renders mermaid diagrams to PNG, and produces a PDF with table of contents, headers/footers, and syntax-highlighted code blocks. The version number is read from `Cargo.toml`.

Prerequisites (not required for building or running maki itself):

- **pandoc** – Document conversion. `brew install pandoc`
- **XeLaTeX** – PDF typesetting with Unicode support. `brew install --cask mactex-no-gui`
- **mermaid-cli** (`mmdc`) – Diagram rendering. `brew install mermaid-cli`

Release Process

1. **Update documentation**: User manual, README, CHANGELOG, and any other docs affected by the release.
2. **Bump version** in `Cargo.toml`:

```
[package]
version = "X.Y.Z"
```

3. **Update lockfile**:

```
cargo build
```

   This regenerates `Cargo.lock` with the new version.

4. **Run all tests — both feature matrices**:

```
cargo test
cargo test --features pro
```

   All tests must pass in both configurations before releasing.

5. **Commit**:

```
git add -A
git commit -m "Release vX.Y.Z -- brief description"
```

6. **Tag**:

```
git tag vX.Y.Z
```

7. **Push**:

```
git push origin main && git push origin vX.Y.Z
```

   Verify both actually landed (`git ls-remote origin` shows the tag and the main commit).

8. **Let the Release workflow publish.** The tag push triggers `.github/workflows/release.yml`, which builds all platform archives, creates the GitHub release, extracts the release body from the CHANGELOG's `## vX.Y.Z` section, and uploads all assets. **Do not run `gh release create` manually** — a manually created release races the workflow: the workflow's token cannot upload assets to a release it doesn't own, and the result is a release with a partial asset set.

9. **Verify the assets** once the workflow completes (~20 minutes):

```
gh release view vX.Y.Z --json assets --jq '.assets[].name'
```

   Expect 11 assets: six platform archives (macOS arm64 / Linux x86_64 / Windows x86_64, each standard + pro), four PDFs (manual, cheat-sheet, search-filters, tagging), and `THIRD_PARTY_LICENSES.md`.

10. **Per-minor cleanup**: only the latest patch of each minor line keeps a GitHub release (all tags are kept). After verifying, delete the previous *published* release of the same minor:

```
gh release delete vX.Y.<previous> --yes --cleanup-tag=false
```

    Check `gh release list` first — pick the previous release that was actually published, not just the previous version number.

Dependencies

Rust Crates

Crate	Purpose
`clap`	CLI argument parsing with derive macros
`sha2`	SHA-256 content hashing
`serde` / `serde_json` / `serde_norway`	Serialization for JSON output, YAML sidecars
`rusqlite`	SQLite database access (bundled)
`kamadak-exif`	EXIF metadata extraction from images
`quick-xml`	XMP/XML parsing and manipulation
`regex`	Pattern matching in query parsing and XMP processing
`image`	Image decoding, resizing, and encoding for previews
`imageproc`	Text rendering on info card previews
`ab_glyph`	Font loading for info card text (embedded DejaVu Sans)
`lofty`	Audio metadata extraction (duration, bitrate, sample rate, channels, embedded tags)
`uuid`	UUID v4 generation (asset IDs) and v5 (deterministic IDs)
`axum`	HTTP web framework for the `serve` command
`askama`	Compile-time HTML template engine
`tokio`	Async runtime for the web server
`tower-http`	Static file serving middleware (`ServeDir` for previews)
`toml` / `toml_edit`	Configuration file parsing (`maki.toml`, `searches.toml`); comment-preserving Settings save
`schemars`	JSON Schema generation for the web Settings form
`printpdf`	Contact-sheet PDF assembly
`zip`	Web export ZIP archives
`rustyline`	Interactive `maki shell` REPL (line editing, completion)
`glob-match`	Filename glob matching for import exclusion patterns
`chrono`	Date/time handling with serde support
`anyhow` / `thiserror`	Error handling
`ort`	ONNX Runtime bindings for AI inference (optional, `ai` feature)
`ndarray`	N-dimensional array operations for tensor manipulation (optional, `ai` feature)
`tokenizers`	HuggingFace tokenizer for SentencePiece text encoding (optional, `ai` feature)

Dev Dependencies

Crate	Purpose
`assert_cmd`	CLI binary testing (running `maki` as a subprocess)
`predicates`	Assertion helpers for CLI output matching

Crate	Purpose
`tempfile`	Temporary directories for test isolation
`tower` / `http-body-util`	In-process axum router test harness (no socket)
`proptest`	Property-based round-trip tests for the XMP reader/writer

External Tools (Highly Recommended)

These tools are not Rust dependencies but are invoked as subprocesses for specific preview generation tasks. Their absence does not prevent the application from running; missing tools result in info card fallback previews.

- **dcraw** or **LibRaw** (`dcraw_emu`) – RAW image preview extraction. Used to decode camera-native formats (NEF, ARW, CR2, CR3, etc.) into RGB data for thumbnail generation. LibRaw's `dcraw_emu` is preferred when available.
- **ffmpeg** / **ffprobe** – Video thumbnail extraction, audio waveform previews (`showwavespic`), and video metadata (duration, codec, resolution, framerate via `ffprobe`, which ships in the ffmpeg package).
- **curl** – Model file download for AI auto-tagging *(Pro)* and VLM image descriptions (`maki describe`). Used to download ONNX model files from HuggingFace. Available by default on macOS and most Linux distributions.
- **keyfinder-cli** / **beat_this** *(optional)* – External audio analyzers for `maki audio analyze` (musical key and BPM detection). Commands configurable via `[audio]` in `maki.toml`; see the Configuration Reference.

Install on macOS (Homebrew):

```
brew install libraw ffmpeg curl
```

Install on Linux (package manager):

```
# Debian/Ubuntu
sudo apt install libraw-bin ffmpeg curl

# Fedora
sudo dnf install LibRaw ffmpeg curl
```

Install on Windows (winget or scoop):

```
# winget
winget install LibRaw.LibRaw Gyan.FFmpeg cURL.cURL

# scoop
scoop install libraw ffmpeg curl
```

To check if these tools are available:

```
# macOS / Linux
which dcraw_emu || which dcraw
which ffmpeg
which curl
```

```
# Windows (PowerShell)
Get-Command dcraw_emu -ErrorAction SilentlyContinue
Get-Command ffmpeg -ErrorAction SilentlyContinue
Get-Command curl -ErrorAction SilentlyContinue
```

Preview generation silently falls back to info cards (metadata display images) when these tools are missing. Use `maki generate-previews --debug` to see external tool invocations and errors.